

FM-Agent 自验证 Bug 清单（主文件：契约不确定与可能有 Bug）

本文件是可操作的实现复核队列，仅保留“契约不确定”和“可能有 Bug”。SPEC 错误与推理误判已拆分到子文件。

最终运行结果：457 个函数；MATCH 145、MISMATCH 312、ERROR 0；confirmed 262、not_confirmed 50。

原唯一 `_split_into_blocks` ERROR 已通过删除旧 SPEC/INFO/结果后用原始验证器 resume，恢复为 MISMATCH/confirmed。

原始函数与 prompt 未修改；报告仍是 PARTIAL / INCONCLUSIVE，因为预检测测试失败、旧索引混用和 trace 损坏问题尚未重跑消除。

全局分类统计

四类结论	数量	文件
契约不确定	91	主文件
可能有 Bug	51	主文件
SPEC 错误	94	SPEC 错误子文件
推理误判	76	推理误判子文件
合计	312	每项恰好出现一次

分类口径

- 可能有 Bug**：probe 复现具体实现风险，但仍是候选，需确认真实调用前置条件。
- 契约不确定**：差异可复现，但现有证据不足以决定应改实现还是改 SPEC。
- SPEC 错误**：生成 SPEC 添加了源码、调用者或文档未证明的格式、类型、异常或边界要求。
- 推理误判**：Validator 未确认，或 probe 直接推翻 Reasoner 的行为描述/反例。

165 项沿用旧 `fm_agent/bug_list.md` 的逐条审计结论；147 项按本轮 Validator、trigger、probe 与 SPEC 证据重新分流。分类是复核队列，不是最终产品裁决。

主文件索引

编号	分类	Phase / 模块	条目	Validator
001	契约 不确定	P1 / configuration	<code>config-py--_LayeredSource::call</code>	confirmed
002	契约 不确定	P1 / configuration	<code>config-py--_LayeredSource::init</code>	confirmed
003	契约 不确定	P1 / configuration	<code>src--configure_llm-py--_fm_agent_config_path</code>	confirmed
007	可能 有 Bug	P1 / configuration	<code>src--configure_llm-py--_private_backup_root</code>	confirmed
013	契约 不确定	P1 / configuration	<code>src--configure_llm-py--apply_configuration</code>	confirmed
014	契约 不确定	P1 / configuration	<code>src--configure_llm-py--apply_llm_settings_update</code>	confirmed

编号	分类	Phase / 模块	条目	Validator
015	契约 不确定	P1 / configuration	src--configure_llm-py--apply_local_backend_configuration	confirmed
016	契约 不确定	P1 / configuration	src--configure_llm-py--backup_file	confirmed
017	契约 不确定	P1 / configuration	src--configure_llm-py--default_paths	confirmed
018	契约 不确定	P1 / configuration	src--configure_llm-py--detect_opencode_config_path	confirmed
019	契约 不确定	P1 / configuration	src--configure_llm-py--main	confirmed
020	契约 不确定	P1 / configuration	src--configure_llm-py--merge_opencode_config	confirmed
023	契约 不确定	P1 / configuration	src--configure_llm-py--remove_legacy_llm_env_overrides	confirmed
024	契约 不确定	P1 / configuration	src--configure_llm-py--run_llm_settings_update	confirmed
025	契约 不确定	P1 / configuration	src--configure_llm-py--run_local_backend_configuration	confirmed
026	契约 不确定	P1 / configuration	src--configure_llm-py--run_wizard	confirmed
027	契约 不确定	P1 / configuration	src--configure_llm-py--secret_path_for_provider	confirmed
028	契约 不确定	P1 / configuration	src--configure_llm-py--update_env_text	confirmed
030	可能 有 Bug	P1 / configuration	src--configure_llm-py--update_llm_settings_toml_text	confirmed
031	契约 不确定	P1 / configuration	src--configure_llm-py--validate_base_url	confirmed
032	契约 不确定	P1 / configuration	src--configure_llm-py--validate_generated_state	confirmed
033	契约 不确定	P1 / configuration	src--configure_llm-py--validate_llm_setting	confirmed

编号	分类	Phase / 模块	条目	Validator
034	契约 不确定	P1 / configuration	src--configure_llm-py--warn_dotenv_overrides_for_updates	confirmed
038	可能 有 Bug	P1 / environment	src--env_check-py--_check_oh_my_openagent	confirmed
040	可能 有 Bug	P1 / environment	src--env_check-py--_save_ignored	confirmed
042	可能 有 Bug	P1 / git_utils	src--git-py--_get_head_commit	confirmed
043	可能 有 Bug	P1 / git_utils	src--git-py--_is_git_repo	confirmed
045	契约 不确定	P1 / git_utils	src--git-py--frozen_worktree	confirmed
046	可能 有 Bug	P1 / git_utils	src--git-py--frozen_worktree::_git	confirmed
051	契约 不确定	P2 / call_graph_edges	src--call_graph_edges-py--_is_path_function_label	confirmed
052	契约 不确定	P2 / call_graph_edges	src--call_graph_edges-py--_load_call_edge_file	confirmed
054	可能 有 Bug	P2 / call_graph_edges	src--call_graph_edges-py--_normalize_endpoint_label	confirmed
058	可能 有 Bug	P2 / call_graph_edges	src--call_graph_edges-py--load_call_edges	confirmed
060	契约 不确定	P2 / codegraph	src--languages--codegraph-py-- CodeGraphExtractor::from_proj_dir	confirmed
061	契约 不确定	P2 / codegraph	src--languages--codegraph-py-- CodeGraphExtractor::get_call_edges	confirmed
062	契约 不确定	P2 / codegraph	src--languages--codegraph-py-- CodeGraphExtractor::get_functions_by_file	confirmed
063	可能 有 Bug	P2 / codegraph	src--languages--codegraph-py--_bare_function_name	confirmed
065	契约 不确定	P2 / codegraph	src--languages--codegraph-py--_extraction_ident	confirmed

编号	分类	Phase / 模块	条目	Validator
066	可能有Bug	P2 / codegraph	src--languages--codegraph-py--_fqn_for	confirmed
067	契约不确定	P2 / codegraph	src--languages--codegraph-py--_qualified_parts	confirmed
068	契约不确定	P2 / codegraph	src--languages--codegraph-py--_warn_on_codegraph_version_mismatch	confirmed
070	契约不确定	P2 / codegraph	src--languages--codegraph-py--try_codegraph_init	confirmed
071	可能有Bug	P2 / extraction	src--extract-py--_extract_func_name_brace	confirmed
073	可能有Bug	P2 / extraction	src--extract-py--_extract_functions_indent	confirmed
075	契约不确定	P2 / extraction	src--extract-py--_function_spans	confirmed
076	可能有Bug	P2 / extraction	src--extract-py--_strip_angle_brackets	confirmed
077	可能有Bug	P2 / extraction	src--extract-py--extract_functions_from_file	confirmed
081	契约不确定	P2 / language_handlers	src--languages--cpp-py--call_edges	confirmed
083	契约不确定	P2 / language_handlers	src--languages--erlang-py--ElpClient::_handle_server_message	confirmed
084	契约不确定	P2 / language_handlers	src--languages--erlang-py--ElpClient::_send	confirmed
085	契约不确定	P2 / language_handlers	src--languages--erlang-py--ElpClient::_wait_for_response	confirmed
086	契约不确定	P2 / language_handlers	src--languages--erlang-py--ElpClient::close	confirmed
087	契约不确定	P2 / language_handlers	src--languages--erlang-py--ElpClient::initialize	confirmed
088	契约不确定	P2 / language_handlers	src--languages--erlang-py--ElpClient::open_document	confirmed

编号	分类	Phase / 模块	条目	Validator
089	契约 不确定	P2 / language_handlers	src--languages--erlang-py--ElpClient::request	confirmed
090	契约 不确定	P2 / language_handlers	src--languages--erlang-py--_JsonRpcReader::run	confirmed
091	契约 不确定	P2 / language_handlers	src--languages--erlang-py--_SourceIndex::build	confirmed
092	契约 不确定	P2 / language_handlers	src--languages--erlang-py--_SourceIndex::position_to_offset	confirmed
096	契约 不确定	P2 / language_handlers	src--languages--erlang-py--_callgraph_project_root	confirmed
097	可能有 Bug	P2 / language_handlers	src--languages--erlang-py--_elp_argv	confirmed
098	可能有 Bug	P2 / language_handlers	src--languages--erlang-py--_escape_component	confirmed
099	可能有 Bug	P2 / language_handlers	src--languages--erlang-py--_function_id	confirmed
101	契约 不确定	P2 / language_handlers	src--languages--erlang-py--_position_to_offset	confirmed
102	可能有 Bug	P2 / language_handlers	src--languages--erlang-py--_project_fingerprint	confirmed
103	契约 不确定	P2 / language_handlers	src--languages--erlang-py--_source_for_range	confirmed
106	可能有 Bug	P2 / language_handlers	src--languages--erlang-py--batch_extract	not_confirmed
107	可能有 Bug	P2 / language_handlers	src--languages--erlang-py--call_edges	confirmed
108	契约 不确定	P2 / language_handlers	src--languages--go-py--batch_extract	confirmed
111	契约 不确定	P2 / language_handlers	src--languages--java-py--batch_extract	confirmed
114	可能有 Bug	P2 / language_handlers	src--languages--python-py--batch_extract	confirmed

编号	分类	Phase / 模块	条目	Validator
119	契约 不确定	P3 / domain_knowledge	src--domain_knowledge-py--_flatten_paths	confirmed
124	契约 不确定	P3 / domain_knowledge	src--domain_knowledge-py--stage_domain_knowledge_files	confirmed
125	契约 不确定	P3 / pipeline_setup	src--pipeline_setup-py--_build_domain_context_regen_prompt	confirmed
128	可能 有 Bug	P3 / pipeline_setup	src--pipeline_setup-py--_collapse_phases_to_one	confirmed
131	可能 有 Bug	P3 / pipeline_setup	src--pipeline_setup-py--_domain_context_complete	confirmed
132	可能 有 Bug	P3 / pipeline_setup	src--pipeline_setup-py--_ensure_source_files_in_phases	confirmed
134	契约 不确定	P3 / pipeline_setup	src--pipeline_setup-py--_phase_plan_schema_errors	confirmed
139	可能 有 Bug	P3 / pipeline_setup	src--pipeline_setup-py--_run_generate_phases	confirmed
143	契约 不确定	P3 / plugins	src--plugin-py--_resolve_command	confirmed
144	契约 不确定	P3 / plugins	src--plugin-py--_validate_plugin_json_content	confirmed
146	可能 有 Bug	P3 / plugins	src--plugin-py--run_plugin_command	confirmed
157	契约 不确定	P4 / llm_client	src--llm_client-py--_matches_inject_target	confirmed
159	可能 有 Bug	P4 / llm_client	src--llm_client-py--_parse_json_response	confirmed
161	契约 不确定	P4 / llm_client	src--llm_client-py--_retry_create	confirmed
164	可能 有 Bug	P4 / opencode_trace	src--opencode_trace-py--_copy_opencode_output	confirmed
165	契约 不确定	P4 / opencode_trace	src--opencode_trace-py--_opencode_env	confirmed

编号	分类	Phase / 模块	条目	Validator
166	可能有Bug	P4 / opencode_trace	src--opencode_trace-py--_opencode_log_path	confirmed
167	可能有Bug	P4 / opencode_trace	src--opencode_trace-py--_opencode_trace_path	confirmed
168	契约不确定	P4 / opencode_trace	src--opencode_trace-py--_payload_dir	confirmed
170	契约不确定	P4 / opencode_trace	src--opencode_trace-py--_trace_dir	confirmed
171	可能有Bug	P4 / opencode_trace	src--opencode_trace-py--_wait_opencode_process	confirmed
177	可能有Bug	P4 / opencode_trace	src--opencode_trace-py--run_opencode_traced	confirmed
182	可能有Bug	P4 / trace_writer	src--trace_writer-py--append_event	confirmed
188	契约不确定	P5 / batch_prompts	src--generate_batch_prompts-py--main	not_confirmed
191	契约不确定	P5 / batch_prompts	src--generate_batch_prompts-py--read_json	confirmed
192	可能有Bug	P5 / file_utils	src--file_utils-py--_get_incomplete_verification_files	confirmed
193	可能有Bug	P5 / file_utils	src--file_utils-py--_get_phase_files	confirmed
198	可能有Bug	P5 / file_utils	src--file_utils-py--_json_file_is_valid	confirmed
199	契约不确定	P5 / file_utils	src--file_utils-py--collect_file_names	not_confirmed
202	契约不确定	P5 / parser	src--parser-py--format_info_for_reasoner	confirmed
203	契约不确定	P5 / parser	src--parser-py--format_spec_for_reasoner	confirmed
205	可能有Bug	P6 / entry_pipeline	src--entry_reasoning_pipeline-py--_count_mismatches	not_confirmed

编号	分类	Phase / 模块	条目	Validator
206	契约 不确定	P6 / entry_pipeline	src--entry_reasoning_pipeline-py--_fqn_to_ident	confirmed
211	契约 不确定	P6 / entry_pipeline	src--entry_reasoning_pipeline-py--_trim_project_in_place	confirmed
214	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_StdoutTee::flush	confirmed
215	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_StdoutTee::write	confirmed
216	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_bare_function_name	confirmed
218	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_codegraph_legacy_coverage	confirmed
220	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_collect_changed_functions::_funcs_from_commit	confirmed
221	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_collect_changed_functions::_git	confirmed
222	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_collect_changed_functions::_is_workspace_file	confirmed
223	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_collect_changed_functions::_path_exists_in_commit	confirmed
224	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_extracted_files_by_method	confirmed
225	可能有 Bug	P6 / incremental_pipeline	src--incremental_reasoner-py--_llm_check_caller_info_update	confirmed
227	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_llm_select_json	confirmed
229	可能有 Bug	P6 / incremental_pipeline	src--incremental_reasoner-py--_opcode_generate_spec	not_confirmed
230	可能有 Bug	P6 / incremental_pipeline	src--incremental_reasoner-py--_opcode_select_json	confirmed
232	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_reconcile_extracted_dir	confirmed

编号	分类	Phase / 模块	条目	Validator
235	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_src_rel_to_func_dir	confirmed
238	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py-- _update_specs_for_intent::_plan_spec_update	confirmed
239	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py-- _update_specs_for_intent::_reconcile_caller	confirmed
242	契约 不确定	P6 / incremental_pipeline	src--incremental_reasoner-py-- _verify_incremental_functions::_validate	confirmed
244	可能 有 Bug	P6 / incremental_pipeline	src--incremental_reasoner-py--check_last_run_existence	confirmed
248	可能 有 Bug	P6 / orchestration	dashboard-py--State::_ingest_event	confirmed
251	契约 不确定	P6 / orchestration	dashboard-py--State::elapsed	confirmed
252	可能 有 Bug	P6 / orchestration	dashboard-py--State::scan_bugs	confirmed
253	契约 不确定	P6 / orchestration	dashboard-py--State::tail_events	confirmed
254	契约 不确定	P6 / orchestration	dashboard-py--State::tail_opencode	confirmed
257	契约 不确定	P6 / orchestration	dashboard-py--_get_nested	confirmed
261	契约 不确定	P6 / orchestration	dashboard-py--_parse_iso	confirmed
264	契约 不确定	P6 / orchestration	dashboard-py--_trace_value	confirmed
269	契约 不确定	P6 / orchestration	dashboard-py--render_llm_status	confirmed
270	契约 不确定	P6 / orchestration	main-py--_clean_previous_run	confirmed
272	可能 有 Bug	P6 / reasoner	src--reasoner-py--_compute_brace_depth_per_line	confirmed

编号	分类	Phase / 模块	条目	Validator
278	可能 有 Bug	P6 / scope_analysis	src--scope-py--_collect_calls	confirmed
284	可能 有 Bug	P6 / scope_analysis	src--scope-py--_llm_rerank	confirmed
288	可能 有 Bug	P6 / scope_analysis	src--scope-py--_parse_issue_signals	confirmed
291	契约 不确定	P6 / scope_analysis	src--scope-py--_score_class	confirmed
293	契约 不确定	P6 / spec_orchestrator	src--spec_generation_and_verification-py--_run_spec_generation_batch	confirmed
301	可能 有 Bug	P6 / topdown_layers	src--generate_topdown_layers-py--_get_call_regex	confirmed
303	可能 有 Bug	P6 / topdown_layers	src--generate_topdown_layers-py--_load_phases	confirmed
310	可能 有 Bug	P6 / verification	src--verification-py--_validate_single_bug	not_confirmed

契约不确定（91 项）

Phase 1 — Initialization & Configuration

模块：[configuration](#)

FMA-MISMATCH-001 — [config-py--_LayeredSource::__call__](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts [1](#)
- Phase / 模块： Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [config.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a dict where each key is a valid field name of the settings class associated with this instance and each value is the configuration value for that field, with a type compatible with that field's Pydantic type annotation
- **Actual behavior:** The method returns the dictionary [self._data](#), which holds the fully resolved and populated configuration data. The instance state is not modified; the return value is exactly the reference to [self._data](#). Formally: [result = self._data](#)
[isinstance\(result, dict\)](#)
[f fields\(self\) : final\(f\) = initial\(f\).](#)
- **Code evidence:** Line 2: [return self._data](#)
- **Trigger condition:** The specification requires that every key in the returned dict is a valid field name of the settings class. The code returns [self._data](#) directly without filtering out extra keys, allowing nonfield keys to appear when the model accepts extra fields (e.g., via [extra='allow'](#)), thereby violating the specification.
- **Validator trigger:** When the settings model accepts extra fields ([extra='allow'](#)), [_LayeredSource.call\(\)](#) returns [self._data](#) unfiltered, leaking non-field TOML sections and env-injected keys into the result.

- **Probe stdout:** CONFIRMED — extra non-field keys leaked through: ['erlang', 'unexpected_section'] (valid fields: ['llm']) | actual extra data: {'erlang': {'command': 'elp', 'timeout_s': '180'}, 'unexpected_section': {'foo': 'bar', 'baz': 42}}

► SPEC sidecar (完整)

```
{
  "signature": "__call__(self) -> dict",
  "pre_condition": "The instance is in a state where its configuration data has been fully resolved and populated",
  "post_condition": "Returns a dict where each key is a valid field name of the settings class associated with this instance and each value is the configuration value for that field, with a type compatible with that field's Pydantic type annotation"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/config-py/_LayeredSource::__call__.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dict where each key is a valid field name of the settings class associated with this instance and each value is the configuration value for that field, with a type compatible with that field's Pydantic type annotation",
    "actual_behavior": "The method returns the dictionary `self._data`, which holds the fully resolved and populated configuration data. The instance state is not modified; the return value is exactly the reference to `self._data`. Formally: `result = self._data` `isinstance(result, dict)` `f fields(self) : final(f) = initial(f)`.",
    "code_evidence": "Line 2: return self._data",
    "trigger_condition": "The specification requires that every key in the returned dict is a valid field name of the settings class. The code returns self._data directly without filtering out extra keys, allowing nonfield keys to appear when the model accepts extra fields (e.g., via extra='allow'), thereby violating the specification."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "config-py--_LayeredSource::__call__",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/config-py/_LayeredSource::__call__.py",
  "function_name": "_LayeredSource::__call__",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_config-py--_LayeredSource::__call__.py",
  "detail_file": "fm_agent/bug_validation/config-py--_LayeredSource::__call__.md",
  "probe_stdout": "CONFIRMED — extra non-field keys leaked through: ['erlang', 'unexpected_section'] (valid fields: ['llm']) | actual extra data: {'erlang': {'command': 'elp', 'timeout_s': '180'}, 'unexpected_section': {'foo': 'bar', 'baz': 42}}",
  "trigger_summary": "When the settings model accepts extra fields (extra='allow'), _LayeredSource.__call__() returns self._data unfiltered, leaking non-field TOML sections and env-injected keys into the result."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:37:20.411813Z	708
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:37:28.957057Z	4360

事件 1 — [llm_55015c8e328d4e559ba71605f5a27af8](#)

► parsed_json (完整)

```
{
  "post_condition": "The method returns the dictionary `self._data`, which holds the fully resolved and populated configuration data. The instance state is not modified; the return value is exactly the reference to `self._data`. Formally: `result =`
```

```
self._data` ^ `isininstance(result, dict)` ^ `∀f ∈ fields(self) : final(f) = initial(f)`."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_55015c8e328d4e559ba71605f5a27af8_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_55015c8e328d4e559ba71605f5a27af8_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_55015c8e328d4e559ba71605f5a27af8_response.txt)

事件 2 — llm_fae358a1f32043eb95579e375555d704

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Consider a Pydantic BaseSettings subclass with config extra='allow' and a declared field 'name'. Create an instance: Settings(name='test', extra_field='value'). The internal _data becomes {'name': 'test', 'extra_field': 'value'}. Returning this dict includes the key 'extra_field', which is not a valid field name of the Settings class.",
  "offending_statements": "Line 2: return self._data",
  "reason": "The specification requires that every key in the returned dict is a valid field name of the settings class. The code returns self._data directly without filtering out extra keys, allowing non-field keys to appear when the model accepts extra fields (e.g., via extra='allow'), thereby violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_fae358a1f32043eb95579e375555d704_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_fae358a1f32043eb95579e375555d704_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_fae358a1f32043eb95579e375555d704_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-002 — config-py --_LayeredSource::__init__

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator：confirmed; attempts 1
- Phase / 模块：Phase 1 — Initialization & Configuration / configuration; 原源码 config.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: After construction, invoking `call()` on this instance returns a dict whose keys are exactly the model field names of `settings_cls`. For each field, the value is resolved by the following precedence (highest first): (1) the value of any process environment variable mapped to that field via the environment-to-field mapping, (2) the value from the TOML file at path when path references an existing regular file containing valid TOML syntax, (3) the field's Pydantic default. An environment variable value, when present, overrides a TOML value for the same field regardless of the TOML value's type. If path does not reference an existing regular file, a message identifying the missing file is written to stderr and the TOML source is treated as empty.
- **Actual behavior**: If no exception occurs during file reading and TOML parsing: `super().init` completed; if `path.is_file()` is True, data is the nested dict from `tomllib.loads(path.read_text())`; if False, a warning is printed to stderr and data is `{}`; then for each (`env_name`, (`section`, `field`)) in `_ENV_MAP`, if `os.environ.get(env_name)` is not None, `data.setdefault(section, {})[field] = value`; finally `self._data = data`. If an exception is raised at line 5: `super().init` completed, no warning, `self._data` is not set by this method.
- **Code evidence**: Line 5: `data = tomllib.loads(path.read_text())`
- **Trigger condition**: The specification states that after construction, `call()` returns a dict with values resolved from environment, TOML (if the file is a valid TOML- containing regular file), or Pydantic defaults. When the file exists but is not valid TOML, step (2) does not apply, so the expected behavior is to use defaults. The code does not handle the `TOMLDecodeError` raised by `tomllib.loads()`, so `self._data` is never set, causing subsequent `call()` to fail with an `AttributeError` instead of returning the expected dict of defaults.
- **Validator trigger**: Constructing `_LayeredSource` with a path pointing to an existing file containing invalid TOML syntax causes an unhandled `tomllib.TOMLDecodeError` instead of treating the invalid TOML as empty and using defaults.
- **Probe stdout**: CONFIRMED — `TOMLDecodeError` raised during `init`; spec requires treating invalid TOML as empty/defaults instead of propagating the error. Exception: Expected ']' at the end of an array declaration (at line 1, column 8)

► SPEC sidecar (完整)

```
{
  "signature": "_LayeredSource.__init__(self, settings_cls, path: Path)",
  "pre_condition": "settings_cls is a Pydantic BaseSettings subclass with model fields defined; path is a Path object that may or may not reference an existing regular file",
}
```

```
"post_condition": "After construction, invoking __call__() on this instance returns a dict whose keys are exactly the model field names of settings_cls. For each field, the value is resolved by the following precedence (highest first): (1) the value of any process environment variable mapped to that field via the environment-to-field mapping, (2) the value from the TOML file at path when path references an existing regular file containing valid TOML syntax, (3) the field's Pydantic default. An environment variable value, when present, overrides a TOML value for the same field regardless of the TOML value's type. If path does not reference an existing regular file, a message identifying the missing file is written to stderr and the TOML source is treated as empty."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "tomllib.loads",
      "signature": "tomllib.loads(text: str) -> dict[str, Any]",
      "pre_condition": "text is a string containing valid TOML syntax",
      "post_condition": "Returns a nested dict representation of the TOML document, preserving section hierarchy. Top-level keys are section names; each section value is a dict of key-value pairs. All TOML value types (strings, integers, floats, booleans, arrays, inline tables) are preserved with their Python equivalents. Raises TOMLDecodeError if text is not syntactically valid TOML."
    },
    {
      "name": "os.environ.get",
      "signature": "os.environ.get(key: str) -> str | None",
      "pre_condition": "key is a non-empty string",
      "post_condition": "Returns the string value of the environment variable named key if it is set in the process environment; returns None if no such variable is set."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/config-py/_LayeredSource::__init__.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "After construction, invoking __call__() on this instance returns a dict whose keys are exactly the model field names of settings_cls. For each field, the value is resolved by the following precedence (highest first): (1) the value of any process environment variable mapped to that field via the environment-to-field mapping, (2) the value from the TOML file at path when path references an existing regular file containing valid TOML syntax, (3) the field's Pydantic default. An environment variable value, when present, overrides a TOML value for the same field regardless of the TOML value's type. If path does not reference an existing regular file, a message identifying the missing file is written to stderr and the TOML source is treated as empty.",
    "actual_behavior": "If no exception occurs during file reading and TOML parsing: super().__init__ completed; if path.is_file() is True, data is the nested dict from tomllib.loads(path.read_text()); if False, a warning is printed to stderr and data is {}; then for each (env_name, (section, field)) in _ENV_MAP, if os.environ.get(env_name) is not None, data.setdefault(section, {}) [field] = value; finally self._data = data. If an exception is raised at line 5: super().__init__ completed, no warning, self._data is not set by this method.",
    "code_evidence": "Line 5: data = tomllib.loads(path.read_text())",
    "trigger_condition": "The specification states that after construction, __call__() returns a dict with values resolved from environment, TOML (if the file is a valid TOML- containing regular file), or Pydantic defaults. When the file exists but is not valid TOML, step (2) does not apply, so the expected behavior is to use defaults. The code does not handle the TOMLDecodeError raised by tomllib.loads(), so self._data is never set, causing subsequent __call__() to fail with an AttributeError instead of returning the expected dict of defaults."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "config-py--_LayeredSource::__init__",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/config.py",
  "function_name": "_LayeredSource.__init__",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe-config-py--_LayeredSource::__init__.py",
  "detail_file": "fm_agent/bug_validation/config-py--_LayeredSource::__init__.md",
  "probe_stdout": "CONFIRMED - TOMLDecodeError raised during __init__; spec requires treating invalid TOML as empty/defaults instead of propagating the error. Exception: Expected ']]' at the end of an array declaration (at line 1, column 8)",
  "trigger_summary": "Constructing _LayeredSource with a path pointing to an existing file containing invalid TOML syntax causes an unhandled tomllib.TOMLDecodeError instead of treating the invalid TOML as empty and using defaults."
}
```

► Reasoner 流程 (3 条事件)

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-27T15:45:02.750878Z	3973
2	generate_block_post_condition	success	2	0/1	2026-07-27T15:45:54.917139Z	1766
3	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:46:04.151331Z	8541

事件 1 — [llm_de8ea9053a514e0c81eb886f001a6594](#)

- parse_error: LLM response contains multiple JSON values
- 载荷: [system_prompt](#) · [user_prompt](#) · [assistant_output](#)

事件 2 — [llm_33c60e73d7dc4d6383d9a8f8dd25e053](#)

► [parsed_json](#) (完整)

```
{
  "post_condition": "If no exception occurs during file reading and TOML parsing: super().__init__ completed; if path.is_file() is True, data is the nested dict from tomlib.loads(path.read_text()); if False, a warning is printed to stderr and data is {}; then for each (env_name, (section, field)) in _ENV_MAP, if os.environ.get(env_name) is not None, data.setdefault(section, {})[field] = value; finally self._data = data. If an exception is raised at line 5: super().__init__ completed, no warning, self._data is not set by this method."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_33c60e73d7dc4d6383d9a8f8dd25e053_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_33c60e73d7dc4d6383d9a8f8dd25e053_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_33c60e73d7dc4d6383d9a8f8dd25e053_message_02_assistant.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_33c60e73d7dc4d6383d9a8f8dd25e053_message_03_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_33c60e73d7dc4d6383d9a8f8dd25e053_response.txt\)](#)

事件 3 — [llm_2c806847ee8d4da0a6cc07567393397b](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "path = Path('/path/to/broken.toml') where the file exists and contains syntactically invalid TOML (e.g., 'not toml content'); settings_cls is any Pydantic Settings class with fields; _ENV_MAP is any non-empty mapping; no relevant environment variables set.",
  "offending_statements": "Line 5: data = tomlib.loads(path.read_text())",
  "reason": "The specification states that after construction, __call__() returns a dict with values resolved from environment, TOML (if the file is a valid TOML- containing regular file), or Pydantic defaults. When the file exists but is not valid TOML, step (2) does not apply, so the expected behavior is to use defaults. The code does not handle the TOMLDecodeError raised by tomlib.loads(), so self._data is never set, causing subsequent __call__() to fail with an AttributeError instead of returning the expected dict of defaults."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_2c806847ee8d4da0a6cc07567393397b_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_2c806847ee8d4da0a6cc07567393397b_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_2c806847ee8d4da0a6cc07567393397b_response.txt\)](#) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-003 — [src--configure_llm-py--fm_agent_config_path](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts 1
- Phase / 模块: Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a Path whose string representation equals the value obtained by resolving the FM_AGENT_CONFIG configuration key through the following precedence: (1) the process environment variable FM_AGENT_CONFIG, if set; (2) the value of the

- FM_AGENT_CONFIG key in the dotenv environment file at project_root/.env, if the file exists and contains the key; (3) the literal filename "fm-agent.toml", resolved relative to project_root.
- **Actual behavior:** The function returns a Path object. If the environment variable 'FM_AGENT_CONFIG' is set and its value is a non-empty string, the returned path is Path(os.environ['FM_AGENT_CONFIG']). Otherwise, if the environment variable is completely absent (None) and the .env file at project_root / '.env' contains a non-empty value for 'FM_AGENT_CONFIG', the returned path is Path(dotenv_values(project_root / '.env').get('FM_AGENT_CONFIG')). In all remaining cases (environment variable present but empty, or absent and .env entry missing or empty), the returned path is project_root / 'fm-agent.toml'. Formally: let env = os.environ.get('FM_AGENT_CONFIG'). If env is not None and env != "": result = Path(env). Else if env is None: let dotenv = dotenv_values(project_root / '.env').get('FM_AGENT_CONFIG'); if dotenv is not None and dotenv != "": result = Path(dotenv); else: result = project_root / 'fm-agent.toml'. Else (env == ""): result = project_root / 'fm-agent.toml'.
 - **Code evidence:** Line 7: return Path(explicit_config) if explicit_config else project_root / "fm-agent.toml"
 - **Trigger condition:** The specification states that if the environment variable FM_AGENT_CONFIG is set (i.e., present in os.environ), its value is used as the path, regardless of whether it is empty. The code incorrectly falls back to the default 'fm-agent.toml' when the variable is set but empty, because the ternary treats an empty string as falsy.
 - **Validator trigger:** FM_AGENT_CONFIG set to empty string triggers fallback to fm-agent.toml because ternary treats empty as falsy
 - **Probe stdout:** CONFIRMED — actual: PosixPath('/tmp/tmp7xzcd6p/fm-agent.toml') | expected: PosixPath('.')

► SPEC sidecar (完整)

```
{
  "signature": "_fm_agent_config_path(project_root: Path) -> Path",
  "pre_condition": "project_root is a Path identifying an existing directory.",
  "post_condition": "Returns a Path whose string representation equals the value obtained by resolving the FM_AGENT_CONFIG configuration key through the following precedence: (1) the process environment variable FM_AGENT_CONFIG, if set; (2) the value of the FM_AGENT_CONFIG key in the dotenv environment file at project_root/.env, if the file exists and contains the key; (3) the literal filename \"fm-agent.toml\", resolved relative to project_root."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/_fm_agent_config_path.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a Path whose string representation equals the value obtained by resolving the FM_AGENT_CONFIG configuration key through the following precedence: (1) the process environment variable FM_AGENT_CONFIG, if set; (2) the value of the FM_AGENT_CONFIG key in the dotenv environment file at project_root/.env, if the file exists and contains the key; (3) the literal filename \"fm-agent.toml\", resolved relative to project_root.",
    "actual_behavior": "The function returns a Path object. If the environment variable 'FM_AGENT_CONFIG' is set and its value is a non-empty string, the returned path is Path(os.environ['FM_AGENT_CONFIG']). Otherwise, if the environment variable is completely absent (None) and the .env file at project_root / '.env' contains a non-empty value for 'FM_AGENT_CONFIG', the returned path is Path(dotenv_values(project_root / '.env').get('FM_AGENT_CONFIG')). In all remaining cases (environment variable present but empty, or absent and .env entry missing or empty), the returned path is project_root / 'fm-agent.toml'. Formally: let env = os.environ.get('FM_AGENT_CONFIG'). If env is not None and env != '': result = Path(env). Else if env is None: let dotenv = dotenv_values(project_root / '.env').get('FM_AGENT_CONFIG'); if dotenv is not None and dotenv != '': result = Path(dotenv); else: result = project_root / 'fm-agent.toml'. Else (env == ''): result = project_root / 'fm-agent.toml'.",
    "code_evidence": "Line 7: return Path(explicit_config) if explicit_config else project_root / \"fm-agent.toml\"",
    "trigger_condition": "The specification states that if the environment variable FM_AGENT_CONFIG is set (i.e., present in os.environ), its value is used as the path, regardless of whether it is empty. The code incorrectly falls back to the default 'fm-agent.toml' when the variable is set but empty, because the ternary treats an empty string as falsy."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--configure_llm-py--_fm_agent_config_path",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/_fm_agent_config_path.py",
  "function_name": "_fm_agent_config_path",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--_fm_agent_config_path.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--_fm_agent_config_path.md",
}
```



```
"probe_stdout": "CONFIRMED - actual: PosixPath('/tmp/tmp7xzc6p/fm-agent.toml') | expected: PosixPath('.')",
"trigger_summary": "FM_AGENT_CONFIG set to empty string triggers fallback to fm-agent.toml because ternary treats empty as falsy"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:08:33.887641Z	2428
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:09:04.304761Z	3715

事件 1 — [llm_ff707c2ee4a94baabb00fe8336c1f257](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a Path object. If the environment variable 'FM_AGENT_CONFIG' is set and its value is a non-empty string, the returned path is Path(os.environ['FM_AGENT_CONFIG']). Otherwise, if the environment variable is completely absent (None) and the .env file at project_root / '.env' contains a non-empty value for 'FM_AGENT_CONFIG', the returned path is Path(dotenv_values(project_root / '.env').get('FM_AGENT_CONFIG')). In all remaining cases (environment variable present but empty, or absent and .env entry missing or empty), the returned path is project_root / 'fm-agent.toml'. Formally: let env = os.environ.get('FM_AGENT_CONFIG'). If env is not None and env != '': result = Path(env). Else if env is None: let dotenv = dotenv_values(project_root / '.env').get('FM_AGENT_CONFIG'); if dotenv is not None and dotenv != '': result = Path(dotenv); else: result = project_root / 'fm-agent.toml'. Else (env == ''): result = project_root / 'fm-agent.toml'."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ff707c2ee4a94baabb00fe8336c1f257_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ff707c2ee4a94baabb00fe8336c1f257_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ff707c2ee4a94baabb00fe8336c1f257_response.txt)

事件 2 — [llm_f7c74d8205e7400c8b00619d554e8e0d](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "project_root=Path('/project'), environment variable FM_AGENT_CONFIG='' (empty string), no .env file or .env file missing",
  "offending_statements": "Line 7: return Path(explicit_config) if explicit_config else project_root / \"fm-agent.toml\"",
  "reason": "The specification states that if the environment variable FM_AGENT_CONFIG is set (i.e., present in os.environ), its value is used as the path, regardless of whether it is empty. The code incorrectly falls back to the default 'fm-agent.toml' when the variable is set but empty, because the ternary treats an empty string as falsy."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f7c74d8205e7400c8b00619d554e8e0d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f7c74d8205e7400c8b00619d554e8e0d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f7c74d8205e7400c8b00619d554e8e0d_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-013 — [src--configure_llm-py--apply_configuration](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#); attempts 1
- Phase / 模块： Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Writes the API key to the .env file at paths.env_path, inserting or replacing the LLM_API_KEY line while preserving all other lines and their ordering unchanged. Writes the non-secret LLM configuration fields (provider_id, provider_name, base_url, api_style, model_id) to the [llm] section of fm-agent.toml at paths.toml_path, preserving every other TOML section, every field within those sections, and all comments exactly as they appeared before the call. Updates the OpenCode provider configuration at paths.opencode_config_path

to reference the provider identified by `config.provider_id` with the API style given by `config.api_style`; when the file does not exist, a new configuration structure is created. Before modifying any file, each file that will be written is backed up to a timestamp-suffixed copy in the same directory. When `validate` is `True`, the written TOML content is verified to be syntactically valid and the generated OpenCode configuration structure is verified to be well-formed before the function returns. The API key written to the provider secret file has its parent directory created with permissions `0700` and the file itself with permissions `0600`. Returns a list of (`modified_file_path`, `backup_path_or_None`) tuples, one entry for each file that was written or would have been written, ordered by the sequence in which the files were written. Raises `ConfigWizardError` when the TOML file at `paths.toml_path` does not exist or is unreadable, when any field of config required for writing is empty, or when `validate` is `True` and any generated configuration file fails its corresponding format validation.

- **Actual behavior:** After the function executes, one of the following holds:

1. Normal return (no exception):

- The return value is a list of 4 tuples of the form (`path`, `backup_path`), where:
 - Tuple 0: (`paths.toml_path`, `backup_file(paths.toml_path)`)
 - Tuple 1: (`paths.env_path`, `backup_file(paths.env_path, private=True)`)
 - Tuple 2: (`paths.opencode_config_path`, `backup_file(paths.opencode_config_path, private=True)`)
 - Tuple 3: (`opencode_secret_path`, `backup_file(opencode_secret_path, private=True)`) where `opencode_secret_path` = `secret_path_for_provider(config)`. For each tuple, the second element is either `None` (if the file did not exist before the backups were made) or a `Path` to a timestamp-suffixed backup copy. Backups with `private=True` have permissions `0600` (disallow group/other read).
- The file at `paths.toml_path` now contains the result of `update_fm_agent_toml_text` applied to the original TOML content. All TOML sections, fields, and comments outside the `[llm]` section are identical to the original. The `[llm]` sections `provider`, `name`, `base_url`, and `api_style` match `config.provider_id`, `config.provider_name`, `config.base_url`, and `config.api_style` respectively.
- The file at `paths.env_path` now contains the result of `update_env_text` applied to the original `.env` content (which was empty if the file did not exist). The `LLM_API_KEY` line is set to `config.api_key`; all other lines are unchanged in order and content.
- The file at `paths.opencode_config_path` contains the JSON representation (indented 2, `ensure_ascii=False`, followed by newline) of a dictionary that represents the merged OpenCode configuration. That dictionary includes a `provider` entry for `config.provider_id` with `provider`: `config.api_style` and `api_key_secret`: the absolute string representation of `opencode_secret_path`. Existing provider entries for other provider IDs are preserved exactly as they were in the original opencode configuration (or empty if none existed).
- The file at `opencode_secret_path` contains `config.api_key + ' '`. Its parent directory now exists and has permission bits `0o700` (`rw-x----`). The file itself has permission bits `0o600` (`rw-----`).
- The original files, if any, have been replaced atomically; no temporary or partial files remain visible at the end of the call.

2. ConfigWizardError raised:

- If the exception is raised by `validate_input` (only when a required field is empty, which does not occur under the given precondition) or by `validate_generated_state` (when the generated TOML or OpenCode configuration is syntactically invalid), the execution stops before any backups or file modifications. All four configuration paths (`toml`, `env`, `opencode`, `secret`) retain their original contents and permissions (or remain nonexistent). No backup files are created. The function does not return a value.

3. Unhandled I/O error (OSError, etc.) during backup or write:

- The function may raise an exception before completing all backups and writes. In this case:
 - The original configuration files (`toml`, `env`, `opencode`, `secret`) are not modified; they retain their previous contents (or nonexistence).
 - Some backup files may have been created for paths that were successfully backed up before the failure (in the order tuples 03). Backups are independent copies; a partial list of backups may exist on disk.
- The return value is never produced.

Under the given precondition (all config fields nonempty; `toml_path` exists and is readable; parent directories of `env_path` and `opencode_config_path` exist or can be created), the normal return path is always taken.

- **Code evidence:** Line 30-38: `backups = [ ... ]` (the order of the list differs from the write order)
- **Trigger condition:** The specification requires the returned list to be ordered by the sequence in which the files were written (`toml`, `env`, `secret`, `opencode config`). The code returns the list in the order `toml`, `env`, `opencode config`, `secret`, which is the order of backup creation, not the write order.
- **Validator trigger:** The returned list of backups is ordered by backup creation sequence (`toml`, `env`, `opencode_config`, `secret`) instead of file write sequence (`toml`, `env`, `secret`, `opencode_config`), swapping the last two entries.
- **Probe stdout:** CONFIRMED — Return order does not match write order. Spec (write order) : `[fm-agent.toml, .env, fm-agent-opencode-api-key.test-provider.793b2b6e7c, opencode.json]` Actual (backup order): `[fm-agent.toml, .env, opencode.json, fm-agent-opencode-api-key.test-provider.793b2b6e7c]` Mismatch at positions 2 and 3: spec expects `secret` before `opencode_config`

► SPEC sidecar (完整)

```
{
  "signature": "apply_configuration(config: LLMConfigInput, paths: WizardPaths, *, validate: bool = True) -> list[tuple[Path, Path | None]]",
  "pre_condition": "config is an LLMConfigInput whose provider_id, provider_name, api_style, base_url, model_id, and api_key fields are all non-empty strings. paths is a WizardPaths whose toml_path references an existing readable fm-agent.toml file. paths.env_path and paths.opencode_config_path reference paths whose parent directories exist or can be created.",
  "post_condition": "Writes the API key to the .env file at paths.env_path, inserting or replacing the LLM_API_KEY line while preserving all other lines and their ordering unchanged. Writes the non-secret LLM configuration fields (provider_id, provider_name, base_url, api_style, model_id) to the [llm] section of fm-agent.toml at paths.toml_path, preserving every other TOML section, every field within those sections, and all comments exactly as they appeared before the call. Updates the OpenCode provider configuration at paths.opencode_config_path to reference the provider identified by config.provider_id with the API style given by config.api_style; when the file does not exist, a new configuration structure is created. Before modifying any file, each file that will be written is backed up to a timestamp-suffixed copy in the same directory. When validate is True, the written TOML content is verified to be syntactically valid and the generated OpenCode configuration structure is verified to be well-formed before the function returns. The API key written to the provider secret file has its parent directory created with permissions 0700 and the file itself with permissions 0600. Returns a list of (modified_file_path, backup_path_or_None) tuples, one entry for each file that was written or would have been written, ordered by the sequence in which the files were written. Raises ConfigWizardError when the TOML file at paths.toml_path does not exist or is unreadable, when any field of config required for writing is empty, or when validate is True and any generated configuration file fails its corresponding format validation."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "validate_input",
      "signature": "validate_input(config: LLMConfigInput) -> None",
      "pre_condition": "config is an LLMConfigInput instance.",
      "post_condition": "Raises ConfigWizardError when any of provider_id, provider_name, api_style, base_url, model_id, or api_key is empty. Returns with no effect when all required fields are non-empty."
    },
    {
      "name": "_read_text_if_exists",
      "signature": "_read_text_if_exists(path: Path) -> str",
      "pre_condition": "path is a Path object.",
      "post_condition": "Returns the file's full text content as a string when the file exists and is readable. Returns an empty string when the file does not exist."
    },
    {
      "name": "secret_path_for_provider",
      "signature": "secret_path_for_provider(config: LLMConfigInput) -> Path",
      "pre_condition": "config is an LLMConfigInput with a non-empty provider_id.",
      "post_condition": "Returns a Path referencing the provider-specific secret key file whose location is determined by the platform-specific user configuration directory and config.provider_id."
    },
    {
      "name": "update_env_text",
      "signature": "update_env_text(env_text: str, api_key: str) -> str",
      "pre_condition": "env_text is a string containing zero or more KEY=VALUE lines separated by newlines; api_key is a non-empty string.",
      "post_condition": "Returns the .env content with the LLM_API_KEY line inserted or replaced with the provided api_key value. All other lines and their ordering are preserved unchanged."
    },
    {
      "name": "update_fm_agent_toml_text",
      "signature": "update_fm_agent_toml_text(toml_text: str, config: LLMConfigInput) -> str",
      "pre_condition": "toml_text is a string containing valid TOML content with an [llm] section; config is an LLMConfigInput with non-empty provider_id, provider_name, base_url, api_style, and model_id fields.",
      "post_condition": "Returns the TOML content with the provider, name, base_url, and api_style fields under [llm] updated to the values from config. All other TOML sections, fields, and comments are preserved exactly as they were."
    },
    {
      "name": "parse_existing_opencode_config",
      "signature": "parse_existing_opencode_config(opencode_text: str) -> dict",
      "pre_condition": "opencode_text is a string, possibly empty.",
      "post_condition": "Returns a dict representing the parsed OpenCode configuration when opencode_text is non-empty and contains valid JSON. Returns an empty dict when opencode_text is empty or not valid JSON."
    },
    {
      "name": "merge_opencode_config",
      "signature": "merge_opencode_config(existing: dict, config: LLMConfigInput, *, opencode_secret_path: Path) -> dict",
      "pre_condition": "existing is a dict representing the current OpenCode configuration; config is an LLMConfigInput with non-empty provider_id and api_style; opencode_secret_path is a Path to the provider secret file.",
      "post_condition": "Returns a dict representing the merged OpenCode configuration that includes a provider entry for config.provider_id configured with the given api_style and referencing the secret key file at opencode_secret_path. Existing provider entries for other provider IDs are preserved unchanged."
    },
    {
      "name": "validate_generated_state",
      "signature": "validate_generated_state(config: LLMConfigInput, merged_opencode: dict, updated_toml: str, *,
```

```

opencode_secret_path: Path) -> None",
    "pre_condition": "config is an LLMConfigInput; merged_opencode is a dict; updated_toml is a string; opencode_secret_path is a Path.",
    "post_condition": "Raises ConfigWizardError when updated_toml is not syntactically valid TOML or when merged_opencode is not well-formed OpenCode configuration. Returns with no effect when all generated outputs pass their respective format validation."
  },
  {
    "name": "backup_file",
    "signature": "backup_file(path: Path, *, private: bool = False) -> Path | None",
    "pre_condition": "path is a Path to a file that may or may not exist.",
    "post_condition": "When the file at path exists: copies it to a timestamp-suffixed filename in the same directory and returns the backup Path. When private is True and the copied file exists, sets its permissions to disallow group and other read access. When the file at path does not exist: returns None without creating anything."
  },
  {
    "name": "atomic_write",
    "signature": "atomic_write(path: Path, content: str) -> None",
    "pre_condition": "path is a Path whose parent directory exists and is writable; content is a non-empty string.",
    "post_condition": "Writes content to a temporary file and atomically renames it to path. After the call, the file at path contains exactly content. If a file already existed at path, its content is replaced."
  }
]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/apply_configuration.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Writes the API key to the .env file at paths.env_path, inserting or replacing the LLM_API_KEY line while preserving all other lines and their ordering unchanged. Writes the non-secret LLM configuration fields (provider_id, provider_name, base_url, api_style, model_id) to the [llm] section of fm-agent.toml at paths.toml_path, preserving every other TOML section, every field within those sections, and all comments exactly as they appeared before the call. Updates the OpenCode provider configuration at paths.opencode_config_path to reference the provider identified by config.provider_id with the API style given by config.api_style; when the file does not exist, a new configuration structure is created. Before modifying any file, each file that will be written is backed up to a timestamp-suffixed copy in the same directory. When validate is True, the written TOML content is verified to be syntactically valid and the generated OpenCode configuration structure is verified to be well-formed before the function returns. The API key written to the provider secret file has its parent directory created with permissions 0700 and the file itself with permissions 0600. Returns a list of (modified_file_path, backup_path_or_None) tuples, one entry for each file that was written or would have been written, ordered by the sequence in which the files were written. Raises ConfigWizardError when the TOML file at paths.toml_path does not exist or is unreadable, when any field of config required for writing is empty, or when validate is True and any generated configuration file fails its corresponding format validation.",
    "actual_behavior": "After the function executes, one of the following holds:\n\n1. **Normal return (no exception):**\n   - The return value is a list of 4 tuples of the form (path, backup_path), where:\n     - Tuple 0: (paths.toml_path, backup_file(paths.toml_path))\n     - Tuple 1: (paths.env_path, backup_file(paths.env_path, private=True))\n     - Tuple 2: (paths.opencode_config_path, backup_file(paths.opencode_config_path, private=True))\n     - Tuple 3: (opencode_secret_path, backup_file(opencode_secret_path, private=True))\n   - where opencode_secret_path = secret_path_for_provider(config).\n   - For each tuple, the second element is either None (if the file did not exist before the backups were made) or a Path to a timestampsuffixed backup copy. Backups with private=True have permissions 0600 (disallow group/other read).\n   - The file at paths.toml_path now contains the result of update_fm_agent_toml_text applied to the original TOML content. All TOML sections, fields, and comments outside the [llm] section are identical to the original. The [llm] sections provider, name, base_url, and api_style match config.provider_id, config.provider_name, config.base_url, and config.api_style respectively.\n   - The file at paths.env_path now contains the result of update_env_text applied to the original .env content (which was empty if the file did not exist). The LLM_API_KEY line is set to config.api_key; all other lines are unchanged in order and content.\n   - The file at paths.opencode_config_path contains the JSON representation (indented 2, ensure_ascii=False, followed by newline) of a dictionary that represents the merged OpenCode configuration. That dictionary includes a provider entry for config.provider_id with provider: config.api_style and api_key_secret: the absolute string representation of opencode_secret_path. Existing provider entries for other provider IDs are preserved exactly as they were in the original opencode configuration (or empty if none existed).\n   - The file at opencode_secret_path contains config.api_key + '\n'. Its parent directory now exists and has permission bits 0o700 (rwx---). The file itself has permission bits 0o600 (rw-----).\n   - The original files, if any, have been replaced atomically; no temporary or partial files remain visible at the end of the call.\n\n2. **ConfigWizardError raised:**\n   - If the exception is raised by validate_input (only when a required field is empty, which does not occur under the given precondition) or by validate_generated_state (when the generated TOML or OpenCode configuration is syntactically invalid), the execution stops before any backups or file modifications. All four configuration paths (toml, env, opencode, secret) retain their original contents and permissions (or remain nonexistent). No backup files are created. The function does not return a value.\n\n3. **Unhandled I/O error (OSError, etc.) during backup or write:**\n   - The function may raise an exception before completing all backups and writes. In this case:\n     - The original configuration files (toml, env, opencode, secret) are not modified; they retain their previous contents (or nonexistence).\n     - Some backup files may have been created for paths that were successfully backed up before the failure (in the order tuples 03). Backups are independent copies; a partial list of backups may exist on disk.\n   - The return value is never produced.\n\nUnder the given precondition (all config fields nonempty; toml_path exists and is readable; parent directories of env_path and opencode_config_path exist or can be created), the normal return path is always taken.",
    "code_evidence": "Line 30-38: backups = [ ... ] (the order of the list differs from the write order)",
    "trigger_condition": "The specification requires the returned list to be ordered by the sequence in which the files were written (toml, env, secret, opencode config). The code returns the list in the order toml, env, opencode config, secret, which is the order of backup creation, not the write order."
  }
}

```

► Bug Validator result (完整)

```
{
  "id": "src--configure_llm-py--apply_configuration",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/apply_configuration.py",
  "function_name": "apply_configuration",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--apply_configuration.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--apply_configuration.md",
  "probe_stdout": "CONFIRMED - Return order does not match write order.\n  Spec (write order) : [fm-agent.toml, .env, fm-agent-opencode-api-key.test-provider.793b2b6e7c, opencode.json]\n  Actual (backup order): [fm-agent.toml, .env, opencode.json, fm-agent-opencode-api-key.test-provider.793b2b6e7c]\n  Mismatch at positions 2 and 3: spec expects secret before opencode_config",
  "trigger_summary": "The returned list of backups is ordered by backup creation sequence (toml, env, opencode_config, secret) instead of file write sequence (toml, env, secret, opencode_config), swapping the last two entries."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:52:31.136432Z	4513
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:53:14.703306Z	7523

事件 1 — [llm_5ad3d441aae043fe9ee0cd101a649d85](#)

► parsed_json（完整）

```
{
  "post_condition": "After the function executes, one of the following holds:\n\n1. **Normal return (no exception):**\n  - The return value is a list of 4 tuples of the form (path, backup_path), where:\n    - Tuple 0: (paths.toml_path, backup_file(paths.toml_path))\n    - Tuple 1: (paths.env_path, backup_file(paths.env_path, private=True))\n    - Tuple 2: (paths.opencode_config_path, backup_file(paths.opencode_config_path, private=True))\n    - Tuple 3: (opencode_secret_path, backup_file(opencode_secret_path, private=True))\n  where opencode_secret_path = secret_path_for_provider(config).\n  For each tuple, the second element is either None (if the file did not exist before the backups were made) or a Path to a timestamp-suffixed backup copy. Backups with private=True have permissions 0600 (disallow group/other read).\n  - The file at paths.toml_path now contains the result of update_fm_agent_toml_text applied to the original TOML content. All TOML sections, fields, and comments outside the [llm] section are identical to the original. The [llm] section's provider, name, base_url, and api_style match config.provider_id, config.provider_name, config.base_url, and config.api_style respectively.\n  - The file at paths.env_path now contains the result of update_env_text applied to the original .env content (which was empty if the file did not exist). The LLM_API_KEY line is set to config.api_key; all other lines are unchanged in order and content.\n  - The file at paths.opencode_config_path contains the JSON representation (indented 2, ensure_ascii=False, followed by newline) of a dictionary that represents the merged OpenCode configuration. That dictionary includes a provider entry for config.provider_id with provider: config.api_style and api_key_secret: the absolute string representation of opencode_secret_path. Existing provider entries for other provider IDs are preserved exactly as they were in the original opencode configuration (or empty if none existed).\n  - The file at opencode_secret_path contains config.api_key + '\n'. Its parent directory now exists and has permission bits 0o700 (rwx---). The file itself has permission bits 0o600 (rw-----).\n  - The original files, if any, have been replaced atomically; no temporary or partial files remain visible at the end of the call.\n\n2. **ConfigWizardError raised:**\n  - If the exception is raised by validate_input (only when a required field is empty, which does not occur under the given pre-condition) or by validate_generated_state (when the generated TOML or OpenCode configuration is syntactically invalid), the execution stops before any backups or file modifications. All four configuration paths (toml, env, opencode, secret) retain their original contents and permissions (or remain non-existent). No backup files are created. The function does not return a value.\n\n3. **Unhandled I/O error (OSError, etc.) during backup or write:**\n  - The function may raise an exception before completing all backups and writes. In this case:\n    - The original configuration files (toml, env, opencode, secret) are not modified; they retain their previous contents (or non-existence).\n    - Some backup files may have been created for paths that were successfully backed up before the failure (in the order tuples 0-3). Backups are independent copies; a partial list of backups may exist on disk.\n  - The return value is never produced.\n\nUnder the given pre-condition (all config fields non-empty; toml_path exists and is readable; parent directories of env_path and opencode_config_path exist or can be created), the normal return path is always taken."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5ad3d441aae043fe9ee0cd101a649d85_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5ad3d441aae043fe9ee0cd101a649d85_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5ad3d441aae043fe9ee0cd101a649d85_response.txt)

事件 2 — [llm_4cb28e0f3ebe48eb9379198f718ce8df](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "config = LLMConfigInput(provider_id='p1', provider_name='P1', api_style='openai', base_url='http://example.com', model_id='m1', api_key='k'); paths = WizardPaths(toml_path=Path('/tmp/fm-agent.toml'), env_path=Path('/tmp/.env'), opencode_config_path=Path('/tmp/opencode.json')); assume /tmp/fm-agent.toml exists with minimal valid TOML content."
```

```
"offending_statements": "Line 30-38: backups = [ ... ] (the order of the list differs from the write order)",
"reason": "The specification requires the returned list to be ordered by the sequence in which the files were written (toml,
env, secret, opencode config). The code returns the list in the order toml, env, opencode config, secret, which is the order of
backup creation, not the write order."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4cb28e0f3ebe48eb9379198f718ce8df_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4cb28e0f3ebe48eb9379198f718ce8df_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4cb28e0f3ebe48eb9379198f718ce8df_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-014 — `src--configure_llm-py--apply_llm_settings_update`

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: `confirmed`; attempts `1`
- **Phase / 模块**: Phase 1 — Initialization & Configuration / `configuration`; 原源码 `src/configure_llm.py`
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: The file at `toml_path` is overwritten such that every key in updates has its value written to the corresponding field path in the `[llm]` section, while all other sections, fields, and the overall TOML structure outside the updated fields are preserved unchanged. Before the write, the original file content is copied to a timestamp-suffixed backup file adjacent to `toml_path`. Returns the backup file Path when a backup was created; returns None when no backup was created. Raises `ConfigWizardError` when `toml_path` does not refer to an existing file or when the updated TOML content is not parseable as valid TOML by the standard library parser.
- **Actual behavior**: After the function execution, one of the following holds: (1) The function raised a `ConfigWizardError` (with message `"fm-agent.toml not found at {toml_path}; refusing to guess a new project config."` or `"Generated fm-agent.toml is invalid TOML."`). In this case, the file at `toml_path` remains unchanged and no backup file is created. (2) The function returned a value `r` normally. Then the file at `toml_path` has been atomically replaced with the TOML text obtained by applying the given `updates` to the original file's `[llm]` section, preserving all other parts of the document. If `r` is a `Path` object, a backup copy of the original file content exists at the path `r` (adjacent to `toml_path` with a timestamp-suffixed name); the content at `r` equals the original file content before the call. If `r` is `None`, no backup copy was created.

Formally, let `orig` be the content of `toml_path` at function entry, and `updated = update_llm_settings_toml_text(orig, updates)`. Let `F_before` and `F_after` be the filesystem state mapping paths to contents immediately before and after the call.

- If the function raises a `ConfigWizardError`: `F_after = F_before` (no files changed or created).
- If the function returns `r` normally: `F_after(toml_path) = updated (r = None p adjacent to toml_path, F_after(p) = F_before(p) no new timestamp-suffixed backup file exists) (isinstance(r, Path) r is a path adjacent to toml_path r did not exist in F_before F_after(r) = orig all other files unchanged)`.
- **Code evidence**: Line 6: `if not toml_text`:
- **Trigger condition**: The code raises a `ConfigWizardError` for an existing empty file, claiming the file is not found, which violates the specification. The specification only allows raising `ConfigWizardError` when `toml_path` does not refer to an existing file or when the updated TOML content is unparseable. An empty file exists, so the code should not raise a 'file not found' error; it should instead proceed to generate updated TOML.
- **Validator trigger**: `apply_llm_settings_update` raises `ConfigWizardError` for an existing empty `fm-agent.toml` file because `_read_text_if_exists` returns `"` for empty files and the `'if not toml_text'` guard treats it identically to a missing file.
- **Probe stdout**: `CONFIRMED -- actual: ConfigWizardError raised: fm-agent.toml not found at /tmp/tmp5zk0ey_k/fm-agent.toml; refusing to guess a new project config. | expected: function should have proceeded normally for an existing empty file`

► SPEC sidecar (完整)

```
{
  "signature": "apply_llm_settings_update(updates: dict[str, str], toml_path: Path) -> Path | None",
  "pre_condition": "updates is a non-empty dict whose keys are dot-separated field paths within the [llm] section of the TOML document and whose values are non-empty strings or booleans representing new configuration values. toml_path identifies an existing, readable, and writable file whose content is valid TOML parseable by the standard library TOML parser.",
  "post_condition": "The file at toml_path is overwritten such that every key in updates has its value written to the corresponding field path in the [llm] section, while all other sections, fields, and the overall TOML structure outside the
```

```

updated fields are preserved unchanged. Before the write, the original file content is copied to a timestamp-suffixed backup file adjacent to toml_path. Returns the backup file Path when a backup was created; returns None when no backup was created. Raises ConfigWizardError when toml_path does not refer to an existing file or when the updated TOML content is not parseable as valid TOML by the standard library parser."
}

```

► INFO / callees sidecar (完整)

```

{
  "callees": [
    {
      "name": "_read_text_if_exists",
      "signature": "_read_text_if_exists(path: Path) -> str",
      "pre_condition": "path is a Path object identifying a filesystem location.",
      "post_condition": "Returns the UTF-8 decoded text content of the file at path. Returns an empty string when no file exists at path."
    },
    {
      "name": "update_llm_settings_toml_text",
      "signature": "update_llm_settings_toml_text(toml_text: str, updates: dict[str, str]) -> str",
      "pre_condition": "toml_text is a non-empty string containing valid TOML with a parseable [llm] section. updates is a non-empty dict whose keys are dot-separated field paths within the [llm] section and whose values are strings.",
      "post_condition": "Returns TOML text where each field path key in updates has its value replaced with the corresponding update value in the [llm] section. All other sections, fields, and TOML structure outside the updated fields are preserved unchanged in the returned text."
    },
    {
      "name": "backup_file",
      "signature": "backup_file(path: Path) -> Path | None",
      "pre_condition": "path identifies an existing, readable file.",
      "post_condition": "Copies the file at path to a new file with a timestamp-suffixed name adjacent to path. Returns the Path of the new backup file when the copy succeeded, or None when a backup could not be created."
    },
    {
      "name": "atomic_write",
      "signature": "atomic_write(path: Path, content: str) -> None",
      "pre_condition": "path is a writable filesystem location. content is a string.",
      "post_condition": "The file at path is overwritten with content such that the file never exists in a partially written state. After the call, the file content at path equals content."
    }
  ]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/apply_llm_settings_update.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "The file at toml_path is overwritten such that every key in updates has its value written to the corresponding field path in the [llm] section, while all other sections, fields, and the overall TOML structure outside the updated fields are preserved unchanged. Before the write, the original file content is copied to a timestamp-suffixed backup file adjacent to toml_path. Returns the backup file Path when a backup was created; returns None when no backup was created. Raises ConfigWizardError when toml_path does not refer to an existing file or when the updated TOML content is not parseable as valid TOML by the standard library parser.",
    "actual_behavior": "After the function execution, one of the following holds: (1) The function raised a `ConfigWizardError` (with message `\"fm-agent.toml not found at {toml_path}\"; refusing to guess a new project config.\"` or `\"Generated fm-agent.toml is invalid TOML.\"`). In this case, the file at `toml_path` remains unchanged and no backup file is created. (2) The function returned a value `r` normally. Then the file at `toml_path` has been atomically replaced with the TOML text obtained by applying the given `updates` to the original file's `[llm]` section, preserving all other parts of the document. If `r` is a `Path` object, a backup copy of the original file content exists at the path `r` (adjacent to `toml_path` with a timestamp-suffixed name); the content at `r` equals the original file content before the call. If `r` is `None`, no backup copy was created.\n\nFormally, let `orig` be the content of `toml_path` at function entry, and `updated` = `update_llm_settings_toml_text(orig, updates)`. Let `F_before` and `F_after` be the filesystem state mapping paths to contents immediately before and after the call.\n\n- If the function raises a `ConfigWizardError`: \n  `F_after` = `F_before` (no files changed or created).\n\n- If the function returns `r` normally: \n  `F_after(toml_path)` = `updated` \n  `(r = None & p adjacent to toml_path, F_after(p) = F_before(p) & no new timestamp-suffixed backup file exists)` \n  `(isinstance(r, Path) & r is a path adjacent to toml_path & r did not exist in F_before & F_after(r) = orig & all other files unchanged)`.",
    "code_evidence": "Line 6: if not toml_text:",
    "trigger_condition": "The code raises a ConfigWizardError for an existing empty file, claiming the file is not found, which violates the specification. The specification only allows raising ConfigWizardError when toml_path does not refer to an existing file or when the updated TOML content is unparseable. An empty file exists, so the code should not raise a 'file not found' error; it should instead proceed to generate updated TOML."
  }
}

```

► Bug Validator result (完整)

► Reasoner 流程 (2 条事件)

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:52:57.153471Z	5017
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:54:04.055816Z	9550

► parsed_json (完整)

```
- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ad914ce7efae43d2a4f031f367e53fcc_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_ad914ce7efae43d2a4f031f367e53fcc_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_ad914ce7efae43d2a4f031f367e53fcc_response.txt)
```

► `parsed_json` (完整)

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1937feee8eb4459ea24da1dc27796dc1_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_1937feee8eb4459ea24da1dc27796dc1_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_1937feee8eb4459ea24da1dc27796dc1_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts [1](#)
- **Phase / 模块**: Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Raises ConfigWizardError when paths.toml_path does not identify a readable fm-agent.toml file. Raises ConfigWizardError when the post-update TOML content is not parseable as valid TOML; when this occurs no file has been modified. Returns a tuple (backups, removed_overrides) where: removed_overrides is a tuple of zero or more environment variable names; for every name in removed_overrides, the corresponding legacy LLM override entry that existed in the project .env file before the call is absent from paths.env_path after the call, while all other .env entries are preserved; paths.env_path is modified if and only if removed_overrides is non-empty. backups is a list of (modified_path, backup_path) pairs. Its first element maps paths.toml_path to the path of a pre-modification backup of that file. When removed_overrides is non-empty, an additional element maps paths.env_path to the path of a pre-modification backup of that file. After return, paths.toml_path contains a valid TOML document in which the LLM-section backend field equals backend and every other LLM-section field has the same value it held before the call. No API key or OpenCode provider configuration is written.
- **Actual behavior**: Post-condition: If `_read_text_if_exists(paths.toml_path)` returns a falsy value (None or empty string), the function raises [ConfigWizardError](#) with a message indicating that fm-agent.toml was not found, and no files are created or modified. If `toml_text` is truthy but `tomllib.loads(updated_toml)` raises a `tomllib.TOMLDecodeError`, the function raises [ConfigWizardError](#) with the message 'Generated fm-agent.toml is invalid TOML.' and no files are written; both `paths.toml_path` and `paths.env_path` remain unchanged. Otherwise, the function completes normally and returns [\(backups, removed_overrides\)](#). In this success case:
 - `updated_toml` (valid TOML incorporating backend and preserved settings) has been atomically written to `paths.toml_path`.
 - If `removed_overrides` is non-empty, `paths.env_path` has been atomically overwritten with `updated_env` (the environment content with those overrides removed); if `removed_overrides` is empty, `paths.env_path` is left in its original state (unchanged or absent if it did not exist before).
 - A backup copy of the original content of `paths.toml_path` exists at the path returned by `backup_file(paths.toml_path)`. If `removed_overrides` is non-empty, a private backup of the original `paths.env_path` also exists at `backup_file(paths.env_path, private=True)`.
 - The returned list `backups` contains [\(paths.toml_path, backup_toml_path\)](#) and, if an env backup was made, [\(paths.env_path, backup_env_path\)](#). The second element `removed_overrides` is the tuple of environment variable names whose legacy overrides were removed.

Formally: (toml_text (raise ConfigWizardError (toml_path unchanged) (env_path unchanged))) (toml_text valid_TOML(updated_toml) (raise ConfigWizardError (toml_path unchanged) (env_path unchanged))) (toml_text valid_TOML(updated_toml) (backup_toml = backup_file(paths.toml_path) atomic_write(paths.toml_path, updated_toml) (removed_overrides = () env_unchanged) (removed_overrides () (updated_env, removed_overrides) = remove_legacy_llm_env_overrides(env_text) backup_env = backup_file(paths.env_path, private=True) atomic_write(paths.env_path, updated_env)) return (backups, removed_overrides) backuups = [(paths.toml_path, backup_toml)] ++ [(paths.env_path, backup_env)] if removed_overrides () else []))

- **Code evidence**: Line 5: `toml_text = _read_text_if_exists(paths.toml_path)`
- **Trigger condition**: When the file is not readable, `_read_text_if_exists` may raise an exception (such as `PermissionError`) that is not caught. The code does not raise `ConfigWizardError`, violating the specification's requirement that `ConfigWizardError` be raised when the toml file is not readable.
- **Validator trigger**: When `paths.toml_path` points to an existing fm-agent.toml with no read permissions, `_read_text_if_exists` raises `PermissionError` that is not caught and wrapped as `ConfigWizardError`.
- **Probe stdout**: `CONFIRMED` — spec requires `ConfigWizardError` for an unreadable toml file, but code raised `PermissionError` (uncaught): `PermissionError: [Errno 13] Permission denied: '/tmp/tmpbiuyjs_/project/fm-agent.toml'`

► SPEC sidecar (完整)

```
{
  "signature": "apply_local_backend_configuration(backend: str, paths: WizardPaths) -> tuple[list[tuple[Path, Path | None]], tuple[str, ...]]",
  "pre_condition": "backend is a non-empty string identifying a local CLI backend that is not 'opencode'. paths is a WizardPaths object whose toml_path and env_path attributes refer to filesystem paths.",
  "post_condition": "Raises ConfigWizardError when paths.toml_path does not identify a readable fm-agent.toml file. Raises ConfigWizardError when the post-update TOML content is not parseable as valid TOML; when this occurs no file has been modified. Returns a tuple (backups, removed_overrides) where: removed_overrides is a tuple of zero or more environment variable names; for every name in removed_overrides, the corresponding legacy LLM override entry that existed in the project .env file before the call is absent from paths.env_path after the call, while all other .env entries are preserved; paths.env_path is modified if and only if removed_overrides is non-empty. backups is a list of (modified_path, backup_path) pairs. Its first element maps paths.toml_path
```



```

the path of a pre-modification backup of that file. When removed_overrides is non-empty, an additional element maps
paths.env_path to the path of a pre-modification backup of that file. After return, paths.toml_path contains a valid TOML document
in which the LLM-section backend field equals backend and every other LLM-section field has the same value it held before the
call. No API key or OpenCode provider configuration is written."
}

```

► INFO / callees sidecar (完整)

```

{
  "callees": [
    {
      "name": "local_backend_toml_updates",
      "signature": "local_backend_toml_updates(env_path: Path, backend: str) -> dict[str, Any]",
      "pre_condition": "env_path is a Path that identifies a readable .env file or does not identify an existing file. backend is
a non-empty string that is not 'opencode'.",
      "post_condition": "Returns a non-empty dict mapping dot-separated TOML field paths within the LLM section to new values
along a key path that sets the LLM backend to backend. When model or effort values exist in the project .env or fm-agent.toml
configuration, their current values are preserved in the returned dict."
    },
    {
      "name": "update_llm_settings_toml_text",
      "signature": "update_llm_settings_toml_text(toml_text: str, updates: dict[str, Any]) -> str",
      "pre_condition": "toml_text is valid TOML text. updates is a non-empty dict whose keys are dot-separated field paths that
each resolve to a writable leaf location in the LLM section of toml_text.",
      "post_condition": "Returns valid TOML text in which every field path present in updates has been set to its corresponding
value and all other content of toml_text is unchanged."
    },
    {
      "name": "remove_legacy_llm_env_overrides",
      "signature": "remove_legacy_llm_env_overrides(env_text: str | None) -> tuple[str | None, tuple[str, ...]]",
      "pre_condition": "env_text is the text content of a .env file or None.",
      "post_condition": "Returns (updated_env, removed_overrides). removed_overrides is a tuple of environment variable names
whose legacy LLM override entries were present in env_text before the call and are absent from updated_env; every other entry from
the original env_text is preserved unchanged in updated_env. When env_text is None or empty, returns (env_text, [])."

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-
py/apply_local_backend_configuration.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Raises ConfigWizardError when paths.toml_path does not identify a readable fm-agent.toml file. Raises
ConfigWizardError when the post-update TOML content is not parseable as valid TOML; when this occurs no file has been modified.
Returns a tuple (backups, removed_overrides) where: removed_overrides is a tuple of zero or more environment variable names; for
every name in removed_overrides, the corresponding legacy LLM override entry that existed in the project .env file before the call
is absent from paths.env_path after the call, while all other .env entries are preserved; paths.env_path is modified if and only
if removed_overrides is non-empty. backups is a list of (modified_path, backup_path) pairs. Its first element maps paths.toml_path
to the path of a pre-modification backup of that file. When removed_overrides is non-empty, an additional element maps
paths.env_path to the path of a pre-modification backup of that file. After return, paths.toml_path contains a valid TOML document
in which the LLM-section backend field equals backend and every other LLM-section field has the same value it held before the
call. No API key or OpenCode provider configuration is written.",
    "actual_behavior": "Post-condition: If `_read_text_if_exists(paths.toml_path)` returns a falsy value (None or empty string),
the function raises `ConfigWizardError` with a message indicating that fm-agent.toml was not found, and no files are created or
modified. If `toml_text` is truthy but `tomllib.loads(updated_toml)` raises a `tomllib.TOMLDecodeError`, the function raises
`ConfigWizardError` with the message 'Generated fm-agent.toml is invalid TOML.' and no files are written; both `paths.toml_path`
and `paths.env_path` remain unchanged. Otherwise, the function completes normally and returns `(backups, removed_overrides)`. In
this success case:\n- `updated_toml` (valid TOML incorporating backend and preserved settings) has been atomically written to
`paths.toml_path`.\n- If `removed_overrides` is non-empty, `paths.env_path` has been atomically overwritten with `updated_env`
(the environment content with those overrides removed); if `removed_overrides` is empty, `paths.env_path` is left in its original
state (unchanged or absent if it did not exist before).\n- A backup copy of the original content of `paths.toml_path` exists at
the path returned by `backup_file(paths.toml_path)`. If `removed_overrides` is non-empty, a private backup of the original
`paths.env_path` also exists at `backup_file(paths.env_path, private=True)`. \n- The returned list `backups` contains
`(paths.toml_path, backup_toml_path)` and, if an env backup was made, `(paths.env_path, backup_env_path)`. The second element
`removed_overrides` is the tuple of environment variable names whose legacy overrides were removed.\n\nFormally:\n(toml_text

```

```
(raise ConfigWizardError (toml_path unchanged) (env_path unchanged))) \n(toml_text valid_TOML(updated_toml) (raise
ConfigWizardError (toml_path unchanged) (env_path unchanged))) \n(toml_text valid_TOML(updated_toml) \n ( backup_toml =
backup_file(paths.toml_path) \n atomic_write(paths.toml_path, updated_toml) \n (removed_overrides = () env_unchanged) \n
(removed_overrides () \n (updated_env, removed_overrides) = remove_legacy_llm_env_overrides(env_text) \n backup_env =
backup_file(paths.env_path, private=True) \n atomic_write(paths.env_path, updated_env)) \n return (backups,
removed_overrides) \n backups = [(paths.toml_path, backup_toml)] ++ [(paths.env_path, backup_env)] if removed_overrides ()
else []))",
"code_evidence": "Line 5: toml_text = _read_text_if_exists(paths.toml_path)",
"trigger_condition": "When the file is not readable, _read_text_if_exists may raise an exception (such as PermissionError)
that is not caught. The code does not raise ConfigWizardError, violating the specification's requirement that ConfigWizardError be
raised when the toml file is not readable."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--apply_local_backend_configuration",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-
py/apply_local_backend_configuration.py",
  "function_name": "apply_local_backend_configuration",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--apply_local_backend_configuration.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--apply_local_backend_configuration.md",
  "probe_stdout": "CONFIRMED - spec requires ConfigWizardError for an unreadable toml file, but code raised PermissionError
(uncaught): PermissionError: [Errno 13] Permission denied: '/tmp/tmpbiyujys_/project/fm-agent.toml'\n",
  "trigger_summary": "When paths.toml_path points to an existing fm-agent.toml with no read permissions, _read_text_if_exists
raises PermissionError that is not caught and wrapped as ConfigWizardError."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:00:06.479020Z	5629
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:00:59.086741Z	9346

事件 1 — [llm_d75ee5c2c7fb4662a966fa581808c2d3](#)

► parsed_json（完整）

```
{
  "post_condition": "Post-condition: If `_read_text_if_exists(paths.toml_path)` returns a falsy value (None or empty string), the
function raises `ConfigWizardError` with a message indicating that fm-agent.toml was not found, and no files are created or
modified. If `toml_text` is truthy but `tomllib.loads(updated_toml)` raises a `tomllib.TOMLDecodeError`, the function raises
`ConfigWizardError` with the message 'Generated fm-agent.toml is invalid TOML.' and no files are written; both `paths.toml_path`
and `paths.env_path` remain unchanged. Otherwise, the function completes normally and returns `(backups, removed_overrides)`. In
this success case:\n- `updated_toml` (valid TOML incorporating backend and preserved settings) has been atomically written to
`paths.toml_path`.\n- If `removed_overrides` is non-empty, `paths.env_path` has been atomically overwritten with `updated_env`
(the environment content with those overrides removed); if `removed_overrides` is empty, `paths.env_path` is left in its original
state (unchanged or absent if it did not exist before).\n- A backup copy of the original content of `paths.toml_path` exists at
the path returned by `backup_file(paths.toml_path)`. If `removed_overrides` is non-empty, a private backup of the original
`paths.env_path` also exists at `backup_file(paths.env_path, private=True)`. \n- The returned list `backups` contains
`(paths.toml_path, backup_toml_path)` and, if an env backup was made, `(paths.env_path, backup_env_path)`. The second element
`removed_overrides` is the tuple of environment variable names whose legacy overrides were removed.\n\nFormally:\n(¬toml_text →
(raise ConfigWizardError ∧ (toml_path unchanged) ∧ (env_path unchanged))) ∧ (toml_text ∧ ¬valid_TOML(updated_toml) → (raise
ConfigWizardError ∧ (toml_path unchanged) ∧ (env_path unchanged))) ∧ (toml_text ∧ valid_TOML(updated_toml) → \n (∃ backup_toml =
backup_file(paths.toml_path) ∧ \n atomic_write(paths.toml_path, updated_toml) ∧ \n (removed_overrides = () → env_unchanged) ∧ \n
(removed_overrides ≠ () → \n (updated_env, removed_overrides) = remove_legacy_llm_env_overrides(env_text) ∧ \n backup_env
= backup_file(paths.env_path, private=True) ∧ \n atomic_write(paths.env_path, updated_env)) ∧ \n return (backups,
removed_overrides) ∧ \n backups = [(paths.toml_path, backup_toml)] ++ [(paths.env_path, backup_env)] if removed_overrides ≠ ()
else []))"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d75ee5c2c7fb4662a966fa581808c2d3_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d75ee5c2c7fb4662a966fa581808c2d3_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d75ee5c2c7fb4662a966fa581808c2d3_response.txt)

事件 2 — [llm_dbbc01df69145eda9077c35fbd68a7d](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "paths.toml_path points to an existing file with no read permission (e.g., chmod 000). backend is a valid non-empty string not equal to 'opencode'.",
  "offending_statements": "Line 5: toml_text = _read_text_if_exists(paths.toml_path)",
  "reason": "When the file is not readable, _read_text_if_exists may raise an exception (such as PermissionError) that is not caught. The code does not raise ConfigWizardError, violating the specification's requirement that ConfigWizardError be raised when the toml file is not readable."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_dbbcb01df69145eda9077c35fbd68a7d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_dbbcb01df69145eda9077c35fbd68a7d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_dbbcb01df69145eda9077c35fbd68a7d_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-016 — [src--configure_llm-py--backup_file](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** When path does not exist, returns None without modifying the filesystem. When path exists and is readable, returns a Path identifying a newly created regular file whose content is byte-for-byte identical to the content of the file at path at the time of the call. The backup file is created in the same directory as path when private is false, and in a private backup directory with owner-restricted permissions when private is true. When path.name is '.env', the backup file permissions restrict read and write access to the owner regardless of the private flag. When private is true, the backup file permissions likewise restrict read and write access to the owner. Every call produces a distinct backup destination no existing backup file is overwritten. When a filesystem operation after destination reservation fails, the reserved empty destination file is removed before the exception propagates to the caller.
- **Actual behavior:** If path does not exist, the function returns None without filesystem changes. If path exists, the function creates a unique backup file named using a timestamp (now or current time) and a disambiguation counter; the backup is created exclusively with mode 0o600 if private is True or path.name == '.env', else 0o644. On successful copy (shutil.copy2), permissions are forced to 0o600 for sensitive sources, and the backup path is returned. If the copy fails, the backup is removed (best-effort) and the exception is re-raised; other exceptions (e.g., from directory creation or exclusive file creation) propagate without cleanup. Formally: (path.exists() return = None filesystem unchanged) (path.exists() ! backup such that (backup created (copy succeeds return = backup backup is copy of path with forced permissions) (copy fails (unlink(backup) attempted; exception re-raised)))).
- **Code evidence:** Line 42: if private or path.name == ".env": Line 43: backup.chmod(0o600)
- **Trigger condition:** The specification requires that when a filesystem operation after destination reservation fails, the reserved empty destination file is removed before the exception propagates. The code only wraps shutil.copy2 in a try/except that performs cleanup; the subsequent chmod for sensitive files (private or .env) is outside that protection. If that chmod fails, the exception propagates without unlinking the backup file, leaving an empty file on disk.
- **Validator trigger:** backup.chmod(0o600) outside try/except: if chmod fails after copy2 succeeds, the backup file is left orphaned on disk without cleanup.
- **Probe stdout:** CONFIRMED — orphaned backup left on disk: ['.env.bak.20260728-001205']

► SPEC sidecar (完整)

```
{
  "signature": "backup_file(path: Path, now: datetime | None = None, *, private: bool = False) -> Path | None",
  "pre_condition": "path is a Path to a filesystem entry that may or may not exist as a readable regular file.",
  "post_condition": "When path does not exist, returns None without modifying the filesystem. When path exists and is readable, returns a Path identifying a newly created regular file whose content is byte-for-byte identical to the content of the file at path at the time of the call. The backup file is created in the same directory as path when private is false, and in a private backup directory with owner-restricted permissions when private is true. When path.name is '.env', the backup file permissions restrict read and write access to the owner regardless of the private flag. When private is true, the backup file permissions likewise restrict read and write access to the owner. Every call produces a distinct backup destination – no existing backup file is overwritten. When a filesystem operation after destination reservation fails, the reserved empty destination file is removed before the exception propagates to the caller."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/backup_file.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When path does not exist, returns None without modifying the filesystem. When path exists and is readable, returns a Path identifying a newly created regular file whose content is byte-for-byte identical to the content of the file at path at the time of the call. The backup file is created in the same directory as path when private is false, and in a private backup directory with owner-restricted permissions when private is true. When path.name is '.env', the backup file permissions restrict read and write access to the owner regardless of the private flag. When private is true, the backup file permissions likewise restrict read and write access to the owner. Every call produces a distinct backup destination no existing backup file is overwritten. When a filesystem operation after destination reservation fails, the reserved empty destination file is removed before the exception propagates to the caller.",
    "actual_behavior": "If path does not exist, the function returns None without filesystem changes. If path exists, the function creates a unique backup file named using a timestamp (now or current time) and a disambiguation counter; the backup is created exclusively with mode 0o600 if private is True or path.name == '.env', else 0o644. On successful copy (shutil.copy2), permissions are forced to 0o600 for sensitive sources, and the backup path is returned. If the copy fails, the backup is removed (best-effort) and the exception is re-raised; other exceptions (e.g., from directory creation or exclusive file creation) propagate without cleanup. Formally: (path.exists() return = None filesystem unchanged) (path.exists() ! backup such that (backup created (copy succeeds return = backup backup is copy of path with forced permissions) (copy fails (unlink(backup) attempted; exception re-raised))))",
    "code_evidence": "Line 42:     if private or path.name == \".env\":\nLine 43:         backup.chmod(0o600)",
    "trigger_condition": "The specification requires that when a filesystem operation after destination reservation fails, the reserved empty destination file is removed before the exception propagates. The code only wraps shutil.copy2 in a try/except that performs cleanup; the subsequent chmod for sensitive files (private or .env) is outside that protection. If that chmod fails, the exception propagates without unlinking the backup file, leaving an empty file on disk."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--backup_file",
  "source_file": "src/configure_llm.py",
  "function_name": "backup_file",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--backup_file.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--backup_file.md",
  "probe_stdout": "CONFIRMED - orphaned backup left on disk: ['.env.bak.20260728-001205']\n",
  "trigger_summary": "backup.chmod(0o600) outside try/except: if chmod fails after copy2 succeeds, the backup file is left orphaned on disk without cleanup."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:08:19.868588Z	4202
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:09:27.101917Z	4227

事件 1 — [llm_297b0df4e56040dea529eb91397dc154](#)

► parsed_json（完整）

```
{
  "post_condition": "If path does not exist, the function returns None without filesystem changes. If path exists, the function creates a unique backup file named using a timestamp (now or current time) and a disambiguation counter; the backup is created exclusively with mode 0o600 if private is True or path.name == '.env', else 0o644. On successful copy (shutil.copy2), permissions are forced to 0o600 for sensitive sources, and the backup path is returned. If the copy fails, the backup is removed (best-effort) and the exception is re-raised; other exceptions (e.g., from directory creation or exclusive file creation) propagate without cleanup. Formally: (~path.exists() => return = None ^ filesystem unchanged) ^ (path.exists() => ∃! backup such that (backup created ^ (copy succeeds => return = backup ^ backup is copy of path with forced permissions) v (copy fails => (unlink(backup) attempted; exception re-raised))))"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_297b0df4e56040dea529eb91397dc154_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_297b0df4e56040dea529eb91397dc154_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_297b0df4e56040dea529eb91397dc154_response.txt)

事件 2 — llm_3b067af153eb418c8b432f8cb2949d06

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "path=PosixPath('/mnt/vfat/.env'), now=None, private=False; the .env file exists and is readable on a vfat filesystem where chmod is not supported (e.g., Linux mount of vfat). The exclusive creation and copy succeed, then the forced chmod at line 43 raises OSError, and because the chmod is not protected by the cleanup logic (which only guards the copy), the empty backup file is left behind, violating the specification's requirement that any filesystem operation failure after destination reservation results in removal of the reserved file.",
  "offending_statements": "Line 42:         if private or path.name == \".env\":\nLine 43:             backup.chmod(0o600)",
  "reason": "The specification requires that when a filesystem operation after destination reservation fails, the reserved empty destination file is removed before the exception propagates. The code only wraps shutil.copy2 in a try/except that performs cleanup; the subsequent chmod for sensitive files (private or .env) is outside that protection. If that chmod fails, the exception propagates without unlinking the backup file, leaving an empty file on disk."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_3b067af153eb418c8b432f8cb2949d06_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_3b067af153eb418c8b432f8cb2949d06_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_3b067af153eb418c8b432f8cb2949d06_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-017 — src--configure_llm-py--default_paths

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a WizardPaths object whose toml_path, env_path, and opencode_config_path attributes each refer to an existing or creatable filesystem path. toml_path is the value of the FM_AGENT_CONFIG environment variable when set, otherwise <project_root>/fm-agent.toml. env_path is <project_root>/env. opencode_config_path is the value of the OPENCODE_CONFIG environment variable when set, otherwise the path to the OpenCode configuration file (opencode.jsonc when present, otherwise opencode.json) in the platform-specific OpenCode configuration directory.
- **Actual behavior**: If project_root is a Path identifying an existing directory, then calling default_paths(project_root) returns a new WizardPaths instance w such that w.project_root == project_root, w.env_path == project_root / '.env', w.toml_path equals the result of _fm_agent_config_path(project_root) (i.e., the Path given by the FM_AGENT_CONFIG environment variable if set, otherwise project_root / 'fm-agent.toml'), and w.opencode_config_path equals the result of detect_opencode_config_path() (i.e., the Path given by the OPENCODE_CONFIG environment variable if set, otherwise the Path to the existing opencode.jsonc in the platform-specific OpenCode configuration directory, or if that file does not exist, the Path to opencode.json in that same directory). No other part of the program state is modified. Formally: w, w = default_paths(project_root) w.project_root = project_root w.env_path = project_root / ".env" (let f = os.environ.get("FM_AGENT_CONFIG") in (f None w.toml_path = Path(f)) (f None w.toml_path = project_root / "fm-agent.toml")) (let o = os.environ.get("OPENCODE_CONFIG") in (o None w.opencode_config_path = Path(o)) (o None (let dir = platform_opencode_config_dir in (isfile(dir / "opencode.jsonc") w.opencode_config_path = dir / "opencode.jsonc") (isfile(dir / "opencode.jsonc") w.opencode_config_path = dir / "opencode.json")))).
- **Code evidence**: Line 6: opencode_config_path=detect_opencode_config_path()
- **Trigger condition**: Specification B requires that every returned path attribute refers to a path that is existing or creatable. The code does not validate environment variables; when OPENCODE_CONFIG is set to a reserved name like 'CON', the returned path is a reserved device name that cannot be created as a regular file, violating the specification.
- **Validator trigger**: Setting OPENCODE_CONFIG to a non-creatable filesystem path (e.g., inside /proc/) causes default_paths() to return an invalid opencode_config_path without any validation.
- **Probe stdout**: CONFIRMED — opencode_config_path (/proc/nonexistent_dir_0123456/test.json) is not an existing or creatable regular-file path. The code returns the raw OPENCODE_CONFIG env-var value without validating that the resulting path can be used.

► SPEC sidecar (完整)


```
{
  "signature": "default_paths(project_root: Path) -> WizardPaths",
  "pre_condition": "project_root is a Path identifying an existing directory.",
  "post_condition": "Returns a WizardPaths object whose toml_path, env_path, and opencode_config_path attributes each refer to an
existing or creatable filesystem path. toml_path is the value of the FM_AGENT_CONFIG environment variable when set, otherwise
<project_root>/fm-agent.toml. env_path is <project_root>/env. opencode_config_path is the value of the OPENCODE_CONFIG
environment variable when set, otherwise the path to the OpenCode configuration file (opencode.jsonc when present, otherwise
opencode.json) in the platform-specific OpenCode configuration directory."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_fm_agent_config_path",
      "signature": "_fm_agent_config_path(project_root: Path) -> Path",
      "pre_condition": "project_root is a Path identifying an existing directory.",
      "post_condition": "Returns a Path: the value of the FM_AGENT_CONFIG environment variable when set, otherwise
<project_root>/fm-agent.toml."
    },
    {
      "name": "detect_opencode_config_path",
      "signature": "detect_opencode_config_path() -> Path",
      "pre_condition": "No caller guarantees required.",
      "post_condition": "Returns a Path to the OpenCode configuration file: the value of the OPENCODE_CONFIG environment variable
when set; otherwise, opencode.jsonc when it exists in the platform-specific OpenCode configuration directory, or opencode.json
otherwise."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/default_paths.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a WizardPaths object whose toml_path, env_path, and opencode_config_path attributes each refer to an
existing or creatable filesystem path. toml_path is the value of the FM_AGENT_CONFIG environment variable when set, otherwise
<project_root>/fm-agent.toml. env_path is <project_root>/env. opencode_config_path is the value of the OPENCODE_CONFIG
environment variable when set, otherwise the path to the OpenCode configuration file (opencode.jsonc when present, otherwise
opencode.json) in the platform-specific OpenCode configuration directory.",
    "actual_behavior": "If project_root is a Path identifying an existing directory, then calling default_paths(project_root)
returns a new WizardPaths instance w such that w.project_root == project_root, w.env_path == project_root / '.env', w.toml_path
equals the result of _fm_agent_config_path(project_root) (i.e., the Path given by the FM_AGENT_CONFIG environment variable if set,
otherwise project_root / 'fm-agent.toml'), and w.opencode_config_path equals the result of detect_opencode_config_path() (i.e.,
the Path given by the OPENCODE_CONFIG environment variable if set, otherwise the Path to the existing opencode.jsonc in the
platformspecific OpenCode configuration directory, or if that file does not exist, the Path to opencode.json in that same
directory). No other part of the program state is modified. Formally:\n w, w = default_paths(project_root) w.project_root =
project_root w.env_path = project_root / \"env\" (let f = os.environ.get(\"FM_AGENT_CONFIG\") in (f None w.toml_path =
Path(f)) (f = None w.toml_path = project_root / \"fm-agent.toml\")) (let o = os.environ.get(\"OPENCODE_CONFIG\") in (o None
w.opencode_config_path = Path(o)) (o = None (let dir = platform_opencode_config_dir in (isfile(dir / \"opencode.jsonc\")
w.opencode_config_path = dir / \"opencode.jsonc\") ( isfile(dir / \"opencode.jsonc\") w.opencode_config_path = dir /
\"opencode.json\"))))).",
    "code_evidence": "Line 6: opencode_config_path=detect_opencode_config_path()",
    "trigger_condition": "Specification B requires that every returned path attribute refers to a path that is existing or
creatable. The code does not validate environment variables; when OPENCODE_CONFIG is set to a reserved name like 'CON', the
returned path is a reserved device name that cannot be created as a regular file, violating the specification."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--configure_llm-py--default_paths",
  "source_file": "src/configure_llm.py",
  "function_name": "default_paths",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--default_paths.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--default_paths.md",
  "probe_stdout": "CONFIRMED - opencode_config_path (/proc/nonexistent_dir_0123456/test.json) is not an existing or creatable
regular-file path. The code returns the raw OPENCODE_CONFIG env-var value without validating that the resulting path can be
used.\n",
  "trigger_summary": "Setting OPENCODE_CONFIG to a non-creatable filesystem path (e.g., inside /proc/) causes default_paths() to
```

```
return an invalid opencode_config_path without any validation."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:00:11.231783Z	2359
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:04:00.258840Z	4476

事件 1 — [llm_cb79b9e63bbe4d80a11b36c9ae9f2c8e](#)

► parsed_json（完整）

```
{
  "post_condition": "If project_root is a Path identifying an existing directory, then calling default_paths(project_root) returns a new WizardPaths instance w such that w.project_root == project_root, w.env_path == project_root / '.env', w.toml_path equals the result of _fm_agent_config_path(project_root) (i.e., the Path given by the FM_AGENT_CONFIG environment variable if set, otherwise project_root / 'fm-agent.toml'), and w.opencode_config_path equals the result of detect_opencode_config_path() (i.e., the Path given by the OPENCODE_CONFIG environment variable if set, otherwise the Path to the existing opencode.jsonc in the platform-specific OpenCode configuration directory, or if that file does not exist, the Path to opencode.json in that same directory). No other part of the program state is modified. Formally:\n∀ w, w = default_paths(project_root) → w.project_root = project_root ∧ w.env_path = project_root / \".env\" ∧ (let f = os.environ.get(\"FM_AGENT_CONFIG\") in (f ≠ None → w.toml_path = Path(f)) ∧ (f = None → w.toml_path = project_root / \"fm-agent.toml\")) ∧ (let o = os.environ.get(\"OPENCODE_CONFIG\") in (o ≠ None → w.opencode_config_path = Path(o)) ∧ (o = None → (let dir = platform_opencode_config_dir in (isfile(dir / \"opencode.jsonc\") → w.opencode_config_path = dir / \"opencode.jsonc\") ∧ (¬ isfile(dir / \"opencode.jsonc\") → w.opencode_config_path = dir / \"opencode.json\")))))."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_cb79b9e63bbe4d80a11b36c9ae9f2c8e_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_cb79b9e63bbe4d80a11b36c9ae9f2c8e_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_cb79b9e63bbe4d80a11b36c9ae9f2c8e_response.txt)

事件 2 — [llm_81461e76bcc1424487cbe6b95a4209c0](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "On a Windows system, set environment variable OPENCODE_CONFIG to 'CON' (a reserved device name). Call default_paths(Path('C:\\\\existing_dir')) where the directory exists. The returned WizardPaths has opencode_config_path = Path('CON'), which is not an existing or creatable filesystem path.",
  "offending_statements": "Line 6: opencode_config_path=detect_opencode_config_path()",
  "reason": "Specification B requires that every returned path attribute refers to a path that is existing or creatable. The code does not validate environment variables; when OPENCODE_CONFIG is set to a reserved name like 'CON', the returned path is a reserved device name that cannot be created as a regular file, violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_81461e76bcc1424487cbe6b95a4209c0_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_81461e76bcc1424487cbe6b95a4209c0_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_81461e76bcc1424487cbe6b95a4209c0_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-018 — [src--configure_llm-py--detect_opencode_config_path](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts 2
- Phase / 模块： Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a Path identifying a candidate location for the OpenCode configuration file. The returned path is determined by the first matching rule in this precedence: (1) when the OPENCODE_CONFIG environment variable is set and non-empty, the variable value

- expanded as a user path; (2) when the `OPENCODE_CONFIG_DIR` environment variable is set and non-empty, `opencode.jsonc` within that directory if the file exists, otherwise `opencode.json` within that directory; (3) otherwise, `opencode.jsonc` within the platform-specific OpenCode configuration directory if the file exists, otherwise `opencode.json` within that same directory. The platform-specific configuration directory is: on Windows, the value of the `APPDATA` environment variable appended with `opencode` when set, otherwise the home directory joined with `AppData/Roaming/opencode`; on other platforms, the value of the `XDG_CONFIG_HOME` environment variable appended with `opencode` when set, otherwise the home directory joined with `.config/opencode`. When home is not None, it replaces `Path.home()` in the default home directory derivation for the platform-specific case only. The returned path may refer to a file that does not yet exist.
- **Actual behavior:** The return value is a Path object `p`. If the environment variable '`OPENCODE_CONFIG`' was set at function entry, then `p = Path(os.environ['OPENCODE_CONFIG']).expanduser()`. Otherwise, let `base_dir` be computed as: if '`OPENCODE_CONFIG_DIR`' was set, `base_dir = Path(os.environ['OPENCODE_CONFIG_DIR']).expanduser()`; else, let `home_dir = home` if home is not None else `Path.home()`; then if `os.name == 'nt'`, if '`APPDATA`' is set, `base_dir = Path(os.environ['APPDATA']) / 'opencode'`, else `base_dir = home_dir / 'AppData' / 'Roaming' / 'opencode'`; else (non-Windows), if '`XDG_CONFIG_HOME`' is set, `base_dir = Path(os.environ['XDG_CONFIG_HOME']) / 'opencode'`, else `base_dir = home_dir / '.config' / 'opencode'`. Then, if `base_dir / 'opencode.jsonc'` exists, `p = base_dir / 'opencode.jsonc'`, else `p = base_dir / 'opencode.json'`. No files are created or modified, and the state of environment variables is unchanged. The input home, if not None, is a Path referring to a directory.
 - **Code evidence:** Line 10: `home = home or Path.home()`
 - **Trigger condition:** The specification states that when 'home' is not None, it replaces `Path.home()` in the default home directory derivation. However, the code on line 10 uses '`home or Path.home()`', which treats a falsy home value (such as an empty Path) as equivalent to None, falling back to `Path.home()` and thus violating the required behavior.
 - **Validator trigger:** Passing a falsy non-None value (e.g., 0) as home causes `home or Path.home()` to fall through to `Path.home()`, violating the spec that any non-None home should replace `Path.home()`.
 - **Probe stdout:** CONFIRMED — actual (home=0): `PosixPath('/home/fancy/.config/opencode/opencode.json')` | expected: would use 0 directly, not fall back to None (`PosixPath('/home/fancy/.config/opencode/opencode.json')`)

► SPEC sidecar (完整)

```
{
  "signature": "detect_opencode_config_path(home: Path | None = None) -> Path",
  "pre_condition": "home, when not None, is a Path referring to a directory.",
  "post_condition": "Returns a Path identifying a candidate location for the OpenCode configuration file. The returned path is
determined by the first matching rule in this precedence: (1) when the OPENCODE_CONFIG environment variable is set and non-empty,
the variable value expanded as a user path; (2) when the OPENCODE_CONFIG_DIR environment variable is set and non-empty,
opencode.jsonc within that directory if the file exists, otherwise opencode.json within that directory; (3) otherwise,
opencode.jsonc within the platform-specific OpenCode configuration directory if the file exists, otherwise opencode.json within
that same directory. The platform-specific configuration directory is: on Windows, the value of the APPDATA environment variable
appended with opencode when set, otherwise the home directory joined with AppData/Roaming/opencode; on other platforms, the value
of the XDG_CONFIG_HOME environment variable appended with opencode when set, otherwise the home directory joined with
.config/opencode. When home is not None, it replaces Path.home() in the default home directory derivation for the platform-
specific case only. The returned path may refer to a file that does not yet exist."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-
py/detect_opencode_config_path.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a Path identifying a candidate location for the OpenCode configuration file. The returned path is
determined by the first matching rule in this precedence: (1) when the OPENCODE_CONFIG environment variable is set and non-empty,
the variable value expanded as a user path; (2) when the OPENCODE_CONFIG_DIR environment variable is set and non-empty,
opencode.jsonc within that directory if the file exists, otherwise opencode.json within that directory; (3) otherwise,
opencode.jsonc within the platform-specific OpenCode configuration directory if the file exists, otherwise opencode.json within
that same directory. The platform-specific configuration directory is: on Windows, the value of the APPDATA environment variable
appended with opencode when set, otherwise the home directory joined with AppData/Roaming/opencode; on other platforms, the value
of the XDG_CONFIG_HOME environment variable appended with opencode when set, otherwise the home directory joined with
.config/opencode. When home is not None, it replaces Path.home() in the default home directory derivation for the platform-
specific case only. The returned path may refer to a file that does not yet exist.",
    "actual_behavior": "The return value is a Path object p. If the environment variable 'OPENCODE_CONFIG' was set at function
entry, then p = Path(os.environ['OPENCODE_CONFIG']).expanduser(). Otherwise, let base_dir be computed as: if 'OPENCODE_CONFIG_DIR'
was set, base_dir = Path(os.environ['OPENCODE_CONFIG_DIR']).expanduser(); else, let home_dir = home if home is not None else
Path.home(); then if os.name == 'nt', if 'APPDATA' is set, base_dir = Path(os.environ['APPDATA']) / 'opencode', else base_dir =
```

```
home_dir / 'AppData' / 'Roaming' / 'opencode'; else (non-Windows), if 'XDG_CONFIG_HOME' is set, base_dir =
Path(os.environ['XDG_CONFIG_HOME']) / 'opencode', else base_dir = home_dir / '.config' / 'opencode'. Then, if base_dir /
'opencode.jsonc' exists, p = base_dir / 'opencode.jsonc', else p = base_dir / 'opencode.json'. No files are created or modified,
and the state of environment variables is unchanged. The input home, if not None, is a Path referring to a directory.",
    "code_evidence": "Line 10: home = home or Path.home()",
    "trigger_condition": "The specification states that when 'home' is not None, it replaces Path.home() in the default home
directory derivation. However, the code on line 10 uses 'home or Path.home()', which treats a falsy home value (such as an empty
Path) as equivalent to None, falling back to Path.home() and thus violating the required behavior."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--detect_opencode_config_path",
  "source_file": "src/configure_llm.py",
  "function_name": "detect_opencode_config_path",
  "confirmation_status": "confirmed",
  "attempts": 2,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--detect_opencode_config_path.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--detect_opencode_config_path.md",
  "probe_stdout": "CONFIRMED - actual (home=0): PosixPath('/home/fancy/.config/opencode/opencode.json') | expected: would use 0
directly, not fall back to None (PosixPath('/home/fancy/.config/opencode/opencode.json'))\n",
  "trigger_summary": "Passing a falsy non-None value (e.g., 0) as home causes `home or Path.home()` to fall through to
Path.home(), violating the spec that any non-None home should replace Path.home()".
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:08:23.876175Z	3584
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:09:10.385428Z	4125

事件 1 — [llm_ee8576cc65cf40268da916888293ebda](#)

► parsed_json（完整）

```
{
  "post_condition": "The return value is a Path object p. If the environment variable 'OPENCODE_CONFIG' was set at function entry,
then p = Path(os.environ['OPENCODE_CONFIG']).expanduser(). Otherwise, let base_dir be computed as: if 'OPENCODE_CONFIG_DIR' was
set, base_dir = Path(os.environ['OPENCODE_CONFIG_DIR']).expanduser(); else, let home_dir = home if home is not None else
Path.home(); then if os.name == 'nt', if 'APPDATA' is set, base_dir = Path(os.environ['APPDATA']) / 'opencode', else base_dir =
home_dir / 'AppData' / 'Roaming' / 'opencode'; else (non-Windows), if 'XDG_CONFIG_HOME' is set, base_dir =
Path(os.environ['XDG_CONFIG_HOME']) / 'opencode', else base_dir = home_dir / '.config' / 'opencode'. Then, if base_dir /
'opencode.jsonc' exists, p = base_dir / 'opencode.jsonc', else p = base_dir / 'opencode.json'. No files are created or modified,
and the state of environment variables is unchanged. The input home, if not None, is a Path referring to a directory."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ee8576cc65cf40268da916888293ebda_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ee8576cc65cf40268da916888293ebda_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ee8576cc65cf40268da916888293ebda_response.txt)

事件 2 — [llm_983e3cffe9ed49c9b2e1c79b78f6b0bd](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Call the function with home=Path('') under a non-Windows OS when no relevant environment variables are set.
The function replaces the empty Path with Path.home(), returning a path under the user's home directory instead of a path relative
to an empty directory as required by the specification.",
  "offending_statements": "Line 10: home = home or Path.home()",
  "reason": "The specification states that when 'home' is not None, it replaces Path.home() in the default home directory
derivation. However, the code on line 10 uses 'home or Path.home()', which treats a falsy home value (such as an empty Path) as
equivalent to None, falling back to Path.home() and thus violating the required behavior."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_983e3cffe9ed49c9b2e1c79b78f6b0bd_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_983e3cffe9ed49c9b2e1c79b78f6b0bd_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_983e3cffe9ed49c9b2e1c79b78f6b0bd_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-019 — `src--configure_llm-py--main`

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: `confirmed`; attempts 1
- **Phase / 模块**: Phase 1 — Initialization & Configuration / `configuration`; 原源码 `src/configure_llm.py`
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Returns 0 on successful completion of the requested configuration operation. Returns 1 if execution is interrupted by KeyboardInterrupt. Returns 2 if the operation fails due to invalid inputs, file access errors, or malformed existing configuration, with a descriptive error message written to stderr. On successful completion (return 0): the fm-agent.toml file at the project root (derived from the script's own filesystem location) reflects the configuration changes requested; the API key is persisted to the project .env file; the OpenCode provider configuration references the chosen provider. On failure (return 2): no configuration files are modified beyond any backups created before the failure point. The project root is resolved relative to the script's own location, not from the current working directory or any argument.
- **Actual behavior**: After the code block executes, the following outcomes are possible:
 1. If `_parse_args(argv)` determines that the arguments are invalid or unrecognised, the process terminates (via system exit) with a nonzero exit code before `main` returns. No further side effects from `main` occur.
 2. Otherwise, `args` is a properly parsed namespace, and `project_root` is the directory one level above the script's location. Then:
 - If `args.command == "set"`, `run_llm_settings_update(project_root, args.updates, assume_yes=args.yes)` is invoked. This function may modify `fm-agent.toml`, create a backup, and write providerspecific configuration files; it returns 0 on success or a nonzero exit code on failure. `main` returns that same integer.
 - If `args.command` is anything else, `run_wizard(project_root)` is called. It conducts an interactive configuration flow and may persist settings to `fm-agent.toml`, `.env`, and the OpenCode provider config. On success it returns 0; on user abort or validation failure it returns a nonzero exit code. `main` returns that integer, unless `run_wizard` raises a `ConfigWizardError`, which is caught and causes `main` to return 2 after printing the error to stderr.
 3. If a `KeyboardInterrupt` is raised at any point, the handler prints `\nAborted.` and `main` returns 1.
 4. Any other exception (not `KeyboardInterrupt` or `ConfigWizardError`) propagates out of `main`; the function does not catch it, so the program may terminate with a traceback and a nonzero exit code.

Formally, let `argv` be the input list or `None`. Let `result` be the value returned by `main` if it returns, and `exception` be any exception raised but not caught. Define the predicate `parse_ok(argv)` to be true when `_parse_args` succeeds without terminating the process. The postcondition is:

```
IF NOT parse_ok(argv) THEN
    process_terminated_with_non_zero_exit_code
ELSE
    args = parsed_result
    project_root = Path(__file__).resolve().parents[1]
    IF args.command == "set" THEN
        result = run_llm_settings_update(project_root, args.updates, args.yes)
        (result = 0  settings_updated_successfully)
        (result  0  update_failed)
    ELSE
        IF run_wizard(project_root) raises ConfigWizardError THEN
            stderr_contains("Error: " + exc_message)
            result = 2
        ELSE
            result = run_wizard(project_root)
            (result = 0  wizard_completed_successfully)
            (result  0  wizard_aborted_or_validation_failed)
        FI
    FI
FI
FI
AND
IF KeyboardInterrupt raised THEN
    stdout_contains("\nAborted.")
    result = 1
AND
IF any_other_exception_raised AND NOT caught THEN
```

```
exception = that_exception
program_terminates_with_non_zero_exit_code
```

In all cases where `main` returns, the return value is an integer. The side effects on files (`fmagent.toml`, `.env`, provider config) and terminal output are exactly those produced by the executed branches.

- **Code evidence:** Line 3: `args = _parse_args(argv)`
- **Trigger condition:** The specification requires that if the operation fails due to invalid inputs, `main` returns 2. Passing an invalid argument (e.g., `--invalid-flag`) causes `_parse_args` to terminate the process via `sys.exit` with a non-zero exit code, preventing `main` from returning at all. Therefore, `main` does not return 2 as required.
- **Validator trigger:** Passing an unrecognized argument causes `argparse` to raise `SystemExit(2)` instead of `main()` returning 2 as required by the specification.
- **Probe stdout:** `usage: probe_src--configure_llm-py--main.py [-h] {set} ... probe_src--configure_llm-py--main.py: error: unrecognized arguments: --invalid-flag CONFIRMED — main() raised SystemExit(2) instead of returning 2`

► SPEC sidecar (完整)

```
{
  "signature": "main(argv: list[str] | None) -> int",
  "pre_condition": "None required; when argv is None, the system argument list is used instead.",
  "post_condition": "Returns 0 on successful completion of the requested configuration operation. Returns 1 if execution is interrupted by KeyboardInterrupt. Returns 2 if the operation fails due to invalid inputs, file access errors, or malformed existing configuration, with a descriptive error message written to stderr. On successful completion (return 0): the fm-agent.toml file at the project root (derived from the script's own filesystem location) reflects the configuration changes requested; the API key is persisted to the project .env file; the OpenCode provider configuration references the chosen provider. On failure (return 2): no configuration files are modified beyond any backups created before the failure point. The project root is resolved relative to the script's own location, not from the current working directory or any argument."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_parse_args",
      "signature": "_parse_args(argv: list[str] | None) -> Namespace",
      "pre_condition": "argv is a list of zero or more strings; when None, the parser uses the process argument list.",
      "post_condition": "Returns a parsed-arguments object with a 'command' attribute whose value is either 'set' or the string representing the default (non-set) mode. When command is 'set', the returned object additionally carries attributes for the updates to apply (key-value pairs mapping TOML field paths to new values) and a boolean flag indicating whether confirmation prompts should be skipped. On unrecognized flags or invalid argument values, the parser terminates the process with a non-zero exit code."
    },
    {
      "name": "run_llm_settings_update",
      "signature": "run_llm_settings_update(project_root: Path, updates: dict, assume_yes: bool) -> int",
      "pre_condition": "project_root is a Path to a directory containing a readable fm-agent.toml file; updates is a dict whose keys are TOML field paths and values are new configuration values (strings or booleans); assume_yes is True when confirmation prompts should be suppressed.",
      "post_condition": "Returns 0 when the requested fields in fm-agent.toml are updated to the provided values and the original file is backed up. Returns a non-zero exit code when the update cannot be completed (e.g., file not writable, invalid key path). Before applying changes, prints the planned modifications to stdout; when assume_yes is False, prompts for confirmation first."
    },
    {
      "name": "run_wizard",
      "signature": "run_wizard(project_root: Path) -> int",
      "pre_condition": "project_root is a Path to a directory containing a readable fm-agent.toml file.",
      "post_condition": "Returns 0 when the user has successfully completed the interactive configuration flow and all resulting configuration has been persisted to fm-agent.toml, .env, and the OpenCode provider config. Returns a non-zero exit code when the user aborts or when validation of user-provided values fails. May raise ConfigWizardError for unrecoverable configuration errors. The API key is written to .env and to a provider-specific key file; non-secret settings are written to fm-agent.toml; the OpenCode provider entry is synchronized with the selected backend and provider."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/main.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns 0 on successful completion of the requested configuration operation. Returns 1 if execution is interrupted by KeyboardInterrupt. Returns 2 if the operation fails due to invalid inputs, file access errors, or malformed
```

```

"actual_behavior": "After the code block executes, the following outcomes are possible:\n\n1. If `_parse_args(argv)` determines that the arguments are invalid or unrecognised, the process terminates (via system exit) with a nonzero exit code before `main` returns. No further side effects from `main` occur.\n\n2. Otherwise, `args` is a properly parsed namespace, and `project_root` is the directory one level above the script's location. Then:\n    - If `args.command == \"set\"`, `run_llm_settings_update(project_root, args.updates, assume_yes=args.yes)` is invoked. This function may modify `fm-agent.toml`, create a backup, and write providerspecific configuration files; it returns 0 on success or a nonzero exit code on failure. `main` returns that same integer.\n    - If `args.command` is anything else, `run_wizard(project_root)` is called. It conducts an interactive configuration flow and may persist settings to `fm-agent.toml`, `.env`, and the OpenCode provider config. On success it returns 0; on user abort or validation failure it returns a nonzero exit code. `main` returns that integer, unless `run_wizard` raises a `ConfigWizardError`, which is caught and causes `main` to return 2 after printing the error to stderr.\n\n3. If a `KeyboardInterrupt` is raised at any point, the handler prints `\\nAborted.` and `main` returns 1.\n\n4. Any other exception (not `KeyboardInterrupt` or `ConfigWizardError`) propagates out of `main`; the function does not catch it, so the program may terminate with a traceback and a nonzero exit code.\n\nFormally, let `argv` be the input list or `None`. Let `result` be the value returned by `main` if it returns, and `exception` be any exception raised but not caught. Define the predicate `parse_ok(argv)` to be true when `_parse_args` succeeds without terminating the process. The postcondition is:\n\n```\nIF NOT parse_ok(argv) THEN\n  process_terminated_with_non_zero_exit_code\nELSE\n  args = parsed_result\n  project_root = Path(__file__).resolve().parents[1]\n  IF args.command == \"set\" THEN\n    result = run_llm_settings_update(project_root, args.updates, args.yes)\n    (result = 0 settings_updated_successfully)\n  ELSE\n    IF run_wizard(project_root) raises ConfigWizardError THEN\n      stderr_contains(\"Error: \" + exc_message)\n    result = 2\n    ELSE\n      result = run_wizard(project_root)\n      (result = 0 wizard_completed_successfully)\n    (result = 0 wizard_aborted_or_validation_failed)\n  FI\n  FI\n  IF KeyboardInterrupt raised THEN\n    stdout_contains(\"\\nAborted.\")\n    result = 1\n  AND\n  IF any_other_exception_raised AND NOT caught THEN\n    exception = that_exception\n  program_terminates_with_non_zero_exit_code\n```\n\nIn all cases where `main` returns, the return value is an integer. The side effects on files (`fmagent.toml`, `.env`, provider config) and terminal output are exactly those produced by the executed branches."

```

Bug Validator result (完整)

Reasoner 流程 (2 条事件)

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:36:52.379327Z	2816
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:37:22.242048Z	5670

► parsed json (完整)

```
{
  "post_condition": "After the code block executes, the following outcomes are possible:\n\n1. If `_parse_args(argv)` determines that the arguments are invalid or unrecognised, the process terminates (via system exit) with a non-zero exit code before `main` returns. No further side effects from `main` occur.\n2. Otherwise, `args` is a properly parsed namespace, and `project_root` is the directory one level above the script's location. Then:\n    - If `args.command == \"set\"`, `run_llm_settings_update(project_root, args.updates, assume_yes=args.yes)` is invoked. This function may modify `fm-agent.toml`, create a backup, and write provider-specific configuration files; it returns 0 on success or a non-zero exit code on failure. `main` returns that same integer.\n    - If `args.command` is anything else, `run_wizard(project_root)` is called. It conducts an interactive configuration flow and may persist settings to `fm-agent.toml`, `.env`, and the OpenCode provider config. On success it returns 0; on user abort or validation failure it returns a non-zero exit code. `main` returns that integer, unless `run_wizard` raises a `ConfigWizardError`, which is caught and causes `main` to return 2 after printing the error to stderr.\n3. If a `KeyboardInterrupt` is raised at any point, the handler prints `\\nAborted.` and `main` returns 1.\n4. Any other exception
```



```
(not `KeyboardInterrupt` or `ConfigWizardError`) propagates out of `main`; the function does not catch it, so the program may
terminate with a traceback and a non-zero exit code.\n\nFormally, let `argv` be the input list or `None`. Let `result` be the
value returned by `main` if it returns, and `exception` be any exception raised but not caught. Define the predicate
`parse_ok(argv)` to be true when `_parse_args` succeeds without terminating the process. The post-condition is:\n\n```\nIF NOT
parse_ok(argv) THEN\n    process_terminated_with_non_zero_exit_code\nELSE\n    args = parsed_result\n    project_root =
Path(__file__).resolve().parents[1]\n    IF args.command == \"set\" THEN\n        result = run_llm_settings_update(project_root,
args.updates, args.yes)\n        (result = 0 => settings_updated_successfully)\n        (result != 0 => update_failed)\n    ELSE\n    IF run_wizard(project_root) raises ConfigWizardError THEN\n        stderr_contains(\"Error: \" + exc_message)\n        result = 2\n    ELSE\n        result = run_wizard(project_root)\n        (result = 0 =>
wizard_completed_successfully)\n        (result != 0 => wizard_aborted_or_validation_failed)\n    FI\n    FI\nFI\nFI\nAND\nIF
KeyboardInterrupt raised THEN\n    stdout_contains(\"\\nAborted.\")\n    result = 1\nAND\nIF any_other_exception_raised AND NOT
caught THEN\n    exception = that_exception\n    program_terminates_with_non_zero_exit_code\n```\n\nIn all cases where `main`
returns, the return value is an integer. The side effects on files (`fm-agent.toml`, `.env`, provider config) and terminal output
are exactly those produced by the executed branches.\"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c989285c76c94a1288d09e2c0450532e_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_c989285c76c94a1288d09e2c0450532e_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_c989285c76c94a1288d09e2c0450532e_response.txt)

事件 2 — llm_01ce331b736a4732999424e1669fbc5c

▶ parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "argv = ['--invalid-flag']",
  "offending_statements": "Line 3: args = _parse_args(argv)",
  "reason": "The specification requires that if the operation fails due to invalid inputs, main returns 2. Passing an invalid
argument (e.g., '--invalid-flag') causes _parse_args to terminate the process via sys.exit with a non-zero exit code, preventing
main from returning at all. Therefore, main does not return 2 as required."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_01ce331b736a4732999424e1669fbc5c_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_01ce331b736a4732999424e1669fbc5c_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_01ce331b736a4732999424e1669fbc5c_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-020 — src--configure_llm-py--merge_opencode_config

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: **confirmed**; attempts 1
- Phase / 模块: Phase 1 — Initialization & Configuration / **configuration**; 原源码 [src/configure_llm.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim:** Returns a new dict not sharing identity with existing. The returned dict includes a '\$schema' key with a fixed schema URL value. The returned dict contains a 'provider' key whose value is a dict where: provider entries from existing for IDs other than config.provider_id are preserved unchanged; an entry for config.provider_id is present with an 'npm' field set according to config.api_style, an 'options' dict containing 'baseURL' set to config.base_url and 'apiKey' set to a file-reference string that points to opencode_secret_path, and a 'models' dict containing all previously existing model entries under config.provider_id plus an entry for config.model_id. When existing or any of its nested fields (provider, config.provider_id entry, options, models) is absent, that field is treated as if it were an empty dict for the merge. When any such field exists but is not a dict, raises ConfigWizardError.
- Actual behavior:** Post-condition: If any of the following type checks fail: the existing 'provider' field (if present) is not a dict, or the entry for config.provider_id is not a dict, or that entry's 'options' or 'models' field is not a dict, or the model entry for config.model_id is not a dict, then a ConfigWizardError is raised with an appropriate message and the existing dict remains completely unmodified. Otherwise, the function returns a new dict (call it result) and existing is unchanged. result is formed as follows: it is a deep copy of existing, then: if '*schema*' was not present in existing, result['schema'] is set to SCHEMA_URL. All other top-level keys remain as in the deep copy. The 'provider' key is a new dict where each existing provider entry is deep-copied, except for config.provider_id. The value for config.provider_id (call it merged_entry) is constructed by: taking a deep copy of the original provider entry if it existed, otherwise using an empty dict; then setting merged_entry['npm'] = adapter_for_api_style(config.api_style); then setting merged_entry['options'] to a new dict that is a deep copy of the original options (if any, else {}) but with 'baseURL' overwritten to config.base_url and 'apiKey' overwritten to the string '{file:{opencode_secret_path}}'; and setting merged_entry['models'] to a dict that is a shallow copy of the original models (if any, else {}) with the key config.model_id pointing to the exact same dict object as the original model entry if it existed, otherwise a new empty dict.

result is then returned. Formally: (isinstance(providers, dict) isinstance(entry, dict) isinstance(options, dict) isinstance(models, dict) isinstance(existing_model, dict)) (ConfigWizardError raised k in existing: existing[k] unchanged) ; else result is a dict such that: result['*schema*'] = *SCHEMA_URL* if '*schema*' not in existing else existing['\$schema'] (through deepcopy); let P = deepcopy(existing.get('provider', {})); then P' = P with P'[config.provider_id] = deepcopy(existing.get('provider', {}).get(config.provider_id, {})) updated: (npm = adapter_for_api_style(config.api_style), options = (deepcopy(original_options) {baseURL: config.base_url, apiKey: 'file:'+opencode_secret_path+'})), models = (shallow_copy(original_models) {config.model_id: original_model_entry if existed else {})); result['provider'] = P' and all other top-level keys equal their deep-copied counterparts from existing.

- **Code evidence:** Line 8: if "*schema*" *not in merged* : *Line 9 : merged*["*schema*"] = *SCHEMA_URL*
- **Trigger condition:** The specification requires the returned dict to always include a '*schema*' key set to a fixed *schema URL*, regardless of any existing value. The code only sets it when not already present, so an existing '*schema*' is preserved instead of being overwritten.
- **Validator trigger:** When existing dict already contains a '\$schema' key, the function preserves it instead of overwriting with *SCHEMA_URL* as the specification requires.
- **Probe stdout:** CONFIRMED — actual: 'https://example.com/not-the-right-schema' | expected: 'https://opencode.ai/config.json'

► SPEC sidecar (完整)

```
{
  "signature": "merge_opencode_config(existing: dict, config: LLMConfigInput, *, opencode_secret_path: Path) -> dict",
  "pre_condition": "existing is a dict representing the current OpenCode configuration. config is an LLMConfigInput with non-empty provider_id and api_style fields. opencode_secret_path is a Path to the provider secret key file.",
  "post_condition": "Returns a new dict not sharing identity with existing. The returned dict includes a '$schema' key with a fixed schema URL value. The returned dict contains a 'provider' key whose value is a dict where: provider entries from existing for IDs other than config.provider_id are preserved unchanged; an entry for config.provider_id is present with an 'npm' field set according to config.api_style, an 'options' dict containing 'baseURL' set to config.base_url and 'apiKey' set to a file-reference string that points to opencode_secret_path, and a 'models' dict containing all previously existing model entries under config.provider_id plus an entry for config.model_id. When existing or any of its nested fields (provider, config.provider_id entry, options, models) is absent, that field is treated as if it were an empty dict for the merge. When any such field exists but is not a dict, raises ConfigWizardError."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "adapter_for_api_style",
      "signature": "adapter_for_api_style(api_style: str) -> str",
      "pre_condition": "api_style is a non-empty string identifying a supported API style.",
      "post_condition": "Returns the npm package name string of the OpenCode adapter corresponding to the given API style."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/merge_opencode_config.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a new dict not sharing identity with existing. The returned dict includes a '$schema' key with a fixed schema URL value. The returned dict contains a 'provider' key whose value is a dict where: provider entries from existing for IDs other than config.provider_id are preserved unchanged; an entry for config.provider_id is present with an 'npm' field set according to config.api_style, an 'options' dict containing 'baseURL' set to config.base_url and 'apiKey' set to a file-reference string that points to opencode_secret_path, and a 'models' dict containing all previously existing model entries under config.provider_id plus an entry for config.model_id. When existing or any of its nested fields (provider, config.provider_id entry, options, models) is absent, that field is treated as if it were an empty dict for the merge. When any such field exists but is not a dict, raises ConfigWizardError.",
    "actual_behavior": "Post-condition: If any of the following type checks fail: the existing 'provider' field (if present) is not a dict, or the entry for config.provider_id is not a dict, or that entry's 'options' or 'models' field is not a dict, or the model entry for config.model_id is not a dict, then a ConfigWizardError is raised with an appropriate message and the existing dict remains completely unmodified. Otherwise, the function returns a new dict (call it result) and existing is unchanged. result is formed as follows: it is a deep copy of existing, then: if '$schema' was not present in existing, result['$schema'] is set to SCHEMA_URL. All other top-level keys remain as in the deep copy. The 'provider' key is a new dict where each existing provider entry is deep-copied, except for config.provider_id. The value for config.provider_id (call it merged_entry) is constructed by: taking a deep copy of the original provider entry if it existed, otherwise using an empty dict; then setting merged_entry['npm'] = adapter_for_api_style(config.api_style); then setting merged_entry['options'] to a new dict that is a deep copy of the original options (if any, else {}) but with 'baseURL' overwritten to config.base_url and 'apiKey' overwritten to the string '{{file: {opencode_secret_path}}}' ; and setting merged_entry['models'] to a dict that is a shallow copy of the original models (if any, else {}) with the key config.model_id pointing to the exact same dict object as the original model entry if it existed, otherwise a new empty dict. result is then returned. Formally: ( isinstance(providers, dict) isinstance(entry, dict) isinstance(options, dict) isinstance(models, dict) isinstance(existing_model, dict) ) ( ConfigWizardError raised k in existing: existing[k] unchanged ) ; else result is a dict such that: result['$schema'] = SCHEMA_URL if '$schema' not in existing else
```



```
existing['$schema'] (through deepcopy); let P = deepcopy(existing.get('provider', {})); then P' = P with P'[config.provider_id] =
deepcopy(existing.get('provider', {}).get(config.provider_id, {})) updated: (npm = adapter_for_api_style(config.api_style),
options = (deepcopy(original_options) {baseUrl: config.base_url, apiKey: '{file:'+opencode_secret_path+'}'}), models =
(shallow_copy(original_models) {config.model_id: original_model_entry if existed else {}})); result['provider'] = P' and all
other top-level keys equal their deep-copied counterparts from existing.",
"code_evidence": "Line 8: if \"'$schema'\" not in merged:\nLine 9: merged[\"'$schema'\"] = SCHEMA_URL",
"trigger_condition": "The specification requires the returned dict to always include a '$schema' key set to a fixed schema
URL, regardless of any existing value. The code only sets it when not already present, so an existing '$schema' is preserved
instead of being overwritten."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--merge_opencode_config",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/merge_opencode_config.py",
  "function_name": "merge_opencode_config",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--merge_opencode_config.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--merge_opencode_config.md",
  "probe_stdout": "CONFIRMED - actual: 'https://example.com/not-the-right-schema' | expected: 'https://opencode.ai/config.json'",
  "trigger_summary": "When existing dict already contains a $schema key, the function preserves it instead of overwriting with
SCHEMA_URL as the specification requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:59:49.509527Z	4667
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:00:44.253465Z	4753

事件 1 — [llm_775413e8f5184322b749f41f1feb2a1d](#)

► parsed_json（完整）

```
{
  "post_condition": "Post-condition: If any of the following type checks fail: the existing 'provider' field (if present) is not a
dict, or the entry for config.provider_id is not a dict, or that entry's 'options' or 'models' field is not a dict, or the model
entry for config.model_id is not a dict, then a ConfigWizardError is raised with an appropriate message and the existing dict
remains completely unmodified. Otherwise, the function returns a new dict (call it result) and existing is unchanged. result is
formed as follows: it is a deep copy of existing, then: if '$schema' was not present in existing, result['$schema'] is set to
SCHEMA_URL. All other top-level keys remain as in the deep copy. The 'provider' key is a new dict where each existing provider
entry is deep-copied, except for config.provider_id. The value for config.provider_id (call it merged_entry) is constructed by:
taking a deep copy of the original provider entry if it existed, otherwise using an empty dict; then setting merged_entry['npm'] =
adapter_for_api_style(config.api_style); then setting merged_entry['options'] to a new dict that is a deep copy of the original
options (if any, else {}) but with 'baseUrl' overwritten to config.base_url and 'apiKey' overwritten to the string '{{file:
{opencode_secret_path}}}' and setting merged_entry['models'] to a dict that is a shallow copy of the original models (if any,
else {}) with the key config.model_id pointing to the exact same dict object as the original model entry if it existed, otherwise
a new empty dict. result is then returned. Formally: ( ¬instance(providers, dict) v ¬instance(existing_model, dict) ) ⇒ ( ConfigWizardError raised ∧ ∀ k in
existing: existing[k] unchanged ) ; else result is a dict such that: result['$schema'] = SCHEMA_URL if '$schema' not in existing
else existing['$schema'] (through deepcopy); let P = deepcopy(existing.get('provider', {})); then P' = P with
P'[config.provider_id] = deepcopy(existing.get('provider', {}).get(config.provider_id, {})) updated: (npm =
adapter_for_api_style(config.api_style), options = (deepcopy(original_options) ∪ {baseUrl: config.base_url, apiKey:
'{file:'+opencode_secret_path+'}'}), models = (shallow_copy(original_models) ∪ {config.model_id: original_model_entry if existed
else {}})); result['provider'] = P' and all other top-level keys equal their deep-copied counterparts from existing."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_775413e8f5184322b749f41f1feb2a1d_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_775413e8f5184322b749f41f1feb2a1d_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_775413e8f5184322b749f41f1feb2a1d_response.txt)

事件 2 — [llm_6761066465894c2baf495d3d5ede3a9f](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
```

```
"counterexample": "existing = {\n'$schema': \"https://old-schema\", \"provider\": {}}; config =\nLLMConfigInput(provider_id='p1', api_style='openai', base_url='https://api.example.com', model_id='gpt-4');\nopcode_secret_path = Path('/secret'),\n  \"offending_statements\": \"Line 8: if \\\"$schema\\\" not in merged:\\nLine 9: merged[\\\"$schema\\\"] = SCHEMA_URL\",\n  \"reason\": \"The specification requires the returned dict to always include a '$schema' key set to a fixed schema URL, regardless\nof any existing value. The code only sets it when not already present, so an existing '$schema' is preserved instead of being\noverwritten.\"\n}\n}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_6761066465894c2baf495d3d5ede3a9f_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_6761066465894c2baf495d3d5ede3a9f_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_6761066465894c2baf495d3d5ede3a9f_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-023 — [src--configure_llm_py--remove_legacy_llm_env_overrides](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#); attempts 1
- Phase / 模块： Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim：** Returns (cleaned_text, removed_names). cleaned_text is text with every line removed whose variable name (after stripping an optional 'export' prefix) belongs to a known set of legacy LLM override variable names environment variables that previously held LLM configuration fields now stored in fm-agent.toml rather than .env. Every line in text whose variable name does not belong to the legacy set appears in cleaned_text unchanged, preserving original content, whitespace, and relative line order. removed_names is a tuple of the distinct legacy LLM override variable names found and removed from text, in the order of their first occurrence. When no legacy LLM override variable appears in text, removed_names is an empty tuple.
- Actual behavior：** The function returns a tuple (new_text, removed_keys) where:
 - new_text is a string identical to the input text except that every line which, after stripping leading whitespace, is not empty, does not start with '#', and after removing an optional 'export' prefix and whitespace, contains a key (the part before '=' stripped of surrounding whitespace) that belongs to the set ENV_LEGACY_LLM_KEYS is completely omitted (including its line ending). All other lines (empty lines, comment lines, and assignments whose key is not in the set) appear exactly as in the input, preserving original order and line endings.
 - removed_keys is a tuple containing each key from ENV_LEGACY_LLM_KEYS that was removed, in the order of their first occurrence in the input. Each removed key appears exactly once, regardless of how many times it appeared.

Formally, let T be the input string. Define lines = T.splitlines(keepends=True). For each line L, let stripped = L.lstrip(); if stripped == "" or stripped.startswith('#'), line is kept. Otherwise, let remainder = stripped after matching _ENV_EXPORT_PREFIX_RE (removing optional 'export' prefix); let key = remainder.partition('=')[0].strip(); if key ENV_LEGACY_LLM_KEYS, line is removed and key is a candidate for removal. Then:

new_text = ".join(L for L in lines if not (condition above and key ENV_LEGACY_LLM_KEYS)) removed_keys = tuple(unique keys from lines where condition holds, preserving order of first appearance)

The output satisfies (new_text, removed_keys) = remove_legacy_llm_env_overrides(T).

- Code evidence：** Line 12: key, sep, _value = working.partition("=") ; Line 13: env_key = key.strip() if sep else ""
- Trigger condition：** The code only considers lines as having a variable name if they contain an equals sign. A line like 'export LLM_MODEL' (without '=') has the variable name 'LLM_MODEL' after removing the export prefix, but the code sets env_key to empty string, keeping the line and not recording the removed key, which violates the specification.
- Validator trigger：** A line like 'export LLM_MODEL' without '=' has LLM_MODEL as the variable name after removing the export prefix, but env_key becomes empty string because no '=' separator exists, so the line is kept and the key is not recorded as removed.
- Probe stdout：** CONFIRMED — actual: ('export LLM_MODEL\n', ()) | expected: ("", ('LLM_MODEL',))

► SPEC sidecar（完整）

```
{\n  \"signature\": \"remove_legacy_llm_env_overrides(text: str) -> tuple[str, tuple[str, ...]]\",\n  \"pre_condition\": \"text is a string whose content follows .env file convention: each line is either empty, a comment (first non-whitespace character is '#'), or a variable assignment optionally prefixed with 'export' followed by KEY=VALUE where KEY consists of uppercase letters, digits, and underscores.\"",
```

```
"post_condition": "Returns (cleaned_text, removed_names). cleaned_text is text with every line removed whose variable name (after stripping an optional 'export' prefix) belongs to a known set of legacy LLM override variable names – environment variables that previously held LLM configuration fields now stored in fm-agent.toml rather than .env. Every line in text whose variable name does not belong to the legacy set appears in cleaned_text unchanged, preserving original content, whitespace, and relative line order. removed_names is a tuple of the distinct legacy LLM override variable names found and removed from text, in the order of their first occurrence. When no legacy LLM override variable appears in text, removed_names is an empty tuple."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/remove_legacy_llm_env_overrides.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns (cleaned_text, removed_names). cleaned_text is text with every line removed whose variable name (after stripping an optional 'export' prefix) belongs to a known set of legacy LLM override variable names environment variables that previously held LLM configuration fields now stored in fm-agent.toml rather than .env. Every line in text whose variable name does not belong to the legacy set appears in cleaned_text unchanged, preserving original content, whitespace, and relative line order. removed_names is a tuple of the distinct legacy LLM override variable names found and removed from text, in the order of their first occurrence. When no legacy LLM override variable appears in text, removed_names is an empty tuple.",
    "actual_behavior": "The function returns a tuple (new_text, removed_keys) where:\n\n- new_text is a string identical to the input text except that every line which, after stripping leading whitespace, is not empty, does not start with '#', and after removing an optional 'export' prefix and whitespace, contains a key (the part before '=' stripped of surrounding whitespace) that belongs to the set ENV_LEGACY_LLM_KEYS is completely omitted (including its line ending). All other lines (empty lines, comment lines, and assignments whose key is not in the set) appear exactly as in the input, preserving original order and line endings.\n\n- removed_keys is a tuple containing each key from ENV_LEGACY_LLM_KEYS that was removed, in the order of their first occurrence in the input. Each removed key appears exactly once, regardless of how many times it appeared.\n\nFormally, let T be the input string. Define lines = T.splitlines(keepends=True). For each line L, let stripped = L.lstrip(); if stripped == '' or stripped.startswith('#'), line is kept. Otherwise, let remainder = stripped after matching _ENV_EXPORT_PREFIX_RE (removing optional 'export' prefix); let key = remainder.partition('=')[0].strip(); if key ENV_LEGACY_LLM_KEYS, line is removed and key is a candidate for removal. Then:\n\n  new_text = ''.join(L for L in lines if not (condition above and key ENV_LEGACY_LLM_KEYS))\n  removed_keys = tuple(unique keys from lines where condition holds, preserving order of first appearance)\n\nThe output satisfies (new_text, removed_keys) = remove_legacy_llm_env_overrides(T).",
    "code_evidence": "Line 12: key, sep, _value = working.partition('=') ; Line 13: env_key = key.strip() if sep else ''",
    "trigger_condition": "The code only considers lines as having a variable name if they contain an equals sign. A line like 'export LLM_MODEL' (without '=') has the variable name 'LLM_MODEL' after removing the export prefix, but the code sets env_key to empty string, keeping the line and not recording the removed key, which violates the specification."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--remove_legacy_llm_env_overrides",
  "source_file": "src/configure_llm.py",
  "function_name": "remove_legacy_llm_env_overrides",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--remove_legacy_llm_env_overrides.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--remove_legacy_llm_env_overrides.md",
  "probe_stdout": "CONFIRMED – actual: ('export LLM_MODEL\n', ()) | expected: ('', ('LLM_MODEL',))",
  "trigger_summary": "A line like 'export LLM_MODEL' without '=' has LLM_MODEL as the variable name after removing the export prefix, but env_key becomes empty string because no '=' separator exists, so the line is kept and the key is not recorded as removed."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:09:21.917822Z	2537
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:10:00.383559Z	4849

► `parsed_json` (完整)

```
{
  "post_condition": "The function returns a tuple (new_text, removed_keys) where:\n\n- new_text is a string identical to the input text except that every line which, after stripping leading whitespace, is not empty, does not start with '#', and after removing an optional 'export' prefix and whitespace, contains a key (the part before '=' stripped of surrounding whitespace) that belongs to the set ENV_LEGACY_LLM_KEYS is completely omitted (including its line ending). All other lines (empty lines, comment lines, and assignments whose key is not in the set) appear exactly as in the input, preserving original order and line endings.\n\n- removed_keys is a tuple containing each key from ENV_LEGACY_LLM_KEYS that was removed, in the order of their first occurrence in the input. Each removed key appears exactly once, regardless of how many times it appeared.\n\nFormally, let T be the input string. Define lines = T.splitlines(keepends=True). For each line L, let stripped = L.lstrip(); if stripped == '' or stripped.startswith('#'), line is kept. Otherwise, let remainder = stripped after matching _ENV_EXPORT_PREFIX_RE (removing optional 'export' prefix); let key = remainder.partition('=')[0].strip(); if key ∈ ENV_LEGACY_LLM_KEYS, line is removed and key is a candidate for removal. Then:\n\n  new_text = ''.join(L for L in lines if not (condition above and key ∈ ENV_LEGACY_LLM_KEYS))\n  removed_keys = tuple(unique keys from lines where condition holds, preserving order of first appearance)\n\nThe output satisfies (new_text, removed_keys) = remove_legacy_llm_env_overrides(T)."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_44cd64dbdfdb407ea5fd245e25eff91b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_44cd64dbdfdb407ea5fd245e25eff91b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_44cd64dbdfdb407ea5fd245e25eff91b_response.txt)

事件 2 — `llm_c5db39387cba471a8d4c56b457ea857f`

► `parsed_json` (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "export LLM_MODEL",
  "offending_statements": "Line 12: key, sep, _value = working.partition('\\n=\\n') ; Line 13: env_key = key.strip() if sep else '\\n\\n'",
  "reason": "The code only considers lines as having a variable name if they contain an equals sign. A line like 'export LLM_MODEL' (without '=') has the variable name 'LLM_MODEL' after removing the export prefix, but the code sets env_key to empty string, keeping the line and not recording the removed key, which violates the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c5db39387cba471a8d4c56b457ea857f_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c5db39387cba471a8d4c56b457ea857f_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c5db39387cba471a8d4c56b457ea857f_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-024 — `src--configure_llm-py--run_llm_settings_update`

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： `confirmed`； attempts 1
- Phase / 模块： Phase 1 — Initialization & Configuration / `configuration`； 原源码 `src/configuration_llm.py`
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim**: Returns 0 when every key in updates has been written to the corresponding TOML field with the provided value and the pre-update TOML content has been backed up to a file adjacent to the original. Before the write, the planned field modifications are printed to stdout, and warnings are emitted for any .env entries or process environment variables whose precedence would override the TOML values being changed. When assume_yes is false and the user declines the confirmation prompt, no file modifications occur and the return value is non-zero. When the TOML file cannot be written or any key path in updates does not resolve to a writable field in the LLM section, the return value is non-zero and no partial updates are persisted.
- Actual behavior**: After execution of `run_llm_settings_update`, the program state satisfies one of the following exhaustive cases. (1) Early termination: `assume_yes` is False, the user responds negatively to the confirmation prompt; the function prints the preview string from `_preview_llm_settings_update(updates, toml_path)`, any warnings from `warn_dotenv_overrides_for_updates` and `warn_live_llm_environment_overrides`, an extra blank line if either warning function flagged active overrides, then prints 'Aborted.' and returns 1. No changes are made to the TOML file. (2) Successful update: either `assume_yes` is True or the user responds affirmatively to the prompt; the same preview and warnings are printed, followed by 'Updated {toml_path}'. `apply_llm_settings_update(updates, toml_path)` is invoked, which writes the new LLM settings into the TOML file while preserving all other sections and fields, and may create a timestamped backup file adjacent to `toml_path`; if a backup was created, the line 'Backed up {toml_path} -> {backup_path}' is printed. The

- function returns 0. (3) Unhandled exception: if any step after printing the preview and warnings raises an exception (e.g., `apply_llm_settings_update` fails due to a non-writable file or TOML serialization error), the exception propagates without printing 'Updated ...' or 'Backed up ...'; the state of the TOML file may be unchanged or partially modified, and no return value is produced. Formally, given the pre-condition P, the post-condition is: (return = 1 assume_yes _prompt_yes_no_result stdout_includes(preview_output, warnings_output, "\n", "\n"[if any overrides exist], "Aborted.") toml_path.unchanged) (return = 0 (assume_yes _prompt_yes_no_result stdout_includes(preview_output, warnings_output, "\n", "\n"[if any overrides exist], "Updated " + toml_path) (backup_created stdout_includes("Backed up " + toml_path + " -> " + backup_path)) toml_path.llm_section = updates other_sections_preserved) (exception_raised stdout_includes(preview_output, warnings_output) (toml_path.unchanged inconsistent) "Updated " + toml_path stdout).
- **Code evidence:** Line 18: `backup = apply_llm_settings_update(updates, toml_path)` no validation of key paths or error handling; the function returns 0 on success (Line 22) instead of returning non-zero for invalid keys as required.
 - **Trigger condition:** Specification B states: 'When ... any key path in updates does not resolve to a writable field in the LLM section, the return value is non-zero and no partial updates are persisted.' The code unconditionally passes updates to `apply_llm_settings_update` without checking validity, and it returns 0 when that call succeeds (or raises an unhandled exception when it fails). Thus, for an invalid key the code either returns 0 (violating the nonzero requirement) or allows an exception to propagate (failing to return any integer).
 - **Validator trigger:** Passing a key not in `_LLM_TOML_KEYS` (e.g., 'invalid_key') to `run_llm_settings_update` causes an unhandled `KeyError` in `_preview_llm_settings_update`, instead of returning a non-zero integer as required by the specification.
 - **Probe stdout:** CONFIRMED — actual: Exception: `KeyError: 'invalid_key'` | expected: non-zero integer ($\neq 0$) — function should not raise

► SPEC sidecar (完整)

```
{
  "signature": "run_llm_settings_update(project_root: Path, updates: dict[str, str], *, assume_yes: bool = False) -> int",
  "pre_condition": "project_root is a Path identifying a directory that contains a readable fm-agent.toml file whose LLM section is parseable. updates is a non-empty dict whose keys are dot-separated field paths within the LLM section of fm-agent.toml, and whose values are non-empty strings or booleans representing new configuration values. assume_yes indicates whether confirmation prompts presented to the user are suppressed.",
  "post_condition": "Returns 0 when every key in updates has been written to the corresponding TOML field with the provided value and the pre-update TOML content has been backed up to a file adjacent to the original. Before the write, the planned field modifications are printed to stdout, and warnings are emitted for any .env entries or process environment variables whose precedence would override the TOML values being changed. When assume_yes is false and the user declines the confirmation prompt, no file modifications occur and the return value is non-zero. When the TOML file cannot be written or any key path in updates does not resolve to a writable field in the LLM section, the return value is non-zero and no partial updates are persisted."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "default_paths",
      "signature": "default_paths(project_root: Path) -> WizardPaths",
      "pre_condition": "project_root is a Path identifying an existing directory.",
      "post_condition": "Returns a WizardPaths object whose toml_path, env_path, and opencode_config_path attributes refer to existing or creatable filesystem paths derived from project_root, following the platform conventions for fm-agent.toml, .env, and the OpenCode configuration file respectively. Each path is determined as: toml_path from the FM_AGENT_CONFIG environment variable when set, otherwise from <project_root>/fm-agent.toml; env_path as <project_root>/env; opencode_config_path from OPENCODE_CONFIG when set, otherwise from the platform-specific OpenCode config directory."
    },
    {
      "name": "_preview_llm_settings_update",
      "signature": "_preview_llm_settings_update(updates: dict[str, str], toml_path: Path) -> str",
      "pre_condition": "updates is a dict mapping TOML field paths to new values; toml_path is a Path identifying an existing TOML file.",
      "post_condition": "Returns a human-readable string describing each key in updates, the current value of that field in the TOML file, and the new value that will be written if applied."
    },
    {
      "name": "warn_dotenv_overrides_for_updates",
      "signature": "warn_dotenv_overrides_for_updates(env_path: Path, updates: dict[str, str]) -> bool",
      "pre_condition": "env_path is a Path to a .env file that may or may not exist; updates is a dict mapping LLM section field paths to new values.",
      "post_condition": "For each key in updates, prints a warning to stdout if the corresponding process environment variable name maps to the same TOML field and a value for that variable is present in the .env file at env_path. Returns True when at least one override warning was printed; returns False when no overrides were found or the .env file is absent or unreadable."
    },
    {
      "name": "warn_live_llm_environment_overrides",
      "signature": "warn_live_llm_environment_overrides() -> None",
      "pre_condition": "None required.",
      "post_condition": "Prints a warning to stdout when any LLM-related environment variable (from the set of variables listed in _ENV_MAP whose target section is 'llm') is currently set in the process environment, indicating that the live environment value will take precedence over any value written to fm-agent.toml. If no such variables are set, prints nothing."
    }
  ]
}
```



```

"name": "live_llm_environment_overrides",
"signature": "live_llm_environment_overrides() -> bool",
"pre_condition": "None required.",
"post_condition": "Returns True when at least one LLM-related environment variable (from the set of variables listed in
_ENV_MAP whose target section is 'llm') is currently set in the process environment; returns False when no such variables are set.
The result is determined solely by the current process environment; it does not examine any file."
},
{
"name": "_prompt_yes_no",
"signature": "_prompt_yes_no(message: str, *, default: bool = True) -> bool",
"pre_condition": "message is a non-empty string; default is True or False.",
"post_condition": "Prints message to stdout followed by a [Y/n] or [y/N] prompt reflecting the default value. Reads one line
from stdin. Returns True when the user's response matches a yes-affirming pattern (case-insensitive 'y' or 'yes') or when the user
provides an empty response and default is True. Returns False when the user's response matches a no-affirming pattern (case-
insensitive 'n' or 'no') or when the user provides an empty response and default is False."
},
{
"name": "apply_llm_settings_update",
"signature": "apply_llm_settings_update(updates: dict[str, str], toml_path: Path) -> Path | None",
"pre_condition": "updates is a dict mapping valid LLM-section TOML field paths to new string or boolean values; toml_path
identifies an existing, readable, and writable TOML file.",
"post_condition": "Writes the provided values to the corresponding fields in the TOML file at toml_path, preserving all
other sections and fields unchanged. Before writing, copies the original file content to a backup file named with a timestamp
suffix adjacent to toml_path. Returns the backup file path if the backup was successfully created; returns None when the file was
updated without creating a backup. The file's TOML structure and formatting outside the modified fields are preserved to the
extent the underlying serializer permits."
}
]
}

```

► Logic verification result（完整）

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/run_llm_settings_update.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns 0 when every key in updates has been written to the corresponding TOML field with the provided value
and the pre-update TOML content has been backed up to a file adjacent to the original. Before the write, the planned field
modifications are printed to stdout, and warnings are emitted for any .env entries or process environment variables whose
precedence would override the TOML values being changed. When assume_yes is false and the user declines the confirmation prompt,
no file modifications occur and the return value is non-zero. When the TOML file cannot be written or any key path in updates does
not resolve to a writable field in the LLM section, the return value is non-zero and no partial updates are persisted.",
    "actual_behavior": "After execution of run_llm_settings_update, the program state satisfies one of the following exhaustive
cases. (1) Early termination: assume_yes is False, the user responds negatively to the confirmation prompt; the function prints
the preview string from _preview_llm_settings_update(updates, toml_path), any warnings from warn_dotenv_overrides_for_updates and
warn_live_llm_environment_overrides, an extra blank line if either warning function flagged active overrides, then prints
'Aborted.' and returns 1. No changes are made to the TOML file. (2) Successful update: either assume_yes is True or the user
responds affirmatively to the prompt; the same preview and warnings are printed, followed by 'Updated {toml_path}'.
apply_llm_settings_update(updates, toml_path) is invoked, which writes the new LLM settings into the TOML file while preserving
all other sections and fields, and may create a timestamped backup file adjacent to toml_path; if a backup was created, the line
'Backed up {toml_path} -> {backup_path}' is printed. The function returns 0. (3) Unhandled exception: if any step after printing
the preview and warnings raises an exception (e.g., apply_llm_settings_update fails due to a non-writable file or TOML
serialization error), the exception propagates without printing 'Updated ...' or 'Backed up ...'; the state of the TOML file may
be unchanged or partially modified, and no return value is produced. Formally, given the pre-condition P, the post-condition is:
(return = 1 assume_yes _prompt_yes_no_result stdout_includes(preview_output, warnings_output, "\\n\\n\\n", "\\n\\n\\n"[if any overrides
exist], "\\Aborted.\\n") toml_path.unchanged) (return = 0 (assume_yes _prompt_yes_no_result) stdout_includes(preview_output,
warnings_output, "\\n\\n\\n", "\\n\\n\\n"[if any overrides exist], "\\Updated \\n" + toml_path) (backup_created stdout_includes("\\Backed up
\\n" + toml_path + \\n" -> \\n" + backup_path)) toml_path.llm_section = updates other_sections_preserved) (exception_raised
stdout_includes(preview_output, warnings_output) (toml_path.unchanged inconsistent) "\\Updated \\n" + toml_path stdout).",
    "code_evidence": "Line 18: backup = apply_llm_settings_update(updates, toml_path) no validation of key paths or error
handling; the function returns 0 on success (Line 22) instead of returning non-zero for invalid keys as required.",
    "trigger_condition": "Specification B states: 'When ... any key path in updates does not resolve to a writable field in the
LLM section, the return value is non-zero and no partial updates are persisted.' The code unconditionally passes updates to
apply_llm_settings_update without checking validity, and it returns 0 when that call succeeds (or raises an unhandled exception
when it fails). Thus, for an invalid key the code either returns 0 (violating the nonzero requirement) or allows an exception to
propagate (failing to return any integer)."

```

► Bug Validator result（完整）

```

{
  "id": "src--configure_llm-py--run_llm_settings_update",
  "source_file": "src/configure_llm.py",
  "function_name": "run_llm_settings_update",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--run_llm_settings_update.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--run_llm_settings_update.md",
  "probe_stdout": "CONFIRMED - actual: Exception: KeyError: 'invalid_key' | expected: non-zero integer (≠ 0) - function should not
raise\\n",
  "trigger_summary": "Passing a key not in _LLM_TOML_KEYS (e.g., 'invalid_key') to run_llm_settings_update causes an unhandled

```



```
KeyError in _preview_llm_settings_update, instead of returning a non-zero integer as required by the specification."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:45:04.753574Z	4238
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:45:49.446069Z	7602

事件 1 — [llm_d18da892a58f4b13ab327f4311afa4f2](#)

► parsed_json（完整）

```
{
  "post_condition": "After execution of run_llm_settings_update, the program state satisfies one of the following exhaustive cases. (1) Early termination: assume_yes is False, the user responds negatively to the confirmation prompt; the function prints the preview string from _preview_llm_settings_update(updates, toml_path), any warnings from warn_dotenv_overrides_for_updates and warn_live_llm_environment_overrides, an extra blank line if either warning function flagged active overrides, then prints 'Aborted.' and returns 1. No changes are made to the TOML file. (2) Successful update: either assume_yes is True or the user responds affirmatively to the prompt; the same preview and warnings are printed, followed by 'Updated {toml_path}'. apply_llm_settings_update(updates, toml_path) is invoked, which writes the new LLM settings into the TOML file while preserving all other sections and fields, and may create a timestamped backup file adjacent to toml_path; if a backup was created, the line 'Backed up {toml_path} -> {backup_path}' is printed. The function returns 0. (3) Unhandled exception: if any step after printing the preview and warnings raises an exception (e.g., apply_llm_settings_update fails due to a non-writable file or TOML serialization error), the exception propagates without printing 'Updated ...' or 'Backed up ...'; the state of the TOML file may be unchanged or partially modified, and no return value is produced. Formally, given the pre-condition P, the post-condition is: (return = 1 ^ ~assume_yes ^ ~_prompt_yes_no_result ^ stdout_includes(preview_output, warnings_output, '\\n\\n', '\\n\\n'[if any overrides exist], '\\Aborted.\\n') ^ toml_path.unchanged) v (return = 0 ^ (assume_yes v _prompt_yes_no_result) ^ stdout_includes(preview_output, warnings_output, '\\n\\n', '\\n\\n'[if any overrides exist], '\\Updated \\n' + toml_path) ^ (backup_created => stdout_includes('\\Backed up \\n' + toml_path + '\\n -> \\n' + backup_path)) ^ toml_path.llm_section = updates ^ other_sections_preserved) v (exception_raised ^ stdout_includes(preview_output, warnings_output) ^ (toml_path.unchanged v inconsistent) ^ '\\Updated \\n' + toml_path <math>\\notin\\math> stdout)."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d18da892a58f4b13ab327f4311afa4f2_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d18da892a58f4b13ab327f4311afa4f2_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d18da892a58f4b13ab327f4311afa4f2_response.txt)

事件 2 — [llm_7ee1942dcaa94175a47833a249fff073](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "project_root = Path('/existing/project'), assume_yes=True, updates = {'nonexistent_field': 'value'} where 'nonexistent_field' does not resolve to a writable field in the LLM section of the existing TOML file. The function will proceed to apply_llm_settings_update, which either raises an exception (not returning a value) or silently writes an invalid key and returns 0, neither of which produces the required non-zero return code.",
  "offending_statements": "Line 18: backup = apply_llm_settings_update(updates, toml_path) - no validation of key paths or error handling; the function returns 0 on success (Line 22) instead of returning non-zero for invalid keys as required.",
  "reason": "Specification B states: 'When ... any key path in updates does not resolve to a writable field in the LLM section, the return value is non-zero and no partial updates are persisted.' The code unconditionally passes updates to apply_llm_settings_update without checking validity, and it returns 0 when that call succeeds (or raises an unhandled exception when it fails). Thus, for an invalid key the code either returns 0 (violating the non-zero requirement) or allows an exception to propagate (failing to return any integer)."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7ee1942dcaa94175a47833a249fff073_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_7ee1942dcaa94175a47833a249fff073_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_7ee1942dcaa94175a47833a249fff073_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-025 — [src--configure_llm-py--run_local_backend_configuration](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： **confirmed**; attempts 1

- **Phase / 模块:** Phase 1 — Initialization & Configuration / `configuration`; 原源码 `src/configure_llm.py`
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns 0 when the `fm-agent.toml` file has been updated to set the LLM backend to the provided backend value, with a backup of the original file created. Returns non-zero when the `fm-agent.toml` file has not been modified. No API key, `.env` modification, or OpenCode provider configuration is performed.
- **Actual behavior:** Natural language: After the execution of `run_local_backend_configuration`, exactly one of the following outcomes occurs, depending on runtime events:
 1. Normal return with 0: The user confirmed the prompt. The standard output contains the preview of the local backend configuration (generated from `_preview_local_backend_configuration`), any warnings about live LLM environment overrides, the prompt `Continue?` followed by the user's affirmative response, and then the messages: `Updated {paths.toml_path}`; if legacy LLM overrides were present, `Removed legacy LLM overrides from {paths.env_path}`; and for every pair `(path, backup)` returned by `apply_local_backend_configuration` where `backup` is not `None`, `Backed up {path} -> {backup}`. The file system has been modified: `paths.toml_path` now contains the local backend configuration for `backend` (as produced by `local_backend_toml_updates`); `paths.env_path` (if it existed and contained legacy LLM overrides) no longer contains those overrides; and for each changed file, a backup file exists at the reported backup path with the original contents. The function returns 0.
 2. Normal return with 1: The user declined the prompt. The standard output contains the same preview, warnings, and the prompt, followed by `Aborted.`. No files were modified on disk. The function returns 1.
 3. Exceptional termination: An exception (e.g., `IOError`, `OSError`, or any exception from the called functions) is raised. The standard output includes all output generated before the exception (which may include the preview, warnings, and possibly partial apply messages). The file system may be in an inconsistent state: some backups may have been created and some file updates written, but not all intended changes are guaranteed. The function does not return to the caller; the exception propagates up the stack.

Formal logic: Let `FS0` be the initial file system state and `STDOUT0` the empty output stream. Let `paths = default_paths(project_root)`, `env_content = _read_text_if_exists(paths.env_path)`, `(updated_env, removed_overrides0) = remove_legacy_llm_env_overrides(env_content)`, `toml_updates = local_backend_toml_updates(paths.env_path, backend)`, `preview_str = _preview_local_backend_configuration(backend, paths, removed_overrides0, toml_updates)`. Let `S_warn` be the string printed by `warn_live_llm_environment_overrides()` (may be empty if no live overrides). Define `live = live_llm_environment_overrides()`.

Post-condition predicate `Q` over the state after the function call:

`Q ((user_confirmed = true) stdout = STDOUT0 + preview_str + "\n" + "\n" + S_warn + ("\n" if live else "") + "Continue?" + user_input_yes + ("Updated " + paths.toml_path + "\n") + (if removed_overrides0 then "Removed legacy LLM overrides from " + paths.env_path + "\n" else "") + (for each (p,b) backups where b None: "Backed up " + p + " -> " + b + "\n") backups, removed_overrides1 : (backups, removed_overrides1) = apply_local_backend_configuration(backend, paths) FS = FS0 after applying all changes: paths.toml_path content = toml_updates applied to original content paths.env_path content = updated_env (i.e., original without legacy overrides) (p, b) backups with b None: backup file b exists with content of p before modification and no other regular files in the project directory are changed. return_value = 0) ((user_confirmed = false) stdout = STDOUT0 + preview_str + "\n" + "\n" + S_warn + ("\n" if live else "") + "Continue?" + user_input_no + "Aborted.\n" FS = FS0 (no file modifications) return_value = 1) (exception E : E was raised during the call stdout contains a prefix of one of the above outputs, up to the point of exception FS may be partially updated (some backups and file changes from apply_local_backend_configuration may have occurred) return_value is undefined (exception propagates))`

Here `'user_input_yes'` and `'user_input_no'` denote the user's typed response, and `'user_confirmed'` is true iff `_prompt_yes_no("Continue?", default=True)` returns True.

- **Code evidence:** Line 22: `backups, removed_overrides = apply_local_backend_configuration(backend, paths)`
- **Trigger condition:** The specification explicitly requires that no `.env` modification is performed. The code, however, modifies the `.env` file by removing legacy LLM overrides when they are present, as performed by `apply_local_backend_configuration` called on line 22.
- **Validator trigger:** When `run_local_backend_configuration` is called with a local CLI backend and the project `.env` contains legacy LLM overrides, the function modifies `.env` by removing those overrides, contradicting the specification that no `.env` modification is performed.
- **Probe stdout:** FM-Agent local CLI backend configuration

Backend: `codex-cli`

The following files will be updated:

- /tmp/bug_probe_ubnz7ezp/fm-agent.toml
- /tmp/bug_probe_ubnz7ezp/.env

The following legacy dotenv overrides will be removed so they do not shadow the selected backend: LLM_MODEL, LLM_EFFORT, LLM_API_BASE_URL

The local CLI model settings retained from .env will be written to TOML:

- name: 'old-model'
- effort: 'high'

No API key or OpenCode provider configuration will be changed.

Warning: these shell environment variables override the saved LLM settings: LLM_API_KEY, LLM_API_BASE_URL, LLM_MODEL, FM_AGENT_MODEL_BACKEND, LLM_EFFORT, OPENCODE_MODEL_PROVIDER The wizard cannot change the shell that launched it. Before starting FM-Agent in this shell, unset them to use the saved configuration: unset LLM_API_KEY LLM_API_BASE_URL LLM_MODEL FM_AGENT_MODEL_BACKEND LLM_EFFORT OPENCODE_MODEL_PROVIDER

Updated /tmp/bug_probe_ubnz7ezp/fm-agent.toml Removed legacy LLM overrides from /tmp/bug_probe_ubnz7ezp/.env Backed up /tmp/bug_probe_ubnz7ezp/fm-agent.toml -> /tmp/bug_probe_ubnz7ezp/fm-agent.toml.bak.20260727-235535 Backed up /tmp/bug_probe_ubnz7ezp/.env -> /tmp/fm-agent-config-backups-uid-1000/tmp_bug_probe_ubnz7ezp__env.bak.20260727-235535
CONFIRMED — .env was modified (legacy LLM overrides removed) result=0 after='# keep this comment\n'

► SPEC sidecar (完整)

```
{
  "signature": "run_local_backend_configuration(project_root: Path, backend: str) -> int",
  "pre_condition": "project_root is a Path identifying a directory containing a readable fm-agent.toml file; backend is a non-empty string identifying a local CLI backend and is not 'opencode'.",
  "post_condition": "Returns 0 when the fm-agent.toml file has been updated to set the LLM backend to the provided backend value, with a backup of the original file created. Returns non-zero when the fm-agent.toml file has not been modified. No API key, .env modification, or OpenCode provider configuration is performed."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/run_local_backend_configuration.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns 0 when the fm-agent.toml file has been updated to set the LLM backend to the provided backend value, with a backup of the original file created. Returns non-zero when the fm-agent.toml file has not been modified. No API key, .env modification, or OpenCode provider configuration is performed.",
    "actual_behavior": "Natural language: After the execution of `run_local_backend_configuration`, exactly one of the following outcomes occurs, depending on runtime events:\n\n1. Normal return with 0: The user confirmed the prompt. The standard output contains the preview of the local backend configuration (generated from `preview_local_backend_configuration`), any warnings about live LLM environment overrides, the prompt `Continue?` followed by the user's affirmative response, and then the messages: `Updated {paths.toml_path}`; if legacy LLM overrides were present, `Removed legacy LLM overrides from {paths.env_path}`; and for every pair `(path, backup)` returned by `apply_local_backend_configuration` where `backup` is not `None`, `Backed up {path} -> {backup}`. The file system has been modified: `paths.toml_path` now contains the local backend configuration for `backend` (as produced by `local_backend_toml_updates`); `paths.env_path` (if it existed and contained legacy LLM overrides) no longer contains those overrides; and for each changed file, a backup file exists at the reported backup path with the original contents. The function returns 0.\n\n2. Normal return with 1: The user declined the prompt. The standard output contains the same preview, warnings, and the prompt, followed by `Aborted.`. No files were modified on disk. The function returns 1.\n\n3. Exceptional termination: An exception (e.g., IOError, OSError, or any exception from the called functions) is raised. The standard output includes all output generated before the exception (which may include the preview, warnings, and possibly partial apply messages). The file system may be in an inconsistent state: some backups may have been created and some file updates written, but not all intended changes are guaranteed. The function does not return to the caller; the exception propagates up the stack.\n\nFormal logic: Let FS0 be the initial file system state and STDOUT0 the empty output stream. Let `paths = default_paths(project_root)`, `env_content = _read_text_if_exists(paths.env_path)`, `(updated_env, removed_overrides0) = remove_legacy_llm_env_overrides(env_content)`, `toml_updates = local_backend_toml_updates(paths.env_path, backend)`, `preview_str = _preview_local_backend_configuration(backend, paths, removed_overrides0, toml_updates)`. Let `S_warn` be the string printed by `warn_live_llm_environment_overrides()` (may be empty if no live overrides). Define `live = live_llm_environment_overrides()`. \n\nPost-condition predicate Q over the state after the function call: \n\nQ \n ( (user_confirmed = true) \n      stdout = STDOUT0 + preview_str + "\\n" + "\\n" + S_warn + ("\\n" if live else "\\n") +
```

```
\nContinue?\n" + user_input_yes (\nUpdated \n" + paths.toml_path + \n\n\n") + (if removed_overrides0 then \nRemoved legacy LLM\noverrides from \n" + paths.env_path + \n\n\n" else \n\n") + (for each(p,b) backups where b None: \nBacked up \n" + p + \n -> \n" +\nb + \n\n\n)\n\n backups, removed_overrides1 : (backups, removed_overrides1) = apply_local_backend_configuration(backend,\npaths) FS = FS0 after applying all changes:\n\n paths.toml_path content = toml_updates applied to original content\npaths.env_path content = updated_env (i.e., original without legacy overrides)\n\n (p, b) backups with b None: backup\nfile b exists with content of p before modification\n\n and no other regular files in the project directory are changed.\n\nreturn_value = 0\n )\n ( (user_confirmed = false)\n\n stdout = STDOUT0 + preview_str + \n\n\n" + \n\n\n" + S_warn +\n(\n\n\n" if live else \n\n") + \nContinue?\n" + user_input_no + \nAborted.\n\n\n" FS = FS0 (no file modifications)\n\nreturn_value = 1\n )\n ( exception E : E was raised during the call\n\n stdout contains a prefix of one of the above\noutputs, up to the point of exception\n\n FS may be partially updated (some backups and file changes from\n`apply_local_backend_configuration` may have occurred)\n\n return_value is undefined (exception propagates)\n )\n\nHere\n'user_input_yes' and 'user_input_no' denote the user's typed response, and 'user_confirmed' is true iff\n`_prompt_yes_no(\nContinue?\n", default=True)` returns True.",\n\n"code_evidence": "Line 22: backups, removed_overrides = apply_local_backend_configuration(backend, paths)",\n"trigger_condition": "The specification explicitly requires that no .env modification is performed. The code, however,\nmodifies the .env file by removing legacy LLM overrides when they are present, as performed by apply_local_backend_configuration\ncalled on line 22." }\n}
```

► Bug Validator result (完整)

```
{\n  "id": "src--configure_llm-py--run_local_backend_configuration",\n  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/configure_llm.py",\n  "function_name": "run_local_backend_configuration",\n  "confirmation_status": "confirmed",\n  "attempts": 1,\n  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--run_local_backend_configuration.py",\n  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--run_local_backend_configuration.md",\n  "probe_stdout": "FM-Agent local CLI backend configuration\n\nBackend: codex-cli\n\nThe following files will be updated:\n -\n/tmp/bug_probe_ubnz7ezp/fm-agent.toml\n - /tmp/bug_probe_ubnz7ezp/.env\n\nThe following legacy dotenv overrides will be removed\nso they do not\nshadow the selected backend: LLM_MODEL, LLM Effort, LLM_API_BASE_URL\n\nThe local CLI model settings retained from\n.env will be written to TOML:\n - name: 'old-model'\n - effort: 'high'\n\nNo API key or OpenCode provider configuration will be\nchanged.\n\nWarning: these shell environment variables override the saved LLM settings:\n LLM_API_KEY, LLM_API_BASE_URL,\nLLM_MODEL, FM_AGENT_MODEL_BACKEND, LLM Effort, OPENCODE_MODEL_PROVIDER\n\nThe wizard cannot change the shell that launched it.\nBefore starting FM-Agent\n\nin this shell, unset them to use the saved configuration:\n unset LLM_API_KEY LLM_API_BASE_URL\nLLM_MODEL FM_AGENT_MODEL_BACKEND LLM Effort OPENCODE_MODEL_PROVIDER\n\nUpdated /tmp/bug_probe_ubnz7ezp/fm-agent.toml\nRemoved\nlegacy LLM overrides from /tmp/bug_probe_ubnz7ezp/.env\n\nBacked up /tmp/bug_probe_ubnz7ezp/fm-agent.toml ->\n/tmp/bug_probe_ubnz7ezp/fm-agent.toml.bak.20260727-235535\n\nBacked up /tmp/bug_probe_ubnz7ezp/.env -> /tmp/fm-agent-config-backups-\nuid-1000/tmp_bug_probe_ubnz7ezp__env.bak.20260727-235535\n\nCONFIRMED - .env was modified (legacy LLM overrides removed) result=0\nafter='# keep this comment\n\n\n',\n\n"trigger_summary": "When run_local_backend_configuration is called with a local CLI backend and the project .env contains legacy\nLLM overrides, the function modifies .env by removing those overrides, contradicting the specification that no .env modification\nis performed." }\n}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:52:21.125478Z	3645
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:53:09.501108Z	3581

事件 1 — [llm_f2ae152d04de4e2d86cd587269f1e326](#)

► parsed_json (完整)

```
{\n  "post_condition": "Natural language: After the execution of `run_local_backend_configuration`, exactly one of the following\noutcomes occurs, depending on runtime events:\n\n1. Normal return with 0: The user confirmed the prompt. The standard output\ncontains the preview of the local backend configuration (generated from `_preview_local_backend_configuration`), any warnings\nabout live LLM environment overrides, the prompt `Continue?` followed by the user's affirmative response, and then the messages:\n`Updated {paths.toml_path}`; if legacy LLM overrides were present, `Removed legacy LLM overrides from {paths.env_path}`; and for\nevery pair `(path, backup)` returned by `apply_local_backend_configuration` where `backup` is not None, `Backed up {path} ->\n{backup}`. The file system has been modified: `paths.toml_path` now contains the local backend configuration for `backend` (as\nproduced by `local_backend_toml_updates`); `paths.env_path` (if it existed and contained legacy LLM overrides) no longer contains\nthose overrides; and for each changed file, a backup file exists at the reported backup path with the original contents. The\nfunction returns 0.\n\n2. Normal return with 1: The user declined the prompt. The standard output contains the same preview,\nwarnings, and the prompt, followed by `Aborted.`. No files were modified on disk. The function returns 1.\n\n3. Exceptional\ntermination: An exception (e.g., IOError, OSError, or any exception from the called functions) is raised. The standard output\nincludes all output generated before the exception (which may include the preview, warnings, and possibly partial apply messages).\nThe file system may be in an inconsistent state: some backups may have been created and some file updates written, but not all\nintended changes are guaranteed. The function does not return to the caller; the exception propagates up the stack.\n\nFormal\nlogic: Let FS0 be the initial file system state and STDOUT0 the empty output stream. Let `paths` = default_paths(project_root),\n`env_content` = _read_text_if_exists(paths.env_path), `(updated_env, removed_overrides0) =`
```

```
remove_legacy_llm_env_overrides(env_content)\`, `toml_updates` = local_backend_toml_updates(paths.env_path, backend)\`, `preview_str` = _preview_local_backend_configuration(backend, paths, removed_overrides0, toml_updates)\`. Let `S_warn` be the string printed by `warn_live_llm_environment_overrides()` (may be empty if no live overrides). Define `live` = live_llm_environment_overrides().\n\nPost-condition predicate Q over the state after the function call:\n\nQ ≡\n ( (user_confirmed = true) \n      ∧ stdout = STDOUT0 + preview_str + "\\n" + "\\n" + S_warn + ("\\n" if live else "\\n") +\n      "\\Continue?" + user_input_yes + ("Updated " + paths.toml_path + "\\n") + (if removed_overrides0 ≠ ∅ then "Removed legacy LLM\n      overrides from " + paths.env_path + "\\n" else "\\n") + (for each (p,b) ∈ backups where b ≠ None: "Backed up " + p + "\n -> "\n      + b + "\\n")\n      ∧ ∃ backups, removed_overrides1 : (backups, removed_overrides1) = apply_local_backend_configuration(backend,\n      paths) ∧ FS = FS0 after applying all changes:\n      paths.toml_path content = toml_updates applied to original content\n      paths.env_path content = updated_env (i.e., original without legacy overrides)\n      ∧ ∀ (p, b) ∈ backups with b ≠ None: backup\n      file b exists with content of p before modification\n      and no other regular files in the project directory are changed.\n      ∧ return_value = 0\n      )\n      ∨ ( (user_confirmed = false)\n      ∧ stdout = STDOUT0 + preview_str + "\\n" + "\\n" + S_warn +\n      ("\\n" if live else "\\n") + "\\Continue?" + user_input_no + "Aborted.\\n")\n      ∧ FS = FS0 (no file modifications)\n      ∧ return_value = 1\n      )\n      ∨ ( ∃ exception E : E was raised during the call\n      ∧ stdout contains a prefix of one of the above\n      outputs, up to the point of exception\n      ∧ FS may be partially updated (some backups and file changes from\n      `apply_local_backend_configuration` may have occurred)\n      ∧ return_value is undefined (exception propagates)\n      )\n      )\n      Here\n      'user_input_yes' and 'user_input_no' denote the user's typed response, and 'user_confirmed' is true iff\n      `prompt_yes_no("\\Continue?", default=True)` returns True."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f2ae152d04de4e2d86cd587269f1e326_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_f2ae152d04de4e2d86cd587269f1e326_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_f2ae152d04de4e2d86cd587269f1e326_response.txt)

事件 2 — llm_b55e85ebf7bb4223ba7e0df0f3518e23

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "project_root = Path('/tmp/test_proj'), backend = 'openai'. The .env file at /tmp/test_proj/.env contains 'OPENAI_API_KEY=sk-abc123'. The user runs run_local_backend_configuration and answers 'yes' to the prompt. The code calls apply_local_backend_configuration, which removes the OPENAI_API_KEY line from .env (a legacy LLM override). This modifies the .env file, violating the specification requirement that no .env modification is performed.",
  "offending_statements": "Line 22: backups, removed_overrides = apply_local_backend_configuration(backend, paths)",
  "reason": "The specification explicitly requires that no .env modification is performed. The code, however, modifies the .env file by removing legacy LLM overrides when they are present, as performed by apply_local_backend_configuration called on line 22."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b55e85ebf7bb4223ba7e0df0f3518e23_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_b55e85ebf7bb4223ba7e0df0f3518e23_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_b55e85ebf7bb4223ba7e0df0f3518e23_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-026 — src--configure_llm-py--run_wizard

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 1 — Initialization & Configuration / configuration; 原源码 src/configure_llm.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Returns 0 when the user has completed the interactive configuration flow and all resulting configuration has been persisted to fm-agent.toml, the project .env file, and the OpenCode provider configuration file. Returns non-zero when the user aborts the flow or when configuration values provided by the user fail validation. When the selected backend is not 'opencode', no API key or OpenCode provider configuration is written and only the backend-specific configuration is applied; the return value reflects whether that backend-specific configuration succeeded. When the selected backend is 'opencode': the API key is written to .env and to a provider-specific key file at the location determined by the OpenCode configuration; non-secret configuration values (provider name, base URL, model ID, API style) are written to fm-agent.toml; the OpenCode provider configuration is updated to reference the chosen provider and backend. Before applying changes, a summary of the planned modifications is printed to stdout, and warnings are emitted for any process environment variables whose precedence would override the configuration being written. When the user declines the confirmation prompt, no file modifications occur and the return value is non-zero. Raises ConfigWizardError when configuration cannot be completed due to an unrecoverable error such as a missing required file or an unwritable configuration path.
- **Actual behavior**: After execution of run_wizard(project_root), the return value r is an integer. Let b = _prompt_backend() be the user-selected backend. If b 'opencode', then: (if project_root/fm-agent.toml is writable, then r = 0, and the file is modified so that the LLM backend is set to b (with no other configuration changes), and a backup copy exists in the same directory; otherwise, r 0 and no files are

modified.) If `b = 'opencode'`, let `(config, validate) = prompt_for_config()` and let `paths = default_paths(project_root)`. If `_prompt_yes_no('Continue?')` returns `False`, then `r = 1` and no files are modified. Otherwise, `apply_configuration(config, paths, validate)` is called, which modifies the `fm-agent.toml`, `.env`, and `OpenCode` config files at `paths.toml_path`, `paths.env_path`, `paths.opencode_config_path` respectively: `fm-agent.toml`'s `[llm]` section is updated with `provider=config.provider_id`, `name=config.provider_name`, `base_url=config.base_url`, `api_style=config.api_style`, other sections unchanged; `.env` gains/replaces the entry `LLM_API_KEY=config.api_key`; `OpenCode` config references the chosen provider and API style; backups of each modified file are created. If `validate` is `true`, the written TOML and `OpenCode` config are syntactically valid. `r = 0`. Warnings about environment overrides are printed but do not affect `r`.

Formal logic: `r . b = _prompt_backend(). if b 'opencode': (writable(project_root / 'fm-agent.toml') r = 0 updated_llm_backend(project_root / 'fm-agent.toml', b) backup_file) (writable(project_root / 'fm-agent.toml') r 0 unchanged(project_root / 'fm-agent.toml') unchanged(env) unchanged(opencode_config)) else: # b = 'opencode' (config, validate) = prompt_for_config() paths = default_paths(project_root) continue = _prompt_yes_no('Continue?') if continue: r = 1 unchanged_all(project_root / 'fm-agent.toml', paths.env_path, paths.opencode_config_path) else: backups = apply_configuration(config, paths, validate) (fm_agent_toml_llm_section_only_modified(project_root / 'fm-agent.toml', config)) (env_has_api_key(paths.env_path, config.api_key)) (opencode_config_updated(paths.opencode_config_path, config.provider_id, config.api_style)) ((orig, bak) backups: bak None backup_of(bak, orig)) (if validate: syntax_valid_toml syntax_valid_open_code) r = 0`

- **Code evidence:** Line 10: `return run_local_backend_configuration(project_root, backend)`
- **Trigger condition:** Specification requires that unrecoverable errors such as unwritable configuration paths raise `ConfigWizardError`. The code instead returns a non-zero value from `run_local_backend_configuration`, which does not raise the required exception.
- **Validator trigger:** When a non-opencode backend is selected and the `fm-agent.toml` path is unwritable, `run_wizard()` raises `PermissionError` instead of the specification-required `ConfigWizardError`.
- **Probe stdout:** FM-Agent LLM configuration

Local CLI backends use their own authentication; no API key or `OpenCode` provider configuration is required. FM-Agent local CLI backend configuration

Backend: auto

The following files will be updated:

- `/tmp/tmpyhlwtr2u/fm-agent.toml`

No legacy LLM overrides were found in the project `.env` file.

No API key or `OpenCode` provider configuration will be changed.

Warning: these shell environment variables override the saved LLM settings: `LLM_API_KEY`, `LLM_API_BASE_URL`, `LLM_MODEL`, `FM_AGENT_MODEL_BACKEND`, `LLM_EFFORT`, `OPENCODE_MODEL_PROVIDER` The wizard cannot change the shell that launched it. Before starting FM-Agent in this shell, unset them to use the saved configuration: `unset LLM_API_KEY LLM_API_BASE_URL LLM_MODEL FM_AGENT_MODEL_BACKEND LLM_EFFORT OPENCODE_MODEL_PROVIDER`

CONFIRMED — `run_wizard()` raised `PermissionError` instead of `ConfigWizardError` for an unwritable path: [Errno 13] Permission denied: `'/tmp/tmpyhlwtr2u/fm-agent.toml.bak.20260727-235010'`

► SPEC sidecar (完整)

```
{
  "signature": "run_wizard(project_root: Path) -> int",
  "pre_condition": "project_root is a Path identifying a directory that contains a readable fm-agent.toml file.",
  "post_condition": "Returns 0 when the user has completed the interactive configuration flow and all resulting configuration has been persisted to fm-agent.toml, the project .env file, and the OpenCode provider configuration file. Returns non-zero when the user aborts the flow or when configuration values provided by the user fail validation. When the selected backend is not 'opencode', no API key or OpenCode provider configuration is written and only the backend-specific configuration is applied; the return value reflects whether that backend-specific configuration succeeded. When the selected backend is 'opencode': the API key is written to .env and to a provider-specific key file at the location determined by the OpenCode configuration; non-secret configuration values (provider name, base URL, model ID, API style) are written to fm-agent.toml; the OpenCode provider configuration is updated to reference the chosen provider and backend. Before applying changes, a summary of the planned modifications is printed to stdout, and warnings are emitted for any process environment variables whose precedence would override the configuration being written. When the user declines the confirmation prompt, no file modifications occur and the return value is non-zero. Raises ConfigWizardError when configuration cannot be completed due to an unrecoverable error such as a missing required file or an unwritable configuration path."
}
```

► INFO / callees sidecar (完整)


```

{
  "callees": [
    {
      "name": "_prompt_backend",
      "signature": "_prompt_backend() -> str",
      "pre_condition": "None required.",
      "post_condition": "Displays the available LLM backend options to the user via stdout, reads the user's selection from stdin, and returns the selected backend identifier as a non-empty string that is one of: 'opencode', 'auto', 'codex-cli', or 'claude-cli'. Re-prompts until the input is recognized as a valid backend identifier."
    },
    {
      "name": "run_local_backend_configuration",
      "signature": "run_local_backend_configuration(project_root: Path, backend: str) -> int",
      "pre_condition": "project_root is a Path identifying a directory containing a readable fm-agent.toml file; backend is a non-empty string identifying a local CLI backend and is not 'opencode'.",
      "post_condition": "Returns 0 when the fm-agent.toml file has been updated to set the LLM backend to the provided backend value, with a backup of the original file created. Returns non-zero when the update cannot be completed because the TOML file is not writable. No API key, .env modification, or OpenCode provider configuration is performed."
    },
    {
      "name": "default_paths",
      "signature": "default_paths(project_root: Path) -> WizardPaths",
      "pre_condition": "project_root is a Path identifying an existing directory.",
      "post_condition": "Returns a WizardPaths object whose toml_path, env_path, and opencode_config_path attributes refer to existing or creatable filesystem paths derived from project_root. toml_path is resolved from the FM_AGENT_CONFIG environment variable when set, otherwise from <project_root>/fm-agent.toml. env_path is <project_root>/.env. opencode_config_path is resolved from OPENCODE_CONFIG when set, otherwise from the platform-specific OpenCode configuration directory."
    },
    {
      "name": "prompt_for_config",
      "signature": "prompt_for_config() -> tuple[LLMConfigInput, bool]",
      "pre_condition": "None required.",
      "post_condition": "Prompts the user interactively via stdout/stdin for each required configuration field: provider ID, provider name, API style, base URL, model ID, and API key. Returns a tuple of (LLMConfigInput, validate_flag) where LLMConfigInput is a frozen dataclass containing the user-provided values as non-empty strings and the API style as a valid ApiStyle enum value, and validate_flag is a boolean indicating whether the user requested syntax validation. Re-prompts for any field whose input does not satisfy per-field validation rules."
    },
    {
      "name": "_preview",
      "signature": "_preview(config: LLMConfigInput, paths: WizardPaths) -> str",
      "pre_condition": "config is an LLMConfigInput with non-empty provider_id, provider_name, api_style, base_url, model_id, and api_key fields; paths is a WizardPaths whose attribute paths exist or can be created.",
      "post_condition": "Returns a human-readable string summarizing which files will be modified and the configuration values that will be written to each: the fm-agent.toml LLM section settings (provider, name, base_url, api_style), the .env API key entry, and the OpenCode provider configuration changes."
    },
    {
      "name": "warn_live_llm_environment_overrides",
      "signature": "warn_live_llm_environment_overrides() -> None",
      "pre_condition": "None required.",
      "post_condition": "Prints a warning to stdout when any LLM-related environment variable (from the set of variables listed in _ENV_MAP whose target section is 'llm') is currently set in the process environment, indicating that the live environment value will take precedence over any value written to configuration files. Prints nothing when no such variables are set."
    },
    {
      "name": "live_llm_environment_overrides",
      "signature": "live_llm_environment_overrides() -> bool",
      "pre_condition": "None required.",
      "post_condition": "Returns True when at least one LLM-related environment variable (from the set of variables listed in _ENV_MAP whose target section is 'llm') is currently set in the process environment; returns False otherwise. The result depends only on the current process environment, not on any file contents."
    },
    {
      "name": "_prompt_yes_no",
      "signature": "_prompt_yes_no(message: str, *, default: bool = True) -> bool",
      "pre_condition": "message is a non-empty string; default is True or False.",
      "post_condition": "Prints message followed by a [Y/n] or [y/N] prompt to stdout reflecting the default value. Reads one line from stdin. Returns True when the user responds with a yes-affirming pattern (case-insensitive 'y' or 'yes') or when the input is empty and default is True. Returns False when the user responds with a no-affirming pattern (case-insensitive 'n' or 'no') or when the input is empty and default is False."
    },
    {
      "name": "apply_configuration",
      "signature": "apply_configuration(config: LLMConfigInput, paths: WizardPaths, *, validate: bool = False) -> list[tuple[Path, Path | None]]",
      "pre_condition": "config is an LLMConfigInput with non-empty provider_id, provider_name, api_style, base_url, model_id, and api_key; paths is a WizardPaths whose attribute paths exist or can be created; validate is True when syntax validation against the target configuration formats is requested.",
      "post_condition": "Writes the API key to the .env file at paths.env_path, inserting or replacing the LLM_API_KEY line without affecting other entries. Writes the non-secret LLM settings (provider, name, base_url, api_style) to the LLM section of fm-agent.toml at paths.toml_path, preserving all other sections and fields unchanged. Updates the OpenCode provider configuration at paths.opencode_config_path to reference the chosen provider and API style, creating the file if it does not exist. Backs up each modified file to a timestamp-suffixed copy in the same directory before writing. When validate is True, verifies that the written TOML content is syntactically valid and that the OpenCode configuration parses correctly. Returns a list of (modified_file_path, backup_path_or_None) tuples, one per modified file, ordered by the sequence of writes."
    }
  ]
}

```

```
]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/run_wizard.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns 0 when the user has completed the interactive configuration flow and all resulting configuration has been persisted to fm-agent.toml, the project .env file, and the OpenCode provider configuration file. Returns non-zero when the user aborts the flow or when configuration values provided by the user fail validation. When the selected backend is not 'opencode', no API key or OpenCode provider configuration is written and only the backend-specific configuration is applied; the return value reflects whether that backend-specific configuration succeeded. When the selected backend is 'opencode': the API key is written to .env and to a provider-specific key file at the location determined by the OpenCode configuration; non-secret configuration values (provider name, base URL, model ID, API style) are written to fm-agent.toml; the OpenCode provider configuration is updated to reference the chosen provider and backend. Before applying changes, a summary of the planned modifications is printed to stdout, and warnings are emitted for any process environment variables whose precedence would override the configuration being written. When the user declines the confirmation prompt, no file modifications occur and the return value is non-zero. Raises ConfigWizardError when configuration cannot be completed due to an unrecoverable error such as a missing required file or an unwritable configuration path.",
    "actual_behavior": "After execution of run_wizard(project_root), the return value r is an integer. Let b = _prompt_backend() be the user-selected backend. If b 'opencode', then: (if project_root/fm-agent.toml is writable, then r = 0, and the file is modified so that the LLM backend is set to b (with no other configuration changes), and a backup copy exists in the same directory; otherwise, r = 0 and no files are modified.) If b = 'opencode', let (config, validate) = prompt_for_config() and let paths = default_paths(project_root). If _prompt_yes_no('Continue?') returns False, then r = 1 and no files are modified. Otherwise, apply_configuration(config, paths, validate) is called, which modifies the fm-agent.toml, .env, and OpenCode config files at paths.toml_path, paths.env_path, paths.opencode_config_path respectively: fm-agent.toml's [llm] section is updated with provider=config.provider_id, name=config.provider_name, base_url=config.base_url, api_style=config.api_style, other sections unchanged; .env gains/replaces the entry LLM_API_KEY=config.api_key; OpenCode config references the chosen provider and API style; backups of each modified file are created. If validate is true, the written TOML and OpenCode config are syntactically valid. r = 0. Warnings about environment overrides are printed but do not affect r.\n\nFormal logic:\nr  \n b = _prompt_backend().\nif b 'opencode':\n  (writable(project_root / 'fm-agent.toml')  r = 0  updated_llm_backend(project_root / 'fm-agent.toml', b) backup_file )\n  (writable(project_root / 'fm-agent.toml')  r 0  unchanged(project_root / 'fm-agent.toml')  unchanged(env) unchanged(opencode_config) )\nelse:  # b = 'opencode'\n  (config, validate) = prompt_for_config()\n  paths = default_paths(project_root)\n  continue = _prompt_yes_no('Continue?')\n  if continue: r = 1  unchanged_all(project_root / 'fm-agent.toml', paths.env_path, paths.opencode_config_path)\n  else:\n    backups = apply_configuration(config, paths, validate)\n    ( fm_agent_toml_llm_section_only_modified(project_root / 'fm-agent.toml', config) )\n    ( env_has_api_key(paths.env_path, config.api_key) )\n    ( opencode_config_updated(paths.opencode_config_path, config.provider_id, config.api_style) )\n    ( (orig, bak) backups: bak None backup_of(bak, orig) )\n    ( if validate: syntax_valid_toml syntax_valid_open_code )\n    r = 0",
    "code_evidence": "Line 10: return run_local_backend_configuration(project_root, backend)",
    "trigger_condition": "Specification requires that unrecoverable errors such as unwritable configuration paths raise ConfigWizardError. The code instead returns a non-zero value from run_local_backend_configuration, which does not raise the required exception."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--run_wizard",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/run_wizard.py",
  "function_name": "run_wizard",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--run_wizard.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--run_wizard.md",
  "probe_stdout": "FM-Agent LLM configuration\n\nLocal CLI backends use their own authentication; no API key or OpenCode provider configuration is required.\n\nFM-Agent local CLI backend configuration\n\nBackend: auto\n\nThe following files will be updated:\n - /tmp/tmpyhlwtr2u/fm-agent.toml\n\nNo legacy LLM overrides were found in the project .env file.\n\nNo API key or OpenCode provider configuration will be changed.\n\nWarning: these shell environment variables override the saved LLM settings:\n  LLM_API_KEY, LLM_API_BASE_URL, LLM_MODEL, FM_AGENT_MODEL_BACKEND, LLM_EFFORT, OPENCODE_MODEL_PROVIDER\n\nThe wizard cannot change the shell that launched it. Before starting FM-Agent\n\nin this shell, unset them to use the saved configuration:\n  unset LLM_API_KEY LLM_API_BASE_URL LLM_MODEL FM_AGENT_MODEL_BACKEND LLM_EFFORT OPENCODE_MODEL_PROVIDER\n\n\nCONFIRMED - run_wizard() raised PermissionError instead of ConfigWizardError for an unwritable path: [Errno 13] Permission denied: '/tmp/tmpyhlwtr2u/fm-agent.toml.bak.20260727-235010'",
  "trigger_summary": "When a non-opencode backend is selected and the fm-agent.toml path is unwritable, run_wizard() raises PermissionError instead of the specification-required ConfigWizardError."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:45:24.772820Z	5776

#	purpose	status	attempt	block	start	tokens
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:46:21.093405Z	5438

事件 1 — [llm_1de5ec839e5240c388ccb9f8dea67924](#)

► [parsed_json](#) (完整)

```
{
  "post_condition": "After execution of run_wizard(project_root), the return value r is an integer. Let b = _prompt_backend() be the user-selected backend. If b ≠ 'opencode', then: (if project_root/fm-agent.toml is writable, then r = 0, and the file is modified so that the LLM backend is set to b (with no other configuration changes), and a backup copy exists in the same directory; otherwise, r ≠ 0 and no files are modified.) If b = 'opencode', let (config, validate) = prompt_for_config() and let paths = default_paths(project_root). If _prompt_yes_no('Continue?') returns False, then r = 1 and no files are modified. Otherwise, apply_configuration(config, paths, validate) is called, which modifies the fm-agent.toml, .env, and OpenCode config files at paths.toml_path, paths.env_path, paths.opencode_config_path respectively: fm-agent.toml's [llm] section is updated with provider=config.provider_id, name=config.provider_name, base_url=config.base_url, api_style=config.api_style, other sections unchanged; .env gains/replaces the entry LLM_API_KEY=config.api_key; OpenCode config references the chosen provider and API style; backups of each modified file are created. If validate is true, the written TOML and OpenCode config are syntactically valid. r = 0. Warnings about environment overrides are printed but do not affect r.\n\nFormal logic:\nr ∈ ℤ.\n∃ b = _prompt_backend().\nif b ≠ 'opencode':\n  ( writable(project_root / 'fm-agent.toml') ⇒ r = 0 ∧ updated_llm_backend(project_root / 'fm-agent.toml', b) ∧ ∃ backup_file )\n  ∧ ( ¬writable(project_root / 'fm-agent.toml') ⇒ r ≠ 0 ∧ unchanged(project_root / 'fm-agent.toml') ∧ unchanged(env) ∧ unchanged(opencode_config) )\nelse: # b = 'opencode'\n  ∃ (config, validate) = prompt_for_config()\n  ∃ paths = default_paths(project_root)\n  ∃ continue = _prompt_yes_no('Continue?')\n  if ¬continue: r = 1 ∧ unchanged_all(project_root / 'fm-agent.toml', paths.env_path, paths.opencode_config_path)\n  else:\n    ∃ backups = apply_configuration(config, paths, validate)\n    ∧ ( fm_agent_toml_llm_section_only_modified(project_root / 'fm-agent.toml', config) )\n    ∧ ( env_has_api_key(paths.env_path, config.api_key) )\n    ∧ ( opencode_config_updated(paths.opencode_config_path, config.provider_id, config.api_style) )\n    ∧ ( ∀ (orig, bak) ∈ backups: bak ≠ None ⇒ backup_of(bak, orig) )\n    ∧ ( if validate: syntax_valid_toml ∧ syntax_valid_open_code )\n    ∧ r = 0"
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1de5ec839e5240c388ccb9f8dea67924_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_1de5ec839e5240c388ccb9f8dea67924_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_1de5ec839e5240c388ccb9f8dea67924_response.txt)

事件 2 — [llm_7f997dbb31394a1e9be519283fe23f41](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "backend = 'claude-cli', project_root/fm-agent.toml exists but is not writable (e.g., permissions 0444).",
  "offending_statements": "Line 10: return run_local_backend_configuration(project_root, backend)",
  "reason": "Specification requires that unrecoverable errors such as unwritable configuration paths raise ConfigWizardError. The code instead returns a non-zero value from run_local_backend_configuration, which does not raise the required exception."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7f997dbb31394a1e9be519283fe23f41_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_7f997dbb31394a1e9be519283fe23f41_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_7f997dbb31394a1e9be519283fe23f41_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-027 — [src--configure_llm-py--secret_path_for_provider](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a Path identifying a filesystem location within the platform-specific user configuration directory. The returned path is deterministically derived from config.provider_id: equal provider_id values always produce equal paths on the same platform, and distinct provider_id values produce distinct paths. The path's filename begins with the prefix 'fm-agent-opencode-api-key.' and ends with a 10-character lowercase hexadecimal suffix. The parent directory of the returned path exists.
- **Actual behavior**: Returns a pathlib.Path object for a file named 'fm-agent-opencode-api-key.<safe_provider_id>.' located in the directory returned by private_opencode_secret_dir(). The directory is guaranteed to exist after the call (created if absent). safe_provider_id is derived from

- `config.provider_id` by replacing any character not in `[A-Za-z0-9.-]` with `'_'`, stripping leading and trailing characters in `'.'`, and defaulting to `'provider'` if the result is empty. `digest` is the first 10 characters of the SHA256 hex digest of `config.provider_id` encoded as UTF8. Formal post-condition: let `raw = config.provider_id`, `safe = (re.sub(r'^A-Za-z0-9.-]+', '', raw).strip('._-'))` or `'provider'`, `digest = hashlib.sha256(raw.encode('utf-8')).hexdigest()[:10]`, `directory = _private_opencode_secret_dir()`; then the return value = `directory / f'fm-agent-opencode-api-key.{safe}.{digest}'` and `directory.exists()` is `True`.
- **Code evidence:** Line 6: `return _private_opencode_secret_dir() / (` Line 7: `f"fm-agent-opencode-api-key.{safe_provider_id}.{digest}"` Line 8: `)`
 - **Trigger condition:** Specification requires the returned Path to be within the platform-specific user configuration directory, but the code builds the path inside `_private_opencode_secret_dir()`, which is only defined as a directory for secret keys and is not guaranteed to be a subdirectory of (or identical to) the user configuration directory. For example, on a typical Linux system the user configuration directory is `~/.config`, while `_private_opencode_secret_dir()` may return `~/.opencode/secrets`, which is outside the required directory.
 - **Validator trigger:** `secret_path_for_provider` places the secret key file under the XDG state directory (`~/.local/state`) instead of the XDG config directory (`~/.config`) as the specification requires.
 - **Probe stdout:** CONFIRMED — `secret_path_for_provider` returns path outside the platform config directory. Returned path :
`/home/fancy/.local/state/fm-agent/opencode/fm-agent-opencode-api-key.test-provider-42.2428fffd89` Config dir : `/home/fancy/.config` State dir :
`/home/fancy/.local/state/fm-agent` The returned path is under the XDG_STATE_HOME / state directory, not under the XDG_CONFIG_HOME / config directory as the spec requires.

► SPEC sidecar (完整)

```
{
  "signature": "secret_path_for_provider(config: LLMConfigInput) -> Path",
  "pre_condition": "config is an LLMConfigInput with a non-empty provider_id string.",
  "post_condition": "Returns a Path identifying a filesystem location within the platform-specific user configuration directory. The returned path is deterministically derived from config.provider_id: equal provider_id values always produce equal paths on the same platform, and distinct provider_id values produce distinct paths. The path's filename begins with the prefix 'fm-agent-opencode-api-key.' and ends with a 10-character lowercase hexadecimal suffix. The parent directory of the returned path exists."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_private_opencode_secret_dir",
      "signature": "_private_opencode_secret_dir() -> Path",
      "pre_condition": "None.",
      "post_condition": "Returns the platform-specific directory path intended for storing OpenCode private secret key files. The returned directory exists on the filesystem, having been created if it was absent before the call."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/secret_path_for_provider.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a Path identifying a filesystem location within the platform-specific user configuration directory. The returned path is deterministically derived from config.provider_id: equal provider_id values always produce equal paths on the same platform, and distinct provider_id values produce distinct paths. The path's filename begins with the prefix 'fm-agent-opencode-api-key.' and ends with a 10-character lowercase hexadecimal suffix. The parent directory of the returned path exists.",
    "actual_behavior": "Returns a pathlib.Path object for a file named 'fm-agent-opencode-api-key.<safe_provider_id>.<digest>' located in the directory returned by _private_opencode_secret_dir(). The directory is guaranteed to exist after the call (created if absent). safe_provider_id is derived from config.provider_id by replacing any character not in [A-Za-z0-9.-] with '_', stripping leading and trailing characters in '._-', and defaulting to 'provider' if the result is empty. digest is the first 10 characters of the SHA256 hex digest of config.provider_id encoded as UTF8. Formal post-condition: let raw = config.provider_id, safe = (re.sub(r'^A-Za-z0-9.-]+', '_', raw).strip('._-')) or 'provider', digest = hashlib.sha256(raw.encode('utf-8')).hexdigest()[:10], directory = _private_opencode_secret_dir(); then the return value = directory / f'fm-agent-opencode-api-key.{safe}.{digest}' and directory.exists() is True.",
    "code_evidence": "Line 6: return _private_opencode_secret_dir() / (\nLine 7:     f\"fm-agent-opencode-api-key.{safe_provider_id}.{digest}\" \nLine 8: )",
    "trigger_condition": "Specification requires the returned Path to be within the platform-specific user configuration directory, but the code builds the path inside _private_opencode_secret_dir(), which is only defined as a directory for secret keys and is not guaranteed to be a subdirectory of (or identical to) the user configuration directory. For example, on a typical Linux system the user configuration directory is ~/.config, while _private_opencode_secret_dir() may return ~/.opencode/secrets, which is outside the required directory."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--configure_llm-py--secret_path_for_provider",
  "source_file": "src/configure_llm.py",
  "function_name": "secret_path_for_provider",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--secret_path_for_provider.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--secret_path_for_provider.md",
  "probe_stdout": "CONFIRMED - secret_path_for_provider returns path outside the platform config directory.
Returned path : /home/fancy/.local/state/fm-agent/opencode/fm-agent-opencode-api-key.test-provider-42.2428fffd89
Config dir : /home/fancy/.config
State dir : /home/fancy/.local/state/fm-agent
The returned path is under the XDG_STATE_HOME / state directory, not under the XDG_CONFIG_HOME / config directory as the spec requires.",
  "trigger_summary": "secret_path_for_provider places the secret key file under the XDG state directory (~/.local/state) instead of the XDG config directory (~/.config) as the specification requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:59:29.488535Z	1506
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:59:50.084037Z	14422

事件 1 — llm_4960d52eecba4109b1eeab03826b0ddd

► parsed_json（完整）

```
{
  "post_condition": "Returns a pathlib.Path object for a file named 'fm-agent-opencode-api-key.<safe_provider_id>.<digest>' located in the directory returned by _private_opencode_secret_dir(). The directory is guaranteed to exist after the call (created if absent). safe_provider_id is derived from config.provider_id by replacing any character not in [A-Za-z0-9._-] with '_', stripping leading and trailing characters in '._-', and defaulting to 'provider' if the result is empty. digest is the first 10 characters of the SHA-256 hex digest of config.provider_id encoded as UTF-8. Formal post-condition: let raw = config.provider_id, safe = (re.sub(r'[A-Za-z0-9._-]+' , '_', raw).strip('._-')) or 'provider', digest = hashlib.sha256(raw.encode('utf-8')).hexdigest()[:10], directory = _private_opencode_secret_dir(); then the return value = directory / f'fm-agent-opencode-api-key.{safe}.{digest}' and directory.exists() is True."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4960d52eecba4109b1eeab03826b0ddd_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4960d52eecba4109b1eeab03826b0ddd_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4960d52eecba4109b1eeab03826b0ddd_response.txt)

事件 2 — llm_0bfed63bbde14dbcb21d8409631a7123

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "config.provider_id = \"example\"",
  "offending_statements": "Line 6: return _private_opencode_secret_dir() / (\nLine 7:      f\"fm-agent-opencode-api-key.{safe_provider_id}.{digest}\" \nLine 8: )",
  "reason": "Specification requires the returned Path to be within the platform-specific user configuration directory, but the code builds the path inside _private_opencode_secret_dir(), which is only defined as a directory for secret keys and is not guaranteed to be a subdirectory of (or identical to) the user configuration directory. For example, on a typical Linux system the user configuration directory is ~/.config, while _private_opencode_secret_dir() may return ~/.opencode/secrets, which is outside the required directory."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_0bfed63bbde14dbcb21d8409631a7123_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_0bfed63bbde14dbcb21d8409631a7123_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_0bfed63bbde14dbcb21d8409631a7123_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-028 — src--configure_llm-py--update_env_text

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。

- **Validator:** [confirmed](#); attempts 1
- **Phase / 模块:** Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a string where (1) exactly one line assigns ENV_SECRET_KEY to api_key, positioned where an existing ENV_SECRET_KEY line appeared or appended at the end if absent; (2) lines whose key, after stripping an optional 'export ' prefix, belongs to a predetermined set of legacy LLM configuration keys are excluded from the output; (3) every other line blank, comment, or assignment appears verbatim in its original relative order. When an existing ENV_SECRET_KEY line carries an 'export ' prefix, the rewritten line preserves that prefix. When the key is appended and the input contains no non-blank, non-comment lines, header comment lines precede the appended key line.
- **Actual behavior:** The function returns a string result obtained by transforming the input text (a .env file content) as follows. Let L = text.splitlines(keepends=True). Process each line L in order: let s = .lstrip(). If s is empty or s.startswith('#'), copy to output. Otherwise, attempt to parse a key-value assignment. Determine leading whitespace and optional 'export ' prefix: let leading = [:len()-len(s)]; if _ENV_EXPORT_PREFIX_RE.match(s) is successful, let export_prefix = leading + match.group(), let working = leading + s[match.end():]; else export_prefix = "", working = . Split working at the first '=': key_candidate, sep, _ = working.partition('='). If sep == " (no '='), copy to output. Otherwise, let env_key = key_candidate.strip(). If env_key == ENV_SECRET_KEY, output the line "{export_prefix}ENV_SECRET_KEY={api_key}\n" and set a flag key_written = True. If env_key in ENV_LEGACY_LLM_KEYS, omit the line (do not copy). Else, copy unchanged. After processing all lines, if key_written is False, append the secret line: if the output list is empty, first prepend the two header comment lines: '# fm-agent secrets gitignored, do not commit.\n', '# Only the LLM API key belongs here.\n'; then append the line ENV_SECRET_KEY={api_key}\n. Otherwise, if the last line in the output has non-empty stripped content, first append a newline '\n', then append the secret line; if the last line is already blank (only whitespace), just append the secret line. Finally, return the concatenation of all output lines. The relative order of lines that are copied from the input is preserved. Formally: key_written {true, false} such that result = ".join(N) where N is a list of strings and the following hold: (1) For each i, = L[i], let c(i) = class of : 'blank' if .strip() == ""; 'comment' if .lstrip().startswith('#'); 'no_eq' if contains no '=' after stripping leading whitespace and a possible 'export ' prefix; 'secret' if extracted key equals ENV_SECRET_KEY; 'legacy' if extracted key ENV_LEGACY_LLM_KEYS; 'other' otherwise. (2) The sequence N is formed by taking, for each i where c(i) is not 'legacy', a string v(i) defined as: if c(i) is 'blank' or 'comment' or 'no_eq' or 'other', v(i)=; if c(i) is 'secret', v(i)=export_prefix + ENV_SECRET_KEY + '=' + api_key + '\n'. (3) If key_written = true, N = [v(i) for i where c(i)'legacy']. (4) If key_written = false, then let M = [v(i) for i where c(i)'legacy']; if M is empty, N = ['# fm-agent secrets gitignored, do not commit.\n', '# Only the LLM API key belongs here.\n', ENV_SECRET_KEY + '=' + api_key + '\n']; else if M[-1].strip() != "", N = M + ['\n', ENV_SECRET_KEY + '=' + api_key + '\n']; else N = M + [ENV_SECRET_KEY + '=' + api_key + '\n']. (5) The order of v(i) in N corresponds to increasing i.
- **Code evidence:** Line 21: if env_key == ENV_SECRET_KEY: Line 22: new_lines.append(f'{export_prefix}{ENV_SECRET_KEY}={api_key}\n') Line 23: key_written = True Line 24: continue
- **Trigger condition:** The code replaces every occurrence of ENV_SECRET_KEY, producing multiple output lines for the secret key when the input contains more than one. The specification requires exactly one line of that key in the output.
- **Validator trigger:** When input contains multiple ENV_SECRET_KEY lines, the function outputs a replacement for every occurrence, violating the spec requirement of exactly one such line.
- **Probe stdout:** CONFIRMED — LLM_API_KEY= appears 2 times (expected exactly 1). Actual output: 'LLM_API_KEY=new-test-api-key\nLLM_API_KEY=new-test-api-key\nOTHER_VAR=keep_me\n'

► SPEC sidecar (完整)

```
{
  "signature": "update_env_text(text: str, api_key: str) -> str",
  "pre_condition": "text is a string whose content follows .env file conventions – each line is blank, a comment beginning with '#', or a KEY=VALUE assignment optionally preceded by 'export ' ; api_key is a non-empty string.",
  "post_condition": "Returns a string where (1) exactly one line assigns ENV_SECRET_KEY to api_key, positioned where an existing ENV_SECRET_KEY line appeared or appended at the end if absent; (2) lines whose key, after stripping an optional 'export ' prefix, belongs to a predetermined set of legacy LLM configuration keys are excluded from the output; (3) every other line – blank, comment, or assignment – appears verbatim in its original relative order. When an existing ENV_SECRET_KEY line carries an 'export ' prefix, the rewritten line preserves that prefix. When the key is appended and the input contains no non-blank, non-comment lines, header comment lines precede the appended key line."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)


```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/update_env_text.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a string where (1) exactly one line assigns ENV_SECRET_KEY to api_key, positioned where an existing ENV_SECRET_KEY line appeared or appended at the end if absent; (2) lines whose key, after stripping an optional 'export ' prefix, belongs to a predetermined set of legacy LLM configuration keys are excluded from the output; (3) every other line blank, comment, or assignment appears verbatim in its original relative order. When an existing ENV_SECRET_KEY line carries an 'export ' prefix, the rewritten line preserves that prefix. When the key is appended and the input contains no non-blank, non-comment lines, header comment lines precede the appended key line.",
    "actual_behavior": "The function returns a string result obtained by transforming the input text (a .env file content) as follows. Let L = text.splitlines(keepends=True). Process each line L in order: let s = .lstrip(). If s is empty or s.startswith('#'), copy to output. Otherwise, attempt to parse a key-value assignment. Determine leading whitespace and optional 'export ' prefix: let leading = [:len()-len(s)]; if _ENV_EXPORT_PREFIX_RE.match(s) is successful, let export_prefix = leading + match.group(), let working = leading + s[match.end():]; else export_prefix = '', working = . Split working at the first '=': key_candidate, sep, _ = working.partition('='). If sep == '' (no '='), copy to output. Otherwise, let env_key = key_candidate.strip(). If env_key == ENV_SECRET_KEY, output the line f"{export_prefix}ENV_SECRET_KEY={api_key}\\n" and set a flag key_written = True. If env_key in ENV_LEGACY_LLM_KEYS, omit the line (do not copy). Else, copy unchanged. After processing all lines, if key_written is False, append the secret line: if the output list is empty, first prepend the two header comment lines: '# fm-agent secrets gitignored, do not commit.\\n', '# Only the LLM API key belongs here.\\n'; then append the line ENV_SECRET_KEY={api_key}\\n. Otherwise, if the last line in the output has non-empty stripped content, first append a newline '\\n', then append the secret line; if the last line is already blank (only whitespace), just append the secret line. Finally, return the concatenation of all output lines. The relative order of lines that are copied from the input is preserved. Formally: key_written {true, false} such that result = ''.join(N) where N is a list of strings and the following hold: (1) For each i, = L[i], let c(i) = class of : 'blank' if .strip() == ''; 'comment' if .lstrip().startswith('#'); 'no_eq' if contains no '=' after stripping leading whitespace and a possible 'export ' prefix; 'secret' if extracted key equals ENV_SECRET_KEY; 'legacy' if extracted key ENV_LEGACY_LLM_KEYS; 'other' otherwise. (2) The sequence N is formed by taking, for each i where c(i) is not 'legacy', a string v(i) defined as: if c(i) is 'blank' or 'comment' or 'no_eq' or 'other', v(i)=; if c(i) is 'secret', v(i)=export_prefix + ENV_SECRET_KEY + '=' + api_key + '\\n'. (3) If key_written = true, N = [v(i) for i where c(i)'legacy']. (4) If key_written = false, then let M = [v(i) for i where c(i)'legacy']; if M is empty, N = ['# fm-agent secrets gitignored, do not commit.\\n', '# Only the LLM API key belongs here.\\n', ENV_SECRET_KEY + '=' + api_key + '\\n']; else if M[-1].strip() != '', N = M + ['\\n', ENV_SECRET_KEY + '=' + api_key + '\\n']; else N = M + [ENV_SECRET_KEY + '=' + api_key + '\\n']. (5) The order of v(i) in N corresponds to increasing i.",
    "code_evidence": "Line 21: if env_key == ENV_SECRET_KEY:\\nLine 22: new_lines.append(f\"{export_prefix}{ENV_SECRET_KEY}={api_key}\\n\\n\")\\nLine 23: key_written = True\\nLine 24: continue",
    "trigger_condition": "The code replaces every occurrence of ENV_SECRET_KEY, producing multiple output lines for the secret key when the input contains more than one. The specification requires exactly one line of that key in the output."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--update_env_text",
  "source_file": "src/configure_llm.py",
  "function_name": "update_env_text",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--update_env_text.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--update_env_text.md",
  "probe_stdout": "CONFIRMED - LLM_API_KEY= appears 2 times (expected exactly 1). Actual output:\\n\\nLLM_API_KEY=new-test-api-key\\n\\nLLM_API_KEY=new-test-api-key\\n\\nOTHER_VAR=keep_me\\n\\n",
  "trigger_summary": "When input contains multiple ENV_SECRET_KEY lines, the function outputs a replacement for every occurrence, violating the spec requirement of exactly one such line."
}
```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-27T15:59:45.503363Z	3839
2	generate_block_post_condition	success	2	0/1	2026-07-27T16:00:36.665004Z	3648
3	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:01:02.271196Z	7307

事件 1 — [llm_82c46a339d0f4f78ab682daaa111343f](#)

- parse_error: [LLM response contains multiple JSON values](#)
- 载荷: [system_prompt](#) · [user_prompt](#) · [assistant_output](#)

事件 2 — [llm_ee09c09f74de4b27be95a14cb98021f8](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string result obtained by transforming the input text (a .env file content) as follows. Let L = text.splitlines(keepends=True). Process each line ℓ ∈ L in order: let s = ℓ.lstrip(). If s is empty or s.startswith('#'), copy ℓ to output. Otherwise, attempt to parse a key-value assignment. Determine leading whitespace and optional 'export ' prefix: let leading = ℓ[:len(ℓ)-len(s)]; if _ENV_EXPORT_PREFIX_RE.match(s) is successful, let export_prefix = leading + match.group(), let working = leading + s[match.end():]; else export_prefix = '', working = ℓ. Split working at the first '=': key_candidate, sep, _ = working.partition('='). If sep == '' (no '='), copy ℓ to output. Otherwise, let env_key = key_candidate.strip(). If env_key == ENV_SECRET_KEY, output the line f\"{export_prefix}ENV_SECRET_KEY={api_key}\\n\" and set a flag key_written = True. If env_key in ENV_LEGACY_LLM_KEYS, omit the line (do not copy). Else, copy ℓ unchanged. After processing all lines, if key_written is False, append the secret line: if the output list is empty, first prepend the two header comment lines: '# fm-agent secrets – gitignored, do not commit.\\n', '# Only the LLM API key belongs here.\\n'; then append the line ENV_SECRET_KEY={api_key}\\n. Otherwise, if the last line in the output has non-empty stripped content, first append a newline '\\n', then append the secret line; if the last line is already blank (only whitespace), just append the secret line. Finally, return the concatenation of all output lines. The relative order of lines that are copied from the input is preserved. Formally: ∃ key_written ∈ {true, false} such that result = ''.join(N) where N is a list of strings and the following hold: (1) For each i, ℓ = L[i], let c(i) = class of ℓ: 'blank' if ℓ.strip() == ''; 'comment' if ℓ.lstrip().startswith('#'); 'no_eq' if ℓ contains no '=' after stripping leading whitespace and a possible 'export ' prefix; 'secret' if extracted key equals ENV_SECRET_KEY; 'legacy' if extracted key ∈ ENV_LEGACY_LLM_KEYS; 'other' otherwise. (2) The sequence N is formed by taking, for each i where c(i) is not 'legacy', a string v(i) defined as: if c(i) is 'blank' or 'comment' or 'no_eq' or 'other', v(i)=ℓ; if c(i) is 'secret', v(i)=export_prefix + ENV_SECRET_KEY + '=' + api_key + '\\n'. (3) If key_written = true, N = [v(i) for i where c(i)≠'legacy']. (4) If key_written = false, then let M = [v(i) for i where c(i)≠'legacy']; if M is empty, N = ['# fm-agent secrets – gitignored, do not commit.\\n', '# Only the LLM API key belongs here.\\n', ENV_SECRET_KEY + '=' + api_key + '\\n']; else if M[-1].strip() != '', N = M + ['\\n', ENV_SECRET_KEY + '=' + api_key + '\\n']; else N = M + [ENV_SECRET_KEY + '=' + api_key + '\\n']. (5) The order of v(i) in N corresponds to increasing i."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_message_02_assistant.txt) · [user_prompt](fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_message_03_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_response.txt)

事件 3 — **llm_18c54f5bdfbf44d08c8728b123988441**

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "text = \\\"FM_AGENT_SECRET_KEY=old\\n\\nOTHER=val\\n\\nFM_AGENT_SECRET_KEY=old2\\n\\n\\\"\\napi_key = \\\"newkey\\\"\\",
  "offending_statements": "Line 21: if env_key == ENV_SECRET_KEY:\\nLine 22: new_lines.append(f\"{export_prefix}{ENV_SECRET_KEY}={api_key}\\n\\n\")\\nLine 23: key_written = True\\nLine 24: continue",
  "reason": "The code replaces every occurrence of ENV_SECRET_KEY, producing multiple output lines for the secret key when the input contains more than one. The specification requires exactly one line of that key in the output."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_18c54f5bdfbf44d08c8728b123988441_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_18c54f5bdfbf44d08c8728b123988441_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_18c54f5bdfbf44d08c8728b123988441_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-031 — [src--configure_llm-py--validate_base_url](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： **confirmed**; attempts 1
- Phase / 模块： Phase 1 — Initialization & Configuration / **configuration**; 源代码 [src/configure_llm.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim**: Raises ConfigWizardError when url is not a syntactically valid absolute URL whose scheme is either 'http' or 'https'. Returns with no effect when url is a syntactically valid absolute URL with scheme 'http' or 'https'.
- Actual behavior**: If the function returns normally (does not raise an exception), then the parsed URL has scheme 'http' or 'https' and a non-empty network location (netloc). Otherwise, a ConfigWizardError is raised with a message indicating the URL is invalid. Formally: (url: str, url " url == url.strip() (after execution: (raised_exception urlparse(url).scheme {'http','https'} urlparse(url).netloc ") (raised_exception raised_exception is ConfigWizardError raised_exception.args[0] starts with 'Base URL must be an absolute http(s) URL, got:'))))
- Code evidence**: Line 2: parsed = urlparse(url) Line 3: if parsed.scheme not in ("http", "https") or not parsed.netloc:

- **Trigger condition:** The code only checks that the scheme is http/https and that netloc is non-empty, but does not validate that the netloc forms a syntactically valid authority (e.g., port must be numeric, hostname must not contain spaces). A URL like 'http://example.com:abc' (port with letters) is not syntactically valid according to RFC 3986, yet the code accepts it because urlparse sets netloc='example.com:abc' which is non-empty. The specification requires raising ConfigWizardError for any non-syntactically-valid absolute URL, so this input violates the specification.
- **Validator trigger:** URL with non-numeric port (http://example.com:abc) passes urlparse netloc check but is syntactically invalid per RFC 3986; the spec requires ConfigWizardError to be raised.
- **Probe stdout:** CONFIRMED — validate_base_url accepted an invalid URL (url='http://example.com:abc') but the spec requires raising ConfigWizardError

► SPEC sidecar（完整）

```
{
  "signature": "validate_base_url(url: str) -> None",
  "pre_condition": "url is a non-empty string with leading and trailing whitespace removed.",
  "post_condition": "Raises ConfigWizardError when url is not a syntactically valid absolute URL whose scheme is either 'http' or 'https'. Returns with no effect when url is a syntactically valid absolute URL with scheme 'http' or 'https'."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/validate_base_url.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Raises ConfigWizardError when url is not a syntactically valid absolute URL whose scheme is either 'http' or 'https'. Returns with no effect when url is a syntactically valid absolute URL with scheme 'http' or 'https'.",
    "actual_behavior": "If the function returns normally (does not raise an exception), then the parsed URL has scheme 'http' or 'https' and a non-empty network location (netloc). Otherwise, a ConfigWizardError is raised with a message indicating the URL is invalid. Formally: ( url: str, url '' url == url.strip() (after execution: (raised_exception urlparse(url).scheme {'http','https'} urlparse(url).netloc '' ) (raised_exception raised_exception is ConfigWizardError raised_exception.args[0] starts with 'Base URL must be an absolute http(s) URL, got:')))",
    "code_evidence": "Line 2: parsed = urlparse(url)\nLine 3: if parsed.scheme not in ('http', 'https') or not parsed.netloc:",
    "trigger_condition": "The code only checks that the scheme is http/https and that netloc is non-empty, but does not validate that the netloc forms a syntactically valid authority (e.g., port must be numeric, hostname must not contain spaces). A URL like 'http://example.com:abc' (port with letters) is not syntactically valid according to RFC 3986, yet the code accepts it because urlparse sets netloc='example.com:abc' which is non-empty. The specification requires raising ConfigWizardError for any non-syntactically-valid absolute URL, so this input violates the specification."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--validate_base_url",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/validate_base_url.py",
  "function_name": "validate_base_url",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--validate_base_url.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--validate_base_url.md",
  "probe_stdout": "CONFIRMED - validate_base_url accepted an invalid URL (url='http://example.com:abc') but the spec requires raising ConfigWizardError\n",
  "trigger_summary": "URL with non-numeric port (http://example.com:abc) passes urlparse netloc check but is syntactically invalid per RFC 3986; the spec requires ConfigWizardError to be raised."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:30:04.731168Z	1122
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:30:19.618859Z	3438

事件 1 — llm_7d13764a8c8e488087cd3c4a95899cc3

► parsed_json（完整）

```
{
  "post_condition": "If the function returns normally (does not raise an exception), then the parsed URL has scheme 'http' or 'https' and a non-empty network location (netloc). Otherwise, a ConfigWizardError is raised with a message indicating the URL is invalid. Formally: (∀ url: str, url ≠ '' ∧ url == url.strip() ⇒ (after execution: (¬raised_exception ⇒ urlparse(url).scheme ∈ {'http','https'} ∧ urlparse(url).netloc ≠ '')) ∧ (raised_exception ⇒ raised_exception is ConfigWizardError ∧ raised_exception.args[0] starts with 'Base URL must be an absolute http(s) URL, got:')))"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7d13764a8c8e488087cd3c4a95899cc3_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_7d13764a8c8e488087cd3c4a95899cc3_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_7d13764a8c8e488087cd3c4a95899cc3_response.txt)

事件 2 — llm_6d2f29f9c232497fa8688b3fa7a5d302

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "http://example.com:abc",
  "offending_statements": "Line 2: parsed = urlparse(url)\nLine 3: if parsed.scheme not in ('http', 'https') or not parsed.netloc:",
  "reason": "The code only checks that the scheme is http/https and that netloc is non-empty, but does not validate that the netloc forms a syntactically valid authority (e.g., port must be numeric, hostname must not contain spaces). A URL like 'http://example.com:abc' (port with letters) is not syntactically valid according to RFC 3986, yet the code accepts it because urlparse sets netloc='example.com:abc' which is non-empty. The specification requires raising ConfigWizardError for any non-syntactically-valid absolute URL, so this input violates the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_6d2f29f9c232497fa8688b3fa7a5d302_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_6d2f29f9c232497fa8688b3fa7a5d302_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_6d2f29f9c232497fa8688b3fa7a5d302_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-032 — src--configure_llm-py--validate_generated_state

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 1 — Initialization & Configuration / configuration; 原源码 src/configure_llm.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim：** Raises ConfigWizardError when updated_toml is not syntactically valid TOML, when merged_opencode is not JSON-serializable, when merged_opencode lacks a 'provider' key whose value is a dict containing an entry keyed by config.provider_id, when that provider entry is not a dict, when its 'npm' field does not equal the adapter name corresponding to config.api_style, when its 'options' field is missing, not a dict, or has an 'apiKey' value that does not equal the string '{file:<opencode_secret_path>}', when its 'models' field is missing, not a dict, or does not contain config.model_id, or when any field of config required for configuration is empty, missing, or malformed. Returns with no effect when all generated outputs pass their respective format and structural validation.
- Actual behavior：** After execution, exactly one of the following outcomes occurs:

- A ConfigWizardError is raised because:
 - validate_input(config) failed (invalid provider configuration), or
 - the generated OpenCode configuration is missing the required provider config.provider_id, or
 - the provider entry is not a dict, or

- the adapter (`npm`) does not match `adapter_for_api_style(config.api_style)`, or
- the options dict is missing or does not contain `'apiKey'` equal to `'{{file:opencode_secret_path}}'`, or
- the models dict is missing or does not contain `config.model_id`.

2. A `TypeError` or `ValueError` is raised from `json.dumps(merged_opencode)` because `merged_opencode` is not JSON-serializable.
3. A `tomllib.TOMLDecodeError` (or similar parsing exception) is raised from `tomllib.loads(updated_toml)` because `updated_toml` is not valid TOML.
4. An exception is propagated from `adapter_for_api_style(config.api_style)` if the API style is unrecognised, causing that call to raise.

5. The function returns `None` (normal termination). In this case all of the following hold:

- `validate_input(config)` succeeded, implying `config` meets all `LLMConfigInput` validity constraints.
- `json.dumps(merged_opencode)` succeeded, i.e., `merged_opencode` is JSON-serializable.
- `tomllib.loads(updated_toml)` succeeded, i.e., `updated_toml` is valid TOML.
- `merged_opencode.get('provider')` is a dict and `config.provider_id` is a key in it. Let `entry = merged_opencode['provider'][config.provider_id]`.
- `entry` is a dict.
- `entry.get('npm') == adapter_for_api_style(config.api_style)`.
- `entry.get('options')` is a dict and `options.get('apiKey') == f'{{file:{opencode_secret_path}}}'`.
- `entry.get('models')` is a dict and `config.model_id` is a key in it.

- **Code evidence:** Line 9: `json.dumps(merged_opencode)` and Line 10: `tomllib.loads(updated_toml)`
- **Trigger condition:** Specification requires raising `ConfigWizardError` when `merged_opencode` is not JSON-serializable or `updated_toml` is invalid TOML, but the code raises `TypeError/ValueError` or `TOMLDecodeError` respectively, not `ConfigWizardError`.
- **Validator trigger:** Non-JSON-serializable `merged_opencode` raises `TypeError` instead of `ConfigWizardError`; invalid TOML update string raises `TOMLDecodeError` instead of `ConfigWizardError`.
- **Probe stdout:** Test 1 (JSON serialization): CONFIRMED - `TypeError` raised instead of `ConfigWizardError`: Object of type bytes is not JSON serializable Test 2 (TOML validation): CONFIRMED - `TOMLDecodeError` raised instead of `ConfigWizardError`: Expected '=' after a key in a key/value pair (at line 1, column 6) CONFIRMED

► SPEC sidecar (完整)

```
{
  "signature": "validate_generated_state(config: LLMConfigInput, merged_opencode: dict, updated_toml: str, *,
opencode_secret_path: Path) -> None",
  "pre_condition": "config is an LLMConfigInput whose provider_id, provider_name, api_style, base_url, model_id, and api_key
fields are all non-empty strings. merged_opencode is a dict representing a candidate OpenCode provider configuration. updated_toml
is a string representing candidate TOML content. opencode_secret_path is a Path identifying the expected location of the provider
secret key file.",
  "post_condition": "Raises ConfigWizardError when updated_toml is not syntactically valid TOML, when merged_opencode is not JSON-
serializable, when merged_opencode lacks a 'provider' key whose value is a dict containing an entry keyed by config.provider_id,
when that provider entry is not a dict, when its 'npm' field does not equal the adapter name corresponding to config.api_style,
when its 'options' field is missing, not a dict, or has an 'apiKey' value that does not equal the string '{file:
<opencode_secret_path>}', when its 'models' field is missing, not a dict, or does not contain config.model_id, or when any field
of config required for configuration is empty, missing, or malformed. Returns with no effect when all generated outputs pass their
respective format and structural validation."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "validate_input",
      "signature": "validate_input(config: LLMConfigInput) -> None",
      "pre_condition": "config is an LLMConfigInput.",
      "post_condition": "Raises ConfigWizardError when any required field of config is empty, malformed, or fails its format
constraint. Returns with no effect when config is valid."
    },
    {
      "name": "adapter_for_api_style",
      "signature": "adapter_for_api_style(api_style: ApiStyle) -> str",
      "pre_condition": "api_style is a recognized API style value.",
      "post_condition": "Returns the npm package name string corresponding to the given API style. Raises when the API style is
not recognized."
    }
  ]
}
```



```
]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/validate_generated_state.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Raises ConfigWizardError when updated_toml is not syntactically valid TOML, when merged_opencode is not JSON-serializable, when merged_opencode lacks a 'provider' key whose value is a dict containing an entry keyed by config.provider_id, when that provider entry is not a dict, when its 'npm' field does not equal the adapter name corresponding to config.api_style, when its 'options' field is missing, not a dict, or has an 'apiKey' value that does not equal the string '{file: <opencode_secret_path>}', when its 'models' field is missing, not a dict, or does not contain config.model_id, or when any field of config required for configuration is empty, missing, or malformed. Returns with no effect when all generated outputs pass their respective format and structural validation.",
    "actual_behavior": "After execution, exactly one of the following outcomes occurs:\n\n1. A `ConfigWizardError` is raised because:\n  - `validate_input(config)` failed (invalid provider configuration), or\n  - the generated OpenCode configuration is missing the required provider `config.provider_id`, or\n  - the provider entry is not a dict, or\n  - the adapter (`npm`) does not match `adapter_for_api_style(config.api_style)`, or\n  - the options dict is missing or does not contain `apiKey` equal to `{file: opencode_secret_path}`, or\n  - the models dict is missing or does not contain `config.model_id`.\n\n2. A `TypeError` or `ValueError` is raised from `json.dumps(merged_opencode)` because `merged_opencode` is not JSON-serializable.\n\n3. A `tomllib.TOMLDecodeError` (or similar parsing exception) is raised from `tomllib.loads(updated_toml)` because `updated_toml` is not valid TOML.\n\n4. An exception is propagated from `adapter_for_api_style(config.api_style)` if the API style is unrecognised, causing that call to raise.\n\n5. The function returns `None` (normal termination). In this case all of the following hold:\n  - `validate_input(config)` succeeded, implying `config` meets all `LLMConfigInput` validity constraints.\n  - `json.dumps(merged_opencode)` succeeded, i.e., `merged_opencode` is JSON-serializable.\n  - `tomllib.loads(updated_toml)` succeeded, i.e., `updated_toml` is valid TOML.\n  - `merged_opencode.get('provider')` is a dict and `config.provider_id` is a key in it. Let `entry = merged_opencode['provider'][config.provider_id]`. \n  - `entry` is a dict.\n  - `entry.get('npm') == adapter_for_api_style(config.api_style)`.\n  - `entry.get('options')` is a dict and `options.get('apiKey') == f'{file: opencode_secret_path}'`.\n  - `entry.get('models')` is a dict and `config.model_id` is a key in it.",
    "code_evidence": "Line 9: json.dumps(merged_opencode) and Line 10: tomllib.loads(updated_toml)",
    "trigger_condition": "Specification requires raising ConfigWizardError when merged_opencode is not JSON-serializable or updated_toml is invalid TOML, but the code raises TypeError/ValueError or TOMLDecodeError respectively, not ConfigWizardError."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--validate_generated_state",
  "source_file": "src/configure_llm-py/validate_generated_state.py",
  "function_name": "validate_generated_state",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--validate_generated_state.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--validate_generated_state.md",
  "probe_stdout": "Test 1 (JSON serialization): CONFIRMED - TypeError raised instead of ConfigWizardError: Object of type bytes is not JSON serializable\nTest 2 (TOML validation): CONFIRMED - TOMLDecodeError raised instead of ConfigWizardError: Expected '=' after a key in a key/value pair (at line 1, column 6)\n\nCONFIRMED",
  "trigger_summary": "Non-JSON-serializable merged_opencode raises TypeError instead of ConfigWizardError; invalid TOML update string raises TOMLDecodeError instead of ConfigWizardError."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:59:23.477029Z	3408
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:00:07.276519Z	4241

事件 1 — [llm_4b29e05cab5f40d08de879666b73e591](#)

► parsed_json（完整）

```
{
  "post_condition": "After execution, exactly one of the following outcomes occurs:\n\n1. A `ConfigWizardError` is raised because:\n  - `validate_input(config)` failed (invalid provider configuration), or\n  - the generated OpenCode configuration is missing the required provider `config.provider_id`, or\n  - the provider entry is not a dict, or\n  - the adapter (`npm`) does not match `adapter_for_api_style(config.api_style)`, or\n  - the options dict is missing or does not contain `apiKey` equal to `{file: opencode_secret_path}`, or\n  - the models dict is missing or does not contain `config.model_id`.\n\n2. A `TypeError` or `ValueError` is raised from `json.dumps(merged_opencode)` because `merged_opencode` is not JSON-serializable.\n\n3. A `tomllib.TOMLDecodeError` (or similar parsing exception) is raised from `tomllib.loads(updated_toml)` because `updated_toml` is
```



```
not valid TOML.\n\n4. An exception is propagated from `adapter_for_api_style(config.api_style)` if the API style is unrecognised, causing that call to raise.\n\n5. The function returns `None` (normal termination). In this case all of the following hold:\n - `validate_input(config)` succeeded, implying `config` meets all `LLMConfigInput` validity constraints.\n - `json.dumps(merged_opencode)` succeeded, i.e., `merged_opencode` is JSON-serializable.\n - `tomllib.loads(updated_toml)` succeeded, i.e., `updated_toml` is valid TOML.\n - `merged_opencode.get('provider')` is a dict and `config.provider_id` is a key in it. Let `entry = merged_opencode['provider'][config.provider_id]`. \n - `entry` is a dict.\n - `entry.get('npm') == adapter_for_api_style(config.api_style)`.\n - `entry.get('options')` is a dict and `options.get('apiKey') == f'{{file: {opencode_secret_path}}}'`.\n - `entry.get('models')` is a dict and `config.model_id` is a key in it."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4b29e05cab5f40d08de879666b73e591_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4b29e05cab5f40d08de879666b73e591_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4b29e05cab5f40d08de879666b73e591_response.txt)

事件 2 — llm_a7cb97340f414912a9358f0b21c6181b

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "merged_opencode = {'provider': {'p1': {}}}, 'non_serializable': object()); config is any valid LLMConfigInput; updated_toml = 'key = \\\"value\\\"'.",
  "offending_statements": "Line 9: json.dumps(merged_opencode) and Line 10: tomllib.loads(updated_toml)",
  "reason": "Specification requires raising ConfigWizardError when merged_opencode is not JSON-serializable or updated_toml is invalid TOML, but the code raises TypeError/ValueError or TOMLDecodeError respectively, not ConfigWizardError."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_a7cb97340f414912a9358f0b21c6181b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_a7cb97340f414912a9358f0b21c6181b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_a7cb97340f414912a9358f0b21c6181b_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-033 — src--configure_llm-py--validate_llm_setting

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Raises ConfigWizardError when key is not a recognized LLM configuration setting key. When key is recognized, raises ConfigWizardError when value fails the validation constraint associated with that key: a constraint requiring a non-empty trimmed string (value is empty or consists only of whitespace after stripping); a constraint requiring a syntactically valid absolute URL with http or https scheme (value does not satisfy it); a constraint requiring membership in a fixed set of recognized identifiers (value is absent from that set); or a constraint delegating to an API style adapter (the adapter rejects value). Returns with no effect when key is recognized and value satisfies all constraints associated with that key.
- **Actual behavior**: The function validate_llm_setting(key, value) either returns None or raises an exception. If key is not in _LLM_TOML_KEYS, it raises ConfigWizardError("Unsupported LLM setting: {key}"). If key == 'provider' and value.strip() is empty, it raises ConfigWizardError("Provider ID must not be empty."). If key == 'base_url', it calls validate_base_url(value.strip()) which may raise an exception if value.strip() is not a syntactically valid absolute URL with http or https scheme; otherwise it does not raise. If key == 'backend' and value not in _BACKENDS, it raises ConfigWizardError with message listing supported backends. If key == 'api_style', it calls adapter_for_api_style(value) which may raise an exception if value is not a recognized API style; otherwise it does not raise. Otherwise, if none of the above conditions cause an exception, the function returns None without side effects.
- **Code evidence**: Line 7: if key == "base_url": Line 8: validate_base_url(value.strip()) Line 14: elif key == "api_style": Line 15: adapter_for_api_style(value)
- **Trigger condition**: For base_url and api_style, the specification requires raising ConfigWizardError when the value fails validation. However, the code delegates to validate_base_url and adapter_for_api_style, which may raise exceptions that are not ConfigWizardError (e.g., ValueError). Thus, the code's behavior does not conform to the required exception type.
- **Validator trigger**: The 'name' and 'effort' recognized keys have no validation — validate_llm_setting('name', "") and validate_llm_setting('effort', 'invalid') return None instead of raising ConfigWizardError as the spec requires.
- **Probe stdout**: CONFIRMED — found 6 spec violation(s):

- 'name' key with '': returned None — spec requires ConfigWizardError for empty/whitespace trimmed value
 - 'name' key with ' ': returned None — spec requires ConfigWizardError for empty/whitespace trimmed value
 - 'effort' key with 'invalid': returned None — spec requires ConfigWizardError for value outside ('', 'low', 'medium', 'high')
 - 'effort' key with 'x': returned None — spec requires ConfigWizardError for value outside ('', 'low', 'medium', 'high')
 - 'effort' key with 'super_high': returned None — spec requires ConfigWizardError for value outside ('', 'low', 'medium', 'high')
 - 'effort' key with 'unknown_effort': returned None — spec requires ConfigWizardError for value outside ('', 'low', 'medium', 'high')
- (base_url and api_style correctly raise ConfigWizardError; the bug is that 'name' and 'effort' keys have no validation against their spec constraints)

► SPEC sidecar (完整)

```
{
  "signature": "validate_llm_setting(key: str, value: str) -> None",
  "pre_condition": "key is a non-empty string; value is a string.",
  "post_condition": "Raises ConfigWizardError when key is not a recognized LLM configuration setting key. When key is recognized,
raises ConfigWizardError when value fails the validation constraint associated with that key: a constraint requiring a non-empty
trimmed string (value is empty or consists only of whitespace after stripping); a constraint requiring a syntactically valid
absolute URL with http or https scheme (value does not satisfy it); a constraint requiring membership in a fixed set of recognized
identifiers (value is absent from that set); or a constraint delegating to an API style adapter (the adapter rejects value).
Returns with no effect when key is recognized and value satisfies all constraints associated with that key."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "validate_base_url",
      "signature": "validate_base_url(url: str) -> None",
      "pre_condition": "url is a non-empty, whitespace-stripped string.",
      "post_condition": "Raises an exception when url is not a syntactically valid absolute URL whose scheme is http or https.
Returns with no effect when url is a valid absolute URL with http or https scheme."
    },
    {
      "name": "adapter_for_api_style",
      "signature": "adapter_for_api_style(style: str) -> None",
      "pre_condition": "style is a string.",
      "post_condition": "Raises an exception when style is not a recognized API style identifier. Returns with no effect when
style is a recognized API style identifier."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/validate_llm_setting.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Raises ConfigWizardError when key is not a recognized LLM configuration setting key. When key is recognized,
raises ConfigWizardError when value fails the validation constraint associated with that key: a constraint requiring a non-empty
trimmed string (value is empty or consists only of whitespace after stripping); a constraint requiring a syntactically valid
absolute URL with http or https scheme (value does not satisfy it); a constraint requiring membership in a fixed set of recognized
identifiers (value is absent from that set); or a constraint delegating to an API style adapter (the adapter rejects value).
Returns with no effect when key is recognized and value satisfies all constraints associated with that key.",
    "actual_behavior": "The function validate_llm_setting(key, value) either returns None or raises an exception. If key is not in
_LLM_TOML_KEYS, it raises ConfigWizardError(\"Unsupported LLM setting: {key}\"). If key == 'provider' and value.strip() is empty,
it raises ConfigWizardError(\"Provider ID must not be empty.\"). If key == 'base_url', it calls validate_base_url(value.strip())
which may raise an exception if value.strip() is not a syntactically valid absolute URL with http or https scheme; otherwise it
does not raise. If key == 'backend' and value not in _BACKENDS, it raises ConfigWizardError with message listing supported
backends. If key == 'api_style', it calls adapter_for_api_style(value) which may raise an exception if value is not a recognized
API style; otherwise it does not raise. Otherwise, if none of the above conditions cause an exception, the function returns None
without side effects.",
    "code_evidence": "Line 7:         if key == \"base_url\":\nLine 8:             validate_base_url(value.strip())\nLine 14:         elif key
== \"api_style\":\nLine 15:             adapter_for_api_style(value)",
    "trigger_condition": "For base_url and api_style, the specification requires raising ConfigWizardError when the value fails
validation. However, the code delegates to validate_base_url and adapter_for_api_style, which may raise exceptions that are not
ConfigWizardError (e.g., ValueError). Thus, the code's behavior does not conform to the required exception type."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--configure_llm-py--validate_llm_setting",
  "source_file": "src/configure_llm.py",
  "function_name": "validate_llm_setting",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--validate_llm_setting.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--validate_llm_setting.md",
  "probe_stdout": "CONFIRMED - found 6 spec violation(s):\n  - 'name' key with '': returned None - spec requires ConfigWizardError for empty/whitespace trimmed value\n  - 'name' key with ' ': returned None - spec requires ConfigWizardError for empty/whitespace trimmed value\n  - 'effort' key with 'invalid': returned None - spec requires ConfigWizardError for value outside ('', 'low', 'medium', 'high')\n  - 'effort' key with 'x': returned None - spec requires ConfigWizardError for value outside ('', 'low', 'medium', 'high')\n  - 'effort' key with 'super_high': returned None - spec requires ConfigWizardError for value outside ('', 'low', 'medium', 'high')\n  - 'effort' key with 'unknown_effort': returned None - spec requires ConfigWizardError for value outside ('', 'low', 'medium', 'high')\n(base_url and api_style correctly raise ConfigWizardError; the bug is that 'name' and 'effort' keys have no validation against their spec constraints)",
  "trigger_summary": "The 'name' and 'effort' recognized keys have no validation - validate_llm_setting('name', '') and validate_llm_setting('effort', 'invalid') return None instead of raising ConfigWizardError as the spec requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:22:40.450123Z	2811
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:23:12.537778Z	3910

事件 1 — [llm_c2dbc8e24dd44e828275b3479a23288f](#)

► parsed_json（完整）

```
{
  "post_condition": "The function validate_llm_setting(key, value) either returns None or raises an exception. If key is not in _LLM_TOML_KEYS, it raises ConfigWizardError(\"Unsupported LLM setting: {key}\"). If key == 'provider' and value.strip() is empty, it raises ConfigWizardError(\"Provider ID must not be empty.\"). If key == 'base_url', it calls validate_base_url(value.strip()) which may raise an exception if value.strip() is not a syntactically valid absolute URL with http or https scheme; otherwise it does not raise. If key == 'backend' and value not in _BACKENDS, it raises ConfigWizardError with message listing supported backends. If key == 'api_style', it calls adapter_for_api_style(value) which may raise an exception if value is not a recognized API style; otherwise it does not raise. Otherwise, if none of the above conditions cause an exception, the function returns None without side effects."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c2dbc8e24dd44e828275b3479a23288f_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c2dbc8e24dd44e828275b3479a23288f_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c2dbc8e24dd44e828275b3479a23288f_response.txt)

事件 2 — [llm_00550c05051a41dfa4de1d3f0fdb0ca6](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "key='base_url', value='invalid'",
  "offending_statements": "Line 7:     if key == \"base_url\":\nLine 8:         validate_base_url(value.strip())\nLine 14: elif key == \"api_style\":\nLine 15:     adapter_for_api_style(value)",
  "reason": "For base_url and api_style, the specification requires raising ConfigWizardError when the value fails validation. However, the code delegates to validate_base_url and adapter_for_api_style, which may raise exceptions that are not ConfigWizardError (e.g., ValueError). Thus, the code's behavior does not conform to the required exception type."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_00550c05051a41dfa4de1d3f0fdb0ca6_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_00550c05051a41dfa4de1d3f0fdb0ca6_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_00550c05051a41dfa4de1d3f0fdb0ca6_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#); attempts [1](#)
- Phase / 模块： Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Prints a warning to stdout when one or more keys in updates correspond to environment variables whose values are present in the .env file at env_path, indicating that those .env values would override the TOML updates. Returns True when at least one warning was printed; returns False when no overrides were found, or the .env file is absent or unreadable.
- **Actual behavior**: After normal execution (i.e., no exceptions), the function returns a boolean indicating whether any legacy environment variable overrides in the .env file shadow the supplied TOML updates. Let `content = _read_text_if_exists(env_path)` and `(_, overrides) = remove_legacy_llm_env_overrides(content)`. Define `shadowing = { name in overrides | _TOML_KEY_BY_ENV_KEY[name] in updates }`. If `shadowing` is empty, the function returns `False` and produces no output. Otherwise, it prints exactly the warning text: 'Warning: these project .env variables still override this TOML update:' followed by a line with the comma-separated list of `shadowing` names, then 'The set command does not modify .env. Remove those lines, or run the', 'interactive wizard to migrate legacy LLM overrides.', and returns `True`. The `env_path` file, the `updates` dictionary, and any global state are unchanged except for the standard output stream.
- **Code evidence**: Line 6: `_updated_env, legacy_overrides = remove_legacy_llm_env_overrides(` Line 7: `_read_text_if_exists(env_path)` Line 8: `)`
- **Trigger condition**: The specification explicitly states that the function should return False when the .env file is absent or unreadable. The code provides no error handling for these scenarios; an unreadable file will cause an exception to be raised, violating the required behavior.
- **Validator trigger**: Pass an unreadable .env file to `warn_dotenv_overrides_for_updates`; spec states it should return False but a `PermissionError` is raised instead.
- **Probe stdout**: CONFIRMED — `PermissionError` raised for unreadable .env file; spec requires returning False

► SPEC sidecar (完整)

```
{
  "signature": "warn_dotenv_overrides_for_updates(env_path: Path, updates: dict[str, str]) -> bool",
  "pre_condition": ".env_path is a Path to a .env file that may or may not exist; updates is a dict mapping LLM section field paths to new values.",
  "post_condition": "Prints a warning to stdout when one or more keys in updates correspond to environment variables whose values are present in the .env file at env_path, indicating that those .env values would override the TOML updates. Returns True when at least one warning was printed; returns False when no overrides were found, or the .env file is absent or unreadable."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/warn_dotenv_overrides_for_updates.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Prints a warning to stdout when one or more keys in updates correspond to environment variables whose values are present in the .env file at env_path, indicating that those .env values would override the TOML updates. Returns True when at least one warning was printed; returns False when no overrides were found, or the .env file is absent or unreadable.",
    "actual_behavior": "After normal execution (i.e., no exceptions), the function returns a boolean indicating whether any legacy environment variable overrides in the .env file shadow the supplied TOML updates. Let `content = _read_text_if_exists(env_path)` and `(_, overrides) = remove_legacy_llm_env_overrides(content)`. Define `shadowing = { name in overrides | _TOML_KEY_BY_ENV_KEY[name] in updates }`. If `shadowing` is empty, the function returns `False` and produces no output. Otherwise, it prints exactly the warning text: 'Warning: these project .env variables still override this TOML update:' followed by a line with the comma-separated list of `shadowing` names, then 'The set command does not modify .env. Remove those lines, or run the', 'interactive wizard to migrate legacy LLM overrides.', and returns `True`. The `env_path` file, the `updates` dictionary, and any global state are unchanged except for the standard output stream.",
    "code_evidence": "Line 6:      _updated_env, legacy_overrides = remove_legacy_llm_env_overrides(\nLine 7:      _read_text_if_exists(env_path)\nLine 8:      )",
    "trigger_condition": "The specification explicitly states that the function should return False when the .env file is absent or unreadable. The code provides no error handling for these scenarios; an unreadable file will cause an exception to be raised, violating the required behavior."
  }
}
```

```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--warn_dotenv_overrides_for_updates",
  "source_file": "src/configure_llm.py",
  "function_name": "warn_dotenv_overrides_for_updates",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--warn_dotenv_overrides_for_updates.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--warn_dotenv_overrides_for_updates.md",
  "probe_stdout": "CONFIRMED - PermissionError raised for unreadable .env file; spec requires returning False",
  "trigger_summary": "Pass an unreadable .env file to warn_dotenv_overrides_for_updates; spec states it should return False but a PermissionError is raised instead."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:52:25.130667Z	2050
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:52:47.714676Z	3505

事件 1 — [llm_bf43f9234a014563ab3b2601af5e874b](#)

► parsed_json（完整）

```
{
  "post_condition": "After normal execution (i.e., no exceptions), the function returns a boolean indicating whether any legacy environment variable overrides in the .env file shadow the supplied TOML updates. Let `content = _read_text_if_exists(env_path)` and `(_, overrides) = remove_legacy_llm_env_overrides(content)`. Define `shadowing = { name in overrides | _TOML_KEY_BY_ENV_KEY[name] in updates }`. If `shadowing` is empty, the function returns `False` and produces no output. Otherwise, it prints exactly the warning text: 'Warning: these project .env variables still override this TOML update:' followed by a line with the comma-separated list of `shadowing` names, then 'The set command does not modify .env. Remove those lines, or run the', 'interactive wizard to migrate legacy LLM overrides.', and returns `True`. The `env_path` file, the `updates` dictionary, and any global state are unchanged except for the standard output stream."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_bf43f9234a014563ab3b2601af5e874b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_bf43f9234a014563ab3b2601af5e874b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_bf43f9234a014563ab3b2601af5e874b_response.txt)

事件 2 — [llm_6b777ed69a924fe280b9bc02fd523196](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input: env_path = Path('/protected/.env') where the file exists but the user lacks read permissions, and updates = {'ANTHROPIC_API_KEY': 'sk-ant-123'}. The code calls _read_text_if_exists(env_path) which attempts to open the unreadable file, raising PermissionError. The function does not catch the exception, so it propagates instead of returning False as required by the specification.",
  "offending_statements": "Line 6:     _updated_env, legacy_overrides = remove_legacy_llm_env_overrides(\nLine 7: _read_text_if_exists(env_path)\nLine 8:     )",
  "reason": "The specification explicitly states that the function should return False when the .env file is absent or unreadable. The code provides no error handling for these scenarios; an unreadable file will cause an exception to be raised, violating the required behavior."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_6b777ed69a924fe280b9bc02fd523196_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_6b777ed69a924fe280b9bc02fd523196_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_6b777ed69a924fe280b9bc02fd523196_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论: 契约不确定
- 分类依据: 沿用旧 `fm_agent/bug_list.md` 的逐条审计结论: 沿用旧清单的逐条审计结论。
- **Validator:** `confirmed`; attempts 1
- **Phase / 模块:** Phase 1 — Initialization & Configuration / `git_utils`; 原源码 `src/git.py`
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Yields the absolute path `wt` to a directory whose contents at yield time are a faithful snapshot of the `proj_dir` filesystem tree at the moment `frozen_worktree` was entered. Subsequent modifications to `proj_dir` are not reflected in the snapshot at `wt`. The snapshot construction never modifies the original `proj_dir` tree, index, or git state. If `proj_dir` is a git repository with at least one commit, the snapshot is a detached git worktree comprising HEAD, all uncommitted tracked edits, and all untracked files present in `proj_dir` at entry time, with entries matching names in `exclude` omitted from the git commit. If `proj_dir` is not a git repository with a valid HEAD, the snapshot is a recursive directory copy of `proj_dir`, with entries matching names in `exclude` omitted. When `copy_excluded` is true, each directory named in `exclude` that exists in `proj_dir` is recursively copied into the snapshot at `wt` after the git worktree or directory copy is constructed. The snapshot at `wt` persists after the context manager exits and is not automatically removed.
- **Actual behavior:** On normal execution, the generator yields `wt`, a directory at `<temp_dir>/snapshot` where `<temp_dir> = tempfile.mkdtemp(prefix="fm_agent_wt-<repo_name>_")` and `<repo_name>` is derived from `proj_dir`. If `proj_dir` was a git repo with a HEAD commit (`is_git` true), then `wt` is a detached worktree for a new commit that captures the working tree (tracked edits + untracked files) at the time `git add -A` ran, with directories in `exclude` removed from the commit via `git rm --cached`. The original git state (HEAD, index, working tree) is unchanged. If `copy_excluded` is true, each excluded directory that existed in `proj_dir` and is absent from `wt` is recursively copied into `wt` as an untracked directory. If `proj_dir` was not a valid git repo with HEAD (`is_git` false), `wt` is a plain copy of `proj_dir` (directories in `exclude` skipped via `shutil.ignore_patterns`), and if `copy_excluded`, those directories are subsequently copied into `wt`. `<temp_dir>` persists after the yield (no automatic deletion). `proj_dir` is unmodified. Messages containing `wt` and cleanup instructions are printed to stdout. On exceptional execution (exception before the yield), `proj_dir` is unchanged, `<temp_dir>` exists but may contain partial artifacts, and `wt` may not exist or be incomplete.

Formally: Let `P = os.path.abspath(proj_dir)`, `R = os.path.basename(P.rstrip(os.sep))` or "repo", `B = tempfile.mkdtemp(prefix="fm_agent_wt_" + R + "_")`, `W = os.path.join(B, "snapshot")`, `G = (subprocess.run(["git", "-C", P, "rev-parse", "--verify", "HEAD"], check=True, capture_output=True, text=True) did not raise CalledProcessError)`. Then the outcome satisfies: (Normal yield `W`) (Exception raised no yield). If normal yield: `G (I = os.path.join(B, "index") that is a valid git index tree = output of _git("write-tree", env=env) snap = output of _git("commit-tree", tree, "-p", "HEAD", "-m", "fm_agent snapshot", env=env) _git("worktree", "add", "--detach", W, snap) succeeded the git state of P (HEAD, index, working tree) is identical to entry state name exclude, let S = os.path.join(P, name), D = os.path.join(W, name): (copy_excluded os.path.isdir(S) os.path.exists(D) D is result of shutil.copytree(S, D, symlinks=True) after worktree creation)) G (W is the result of shutil.copytree(P, W, ignore=shutil.ignore_patterns(*exclude), symlinks=True) name exclude: (copy_excluded os.path.isdir(os.path.join(P, name)) os.path.exists(os.path.join(W, name)) shutil.copytree(os.path.join(P, name), os.path.join(W, name), symlinks=True) executed after the initial copytree)) B exists after yield, and stdout contains W. If exception raised: P and all its contents are identical to entry state B exists (W may not exist or be incomplete).`

- **Code evidence:** Line 47: `_git("rm", "-r", "--cached", "--quiet", "--ignore-unmatch", "--",` Line 48: `*exclude, env=env)`
- **Trigger condition:** The code uses 'git rm --cached' with only the top-level exclude names, so nested directories/files with the same name remain in the commit. The specification says 'entries matching names in exclude omitted', which requires omission of all such entries at any depth.
- **Validator trigger:** `frozen_worktree()` uses `git rm --cached` with only top-level exclude names, so nested directories with the same name (e.g. `testdata/fm_agent/`) remain in the commit and leak into the snapshot.
- **Probe stdout:** [Pipeline] Snapshot created at: `/tmp/fm_agent_wt_repo_54som_02/snapshot` [Pipeline] Snapshot is kept after the run. Remove with: `git -C /tmp/probe_fwtf_15ct/repo worktree remove --force /tmp/fm_agent_wt_repo_54som_02/snapshot CONFIRMED` — nested `fm_agent/` present in snapshot at `/tmp/fm_agent_wt_repo_54som_02/snapshot/testdata/fm_agent` (contains: `['nested.txt']`) — spec requires exclusion at all depths

► SPEC sidecar (完整)

```
{
  "signature": "frozen_worktree(proj_dir, exclude=('fm_agent',), copy_excluded=True) -> yields str",
  "pre_condition": "proj_dir is a string path to an existing directory. exclude is an iterable of relative directory names. copy_excluded is a boolean.",
  "post_condition": "Yields the absolute path wt to a directory whose contents at yield time are a faithful snapshot of the proj_dir filesystem tree at the moment frozen_worktree was entered. Subsequent modifications to proj_dir are not reflected in the"
```



```

snapshot at wt. The snapshot construction never modifies the original proj_dir tree, index, or git state. If proj_dir is a git repository with at least one commit, the snapshot is a detached git worktree comprising HEAD, all uncommitted tracked edits, and all untracked files present in proj_dir at entry time, with entries matching names in exclude omitted from the git commit. If proj_dir is not a git repository with a valid HEAD, the snapshot is a recursive directory copy of proj_dir, with entries matching names in exclude omitted. When copy_excluded is true, each directory named in exclude that exists in proj_dir is recursively copied into the snapshot at wt after the git worktree or directory copy is constructed. The snapshot at wt persists after the context manager exits and is not automatically removed."
}

```

► INFO / callees sidecar (完整)

```

{
  "callees": [
    {
      "name": "os.path.abspath",
      "signature": "os.path.abspath(path: str) -> str",
      "pre_condition": "path is a string.",
      "post_condition": "Returns a normalized absolute path equivalent to path without accessing the filesystem."
    },
    {
      "name": "tempfile.mkdtemp",
      "signature": "tempfile.mkdtemp(prefix: str | None) -> str",
      "pre_condition": "The parent temporary directory exists and is writable.",
      "post_condition": "Creates a unique directory whose name begins with prefix when prefix is non-None. Returns the absolute path to the created directory."
    },
    {
      "name": "subprocess.run",
      "signature": "subprocess.run(args: list[str], *, check: bool, capture_output: bool, text: bool, env: dict | None) -> CompletedProcess",
      "pre_condition": "args is a non-empty list of strings naming an executable and its arguments. When env is specified, it maps environment variable names to values.",
      "post_condition": "Executes the command specified by args in a child process. When capture_output is true, captures stdout and stderr into the result's stdout and stderr attributes. When text is true, captured output is decoded as strings. When check is true, raises CalledProcessError if the child exits with a non-zero status. Returns a CompletedProcess whose stdout attribute is the stripped captured standard output."
    },
    {
      "name": "shutil.copytree",
      "signature": "shutil.copytree(src: str, dst: str, *, ignore: callable | None, symlinks: bool) -> str",
      "pre_condition": "src is an existing directory. dst does not exist.",
      "post_condition": "Recursively copies the directory tree rooted at src to dst. When symlinks is true, symbolic links are preserved as links. When ignore is provided, entries matched by the ignore callable are excluded from the copy. Returns dst."
    },
    {
      "name": "os.path.isdir",
      "signature": "os.path.isdir(path: str) -> bool",
      "pre_condition": "path is a string.",
      "post_condition": "Returns True if path exists and refers to a directory; False otherwise."
    },
    {
      "name": "os.path.exists",
      "signature": "os.path.exists(path: str) -> bool",
      "pre_condition": "path is a string.",
      "post_condition": "Returns True if path exists in the filesystem; False otherwise, including for broken symbolic links."
    }
  ]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/git-py/frozen_worktree.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Yields the absolute path wt to a directory whose contents at yield time are a faithful snapshot of the proj_dir filesystem tree at the moment frozen_worktree was entered. Subsequent modifications to proj_dir are not reflected in the snapshot at wt. The snapshot construction never modifies the original proj_dir tree, index, or git state. If proj_dir is a git repository with at least one commit, the snapshot is a detached git worktree comprising HEAD, all uncommitted tracked edits, and all untracked files present in proj_dir at entry time, with entries matching names in exclude omitted from the git commit. If proj_dir is not a git repository with a valid HEAD, the snapshot is a recursive directory copy of proj_dir, with entries matching names in exclude omitted. When copy_excluded is true, each directory named in exclude that exists in proj_dir is recursively copied into the snapshot at wt after the git worktree or directory copy is constructed. The snapshot at wt persists after the context manager exits and is not automatically removed.",
    "actual_behavior": "On normal execution, the generator yields `wt`, a directory at `/snapshot` where `` = `tempfile.mkdtemp(prefix='fm_agent_wt_<repo_name>\\')` and `` is derived from `proj_dir`. If `proj_dir` was a git repo with a HEAD commit (`is_git` true), then `wt` is a detached worktree for a new commit that captures the working tree (tracked edits + untracked files) at the time `git add -A` ran, with directories in `exclude` removed from the commit via `git rm --cached`. The original git state (`HEAD`, index, working tree) is unchanged. If `copy_excluded` is true, each excluded directory that existed in `proj_dir` and is absent from `wt` is recursively copied into `wt` as an untracked directory. If `proj_dir` was not a valid git repo with HEAD (`is_git` false), `wt` is a plain copy of `proj_dir` (directories in `exclude` skipped via `shutil.ignore_patterns`), and if `copy_excluded`, those directories are subsequently copied into `wt`. `` persists"
  }
}

```

```
after the yield (no automatic deletion). `proj_dir` is unmodified. Messages containing `wt` and cleanup instructions are printed to stdout. On exceptional execution (exception before the yield), `proj_dir` is unchanged, `` exists but may contain partial artifacts, and `wt` may not exist or be incomplete.\n\nFormally:\nLet P = os.path.abspath(proj_dir), R = os.path.basename(P.rstrip(os.sep)) or `repo`, B = tempfile.mkdtemp(prefix=`fm_agent_wt` + R + `_\`), W = os.path.join(B, `snapshot`), G = (subprocess.run([`git`, `--C`, P, `rev-parse`, `--verify`, `HEAD`], check=True, capture_output=True, text=True) did not raise CalledProcessError). Then the outcome satisfies:\n (Normal yield W) (Exception raised no yield).\n If normal yield:\n G \n ( I = os.path.join(B, `index`) that is a valid git index \n tree = output of _git(`write-tree`, env=env) \n snap = output of _git(`commit-tree`, tree, `p`, `HEAD`, `m`, `fm_agent_snapshot`, env=env) \n _git(`worktree`, `add`, `--detach`, W, snap) succeeded \n the git state of P (HEAD, index, working tree) is identical to entry state \n name exclude, let S = os.path.join(P, name), D = os.path.join(W, name):\n (copy_excluded os.path.isdir(S) os.path.exists(D) D is result of shutil.copytree(S, D, symlinks=True) after worktree creation) \n G \n ( W is the result of shutil.copytree(P, W, ignore=shutil.ignore_patterns(*exclude), symlinks=True) \n name exclude: (copy_excluded os.path.isdir(os.path.join(P, name)) os.path.exists(os.path.join(W, name)) \n shutil.copytree(os.path.join(P, name), os.path.join(W, name), symlinks=True) executed after the initial copytree) \n B exists after yield, and stdout contains W.\n If exception raised:\n P and all its contents are identical to entry state B exists (W may not exist or be incomplete).",\n "code_evidence": "Line 47: _git(`rm`, `r`, `--cached`, `--quiet`, `--ignore-unmatch`, `--`, \nLine 48: *exclude, env=env)",\n "trigger_condition": "The code uses 'git rm --cached' with only the top-level exclude names, so nested directories/files with the same name remain in the commit. The specification says 'entries matching names in exclude omitted', which requires omission of all such entries at any depth." }\n}
```

► Bug Validator result (完整)

```
{\n  "id": "src--git-py--frozen_worktree",\n  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/git-py/frozen_worktree.py",\n  "function_name": "frozen_worktree",\n  "confirmation_status": "confirmed",\n  "attempts": 1,\n  "probe_script": "fm_agent/bug_validation/probe_src--git-py--frozen_worktree.py",\n  "detail_file": "fm_agent/bug_validation/src--git-py--frozen_worktree.md",\n  "probe_stdout": "[Pipeline] Snapshot created at: /tmp/fm_agent_wt_repo_54som_02/snapshot\n[Pipeline] Snapshot is kept after the run. Remove with: git -C /tmp/probe_fwt_frax15ct/repo worktree remove --force /tmp/fm_agent_wt_repo_54som_02/snapshot\nCONFIRMED - nested fm_agent/ present in snapshot at /tmp/fm_agent_wt_repo_54som_02/snapshot/testdata/fm_agent (contains: ['nested.txt']) - spec requires exclusion at all depths",\n  "trigger_summary": "frozen_worktree() uses git rm --cached with only top-level exclude names, so nested directories with the same name (e.g. testdata/fm_agent/) remain in the commit and leak into the snapshot." \n}
```

► Reasoner 流程 (3 条事件)

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-27T15:37:22.417794Z	8614
2	generate_block_post_condition	success	2	0/1	2026-07-27T15:39:02.286524Z	5559
3	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:39:41.039217Z	11504

事件 1 — [llm_42242808c65246bf948fa3f52184c058](#)

- parse_error: LLM response is not valid JSON: Invalid \\escape: line 2 column 66 (char 67)
- 载荷: [system_prompt](#) · [user_prompt](#) · [assistant_output](#)

事件 2 — [llm_01aca045e7fa434eab75c67be53662bf](#)

► parsed_json (完整)

```
{\n  "post_condition": "On normal execution, the generator yields `wt`, a directory at `/snapshot` where `` = tempfile.mkdtemp(prefix=`fm_agent_wt_<repo_name>_`) and `` is derived from `proj_dir`. If `proj_dir` was a git repo with a HEAD commit (`is_git` true), then `wt` is a detached worktree for a new commit that captures the working tree (tracked edits + untracked files) at the time `git add -A` ran, with directories in `exclude` removed from the commit via `git rm --cached`. The original git state (`HEAD`, index, working tree) is unchanged. If `copy_excluded` is true, each excluded directory that existed in `proj_dir` and is absent from `wt` is recursively copied into `wt` as an untracked directory. If `proj_dir` was not a valid git repo with HEAD (`is_git` false), `wt` is a plain copy of `proj_dir` (directories in `exclude` skipped via `shutil.ignore_patterns`), and if `copy_excluded`, those directories are subsequently copied into `wt`. `` persists after the yield (no automatic deletion). `proj_dir` is unmodified. Messages containing `wt` and cleanup instructions are printed to stdout. On exceptional execution (exception before the yield), `proj_dir` is unchanged, `` exists but may contain partial artifacts, and `wt` may not exist or be incomplete.\n\nFormally:\nLet P = os.path.abspath(proj_dir), R = os.path.basename(P.rstrip(os.sep)) or `repo`, B = tempfile.mkdtemp(prefix=`fm_agent_wt` + R + `_\`), W = os.path.join(B, `snapshot`), G = (subprocess.run([`git`, `--C`, P, `rev-parse`, `--verify`, `HEAD`], check=True, capture_output=True, text=True) did not raise CalledProcessError). Then the outcome satisfies:\n (Normal yield W) v (Exception raised ^ no yield).\n
```

```

If normal yield:\n      G =>\n      ( \exists I = os.path.join(B, \"index\") that is a valid git index ^\n      \exists tree = output of\n      _git(\"write-tree\", env=env) ^\n      \exists snap = output of _git(\"commit-tree\", tree, \"-p\", \"HEAD\", \"-m\", \"fm_agent\n      snapshot\", env=env) ^\n      _git(\"worktree\", \"add\", \"--detach\", W, snap) succeeded ^\n      the git state of P (HEAD,\n      index, working tree) is identical to entry state ^\n      \forall name \in exclude, let S = os.path.join(P, name), D = os.path.join(W,\n      name):\n      (copy_excluded \wedge os.path.isdir(S) \wedge \neg os.path.exists(D) \rightarrow D is result of shutil.copytree(S, D, symlinks=True)\n      after worktree creation) )\n      -G =>\n      ( W is the result of shutil.copytree(P, W, ignore=shutil.ignore_patterns(*exclude),\n      symlinks=True) ^\n      \forall name \in exclude: (copy_excluded \wedge os.path.isdir(os.path.join(P, name)) \wedge\n      \neg os.path.exists(os.path.join(W, name)) \rightarrow\n      shutil.copytree(os.path.join(P, name), os.path.join(W, name), symlinks=True)\n      executed after the initial copytree) )\n      \wedge B exists after yield, and stdout contains W.\n      If exception raised:\n      P and all\n      its contents are identical to entry state \wedge B exists \wedge (W may not exist or be incomplete).\"
}

```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_message_00_system.txt) · [user_prompt]
 (fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_message_01_user.txt) · [assistant_output]
 (fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_message_02_assistant.txt) · [user_prompt]
 (fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_message_03_user.txt) · [assistant_output]
 (fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_response.txt)

事件 3 — llm_11f0918b468947608ab50c754b1dc97b

► parsed_json (完整)

```

{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir is a git repository with at least one commit. Its working tree contains a top-level directory\n      'fm_agent' and also a subdirectory 'subdir/fm_agent/important.txt'. Invoke frozen_worktree(proj_dir) with default exclude=\n      ('fm_agent',). The snapshot worktree will contain 'subdir/fm_agent/important.txt', but the specification requires all entries\n      matching names in exclude to be omitted from the snapshot.",
  "offending_statements": "Line 47:      _git(\"rm\", \"-r\", \"--cached\", \"--quiet\", \"--ignore-unmatch\", \"--\n      \",\n      \n      Line 48:      *exclude, env=env)",
  "reason": "The code uses 'git rm --cached' with only the top-level exclude names, so nested directories/files with the same name\n      remain in the commit. The specification says 'entries matching names in exclude omitted', which requires omission of all such\n      entries at any depth."
}

```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_11f0918b468947608ab50c754b1dc97b_message_00_system.txt) · [user_prompt]
 (fm_agent/trace/payloads/llm_11f0918b468947608ab50c754b1dc97b_message_01_user.txt) · [assistant_output]
 (fm_agent/trace/payloads/llm_11f0918b468947608ab50c754b1dc97b_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

Phase 2 — Language Backend & Function Extraction

模块: call_graph_edges

FMA-MISMATCH-051 — src--call_graph_edges.py--_is_path_function_label

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / call_graph_edges; 原源码 src/call_graph_edges.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim:** Returns True when label contains the substring '::', the substring before the last '::' contains the character '/', and the final POSIX-path component of that substring (the portion following the last '/') contains the character '.'. Returns False otherwise.
- Actual behavior:** The function returns a boolean. It returns True if and only if the string label contains '::' and the substring before the last occurrence of '::' contains '/' and the last component of that substring when treated as a Posix path (via PurePosixPath) contains a period ('.'). Otherwise it returns False. No side effects occur and no exceptions are raised. Formally: let $s = \text{label}$, $k = s.rfind('::')$. If $k == -1$, the result is False. Otherwise, let $p = s[:k]$; the result is True iff '/' in p and '.' in PurePosixPath(p).name.
- Code evidence:** Line 4: path, _func = label.rsplit("::", 1) Line 5: return "/" in path and "." in PurePosixPath(path).name
- Trigger condition:** The specification requires checking the substring after the last '/' literally, which is empty for a trailing slash. The code uses PurePosixPath(path).name, which strips trailing slashes and returns the last nonempty component. For label='a/b.c::func', the

substring before ':' is 'a/b.c/', its final POSIXpath component per spec is " (no '.'), so spec returns False. The code returns True because PurePosixPath('a/b.c/').name is 'b.c', which contains '.'. Thus a violation occurs.

- **Validator trigger:** For label 'a/b.c::func', PurePosixPath strips the trailing slash making name='b.c' (contains '.') when spec says final component after last '/' is " (no '.')
- **Probe stdout:** CONFIRMED — actual: 'a::b-c::func' | expected: 'a/b.c::func'

► SPEC sidecar（完整）

```
{
  "signature": "_is_path_function_label(label: str) -> bool",
  "pre_condition": "label is a string.",
  "post_condition": "Returns True when label contains the substring '::', the substring before the last '::' contains the character '/', and the final POSIX-path component of that substring (the portion following the last '/') contains the character '.'. Returns False otherwise."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-py/_is_path_function_label.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns True when label contains the substring '::', the substring before the last '::' contains the character '/', and the final POSIX-path component of that substring (the portion following the last '/') contains the character '.'. Returns False otherwise.",
    "actual_behavior": "The function returns a boolean. It returns True if and only if the string `label` contains '::' and the substring before the last occurrence of '::' contains '/' and the last component of that substring when treated as a Posix path (via `PurePosixPath`) contains a period ('.'). Otherwise it returns False. No side effects occur and no exceptions are raised. Formally: let `s = label`, `k = s.rfind('::')`. If `k == -1`, the result is `False`. Otherwise, let `p = s[:k]`; the result is `True` iff ` '/' in p` and `.` in PurePosixPath(p).name`.",
    "code_evidence": "Line 4: path, _func = label.rsplit(':', 1)\nLine 5: return '/' in path and '.' in PurePosixPath(path).name",
    "trigger_condition": "The specification requires checking the substring after the last '/' literally, which is empty for a trailing slash. The code uses PurePosixPath(path).name, which strips trailing slashes and returns the last nonempty component. For label='a/b.c::func', the substring before '::' is 'a/b.c/', its final POSIXpath component per spec is ' ' (no '.'), so spec returns False. The code returns True because PurePosixPath('a/b.c/').name is 'b.c', which contains '.'. Thus a violation occurs."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--call_graph_edges-py--_is_path_function_label",
  "source_file": "src/call_graph_edges.py",
  "function_name": "_is_path_function_label",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--call_graph_edges-py--_is_path_function_label.py",
  "detail_file": "fm_agent/bug_validation/src--call_graph_edges-py--_is_path_function_label.md",
  "probe_stdout": "CONFIRMED — actual: 'a::b-c::func' | expected: 'a/b.c::func'",
  "trigger_summary": "For label 'a/b.c::func', PurePosixPath strips the trailing slash making name='b.c' (contains '.') when spec says final component after last '/' is ' ' (no '.')
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:55:03.638296Z	2226
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:55:36.776818Z	3874

事件 1 — [llm_7ec729f2c52b41e681cc98bb73287272](#)

► [parsed_json](#) (完整)

```
{
  "post_condition": "The function returns a boolean. It returns True if and only if the string `label` contains ':' and the substring before the last occurrence of ':' contains '/' and the last component of that substring when treated as a Posix path (via `PurePosixPath`) contains a period ('.'). Otherwise it returns False. No side effects occur and no exceptions are raised. Formally: let `s = label`, `k = s.rfind(':')`. If `k == -1`, the result is `False`. Otherwise, let `p = s[k:]`; the result is `True` iff ` '/' in p` and ` '.' in PurePosixPath(p).name`."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7ec729f2c52b41e681cc98bb73287272_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_7ec729f2c52b41e681cc98bb73287272_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_7ec729f2c52b41e681cc98bb73287272_response.txt)

事件 2 — [llm_013fcc3b470448b3901c5c8a15d23a4c](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "a/b.c/::func",
  "offending_statements": "Line 4: path, _func = label.rsplit(':', 1)\nLine 5: return \" / \" in path and \" . \" in PurePosixPath(path).name",
  "reason": "The specification requires checking the substring after the last '/' literally, which is empty for a trailing slash. The code uses PurePosixPath(path).name, which strips trailing slashes and returns the last non-empty component. For label='a/b.c/::func', the substring before ':' is 'a/b.c/', its final POSIX-path component per spec is '' (no '.'), so spec returns False. The code returns True because PurePosixPath('a/b.c/').name is 'b.c', which contains '.'. Thus a violation occurs."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_013fcc3b470448b3901c5c8a15d23a4c_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_013fcc3b470448b3901c5c8a15d23a4c_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_013fcc3b470448b3901c5c8a15d23a4c_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-052 — [src--call_graph_edges-py--_load_call_edge_file](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#)； attempts 1
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / [call_graph_edges](#)； 源代码 [src/call_graph_edges.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim**: When the file content is empty or consists solely of Unicode whitespace characters, returns an empty list. When the file content is non-empty valid JSON, returns a list containing one CallEdge object for each element in the 'edges' array, in the same order the elements appear in the array. If the 'edges' array is present but contains zero elements, returns an empty list. Raises FileNotFoundError if edge_path does not refer to an existing file. Raises JSONDecodeError (or a subclass thereof) if the file content is non-empty but is not valid JSON conforming to the expected schema.
- Actual behavior**: The function returns a list of CallEdge objects. Let file_contents = the string obtained by reading edge_path with replacement errors. If file_contents.strip() == "", then the returned list is empty ([]). Otherwise, file_contents is a nonempty, nonwhitespace string containing valid JSON with a toplevel key 'edges' whose value is a JSON array of valid edge objects, and the function returns _load_json_edges(file_contents, str(edge_path)), which produces a list of CallEdge objects in onetoone order with the elements of the 'edges' array. Formally:

```
( f Files f = edge_path readable(f) (content(f) = content(f) = whitespace validJSONEdges(content(f)))) result = (if strip(read_text(f, errors='replace')) = then [] else (let t = read_text(f, errors='replace') in let edges_list = decodeEdgesArray(t) in [CallEdge(e) | e edges_list]))
```

where content(f) is the raw file content, denotes empty/whitespaceonly text, and decodeEdgesArray(t) extracts the 'edges' array from the JSON in t, preserving order.

- **Code evidence:** Lines 3-5: the code only checks for empty/whitespace content and otherwise calls `_load_json_edges` without validating JSON conformance; it fails to raise `JSONDecodeError` when the file contains invalid JSON.
- **Trigger condition:** Specification requires raising `JSONDecodeError` (or subclass) if the file content is non-empty but not valid JSON conforming to the expected schema. The code does not guarantee this behavior; it passes the text directly to `_load_json_edges`, whose pre-condition requires valid JSON, leading to undefined behavior for invalid JSON input.
- **Validator trigger:** Provide a file with non-empty, non-whitespace, invalid JSON content; the function raises `ValueError` (wrapping `JSONDecodeError`) instead of `JSONDecodeError` directly.
- **Probe stdout:** CONFIRMED — `_load_call_edge_file` raises `ValueError` instead of `JSONDecodeError`: `/tmp/tmp_pj275qt/invalid.json: invalid JSON: Expecting value: line 1 column 1 (char 0)`

► SPEC sidecar (完整)

```
{
  "signature": "_load_call_edge_file(edge_path: Path) -> list[CallEdge]",
  "pre_condition": "edge_path refers to an existing, readable file whose content is either empty/whitespace-only or valid JSON containing an 'edges' array of edge objects.",
  "post_condition": "When the file content is empty or consists solely of Unicode whitespace characters, returns an empty list. When the file content is non-empty valid JSON, returns a list containing one CallEdge object for each element in the 'edges' array, in the same order the elements appear in the array. If the 'edges' array is present but contains zero elements, returns an empty list. Raises FileNotFoundError if edge_path does not refer to an existing file. Raises JSONDecodeError (or a subclass thereof) if the file content is non-empty but is not valid JSON conforming to the expected schema."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_load_json_edges",
      "signature": "_load_json_edges(text: str, source: str) -> list[CallEdge]",
      "pre_condition": "text is a non-empty, non-whitespace-only string containing valid JSON whose top-level object has an 'edges' key whose value is a JSON array of valid edge objects.",
      "post_condition": "Returns a list of CallEdge objects, one for each element in the 'edges' array, preserving the order of the elements as they appear in the array."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-py/_load_call_edge_file.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When the file content is empty or consists solely of Unicode whitespace characters, returns an empty list. When the file content is non-empty valid JSON, returns a list containing one CallEdge object for each element in the 'edges' array, in the same order the elements appear in the array. If the 'edges' array is present but contains zero elements, returns an empty list. Raises FileNotFoundError if edge_path does not refer to an existing file. Raises JSONDecodeError (or a subclass thereof) if the file content is non-empty but is not valid JSON conforming to the expected schema.",
    "actual_behavior": "The function returns a list of CallEdge objects. Let file_contents = the string obtained by reading edge_path with replacement errors. If file_contents.strip() == \"\", then the returned list is empty ([]). Otherwise, file_contents is a nonempty, nonwhitespace string containing valid JSON with a toplevel key 'edges' whose value is a JSON array of valid edge objects, and the function returns _load_json_edges(file_contents, str(edge_path)), which produces a list of CallEdge objects in onetoone order with the elements of the 'edges' array. Formally:\n\n( f Files f = edge_path readable(f) (content(f) = content(f) = whitespace validJSONEdges(content(f)))) \nresult = (if strip(read_text(f, errors='replace')) = then [] else (let t = read_text(f, errors='replace') in let edges_list = decodeEdgesArray(t) in [CallEdge(e) | e edges_list]))\n\nwhere content(f) is the raw file content, denotes empty/whitespaceonly text, and decodeEdgesArray(t) extracts the 'edges' array from the JSON in t, preserving order.",
    "code_evidence": "Lines 3-5: the code only checks for empty/whitespace content and otherwise calls _load_json_edges without validating JSON conformance; it fails to raise JSONDecodeError when the file contains invalid JSON.",
    "trigger_condition": "Specification requires raising JSONDecodeError (or subclass) if the file content is non-empty but not valid JSON conforming to the expected schema. The code does not guarantee this behavior; it passes the text directly to _load_json_edges, whose pre-condition requires valid JSON, leading to undefined behavior for invalid JSON input."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--call_graph_edges-py--_load_call_edge_file",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-py/_load_call_edge_file.py",

```



```
"function_name": "_load_call_edge_file",
"confirmation_status": "confirmed",
"attempts": 1,
"probe_script": "fm_agent/bug_validation/probe_src--call_graph_edges-py--_load_call_edge_file.py",
"detail_file": "fm_agent/bug_validation/src--call_graph_edges-py--_load_call_edge_file.md",
"probe_stdout": "CONFIRMED - _load_call_edge_file raises ValueError instead of JSONDecodeError: /tmp/tmp_pj275qt/invalid.json:
invalid JSON: Expecting value: line 1 column 1 (char 0)\n",
"trigger_summary": "Provide a file with non-empty, non-whitespace, invalid JSON content; the function raises ValueError
(wrapping JSONDecodeError) instead of JSONDecodeError directly."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:47:24.160270Z	1025
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:47:35.287225Z	3110

事件 1 — [llm_bcf8306a9f5344568cef866573b15665](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a list of CallEdge objects. Let file_contents = the string obtained by reading edge_path
with replacement errors. If file_contents.strip() == \"\", then the returned list is empty ([]). Otherwise, file_contents is a
non-empty, non-whitespace string containing valid JSON with a top-level key 'edges' whose value is a JSON array of valid edge
objects, and the function returns _load_json_edges(file_contents, str(edge_path)), which produces a list of CallEdge objects in
one-to-one order with the elements of the 'edges' array. Formally:\n\n(∃ f ∈ Files · f = edge_path ∧ readable(f) ∧ (content(f) = ε
∨ content(f) = whitespace ∨ validJSONEdges(content(f)))) ∧\nresult = (if strip(read_text(f, errors='replace')) = ε then [] else
(let t = read_text(f, errors='replace') in let edges_list = decodeEdgesArray(t) in [CallEdge(e) | e ← edges_list]))\n\nwhere
content(f) is the raw file content, ε denotes empty/whitespace-only text, and decodeEdgesArray(t) extracts the 'edges' array from
the JSON in t, preserving order."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_bcf8306a9f5344568cef866573b15665_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_bcf8306a9f5344568cef866573b15665_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_bcf8306a9f5344568cef866573b15665_response.txt)

事件 2 — [llm_f3014269c1df483ea246c1cccb2a650e](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "edge_path pointing to a non-empty file containing the string 'not valid json' (not whitespace, not valid
JSON)",
  "offending_statements": "Lines 3-5: the code only checks for empty/whitespace content and otherwise calls _load_json_edges
without validating JSON conformance; it fails to raise JSONDecodeError when the file contains invalid JSON.",
  "reason": "Specification requires raising JSONDecodeError (or subclass) if the file content is non-empty but not valid JSON
conforming to the expected schema. The code does not guarantee this behavior; it passes the text directly to _load_json_edges,
whose pre-condition requires valid JSON, leading to undefined behavior for invalid JSON input."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f3014269c1df483ea246c1cccb2a650e_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_f3014269c1df483ea246c1cccb2a650e_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_f3014269c1df483ea246c1cccb2a650e_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: **codegraph**

FMA-MISMATCH-060 — [src--languages--codegraph-py--CodeGraphExtractor::from_proj_dir](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: **confirmed**; attempts 1
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / **codegraph**; 原源码 [src/languages/codegraph.py](#)

冲突摘要

- **SPEC claim:** Returns a CodeGraphExtractor instance backed by a .codegraph/codegraph.db SQLite database when the file proj_dir/.codegraph/codegraph.db or <parent-of-proj_dir>/.codegraph/codegraph.db exists on the filesystem. Returns None when neither database file exists. A non-None return guarantees that the returned instance supports function-extraction, function-span, and call-edge queries for every language indexed in the database.
- **Actual behavior:** The method returns an instance of cls initialized with the path to an existing .codegraph/codegraph.db file found by checking first in proj_dir itself, and then in its parent directory (the directory of the absolute path of proj_dir). If neither location contains that file, it returns None. The filesystem is not altered. Formal logic: Let candidates = [proj_dir, os.path.dirname(os.path.abspath(proj_dir))] and define db_path(d) = os.path.join(d, '.codegraph', 'codegraph.db'). The return value R satisfies: if d candidates such that os.path.exists(db_path(d)), then R = cls(db_path(d)) where d is the first element of candidates with existing db_path; otherwise R = None.
- **Code evidence:** Line 9: if os.path.exists(db_path): Line 10: return cls(db_path)
- **Trigger condition:** The code only checks file existence and never validates that the database is a valid CodeGraph database supporting required queries. When the file exists but is invalid or empty, the returned instance violates the specification's required guarantee.
- **Validator trigger:** from_proj_dir only checks file existence (os.path.exists) and never validates that the .codegraph/codegraph.db file is a valid CodeGraph database; placing an empty file at that path causes a non-None return while queries fail, violating the specification guarantee.
- **Probe stdout:** CONFIRMED — Bug reproduced: from_proj_dir returned non-None for empty/invalid .codegraph/codegraph.db (spec requires None for databases that cannot support queries). Queries fail.\n

► SPEC sidecar (完整)

```
{
  "signature": "CodeGraphExtractor.from_proj_dir(proj_dir: str) -> CodeGraphExtractor | None",
  "pre_condition": "proj_dir is a string path to an existing directory.",
  "post_condition": "Returns a CodeGraphExtractor instance backed by a .codegraph/codegraph.db SQLite database when the file proj_dir/.codegraph/codegraph.db or <parent-of-proj_dir>/.codegraph/codegraph.db exists on the filesystem. Returns None when neither database file exists. A non-None return guarantees that the returned instance supports function-extraction, function-span, and call-edge queries for every language indexed in the database."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/CodeGraphExtractor::from_proj_dir.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a CodeGraphExtractor instance backed by a .codegraph/codegraph.db SQLite database when the file proj_dir/.codegraph/codegraph.db or <parent-of-proj_dir>/.codegraph/codegraph.db exists on the filesystem. Returns None when neither database file exists. A non-None return guarantees that the returned instance supports function-extraction, function-span, and call-edge queries for every language indexed in the database.",
    "actual_behavior": "The method returns an instance of cls initialized with the path to an existing .codegraph/codegraph.db file found by checking first in `proj_dir` itself, and then in its parent directory (the directory of the absolute path of `proj_dir`). If neither location contains that file, it returns `None`. The filesystem is not altered. Formal logic: Let candidates = [proj_dir, os.path.dirname(os.path.abspath(proj_dir))] and define db_path(d) = os.path.join(d, '.codegraph', 'codegraph.db'). The return value R satisfies: if d candidates such that os.path.exists(db_path(d)), then R = cls(db_path(d)) where d is the first element of candidates with existing db_path; otherwise R = None.",
    "code_evidence": "Line 9: if os.path.exists(db_path):\nLine 10: return cls(db_path)",
    "trigger_condition": "The code only checks file existence and never validates that the database is a valid CodeGraph database supporting required queries. When the file exists but is invalid or empty, the returned instance violates the specification's required guarantee."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--codegraph-py--CodeGraphExtractor::from_proj_dir",
  "source_file": "/home/fancy/Projects/Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/CodeGraphExtractor::from_proj_dir.py",
  "function_name": "CodeGraphExtractor::from_proj_dir",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--CodeGraphExtractor::from_proj_dir.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--CodeGraphExtractor::from_proj_dir.md",
  "probe_stdout": "CONFIRMED – Bug reproduced: from_proj_dir returned non-None for empty/invalid .codegraph/codegraph.db (spec requires None for databases that cannot support queries). Queries fail.\\n",
  "trigger_summary": "from_proj_dir only checks file existence (os.path.exists) and never validates that the .codegraph/codegraph.db file is a valid CodeGraph database; placing an empty file at that path causes a non-None return while queries fail, violating the specification guarantee."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:54:30.596004Z	1358
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:54:43.764850Z	2946

事件 1 — [llm_f77cebce577a43c29746db641e4d90f0](#)

► parsed_json（完整）

```
{
  "post_condition": "The method returns an instance of cls initialized with the path to an existing `.codegraph/codegraph.db` file found by checking first in `proj_dir` itself, and then in its parent directory (the directory of the absolute path of `proj_dir`). If neither location contains that file, it returns `None`. The filesystem is not altered. Formal logic: Let candidates = [proj_dir, os.path.dirname(os.path.abspath(proj_dir))] and define db_path(d) = os.path.join(d, '.codegraph', 'codegraph.db'). The return value R satisfies: if ∃ d ∈ candidates such that os.path.exists(db_path(d)), then R = cls(db_path(d_0)) where d_0 is the first element of candidates with existing db_path; otherwise R = None."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f77cebce577a43c29746db641e4d90f0_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f77cebce577a43c29746db641e4d90f0_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f77cebce577a43c29746db641e4d90f0_response.txt)

事件 2 — [llm_d3800ea770c941f695a2599ce1bf76f7](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp/test_project' with an empty file at '/tmp/test_project/.codegraph/codegraph.db' (or any non-CodeGraph SQLite file). The method returns an instance, but it does not support function-extraction, function-span, or call-edge queries, failing the specification's guarantee for non-None returns.",
  "offending_statements": "Line 9: if os.path.exists(db_path):\\nLine 10: return cls(db_path)",
  "reason": "The code only checks file existence and never validates that the database is a valid CodeGraph database supporting required queries. When the file exists but is invalid or empty, the returned instance violates the specification's required guarantee."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d3800ea770c941f695a2599ce1bf76f7_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d3800ea770c941f695a2599ce1bf76f7_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d3800ea770c941f695a2599ce1bf76f7_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-061 — [src--languages--codegraph-py--CodeGraphExtractor::get_call_edges](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#)； attempts 1
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / [codegraph](#)； 源代码 [src/languages/codegraph.py](#)

冲突摘要

- **SPEC claim:** Returns a dict mapping each caller FQN (str) to a non-empty set of callee FQNs (set[str]) for all direct call relationships discovered in source files of the given language within the indexed project. Both keys and values use canonicalized FQNs with :: as the component separator. Call edges include both function-call edges and constructor invocations synthesized from class-instantiation relationships. FQN identity is resolved by codegraph node ID, preserving the precise caller/callee association for same-named functions in different files. Returns an empty dict when lang_key is not a recognized language or when no call edges exist for the given language.
- **Actual behavior:** If no exception is raised, then: If the lookup _CG_LANG.get(lang_key) yields a non-empty list cg_langs, then the method opens a connection to self._db, obtains a cursor, computes fqn_of = _node_fqn_map(cur, cg_langs), then constructs a mapping result where for each (src, tgt) from edges.kind='calls' with source language in cg_langs, if both fqn_of[src] and fqn_of[tgt] are not None, then result[fqn_of[src]] contains fqn_of[tgt]; additionally if a ctor_filter = _CONSTRUCTOR_FILTER.get(lang_key) exists, for each (src, ctor_id) from the second query (edges.kind='instantiates' with source language in cg_langs, target class containing a method/function matching ctor_filter), if both fqn_of[src] and fqn_of[ctor_id] are not None, then result[fqn_of[src]] contains fqn_of[ctor_id]. The connection is closed before return. The method returns dict(result). If _CG_LANG.get(lang_key) yields a falsy value, the method returns {} without any side effects. In all cases, self remains unmodified, self._db is unchanged, no resources leaked.
- **Code evidence:** Line 40: caller, callee = fqn_of.get(src_id), fqn_of.get(tgt_id) Line 41: if caller and callee: Line 42: result[caller].add(callee)
- **Trigger condition:** The code omits call edges where the callee's FQN is not present in fqn_of, which only covers nodes of the requested languages. However, the specification demands all direct call relationships discovered in source files of the given language, including calls to functions defined in other languages. This leads to a missing callee in the output dict.
- **Validator trigger:** Calls from the requested language to functions in other languages are dropped because _node_fqn_map only maps nodes of the requested language, so fqn_of.get returns None for cross-language callees.
- **Probe stdout:** CONFIRMED — Bug reproduced in attempt 1: cross-language call edge from Python to C was dropped. actual={} | expected={'src::py_code-py::py_func': {'src::c_code-c::c_func'}}

► SPEC sidecar (完整)

```
{
  "signature": "CodeGraphExtractor.get_call_edges(lang_key: str) -> dict",
  "pre_condition": "self is a CodeGraphExtractor instance initialized with a valid and uncorrupted codegraph database containing call-edge data; lang_key is a string value.",
  "post_condition": "Returns a dict mapping each caller FQN (str) to a non-empty set of callee FQNs (set[str]) for all direct call relationships discovered in source files of the given language within the indexed project. Both keys and values use canonicalized FQNs with :: as the component separator. Call edges include both function-call edges and constructor invocations synthesized from class-instantiation relationships. FQN identity is resolved by codegraph node ID, preserving the precise caller/callee association for same-named functions in different files. Returns an empty dict when lang_key is not a recognized language or when no call edges exist for the given language."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/CodeGraphExtractor::get_call_edges.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dict mapping each caller FQN (str) to a non-empty set of callee FQNs (set[str]) for all direct call relationships discovered in source files of the given language within the indexed project. Both keys and values use canonicalized FQNs with :: as the component separator. Call edges include both function-call edges and constructor invocations synthesized from class-instantiation relationships. FQN identity is resolved by codegraph node ID, preserving the precise caller/callee association for same-named functions in different files. Returns an empty dict when lang_key is not a recognized language or when no call edges exist for the given language.",
    "actual_behavior": "If no exception is raised, then: If the lookup _CG_LANG.get(lang_key) yields a non-empty list cg_langs, then the method opens a connection to self._db, obtains a cursor, computes fqn_of = _node_fqn_map(cur, cg_langs), then constructs a mapping result where for each (src, tgt) from edges.kind='calls' with source language in cg_langs, if both fqn_of[src] and fqn_of[tgt] are not None, then result[fqn_of[src]] contains fqn_of[tgt]; additionally if a ctor_filter = _CONSTRUCTOR_FILTER.get(lang_key) exists, for each (src, ctor_id) from the second query (edges.kind='instantiates' with source language in cg_langs, target class containing a method/function matching ctor_filter), if both fqn_of[src] and fqn_of[ctor_id] are not None, then result[fqn_of[src]] contains fqn_of[ctor_id]. The connection is closed before return. The method returns dict(result). If _CG_LANG.get(lang_key) yields a falsy value, the method returns {} without any side effects. In all cases, self
```

```
remains unmodified, self._db is unchanged, no resources leaked.",
"code_evidence": "Line 40: caller, callee = fqn_of.get(src_id), fqn_of.get(tgt_id)\nLine 41: if caller
and callee:\nLine 42: result[caller].add(callee)",
"trigger_condition": "The code omits call edges where the callee's FQN is not present in fqn_of, which only covers nodes of
the requested languages. However, the specification demands all direct call relationships discovered in source files of the given
language, including calls to functions defined in other languages. This leads to a missing callee in the output dict."
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--codegraph-py--CodeGraphExtractor::get_call_edges",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-
py/CodeGraphExtractor::get_call_edges.py",
  "function_name": "CodeGraphExtractor::get_call_edges",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--CodeGraphExtractor::get_call_edges.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--CodeGraphExtractor::get_call_edges.md",
  "probe_stdout": "CONFIRMED - Bug reproduced in attempt 1: cross-language call edge from Python to C was dropped. actual={} |
expected={'src::py_code-py::py_func': {'src::c_code-c::c_func'}}",
  "trigger_summary": "Calls from the requested language to functions in other languages are dropped because _node_fqn_map only
maps nodes of the requested language, so fqn_of.get returns None for cross-language callees."
}
```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-27T16:54:34.609052Z	2666
2	generate_block_post_condition	success	2	0/1	2026-07-27T16:54:56.276908Z	3284
3	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:55:14.298556Z	5818

事件 1 — [llm_0dbae84127b946c290dc6217ff8e1961](#)

► parsed_json（完整）

```
{}
```

- parse_error: `post-condition JSON requires a non-empty string field: post\_condition` - 载荷: [system_prompt](fm_agent/trace/payloads/llm_0dbae84127b946c290dc6217ff8e1961_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_0dbae84127b946c290dc6217ff8e1961_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_0dbae84127b946c290dc6217ff8e1961_response.txt)

事件 2 — [llm_75e849652bbb432b96f4e7dcae0059cc](#)

► parsed_json（完整）

```
{
  "post_condition": "If no exception is raised, then: If the lookup _CG_LANG.get(lang_key) yields a non-empty list cg_langs, then
the method opens a connection to self._db, obtains a cursor, computes fqn_of = _node_fqn_map(cur, cg_langs), then constructs a
mapping result where for each (src, tgt) from edges.kind='calls' with source language in cg_langs, if both fqn_of[src] and
fqn_of[tgt] are not None, then result[fqn_of[src]] contains fqn_of[tgt]; additionally if a ctor_filter =
_CONSTRUCTOR_FILTER.get(lang_key) exists, for each (src, ctor_id) from the second query (edges.kind='instantiates' with source
language in cg_langs, target class containing a method/function matching ctor_filter), if both fqn_of[src] and fqn_of[ctor_id] are
not None, then result[fqn_of[src]] contains fqn_of[ctor_id]. The connection is closed before return. The method returns
dict(result). If _CG_LANG.get(lang_key) yields a falsy value, the method returns {} without any side effects. In all cases, self
remains unmodified, self._db is unchanged, no resources leaked."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_message_02_assistant.txt) · [user_prompt](fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_message_03_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_response.txt)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Consider lang_key='python', with _CG_LANG['python']=['python']. The database contains a 'calls' edge from
source node 1 (language='python', FQN='src::file::foo') to target node 2 (language='cpp', FQN='other::file::bar'). The code builds
fq_n_of only for nodes in ['python'], so fq_n_of.get(2) is None. The edge is skipped, and the method returns {}. The specification
requires {'src::file::foo': {'other::file::bar'}}.",
  "offending_statements": "Line 40:         caller, callee = fq_n_of.get(src_id), fq_n_of.get(tgt_id)\nLine 41:         if
caller and callee:\n                        result[caller].add(callee)",
  "reason": "The code omits call edges where the callee's FQN is not present in fq_n_of, which only covers nodes of the requested
languages. However, the specification demands all direct call relationships discovered in source files of the given language,
including calls to functions defined in other languages. This leads to a missing callee in the output dict."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f74a6de9292c40bda7c8ff45e97964d8_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_f74a6de9292c40bda7c8ff45e97964d8_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_f74a6de9292c40bda7c8ff45e97964d8_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-062 — [src--languages--codegraph-py--CodeGraphExtractor::get_functions_by_file](#)

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts [1](#)
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / [codegraph](#); 原源码 [src/languages/codegraph.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: When lang_key is recognized by the code analysis backend: returns a dictionary mapping each absolute source file path (str) to a list of (function_name: str, body_text: str) tuples. Each tuple corresponds to one function or method definition in that file. function_name is a class-qualified identifier for methods or a bare function name for free functions; when multiple definitions share the same function_name within the same file, the first occurrence retains the bare name and all subsequent occurrences append a deterministic underscore-suffixed numeric index (beginning with _1) by source-order position. body_text is the complete source text of the function definition from its starting line through its ending line, with a trailing newline. Files present in the index but unreadable on disk are excluded from the returned dictionary. Returns an empty dictionary when lang_key is not recognized by the code analysis backend.
- **Actual behavior**: If _CG_LANG.get(lang_key) is falsy (None or empty), the method returns an empty dictionary and no database connection is opened. Otherwise, a SQLite connection to self._db is opened; if a sqlite3.Error or other unhandled exception occurs during the database operations (connect, cursor, execute, fetchall) or before the explicit close, the connection may not be closed, no return value is produced, and the exception propagates. If the database interaction completes without exception, the connection is closed after fetching all rows ordered by file_path, start_line. The rows are grouped by file_path into a mapping (by_file), preserving the row order, where each value is a list of tuples (ident, start_line, end_line) with ident = *extraction_ident(name, qualified_name)* and start_line/end_line converted to int. Then an empty dictionary result is built. For each file_path in by_file, abs_path is computed as os.path.join(proj_dir, file_path) if proj_dir is not None, else file_path. An attempt is made to open abs_path in text mode with errors='replace'; if an OSError occurs, the file is skipped (continue), leaving the result unchanged for that path. If the file is opened successfully, its lines are read. Then for each function entry in the per-file list (ordered by start_line), a deduplicated identifier deduped is generated: if the identifier ident has appeared count times before in the same file (tracked per file), deduped = ident if count == 0 else f'{ident}{count}'. The body is extracted as the concatenation of lines[start_line - 1 : end_line]; if the resulting body does not end with '\n', a newline is appended. The pair (deduped, body) is appended to a list file_funcs in that order. After processing all functions for the file, result[abs_path] is set to file_funcs. Once all files are processed, the method returns result. If any other unexpected exception occurs during file processing after the database phase, it propagates and no return occurs. Formal logic: Let L = _CG_LANG.get(lang_key). If L is falsy, return {}. Otherwise, let R = the set of rows obtained by executing the SQL query with parameters L, ordered by file_path, start_line. Let G = group-preserving-order(R by file_path). Let F = {}. For each (fp, entries) in G.items(): let p = os.path.join(proj_dir, fp) if proj_dir else fp. If opening p raises OSError, skip this fp. Else, let lines = file.readlines(). Let C = empty map. For each (name, qname, _, start, end) in entries: ident := *extraction_ident(name, qname)*; cnt := C[ident] default 0; C[ident] := cnt+1; deduped := ident if cnt == 0 else ident + " + str(cnt); body := ".join(lines[start-1 : end]); if not body.endswith('\n') then body += '\n'; append (deduped, body) to list L_fp. Then F[p] := L_fp. End for. Return F. The method does not modify self or its database file; side effects are limited to reading the database and file system.
- **Code evidence**: Line 32: abs_path = os.path.join(proj_dir, file_path) if proj_dir else file_path

- **Trigger condition:** The specification requires the returned dictionary to map absolute source file paths, but when `proj_dir` is `None`, the path used is the raw relative `file_path`, violating the absolute requirement.
- **Validator trigger:** When `proj_dir` is `None`, `get_functions_by_file` uses raw relative `file_path` as dict keys instead of absolute paths, violating the specification's absolute-path guarantee.
- **Probe stdout:** CONFIRMED — returned dict keys are not all absolute: relative keys=`['test_module.py']`, absolute keys=`[]`

► SPEC sidecar (完整)

```
{
  "signature": "CodeGraphExtractor.get_functions_by_file(self, lang_key: str, proj_dir: str) -> dict",
  "pre_condition": "self is an initialized CodeGraphExtractor with a valid project index; lang_key is a string language identifier; proj_dir is a string path to the project root directory",
  "post_condition": "When lang_key is recognized by the code analysis backend: returns a dictionary mapping each absolute source file path (str) to a list of (function_name: str, body_text: str) tuples. Each tuple corresponds to one function or method definition in that file. function_name is a class-qualified identifier for methods or a bare function name for free functions; when multiple definitions share the same function_name within the same file, the first occurrence retains the bare name and all subsequent occurrences append a deterministic underscore-suffixed numeric index (beginning with _1) by source-order position. body_text is the complete source text of the function definition from its starting line through its ending line, with a trailing newline. Files present in the index but unreadable on disk are excluded from the returned dictionary. Returns an empty dictionary when lang_key is not recognized by the code analysis backend."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/CodeGraphExtractor::get_functions_by_file.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When lang_key is recognized by the code analysis backend: returns a dictionary mapping each absolute source file path (str) to a list of (function_name: str, body_text: str) tuples. Each tuple corresponds to one function or method definition in that file. function_name is a class-qualified identifier for methods or a bare function name for free functions; when multiple definitions share the same function_name within the same file, the first occurrence retains the bare name and all subsequent occurrences append a deterministic underscore-suffixed numeric index (beginning with _1) by source-order position. body_text is the complete source text of the function definition from its starting line through its ending line, with a trailing newline. Files present in the index but unreadable on disk are excluded from the returned dictionary. Returns an empty dictionary when lang_key is not recognized by the code analysis backend.",
    "actual_behavior": "If _CG_LANG.get(lang_key) is falsy (None or empty), the method returns an empty dictionary and no database connection is opened. Otherwise, a SQLite connection to self._db is opened; if a sqlite3.Error or other unhandled exception occurs during the database operations (connect, cursor, execute, fetchall) or before the explicit close, the connection may not be closed, no return value is produced, and the exception propagates. If the database interaction completes without exception, the connection is closed after fetching all rows ordered by file_path, start_line. The rows are grouped by file_path into a mapping (by_file), preserving the row order, where each value is a list of tuples (ident, start_line, end_line) with ident = _extraction_ident(name, qualified_name) and start_line/end_line converted to int. Then an empty dictionary result is built. For each file_path in by_file, abs_path is computed as os.path.join(proj_dir, file_path) if proj_dir is not None, else file_path. An attempt is made to open abs_path in text mode with errors='replace'; if an OSError occurs, the file is skipped (continue), leaving the result unchanged for that path. If the file is opened successfully, its lines are read. Then for each function entry in the per-file list (ordered by start_line), a deduplicated identifier deduped is generated: if the identifier ident has appeared count times before in the same file (tracked per file), deduped = ident if count == 0 else f\"{ident}_{count}\". The body is extracted as the concatenation of lines[start_line - 1 : end_line]; if the resulting body does not end with '\\n', a newline is appended. The pair (deduped, body) is appended to a list file_funcs in that order. After processing all functions for the file, result[abs_path] is set to file_funcs. Once all files are processed, the method returns result. If any other unexpected exception occurs during file processing after the database phase, it propagates and no return occurs. Formal logic: Let L = _CG_LANG.get(lang_key). If L is falsy, return {}. Otherwise, let R = the set of rows obtained by executing the SQL query with parameters L, ordered by file_path, start_line. Let G = group-preserving-order(R by file_path). Let F = {}. For each (fp, entries) in G.items(): let p = os.path.join(proj_dir, fp) if proj_dir else fp. If opening p raises OSError, skip this fp. Else, let lines = file.readlines(). Let C = empty map. For each (name, qname, _, start, end) in entries: ident := _extraction_ident(name, qname); cnt := C[ident] default 0; C[ident] := cnt+1; deduped := ident if cnt == 0 else ident + '_' + str(cnt); body := ''.join(lines[start-1 : end]); if not body.endswith('\\n') then body += '\\n'; append (deduped, body) to list L_fp. Then F[p] := L_fp. End for. Return F. The method does not modify self or its database file; side effects are limited to reading the database and file system.",
    "code_evidence": "Line 32: abs_path = os.path.join(proj_dir, file_path) if proj_dir else file_path",
    "trigger_condition": "The specification requires the returned dictionary to map absolute source file paths, but when proj_dir is None, the path used is the raw relative file_path, violating the absolute requirement."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--codegraph-py--CodeGraphExtractor::get_functions_by_file",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/CodeGraphExtractor::get_functions_by_file.py",
  "function_name": "get_functions_by_file",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--CodeGraphExtractor::get_functions_by_file.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--CodeGraphExtractor::get_functions_by_file.md",
  "probe_stdout": "CONFIRMED – returned dict keys are not all absolute: relative keys=['test_module.py'], absolute keys=[]",
  "trigger_summary": "When proj_dir is None, get_functions_by_file uses raw relative file_path as dict keys instead of absolute paths, violating the specification's absolute-path guarantee."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:48:08.386689Z	4271
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:48:59.196132Z	7319

事件 1 — [llm_854587b2c13446bd91d2e7b52c85cf60](#)

► parsed_json（完整）

```
{
  "post_condition": "If _CG_LANG.get(lang_key) is falsy (None or empty), the method returns an empty dictionary and no database connection is opened. Otherwise, a SQLite connection to self._db is opened; if a sqlite3.Error or other unhandled exception occurs during the database operations (connect, cursor, execute, fetchall) or before the explicit close, the connection may not be closed, no return value is produced, and the exception propagates. If the database interaction completes without exception, the connection is closed after fetching all rows ordered by file_path, start_line. The rows are grouped by file_path into a mapping (by_file), preserving the row order, where each value is a list of tuples (ident, start_line, end_line) with ident = _extraction_ident(name, qualified_name) and start_line/end_line converted to int. Then an empty dictionary result is built. For each file_path in by_file, abs_path is computed as os.path.join(proj_dir, file_path) if proj_dir is not None, else file_path. An attempt is made to open abs_path in text mode with errors='replace'; if an OSError occurs, the file is skipped (continue), leaving the result unchanged for that path. If the file is opened successfully, its lines are read. Then for each function entry in the per-file list (ordered by start_line), a deduplicated identifier deduped is generated: if the identifier ident has appeared count times before in the same file (tracked per file), deduped = ident if count == 0 else f\"{ident}_{count}\". The body is extracted as the concatenation of lines[start_line - 1 : end_line]; if the resulting body does not end with '\\n', a newline is appended. The pair (deduped, body) is appended to a list file_funcs in that order. After processing all functions for the file, result[abs_path] is set to file_funcs. Once all files are processed, the method returns result. If any other unexpected exception occurs during file processing after the database phase, it propagates and no return occurs. Formal logic: Let L = _CG_LANG.get(lang_key). If L is falsy, return {}. Otherwise, let R = the set of rows obtained by executing the SQL query with parameters L, ordered by file_path, start_line. Let G = group-preserving-order(R by file_path). Let F = {}. For each (fp, entries) in G.items(): let p = os.path.join(proj_dir, fp) if proj_dir else fp. If opening p raises OSError, skip this fp. Else, let lines = file.readlines(). Let C = empty map. For each (name, qname, _, start, end) in entries: ident := _extraction_ident(name, qname); cnt := C[ident] default 0; C[ident] := cnt+1; deduped := ident if cnt == 0 else ident + '_' + str(cnt); body := ''.join(lines[start-1 : end]); if not body.endswith('\\n') then body += '\\n'; append (deduped, body) to list L_fp. Then F[p] := L_fp. End for. Return F. The method does not modify self or its database file; side effects are limited to reading the database and file system."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_854587b2c13446bd91d2e7b52c85cf60_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_854587b2c13446bd91d2e7b52c85cf60_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_854587b2c13446bd91d2e7b52c85cf60_response.txt)

事件 2 — [llm_5b5840e9deb74d11b39872254ccfa139](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Call get_functions_by_file with a recognized lang_key (e.g., 'python'), proj_dir=None, and a database containing a relative file_path (e.g., 'src/foo.py'). The file 'src/foo.py' exists and contains a function. The method returns a dictionary with the key 'src/foo.py' (relative path) instead of an absolute path.",
  "offending_statements": "Line 32: abs_path = os.path.join(proj_dir, file_path) if proj_dir else file_path",
  "reason": "The specification requires the returned dictionary to map absolute source file paths, but when proj_dir is None, the path used is the raw relative file_path, violating the absolute requirement."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5b5840e9deb74d11b39872254ccfa139_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_5b5840e9deb74d11b39872254ccfa139_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_5b5840e9deb74d11b39872254ccfa139_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-065 — [src](#) · [languages](#) · [codegraph.py](#) · [_extraction_ident](#)

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts [1](#)
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / [codegraph](#); 源代码 [src/languages/codegraph.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a canonicalized fully-qualified function identifier using ':' as the component separator. Each component of the qualified name is first stripped of tree-sitter decorations (type signatures, template bodies, and parameter lists) to yield the bare identifier, then transformed for filesystem and FQN safety by replacing every '/' character with '_'. The number of components equals the number of identifier segments after splitting qualified_name on ':' and '.' boundaries, preserving left-to-right order.
- **Actual behavior**: The function returns a string R that satisfies: Let C = _qualified_parts(name, qualified_name), a list of component strings obtained by splitting qualified_name on ':' and '.' boundaries, defaulting to [name] if no components arise. Then R = ':'.join(canonicalize(_bare_function_name(p)) for p in C). For each p, *bare_function_name(p) strips tree-sitter decorations, leaving the bare identifier, and canonicalize(s) replaces every '/' with '_'*. Hence R is a class-qualified identifier with ':' separators, free of slashes and tree-sitter artifacts, and safe for filesystem use.
- **Code evidence**: Line 15: return ":".join(Line 16: canonicalize(_bare_function_name(p)) Line 17: for p in _qualified_parts(name, qualified_name) Line 18:)
- **Trigger condition**: When qualified_name starts with ':' (e.g., ':foo'), _qualified_parts splits on ':' boundaries yielding an empty first component. The code includes this empty component in the joined result, producing ':foo'. The specification requires the number of components to equal the number of identifier segments (i.e., non-empty parts), so the expected result is 'foo'.
- **Validator trigger**: qualified_name with whitespace after ':' like ': foo' causes _bare_function_name to produce an empty component, resulting in a leading ':' separator in the output where only 'foo' is expected.
- **Probe stdout**: CONFIRMED — actual: ':foo' | expected: 'foo'

► SPEC sidecar (完整)

```
{
  "signature": "_extraction_ident(name: str, qualified_name: str) -> str",
  "pre_condition": "name and qualified_name are strings from a codegraph nodes table row, where qualified_name is a module-qualified identifier using ':' or '.' as component separators.",
  "post_condition": "Returns a canonicalized fully-qualified function identifier using ':' as the component separator. Each component of the qualified name is first stripped of tree-sitter decorations (type signatures, template bodies, and parameter lists) to yield the bare identifier, then transformed for filesystem and FQN safety by replacing every '/' character with '_'. The number of components equals the number of identifier segments after splitting qualified_name on ':' and '.' boundaries, preserving left-to-right order."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_qualified_parts",
      "signature": "_qualified_parts(name: str, qualified_name: str) -> list[str]",
      "pre_condition": "name and qualified_name are string values from a codegraph nodes table row.",
      "post_condition": "Returns a list of identifier component strings obtained by splitting qualified_name on ':' and '.' boundaries; when the split yields no components, returns a single-element list containing name."
    },
    {
      "name": "_bare_function_name",
      "signature": "_bare_function_name(name: str) -> str",
      "pre_condition": "name is a string component that may contain tree-sitter decorations (full function signature, template body, or parameter lists).",
      "post_condition": "Returns the bare function identifier with any tree-sitter decorations, type annotations, template parameters, and parameter lists removed; the result is the unqualified function or method name only."
    },
    {
      "name": "canonicalize",
      "signature": "canonicalize(func_name: str) -> str",
    }
  ]
}
```

```
"pre_condition": "func_name is a function identifier string.",
"post_condition": "Returns a filesystem- and FQN-safe version of func_name where every '/' character is replaced with '_';
all other characters are preserved unchanged."
}
]
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/_extraction_ident.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a canonicalized fully-qualified function identifier using '::' as the component separator. Each
component of the qualified name is first stripped of tree-sitter decorations (type signatures, template bodies, and parameter
lists) to yield the bare identifier, then transformed for filesystem and FQN safety by replacing every '/' character with '_'. The
number of components equals the number of identifier segments after splitting qualified_name on '::' and '.' boundaries,
preserving left-to-right order.",
    "actual_behavior": "The function returns a string R that satisfies: Let C = _qualified_parts(name, qualified_name), a list of
component strings obtained by splitting qualified_name on '::' and '.' boundaries, defaulting to [name] if no components arise.
Then R = '::'.join(canonicalize(_bare_function_name(p)) for p in C). For each p, _bare_function_name(p) strips tree-sitter
decorations, leaving the bare identifier, and canonicalize(s) replaces every '/' with '_'. Hence R is a class-qualified identifier
with '::' separators, free of slashes and tree-sitter artifacts, and safe for filesystem use.",
    "code_evidence": "Line 15: return \"::\".join(\\nLine 16:     canonicalize(_bare_function_name(p))\\nLine 17:     for p in
_qualified_parts(name, qualified_name)\\nLine 18: )",
    "trigger_condition": "When qualified_name starts with '::' (e.g., '::foo'), _qualified_parts splits on '::' boundaries
yielding an empty first component. The code includes this empty component in the joined result, producing '::foo'. The
specification requires the number of components to equal the number of identifier segments (i.e., non-empty parts), so the
expected result is 'foo'."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--codegraph-py--_extraction_ident",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-
py/_extraction_ident.py",
  "function_name": "_extraction_ident",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--_extraction_ident.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--_extraction_ident.md",
  "probe_stdout": "CONFIRMED - actual: '::foo' | expected: 'foo\\n",
  "trigger_summary": "qualified_name with whitespace after '::' like ':: foo' causes _bare_function_name to produce an empty
component, resulting in a leading '::' separator in the output where only 'foo' is expected."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:29:42.098401Z	3109
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:30:22.301573Z	5425

事件 1 — [llm_ca4f7f6476d44449bfb92f9b3381eed0](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string R that satisfies: Let C = _qualified_parts(name, qualified_name), a list of
component strings obtained by splitting qualified_name on '::' and '.' boundaries, defaulting to [name] if no components arise.
Then R = '::'.join(canonicalize(_bare_function_name(p)) for p in C). For each p, _bare_function_name(p) strips tree-sitter
decorations, leaving the bare identifier, and canonicalize(s) replaces every '/' with '_'. Hence R is a class-qualified identifier
with '::' separators, free of slashes and tree-sitter artifacts, and safe for filesystem use."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ca4f7f6476d44449bfb92f9b3381eed0_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_ca4f7f6476d44449bfb92f9b3381eed0_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_ca4f7f6476d44449bfb92f9b3381eed0_response.txt)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "name='foo', qualified_name='::foo'",
  "offending_statements": "Line 15: return \"::\".join(\\nLine 16:         canonicalize(_bare_function_name(p))\\nLine 17:         for p in
_qualified_parts(name, qualified_name)\\nLine 18: )",
  "reason": "When qualified_name starts with '::' (e.g., '::foo'), _qualified_parts splits on ':' boundaries yielding an empty
first component. The code includes this empty component in the joined result, producing '::foo'. The specification requires the
number of components to equal the number of identifier segments (i.e., non-empty parts), so the expected result is 'foo'."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_137c7d892e5440e892b97c9cb896471d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_137c7d892e5440e892b97c9cb896471d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_137c7d892e5440e892b97c9cb896471d_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-067 — src--languages--codegraph-py--_qualified_parts

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / codegraph; 源代码 src/languages/codegraph.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim:** Returns a non-empty list of identifier component strings whose final element is name. If qualified_name is empty or does not end with name, the result is [name]. Otherwise, the portion of qualified_name that precedes name is split on each occurrence of '::' or '.', empty components are discarded, and the surviving non-empty components form the prefix of the returned list in the left-to-right order they appear in qualified_name.
- Actual behavior:** The function returns a list of strings R. Let s = (qualified_name or "").strip(). If s == "" or not s.endswith(name): R = [name]. Otherwise, define scope = s[:-len(name)].rstrip('.:'). If scope == "": R = [name]. Else, let parts be the non-empty segments obtained by splitting scope on occurrences of '::' or '.' (using re.split); then R = parts + [name]. No exceptions are raised, and the list contains at least one element (name). Formally, R [name] ({[p for p in re.split(r'::|.', scope) if p] + [name]} where scope = (qualified_name or "").strip()[:-len(name)].rstrip('.:') and (qualified_name or "").strip().endswith(name)).
- Code evidence:** Line 14: q = (qualified_name or "").strip() Line 15: if not q or not q.endswith(name):
- Trigger condition:** Specification requires checking whether qualified_name (without stripping) ends with name. The code strips whitespace first, so for ' Foo::bar ' it does not end with 'bar', but after stripping it does, causing the code to incorrectly return ['Foo', 'bar'] instead of the required ['bar'].
- Validator trigger:** qualified_name with leading/trailing whitespace causes the stripped ends with check to misclassify the string, producing scope parts that should not appear per spec.
- Probe stdout:** CONFIRMED — actual: ['Foo', 'bar'] | expected: ['bar'] (code strips whitespace before ends with check, spec requires checking raw qualified_name)

► SPEC sidecar (完整)

```
{
  "signature": "_qualified_parts(name: str, qualified_name: str) -> list[str]",
  "pre_condition": "name and qualified_name are string values from a codegraph nodes table row.",
  "post_condition": "Returns a non-empty list of identifier component strings whose final element is name. If qualified_name is
empty or does not end with name, the result is [name]. Otherwise, the portion of qualified_name that precedes name is split on
each occurrence of '::' or '.', empty components are discarded, and the surviving non-empty components form the prefix of the
returned list in the left-to-right order they appear in qualified_name."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/_qualified_parts.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a non-empty list of identifier component strings whose final element is name. If qualified_name is empty or does not end with name, the result is [name]. Otherwise, the portion of qualified_name that precedes name is split on each occurrence of ':' or '.', empty components are discarded, and the surviving non-empty components form the prefix of the returned list in the left-to-right order they appear in qualified_name.",
    "actual_behavior": "The function returns a list of strings R. Let s = (qualified_name or '').strip(). If s == '' or not s.endswith(name): R = [name]. Otherwise, define scope = s[:-len(name)].rstrip(':.'). If scope == '': R = [name]. Else, let parts be the non-empty segments obtained by splitting scope on occurrences of ':' or '.' (using re.split); then R = parts + [name]. No exceptions are raised, and the list contains at least one element (name). Formally, R [name] ({[p for p in re.split(r'::|\\.', scope) if p] + [name]} where scope = (qualified_name or '').strip()[:-len(name)].rstrip(':.') and (qualified_name or '').strip().endswith(name)).",
    "code_evidence": "Line 14: q = (qualified_name or '\\').strip()\\nLine 15: if not q or not q.endswith(name):",
    "trigger_condition": "Specification requires checking whether qualified_name (without stripping) ends with name. The code strips whitespace first, so for ' Foo::bar ' it does not end with 'bar', but after stripping it does, causing the code to incorrectly return ['Foo', 'bar'] instead of the required ['bar'].\"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--codegraph-py--_qualified_parts",
  "source_file": "src/languages/codegraph-py/_qualified_parts.py",
  "function_name": "_qualified_parts",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--_qualified_parts.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--_qualified_parts.md",
  "probe_stdout": "CONFIRMED – actual: ['Foo', 'bar'] | expected: ['bar'] (code strips whitespace before ends with check, spec requires checking raw qualified_name)\\n",
  "trigger_summary": "qualified_name with leading/trailing whitespace causes the stripped ends with check to misclassify the string, producing scope parts that should not appear per spec.\"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:38:54.847122Z	1924
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:39:19.154339Z	4236

事件 1 — [llm_1cb51b7f4f3d457fb5732b972a94e8d7](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a list of strings R. Let s = (qualified_name or '').strip(). If s == '' or not s.endswith(name): R = [name]. Otherwise, define scope = s[:-len(name)].rstrip(':.'). If scope == '': R = [name]. Else, let parts be the non-empty segments obtained by splitting scope on occurrences of ':' or '.' (using re.split); then R = parts + [name]. No exceptions are raised, and the list contains at least one element (name). Formally, R ∈ [name] ∪ ({[p for p in re.split(r'::|\\.', scope) if p] + [name]} where scope = (qualified_name or '').strip()[:-len(name)].rstrip(':.') and (qualified_name or '').strip().endswith(name)).\"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1cb51b7f4f3d457fb5732b972a94e8d7_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_1cb51b7f4f3d457fb5732b972a94e8d7_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_1cb51b7f4f3d457fb5732b972a94e8d7_response.txt)

事件 2 — [llm_1682d44c706d4af3a442634932dbddb2](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
```



```
"counterexample": "name = 'bar', qualified_name = '    Foo::bar    '",
"offending_statements": "Line 14: q = (qualified_name or '').strip()\nLine 15: if not q or not q.endswith(name):",
"reason": "Specification requires checking whether qualified_name (without stripping) ends with name. The code strips whitespace first, so for '    Foo::bar    ' it does not end with 'bar', but after stripping it does, causing the code to incorrectly return ['Foo', 'bar'] instead of the required ['bar'].",
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1682d44c706d4af3a442634932dbddb2_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_1682d44c706d4af3a442634932dbddb2_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_1682d44c706d4af3a442634932dbddb2_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-068 — [src](#) - [--languages](#) - [codegraph.py](#) - [_warn_on_codegraph_version_mismatch](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts [1](#)
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / [codegraph](#); 原源码 [src/languages/codegraph.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: If the configured codegraph.version (after stripping whitespace and removing any leading 'v') is empty, returns immediately with no side effects. If the executable at cmd cannot be reached or fails to report its version, returns silently with no side effects. If the version reported by the executable differs from the configured version, a warning is logged containing both version strings. The function never raises an exception and never blocks or terminates the calling process.
- **Actual behavior**: The function returns None. No exception is propagated. The global state (including [settings.codegraph.version](#)) is unchanged. If [want](#) (computed as [settings.codegraph.version.strip\(\).removeprefix\('v'\)](#)) is empty, the function returns immediately with no side effects. If [want](#) is nonempty, the subprocess [cmd --version](#) is run with [capture_output=True, text=True, timeout=10](#). If an [OSError](#) or [subprocess.SubprocessError](#) is raised, it is caught and the function returns without logging. If the subprocess completes successfully, its stdout is stripped into [got](#). If [got](#) is nonempty and [got != want](#), then [logging.warning](#) is called with the specific message containing [got](#) and [want](#); otherwise, no warning is logged. No other output or state change occurs.

Formally: Let [want = settings.codegraph.version.strip\(\).removeprefix\('v'\)](#). Post-condition: return = None (want = " no warning logged subprocess not executed) (want "

((subprocess.run raises [OSError](#) [subprocess.SubprocessError](#)) no warning logged) (no such exception occurs let [got = subprocess_result.stdout.strip\(\)](#) in ((got " got want) [warning_logged_with\(got, want\)](#))))

- **Code evidence**: Line 15: if got and got != want:
- **Trigger condition**: The code's condition requires [got](#) to be truthy before checking inequality, so it skips the warning when the executable reports an empty version string. The specification demands a warning whenever the reported version differs, even if it is empty.
- **Validator trigger**: When the codegraph executable reports an empty version string (got="), the condition 'if got and got != want' short-circuits on falsy got, skipping the spec-required warning for version mismatch.
- **Probe stdout**: CONFIRMED — bug reproduced: no warning logged when executable reports empty version string, but spec requires warning when versions differ want: '1.0.0' (from CODEGRAPH_VERSION=v1.0.0) got: " (empty, from mocked subprocess.run stdout='\n') Root cause: 'if got and got != want' short-circuits on falsy got before checking inequality

► SPEC sidecar (完整)

```
{
  "signature": "_warn_on_codegraph_version_mismatch(cmd: str) -> None",
  "pre_condition": "cmd is a non-empty string representing a codegraph executable path; the configuration key codegraph.version is set.",
  "post_condition": "If the configured codegraph.version (after stripping whitespace and removing any leading 'v') is empty, returns immediately with no side effects. If the executable at cmd cannot be reached or fails to report its version, returns silently with no side effects. If the version reported by the executable differs from the configured version, a warning is logged containing both version strings. The function never raises an exception and never blocks or terminates the calling process."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

```
{
  "post_condition": "The function returns None. No exception is propagated. The global state (including
`settings.codegraph.version`) is unchanged. If `want` (computed as `settings.codegraph.version.strip().removeprefix('v')`) is
empty, the function returns immediately with no side effects. If `want` is non-empty, the subprocess `cmd --version` is run with
`capture_output=True`, `text=True`, `timeout=10`. If an `OSError` or `subprocess.SubprocessError` is raised, it is caught and the
function returns without logging. If the subprocess completes successfully, its stdout is stripped into `got`. If `got` is
non-empty and `got != want`, then `logging.warning` is called with the specific message containing `got` and `want`; otherwise, no
warning is logged. No other output or state change occurs.\n\nFormally:\nLet `want` =
settings.codegraph.version.strip().removeprefix('v')\n\nPost-condition:\n  return = None\n  ^ (want = '' => no warning logged ^
subprocess not executed)\n  ^ (want != '' => ( subprocess.run raises OSError v subprocess.SubprocessError)\n              => no
warning logged )\n  ^ ( no such exception occurs => let `got` = subprocess_result.stdout.strip() in\n              (
```

```
(got != '' ^ got != want) == warning_logged_with(got, want) )\n    )\n    )"
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_2a46b5c352df41f1bc920481c0a52a89_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_2a46b5c352df41f1bc920481c0a52a89_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_2a46b5c352df41f1bc920481c0a52a89_response.txt)

事件 2 — llm_576593e0e03945a6ba3757a29ab8b252

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Configured version is 'v1.0.0' (want='1.0.0'). The executable `cmd` runs successfully with empty stdout, so `got=''`. The code executes without logging any warning, but the specification requires a warning because the reported empty version differs from the configured version.",
  "offending_statements": "Line 15: if got and got != want:",
  "reason": "The code's condition requires `got` to be truthy before checking inequality, so it skips the warning when the executable reports an empty version string. The specification demands a warning whenever the reported version differs, even if it is empty."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_576593e0e03945a6ba3757a29ab8b252_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_576593e0e03945a6ba3757a29ab8b252_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_576593e0e03945a6ba3757a29ab8b252_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-070 — src -- languages -- codegraph.py -- try_codegraph_init

- 四类结论：契约不确定
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator：confirmed；attempts 2
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / codegraph；原源码 src/languages/codegraph.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: If force is true, any pre-existing CodeGraph index directory at <proj_dir>/.codegraph/ is removed before attempting to build a new one. If force is false and the index database <proj_dir>/.codegraph/codegraph.db already exists, the function returns immediately without modifying the filesystem. Otherwise, the function invokes the 'codegraph init' subcommand with proj_dir as the working directory. The function never raises an exception: if the codegraph executable is not found on the system PATH, the function returns silently; if the 'init' subcommand exits with a non-zero status code, a warning is logged and the function returns without error.
- **Actual behavior**: After execution, the function has attempted to build or rebuild the codegraph index for proj_dir. No exception is raised; all internal errors are handled (silent return on missing codegraph, warning on nonzero exit). The exact effects depend on the initial state and the force parameter:
- Let db_path = os.path.join(os.path.join(proj_dir, '.codegraph'), 'codegraph.db').
- Let E_before = os.path.exists(db_path) initially.
- Let CMD_EXIST be true if the codegraph command (as returned by _codegraph_cmd()) is present and executable, otherwise false.
- Let INIT_SUCCEED be true when the subprocess [cmd, 'init'] runs and returns exit code 0, otherwise false (including when the command is missing).

Post-condition:

1. If E_before force: the function returns immediately. No filesystem changes occur. No output is printed. db_path still exists.
2. If (E_before force) E_before:
 - A message is printed:
 - If E_before force: "[Pipeline] Rebuilding codegraph index for current working tree..." is printed, and the directory containing db_path is removed (codegraph_dir is deleted via shutil.rmtree with ignore_errors=True).
 - If E_before: "[Pipeline] Building codegraph index..." is printed, no prior removal is done.

- The function `_warn_on_codegraph_version_mismatch(cmd)` is called, which may emit a versionmismatch warning.
- Then `subprocess.run([cmd, 'init'], cwd=proj_dir, capture_output=True, text=True)` is attempted.
- If `CMD_EXIST` is false (`FileNotFoundError`), the function returns silently; no further output is produced.
- If `CMD_EXIST`:
 - If `INIT_SUCCEED`: "[Pipeline] codegraph index built." is printed.
 - If `INIT_SUCCEED`: a warning is logged with the first 300 characters of `stderr`.

Formally, the final existence of `db_path` is given by: `exists(db_path) (E_before force) (INIT_SUCCEED CMD_EXIST)`.

In particular, the database file is guaranteed to be present after the call exactly in the following situations:

- The database already existed and `force` is false, or
- The codegraph command was available and its `init` subcommand succeeded (whether or not a prior directory was removed).

All printed output is on `stdout`; warnings are emitted through the logging module.

- **Code evidence:** Line 19: `if os.path.exists(db_path):`
- **Trigger condition:** The specification requires that when `force` is `True`, any pre-existing `.codegraph/` directory is removed before building. The code only removes the directory when `codegraph.db` exists (Line 19). If `.codegraph/` exists without `codegraph.db`, the code does not remove it, violating the specification.
- **Validator trigger:** When `.codegraph/` directory exists without `codegraph.db` inside, `force=True` does not remove it (spec requires removal of any pre-existing directory).
- **Probe stdout:** [Pipeline] Building codegraph index... WARNING:root:codegraph '1.3.0-fmagent.1' does not match the pinned '1.5.0-fmagent.1' (fm-agent.toml [codegraph].version); re-run install.sh to update. [Pipeline] codegraph index built. CONFIRMED — .codegraph/ dir survived force=True rebuild (expected: removed by spec, actual: still present). `db_exists_before=False`, `cgdir_exists_before=True`, `cgdir_exists_after=True`, `db_exists_after=True`

► SPEC sidecar (完整)

```
{
  "signature": "try_codegraph_init(proj_dir: str, force: bool = True) -> None",
  "pre_condition": "proj_dir is a string representing an existing filesystem directory.",
  "post_condition": "If force is true, any pre-existing CodeGraph index directory at <proj_dir>/codegraph/ is removed before attempting to build a new one. If force is false and the index database <proj_dir>/codegraph/codegraph.db already exists, the function returns immediately without modifying the filesystem. Otherwise, the function invokes the 'codegraph init' subcommand with proj_dir as the working directory. The function never raises an exception: if the codegraph executable is not found on the system PATH, the function returns silently; if the 'init' subcommand exits with a non-zero status code, a warning is logged and the function returns without error."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/try_codegraph_init.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "If force is true, any pre-existing CodeGraph index directory at <proj_dir>/codegraph/ is removed before attempting to build a new one. If force is false and the index database <proj_dir>/codegraph/codegraph.db already exists, the function returns immediately without modifying the filesystem. Otherwise, the function invokes the 'codegraph init' subcommand with proj_dir as the working directory. The function never raises an exception: if the codegraph executable is not found on the system PATH, the function returns silently; if the 'init' subcommand exits with a non-zero status code, a warning is logged and the function returns without error.",
    "actual_behavior": "After execution, the function has attempted to build or rebuild the codegraph index for proj_dir. No exception is raised; all internal errors are handled (silent return on missing codegraph, warning on nonzero exit). The exact effects depend on the initial state and the force parameter:\n\n- Let db_path = os.path.join(os.path.join(proj_dir, '.codegraph'), 'codegraph.db').\n- Let E_before = os.path.exists(db_path) initially.\n- Let CMD_EXIST be true if the codegraph command (as returned by _codegraph_cmd()) is present and executable, otherwise false.\n- Let INIT_SUCCEED be true when the subprocess `[cmd, 'init']` runs and returns exit code 0, otherwise false (including when the command is missing).\n\nPost-condition:\n1. If E_before force: the function returns immediately. No filesystem changes occur. No output is printed. db_path still exists.\n2. If (E_before force) E_before:\n   - A message is printed:\n       * If E_before force: \"[Pipeline] Rebuilding codegraph index for current working tree...\" is printed, and the directory containing db_path is removed (codegraph_dir is deleted via `shutil.rmtree` with `ignore_errors=True`).\n       * If E_before: \"[Pipeline] Building codegraph index...\" is printed, no prior removal is done.\n   - The function `_warn_on_codegraph_version_mismatch(cmd)` is called, which may emit a versionmismatch warning.\n   - Then
```

```
` subprocess.run([cmd, 'init'], cwd=proj_dir, capture_output=True, text=True)` is attempted.\n    - If CMD_EXIST is false (FileNotFoundError), the function returns silently; no further output is produced.\n    - If CMD_EXIST:\n        * If INIT_SUCCEEDED:\n            \"[Pipeline] codegraph index built.\" is printed.\n        * If INIT_SUCCEEDED: a warning is logged with the first 300 characters of stderr.\n\nFormally, the final existence of db_path is given by:\n    exists(db_path)  $\Leftrightarrow$  (E_before  $\wedge$  force)  $\vee$  (INIT_SUCCEEDED  $\wedge$  CMD_EXIST).\n\nIn particular, the database file is guaranteed to be present after the call exactly in the following situations:\n    - The database already existed and force is false, or\n    - The codegraph command was available and its `init` subcommand succeeded (whether or not a prior directory was removed).\n\nAll printed output is on stdout; warnings are emitted through the logging module.",\n    "code_evidence": "Line 19: if os.path.exists(db_path):",\n    "trigger_condition": "The specification requires that when force is True, any pre-existing `.codegraph/` directory is removed before building. The code only removes the directory when `codegraph.db` exists (Line 19). If `.codegraph/` exists without `codegraph.db`, the code does not remove it, violating the specification." }\n}
```

► Bug Validator result (完整)

```
{\n  "id": "src--languages--codegraph-py--try_codegraph_init",\n  "source_file": "/home/fancy/Projects/Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/try_codegraph_init.py",\n  "function_name": "try_codegraph_init",\n  "confirmation_status": "confirmed",\n  "attempts": 2,\n  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--try_codegraph_init.py",\n  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--try_codegraph_init.md",\n  "probe_stdout": "[Pipeline] Building codegraph index...\nWARNING:root:codegraph '1.3.0-fmagent.1' does not match the pinned '1.5.0-fmagent.1' (fm-agent.toml [codegraph].version); re-run install.sh to update.\n[Pipeline] codegraph index built.\nCONFIRMED\n.codegraph/ dir survived force=True rebuild (expected: removed by spec, actual: still present).\ndb_exists_before=False,\ncgdir_exists_before=True, cgdir_exists_after=True, db_exists_after=True",\n  "trigger_summary": "When .codegraph/ directory exists without codegraph.db inside, force=True does not remove it (spec requires removal of any pre-existing directory)."\n}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:34:17.847918Z	3927
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:35:12.412468Z	6054

事件 1 — [llm_3623b0a19ca94dcb86a57ffb018ce69c](#)

► parsed_json (完整)

```
{\n  "post_condition": "After execution, the function has attempted to build or rebuild the codegraph index for proj_dir. No exception is raised; all internal errors are handled (silent return on missing codegraph, warning on non-zero exit). The exact effects depend on the initial state and the force parameter:\n\n- Let db_path = os.path.join(os.path.join(proj_dir, '.codegraph'), 'codegraph.db').\n- Let E_before = os.path.exists(db_path) initially.\n- Let CMD_EXIST be true if the codegraph command (as returned by `_codegraph_cmd()`) is present and executable, otherwise false.\n- Let INIT_SUCCEEDED be true when the subprocess `[cmd, 'init']` runs and returns exit code 0, otherwise false (including when the command is missing).\n\nPost-condition:\n1. If E_before  $\wedge$  force: the function returns immediately. No filesystem changes occur. No output is printed. db_path still exists.\n2. If (E_before  $\wedge$  force)  $\vee$   $\neg$ E_before:\n    - A message is printed:\n        * If E_before  $\wedge$  force: \"[Pipeline] Rebuilding codegraph index for current working tree...\" is printed, and the directory containing db_path is removed (codegraph_dir is deleted via `shutil.rmtree` with `ignore_errors=True`).\n        * If  $\neg$ E_before: \"[Pipeline] Building codegraph index...\" is printed, no prior removal is done.\n    - The function `_warn_on_codegraph_version_mismatch(cmd)` is called, which may emit a version-mismatch warning.\n    - Then `subprocess.run([cmd, 'init'], cwd=proj_dir, capture_output=True, text=True)` is attempted.\n    - If CMD_EXIST is false (FileNotFoundError), the function returns silently; no further output is produced.\n    - If CMD_EXIST:\n        * If INIT_SUCCEEDED:\n            \"[Pipeline] codegraph index built.\" is printed.\n        * If  $\neg$ INIT_SUCCEEDED: a warning is logged with the first 300 characters of stderr.\n\nFormally, the final existence of db_path is given by:\n    exists(db_path)  $\Leftrightarrow$  (E_before  $\wedge$  force)  $\vee$  (INIT_SUCCEEDED  $\wedge$  CMD_EXIST).\n\nIn particular, the database file is guaranteed to be present after the call exactly in the following situations:\n    - The database already existed and force is false, or\n    - The codegraph command was available and its `init` sub-command succeeded (whether or not a prior directory was removed).\n\nAll printed output is on stdout; warnings are emitted through the logging module." }\n}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_3623b0a19ca94dcb86a57ffb018ce69c_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_3623b0a19ca94dcb86a57ffb018ce69c_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_3623b0a19ca94dcb86a57ffb018ce69c_response.txt)

事件 2 — [llm_b30729f439174a43813d172dbb643773](#)


```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/tmp/testproj'; inside it, '.codegraph' directory exists but contains no 'codegraph.db' file;
force = True",
  "offending_statements": "Line 19: if os.path.exists(db_path):",
  "reason": "The specification requires that when force is True, any pre-existing `.codegraph/` directory is removed before
building. The code only removes the directory when `codegraph.db` exists (Line 19). If `.codegraph/` exists without
`codegraph.db`, the code does not remove it, violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b30729f439174a43813d172dbb643773_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_b30729f439174a43813d172dbb643773_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_b30729f439174a43813d172dbb643773_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块：extraction

FMA-MISMATCH-075 — src--extract-py--_function_spans

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / [extraction](#); 原源码 [src/extract.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a pair (spans, raw_lines). raw_lines is a list of strings containing every line of the file, each with its trailing newline character preserved; the file is decoded with replacement characters on decoding error. spans is a list of (name, start_idx, end_idx) tuples where start_idx and end_idx are 0-based inclusive line indices delimiting each function definition found in the file. Each name is the function's canonicalized identifier free of characters unsafe for filesystem paths and FQNs with a disambiguation suffix appended when multiple distinct functions share the same canonicalized name: the first occurrence retains the bare canonicalized name, while each subsequent occurrence appends '_N' where N is the 1-based count of prior occurrences of that canonicalized name. The ordering of spans reflects the order in which function definitions appear in the source file.
- **Actual behavior**: After successful execution (no uncaught exceptions), the function returns a tuple (spans, raw_lines) such that:
 1. raw_lines = list of strings, each representing a line from the file at [filepath](#), read via [f.readlines\(\)](#) with [errors='replace'](#). Newline characters are preserved exactly as present in the file.
 2. Let [norm_lines](#) = [l.rstrip("\n").rstrip("\r") for l in raw_lines], [lang_cfg](#) = LANG_CONFIG[lang_key].
 3. The detection source for function boundaries is determined as:
 - If [proj_dir](#) is not None and [function_spans_for_file\(proj_dir, filepath, lang_key\)](#) returns a list (i.e. not None), that list is used as [raw_funcs](#).
 - Otherwise, if the return value from the codegraph call is None (or if [proj_dir](#) is None), then:
 - if [lang_cfg\['body'\]](#) == 'brace': [raw_funcs](#) = [_extract_functions_brace\(norm_lines, lang_key, lang_cfg\)](#)
 - else: [raw_funcs](#) = [_extract_functions_indent\(norm_lines, lang_cfg\)](#). [raw_funcs](#) is a list of triples (name, start_idx, end_idx) with 0based inclusive line indices, possibly empty if no functions are found.
 4. Let [name_counts](#) be an initially empty dictionary, and [spans](#) an empty list. For each (name, start, end) in [raw_funcs](#) in order: a. [cname](#) = canonicalize(name), which replaces all '/' characters with '_'. b. [count](#) = current value of [name_counts\[cname\]](#) (0 if not present). c. If [count](#) == 0, then [deduped](#) = [cname](#); otherwise [deduped](#) = f'{cname}_{count}'. d. Append (deduped, start, end) to [spans](#). e. [name_counts\[cname\]](#) = count + 1.
 5. The function returns (spans, raw_lines), where [spans](#) is the list of deduplicated named ranges and [raw_lines](#) the unmodified file lines.

Formally, let [DETECT](#) be the overall function extraction (as in step 3) that yields a sequence of m triples ((n_i, s_i, e_i)). For (i=1 \dots m), define (C_i = \texttt{canonicalize}(n_i)), (k_i = |\{ j < i \mid \texttt{canonicalize}(n_j) = C_i \}|). Then (\texttt{spans} = [(D_i, s_i, e_i) \mid i=1 \dots m, j) where (D_i = C_i) if (k_i = 0), else (D_i = C_i + _! + \texttt{str}(k_i)). (\texttt{raw_lines}) is the list of lines as described. The returned

tuple is ((texttt{spans}, texttt{raw_lines})). No exceptions are raised because the preconditions guarantee readable file, valid language configuration, and valid backup/fallback extraction functions.

- **Code evidence:** Line 15: with open(filepath, "r", errors="replace") as f:
- **Trigger condition:** The code opens the file in default text mode without newline="", causing universal newline translation. Consequently, raw_lines will have '\n' line endings instead of the file's original '\r\n' endings, violating the requirement that each line preserve its trailing newline character.
- **Validator trigger:** Opening file in text mode without newline="" causes universal newline translation, converting \r\n to \n in raw_lines.
- **Probe stdout:** CONFIRMED — raw_lines lost \r\n (universal newline translation): ["class Foo:\n", "" def bar(self):\n", "" pass\n"]

► SPEC sidecar (完整)

```
{
  "signature": "_function_spans(filepath: str, lang_key: str, proj_dir: str | None = None) -> tuple[list[tuple[str, int, int]], list[str]]",
  "pre_condition": "filepath refers to a readable source file. lang_key is a key present in LANG_CONFIG. proj_dir is None or a path to a project root directory.",
  "post_condition": "Returns a pair (spans, raw_lines). raw_lines is a list of strings containing every line of the file, each with its trailing newline character preserved; the file is decoded with replacement characters on decoding error. spans is a list of (name, start_idx, end_idx) tuples where start_idx and end_idx are 0-based inclusive line indices delimiting each function definition found in the file. Each name is the function's canonicalized identifier – free of characters unsafe for filesystem paths and FQNs – with a disambiguation suffix appended when multiple distinct functions share the same canonicalized name: the first occurrence retains the bare canonicalized name, while each subsequent occurrence appends '_N' where N is the 1-based count of prior occurrences of that canonicalized name. The ordering of spans reflects the order in which function definitions appear in the source file."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "function_spans_for_file",
      "signature": "function_spans_for_file(proj_dir: str, filepath: str, lang_key: str) -> list[tuple[str, int, int]] | None",
      "pre_condition": "proj_dir is a valid project root directory path. filepath refers to a source file. lang_key is a registered language key.",
      "post_condition": "Returns a list of (name, start_idx, end_idx) tuples representing function definitions found in the file by the codegraph index, with start_idx and end_idx as 0-based inclusive line indices. Returns None when the index does not cover this file."
    },
    {
      "name": "_extract_functions_brace",
      "signature": "_extract_functions_brace(norm_lines: list[str], lang_key: str, lang_cfg: dict) -> list[tuple[str, int, int]]",
      "pre_condition": "norm_lines is a list of newline-stripped source code lines from a file whose language uses brace-delimited blocks. lang_key is a registered language key. lang_cfg is the configuration dictionary for that language.",
      "post_condition": "Returns a list of (name, start_idx, end_idx) tuples for each function definition found via brace-matching, with start_idx and end_idx as 0-based inclusive line indices. Returns an empty list if no function definitions are found."
    },
    {
      "name": "_extract_functions_indent",
      "signature": "_extract_functions_indent(norm_lines: list[str], lang_cfg: dict) -> list[tuple[str, int, int]]",
      "pre_condition": "norm_lines is a list of newline-stripped source code lines from a file whose language uses indentation-delimited blocks. lang_cfg is the configuration dictionary for that language.",
      "post_condition": "Returns a list of (name, start_idx, end_idx) tuples for each function definition found via indentation matching, with start_idx and end_idx as 0-based inclusive line indices. Returns an empty list if no function definitions are found."
    },
    {
      "name": "canonicalize",
      "signature": "canonicalize(name: str) -> str",
      "pre_condition": "name is a string representing a function name or identifier that may contain characters unsafe for filesystem paths or FQNs.",
      "post_condition": "Returns a string where all occurrences of '/' have been translated to '_'. The result is safe for use as a filesystem path component and as a segment in a ':'-delimited FQN."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/_function_spans.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a pair (spans, raw_lines). raw_lines is a list of strings containing every line of the file, each with
```

its trailing newline character preserved. The file is a list of (name, start_idx, end_idx) tuples where start_idx and end_idx are 0-based inclusive line indices delimiting each function definition found in the file. Each name is the function's canonicalized identifier free of characters unsafe for filesystem paths and FQNs with a disambiguation suffix appended when multiple distinct functions share the same canonicalized name: the first occurrence retains the bare canonicalized name, while each subsequent occurrence appends '_N' where N is the 1-based count of prior occurrences of that canonicalized name. The ordering of spans reflects the order in which function definitions appear in the source file.",

"actual_behavior": "After successful execution (no uncaught exceptions), the function returns a tuple (spans, raw_lines) such that:\n\n1. raw_lines = list of strings, each representing a line from the file at `filepath`, read via `f.readlines()` with `errors='replace'`. Newline characters are preserved exactly as present in the file.\n\n2. Let `norm\_lines` = [l.rstrip('\n').rstrip('\r') for l in raw_lines], `lang\_cfg` = LANG_CONFIG[lang_key].\n\n3. The detection source for function boundaries is determined as:\n - If `proj\_dir` is not None and `function\_spans\_for\_file(proj\_dir, filepath, lang\_key)` returns a list (i.e. not `None`), that list is used as `raw\_funcs`.\n - Otherwise, if the return value from the codegraph call is `None` (or if `proj\_dir` is None), then:\n - if `lang\_cfg['body'] == 'brace': `raw_funcs` = \_extract\_functions\_brace(norm\_lines, lang\_key, lang\_cfg)\n - else: `raw_funcs` = \_extract\_functions\_indent(norm\_lines, lang\_cfg).\n `raw_funcs` is a list of triples `(name, start_idx, end_idx)` with 0-based inclusive line indices, possibly empty if no functions are found.\n\n4. Let `name_counts` be an initially empty dictionary, and `spans` an empty list. For each `(name, start, end)` in `raw_funcs` in order:\n a. `cname` = canonicalize(name), which replaces all '/' characters with '\_'.\n b. `count` = current value of `name_counts[cname]` (0 if not present).\n c. If `count == 0`, then `deduped` = `cname`; otherwise `deduped` = `f'{cname}_{count}'`.\n d. Append `(deduped, start, end)` to `spans`.\n e. `name_counts[cname]` = `count + 1`.\n\n5. The function returns `(spans, raw_lines)`, where `spans` is the list of deduplicated named ranges and `raw_lines` the unmodified file lines.\n\nFormally, let `DETECT` be the overall function extraction (as in step 3) that yields a sequence of m triples \(\ (n_i, s_i, e_i) \). For \(\ (i=1 \dots m) \), define \(\ (C_i = \text{canonicalize}(n_i) \), \(\ (k_i = |\{ j < i \mid \text{canonicalize}(n_j) = C_i \}| \). Then \(\ (\text{spans} = [\, (D_i, s_i, e_i) \mid i=1 \dots m \,]) \) where \(\ (D_i = C_i \) if \(\ (k_i = 0 \), else \(\ (D_i = C_i + _ \) + \(\ (\text{str}(k_i) \). \(\ (\text{raw_lines}) \) is the list of lines as described. The returned tuple is \(\ ((\text{spans}), \text{raw_lines}) \). No exceptions are raised because the preconditions guarantee readable file, valid language configuration, and valid backup/fallback extraction functions.",

"code_evidence": "Line 15: with open(filepath, "r", errors="replace") as f:",

"trigger_condition": "The code opens the file in default text mode without newline='', causing universal newline translation. Consequently, raw_lines will have '\n' line endings instead of the file's original '\r\n' endings, violating the requirement that each line preserve its trailing newline character."

```
}
}
```

► Bug Validator result (完整)

```
{
  "id": "src--extract-py--_function_spans",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/_function_spans.py",
  "function_name": "_function_spans",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--extract-py--_function_spans.py",
  "detail_file": "fm_agent/bug_validation/src--extract-py--_function_spans.md",
  "probe_stdout": "CONFIRMED - raw_lines lost \r\n (universal newline translation): [\"'class Foo:\\n\\n', '\" def bar(self):\\n\\n', '\" pass\\n\\n']",
  "trigger_summary": "Opening file in text mode without newline='' causes universal newline translation, converting \r\n to \n in raw_lines."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:35:39.624708Z	4159
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:36:25.514646Z	6908

事件 1 — [llm_969a515fb36f43feb7c58ea591d15985](#)

► parsed_json (完整)

```
{
  "post_condition": "After successful execution (no uncaught exceptions), the function returns a tuple (spans, raw_lines) such that:\n\n1. raw_lines = list of strings, each representing a line from the file at `filepath`, read via `f.readlines()` with `errors='replace'`. Newline characters are preserved exactly as present in the file.\n\n2. Let `norm_lines` = [l.rstrip('\n').rstrip('\r') for l in raw_lines], `lang_cfg` = LANG_CONFIG[lang_key].\n\n3. The detection source for function boundaries is determined as:\n    - If `proj_dir` is not None and `function_spans_for_file(proj_dir, filepath, lang_key)` returns a list (i.e. not `None`), that list is used as `raw_funcs`.\n    - Otherwise, if the return value from the codegraph call is `None` (or if `proj_dir` is None), then:\n        - if `lang_cfg['body'] == 'brace': `raw_funcs` = _extract_functions_brace(norm_lines, lang_key, lang_cfg)\n        - else: `raw_funcs` = _extract_functions_indent(norm_lines, lang_cfg).\n    `raw_funcs` is a list of triples `(name, start_idx, end_idx)` with 0-based inclusive line indices, possibly empty if no functions are found.\n\n4. Let `name_counts` be an initially empty dictionary, and `spans` an empty list. For each `(name, start, end)` in `raw_funcs` in order:\n    a. `cname` = canonicalize(name), which replaces all '/' characters with '_'.\n    b. `count` = current value of `name_counts[cname]` (0 if not present).\n    c. If `count == 0`, then `deduped` = `cname`; otherwise `deduped` = `f'{cname}_{count}'`.\n    d. Append `(deduped, start, end)` to `spans`.\n    e. `name_counts[cname]` = `count + 1`.\n\n5. The function returns `(spans, raw_lines)`, where `spans` is the list of deduplicated named ranges and `raw_lines` the unmodified file
```

```
lines.\n\nFormally, let `DETECT` be the overall function extraction (as in step 3) that yields a sequence of m triples  $((n_i, s_i, e_i))$ . For  $(i=1 \dots m)$ , define  $(C_i = \text{canonicalize}(n_i))$ ,  $(k_i = |\{j < i \mid \text{canonicalize}(n_j) = C_i\}|)$ . Then  $((\text{spans} = [(), (D_i, s_i, e_i) \mid i=1 \dots m],))$  where  $(D_i = C_i)$  if  $(k_i = 0)$ , else  $(D_i = C_i + \_! + \text{str}(k_i))$ .  $(\text{raw\_lines})$  is the list of lines as described. The returned tuple is  $((\text{spans}, \text{raw\_lines}))$ . No exceptions are raised because the pre-conditions guarantee readable file, valid language configuration, and valid backup/fallback extraction functions."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_969a515fb36f43feb7c58ea591d15985_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_969a515fb36f43feb7c58ea591d15985_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_969a515fb36f43feb7c58ea591d15985_response.txt)

事件 2 — llm_4136e42fb41545a18760b49fe8dceed7

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "filepath points to a file containing 'def foo():\r\n    pass\r\n', lang_key='python', proj_dir=None. The file has CRLF line endings.",
  "offending_statements": "Line 15: with open(filepath, \"r\", errors=\"replace\") as f:",
  "reason": "The code opens the file in default text mode without newline='\\n', causing universal newline translation. Consequently, raw_lines will have '\\n' line endings instead of the file's original '\\r\\n' endings, violating the requirement that each line preserve its trailing newline character."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4136e42fb41545a18760b49fe8dceed7_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4136e42fb41545a18760b49fe8dceed7_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4136e42fb41545a18760b49fe8dceed7_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: language_handlers

FMA-MISMATCH-081 — src--languages--cpp-py--call_edges

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / language_handlers; 原源码 src/languages/cpp.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim:** When a CodeGraph index can be initialized for the project, returns a dict mapping each caller's fully-qualified name to a set of callee fully-qualified names for C++ source files within the project. When CodeGraph initialization is not possible, returns None.
- Actual behavior:** The result is None if CodeGraphExtractor.from_proj_dir(proj_dir) is None or if the returned extractor's get_call_edges('cpp') method returns None. Otherwise, the result is a dictionary mapping each caller identifier (represented as a tuple of (caller_stem, caller_module)) to a set of callee stems (strings). Formally, let cg = CodeGraphExtractor.from_proj_dir(proj_dir). Then (cg is None (cg is not None cg.get_call_edges('cpp') is None)) result is None. Otherwise, result = cg.get_call_edges('cpp') and is a dict where each key is a tuple[str, str] and each value is a set[str].
- Code evidence:** Line 4: return cg.get_call_edges("cpp") if cg else None
- Trigger condition:** The specification requires a dict when CodeGraph initialization is successful. The code returns None if get_call_edges('cpp') returns None (e.g., C++ not indexed), violating the spec's guarantee of a dict in that case.
- Validator trigger:** When CodeGraph init succeeds but get_call_edges('cpp') returns None, the code returns None instead of the spec-required dict.
- Probe stdout:** CONFIRMED — actual: None | expected: {}

► SPEC sidecar (完整)

```
{
  "signature": "call_edges(proj_dir: str) -> dict | None",
  "pre_condition": "proj_dir is a string path to a project directory",
  "post_condition": "When a CodeGraph index can be initialized for the project, returns a dict mapping each caller's fully-qualified name to a set of callee fully-qualified names for C++ source files within the project. When CodeGraph initialization is
```

```
not possible, returns None."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "CodeGraphExtractor.from_proj_dir",
      "signature": "CodeGraphExtractor.from_proj_dir(proj_dir: str) -> CodeGraphExtractor | None",
      "pre_condition": "proj_dir is a string path to a project directory",
      "post_condition": "Returns a CodeGraphExtractor instance initialized with the project's source index when the project can be indexed, or None when indexing is unavailable"
    },
    {
      "name": "CodeGraphExtractor.get_call_edges",
      "signature": "CodeGraphExtractor.get_call_edges(language: str) -> dict | None",
      "pre_condition": "The CodeGraphExtractor instance is initialized and non-None; language is a valid language key string",
      "post_condition": "Returns a dict mapping each caller's fully-qualified name to a set of callee fully-qualified names for the specified language, or None when the language is not indexed"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/cpp-py/call_edges.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When a CodeGraph index can be initialized for the project, returns a dict mapping each caller's fully-qualified name to a set of callee fully-qualified names for C++ source files within the project. When CodeGraph initialization is not possible, returns None.",
    "actual_behavior": "The result is None if CodeGraphExtractor.from_proj_dir(proj_dir) is None or if the returned extractor's get_call_edges('cpp') method returns None. Otherwise, the result is a dictionary mapping each caller identifier (represented as a tuple of (caller_stem, caller_module)) to a set of callee stems (strings). Formally, let cg = CodeGraphExtractor.from_proj_dir(proj_dir). Then (cg is None (cg is not None cg.get_call_edges('cpp') is None)) result is None. Otherwise, result = cg.get_call_edges('cpp') and is a dict where each key is a tuple[str, str] and each value is a set[str].",
    "code_evidence": "Line 4: return cg.get_call_edges(\"cpp\") if cg else None",
    "trigger_condition": "The specification requires a dict when CodeGraph initialization is successful. The code returns None if get_call_edges('cpp') returns None (e.g., C++ not indexed), violating the spec's guarantee of a dict in that case."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--cpp-py--call_edges",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/cpp-py/call_edges.py",
  "function_name": "call_edges",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--languages--cpp-py--call_edges.py",
  "detail_file": "fm_agent/bug_validation/src--languages--cpp-py--call_edges.md",
  "probe_stdout": "CONFIRMED - actual: None | expected: {}",
  "trigger_summary": "When CodeGraph init succeeds but get_call_edges('cpp') returns None, the code returns None instead of the spec-required dict."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:33:36.973281Z	2103
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:34:00.623567Z	2022

事件 1 — [llm_9969b326e53c4eb690ddb9c5c92306c1](#)

► parsed_json (完整)

```
{
  "post_condition": "The result is None if CodeGraphExtractor.from_proj_dir(proj_dir) is None or if the returned extractor's
get_call_edges('cpp') method returns None. Otherwise, the result is a dictionary mapping each caller identifier (represented as a
tuple of (caller_stem, caller_module)) to a set of callee stems (strings). Formally, let cg =
CodeGraphExtractor.from_proj_dir(proj_dir). Then (cg is None v (cg is not None ^ cg.get_call_edges('cpp') is None)) => result is
None. Otherwise, result = cg.get_call_edges('cpp') and is a dict where each key is a tuple[str, str] and each value is a
set[str]."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_9969b326e53c4eb690ddb9c5c92306c1_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_9969b326e53c4eb690ddb9c5c92306c1_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_9969b326e53c4eb690ddb9c5c92306c1_response.txt)

事件 2 — [llm_a75585ad7b93425a9b36094f46884316](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/path/to/pure/python/project' where the project has no C++ files, so from_proj_dir returns a valid
CodeGraphExtractor, but get_call_edges('cpp') returns None.",
  "offending_statements": "Line 4: return cg.get_call_edges(\"cpp\") if cg else None",
  "reason": "The specification requires a dict when CodeGraph initialization is successful. The code returns None if
get_call_edges('cpp') returns None (e.g., C++ not indexed), violating the spec's guarantee of a dict in that case."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_a75585ad7b93425a9b36094f46884316_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_a75585ad7b93425a9b36094f46884316_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_a75585ad7b93425a9b36094f46884316_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-083 — [src--languages--erlang-py--ElpClient::_handle_server_message](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts [1](#)
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 原源码 [src/languages/erlang.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: When message has a 'method' key with value 'elp/status' and a 'params' key, self._status is updated to the value of params['status'] (or a falsy default if 'status' is absent from params). When message has both an 'id' key and a truthy 'method' key, a JSON-RPC 2.0 response is sent via self._send with: 'jsonrpc' set to '2.0', 'id' set to message['id'], and 'result' determined by the method for 'workspace/configuration', result is a list of None values whose length equals the count of entries in params['items'] (or a zero-length list if 'items' is absent from params); for 'workspace/workspaceFolders', result is a list containing a single dict with 'uri' set to self.root_uri and 'name' set to the last path component of self.proj_dir; for 'workspace/applyEdit', result is {'applied': False}; for any other method, result is None. When message lacks an 'id' key or lacks a truthy 'method' key, no response is sent. The function returns None.
- **Actual behavior**: Natural language: After method execution, if the message 'method' is 'elp/status', self._status is updated to the value associated with 'status' in params (or None if not present), where params defaults to {} if absent. If the message contains an 'id' key and a truthy 'method' value, a JSON-RPC response is sent via self._send with 'jsonrpc': '2.0', 'id': message['id'], and 'result' computed as: if method=='workspace/configuration' then a list of None of length equal to len(items) with items from params (or [] if absent); if method=='workspace/workspaceFolders' then [{uri:self.root_uri, name:os.path.basename(self.proj_dir)}]; if method=='workspace/applyEdit' then {'applied': False}; otherwise None. If the message lacks an 'id' or the method is falsey, no response is sent. No other attributes are modified, and no exceptions are raised. Formal: (message.method = 'elp/status' self._status = (params.status if paramsdict else None)) ((message.method = 'elp/status') self._status = self._status_pre) ((id:message.message.id None message.method None message.method) r : (r = case message.method of 'workspace/configuration': [None | _ items] where items = params.get('items', []); 'workspace/workspaceFolders': [{uri:self.root_uri, name:os.path.basename(self.proj_dir)}]; 'workspace/applyEdit': {'applied': False}; other: None) self._send({'jsonrpc': '2.0', 'id': message.id, 'result': r}) ((id:message.message.method truthy) m s.t. self._send(m))
- **Code evidence**: Line 4: if params is None: Line 5: params = {} Line 7: self._status = params.get("status")
- **Trigger condition**: The specification requires self._status to be updated only when the message contains a 'params' key. The code defaults a missing 'params' to an empty dict, causing self._status to be unconditionally set to None (or another falsy value) even when no

params' key is present, which violates the specification.

- **Validator trigger:** Send a server message with method='elp/status' but without a 'params' key — self._status is overwritten to None instead of being left unchanged.
- **Probe stdout:** CONFIRMED — _status was overwritten to None from 'INITIAL_SENTINEL' despite the message lacking a 'params' key. The specification requires _status to be updated ONLY when the message contains a 'params' key.

► SPEC sidecar (完整)

```
{
  "signature": "ElpClient._handle_server_message(self, message: dict) -> None",
  "pre_condition": "self is an ElpClient instance with an active connection to an ELP server and the attributes self.root_uri (a valid URI string), self.proj_dir (a valid directory path string), self._status (tracks server status), and self._send (a callable for sending JSON-RPC responses). message is a dict representing a parsed JSON-RPC message received from the ELP server.",
  "post_condition": "When message has a 'method' key with value 'elp/status' and a 'params' key, self._status is updated to the value of params['status'] (or a falsy default if 'status' is absent from params). When message has both an 'id' key and a truthy 'method' key, a JSON-RPC 2.0 response is sent via self._send with: 'jsonrpc' set to '2.0', 'id' set to message['id'], and 'result' determined by the method – for 'workspace/configuration', result is a list of None values whose length equals the count of entries in params['items'] (or a zero-length list if 'items' is absent from params); for 'workspace/workspaceFolders', result is a list containing a single dict with 'uri' set to self.root_uri and 'name' set to the last path component of self.proj_dir; for 'workspace/applyEdit', result is {'applied': False}; for any other method, result is None. When message lacks an 'id' key or lacks a truthy 'method' key, no response is sent. The function returns None."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "self._send",
      "signature": "self._send(message: dict) -> None",
      "pre_condition": "message is a JSON-RPC 2.0 response dict containing 'jsonrpc', 'id', and 'result' keys with values conforming to the JSON-RPC 2.0 specification.",
      "post_condition": "The JSON-RPC response represented by message is transmitted to the ELP server via the active connection."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::_handle_server_message.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When message has a 'method' key with value 'elp/status' and a 'params' key, self._status is updated to the value of params['status'] (or a falsy default if 'status' is absent from params). When message has both an 'id' key and a truthy 'method' key, a JSON-RPC 2.0 response is sent via self._send with: 'jsonrpc' set to '2.0', 'id' set to message['id'], and 'result' determined by the method for 'workspace/configuration', result is a list of None values whose length equals the count of entries in params['items'] (or a zero-length list if 'items' is absent from params); for 'workspace/workspaceFolders', result is a list containing a single dict with 'uri' set to self.root_uri and 'name' set to the last path component of self.proj_dir; for 'workspace/applyEdit', result is {'applied': False}; for any other method, result is None. When message lacks an 'id' key or lacks a truthy 'method' key, no response is sent. The function returns None.",
    "actual_behavior": "Natural language: After method execution, if the message 'method' is 'elp/status', self._status is updated to the value associated with 'status' in params (or None if not present), where params defaults to {} if absent. If the message contains an 'id' key and a truthy 'method' value, a JSON-RPC response is sent via self._send with 'jsonrpc': '2.0', 'id': message['id'], and 'result' computed as: if method=='workspace/configuration' then a list of None of length equal to len(items) with items from params (or [] if absent); if method=='workspace/workspaceFolders' then [{uri:self.root_uri, name:os.path.basename(self.proj_dir)}]; if method=='workspace/applyEdit' then {'applied': False}; otherwise None. If the message lacks an 'id' or the method is falsey, no response is sent. No other attributes are modified, and no exceptions are raised. Formal: (message.method = 'elp/status' self._status = (params.status if paramsdict else None)) ((message.method = 'elp/status') self._status = self._status_pre) (('id:message.id None message.method None message.method '' r : (r = case message.method of 'workspace/configuration': [None | _ items] where items = params.get('items', []); 'workspace/workspaceFolders': [{uri:self.root_uri, name:os.path.basename(self.proj_dir)}]; 'workspace/applyEdit': {'applied': False}; other: None) self._send({'jsonrpc': '2.0', 'id': message.id, 'result': r}) ) (('id:message.id None message.method truthy) m s.t. self._send(m))",
    "code_evidence": "Line 4: if params is None:\nLine 5: params = {}\nLine 7: self._status = params.get('status')",
    "trigger_condition": "The specification requires self._status to be updated only when the message contains a 'params' key. The code defaults a missing 'params' to an empty dict, causing self._status to be unconditionally set to None (or another falsy value) even when no 'params' key is present, which violates the specification."
  }
}
```

► Bug Validator result (完整)


```
{
  "id": "src--languages--erlang-py--ElpClient::_handle_server_message",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/languages/erlang-py/ElpClient::_handle_server_message.py",
  "function_name": "_handle_server_message",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--ElpClient::_handle_server_message.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--ElpClient::_handle_server_message.md",
  "probe_stdout": "CONFIRMED - _status was overwritten to None from 'INITIAL_SENTINEL' despite the message lacking a 'params' key.
The specification requires _status to be updated ONLY when the message contains a 'params' key.\n",
  "trigger_summary": "Send a server message with method='elp/status' but without a 'params' key - self._status is overwritten to
None instead of being left unchanged."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:55:03.636129Z	2396
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:55:31.905827Z	4613

事件 1 — llm_cae92a9feab04d42a37cfcf59b28493f

► parsed_json（完整）

```
{
  "post_condition": "Natural language: After method execution, if the message 'method' is 'elp/status', self._status is updated to
the value associated with 'status' in params (or None if not present), where params defaults to {} if absent. If the message
contains an 'id' key and a truthy 'method' value, a JSON-RPC response is sent via self._send with 'jsonrpc':'2.0',
'id':message['id'], and 'result' computed as: if method=='workspace/configuration' then a list of None of length equal to
len(items) with items from params (or [] if absent); if method=='workspace/workspaceFolders' then
[{'uri':self.root_uri,'name':os.path.basename(self.proj_dir)}]; if method=='workspace/applyEdit' then {'applied':False}; otherwise
None. If the message lacks an 'id' or the method is falsey, no response is sent. No other attributes are modified, and no
exceptions are raised. Formal: (message.method = 'elp/status' => self._status = (params.status if params!=dict else None)) ^
(¬(message.method = 'elp/status') => self._status = self._status_pre) ^ ((('id'∈message ^ message.id ≠ None ^ message.method ≠ None
^ message.method ≠ '' ) => ∃ r : (r = case message.method of 'workspace/configuration': [None | _ ∈ items] where items =
params.get('items',[ ]); 'workspace/workspaceFolders': [{ 'uri':self.root_uri,'name':os.path.basename(self.proj_dir)}];
'workspace/applyEdit': {'applied':False}; other: None) ^ self._send({'jsonrpc':'2.0','id':message.id,'result':r}) ) ^
(¬('id'∈message ^ message.method truthy) => ¬∃m s.t. self._send(m))"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_cae92a9feab04d42a37cfcf59b28493f_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_cae92a9feab04d42a37cfcf59b28493f_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_cae92a9feab04d42a37cfcf59b28493f_response.txt)

事件 2 — llm_d26842192f5d498ca769510ead1a30c4

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "message = {\"method\": \"elp/status\"}",
  "offending_statements": "Line 4:           if params is None:\nLine 5:                 params = {}\nLine 7:                 self._status =
params.get(\"status\")",
  "reason": "The specification requires self._status to be updated only when the message contains a 'params' key. The code
defaults a missing 'params' to an empty dict, causing self._status to be unconditionally set to None (or another falsy value) even
when no 'params' key is present, which violates the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d26842192f5d498ca769510ead1a30c4_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d26842192f5d498ca769510ead1a30c4_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d26842192f5d498ca769510ead1a30c4_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-084 — src--languages--erlang-py--ElpClient::_send

- 四类结论： 契约不确定

- 分类依据: [probe](#) 已复现 [SPEC](#) 与实现的差异, 但现有证据不足以决定应修改实现还是修改 [SPEC](#)。
- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 原源码 [src/languages/erlang.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** If the server process is not running or its stdin stream is not writable, raises `RuntimeError`. Otherwise: message is serialized to a UTF-8 JSON byte sequence with minimal separators (commas and colons without whitespace) and without ASCII-escaping non-ASCII characters; the serialized byte sequence is prepended with an LSP Content-Length header (Content-Length: <octet_count>\r\n\r\n) to form a complete frame; the complete frame is written to the server process's stdin under exclusive acquisition of the write lock and the stream is flushed, ensuring the frame is delivered to the server as a single atomic unit. Returns `None` after successful transmission. Raises an `OSError` (or subclass, such as `BrokenPipeError`) when the stdin write or flush operation fails.
- **Actual behavior:** If the function raises no exception, then `self._proc` and `self._proc.stdin` are not `None`, and the JSON-RPC frame constructed from `message` (UTF-8 encoded JSON with ensured non-ASCII and compact separators) has been written and flushed to the server's stdin within the `with self._write_lock` block, so the lock is now released. The frame content exactly is `Content-Length: <payload_length>\r\n\r\n<payload_utf8>`. The message argument remains unchanged. If the function raises `RuntimeError`, then `self._proc` is `None` or `self._proc.stdin` is `None` is true, no frame is written, the lock is unaffected, and the server stdin is unmodified. Formal logic: (Exception (self._proc None self._proc.stdin None) sent(frame) lock_released(self._write_lock)) (Exception(RuntimeError) (self._proc = None self._proc.stdin = None) sent(frame) lock_unacquired(self._write_lock) stdin_unchanged(self._proc.stdin)).
- **Code evidence:** Line 2: if `self._proc` is `None` or `self._proc.stdin` is `None`:
- **Trigger condition:** The specification requires raising `RuntimeError` when the server process is not running or its stdin stream is not writable. The code only checks for `None` values, missing cases where the process has exited but the `Popen` object and `stdin` still exist. In such a case, the code raises an `OSError` instead of `RuntimeError`, violating the contract.
- **Validator trigger:** The code only checks `self._proc` is `None`, missing the case where the process has exited but the `Popen` object still exists; writing to dead stdin raises `BrokenPipeError` instead of `RuntimeError`.
- **Probe stdout:** **CONFIRMED** — actual: `BrokenPipeError` raised | expected: `RuntimeError` | process exited (`poll()==0`) but `self._proc` is not `None`, so `None` check passes; write to dead pipe raises `BrokenPipeError` instead of `RuntimeError`

► SPEC sidecar (完整)

```
{
  "signature": "_send(self, message: dict) -> None",
  "pre_condition": "self holds an active connection to an ELP server process whose stdin stream is writable. self._write_lock is a lock that serializes concurrent writes to the server's stdin. message is a dict forming a JSON-RPC 2.0 message whose top-level values are JSON-serializable.",
  "post_condition": "If the server process is not running or its stdin stream is not writable, raises RuntimeError. Otherwise: message is serialized to a UTF-8 JSON byte sequence with minimal separators (commas and colons without whitespace) and without ASCII-escaping non-ASCII characters; the serialized byte sequence is prepended with an LSP Content-Length header (Content-Length: <octet_count>\\r\\n\\r\\n) to form a complete frame; the complete frame is written to the server process's stdin under exclusive acquisition of the write lock and the stream is flushed, ensuring the frame is delivered to the server as a single atomic unit. Returns None after successful transmission. Raises an OSError (or subclass, such as BrokenPipeError) when the stdin write or flush operation fails."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::_send.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "If the server process is not running or its stdin stream is not writable, raises RuntimeError. Otherwise: message is serialized to a UTF-8 JSON byte sequence with minimal separators (commas and colons without whitespace) and without ASCII-escaping non-ASCII characters; the serialized byte sequence is prepended with an LSP Content-Length header (Content-Length: <octet_count>\\r\\n\\r\\n) to form a complete frame; the complete frame is written to the server process's stdin under exclusive acquisition of the write lock and the stream is flushed, ensuring the frame is delivered to the server as a single atomic unit. Returns None after successful transmission. Raises an OSError (or subclass, such as BrokenPipeError) when the stdin write or flush operation fails.",
    "actual_behavior": "If the function raises no exception, then self._proc and self._proc.stdin are not None, and the JSON-RPC
```

```
frame constructed from `message` (UTF-8 encoded JSON with ensured non-ASCII and compact separators) has been written and flushed
to the server's stdin within the `with self._write_lock` block, so the lock is now released. The frame content exactly is
`Content-Length: <payload_length>\\r\\n\\r\\n<payload_utf8>`. The message argument remains unchanged. If the function raises
`RuntimeError`, then `self._proc is None or self._proc.stdin is None` is true, no frame is written, the lock is unaffected, and
the server stdin is unmodified. Formal logic: (Exception (self._proc None self._proc.stdin None) sent(frame)
lock_released(self._write_lock)) (Exception(RuntimeError) (self._proc = None self._proc.stdin = None) sent(frame)
lock_unacquired(self._write_lock) stdin_unchanged(self._proc.stdin)).",
"code_evidence": "Line 2:             if self._proc is None or self._proc.stdin is None:",
"trigger_condition": "The specification requires raising RuntimeError when the server process is not running or its stdin
stream is not writable. The code only checks for None values, missing cases where the process has exited but the Popen object and
stdin still exist. In such a case, the code raises an OSError instead of RuntimeError, violating the contract."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--ElpClient::_send",
  "source_file": "/home/fancy/Projects/Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::_send.py",
  "function_name": "ElpClient::_send",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--ElpClient::_send.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--ElpClient::_send.md",
  "probe_stdout": "CONFIRMED - actual: BrokenPipeError raised | expected: RuntimeError | process exited (poll())=0) but self._proc
is not None, so None check passes; write to dead pipe raises BrokenPipeError instead of RuntimeError\n",
  "trigger_summary": "The code only checks self._proc is None, missing the case where the process has exited but the Popen object
still exists; writing to dead stdin raises BrokenPipeError instead of RuntimeError."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:01:21.906983Z	2345
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:01:55.844847Z	3971

事件 1 — [llm_2ba1a5ae88914fa09916c958cd2935b5](#)

► parsed_json（完整）

```
{
  "post_condition": "If the function raises no exception, then self._proc and self._proc.stdin are not None, and the JSON-RPC
frame constructed from `message` (UTF-8 encoded JSON with ensured non-ASCII and compact separators) has been written and flushed
to the server's stdin within the `with self._write_lock` block, so the lock is now released. The frame content exactly is
`Content-Length: <payload_length>\\r\\n\\r\\n<payload_utf8>`. The message argument remains unchanged. If the function raises
`RuntimeError`, then `self._proc is None or self._proc.stdin is None` is true, no frame is written, the lock is unaffected, and
the server stdin is unmodified. Formal logic: (¬Exception ⇒ (self._proc ≠ None ∧ self._proc.stdin ≠ None) ∧ sent(frame) ∧
lock_released(self._write_lock)) ∧ (Exception(RuntimeError) ⇒ (self._proc = None ∨ self._proc.stdin = None) ∧ ¬sent(frame) ∧
lock_unacquired(self._write_lock) ∧ stdin_unchanged(self._proc.stdin))."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_2ba1a5ae88914fa09916c958cd2935b5_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_2ba1a5ae88914fa09916c958cd2935b5_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_2ba1a5ae88914fa09916c958cd2935b5_response.txt)

事件 2 — [llm_5297329890924aaeb8db853985e24db4](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Let self._proc be a Popen object whose process has already terminated, but self._proc is not None and
self._proc.stdin is not None (the pipe is still open). Call _send with any message dict, e.g., {\"method\": \"test\"}. The code
will not raise RuntimeError, proceed to write, and raise BrokenPipeError, which is an OSError, not RuntimeError as required by the
specification.",
  "offending_statements": "Line 2:             if self._proc is None or self._proc.stdin is None:",
  "reason": "The specification requires raising RuntimeError when the server process is not running or its stdin stream is not
writable. The code only checks for None values, missing cases where the process has exited but the Popen object and stdin still
```

```
exist. In such a case, the code raises an OSError instead of RuntimeError, violating the contract."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5297329890924aaeb8db853985e24db4_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5297329890924aaeb8db853985e24db4_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5297329890924aaeb8db853985e24db4_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-085 — `src--languages--erlang-py--ElpClient::_wait_for_response`

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: `confirmed`; attempts 1
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / `language_handlers`; 原源码 `src/languages/erlang.py`
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns the 'result' field of the server's JSON-RPC response whose 'id' equals request_id and which lacks a 'method' key. If the matching response contains an 'error' object whose 'code' field equals the content-modified error code, raises `_ContentModifiedError`. If the matching response contains any other 'error' object, raises `RuntimeError` with a message describing the error. Messages from the server that do not match request_id including server notifications identified by the presence of a 'method' key are delegated to message handling without returning control to the caller.
- **Actual behavior**: After execution of `_wait_for_response`, if no exception is raised, the returned value is the 'result' field of a JSONRPC response message that has `id` equal to the given `request_id`, has no 'method' key, and contains no 'error' field. All server messages received before that response (i.e., messages with a different `id` or containing a 'method' key) have been processed by `_handle_server_message`. If a `_ContentModifiedError` is raised, then the response for `request_id` contained an 'error' field that is a dictionary with 'code' equal to `_CONTENT_MODIFIED_ERROR`. If a `RuntimeError` is raised, then the response contained an 'error' that does not match that specific code. If an exception is raised from `self._next_message(deadline)`, then either the deadline was reached without receiving a message or the server connection was lost. Formal logic: Let `M` be the last message retrieved by `self._next_message` before the method exits.
- `(return r) M.get('id') == request_id 'method' M.keys() M.get('error') is None r == M.get('result') M received earlier: (M.get('id') != request_id 'method' M.keys()) processed_by_handle(M).`
- `(_ContentModifiedError) M.get('id') == request_id 'method' M.keys() isinstance(M.get('error'), dict) M.get('error').get('code') == _CONTENT_MODIFIED_ERROR.`
- `(RuntimeError) M.get('id') == request_id 'method' M.keys() M.get('error') is not None (isinstance(M.get('error'), dict) M.get('error').get('code') == _CONTENT_MODIFIED_ERROR).`
- (exception from `_next_message`) no message satisfying the response condition was received before `deadline` or the connection was lost.
- **Code evidence**: Line 6: if error:
- **Trigger condition**: An error object that is an empty dict is falsy in Python, causing the code to skip error handling and return the result. However, the specification requires that any error object other than the specific content-modified one must raise `RuntimeError`. This violates the specification when a matching response contains an empty 'error' object.
- **Validator trigger**: An empty error dict `{}` in a matching JSON-RPC response is falsy in Python, causing the code to skip error handling and return `None` instead of raising `RuntimeError`.
- **Probe stdout**: CONFIRMED — actual: `None` | expected: `RuntimeError` for any non-content-modified error object

► SPEC sidecar (完整)

```
{
  "signature": "ElpClient._wait_for_response(self, request_id: int, deadline: float) -> Any",
  "pre_condition": "A JSON-RPC request with the given request_id has been sent to the ELP server. deadline is a monotonic timestamp by which a response must be received.",
  "post_condition": "Returns the 'result' field of the server's JSON-RPC response whose 'id' equals request_id and which lacks a 'method' key. If the matching response contains an 'error' object whose 'code' field equals the content-modified error code, raises _ContentModifiedError. If the matching response contains any other 'error' object, raises RuntimeError with a message describing the error. Messages from the server that do not match request_id – including server notifications identified by the presence of a 'method' key – are delegated to message handling without returning control to the caller."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_next_message",
      "signature": "ElpClient._next_message(self, deadline: float) -> dict",
      "pre_condition": "deadline is a monotonic timestamp in the future. self has an active connection to the ELP server.",
      "post_condition": "Returns the next complete JSON-RPC message from the server as a dict. Blocks until a message arrives or the deadline is reached. Raises an exception when the deadline expires before a message is received or when the connection to the server is lost."
    },
    {
      "name": "_handle_server_message",
      "signature": "ElpClient._handle_server_message(self, message: dict)",
      "pre_condition": "message is a dict representing a JSON-RPC message from the server that is not the response to the current request (either a server notification or a response for a different request_id).",
      "post_condition": "Processes the server message according to its type and content. Does not return a value relevant to the caller."
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::_wait_for_response.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns the 'result' field of the server's JSON-RPC response whose 'id' equals request_id and which lacks a 'method' key. If the matching response contains an 'error' object whose 'code' field equals the content-modified error code, raises _ContentModifiedError. If the matching response contains any other 'error' object, raises RuntimeError with a message describing the error. Messages from the server that do not match request_id including server notifications identified by the presence of a 'method' key are delegated to message handling without returning control to the caller.",
    "actual_behavior": "After execution of `_wait_for_response`, if no exception is raised, the returned value is the 'result' field of a JSONRPC response message that has `id` equal to the given `request_id`, has no `method` key, and contains no `error` field. All server messages received before that response (i.e., messages with a different `id` or containing a `method` key) have been processed by `_handle_server_message`. If a `_ContentModifiedError` is raised, then the response for `request_id` contained an `error` field that is a dictionary with `code` equal to `_CONTENT_MODIFIED_ERROR`. If a `RuntimeError` is raised, then the response contained an `error` that does not match that specific code. If an exception is raised from `self._next_message(deadline)`, then either the deadline was reached without receiving a message or the server connection was lost. Formal logic: Let `M` be the last message retrieved by `self._next_message` before the method exits.\n- (return `r`) `M.get('id') == request_id` `method` `M.keys()` `M.get('error')` is None `r == M.get('result')` `M` received earlier: `(M.get('id') != request_id` `method` `M.keys()`) processed_by_handle(M)`.\n- (`_ContentModifiedError` `M.get('id') == request_id` `method` `M.keys()`) isinstance(M.get('error'), dict) `M.get('error').get('code') == _CONTENT_MODIFIED_ERROR`.\n- (RuntimeError `M.get('id') == request_id` `method` `M.keys()`) `M.get('error')` is not None (isinstance(M.get('error'), dict) `M.get('error').get('code') == _CONTENT_MODIFIED_ERROR`).\n- (exception from `_next_message`) no message satisfying the response condition was received before `deadline` or the connection was lost.",
    "code_evidence": "Line 6: if error:",
    "trigger_condition": "An error object that is an empty dict is falsy in Python, causing the code to skip error handling and return the result. However, the specification requires that any error object other than the specific content-modified one must raise RuntimeError. This violates the specification when a matching response contains an empty 'error' object."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--ElpClient::_wait_for_response",
  "source_file": "src/languages/erlang-py/ElpClient::_wait_for_response.py",
  "function_name": "ElpClient::_wait_for_response",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--ElpClient::_wait_for_response.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--ElpClient::_wait_for_response.md",
  "probe_stdout": "CONFIRMED – actual: None | expected: RuntimeError for any non-content-modified error object",
  "trigger_summary": "An empty error dict {} in a matching JSON-RPC response is falsy in Python, causing the code to skip error handling and return None instead of raising RuntimeError."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:49:03.385248Z	2920

#	purpose	status	attempt	block	start	tokens
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:49:43.741586Z	10041

事件 1 — [llm_d1ca897247dd4b178fc388b9534aaa78](#)

► [parsed_json](#) (完整)

```
{
  "post_condition": "After execution of `_wait_for_response`, if no exception is raised, the returned value is the 'result' field of a JSON-RPC response message that has `id` equal to the given `request_id`, has no `method` key, and contains no `error` field. All server messages received before that response (i.e., messages with a different `id` or containing a `method` key) have been processed by `_handle_server_message`. If a `_ContentModifiedError` is raised, then the response for `request_id` contained an `error` field that is a dictionary with `code` equal to `_CONTENT_MODIFIED_ERROR`. If a `RuntimeError` is raised, then the response contained an `error` that does not match that specific code. If an exception is raised from `self._next_message(deadline)`, then either the deadline was reached without receiving a message or the server connection was lost. Formal logic: Let `M` be the last message retrieved by `self._next_message` before the method exits.\n- (return `r`) => `M.get('id') == request_id` ^ `method` <math>\notin</math> `M.keys()` ^ `M.get('error')` is None ^ `r == M.get('result')` ^  $\forall M'$  received earlier: (`M'.get('id') != request_id` v `method' <math>\in</math> `M'.keys()`) => `processed_by_handle(M')`.\n- (_ContentModifiedError) => `M.get('id') == request_id` ^ `method` <math>\notin</math> `M.keys()` ^ `isinstance(M.get('error'), dict)` ^ `M.get('error').get('code') == _CONTENT_MODIFIED_ERROR`.\n- (RuntimeError) => `M.get('id') == request_id` ^ `method` <math>\notin</math> `M.keys()` ^ `M.get('error')` is not None ^  $\neg$ (`isinstance(M.get('error'), dict)` ^ `M.get('error').get('code') == _CONTENT_MODIFIED_ERROR`).\n- (exception from `_next_message`) => no message satisfying the response condition was received before `deadline` or the connection was lost."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d1ca897247dd4b178fc388b9534aaa78_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d1ca897247dd4b178fc388b9534aaa78_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d1ca897247dd4b178fc388b9534aaa78_response.txt)

事件 2 — [llm_2aa46f818b904260ba48a0f2d91336fb](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "request_id=1, deadline far ahead, _next_message returns {'id': 1, 'error': {}, 'result': 'data'} (no 'method' key).",
  "offending_statements": "Line 6: if error:",
  "reason": "An error object that is an empty dict is falsy in Python, causing the code to skip error handling and return the result. However, the specification requires that any error object other than the specific content-modified one must raise RuntimeError. This violates the specification when a matching response contains an empty 'error' object."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_2aa46f818b904260ba48a0f2d91336fb_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_2aa46f818b904260ba48a0f2d91336fb_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_2aa46f818b904260ba48a0f2d91336fb_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-086 — [src--languages--erlang-py--ElpClient::close](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts 2
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 原源码 [src/languages/erlang.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim:** The ELP subprocess referenced by self._proc is terminated and its stdin and stdout pipes are closed. self._proc is set to None. The background reader thread self._reader is stopped and no longer consuming from the subprocess stdout. All OS-level resources (pipes, process handle) associated with the subprocess are released.
- Actual behavior:** After execution, self._proc is None. The original subprocess object (if it existed) has been properly handled: if it was still running, an attempt was made to gracefully shut it down (via request('shutdown'), notify('exit'), and wait with timeout), and if necessary it was terminated or killed; ultimately the process is no longer running and its standard input and output streams have been closed (any OSError during close is caught). The self._reader attribute remains unchanged, still referencing an active background reader thread. No unhandled exceptions propagate from this method. Formally: (self._proc = None) (s { proc.stdin, proc.stdout } : s is closed) (self._reader is an active thread) (the subprocess originally referenced is not running and its termination has been properly waited).

- **Code evidence:** Line 1-30: the entire close() method does not contain any code to stop or join self._reader.
- **Trigger condition:** The specification requires the background reader thread self._reader to be stopped and no longer consuming from the subprocess stdout. The code only terminates the subprocess and closes its streams, leaving self._reader untouched and still active, which violates the specification.
- **Validator trigger:** ElpClient::close() terminates the subprocess and closes its streams, but never stops, joins, or sets to None the background reader thread self._reader, leaving it alive and still referencing the closed subprocess stdout.
- **Probe stdout:** CONFIRMED — _proc is None=True, _reader is alive=True, _reader is None=False

► SPEC sidecar（完整）

```
{
  "signature": "ElpClient.close(self)",
  "pre_condition": "self._proc references an active ELP subprocess and self._reader references an active background reader thread.",
  "post_condition": "The ELP subprocess referenced by self._proc is terminated and its stdin and stdout pipes are closed. self._proc is set to None. The background reader thread self._reader is stopped and no longer consuming from the subprocess stdout. All OS-level resources (pipes, process handle) associated with the subprocess are released."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::close.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "The ELP subprocess referenced by self._proc is terminated and its stdin and stdout pipes are closed. self._proc is set to None. The background reader thread self._reader is stopped and no longer consuming from the subprocess stdout. All OS-level resources (pipes, process handle) associated with the subprocess are released.",
    "actual_behavior": "After execution, self._proc is None. The original subprocess object (if it existed) has been properly handled: if it was still running, an attempt was made to gracefully shut it down (via request('shutdown'), notify('exit'), and wait with timeout), and if necessary it was terminated or killed; ultimately the process is no longer running and its standard input and output streams have been closed (any OSError during close is caught). The self._reader attribute remains unchanged, still referencing an active background reader thread. No unhandled exceptions propagate from this method. Formally: (self._proc = None) ( s { proc.stdin, proc.stdout } : s is closed) (self._reader is an active thread) (the subprocess originally referenced is not running and its termination has been properly waited).",
    "code_evidence": "Line 1-30: the entire close() method does not contain any code to stop or join self._reader.",
    "trigger_condition": "The specification requires the background reader thread self._reader to be stopped and no longer consuming from the subprocess stdout. The code only terminates the subprocess and closes its streams, leaving self._reader untouched and still active, which violates the specification."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--ElpClient::close",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::close.py",
  "function_name": "close",
  "confirmation_status": "confirmed",
  "attempts": 2,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--ElpClient::close.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--ElpClient::close.md",
  "probe_stdout": "CONFIRMED — _proc is None=True, _reader is alive=True, _reader is None=False\n",
  "trigger_summary": "ElpClient::close() terminates the subprocess and closes its streams, but never stops, joins, or sets to None the background reader thread self._reader, leaving it alive and still referencing the closed subprocess stdout."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:47:42.200995Z	2577

#	purpose	status	attempt	block	start	tokens
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:48:15.423432Z	2346

事件 1 — llm_779cf3cf1cfc4a2bbdce4b1c6e306f36

► parsed_json (完整)

```
{
  "post_condition": "After execution, self._proc is None. The original subprocess object (if it existed) has been properly handled: if it was still running, an attempt was made to gracefully shut it down (via request('shutdown'), notify('exit'), and wait with timeout), and if necessary it was terminated or killed; ultimately the process is no longer running and its standard input and output streams have been closed (any OSError during close is caught). The self._reader attribute remains unchanged, still referencing an active background reader thread. No unhandled exceptions propagate from this method. Formally: (self._proc = None) ^ (forall s in { proc.stdin, proc.stdout } : s is closed) ^ (self._reader is an active thread) ^ (the subprocess originally referenced is not running and its termination has been properly waited)."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_779cf3cf1cfc4a2bbdce4b1c6e306f36_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_779cf3cf1cfc4a2bbdce4b1c6e306f36_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_779cf3cf1cfc4a2bbdce4b1c6e306f36_response.txt)

事件 2 — llm_e1c4ad391d9540bbbf8307819b216a41

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Call close() on an instance where self._proc is a running subprocess.Popen object and self._reader is an active threading.Thread that loops reading from self._proc.stdout. After close() returns, self._proc is None and pipes are closed, but self._reader is still running and may still be consuming from the buffer of the closed stdout pipe.",
  "offending_statements": "Line 1-30: the entire close() method does not contain any code to stop or join self._reader.",
  "reason": "The specification requires the background reader thread self._reader to be stopped and no longer consuming from the subprocess stdout. The code only terminates the subprocess and closes its streams, leaving self._reader untouched and still active, which violates the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_e1c4ad391d9540bbbf8307819b216a41_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_e1c4ad391d9540bbbf8307819b216a41_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_e1c4ad391d9540bbbf8307819b216a41_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-087 — src--languages--erlang-py--ElpClient::initialize

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / language_handlers; 源代码 src/languages/erlang.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim:** The ELP server has received the LSP initialize request conveying: the workspace root URI, the workspace folder list (containing a single folder at self.root_uri named after the project directory), client identity 'fm-agent' version '0.1.0', and requested capabilities (server status notifications, workspace configuration, hierarchical document symbols with no dynamic registration). The ELP server has received the 'initialized' notification. The text document at bootstrap_path has been opened on the server when bootstrap_source is not None, the server uses the provided source text; when bootstrap_source is None, the server reads the file from disk. The ELP server has reached the 'running' status after processing all server messages up to and including the status transition. When the initialize response is a dict, returns the value associated with the key 'serverInfo' from that dict, or None if the key is absent. When the initialize response is not a dict, returns None. Raises an exception when the server is unreachable, when the server does not reach running status before the message-processing deadline, or when a malformed server message is received.
- Actual behavior:** If the method completes without raising an exception, it returns a value server_info equal to (result.get('serverInfo') if isinstance(result, dict) else None), where result is the parsed JSON-RPC response of the 'initialize' request. The server connection remains active, the 'initialized' notification has been sent, the document at bootstrap_path has been opened (using bootstrap_source if not None, otherwise referencing the file on disk), and

`str(self._status).lower() == 'running'` holds, meaning the server has reached the running state. The polling loop was bounded by a deadline of `time.monotonic() + self.timeout`. If any step fails (server unreachable, JSON-RPC error response, connection loss, message timeout, or malformed server message), an exception is raised and the method does not return; the state of the client and server may be inconsistent. Formally: `Pre(self, bootstrap_path, bootstrap_source) ((NormalReturn (server_info: return_value = server_info server_info = (resp['serverInfo'] if isinstance(resp, dict) else None) self.connection_active str(self._status).lower() = 'running' notified('initialized') doc_opened(bootstrap_path, bootstrap_source))) (Exception))`.

- **Code evidence:** Line 2: `result = self.request(`
- **Trigger condition:** The specification lists only three specific conditions under which an exception is raised: server unreachable, server does not reach running status before deadline, or malformed server message. The code raises an exception when the server returns a JSON-RPC error response (e.g., 'Method not found') which does not fall under any of those conditions. According to the specification, the method should return `server_info` in all other cases, so a JSON-RPC error leads to a mismatch.
- **Validator trigger:** A JSON-RPC error response from the ELP server (e.g. 'Method not found') causes `RuntimeError`, but the spec only allows exceptions for server unreachable, timeout, or malformed server messages.
- **Probe stdout:** CONFIRMED — bug confirmed: `RuntimeError` raised for JSON-RPC error response: ELP request failed: `{'code': -32601, 'message': 'Method not found'}`

► SPEC sidecar (完整)

```
{
  "signature": "ElpClient.initialize(self, bootstrap_path: str, bootstrap_source: str | None = None) -> dict | None",
  "pre_condition": "self is an ElpClient instance with an active connection to a running ELP server process. self is configured with a valid project workspace root URI, a project directory path, and a positive message-processing timeout (in seconds). self._status reflects the current server connection state. bootstrap_path is a string.",
  "post_condition": "The ELP server has received the LSP initialize request conveying: the workspace root URI, the workspace folder list (containing a single folder at self.root_uri named after the project directory), client identity 'fm-agent' version '0.1.0', and requested capabilities (server status notifications, workspace configuration, hierarchical document symbols with no dynamic registration). The ELP server has received the 'initialized' notification. The text document at bootstrap_path has been opened on the server – when bootstrap_source is not None, the server uses the provided source text; when bootstrap_source is None, the server reads the file from disk. The ELP server has reached the 'running' status after processing all server messages up to and including the status transition. When the initialize response is a dict, returns the value associated with the key 'serverInfo' from that dict, or None if the key is absent. When the initialize response is not a dict, returns None. Raises an exception when the server is unreachable, when the server does not reach running status before the message-processing deadline, or when a malformed server message is received."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "ElpClient.request",
      "signature": "ElpClient.request(self, method: str, params: dict) -> Any",
      "pre_condition": "self has an active connection to the ELP server. method is the LSP method name as a string. params is a dict of JSON-serializable parameters for the LSP request.",
      "post_condition": "Returns the 'result' field of the server's JSON-RPC response, parsed to the appropriate Python type for the method. Raises an exception when the server is unreachable, the server returns a JSON-RPC error response, or the connection is lost during the request."
    },
    {
      "name": "ElpClient.notify",
      "signature": "ElpClient.notify(self, method: str) -> None",
      "pre_condition": "self has an active connection to the ELP server. method is the LSP notification method name as a string.",
      "post_condition": "The notification has been delivered to the server. Does not wait for or return a server response. Raises an exception when the server is unreachable or the connection is lost during delivery."
    },
    {
      "name": "ElpClient.open_document",
      "signature": "ElpClient.open_document(self, path: str, source: str | None) -> None",
      "pre_condition": "self has an active connection to the ELP server. path is a string representing a file URI. source is either None or a string containing source code text.",
      "post_condition": "The server has been notified that the text document at path is open. When source is not None, the server holds the provided text content for subsequent document symbol and call-hierarchy queries. When source is None, the server associates the URI with the file content on disk. Raises an exception when the server is unreachable, the connection is lost, or the server rejects the document-open notification."
    },
    {
      "name": "ElpClient._next_message",
      "signature": "ElpClient._next_message(self, deadline: float) -> Any",
      "pre_condition": "self has an active connection to the ELP server. deadline is an absolute monotonic time value (as returned by time.monotonic()) representing the latest acceptable arrival time for the next message.",
      "post_condition": "Returns the next message received from the ELP server, parsed into the appropriate Python type. Blocks the caller until a message arrives. Raises an exception when no message arrives before deadline or when the server connection is lost."
    }
  ]
}
```

```
"name": "ElpClient._handle_server_message",
"signature": "ElpClient._handle_server_message(self, msg: Any) -> None",
"pre_condition": "msg is a parsed server message received from the ELP server, in the format produced by _next_message.",
"post_condition": "The message has been processed. May update self._status and other internal server-state attributes on self. When the message indicates the server has reached the running status, subsequent checks of self._status reflect this. Raises an exception when the message is malformed or indicates a server error that cannot be handled."
}
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::initialize.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "The ELP server has received the LSP initialize request conveying: the workspace root URI, the workspace folder list (containing a single folder at self.root_uri named after the project directory), client identity 'fm-agent' version '0.1.0', and requested capabilities (server status notifications, workspace configuration, hierarchical document symbols with no dynamic registration). The ELP server has received the 'initialized' notification. The text document at bootstrap_path has been opened on the server when bootstrap_source is not None, the server uses the provided source text; when bootstrap_source is None, the server reads the file from disk. The ELP server has reached the 'running' status after processing all server messages up to and including the status transition. When the initialize response is a dict, returns the value associated with the key 'serverInfo' from that dict, or None if the key is absent. When the initialize response is not a dict, returns None. Raises an exception when the server is unreachable, when the server does not reach running status before the message-processing deadline, or when a malformed server message is received.",
    "actual_behavior": "If the method completes without raising an exception, it returns a value `server_info` equal to `(result.get('serverInfo') if isinstance(result, dict) else None)`, where `result` is the parsed JSON-RPC response of the 'initialize' request. The server connection remains active, the 'initialized' notification has been sent, the document at `bootstrap_path` has been opened (using `bootstrap_source` if not None, otherwise referencing the file on disk), and `str(self._status).lower() == 'running'` holds, meaning the server has reached the running state. The polling loop was bounded by a deadline of `time.monotonic() + self.timeout`. If any step fails (server unreachable, JSON-RPC error response, connection loss, message timeout, or malformed server message), an exception is raised and the method does not return; the state of the client and server may be inconsistent. Formally: Pre(self, bootstrap_path, bootstrap_source) ((NormalReturn ( server_info: return_value = server_info server_info = (resp['serverInfo'] if isinstance(resp, dict) else None) self.connection_active str(self._status).lower() = 'running' notified('initialized') doc_opened(bootstrap_path, bootstrap_source))) (Exception ))).",
    "code_evidence": "Line 2:         result = self.request(",
    "trigger_condition": "The specification lists only three specific conditions under which an exception is raised: server unreachable, server does not reach running status before deadline, or malformed server message. The code raises an exception when the server returns a JSON-RPC error response (e.g., 'Method not found') which does not fall under any of those conditions. According to the specification, the method should return server_info in all other cases, so a JSON-RPC error leads to a mismatch."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--ElpClient::initialize",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::initialize.py",
  "function_name": "ElpClient::initialize",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--ElpClient::initialize.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--ElpClient::initialize.md",
  "probe_stdout": "CONFIRMED – bug confirmed: RuntimeError raised for JSON-RPC error response: ELP request failed: {'code': -32601, 'message': 'Method not found'}",
  "trigger_summary": "A JSON-RPC error response from the ELP server (e.g. 'Method not found') causes RuntimeError, but the spec only allows exceptions for server unreachable, timeout, or malformed server messages."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:28:42.374777Z	4042
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:29:30.924484Z	7648

事件 1 — [llm_0086c51003c84fcdb4392455b96e30b6](#)

► parsed_json（完整）

```
{
  "post_condition": "If the method completes without raising an exception, it returns a value `server_info` equal to `(result.get('serverInfo') if isinstance(result, dict) else None)`, where `result` is the parsed JSON-RPC response of the 'initialize' request. The server connection remains active, the 'initialized' notification has been sent, the document at `bootstrap_path` has been opened (using `bootstrap_source` if not None, otherwise referencing the file on disk), and `str(self._status).lower() == 'running'` holds, meaning the server has reached the running state. The polling loop was bounded by a deadline of `time.monotonic() + self.timeout`. If any step fails (server unreachable, JSON-RPC error response, connection loss, message timeout, or malformed server message), an exception is raised and the method does not return; the state of the client and server may be inconsistent. Formally: Pre(self, bootstrap_path, bootstrap_source) => ((NormalReturn => (exists server_info: return_value = server_info ^ server_info = (resp['serverInfo'] if isinstance(resp, dict) else None) ^ self.connection_active ^ str(self._status).lower() = 'running' ^ notified('initialized') ^ doc_opened(bootstrap_path, bootstrap_source))) ^ (Exception => T))."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_0086c51003c84fdb4392455b96e30b6_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_0086c51003c84fdb4392455b96e30b6_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_0086c51003c84fdb4392455b96e30b6_response.txt)

事件 2 — llm_c7939748f5cd482f91a80c37efc63d1b

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "An ElpClient instance with an active connection to a reachable ELP server. bootstrap_path = 'file:///test.py', bootstrap_source = None, self.timeout = 30. When the server receives the 'initialize' request, it responds with a valid JSON-RPC error message: {\"jsonrpc\": \"2.0\", \"error\": {\"code\": -32601, \"message\": \"Method not found\"}, \"id\": 1}.\", \"offending_statements\": \"Line 2: result = self.request(\"\", \"reason\": \"The specification lists only three specific conditions under which an exception is raised: server unreachable, server does not reach running status before deadline, or malformed server message. The code raises an exception when the server returns a JSON-RPC error response (e.g., 'Method not found') which does not fall under any of those conditions. According to the specification, the method should return server_info in all other cases, so a JSON-RPC error leads to a mismatch.\""}
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c7939748f5cd482f91a80c37efc63d1b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c7939748f5cd482f91a80c37efc63d1b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c7939748f5cd482f91a80c37efc63d1b_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-088 — src -- languages -- erlang.py -- ElpClient::open_document

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator：confirmed; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / language_handlers; 源代码 src/languages/erlang.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim：**The ELP server has been notified via an LSP 'textDocument/didOpen' notification that the text document at the resolved absolute path is open. When source is not None, the server has received the provided source text as the document content. When source is None, the server has received the on-disk file content read as UTF-8 with replacement characters substituted for any undecodable bytes. The opened document is registered with languageId 'erlang' and version 1. Raises an exception when the server is unreachable, the server rejects the notification, or the connection is lost.
- Actual behavior：**After the execution of the method open_document, one of the following holds: (1) No exception is raised; then the ELP server has received the textDocument/didOpen notification with uri equal to the file URI of the resolved absolute path of path, languageId set to "erlang", version set to 1, and text equal to the source argument if provided, otherwise the file content read using UTF-8 encoding with error replacement; and self's connection to the ELP server remains active. (2) An exception is raised. If the exception is a file-read error (e.g., FileNotFoundError, PermissionError), then no textDocument/didOpen notification was sent and the connection remains active. If the exception originates from the notify call (due to server unreachable, connection loss, or notification rejection), then the notification was not delivered and the connection is no longer active. Formal post-condition: Let normal_exit be true if the block completed without an exception, active the predicate that self has an active ELP connection, receive(N, params) that the server received notification N with params, file_exc and notify_exc the respective exception conditions. Then (normal_exit (receive("textDocument/didOpen", {uri: URI, languageId: "erlang", version: 1, text: text_val}) active)) (normal_exit ((file_exc (receive("textDocument/didOpen", _) active)) (notify_exc (receive("textDocument/didOpen", _) active)))).

- **Code evidence:** Line 4: `source = document.read_text(encoding="utf-8", errors="replace")`
- **Trigger condition:** The code raises a `FileNotFoundError` (a file-read exception) when the file does not exist, but the specification only allows exceptions when the server is unreachable, the server rejects the notification, or the connection is lost.
- **Validator trigger:** `open_document()` with `source=None` and a non-existent file path raises `FileNotFoundError`, which is not among the spec's allowed exception categories (server unreachable, notification rejection, connection loss).
- **Probe stdout:** CONFIRMED — `FileNotFoundError` raised by `open_document()` when `source=None` and the file does not exist. The spec only allows exceptions for server unreachable, notification rejection, or connection loss. Exception: `FileNotFoundError: [Errno 2] No such file or directory: '/tmp/probe_open_document_mzgd9rwi/nonexistent_file.erl'`

► SPEC sidecar (完整)

```
{
  "signature": "ElpClient.open_document(self, path: str, source: str | None = None) -> None",
  "pre_condition": "self has an active connection to the ELP server. path is a string representing a filesystem path to an Erlang source file.",
  "post_condition": "The ELP server has been notified via an LSP 'textDocument/didOpen' notification that the text document at the resolved absolute path is open. When source is not None, the server has received the provided source text as the document content. When source is None, the server has received the on-disk file content read as UTF-8 with replacement characters substituted for any undecodable bytes. The opened document is registered with languageId 'erlang' and version 1. Raises an exception when the server is unreachable, the server rejects the notification, or the connection is lost."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "ElpClient.notify",
      "signature": "ElpClient.notify(self, method: str, params: dict) -> None",
      "pre_condition": "self has an active connection to the ELP server. method is an LSP notification method name. params is a dict of JSON-serializable parameters.",
      "post_condition": "The ELP server has received the notification with the given method and parameters. Raises an exception when the server is unreachable, the server rejects the notification, or the connection is lost."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::open_document.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "The ELP server has been notified via an LSP 'textDocument/didOpen' notification that the text document at the resolved absolute path is open. When source is not None, the server has received the provided source text as the document content. When source is None, the server has received the on-disk file content read as UTF-8 with replacement characters substituted for any undecodable bytes. The opened document is registered with languageId 'erlang' and version 1. Raises an exception when the server is unreachable, the server rejects the notification, or the connection is lost.",
    "actual_behavior": "After the execution of the method 'open_document', one of the following holds: (1) No exception is raised; then the ELP server has received the 'textDocument/didOpen' notification with 'uri' equal to the file URI of the resolved absolute path of 'path', 'languageId' set to 'erlang', 'version' set to '1', and 'text' equal to the 'source' argument if provided, otherwise the file content read using UTF-8 encoding with error replacement; and 'self's connection to the ELP server remains active. (2) An exception is raised. If the exception is a file-read error (e.g., 'FileNotFoundError', 'PermissionError'), then no 'textDocument/didOpen' notification was sent and the connection remains active. If the exception originates from the 'notify' call (due to server unreachable, connection loss, or notification rejection), then the notification was not delivered and the connection is no longer active. Formal post-condition: Let 'normal_exit' be true if the block completed without an exception, 'active' the predicate that self has an active ELP connection, 'receive(N, params)' that the server received notification N with params, 'file_exc' and 'notify_exc' the respective exception conditions. Then (normal_exit (receive('textDocument/didOpen', {uri: URI, languageId: 'erlang', version: 1, text: text_val}) active)) (normal_exit ((file_exc (receive('textDocument/didOpen', _) active)) (notify_exc (receive('textDocument/didOpen', _) active))))).",
    "code_evidence": "Line 4: source = document.read_text(encoding='utf-8', errors='replace')",
    "trigger_condition": "The code raises a FileNotFoundError (a file-read exception) when the file does not exist, but the specification only allows exceptions when the server is unreachable, the server rejects the notification, or the connection is lost."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--ElpClient::open_document",
  "source_file": "src/languages/erlang.py",
}
```



```
{
  "function_name": "ElpClient.open_document",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--ElpClient::open_document.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--ElpClient::open_document.md",
  "probe_stdout": "CONFIRMED - FileNotFoundError raised by open_document() when source=None and the file does not exist. The spec
only allows exceptions for server unreachable, notification rejection, or connection loss. Exception: FileNotFoundError: [Errno 2]
No such file or directory: '/tmp/probe_open_document_mzgd9rwi/nonexistent_file.erl'",
  "trigger_summary": "open_document() with source=None and a non-existent file path raises FileNotFoundError, which is not among
the spec's allowed exception categories (server unreachable, notification rejection, connection loss).\"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:39:18.892547Z	4837
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:40:28.417656Z	6268

事件 1 — [llm_4d0a7143f9a34edaaff0099ccfa420a7](#)

► parsed_json（完整）

```
{
  "post_condition": "After the execution of the method `open_document`, one of the following holds: (1) No exception is raised;
then the ELP server has received the `textDocument/didOpen` notification with `uri` equal to the file URI of the resolved absolute
path of `path`, `languageId` set to `\"erlang\"`, `version` set to `1`, and `text` equal to the `source` argument if provided,
otherwise the file content read using UTF-8 encoding with error replacement; and `self`'s connection to the ELP server remains
active. (2) An exception is raised. If the exception is a file-read error (e.g., `FileNotFoundError`, `PermissionError`), then no
`textDocument/didOpen` notification was sent and the connection remains active. If the exception originates from the `notify` call
(due to server unreachable, connection loss, or notification rejection), then the notification was not delivered and the
connection is no longer active. Formal post-condition: Let `normal_exit` be true if the block completed without an exception,
`active` the predicate that self has an active ELP connection, `receive(N, params)` that the server received notification N with
params, `file_exc` and `notify_exc` the respective exception conditions. Then (normal_exit => (receive(`\"textDocument/didOpen\"`,
{uri: URI, languageId: \"erlang\", version: 1, text: text_val}) & active)) & (~normal_exit => ((file_exc =>
(~receive(`\"textDocument/didOpen\"`, _) & active)) & (notify_exc => (~receive(`\"textDocument/didOpen\"`, _) & ~active))))).\"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4d0a7143f9a34edaaff0099ccfa420a7_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_4d0a7143f9a34edaaff0099ccfa420a7_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_4d0a7143f9a34edaaff0099ccfa420a7_response.txt)

事件 2 — [llm_295d175772c642d487d0cf25529a3ed7](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "open_document('/nonexistent.erl')",
  "offending_statements": "Line 4: source = document.read_text(encoding=\"utf-8\", errors=\"replace\")",
  "reason": "The code raises a FileNotFoundError (a file-read exception) when the file does not exist, but the specification only
allows exceptions when the server is unreachable, the server rejects the notification, or the connection is lost.\"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_295d175772c642d487d0cf25529a3ed7_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_295d175772c642d487d0cf25529a3ed7_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_295d175772c642d487d0cf25529a3ed7_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-089 — [src--languages--erlang-py--ElpClient::request](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts 1
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 原源码 [src/languages/erlang.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

- **SPEC claim:** Sends a JSON-RPC 2.0 request with the given method and params to the ELP server. When params is None, an empty dict is used in its place. Returns the 'result' field of the server's JSON-RPC response, parsed to the appropriate Python type. Transparently retries on transient server-side content-modification errors with exponential backoff bounded by the instance's configured message-processing timeout. Raises RuntimeError when all retries are exhausted on a content-modification error. Raises TimeoutError when the overall message-processing deadline expires before a response is received. Raises an exception when the server is unreachable, the server returns a non-retriable JSON-RPC error response, or the connection is lost.
- **Actual behavior:** After the method 'request' executes, the following post-conditions hold based on the outcome:
 - If the method returns a value v: A JSON-RPC 2.0 request with a unique integer id (equal to self._next_id at the start of the attempt) was sent, and before the deadline the server responded with a non-ContentModifiedError success response matching that id, with 'result' v. The method may have retried after receiving ContentModifiedError responses, each time sending a new request with incremented id. The returned v is the result from the first attempt where no ContentModifiedError occurred. The connection remains active. Formal: $i \in [0, _MAX_CONTENT_MODIFIED_RETRIES)$ such that $(\forall j < i, \text{attempt } j \text{ received ContentModifiedError})$ (attempt i received success with result v) (time_monotonic() deadline at response) (connection active).
 - If the method raises RuntimeError: All $_MAX_CONTENT_MODIFIED_RETRIES$ attempts received ContentModifiedError, and the deadline was not exceeded before the final attempt's error catch. No successful response was returned. Formal: $(i \in [0, _MAX_CONTENT_MODIFIED_RETRIES), \text{attempt } i \text{ received ContentModifiedError})$ (deadline - time_monotonic() after final ContentModifiedError > 0 but retries exhausted).
 - If the method raises TimeoutError: Either (a) a `\_wait_for_response` call raised TimeoutError because the deadline was reached without a response for the sent id, or (b) after a ContentModifiedError, the remaining time (deadline - time_monotonic()) was 0, causing an explicit TimeoutError. In both cases, no successful response was obtained. Formal: $i \in [0, _MAX_CONTENT_MODIFIED_RETRIES)$ such that $(\forall j < i, \text{attempt } j \text{ received ContentModifiedError})$ ((attempt i raised TimeoutError during `\_wait_for_response`) (attempt i received ContentModifiedError deadline - time_monotonic() 0)).
 - If any other exception propagates: An exception from `\_send` (e.g., connection failure) or from `\_wait_for_response` (e.g., JSON-RPC error response, connection lost) is raised, and no result is returned. The connection state may be unclear.
 - State change: self._next_id is incremented by the number of attempts started $(1, _MAX_CONTENT_MODIFIED_RETRIES)$.
- **Code evidence:** Line 18: if attempt + 1 == $_MAX_CONTENT_MODIFIED_RETRIES$: Line 19: raise RuntimeError(Line 20: f"ELP request {method} repeatedly failed: {exc.error}" Line 21:) from exc
- **Trigger condition:** When the last retry receives a ContentModifiedError after the overall deadline has expired, the code immediately raises RuntimeError without checking the remaining time. The specification requires TimeoutError when the deadline expires before a response is received, which should take precedence over RuntimeError.
- **Validator trigger:** When the last retry receives a ContentModifiedError after the overall deadline has expired, the code immediately raises RuntimeError without checking the remaining time.
- **Probe stdout:** CONFIRMED — actual: RuntimeError raised | expected: TimeoutError | deadline=10.0, monotonic_reached=11.0, deadline expired on final retry

► SPEC sidecar (完整)

```
{
  "signature": "ElpClient.request(self, method: str, params: dict | list | None = None) -> Any",
  "pre_condition": "self has an active connection to the ELP server. method is an LSP method name as a string. params is a dict or list of JSON-serializable parameters, or None.",
  "post_condition": "Sends a JSON-RPC 2.0 request with the given method and params to the ELP server. When params is None, an empty dict is used in its place. Returns the 'result' field of the server's JSON-RPC response, parsed to the appropriate Python type. Transparently retries on transient server-side content-modification errors with exponential backoff bounded by the instance's configured message-processing timeout. Raises RuntimeError when all retries are exhausted on a content-modification error. Raises TimeoutError when the overall message-processing deadline expires before a response is received. Raises an exception when the server is unreachable, the server returns a non-retriable JSON-RPC error response, or the connection is lost."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
```

```

"name": "ElpClient._send",
"signature": "ElpClient._send(self, message: dict) -> None",
"pre_condition": "self has an active connection to the ELP server. message is a dict forming a well-formed JSON-RPC 2.0 request with 'jsonrpc', 'id', 'method', and 'params' fields.",
"post_condition": "The message has been serialized to JSON and written to the ELP server process's stdin. Raises an exception when the server is unreachable or the write fails."
},
{
"name": "ElpClient._wait_for_response",
"signature": "ElpClient._wait_for_response(self, request_id: int, deadline: float) -> Any",
"pre_condition": "A JSON-RPC request with the given request_id has been sent to the ELP server. deadline is a monotonic timestamp by which a response must arrive.",
"post_condition": "Returns the 'result' field of the server's JSON-RPC response matching the given request_id. Raises an exception when the response indicates a content-modified error, when the server returns a JSON-RPC error response, when the deadline is exceeded without a matching response, or when the connection is lost."
}
]
}

```

► Logic verification result（完整）

```

{
"function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::request.py",
"verdict": "MISMATCH",
"gaps": {
"spec_claim": "Sends a JSON-RPC 2.0 request with the given method and params to the ELP server. When params is None, an empty dict is used in its place. Returns the 'result' field of the server's JSON-RPC response, parsed to the appropriate Python type. Transparently retries on transient server-side content-modification errors with exponential backoff bounded by the instance's configured message-processing timeout. Raises RuntimeError when all retries are exhausted on a content-modification error. Raises TimeoutError when the overall message-processing deadline expires before a response is received. Raises an exception when the server is unreachable, the server returns a non-retriable JSON-RPC error response, or the connection is lost.",
"actual_behavior": "After the method 'request' executes, the following post-conditions hold based on the outcome:\n\n- If the method returns a value v:\n  A JSON-RPC 2.0 request with a unique integer id (equal to self._next_id at the start of the attempt) was sent, and before the deadline the server responded with a non-ContentModifiedError success response matching that id, with 'result' v. The method may have retried after receiving ContentModifiedError responses, each time sending a new request with incremented id. The returned v is the result from the first attempt where no ContentModifiedError occurred. The connection remains active.\n  Formal:  $\exists i \in [0, \text{\_MAX\_CONTENT\_MODIFIED\_RETRIES})$  such that  $(\forall j < i, \text{attempt } j \text{ received ContentModifiedError}) \wedge (\text{attempt } i \text{ received success with result } v) \wedge (\text{time\_monotonic}() < \text{deadline at response}) \wedge (\text{connection active})$ .\n\n- If the method raises RuntimeError:\n  All  $\text{\_MAX\_CONTENT\_MODIFIED\_RETRIES}$  attempts received ContentModifiedError, and the deadline was not exceeded before the final attempt's error catch. No successful response was returned.\n  Formal:  $(\exists i \in [0, \text{\_MAX\_CONTENT\_MODIFIED\_RETRIES}), \text{attempt } i \text{ received ContentModifiedError}) \wedge (\text{deadline} - \text{time\_monotonic}() > 0 \text{ but retries exhausted})$ .\n\n- If the method raises TimeoutError:\n  Either (a) a _wait_for_response call raised TimeoutError because the deadline was reached without a response for the sent id, or (b) after a ContentModifiedError, the remaining time ( $\text{deadline} - \text{time\_monotonic}()$ ) was 0, causing an explicit TimeoutError. In both cases, no successful response was obtained.\n  Formal:  $\exists i \in [0, \text{\_MAX\_CONTENT\_MODIFIED\_RETRIES})$  such that  $(\forall j < i, \text{attempt } j \text{ received ContentModifiedError}) \wedge ((\text{attempt } i \text{ raised TimeoutError during } \text{\_wait\_for\_response}) \vee (\text{attempt } i \text{ received ContentModifiedError} \wedge \text{deadline} - \text{time\_monotonic}() = 0))$ .\n\n- If any other exception propagates:\n  An exception from _send (e.g., connection failure) or from _wait_for_response (e.g., JSON-RPC error response, connection lost) is raised, and no result is returned. The connection state may be unclear.\n\n- State change: self._next_id is incremented by the number of attempts started (1,  $\text{\_MAX\_CONTENT\_MODIFIED\_RETRIES}$ ).",
"code_evidence": "Line 18:         if attempt + 1 == _MAX_CONTENT_MODIFIED_RETRIES:\nLine 19:             raise RuntimeError(\nLine 20:                 f\"ELP request {method} repeatedly failed: {exc.error}\"\nLine 21:             ) from exc",
"trigger_condition": "When the last retry receives a ContentModifiedError after the overall deadline has expired, the code immediately raises RuntimeError without checking the remaining time. The specification requires TimeoutError when the deadline expires before a response is received, which should take precedence over RuntimeError."
}
}

```

► Bug Validator result（完整）

```

{
"id": "src--languages--erlang-py--ElpClient::request",
"source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::request.py",
"function_name": "ElpClient::request",
"confirmation_status": "confirmed",
"attempts": 1,
"probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--ElpClient::request.py",
"detail_file": "fm_agent/bug_validation/src--languages--erlang-py--ElpClient::request.md",
"probe_stdout": "CONFIRMED - actual: RuntimeError raised | expected: TimeoutError | deadline=10.0, monotonic_reached=11.0, deadline expired on final retry\n",
"trigger_summary": "When the last retry receives a ContentModifiedError after the overall deadline has expired, the code immediately raises RuntimeError without checking the remaining time."
}

```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:40:10.016936Z	4233
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:41:12.106102Z	10037

事件 1 — [llm_eaf4d36f15754c24a04a66e762ca4cca](#)

► [parsed_json](#) (完整)

```
{
  "post_condition": "After the method 'request' executes, the following post-conditions hold based on the outcome:\n\n- If the method returns a value v:\n  A JSON-RPC 2.0 request with a unique integer id (equal to self._next_id at the start of the attempt) was sent, and before the deadline the server responded with a non-ContentModifiedError success response matching that id, with 'result' v. The method may have been retried after receiving ContentModifiedError responses, each time sending a new request with incremented id. The returned v is the result from the first attempt where no ContentModifiedError occurred. The connection remains active.\n  Formal:  $\exists i \in [0, \_MAX\_CONTENT\_MODIFIED\_RETRIES)$  such that  $(\forall j < i, \text{attempt } j \text{ received ContentModifiedError}) \wedge (\text{attempt } i \text{ received success with result } v) \wedge (\text{time\_monotonic}() \leq \text{deadline at response}) \wedge (\text{connection active}).$ \n\n- If the method raises RuntimeError:\n  All  $\_MAX\_CONTENT\_MODIFIED\_RETRIES$  attempts received ContentModifiedError, and the deadline was not exceeded before the final attempt's error catch. No successful response was returned.\n  Formal:  $(\forall i \in [0, \_MAX\_CONTENT\_MODIFIED\_RETRIES), \text{attempt } i \text{ received ContentModifiedError}) \wedge (\text{deadline} - \text{time\_monotonic}()) \text{ after final ContentModifiedError} > 0 \text{ but retries exhausted}).$ \n\n- If the method raises TimeoutError:\n  Either (a) a _wait_for_response call raised TimeoutError because the deadline was reached without a response for the sent id, or (b) after a ContentModifiedError, the remaining time  $(\text{deadline} - \text{time\_monotonic}())$  was  $\leq 0$ , causing an explicit TimeoutError. In both cases, no successful response was obtained.\n  Formal:  $\exists i \in [0, \_MAX\_CONTENT\_MODIFIED\_RETRIES)$  such that  $( (\forall j < i, \text{attempt } j \text{ received ContentModifiedError}) \wedge (\text{attempt } i \text{ raised TimeoutError during } \_wait\_for\_response) \vee (\text{attempt } i \text{ received ContentModifiedError} \wedge \text{deadline} - \text{time\_monotonic}() \leq 0) ) ).$ \n\n- If any other exception propagates:\n  An exception from _send (e.g., connection failure) or from _wait_for_response (e.g., JSON-RPC error response, connection lost) is raised, and no result is returned. The connection state may be unclear.\n\nState change: self._next_id is incremented by the number of attempts started ( $\geq 1, \leq \_MAX\_CONTENT\_MODIFIED\_RETRIES$ )."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_eaf4d36f15754c24a04a66e762ca4cca_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_eaf4d36f15754c24a04a66e762ca4cca_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_eaf4d36f15754c24a04a66e762ca4cca_response.txt)

事件 2 — [llm_b754625cda274fcd8c9d069db0c596da](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Instance with timeout=0.001s,  $\_MAX\_CONTENT\_MODIFIED\_RETRIES=1$ . Call request('test', None). Server sends a ContentModifiedError response after the deadline (e.g., at 0.002s). _wait_for_response raises _ContentModifiedError despite the expired deadline. Code then raises RuntimeError instead of TimeoutError.",
  "offending_statements": "Line 18:                                if attempt + 1 ==  $\_MAX\_CONTENT\_MODIFIED\_RETRIES$ :\nLine 19:                                raise RuntimeError(\nLine 20:                                    f\"ELP request {method} repeatedly failed: {exc.error}\" \nLine 21:                                ) from exc",
  "reason": "When the last retry receives a ContentModifiedError after the overall deadline has expired, the code immediately raises RuntimeError without checking the remaining time. The specification requires TimeoutError when the deadline expires before a response is received, which should take precedence over RuntimeError."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b754625cda274fcd8c9d069db0c596da_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_b754625cda274fcd8c9d069db0c596da_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_b754625cda274fcd8c9d069db0c596da_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-090 — [src - languages - erlang-py - _JsonRpcReader::run](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#); attempts 1
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 源代码 [src/languages/erlang.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim:** Each complete JSON-RPC message received on self._stream framed by HTTP-style headers that terminate with an empty line and contain a Content-Length field is decoded from JSON and placed as a Python object onto self._messages, in the order received. When self._stream reaches EOF, or when a payload is shorter than its declared Content-Length, an EOFError is placed on self._messages

- and the method returns. When any other exception is raised during processing, that exception is placed on `self._messages` and the method returns. The method never raises an exception to its caller.
- **Actual behavior:** If the method `run()` returns (it may never return if no exception occurs), then the following holds. The binary stream `self._stream` remains open but its read position is advanced to an indeterminate point after the last read operation (successful or failed). The thread-safe queue `self._messages` now contains all elements it had before `run()` was called, followed by zero or more successfully parsed JSON messages (each a Python object obtained from a complete `content-length` delimited payload read after a set of HTTP-like headers), then followed by a single exception object `exc` that caused the loop to terminate. This exception is an instance of `BaseException` (or a subclass) and was placed into the queue by the `except` block. Formally, let `Q_before` be the sequence of items in `self._messages` before the call. After the call, the queue contains the sequence `Q_after = Q_before + [m_1, ..., m_k, exc]` where `k >= 0`, each `m_i` is the result of `json.loads(payload_i.decode('utf-8'))` for some `payload_i` read from the stream exactly after correctly parsing a header set that included a `content-length` field whose value was a non-negative integer, and `exc` is the exception instance that was caught in the `except BaseException as exc` block.
 - **Code evidence:** Line 20: `self._messages.put(exc)`
 - **Trigger condition:** The specification (B) states 'The method never raises an exception to its caller.' The code's `except` block calls `self._messages.put(exc)` without a `try/except`. If that `put()` call raises an exception, it will propagate to the caller, directly contradicting the specification.
 - **Validator trigger:** `self._messages.put(exc)` in the `except` block (line 90) is not wrapped in `try/except`; if `put()` raises, the exception escapes `run()` to the caller, violating the spec guarantee that `run()` never raises.
 - **Probe stdout:** CONFIRMED — `RuntimeError` propagated to caller: queue put failure

► SPEC sidecar (完整)

```
{
  "signature": "_JsonRpcReader::run(self)",
  "pre_condition": "self._stream is an open readable binary stream. self._messages is a thread-safe queue.",
  "post_condition": "Each complete JSON-RPC message received on self._stream – framed by HTTP-style headers that terminate with an empty line and contain a Content-Length field – is decoded from JSON and placed as a Python object onto self._messages, in the order received. When self._stream reaches EOF, or when a payload is shorter than its declared Content-Length, an EOFError is placed on self._messages and the method returns. When any other exception is raised during processing, that exception is placed on self._messages and the method returns. The method never raises an exception to its caller."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_JsonRpcReader::run.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Each complete JSON-RPC message received on self._stream framed by HTTP-style headers that terminate with an empty line and contain a Content-Length field is decoded from JSON and placed as a Python object onto self._messages, in the order received. When self._stream reaches EOF, or when a payload is shorter than its declared Content-Length, an EOFError is placed on self._messages and the method returns. When any other exception is raised during processing, that exception is placed on self._messages and the method returns. The method never raises an exception to its caller.",
    "actual_behavior": "If the method `run()` returns (it may never return if no exception occurs), then the following holds. The binary stream `self._stream` remains open but its read position is advanced to an indeterminate point after the last read operation (successful or failed). The thread-safe queue `self._messages` now contains all elements it had before `run()` was called, followed by zero or more successfully parsed JSON messages (each a Python object obtained from a complete `content-length` delimited payload read after a set of HTTP-like headers), then followed by a single exception object `exc` that caused the loop to terminate. This exception is an instance of `BaseException` (or a subclass) and was placed into the queue by the `except` block. Formally, let `Q_before` be the sequence of items in `self._messages` before the call. After the call, the queue contains the sequence `Q_after = Q_before + [m_1, ..., m_k, exc]` where `k >= 0`, each `m_i` is the result of `json.loads(payload_i.decode('utf-8'))` for some `payload_i` read from the stream exactly after correctly parsing a header set that included a `content-length` field whose value was a non-negative integer, and `exc` is the exception instance that was caught in the `except BaseException as exc` block.",
    "code_evidence": "Line 20: self._messages.put(exc)",
    "trigger_condition": "The specification (B) states 'The method never raises an exception to its caller.' The code's except block calls self._messages.put(exc) without a try/except. If that put() call raises an exception, it will propagate to the caller, directly contradicting the specification."
  }
}
```

► Bug Validator result (完整)


```
{
  "id": "src--languages--erlang-py--_JsonRpcReader::run",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/languages/erlang.py",
  "function_name": "_JsonRpcReader::run",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_JsonRpcReader::run.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_JsonRpcReader::run.md",
  "probe_stdout": "CONFIRMED - RuntimeError propagated to caller: queue put failure\n",
  "trigger_summary": "self._messages.put(exc) in the except block (line 90) is not wrapped in try/except; if put() raises, the exception escapes run() to the caller, violating the spec guarantee that run() never raises."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:55:20.641281Z	2343
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:55:55.370083Z	18128

事件 1 — [llm_c182e0f48c3b4b9d9f18637fce0fbeb5](#)

► parsed_json（完整）

```
{
  "post_condition": "If the method `run()` returns (it may never return if no exception occurs), then the following holds. The binary stream `self._stream` remains open but its read position is advanced to an indeterminate point after the last read operation (successful or failed). The thread-safe queue `self._messages` now contains all elements it had before `run()` was called, followed by zero or more successfully parsed JSON messages (each a Python object obtained from a complete `content-length` delimited payload read after a set of HTTP-like headers), then followed by a single exception object `exc` that caused the loop to terminate. This exception is an instance of `BaseException` (or a subclass) and was placed into the queue by the `except` block. Formally, let `Q_before` be the sequence of items in `self._messages` before the call. After the call, the queue contains the sequence `Q_after = Q_before + [m_1, ..., m_k, exc]` where `k >= 0`, each `m_i` is the result of `json.loads(payload_i.decode('utf-8'))` for some `payload_i` read from the stream exactly after correctly parsing a header set that included a `content-length` field whose value was a non-negative integer, and `exc` is the exception instance that was caught in the `except BaseException as exc` block."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c182e0f48c3b4b9d9f18637fce0fbeb5_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c182e0f48c3b4b9d9f18637fce0fbeb5_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c182e0f48c3b4b9d9f18637fce0fbeb5_response.txt)

事件 2 — [llm_f2c186c9c6684a3ea3a858573c36d3d4](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input: self._stream is an empty io.BytesIO() (immediate EOF); self._messages is an object with a put() method that raises ValueError('queue closed'). Trace: run() calls readline() which returns b'', immediately raising EOFError('ELP closed its stdout') at line 8. Execution enters the except block at line 19. Line 20 calls self._messages.put(exc), which raises ValueError. This ValueError propagates out of run(), violating the requirement that the method never raises an exception to its caller.",
  "offending_statements": "Line 20: self._messages.put(exc)",
  "reason": "The specification (B) states 'The method never raises an exception to its caller.' The code's except block calls self._messages.put(exc) without a try/except. If that put() call raises an exception, it will propagate to the caller, directly contradicting the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f2c186c9c6684a3ea3a858573c36d3d4_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f2c186c9c6684a3ea3a858573c36d3d4_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f2c186c9c6684a3ea3a858573c36d3d4_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-091 — [src--languages--erlang-py--_SourceIndex::build](#)

- 四类结论： 契约不确定

- **分类依据:** [probe](#) 已复现 [SPEC](#) 与实现的差异, 但现有证据不足以决定应修改实现还是修改 [SPEC](#)。
- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 原源码 [src/languages/erlang.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a `_SourceIndex` instance that indexes source by line boundaries. The returned instance contains the full source text verbatim (with all line-ending characters preserved) and a list of zero-indexed byte offsets where each line begins within source. The first offset is always 0. Successive offsets increase monotonically each offset equals the sum of byte lengths of all preceding lines (including their line-ending characters). The number of offsets equals the number of newline-delimited lines in source. The returned instance enables callers to map any valid line index and character position to the corresponding byte offset within source, and to extract contiguous substrings spanning an inclusive start byte offset to an exclusive end byte offset.
- **Actual behavior:** Returns an instance `r` of `_SourceIndex` such that `r.source == source`, `r.lines == source.splitlines(keepends=True)`, and for all `i` in `0..len(r.lines)-1`: `r.line_offsets[i] == sum(len(r.lines[j]) for j in range(i))`.
- **Code evidence:** Line 7: `offset += len(line)`
- **Trigger condition:** The specification explicitly requires byte offsets into the source text, but the code uses `len(line)` which returns the number of Unicode code points (characters). For strings containing multi-byte characters (e.g., non-ASCII), character length differs from byte length, causing incorrect byte offsets. The counterexample demonstrates this with a string containing the two-byte character `"`.
- **Validator trigger:** Multi-byte UTF-8 character (`é`) causes `len()` to return character count (3) instead of byte length (4), producing incorrect byte offsets in `line_offsets`.
- **Probe stdout:** `CONFIRMED` — actual (char offsets): `[0, 3]` | expected (byte offsets): `[0, 4, 6]`

► [SPEC sidecar \(完整\)](#)

```
{
  "signature": "_SourceIndex.build(cls, source: str) -> _SourceIndex",
  "pre_condition": "source is a non-empty string containing source code text.",
  "post_condition": "Returns a _SourceIndex instance that indexes source by line boundaries. The returned instance contains the full source text verbatim (with all line-ending characters preserved) and a list of zero-indexed byte offsets where each line begins within source. The first offset is always 0. Successive offsets increase monotonically – each offset equals the sum of byte lengths of all preceding lines (including their line-ending characters). The number of offsets equals the number of newline-delimited lines in source. The returned instance enables callers to map any valid line index and character position to the corresponding byte offset within source, and to extract contiguous substrings spanning an inclusive start byte offset to an exclusive end byte offset."
}
```

► [INFO / callees sidecar \(完整\)](#)

```
{
  "callees": []
}
```

► [Logic verification result \(完整\)](#)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_SourceIndex::build.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a _SourceIndex instance that indexes source by line boundaries. The returned instance contains the full source text verbatim (with all line-ending characters preserved) and a list of zero-indexed byte offsets where each line begins within source. The first offset is always 0. Successive offsets increase monotonically each offset equals the sum of byte lengths of all preceding lines (including their line-ending characters). The number of offsets equals the number of newline-delimited lines in source. The returned instance enables callers to map any valid line index and character position to the corresponding byte offset within source, and to extract contiguous substrings spanning an inclusive start byte offset to an exclusive end byte offset.",
    "actual_behavior": "Returns an instance r of _SourceIndex such that r.source == source, r.lines == source.splitlines(keepends=True), and for all i in 0..len(r.lines)-1: r.line_offsets[i] == sum(len(r.lines[j]) for j in range(i)).",
    "code_evidence": "Line 7: offset += len(line)",
    "trigger_condition": "The specification explicitly requires byte offsets into the source text, but the code uses len(line) which returns the number of Unicode code points (characters). For strings containing multi-byte characters (e.g., non-ASCII), character length differs from byte length, causing incorrect byte offsets. The counterexample demonstrates this with a string containing the two-byte character '\".'"
  }
}
```

► [Bug Validator result \(完整\)](#)

```
{
  "id": "src--languages--erlang-py--_SourceIndex::build",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/languages/erlang.py",
  "function_name": "_SourceIndex.build",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_SourceIndex::build.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_SourceIndex::build.md",
  "probe_stdout": "CONFIRMED - actual (char offsets): [0, 3] | expected (byte offsets): [0, 4, 6]\n",
  "trigger_summary": "Multi-byte UTF-8 character (é) causes len() to return character count (3) instead of byte length (4), producing incorrect byte offsets in line_offsets."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:29:08.385080Z	1438
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:29:24.499742Z	3228

事件 1 — [llm_023b2ff038a04aa9bf1b5fb4dcc6605b](#)

► parsed_json（完整）

```
{
  "post_condition": "Returns an instance r of _SourceIndex such that r.source == source, r.lines == source.splitlines(keepends=True), and for all i in 0..len(r.lines)-1: r.line_offsets[i] == sum(len(r.lines[j]) for j in range(i))."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_023b2ff038a04aa9bf1b5fb4dcc6605b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_023b2ff038a04aa9bf1b5fb4dcc6605b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_023b2ff038a04aa9bf1b5fb4dcc6605b_response.txt)

事件 2 — [llm_c42243a98c914b0e8d492b350cccace5](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input source = \"café\\nhello\". In UTF-8, the first line 'café\\n' is 6 bytes, but len('café\\n') returns 5. The code computes line_offsets = [0, 5], while the specification requires byte offsets, so the correct starting byte offset for the second line is 6. The returned instance fails to provide the correct byte offset.",
  "offending_statements": "Line 7: offset += len(line)",
  "reason": "The specification explicitly requires byte offsets into the source text, but the code uses len(line) which returns the number of Unicode code points (characters). For strings containing multi-byte characters (e.g., non-ASCII), character length differs from byte length, causing incorrect byte offsets. The counterexample demonstrates this with a string containing the two-byte character 'é'."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c42243a98c914b0e8d492b350cccace5_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c42243a98c914b0e8d492b350cccace5_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c42243a98c914b0e8d492b350cccace5_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-092 — [src--languages--erlang-py--_SourceIndex::position_to_offset](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#)； attempts 1
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / [language_handlers](#)； 原源码 [src/languages/erlang.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns an integer in the closed interval $[0, \text{len}(\text{self.source})]$ representing the byte offset within `self.source` that corresponds to the given position. When the line number is at or beyond the number of indexed lines, returns `len(self.source)`. When the UTF-16 code-unit column offset meets or exceeds the number of UTF-16 code units on the target line, returns the byte offset of the first byte after that line. Characters with Unicode code points above U+FFFF advance the UTF-16 code-unit column by 2; characters at or below U+FFFF advance it by 1. For any two positions `p1` and `p2` where `p1` does not follow `p2` in document order relative to the indexed source, `position_to_offset(self, p1) <= position_to_offset(self, p2)`.
- **Actual behavior:** The method returns an integer offset `r` with no side effects. Let `line_number = max(0, int(position.get('line', 0)))` and `utf16_target = max(0, int(position.get('character', 0)))`. If `line_number >= len(self.lines)`, then `r = len(self.source)`. Otherwise, let `base = self.line_offsets[line_number]` and `L = self.lines[line_number]`. Define `unit(c) = 2` if `ord(c) > 0xFFFF` else 1. Let `index = max { k [0, len(L)] |  $\sum_{i=0}^{k-1} \text{unit}(L[i]) \leq \text{utf16\_target}$  }`. Then `r = base + index`. The mapping from position to offset uses character-based indexing on a line, adding to a byte-offset base, which may not correspond to the true byte offset if source contains multi-byte characters.
- **Code evidence:** Line 16: `return base + index`
- **Trigger condition:** The code treats the offset as a character-based index added to a byte-offset base. This only yields correct byte offsets when all characters are singlebyte (ASCII). With multibyte characters, the return value is not the required byte offset within `self.source`. For a character above U+FFFF, such as U+1F600 ("😄"), `len(line)=1` but its byte length is 4, so `base + index = 0 + 1 = 1`, which violates the specification that expects 4.
- **Validator trigger:** Call `_position_to_offset` with source '😄\n' and position {line: 0, character: 2}; returns character index 1 instead of byte offset 4 because `index += 1` counts characters, not bytes, for multi-byte UTF-8 characters.
- **Probe stdout:** CONFIRMED — actual: 1 | expected: 4

► SPEC sidecar (完整)

```
{
  "signature": "position_to_offset(self, position: dict) -> int",
  "pre_condition": "self is a _SourceIndex instance whose source, lines, and line_offsets attributes are initialized from the same source text; each entry in line_offsets is the byte offset within source of the corresponding line in lines. position is a dict with optional 'line' entry (defaulting to 0) representing a zero-indexed line number and optional 'character' entry (defaulting to 0) representing a zero-based UTF-16 code-unit column offset within that line.",
  "post_condition": "Returns an integer in the closed interval [0, len(self.source)] representing the byte offset within self.source that corresponds to the given position. When the line number is at or beyond the number of indexed lines, returns len(self.source). When the UTF-16 code-unit column offset meets or exceeds the number of UTF-16 code units on the target line, returns the byte offset of the first byte after that line. Characters with Unicode code points above U+FFFF advance the UTF-16 code-unit column by 2; characters at or below U+FFFF advance it by 1. For any two positions p1 and p2 where p1 does not follow p2 in document order relative to the indexed source, position_to_offset(self, p1) <= position_to_offset(self, p2)."
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_SourceIndex:position_to_offset.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns an integer in the closed interval [0, len(self.source)] representing the byte offset within self.source that corresponds to the given position. When the line number is at or beyond the number of indexed lines, returns len(self.source). When the UTF-16 code-unit column offset meets or exceeds the number of UTF-16 code units on the target line, returns the byte offset of the first byte after that line. Characters with Unicode code points above U+FFFF advance the UTF-16 code-unit column by 2; characters at or below U+FFFF advance it by 1. For any two positions p1 and p2 where p1 does not follow p2 in document order relative to the indexed source, position_to_offset(self, p1) <= position_to_offset(self, p2).",
    "actual_behavior": "The method returns an integer offset r with no side effects. Let line_number = max(0, int(position.get('line', 0))) and utf16_target = max(0, int(position.get('character', 0))). If line_number >= len(self.lines), then r = len(self.source). Otherwise, let base = self.line_offsets[line_number] and L = self.lines[line_number]. Define unit(c) = 2 if ord(c) > 0xFFFF else 1. Let index = max { k [0, len(L)] |  $\sum_{i=0}^{k-1} \text{unit}(L[i]) \leq \text{utf16\_target}$  }. Then r = base + index. The mapping from position to offset uses character-based indexing on a line, adding to a byte-offset base, which may not correspond to the true byte offset if source contains multi-byte characters.",
    "code_evidence": "Line 16: return base + index",
    "trigger_condition": "The code treats the offset as a character-based index added to a byte-offset base. This only yields correct byte offsets when all characters are singlebyte (ASCII). With multibyte characters, the return value is not the required byte offset within self.source. For a character above U+FFFF, such as U+1F600 ('😄'), len(line)=1 but its byte length is 4, so base + index = 0 + 1 = 1, which violates the specification that expects 4."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--_SourceIndex::position_to_offset",
  "source_file": "/home/fancy/Projects/Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_SourceIndex::position_to_offset.py",
  "function_name": "_SourceIndex::position_to_offset",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_SourceIndex::position_to_offset.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_SourceIndex::position_to_offset.md",
  "probe_stdout": "CONFIRMED — actual: 1 | expected: 4\n",
  "trigger_summary": "Call _position_to_offset with source '😊\n' and position {line: 0, character: 2}; returns character index 1 instead of byte offset 4 because index += 1 counts characters, not bytes, for multi-byte UTF-8 characters."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:39:06.866213Z	2851
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:39:54.207265Z	4402

事件 1 — [llm_b8564e5e13df42298ef40fa2f7d79850](#)

► parsed_json（完整）

```
{
  "post_condition": "The method returns an integer offset r with no side effects. Let line_number = max(0, int(position.get('line', 0))) and utf16_target = max(0, int(position.get('character', 0))). If line_number >= len(self.lines), then r = len(self.source). Otherwise, let base = self.line_offsets[line_number] and L = self.lines[line_number]. Define unit(c) = 2 if ord(c) > 0xFFFF else 1. Let index = max { k ∈ [0, len(L)] | Σ_{i=0}^{k-1} unit(L[i]) <= utf16_target }. Then r = base + index. The mapping from position to offset uses character-based indexing on a line, adding to a byte-offset base, which may not correspond to the true byte offset if source contains multi-byte characters."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b8564e5e13df42298ef40fa2f7d79850_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_b8564e5e13df42298ef40fa2f7d79850_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_b8564e5e13df42298ef40fa2f7d79850_response.txt)

事件 2 — [llm_d882e6f0c15347e3a2ad18773a09a4af](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "self.lines = ['\\U0001F600']; self.line_offsets = [0]; self.source = '\\U0001F600'; position = {'line': 0, 'character': 2}; self.position_to_offset(position) returns 1, but the specification requires the byte offset after the line, which is 4 (e.g., for UTF-8 encoding).",
  "offending_statements": "Line 16: return base + index",
  "reason": "The code treats the offset as a character-based index added to a byte-offset base. This only yields correct byte offsets when all characters are single-byte (ASCII). With multi-byte characters, the return value is not the required byte offset within self.source. For a character above U+FFFF, such as U+1F600 ('😊'), len(line)=1 but its byte length is 4, so base + index = 0 + 1 = 1, which violates the specification that expects 4."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d882e6f0c15347e3a2ad18773a09a4af_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d882e6f0c15347e3a2ad18773a09a4af_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d882e6f0c15347e3a2ad18773a09a4af_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-096 — [src--languages--erlang-py--_callgraph_project_root](#)

- 四类结论: 契约不确定
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1

- **Phase / 模块:** Phase 2 — Language Backend & Function Extraction / `language_handlers`; 原源码 `src/languages/erlang.py`
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns an absolute path string that identifies the original Erlang source project root. When `proj_dir` does not contain an 'extracted_functions' subdirectory, the absolute path of `proj_dir` is returned unchanged. When `proj_dir` contains an 'extracted_functions' subdirectory and its parent directory contains at least one file with the `.erl` extension, the absolute path of the parent directory is returned. Otherwise, the absolute path of `proj_dir` is returned.
- **Actual behavior:** The function returns a string that is the resolved original source-project root. Let `root = os.path.abspath(proj_dir)`. The following outcomes cover normal execution (no exceptions):
 1. If `os.path.isdir(os.path.join(root, "extracted_functions"))` is `False`, the function returns `root`.
 2. If `os.path.isdir(os.path.join(root, "extracted_functions"))` is `True`:
 - Let `parent = os.path.dirname(root)`.
 - If `parent == root`, the function returns `root`.
 - Else, if `parent` is an existing directory and `next(_iter_project_files(parent, {".erl"}), None)` is not `None` (i.e., `parent` contains at least one `.erl` file belonging to the original project source tree and not excluded as pipeline workspace artifact), the function returns `parent`.
 - Else (the iterator is exhausted or `parent` contains no such `.erl` files), the function returns `root`.

Formally, with `R = os.path.abspath(proj_dir)`, `E = os.path.join(R, "extracted_functions")`, `P = os.path.dirname(R)`:

- `((not os.path.isdir(E)) (os.path.isdir(E) (P = R (os.path.isdir(P) next(_iter_project_files(P, {".erl"}), None) is None)))` return value is `R`.
- `os.path.isdir(E) (P R) os.path.isdir(P) (next(_iter_project_files(P, {".erl"}), None) is not None)` return value is `P`.

Potential exceptions: If `os.path.isdir(E)` raises an `OSError` (e.g., due to inaccessible paths) the function aborts with that exception. If `os.path.isdir(E)` succeeds and we proceed to the branch that calls `_iter_project_files(P, ...)` but `P` does not exist or is not a directory, `_iter_project_files` raises a `FileNotFoundError` (or `NotADirectoryError`) because its precondition requires an existing directory. Any other I/O error during filesystem operations may also propagate as an exception.

- **Code evidence:** Line 11: if `next(_iter_project_files(parent, {".erl"}), None)` is not `None`:
- **Trigger condition:** The code uses `_iter_project_files` which skips certain directories (e.g., `fm_agent`) when scanning for `.erl` files. The specification requires returning the parent directory if it contains any `.erl` file, without exclusions. If all `.erl` files in the parent reside in excluded directories, the code returns `root` instead of `parent`, violating the specification.
- **Validator trigger:** `_callgraph_project_root` uses `_iter_project_files` which skips `_SKIP_DIRS`-excluded directories; `.erl` files inside excluded directories are invisible causing the function to return `root` instead of `parent`.
- **Probe stdout:** CONFIRMED — actual: `'/tmp/probe_callgraph_root_e2yt7cf1/erlang_project'` | expected: `'/tmp/probe_callgraph_root_e2yt7cf1'`

► SPEC sidecar (完整)

```
{
  "signature": "_callgraph_project_root(proj_dir: str) -> str",
  "pre_condition": "proj_dir is a string referring to a filesystem path. The path may be either the original Erlang source project root or a pipeline workspace directory derived from it.",
  "post_condition": "Returns an absolute path string that identifies the original Erlang source project root. When proj_dir does not contain an 'extracted_functions' subdirectory, the absolute path of proj_dir is returned unchanged. When proj_dir contains an 'extracted_functions' subdirectory and its parent directory contains at least one file with the .erl extension, the absolute path of the parent directory is returned. Otherwise, the absolute path of proj_dir is returned."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_iter_project_files",
      "signature": "_iter_project_files(dir_path: str, exts: set[str]) -> Iterator[str]",
      "pre_condition": "dir_path is a string referring to an existing directory. exts is a non-empty set of file extension strings, each including the leading dot.",
      "post_condition": "Returns an iterator that yields, one at a time, the absolute paths of files under dir_path (recursively) whose file extension is a member of exts. Only yields files belonging to the original project source tree – files under pipeline workspace artifact directories are excluded. If no matching files exist under dir_path, the iterator is immediately exhausted (the"
    }
  ]
}
```

```
first call to next() raises StopIteration)."
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-
py/_callgraph_project_root.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns an absolute path string that identifies the original Erlang source project root. When proj_dir does not
contain an 'extracted_functions' subdirectory, the absolute path of proj_dir is returned unchanged. When proj_dir contains an
'extracted_functions' subdirectory and its parent directory contains at least one file with the .erl extension, the absolute path
of the parent directory is returned. Otherwise, the absolute path of proj_dir is returned.",
    "actual_behavior": "The function returns a string that is the resolved original source-project root. Let `root =
os.path.abspath(proj_dir)`. The following outcomes cover normal execution (no exceptions):\n\n1. If
`os.path.isdir(os.path.join(root, \"extracted_functions\"))` is `False`, the function returns `root`. \n2. If
`os.path.isdir(os.path.join(root, \"extracted_functions\"))` is `True`: \n - Let `parent = os.path.dirname(root)`. \n - If
`parent == root`, the function returns `root`. \n - Else, if `parent` is an existing directory and
`next(_iter_project_files(parent, {\".erl\"}), None)` is not None` (i.e., `parent` contains at least one `.erl` file belonging to
the original project source tree and not excluded as pipeline workspace artifact), the function returns `parent`. \n - Else (the
iterator is exhausted or `parent` contains no such `.erl` files), the function returns `root`. \n\nFormally, with `R =
os.path.abspath(proj_dir)`, `E = os.path.join(R, \"extracted_functions\")`, `P = os.path.dirname(R)`: \n- `(not os.path.isdir(E))
(os.path.isdir(E) (P = R (os.path.isdir(P) next(_iter_project_files(P, {\".erl\"}), None) is None)))` return value is `R`. \n-
`os.path.isdir(E) (P = R) os.path.isdir(P) (next(_iter_project_files(P, {\".erl\"}), None) is not None)` return value is
`P`. \n\nPotential exceptions: If `os.path.isdir(E)` raises an `OSError` (e.g., due to inaccessible paths) the function aborts with
that exception. If `os.path.isdir(E)` succeeds and we proceed to the branch that calls `_iter_project_files(P, ...)` but `P` does
not exist or is not a directory, `_iter_project_files` raises a `FileNotFoundError` (or `NotADirectoryError`) because its
precondition requires an existing directory. Any other I/O error during filesystem operations may also propagate as an
exception.",
    "code_evidence": "Line 11: if next(_iter_project_files(parent, {\".erl\"}), None) is not None:",
    "trigger_condition": "The code uses _iter_project_files which skips certain directories (e.g., fm_agent) when scanning for
.erl files. The specification requires returning the parent directory if it contains any .erl file, without exclusions. If all
.erl files in the parent reside in excluded directories, the code returns root instead of parent, violating the specification."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--callgraph_project_root",
  "source_file": "fm_agent/extracted_functions/src/languages/erlang-py/_callgraph_project_root.py",
  "function_name": "_callgraph_project_root",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--callgraph_project_root.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--callgraph_project_root.md",
  "probe_stdout": "CONFIRMED - actual: '/tmp/probe_callgraph_root_e2yt7cf1/erlang_project' | expected:
'/tmp/probe_callgraph_root_e2yt7cf1'",
  "trigger_summary": "_callgraph_project_root uses _iter_project_files which skips _SKIP_DIRS-excluded directories; .erl files
inside excluded directories are invisible causing the function to return root instead of parent."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:48:30.737410Z	3707
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:49:13.090928Z	5073

事件 1 — [llm_6fe5ca04c9584740be2748c01991b57f](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string that is the resolved original source-project root. Let `root =
os.path.abspath(proj_dir)`. The following outcomes cover normal execution (no exceptions):\n\n1. If
`os.path.isdir(os.path.join(root, \"extracted_functions\"))` is `False`, the function returns `root`. \n2. If
`os.path.isdir(os.path.join(root, \"extracted_functions\"))` is `True`: \n - Let `parent = os.path.dirname(root)`. \n - If
`parent == root`, the function returns `root`. \n - Else, if `parent` is an existing directory and
`next(_iter_project_files(parent, {\".erl\"}), None)` is not None` (i.e., `parent` contains at least one `.erl` file belonging to
```



```

the original project source tree and not excluded as pipeline workspace artifact), the function returns `parent`.\\n - Else (the
iterator is exhausted or `parent` contains no such `.erl` files), the function returns `root`.\\n\\nFormally, with `R =
os.path.abspath(proj_dir)`, `E = os.path.join(R, \"extracted_functions\")`, `P = os.path.dirname(R):\\n- `(not os.path.isdir(E)) v
(os.path.isdir(E) ^ (P = R v (os.path.isdir(P) ^ next(_iter_project_files(P, {\".erl\"})), None) is None)))` ⇒ return value is
`R`.\\n- `os.path.isdir(E) ^ (P ≠ R) ^ os.path.isdir(P) ^ (next(_iter_project_files(P, {\".erl\"})), None) is not None)` ⇒ return
value is `P`.\\n\\nPotential exceptions: If `os.path.isdir(E)` raises an `OSError` (e.g., due to inaccessible paths) the function
aborts with that exception. If `os.path.isdir(E)` succeeds and we proceed to the branch that calls `_iter_project_files(P, ...)`
but `P` does not exist or is not a directory, `_iter_project_files` raises a `FileNotFoundError` (or `NotADirectoryError`) because
its pre-condition requires an existing directory. Any other I/O error during filesystem operations may also propagate as an
exception.\"
}

```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_6fe5ca04c9584740be2748c01991b57f_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_6fe5ca04c9584740be2748c01991b57f_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_6fe5ca04c9584740be2748c01991b57f_response.txt)

事件 2 — llm_db96a716490346b88e96fe141178395f

▶ parsed_json (完整)

```

{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp/test/project', where '/tmp/test/project/extracted_functions' is a directory and
'/tmp/test/fm_agent/test.erl' exists.",
  "offending_statements": "Line 11: if next(_iter_project_files(parent, {\".erl\"})), None) is not None:",
  "reason": "The code uses _iter_project_files which skips certain directories (e.g., fm_agent) when scanning for .erl files. The
specification requires returning the parent directory if it contains any .erl file, without exclusions. If all .erl files in the
parent reside in excluded directories, the code returns root instead of parent, violating the specification."
}

```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_db96a716490346b88e96fe141178395f_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_db96a716490346b88e96fe141178395f_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_db96a716490346b88e96fe141178395f_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-101 — src--languages--erlang-py--_position_to_offset

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: **confirmed**; attempts 1
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / **language_handlers**; 原源码 **src/languages/erlang.py**
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Returns the zero-indexed byte offset within source that corresponds to the given position. When the line index is at or beyond the number of lines in source, returns len(source). When the character value is at or beyond the end of the target line in UTF-16 code units, returns the byte offset of the last byte of that line. Characters with Unicode code points above U+FFFF advance the UTF-16 column by 2 units per character. The result is in the range [0, len(source)].
- **Actual behavior**: Returns the zero-indexed byte offset into the given **source** string corresponding to the position described by the **position** dictionary (with 'line' defaulting to 0 and 'character' defaulting to 0). The offset is clamped to the inclusive range [0, len(source)] and **character** counts in UTF-16 code units (code points above U+FFFF count as two units). For all valid inputs, the return value **r** is an integer such that:

$r == \min(\max(\text{offset_of_position}(\text{source}, \text{position.get('line', 0)}, \text{position.get('character', 0)}), 0), \text{len}(\text{source}))$

where **offset_of_position(s, l, c)** computes the byte offset in **s** of the first code unit of the character at line index **l** and UTF-16 column **c**, using line break boundaries in **s** to determine line lengths measured in UTF-16 code units.

- **Code evidence**: Line 3: `return _SourceIndex.build(source).position_to_offset(position)`
- **Trigger condition**: Specification requires that when the character value is at or beyond the end of the target line, the function returns the byte offset of the last byte of that line. For `source = "a\nb"`, line 0 is "a" (1 code unit). With `character = 1` (beyond the line), the expected offset is 0 (the byte offset of 'a'). However, the code delegates to `_SourceIndex.position_to_offset`, which, according to Condition A, computes the offset of the first code unit at the given line and column and then clamps it between 0 and `len(source)`. Since the column is beyond the line, `offset_of_position` likely returns the offset of the newline (1) or the start of the next line (2), which after clamping would be 1 or 2, violating the specification.

- **Validator trigger:** When character is at or beyond the line's UTF-16 length, `_SourceIndex.position_to_offset` returns `base+len(line)` (offset right after the line) instead of the byte offset of the last byte of that line.
- **Probe stdout:** CONFIRMED — source='a', char=1 beyond single-char line | actual: 1 | expected: 0 CONFIRMED — source='a\nb', line=0 char=2 beyond line | actual: 2 | expected: 1 CONFIRMED — source='x\ny\nz', line=1 char=2 beyond 'y\n' | actual: 4 | expected: 3

► SPEC sidecar (完整)

```
{
  "signature": "_position_to_offset(source, position) -> int",
  "pre_condition": "source is a non-empty string containing source code text. position is a dict whose 'line' entry (default 0) is a non-negative integer line index and whose 'character' entry (default 0) is a non-negative integer column offset in UTF-16 code units.",
  "post_condition": "Returns the zero-indexed byte offset within source that corresponds to the given position. When the line index is at or beyond the number of lines in source, returns len(source). When the character value is at or beyond the end of the target line in UTF-16 code units, returns the byte offset of the last byte of that line. Characters with Unicode code points above U+FFFF advance the UTF-16 column by 2 units per character. The result is in the range [0, len(source)]."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_SourceIndex.build",
      "signature": "build(cls, source: str) -> _SourceIndex",
      "pre_condition": "source is a non-empty string containing source code text.",
      "post_condition": "Returns a _SourceIndex instance that indexes the source text by line boundaries, enabling position-to-offset and range-to-source queries for the given source."
    },
    {
      "name": "_SourceIndex.position_to_offset",
      "signature": "position_to_offset(self, position: dict) -> int",
      "pre_condition": "self is a _SourceIndex instance built from the same source string handed to _position_to_offset. position is a dict with 'line' and 'character' entries (each defaulting to 0) representing a location in the indexed source.",
      "post_condition": "Returns the zero-indexed byte offset in the indexed source corresponding to the given position, clamped to [0, len(source)]. Characters above U+FFFF count as 2 UTF-16 code units."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_position_to_offset.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns the zero-indexed byte offset within source that corresponds to the given position. When the line index is at or beyond the number of lines in source, returns len(source). When the character value is at or beyond the end of the target line in UTF-16 code units, returns the byte offset of the last byte of that line. Characters with Unicode code points above U+FFFF advance the UTF-16 column by 2 units per character. The result is in the range [0, len(source)].",
    "actual_behavior": "Returns the zero-indexed byte offset into the given `source` string corresponding to the position described by the `position` dictionary (with 'line' defaulting to 0 and 'character' defaulting to 0). The offset is clamped to the inclusive range [0, len(source)] and `character` counts in UTF-16 code units (code points above U+FFFF count as two units). For all valid inputs, the return value `r` is an integer such that:\n\nr == min(max(offset_of_position(source, position.get('line', 0), position.get('character', 0)), 0), len(source))\n\nwhere `offset_of_position(s, l, c)` computes the byte offset in `s` of the first code unit of the character at line index `l` and UTF-16 column `c`, using line break boundaries in `s` to determine line lengths measured in UTF-16 code units.",
    "code_evidence": "Line 3: return _SourceIndex.build(source).position_to_offset(position)",
    "trigger_condition": "Specification requires that when the character value is at or beyond the end of the target line, the function returns the byte offset of the last byte of that line. For source = `a\\nb\\`, line 0 is `a\\` (1 code unit). With character = 1 (beyond the line), the expected offset is 0 (the byte offset of 'a'). However, the code delegates to _SourceIndex.position_to_offset, which, according to Condition A, computes the offset of the first code unit at the given line and column and then clamps it between 0 and len(source). Since the column is beyond the line, offset_of_position likely returns the offset of the newline (1) or the start of the next line (2), which after clamping would be 1 or 2, violating the specification."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--_position_to_offset",
  "source_file": "src/languages/erlang-py/_position_to_offset.py",
  "function_name": "_position_to_offset",
  "confirmation_status": "confirmed",
  "attempts": 1,
```

```
"probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_position_to_offset.py",
"detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_position_to_offset.md",
"probe_stdout": "CONFIRMED - source='a', char=1 beyond single-char line | actual: 1 | expected: 0\nCONFIRMED - source='a\\nb', line=0 char=2 beyond line | actual: 2 | expected: 1\nCONFIRMED - source='x\\ny\\nz', line=1 char=2 beyond 'y\\n' | actual: 4 | expected: 3",
"trigger_summary": "When character is at or beyond the line's UTF-16 length, _SourceIndex.position_to_offset returns base+len(line) (offset right after the line) instead of the byte offset of the last byte of that line."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:33:53.013605Z	2086
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:34:21.192054Z	5885

事件 1 — [llm_3f92cfe222314276a2a522b376aada72](#)

► parsed_json（完整）

```
{
  "post_condition": "Returns the zero-indexed byte offset into the given `source` string corresponding to the position described by the `position` dictionary (with 'line' defaulting to 0 and 'character' defaulting to 0). The offset is clamped to the inclusive range [0, len(source)] and `character` counts in UTF-16 code units (code points above U+FFFF count as two units). For all valid inputs, the return value `r` is an integer such that:\n\nr == min(max(offset_of_position(source, position.get('line', 0), position.get('character', 0)), 0), len(source))\n\nwhere `offset_of_position(s, l, c)` computes the byte offset in `s` of the first code unit of the character at line index `l` and UTF-16 column `c`, using line break boundaries in `s` to determine line lengths measured in UTF-16 code units."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_3f92cfe222314276a2a522b376aada72_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_3f92cfe222314276a2a522b376aada72_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_3f92cfe222314276a2a522b376aada72_response.txt)

事件 2 — [llm_c484863dad754f5ea623905d1fb782cb](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "source = \"a\\nb\\n\", position = {\"line\": 0, \"character\": 1}",
  "offending_statements": "Line 3: return _SourceIndex.build(source).position_to_offset(position)",
  "reason": "Specification requires that when the character value is at or beyond the end of the target line, the function returns the byte offset of the last byte of that line. For source = \"a\\nb\\n\", line 0 is \"a\" (1 code unit). With character = 1 (beyond the line), the expected offset is 0 (the byte offset of 'a'). However, the code delegates to _SourceIndex.position_to_offset, which, according to Condition A, computes the offset of the first code unit at the given line and column and then clamps it between 0 and len(source). Since the column is beyond the line, offset_of_position likely returns the offset of the newline (1) or the start of the next line (2), which after clamping would be 1 or 2, violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c484863dad754f5ea623905d1fb782cb_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c484863dad754f5ea623905d1fb782cb_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c484863dad754f5ea623905d1fb782cb_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-103 — [src--languages--erlang-py--_source_for_range](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#)； attempts 2
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / [language_handlers](#)； 原源码 [src/languages/erlang.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns the substring of source spanning from the byte offset corresponding to `lsp_range['start']` (inclusive) to the byte offset corresponding to `lsp_range['end']` (exclusive). Position-to-offset mapping follows the same rules as `_position_to_offset`: lines are zero-indexed, characters are counted in UTF-16 code units, characters above U+FFFF consume 2 code units, and out-of-range positions are clamped to source boundaries. The returned string contains all bytes of source between the resolved start and end offsets.
- **Actual behavior:** The return value is the substring of the input 'source' that lies between the byte offsets of `lsp_range['start']` (inclusive) and `lsp_range['end']` (exclusive), where the byte offsets are computed by the `_SourceIndex` built from 'source' using line boundaries and UTF-16 code unit positions. Formally, let `start_offset` be the byte offset in 'source' corresponding to `lsp_range['start']`, and `end_offset` be the byte offset corresponding to `lsp_range['end']`; then the function returns `source[start_offset:end_offset]`.
- **Code evidence:** Line 3: `return _SourceIndex.build(source).source_for_range(lsp_range)`
- **Trigger condition:** The code does not handle out-of-range positions; it blindly delegates to `_SourceIndex`. The specification requires clamping out-of-range positions to source boundaries, guaranteeing a string result. For `line=2` (past the end of 'a'), `_SourceIndex` may fail or return an offset that does not clamp, causing a mismatch.
- **Validator trigger:** When `start character > end character`, `source[start:end]` yields empty string but spec expects substring between resolved positions regardless of order.
- **Probe stdout:** `[bug_reproduction] actual=" expected='el'`

CONFIRMED — Code returns empty string when `start > end`, spec requires returning substring between resolved positions

► SPEC sidecar (完整)

```
{
  "signature": "_source_for_range(source, lsp_range) -> str",
  "pre_condition": "source is a non-empty string containing source code text. lsp_range is a dict with a 'start' entry and an 'end' entry, each being a dict with 'line' (non-negative integer) and 'character' (non-negative integer in UTF-16 code units) entries, where start does not follow end in the source.",
  "post_condition": "Returns the substring of source spanning from the byte offset corresponding to lsp_range['start'] (inclusive) to the byte offset corresponding to lsp_range['end'] (exclusive). Position-to-offset mapping follows the same rules as _position_to_offset: lines are zero-indexed, characters are counted in UTF-16 code units, characters above U+FFFF consume 2 code units, and out-of-range positions are clamped to source boundaries. The returned string contains all bytes of source between the resolved start and end offsets."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_SourceIndex.build",
      "signature": "build(cls, source: str) -> _SourceIndex",
      "pre_condition": "source is a non-empty string containing source code text.",
      "post_condition": "Returns a _SourceIndex instance that indexes the source text by line boundaries, enabling position-to-offset and range-to-source queries for the given source."
    },
    {
      "name": "_SourceIndex.source_for_range",
      "signature": "source_for_range(self, lsp_range: dict) -> str",
      "pre_condition": "self is a _SourceIndex instance built from the same source string handed to _source_for_range. lsp_range is a dict with 'start' and 'end' entries, each being a dict with 'line' and 'character' entries, where start does not follow end in the indexed source.",
      "post_condition": "Returns the substring of the indexed source from the byte offset of lsp_range['start'] (inclusive) to the byte offset of lsp_range['end'] (exclusive)."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_source_for_range.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns the substring of source spanning from the byte offset corresponding to lsp_range['start'] (inclusive) to the byte offset corresponding to lsp_range['end'] (exclusive). Position-to-offset mapping follows the same rules as _position_to_offset: lines are zero-indexed, characters are counted in UTF-16 code units, characters above U+FFFF consume 2 code units, and out-of-range positions are clamped to source boundaries. The returned string contains all bytes of source between the resolved start and end offsets.",
    "actual_behavior": "The return value is the substring of the input 'source' that lies between the byte offsets of lsp_range['start'] (inclusive) and lsp_range['end'] (exclusive), where the byte offsets are computed by the _SourceIndex built from 'source' using line boundaries and UTF-16 code unit positions. Formally, let start_offset be the byte offset in 'source' corresponding to lsp_range['start'], and end_offset be the byte offset corresponding to lsp_range['end']; then the function returns source[start_offset:end_offset].",
    "code_evidence": "Line 3: return _SourceIndex.build(source).source_for_range(lsp_range)",
  }
}
```

```
    "trigger_condition": "The code does not handle out-of-range positions; it blindly delegates to _SourceIndex. The specification requires clamping out-of-range positions to source boundaries, guaranteeing a string result. For line=2 (past the end of 'a'), _SourceIndex may fail or return an offset that does not clamp, causing a mismatch."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--_source_for_range",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_source_for_range.py",
  "function_name": "_source_for_range",
  "confirmation_status": "confirmed",
  "attempts": "2",
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_source_for_range.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_source_for_range.md",
  "probe_stdout": "[bug_reproduction] actual='' expected='el'\n\n\nCONFIRMED – Code returns empty string when start > end, spec requires returning substring between resolved positions\n",
  "trigger_summary": "When start character > end character, source[start:end] yields empty string but spec expects substring between resolved positions regardless of order."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:34:13.341123Z	1784
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:34:33.248943Z	6517

事件 1 — [llm_51449b121c3f4cf6ba28ca47366b2198](#)

► parsed_json（完整）

```
{
  "post_condition": "The return value is the substring of the input 'source' that lies between the byte offsets of lsp_range['start'] (inclusive) and lsp_range['end'] (exclusive), where the byte offsets are computed by the _SourceIndex built from 'source' using line boundaries and UTF-16 code unit positions. Formally, let start_offset be the byte offset in 'source' corresponding to lsp_range['start'], and end_offset be the byte offset corresponding to lsp_range['end']; then the function returns source[start_offset:end_offset]."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_51449b121c3f4cf6ba28ca47366b2198_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_51449b121c3f4cf6ba28ca47366b2198_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_51449b121c3f4cf6ba28ca47366b2198_response.txt)

事件 2 — [llm_5dfdd76f0f1b4c5592343ea84563bc43](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "source = 'a', lsp_range = {'start': {'line': 2, 'character': 0}, 'end': {'line': 2, 'character': 1}}. Expected: specification (B) requires clamping out-of-range positions to source boundaries, yielding offsets (1, 1) and an empty string. Actual: the code delegates to _SourceIndex, which likely raises an exception or returns an incorrect substring for an out-of-range line, violating the requirement to return a string.",
  "offending_statements": "Line 3: return _SourceIndex.build(source).source_for_range(lsp_range)",
  "reason": "The code does not handle out-of-range positions; it blindly delegates to _SourceIndex. The specification requires clamping out-of-range positions to source boundaries, guaranteeing a string result. For line=2 (past the end of 'a'), _SourceIndex may fail or return an offset that does not clamp, causing a mismatch."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5dfdd76f0f1b4c5592343ea84563bc43_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5dfdd76f0f1b4c5592343ea84563bc43_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5dfdd76f0f1b4c5592343ea84563bc43_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts [1](#)
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 原源码 [src/languages/go.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a dict mapping absolute file paths to lists of (func_name, body_text) tuples for all Go source files in proj_dir when a CodeGraph index for the project is available. Returns an empty dict when CodeGraph initialization fails or is not available for the project. Each key is an absolute filepath string; each value is a list of 2-tuples whose first element is a function name and second element is the function body text.
- **Actual behavior**: If the call to CodeGraphExtractor.from_proj_dir(proj_dir) raises an exception, the function batch_extract terminates abnormally with that exception. If no exception occurs, let cg = CodeGraphExtractor.from_proj_dir(proj_dir). Then the function returns: if cg is None, the value {} (empty dictionary); otherwise, the value cg.get_functions_by_file("go", proj_dir), which is a dictionary mapping absolute file paths (str) to lists of (func_name, body) tuples for all Go source files under the project root. Formally: (Exception) ((cg = None) result = {}) ((cg None) result = cg.get_functions_by_file("go", proj_dir) k,v result: isinstance(k, str) isabs(k) isinstance(v, list) t v: isinstance(t, tuple) len(t)=2).
- **Code evidence**: Line 3: cg = CodeGraphExtractor.from_proj_dir(proj_dir)
- **Trigger condition**: The specification requires returning an empty dict when CodeGraph initialization fails, which includes any exception from from_proj_dir. The code directly calls from_proj_dir without exception handling, so if it raises (e.g., FileNotFoundError for an invalid path), batch_extract propagates the exception instead of returning {}. This violates the specification.
- **Validator trigger**: Pass None as proj_dir to trigger TypeError in os.path.join inside CodeGraphExtractor.from_proj_dir, which batch_extract propagates instead of returning {}
- **Probe stdout**: CONFIRMED — batch_extract(None) raised TypeError instead of returning {}: expected str, bytes or os.PathLike object, not NoneType

► SPEC sidecar (完整)

```
{
  "signature": "batch_extract(proj_dir: str) -> dict",
  "pre_condition": "proj_dir is a non-empty string identifying a project root directory on disk.",
  "post_condition": "Returns a dict mapping absolute file paths to lists of (func_name, body_text) tuples for all Go source files in proj_dir when a CodeGraph index for the project is available. Returns an empty dict when CodeGraph initialization fails or is not available for the project. Each key is an absolute filepath string; each value is a list of 2-tuples whose first element is a function name and second element is the function body text."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "CodeGraphExtractor.from_proj_dir",
      "signature": "from_proj_dir(proj_dir: str) -> CodeGraphExtractor | None",
      "pre_condition": "proj_dir is a non-empty string identifying a project root directory.",
      "post_condition": "Returns a CodeGraphExtractor instance initialized with the project's code index if the index exists and is loadable. Returns None when no CodeGraph index is available for the project or initialization fails."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/go-py/batch_extract.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dict mapping absolute file paths to lists of (func_name, body_text) tuples for all Go source files in proj_dir when a CodeGraph index for the project is available. Returns an empty dict when CodeGraph initialization fails or is not available for the project. Each key is an absolute filepath string; each value is a list of 2-tuples whose first element is a function name and second element is the function body text.",
    "actual_behavior": "If the call to CodeGraphExtractor.from_proj_dir(proj_dir) raises an exception, the function batch_extract terminates abnormally with that exception. If no exception occurs, let cg = CodeGraphExtractor.from_proj_dir(proj_dir). Then the function returns: if cg is None, the value {} (empty dictionary); otherwise, the value cg.get_functions_by_file(\"go\", proj_dir),"
```



```
which is a dictionary mapping absolute file paths (str) to lists of (func_name, body) tuples for all Go source files under the project root. Formally: ( Exception) ((cg = None) result = {}) ((cg None) result = cg.get_functions_by_file(\"go\", proj_dir) k,v result: isinstance(k, str) isabs(k) isinstance(v, list) t v: isinstance(t, tuple) len(t)=2).",
"code_evidence": "Line 3: cg = CodeGraphExtractor.from_proj_dir(proj_dir)",
"trigger_condition": "The specification requires returning an empty dict when CodeGraph initialization fails, which includes any exception from from_proj_dir. The code directly calls from_proj_dir without exception handling, so if it raises (e.g., FileNotFoundError for an invalid path), batch_extract propagates the exception instead of returning {}. This violates the specification."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--go-py--batch_extract",
  "source_file": "src/languages/go.py",
  "function_name": "batch_extract",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--languages--go-py--batch_extract.py",
  "detail_file": "fm_agent/bug_validation/src--languages--go-py--batch_extract.md",
  "probe_stdout": "CONFIRMED - batch_extract(None) raised TypeError instead of returning {}: expected str, bytes or os.PathLike object, not NoneType\n",
  "trigger_summary": "Pass None as proj_dir to trigger TypeError in os.path.join inside CodeGraphExtractor.from_proj_dir, which batch_extract propagates instead of returning {}"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:33:51.004029Z	1519
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:34:08.415357Z	2883

事件 1 — [llm_74fd594f6f284436b5e5f4f88550c976](#)

► parsed_json（完整）

```
{
  "post_condition": "If the call to CodeGraphExtractor.from_proj_dir(proj_dir) raises an exception, the function batch_extract terminates abnormally with that exception. If no exception occurs, let cg = CodeGraphExtractor.from_proj_dir(proj_dir). Then the function returns: if cg is None, the value {} (empty dictionary); otherwise, the value cg.get_functions_by_file(\"go\", proj_dir), which is a dictionary mapping absolute file paths (str) to lists of (func_name, body) tuples for all Go source files under the project root. Formally: (¬ Exception) ⇒ ((cg = None) ⇒ result = {}) ∧ ((cg ≠ None) ⇒ result = cg.get_functions_by_file(\"go\", proj_dir) ∧ ∀k,v ∈ result: isinstance(k, str) ∧ isabs(k) ∧ isinstance(v, list) ∧ ∀t ∈ v: isinstance(t, tuple) ∧ len(t)=2).\"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_74fd594f6f284436b5e5f4f88550c976_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_74fd594f6f284436b5e5f4f88550c976_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_74fd594f6f284436b5e5f4f88550c976_response.txt)

事件 2 — [llm_bf9f88aee4654e998056c532e5a5fe1e](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/nonexistent/path' (a nonexistent directory causing from_proj_dir to raise an exception)",
  "offending_statements": "Line 3: cg = CodeGraphExtractor.from_proj_dir(proj_dir)",
  "reason": "The specification requires returning an empty dict when CodeGraph initialization fails, which includes any exception from from_proj_dir. The code directly calls from_proj_dir without exception handling, so if it raises (e.g., FileNotFoundError for an invalid path), batch_extract propagates the exception instead of returning {}. This violates the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_bf9f88aee4654e998056c532e5a5fe1e_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_bf9f88aee4654e998056c532e5a5fe1e_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_bf9f88aee4654e998056c532e5a5fe1e_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-111 — [src--languages--java-py--batch_extract](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异， 但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#)； attempts [1](#)
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / [language_handlers](#)； 原源码 [src/languages/java.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**： Returns a dict mapping each absolute file path (str) to a list of (function_name: str, function_body: str) tuples for all discovered functions in Java source files under proj_dir. Each function_name is a canonicalized fully-qualified name using '::' as the separator with path-unsafe characters translated. Each function_body contains the complete source text of that function. Returns an empty dict if the CodeGraph backend cannot be initialized for the project.
- **Actual behavior**： If an exception is raised during CodeGraphExtractor.from_proj_dir(proj_dir), the function batch_extract terminates with that exception and no further state holds. Otherwise, the function returns normally. Let result be the returned value. If the call from_proj_dir(proj_dir) returned a falsy value (None or otherwise), result is the empty dict {}. If it returned a truthy CodeGraphExtractor instance cg, result is the dict produced by cg.get_functions_by_file('java', proj_dir). This dict maps each absolute file path (str) for a Java source file in proj_dir to a list of (function_name: str, function_body: str) tuples. Each function_name is a canonicalized fully-qualified name using '::' as separator. No other keys or values are present, and proj_dir remains unchanged.
- **Code evidence**： Line 3: cg = CodeGraphExtractor.from_proj_dir(proj_dir)
- **Trigger condition**： The specification requires that batch_extract returns an empty dict if the CodeGraph backend cannot be initialized, which implies graceful handling of any initialization failure, including exceptions. The code does not catch exceptions from from_proj_dir, so if that call raises an exception (e.g., FileNotFoundError for a nonexistent directory), the function propagates the exception instead of returning an empty dict, violating the specification.
- **Validator trigger**： Calling batch_extract(None) causes os.path.abspath to raise TypeError, which propagates uncaught instead of returning {} as the spec requires.
- **Probe stdout**： CONFIRMED: batch_extract(None) raised TypeError: expected str, bytes or os.PathLike object, not NoneType Spec requires returning {}; got exception instead PASS: batch_extract('/tmp/x00test') returned {} as spec requires CONFIRMED — batch_extract propagates exception(s) instead of returning {}

► SPEC sidecar（完整）

```
{
  "signature": "batch_extract(proj_dir) -> dict",
  "pre_condition": "proj_dir is a string path to an existing project root directory containing Java source files.",
  "post_condition": "Returns a dict mapping each absolute file path (str) to a list of (function_name: str, function_body: str) tuples for all discovered functions in Java source files under proj_dir. Each function_name is a canonicalized fully-qualified name using '::' as the separator with path-unsafe characters translated. Each function_body contains the complete source text of that function. Returns an empty dict if the CodeGraph backend cannot be initialized for the project."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "CodeGraphExtractor.from_proj_dir",
      "signature": "from_proj_dir(proj_dir) -> CodeGraphExtractor | None",
      "pre_condition": "proj_dir is a string path to an existing project root directory with source files.",
      "post_condition": "Returns a CodeGraphExtractor instance initialized with the project's code graph index if the index exists and is valid; returns None or a falsy value if initialization fails."
    },
    {
      "name": "CodeGraphExtractor.get_functions_by_file",
      "signature": "get_functions_by_file(language_key, proj_dir) -> dict",
      "pre_condition": "The receiver is a valid, initialized CodeGraphExtractor instance; language_key is a string identifying a supported language; proj_dir is a string path to the project root.",
      "post_condition": "Returns a dict mapping each absolute file path (str) to a list of (function_name: str, function_body: str) tuples for all functions extracted from source files matching the given language key. Each function_name is a canonicalized fully-qualified name using '::' as the separator."
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/java-py/batch_extract.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dict mapping each absolute file path (str) to a list of (function_name: str, function_body: str) tuples for all discovered functions in Java source files under proj_dir. Each function_name is a canonicalized fully-qualified name using '::' as the separator with path-unsafe characters translated. Each function_body contains the complete source text of that function. Returns an empty dict if the CodeGraph backend cannot be initialized for the project.",
    "actual_behavior": "If an exception is raised during CodeGraphExtractor.from_proj_dir(proj_dir), the function batch_extract terminates with that exception and no further state holds. Otherwise, the function returns normally. Let result be the returned value. If the call from_proj_dir(proj_dir) returned a falsy value (None or otherwise), result is the empty dict {}. If it returned a truthy CodeGraphExtractor instance cg, result is the dict produced by cg.get_functions_by_file('java', proj_dir). This dict maps each absolute file path (str) for a Java source file in proj_dir to a list of (function_name: str, function_body: str) tuples. Each function_name is a canonicalized fully-qualified name using '::' as separator. No other keys or values are present, and proj_dir remains unchanged.",
    "code_evidence": "Line 3: cg = CodeGraphExtractor.from_proj_dir(proj_dir)",
    "trigger_condition": "The specification requires that batch_extract returns an empty dict if the CodeGraph backend cannot be initialized, which implies graceful handling of any initialization failure, including exceptions. The code does not catch exceptions from from_proj_dir, so if that call raises an exception (e.g., FileNotFoundError for a nonexistent directory), the function propagates the exception instead of returning an empty dict, violating the specification."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--java-py--batch_extract",
  "source_file": "src/languages/java.py",
  "function_name": "batch_extract",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--java-py--batch_extract.py",
  "detail_file": "fm_agent/bug_validation/src--languages--java-py--batch_extract.md",
  "probe_stdout": "CONFIRMED: batch_extract(None) raised TypeError: expected str, bytes or os.PathLike object, not NoneType\nSpec requires returning {}; got exception instead\nPASS: batch_extract('/tmp/\\x00test') returned {} as spec requires\nCONFIRMED - batch_extract propagates exception(s) instead of returning {}",
  "trigger_summary": "Calling batch_extract(None) causes os.path.abspath to raise TypeError, which propagates uncaught instead of returning {} as the spec requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:35:49.795569Z	1711
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:36:11.161867Z	2848

事件 1 — [llm_c06788b7e4984b5fb2d60e7e4a5de2d5](#)

► parsed_json（完整）

```
{
  "post_condition": "If an exception is raised during CodeGraphExtractor.from_proj_dir(proj_dir), the function batch_extract terminates with that exception and no further state holds. Otherwise, the function returns normally. Let result be the returned value. If the call from_proj_dir(proj_dir) returned a falsy value (None or otherwise), result is the empty dict {}. If it returned a truthy CodeGraphExtractor instance cg, result is the dict produced by cg.get_functions_by_file('java', proj_dir). This dict maps each absolute file path (str) for a Java source file in proj_dir to a list of (function_name: str, function_body: str) tuples. Each function_name is a canonicalized fully-qualified name using '::' as separator. No other keys or values are present, and proj_dir remains unchanged."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c06788b7e4984b5fb2d60e7e4a5de2d5_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c06788b7e4984b5fb2d60e7e4a5de2d5_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c06788b7e4984b5fb2d60e7e4a5de2d5_response.txt)

事件 2 — [llm_344e30602b794516b7ad72b0ad58cd27](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/non_existent_project_path'",
  "offending_statements": "Line 3: cg = CodeGraphExtractor.from_proj_dir(proj_dir)",
  "reason": "The specification requires that batch_extract returns an empty dict if the CodeGraph backend cannot be initialized, which implies graceful handling of any initialization failure, including exceptions. The code does not catch exceptions from from_proj_dir, so if that call raises an exception (e.g., FileNotFoundError for a nonexistent directory), the function propagates the exception instead of returning an empty dict, violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_344e30602b794516b7ad72b0ad58cd27_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_344e30602b794516b7ad72b0ad58cd27_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_344e30602b794516b7ad72b0ad58cd27_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

Phase 3 — Pipeline Planning & Domain Knowledge

模块: domain_knowledge

FMA-MISMATCH-119 — [src--domain_knowledge-py--_flatten_paths](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 3 — Pipeline Planning & Domain Knowledge / [domain_knowledge](#); 原源码 [src/domain_knowledge.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a flat list of string elements obtained by recursively flattening all nested iterables in values into a single list, preserving the relative order of elements from the original structure. Elements that are empty strings or otherwise falsy are excluded from the result.
- **Actual behavior:** The function returns a flat list containing all nonempty strings from the input [values](#) (or an empty list if [values](#) is [None](#) [✔](#)if x != " else []; else if x is a list or tuple: concatenation over e in x of flatten(e). Then the return value R satisfies R = flatten(values).
- **Code evidence:** Line 6: elif value: Line 7: paths.append(value)
- **Trigger condition:** The specification requires a flat list of string elements, but the code appends any truthy non-list/non-tuple value directly, including non-strings like integers. This violates the output type requirement.
- **Validator trigger:** Passing non-string truthy values (integers) to [_flatten_paths](#) causes them to be included in the output, violating the spec requirement of a flat list of string elements.
- **Probe stdout:** CONFIRMED — Test 1 passed: [_flatten_paths](#)([1, 2, 3]) returned [1, 2, 3] but spec expects only strings ([]) CONFIRMED — Test 2 passed: [_flatten_paths](#)(['a', 1, 'b']) returned ['a', 1, 'b'] but spec expects only strings (['a', 'b']) CONFIRMED — Test 3 passed: [_flatten_paths](#)([[1, 2], 'a']) returned [1, 2, 'a'] but spec expects only strings (['a'])

► SPEC sidecar (完整)

```
{
  "signature": "_flatten_paths(values)",
  "pre_condition": "values, if provided, is an iterable whose elements may be strings or nested iterables of strings; None is treated as an empty iterable",
  "post_condition": "Returns a flat list of string elements obtained by recursively flattening all nested iterables in values into a single list, preserving the relative order of elements from the original structure. Elements that are empty strings or otherwise falsy are excluded from the result."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/domain_knowledge-py/_flatten_paths.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a flat list of string elements obtained by recursively flattening all nested iterables in values into a single list, preserving the relative order of elements from the original structure. Elements that are empty strings or otherwise falsy are excluded from the result.",
    "actual_behavior": "The function returns a flat list containing all nonempty strings from the input `values` (or an empty list if `values` is `None`). Recursively, for any nested list or tuple, all its elements are flattened in depthfirst order, preserving the original sequence. Falsy strings (e.g., empty strings) are omitted. Formally, define function flatten(x) as: if x is None or x is empty iterable: []; else if x is a string: [x] if x != '' else []; else if x is a list or tuple: concatenation over e in x of flatten(e). Then the return value R satisfies R = flatten(values).",
    "code_evidence": "Line 6: elif value:\nLine 7:         paths.append(value)",
    "trigger_condition": "The specification requires a flat list of string elements, but the code appends any truthy non-list/non-tuple value directly, including non-strings like integers. This violates the output type requirement."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--domain_knowledge-py--_flatten_paths",
  "source_file": "fm_agent/extracted_functions/src/domain_knowledge-py/_flatten_paths.py",
  "function_name": "_flatten_paths",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--domain_knowledge-py--_flatten_paths.py",
  "detail_file": "fm_agent/bug_validation/src--domain_knowledge-py--_flatten_paths.md",
  "probe_stdout": "CONFIRMED - Test 1 passed: _flatten_paths([1, 2, 3]) returned [1, 2, 3] but spec expects only strings ([])\nCONFIRMED - Test 2 passed: _flatten_paths(['a', 1, 'b']) returned ['a', 1, 'b'] but spec expects only strings (['a', 'b'])\nCONFIRMED - Test 3 passed: _flatten_paths([[1, 2], 'a']) returned [1, 2, 'a'] but spec expects only strings (['a'])",
  "trigger_summary": "Passing non-string truthy values (integers) to _flatten_paths causes them to be included in the output, violating the spec requirement of a flat list of string elements."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:31:59.380003Z	1275
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:32:12.716335Z	1924

事件 1 — [llm_24c575e76533449a89195f0dae949ff1](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a flat list containing all non-empty strings from the input `values` (or an empty list if `values` is `None`). Recursively, for any nested list or tuple, all its elements are flattened in depth-first order, preserving the original sequence. Falsy strings (e.g., empty strings) are omitted. Formally, define function flatten(x) as: if x is None or x is empty iterable: []; else if x is a string: [x] if x != '' else []; else if x is a list or tuple: concatenation over e in x of flatten(e). Then the return value R satisfies R = flatten(values)."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_24c575e76533449a89195f0dae949ff1_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_24c575e76533449a89195f0dae949ff1_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_24c575e76533449a89195f0dae949ff1_response.txt)

事件 2 — [llm_e93e6e1c4c2f402fafd6b399aa683b65](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "[1]",
  "offending_statements": "Line 6: elif value:\nLine 7:           paths.append(value)",
  "reason": "The specification requires a flat list of string elements, but the code appends any truthful non-list/non-tuple value directly, including non-strings like integers. This violates the output type requirement."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_e93e6e1c4c2f402fafd6b399aa683b65_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_e93e6e1c4c2f402fafd6b399aa683b65_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_e93e6e1c4c2f402fafd6b399aa683b65_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-124 — [src--domain_knowledge-py--stage_domain_knowledge_files](#)

- 四类结论：契约不确定
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**： [confirmed](#)； [attempts 1](#)
- **Phase / 模块**： Phase 3 — Pipeline Planning & Domain Knowledge / [domain_knowledge](#)； 原源码 [src/domain_knowledge.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**： When `markdown_paths` is falsy (None or empty): the contents of the directory `spec_prompts/domain_context/user_knowledge/` under `work_dir` are unchanged. A sorted list of relative path strings (using `/` separator, prefixed with `fm_agent/`) to all markdown files present in that directory is returned. When `markdown_paths` is truthful: the directory `spec_prompts/domain_context/user_knowledge/` under `work_dir` is atomically cleared and repopulated with exactly one copy of each markdown file resolved from `markdown_paths` plus a manifest.json. Each staged file is given a name derived from its source basename with special characters replaced by underscores; if the derived name collides with an already-used name in the batch, a numeric suffix is appended to ensure uniqueness. The manifest.json file in that directory records, for every staged file, the original absolute source path (`source_path`) and the staged relative path prefixed with `fm_agent/` (`staged_path`). A sorted list of relative path strings (using `/` separator, prefixed with `fm_agent/`) to all staged markdown files in the directory is returned.
- **Actual behavior**： If the function terminates normally (no exception is raised):
 - If `markdown_paths` is falsy (None or an empty sequence), the function returns `list_staged_domain_knowledge_relpaths(work_dir)` and leaves the filesystem unmodified.
 - If `markdown_paths` is a nonempty sequence, let `resolved = resolve_domain_knowledge_paths(markdown_paths, base_dir=proj_dir, fallback_base_dir=os.getcwd())`; assume this call succeeds (otherwise an exception would be thrown). Define `target_dir = os.path.join(work_dir, USER_KNOWLEDGE_REL_DIR)` where `USER_KNOWLEDGE_REL_DIR` is the constant path segment `spec_prompts/domain_context/user_knowledge`. After normal execution:
 - `target_dir` exists and contains exactly one file per element of `resolved` and a JSON manifest file named `USER_KNOWLEDGE_MANIFEST`.
 - For each `i = 0, ..., len(resolved)-1`, let `name_i = _safe_staged_name(resolved[i], {name_0, ..., name_{i-1}})` (with an empty set for `i=0`). Then `target_dir/name_i` is a copy of the source file `resolved[i]` (preserving file metadata via `shutil.copy2`), and all `name_i` are pairwise distinct.
 - The manifest file contains `{"files": entries}` where `entries` is a list of length `len(resolved)`. For each index `i`, the `i`th entry has `"source_path": resolved[i]` and `"staged_path": "fm_agent/" + rel_path_i` where `rel_path_i` is the path of `target_dir/name_i` relative to `work_dir` with all directory separators replaced by `/`.
 - The replacement is atomic: either the old `target_dir` remains untouched (if a failure occurs before the final `os.replace`) or it is completely replaced by the new content; no partial or mixed state can be observed.
 - The function returns `list_staged_domain_knowledge_relpaths(work_dir)`. According to its specification, this return value is a sorted list of `fm_agent/`prefixed relative paths (using `/` separators, in lexicographic ascending order) to every markdown file present in `target_dir`. The manifest file is not included in this list.

If an exception is raised at any point (for example from `resolve_domain_knowledge_paths`, `os.makedirs`, `shutil.copy2`, JSON writing, or `os.replace`), the function does not return normally, and the original `target_dir` (the userknowledge directory) remains exactly as it was before the call no partial modifications persist.

- **Code evidence**： Line 36: `shutil.rmtree(target_dir, ignore_errors=True)` Line 37: `os.replace(tmp_dir, target_dir)`
- **Trigger condition**： The specification requires that the directory be 'atomically cleared and repopulated', meaning there should be no observable intermediate state where the directory is empty. The code deletes the target directory on Line 36 before replacing it on Line 37.

This creates a window where the directory does not exist, violating the atomicity requirement. Any non-empty markdown_paths input triggers this violation.

- **Validator trigger:** Any non-empty markdown_paths triggers shutil.rmtree before os.replace, creating an observable window where target_dir does not exist, violating the atomicity requirement.
- **Probe stdout:** CONFIRMED — actual: 'directory was missing between rmtree and replace (atomicity violated)' | expected: 'atomically cleared and repopulated (no window where directory missing)'

► SPEC sidecar (完整)

```
{
  "signature": "stage_domain_knowledge_files(proj_dir, work_dir, markdown_paths=None) -> list[str]",
  "pre_condition": "work_dir is an absolute path to an existing, writable directory managed by FM-Agent. markdown_paths is either None, an empty sequence, or a non-empty sequence of file path strings.",
  "post_condition": "When markdown_paths is falsy (None or empty): the contents of the directory spec_prompts/domain_context/user_knowledge/ under work_dir are unchanged. A sorted list of relative path strings (using `\\` separator, prefixed with `fm_agent/`) to all markdown files present in that directory is returned. When markdown_paths is truthy: the directory spec_prompts/domain_context/user_knowledge/ under work_dir is atomically cleared and repopulated with exactly one copy of each markdown file resolved from markdown_paths plus a manifest.json. Each staged file is given a name derived from its source basename with special characters replaced by underscores; if the derived name collides with an already-used name in the batch, a numeric suffix is appended to ensure uniqueness. The manifest.json file in that directory records, for every staged file, the original absolute source path (`source_path`) and the staged relative path prefixed with `fm_agent/` (`staged_path`). A sorted list of relative path strings (using `\\` separator, prefixed with `fm_agent/`) to all staged markdown files in the directory is returned."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "resolve_domain_knowledge_paths",
      "signature": "resolve_domain_knowledge_paths(paths, base_dir, fallback_base_dir)",
      "pre_condition": "paths is a non-empty sequence of file path strings. base_dir is an absolute path to an existing directory. fallback_base_dir is an absolute path to an existing directory.",
      "post_condition": "Returns a list of absolute, validated paths to existing markdown files. Every returned path is a valid markdown file accessible on the filesystem."
    },
    {
      "name": "_safe_staged_name",
      "signature": "_safe_staged_name(source_path, used_names) -> str",
      "pre_condition": "source_path is an absolute path to an existing file. used_names is a set of strings representing already-reserved staged filenames.",
      "post_condition": "Returns a filename string that is derived from the source file's basename with all special characters replaced by underscores. The returned name is guaranteed distinct from every name in used_names. When the derived name is not in used_names, it is returned as-is; when it collides with a name in used_names, a numeric suffix is appended to produce a unique name."
    },
    {
      "name": "list_staged_domain_knowledge_relpaths",
      "signature": "list_staged_domain_knowledge_relpaths(work_dir) -> list[str]",
      "pre_condition": "work_dir is an absolute path to an existing, FM-Agent-managed workspace directory. The directory spec_prompts/domain_context/user_knowledge/ exists under work_dir.",
      "post_condition": "Returns a sorted list of relative path strings (using `\\` separator, prefixed with `fm_agent/`) to every markdown file present in the user knowledge directory under work_dir. The list is in lexicographic ascending order."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/domain_knowledge-py/stage_domain_knowledge_files.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When markdown_paths is falsy (None or empty): the contents of the directory spec_prompts/domain_context/user_knowledge/ under work_dir are unchanged. A sorted list of relative path strings (using `\\` separator, prefixed with `fm_agent/`) to all markdown files present in that directory is returned. When markdown_paths is truthy: the directory spec_prompts/domain_context/user_knowledge/ under work_dir is atomically cleared and repopulated with exactly one copy of each markdown file resolved from markdown_paths plus a manifest.json. Each staged file is given a name derived from its source basename with special characters replaced by underscores; if the derived name collides with an already-used name in the batch, a numeric suffix is appended to ensure uniqueness. The manifest.json file in that directory records, for every staged file, the original absolute source path (`source_path`) and the staged relative path prefixed with `fm_agent/` (`staged_path`). A sorted list of relative path strings (using `\\` separator, prefixed with `fm_agent/`) to all staged markdown files in the directory is returned.",
    "actual_behavior": "If the function terminates normally (no exception is raised):\n - If `markdown_paths` is falsy (None or an empty sequence), the function returns `list_staged_domain_knowledge_relpaths(work_dir)` and leaves the filesystem unmodified.\n"
```

```

    If `markdown_paths` is a non-empty sequence, let `resolved` = resolve_domain_knowledge_paths(markdown_paths, base_dir=proj_dir,
    fallback_base_dir=os.getcwd()); assume this call succeeds (otherwise an exception would be thrown). Define `target_dir` =
    os.path.join(work_dir, USER_KNOWLEDGE_REL_DIR) where `USER_KNOWLEDGE_REL_DIR` is the constant path segment
    `spec_prompts/domain_context/user_knowledge`. After normal execution:\n    * `target_dir` exists and contains exactly one file per
    element of `resolved` and a JSON manifest file named `USER_KNOWLEDGE_MANIFEST`.\n    * For each `i = 0,...,len(resolved)-1`, let
    `name_i` = _safe_staged_name(resolved[i], {name_0,...,name_{i-1}}) (with an empty set for `i=0`). Then `target_dir/name_i` is a
    copy of the source file `resolved[i]` (preserving file metadata via `shutil.copy2`), and all `name_i` are pairwise distinct.\n
    * The manifest file contains `{"files": entries}` where `entries` is a list of length `len(resolved)`. For each index `i`, the
    i-th entry has `{"source_path": resolved[i]` and `{"staged_path": `fm_agent/" + rel_path_i` where `rel_path_i` is the path of
    `target_dir/name_i` relative to `work_dir` with all directory separators replaced by `/.`.\n    * The replacement is atomic: either
    the old `target_dir` remains untouched (if a failure occurs before the final `os.replace`) or it is completely replaced by the new
    content; no partial or mixed state can be observed.\n    * The function returns `list_staged_domain_knowledge_relpaths(work_dir)`.
    According to its specification, this return value is a sorted list of `fm_agent/`-prefixed relative paths (using `/.` separators, in
    lexicographic ascending order) to every markdown file present in `target_dir`. The manifest file is not included in this
    list.\n\nIf an exception is raised at any point (for example from `resolve_domain_knowledge_paths`, `os.makedirs`, `shutil.copy2`,
    JSON writing, or `os.replace`), the function does not return normally, and the original `target_dir` (the userknowledge directory)
    remains exactly as it was before the call - no partial modifications persist.",
    "code_evidence": "Line 36: shutil.rmtree(target_dir, ignore_errors=True)\nLine 37: os.replace(tmp_dir, target_dir)",
    "trigger_condition": "The specification requires that the directory be 'atomically cleared and repopulated', meaning there
    should be no observable intermediate state where the directory is empty. The code deletes the target directory on Line 36 before
    replacing it on Line 37. This creates a window where the directory does not exist, violating the atomicity requirement. Any non-
    empty markdown_paths input triggers this violation."
  }
}
```

► Bug Validator result（完整）

```

{
  "id": "src--domain_knowledge-py--stage_domain_knowledge_files",
  "source_file": "/home/fancy/Projects/Vault/FM-Agent/fm_agent/extracted_functions/src/domain_knowledge-
py/stage_domain_knowledge_files.py",
  "function_name": "stage_domain_knowledge_files",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--domain_knowledge-py--stage_domain_knowledge_files.py",
  "detail_file": "fm_agent/bug_validation/src--domain_knowledge-py--stage_domain_knowledge_files.md",
  "probe_stdout": "CONFIRMED - actual: 'directory was missing between rmtree and replace (atomicity violated)' | expected:
'atomically cleared and repopulated (no window where directory missing)'\n",
  "trigger_summary": "Any non-empty markdown_paths triggers shutil.rmtree before os.replace, creating an observable window where
target_dir does not exist, violating the atomicity requirement."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:06:04.124441Z	5249
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:07:08.085339Z	11025

事件 1 — [llm_b5ff27558f624e1085a9dfd03281035e](#)

► parsed_json（完整）

```

{
  "post_condition": "If the function terminates normally (no exception is raised):\n - If `markdown_paths` is falsy (None or an
empty sequence), the function returns `list_staged_domain_knowledge_relpaths(work_dir)` and leaves the filesystem unmodified.\n -
If `markdown_paths` is a non-empty sequence, let `resolved` = resolve_domain_knowledge_paths(markdown_paths, base_dir=proj_dir,
fallback_base_dir=os.getcwd()); assume this call succeeds (otherwise an exception would be thrown). Define `target_dir` =
os.path.join(work_dir, USER_KNOWLEDGE_REL_DIR) where `USER_KNOWLEDGE_REL_DIR` is the constant path segment
`spec_prompts/domain_context/user_knowledge`. After normal execution:\n    * `target_dir` exists and contains exactly one file per
element of `resolved` and a JSON manifest file named `USER_KNOWLEDGE_MANIFEST`.\n    * For each `i = 0,...,len(resolved)-1`, let
`name_i` = _safe_staged_name(resolved[i], {name_0,...,name_{i-1}}) (with an empty set for `i=0`). Then `target_dir/name_i` is a
copy of the source file `resolved[i]` (preserving file metadata via `shutil.copy2`), and all `name_i` are pairwise distinct.\n
    * The manifest file contains `{"files": entries}` where `entries` is a list of length `len(resolved)`. For each index `i`, the
i-th entry has `{"source_path": resolved[i]` and `{"staged_path": `fm_agent/" + rel_path_i` where `rel_path_i` is the path of
`target_dir/name_i` relative to `work_dir` with all directory separators replaced by `/.`.\n    * The replacement is atomic: either
the old `target_dir` remains untouched (if a failure occurs before the final `os.replace`) or it is completely replaced by the new
content; no partial or mixed state can be observed.\n    * The function returns `list_staged_domain_knowledge_relpaths(work_dir)`.
According to its specification, this return value is a sorted list of `fm_agent/`-prefixed relative paths (using `/.` separators,
in lexicographic ascending order) to every markdown file present in `target_dir`. The manifest file is not included in this
list.\n\nIf an exception is raised at any point (for example from `resolve_domain_knowledge_paths`, `os.makedirs`, `shutil.copy2`,
JSON writing, or `os.replace`), the function does not return normally, and the original `target_dir` (the user-knowledge
directory) remains exactly as it was before the call - no partial modifications persist."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b5ff27558f624e1085a9dfd03281035e_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_b5ff27558f624e1085a9dfd03281035e_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_b5ff27558f624e1085a9dfd03281035e_response.txt)

事件 2 — [llm_5b3402e7bcfe43e6adcaf314773ef868](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp/proj', work_dir='/tmp/work', markdown_paths=['doc.md'] (assuming necessary directories and source file exist)",
  "offending_statements": "Line 36: shutil.rmtree(target_dir, ignore_errors=True)\nLine 37: os.replace(tmp_dir, target_dir)",
  "reason": "The specification requires that the directory be 'atomically cleared and repopulated', meaning there should be no observable intermediate state where the directory is empty. The code deletes the target directory on Line 36 before replacing it on Line 37. This creates a window where the directory does not exist, violating the atomicity requirement. Any non-empty markdown_paths input triggers this violation."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5b3402e7bcfe43e6adcaf314773ef868_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_5b3402e7bcfe43e6adcaf314773ef868_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_5b3402e7bcfe43e6adcaf314773ef868_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: pipeline_setup

FMA-MISMATCH-125 — [src--pipeline_setup-py--_build_domain_context_regen_prompt](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts 1
- Phase / 模块: Phase 3 — Pipeline Planning & Domain Knowledge / [pipeline_setup](#); 原源码 [src/pipeline_setup.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim:** Returns a string containing a complete agent instruction prompt for regenerating per-phase domain-context type files. The prompt lists each phase number in phase_source_files in ascending order, each with the directive to rewrite the corresponding phase_NN_types.txt file from scratch using the phase's current source files. When phase_source_files is empty or yields no entries, the prompt states that no phase type files need regeneration. The prompt forbids modifying files under user_knowledge/ and restricts edits to engine_overview.txt and phase_NN_types.txt only. If phase_cleanup, after filtering out None-valued entries and identity renumberings (old == new), contains at least one removed phase or one renumbered phase where old differs from new, the prompt instructs the agent to update engine_overview.txt to match the final phase numbers after cleanup. Otherwise, the prompt instructs the agent to modify engine_overview.txt only if it explicitly references a changed phase number.
- Actual behavior:** Natural language: The function build_domain_context_regen_prompt returns a string S and has no side effects on the input arguments, no exceptions, and no other observable state changes. S follows the pattern: 'Regenerate per-phase domain-context files under fm_agent/spec_prompts/domain_context/ (named phase_NN_types.txt) so each reflects its phase's CURRENT source_files:\n\n' + changes_text + '\n\nRules:\n- Base each file on the types in that phase's source files; do not invent new types.\n- Do NOT modify any other files. Only edit engine_overview.txt and phase_NN_types.txt files directly under fm_agent/spec_prompts/domain_context/. Do NOT edit files under fm_agent/spec_prompts/domain_context/user_knowledge/. \n' + overview_rule. The changes_text is built as: if phase_source_files is empty, it is ' - No phase_NN_types.txt files need regeneration; only update engine_overview.txt if cleanup made its phase references stale.'; otherwise, it is a newline-separated list of lines, one per phase number in sorted ascending order, each line containing the instruction ' - phase : REGENERATE phasesum:02dtypes.txt from scratch. Its source_files are now: . READ them in the project and write the real types/structs/invariants they define. Do not leave the file empty or invent content.' The overview_rule is determined by processing phase_cleanup (None is treated as {}). removed_phases is the list obtained from phase_cleanup['removed_phases'] with any None entries removed; renumbered_changes is the dict from phase_cleanup['renumbered'] excluding any pair where old is None, new is None, or old equals new. If removed_phases is non-empty or renumbered_changes is non-empty, overview_rule = ' - Review engine_overview.txt and update any phase-numbered references so they match the final phases.json after cleanup (' + cleanup_details + ').' where cleanup_details is a semicolon-joined string of the removal list (sorted removed phase numbers prefixed 'removed old phase number(s): ') and the renumbering list (sorted old->new pairs prefixed 'renumbered surviving phase(s): ') as applicable. Otherwise, overview_rule = ' - engine_overview.txt does not need updating; touch it only if it explicitly names a phase number that changed.' Formal logic: For any psf: dict[int, list[str]], pc: None | dict, let effective_pc = pc if pc is not None else {}; let removed = [p for p in effective_pc.get('removed_phases', []) if p is not None]; let renum = {o: n for o, n in effective_pc.get('renumbered',

}}.items() if o is not None and n is not None and o != n}. Let S be the return value. Then S = 'Regenerate per-phase domain-context files under fm_agent/spec_prompts/domain_context/ (named phase_NN_types.txt) so each reflects its phase's CURRENT source_files:\n\n' + C + '\n\nRules:\n- Base each file on the types in that phase's source files; do not invent new types.\n- Do NOT modify any other files. Only edit engine_overview.txt and phase_NN_types.txt files directly under fm_agent/spec_prompts/domain_context/. Do NOT edit files under fm_agent/spec_prompts/domain_context/user_knowledge/. \n' + R, where C equals ' - No phase_NN_types.txt files need regeneration; only update engine_overview.txt if cleanup made its phase references stale.' if len(psf) == 0 else '\n'.join([' - phase ' + str(num) + ': REGENERATE phase' + f'{num:02d}' + '_types.txt from scratch. Its source_files are now: ' + ', '.join(psf[num]) + '. READ them in the project and write the real types/structs/invariants they define. Do not leave the file empty or invent content.' for num in sorted(psf)]), and R equals ' - Review engine_overview.txt and update any phase-numbered references so they match the final phases.json after cleanup (' + cleanup_strs + ').' if (removed != [] or renum != {}) else ' - engine_overview.txt does not need updating; touch it only if it explicitly names a phase number that changed.', where cleanup_strs = '; '.join(filter(None, [(f'removed old phase number(s): ' + ', '.join(str(p) for p in sorted(removed))) if removed else None, (f'renumbered surviving phase(s): ' + ', '.join(f'{o} -> {n}' for o,n in sorted(renum.items())) if renum else None])). The function raises no exception and does not mutate any inputs.

- Code evidence:** Line 31: renumbered_changes = { Line 32: old: new for old, new in renumbered.items() Line 33: if old is not None and new is not None and old != new Line 34: }
 - Trigger condition:** The specification says to filter out 'None-valued entries' from the renumbered mapping, which conventionally means entries whose value is None. The code additionally filters out entries where the key (old) is None. For an input like {None: 2}, the specification would retain the entry and thus require an engine_overview.txt update instruction. The code discards it, resulting in a different instruction string.
 - Validator trigger:** When phase_cleanup[renumbered] contains a key of None with a non-None value (e.g. {None: 2}), the code discards the entry due to the **old is not None** check at line 84, though the spec only requires filtering None-valued entries (where value is None).
 - Probe stdout:** CONFIRMED — actual output says 'does not need updating' (None-keyed entry {None: 2} was discarded by the code); expected output should include 'Review engine_overview.txt and update' (spec keeps None-keyed entry when value is not None)
- SPEC sidecar (完整)

```
{
  "signature": "_build_domain_context_regen_prompt(phase_source_files, phase_cleanup=None) -> str",
  "pre_condition": "phase_source_files is a dict mapping phase numbers (integers) to lists of source file path strings for phases needing domain-context regeneration; phase_cleanup is either None or a dict with optional keys 'removed_phases' (list of phase numbers) and 'renumbered' (dict mapping old to new phase numbers)",
  "post_condition": "Returns a string containing a complete agent instruction prompt for regenerating per-phase domain-context type files. The prompt lists each phase number in phase_source_files in ascending order, each with the directive to rewrite the corresponding phase_NN_types.txt file from scratch using the phase's current source files. When phase_source_files is empty or yields no entries, the prompt states that no phase type files need regeneration. The prompt forbids modifying files under user_knowledge/ and restricts edits to engine_overview.txt and phase_NN_types.txt only. If phase_cleanup, after filtering out None-valued entries and identity renumberings (old == new), contains at least one removed phase or one renumbered phase where old differs from new, the prompt instructs the agent to update engine_overview.txt to match the final phase numbers after cleanup. Otherwise, the prompt instructs the agent to modify engine_overview.txt only if it explicitly references a changed phase number."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-py/_build_domain_context_regen_prompt.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a string containing a complete agent instruction prompt for regenerating per-phase domain-context type files. The prompt lists each phase number in phase_source_files in ascending order, each with the directive to rewrite the corresponding phase_NN_types.txt file from scratch using the phase's current source files. When phase_source_files is empty or yields no entries, the prompt states that no phase type files need regeneration. The prompt forbids modifying files under user_knowledge/ and restricts edits to engine_overview.txt and phase_NN_types.txt only. If phase_cleanup, after filtering out None-valued entries and identity renumberings (old == new), contains at least one removed phase or one renumbered phase where old differs from new, the prompt instructs the agent to update engine_overview.txt to match the final phase numbers after cleanup. Otherwise, the prompt instructs the agent to modify engine_overview.txt only if it explicitly references a changed phase number.",
    "actual_behavior": "Natural language: The function _build_domain_context_regen_prompt returns a string S and has no side effects on the input arguments, no exceptions, and no other observable state changes. S follows the pattern: 'Regenerate per-phase domain-context files under fm_agent/spec_prompts/domain_context/ (named phase_NN_types.txt) so each reflects its phase's CURRENT source_files:\n\n' + changes_text + '\n\nRules:\n- Base each file on the types in that phase\'s source files; do not invent new types.\n- Do NOT modify any other files. Only edit engine_overview.txt and phase_NN_types.txt files directly under fm_agent/spec_prompts/domain_context/. Do NOT edit files under fm_agent/spec_prompts/domain_context/user_knowledge/. \n' + overview_rule. The changes_text is built as: if phase_source_files is empty, it is ' - No phase_NN_types.txt files need regeneration; only update engine_overview.txt if cleanup made its phase references stale.'; otherwise, it is a newline-separated list of lines, one per phase number in sorted ascending order, each line containing the instruction ' - phase <num>: REGENERATE
```

```
phase_<num:02d>_types.txt from scratch. Its source_files are now: <comma-space-separated list of file paths>. READ them in the project and write the real types/structs/invariants they define. Do not leave the file empty or invent content.' The overview_rule is determined by processing phase_cleanup (None is treated as {}). removed_phases is the list obtained from phase_cleanup['removed_phases'] with any None entries removed; renumbered_changes is the dict from phase_cleanup['renumbered'] excluding any pair where old is None, new is None, or old equals new. If removed_phases is non-empty or renumbered_changes is non-empty, overview_rule = '- Review engine_overview.txt and update any phase-numbered references so they match the final phases.json after cleanup (' + cleanup_details + ') where cleanup_details is a semicolon-joined string of the removal list (sorted removed phase numbers prefixed 'removed old phase number(s): ') and the renumbering list (sorted old->new pairs prefixed 'renumbered surviving phase(s): ') as applicable. Otherwise, overview_rule = '- engine_overview.txt does not need updating; touch it only if it explicitly names a phase number that changed.' Formal logic: For any psf: dict[int, list[str]], pc: None | dict, let effective_pc = pc if pc is not None else {}; let removed = [p for p in effective_pc.get('removed_phases', []) if p is not None]; let renum = {o: n for o, n in effective_pc.get('renumbered', {}).items()} if o is not None and n is not None and o != n. Let S be the return value. Then S = 'Regenerate per-phase domain-context files under fm_agent/spec_prompts/domain_context/ (named phase_NN_types.txt) so each reflects its phase\\'s CURRENT source_files:\\n\\n' + C + '\\n\\nRules:\\n- Base each file on the types in that phase\\'s source files; do not invent new types.\\n- Do NOT modify any other files. Only edit engine_overview.txt and phase_NN_types.txt files directly under fm_agent/spec_prompts/domain_context/. Do NOT edit files under fm_agent/spec_prompts/domain_context/user_knowledge/.\\n' + R, where C equals ' - No phase_NN_types.txt files need regeneration; only update engine_overview.txt if cleanup made its phase references stale.' if len(psf) == 0 else '\\n'.join([' - phase ' + str(num) + ': REGENERATE phase_' + f'{num:02d}' + '_types.txt from scratch. Its source_files are now: ' + ', '.join(psf[num]) + '. READ them in the project and write the real types/structs/invariants they define. Do not leave the file empty or invent content.' for num in sorted(psf)]), and R equals '- Review engine_overview.txt and update any phase-numbered references so they match the final phases.json after cleanup (' + cleanup_strs + ') if (removed != [] or renum != {}) else '- engine_overview.txt does not need updating; touch it only if it explicitly names a phase number that changed.', where cleanup_strs = '; '.join(filter(None, [('removed old phase number(s): ' + ', '.join(str(p) for p in sorted(removed)) if removed else None, ('renumbered surviving phase(s): ' + ', '.join(f'{o} -> {n}' for o,n in sorted(renum.items())) if renum else None])). The function raises no exception and does not mutate any inputs.",
"code_evidence": "Line 31:         renumbered_changes = {\nLine 32:             old: new for old, new in renumbered.items()\nLine 33: if old is not None and new is not None and old != new\nLine 34:         }",
"trigger_condition": "The specification says to filter out 'None-valued entries' from the renumbered mapping, which conventionally means entries whose value is None. The code additionally filters out entries where the key (old) is None. For an input like {None: 2}, the specification would retain the entry and thus require an engine_overview.txt update instruction. The code discards it, resulting in a different instruction string."
}
```

► Bug Validator result (完整)

```
{
  "id": "src--pipeline_setup-py--_build_domain_context_regen_prompt",
  "source_file": "/home/fancy/Projects/Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-py/_build_domain_context_regen_prompt.py",
  "function_name": "_build_domain_context_regen_prompt",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--pipeline_setup-py--_build_domain_context_regen_prompt.py",
  "detail_file": "fm_agent/bug_validation/src--pipeline_setup-py--_build_domain_context_regen_prompt.md",
  "probe_stdout": "CONFIRMED - actual output says 'does not need updating' (None-keyed entry {None: 2} was discarded by the code); expected output should include 'Review engine_overview.txt and update' (spec keeps None-keyed entry when value is not None)\n",
  "trigger_summary": "When phase_cleanup['renumbered'] contains a key of None with a non-None value (e.g. {None: 2}), the code discards the entry due to the `old is not None` check at line 84, though the spec only requires filtering None-valued entries (where value is None)."
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:19:54.756440Z	4518
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:20:36.875400Z	5855

事件 1 — [llm_2b17b606da3e4c4bb7328325a78beee3](#)

► parsed_json (完整)

```
{
  "post_condition": "Natural language: The function _build_domain_context_regen_prompt returns a string S and has no side effects on the input arguments, no exceptions, and no other observable state changes. S follows the pattern: 'Regenerate per-phase domain-context files under fm_agent/spec_prompts/domain_context/ (named phase_NN_types.txt) so each reflects its phase's CURRENT source_files:\\n\\n' + changes_text + '\\n\\nRules:\\n- Base each file on the types in that phase\\'s source files; do not invent new types.\\n- Do NOT modify any other files. Only edit engine_overview.txt and phase_NN_types.txt files directly under fm_agent/spec_prompts/domain_context/. Do NOT edit files under fm_agent/spec_prompts/domain_context/user_knowledge/.\\n' + overview_rule. The changes_text is built as: if phase_source_files is empty, it is ' - No phase_NN_types.txt files need regeneration; only update engine_overview.txt if cleanup made its phase references stale.'; otherwise, it is a newline-separated list of lines, one per phase number in sorted ascending order, each line containing the instruction ' - phase <num>: REGENERATE phase_<num:02d>_types.txt from scratch. Its source_files are now: <comma-space-separated list of file paths>. READ them in the project and write the real types/structs/invariants they define. Do not leave the file empty or invent content.' The overview_rule
```



```
is determined by processing phase_cleanup (None is treated as {}). removed_phases is the list obtained from
phase_cleanup['removed_phases'] with any None entries removed; renumbered_changes is the dict from phase_cleanup['renumbered']
excluding any pair where old is None, new is None, or old equals new. If removed_phases is non-empty or renumbered_changes is non-
empty, overview_rule = '- Review engine_overview.txt and update any phase-numbered references so they match the final phases.json
after cleanup (' + cleanup_details + ').' where cleanup_details is a semicolon-joined string of the removal list (sorted removed
phase numbers prefixed 'removed old phase number(s): ') and the renumbering list (sorted old->new pairs prefixed 'renumbered
surviving phase(s): ') as applicable. Otherwise, overview_rule = '- engine_overview.txt does not need updating; touch it only if
it explicitly names a phase number that changed.' Formal logic: For any psf: dict[int, list[str]], pc: None | dict, let
effective_pc = pc if pc is not None else {}; let removed = [p for p in effective_pc.get('removed_phases', []) if p is not None];
let renum = {o: n for o, n in effective_pc.get('renumbered', {}).items() if o is not None and n is not None and o != n}. Let S be
the return value. Then S = 'Regenerate per-phase domain-context files under fm_agent/spec_prompts/domain_context/ (named
phase_NN_types.txt) so each reflects its phase\\'s CURRENT source_files:\\n\\n' + C + '\\n\\nRules:\\n- Base each file on the
types in that phase\\'s source files; do not invent new types.\\n- Do NOT modify any other files. Only edit engine_overview.txt
and phase_NN_types.txt files directly under fm_agent/spec_prompts/domain_context/. Do NOT edit files under
fm_agent/spec_prompts/domain_context/user_knowledge/.\\n' + R, where C equals ' - No phase_NN_types.txt files need regeneration;
only update engine_overview.txt if cleanup made its phase references stale.' if len(psf) == 0 else '\\n'.join([' - phase ' +
str(num) + ': REGENERATE phase_' + f'{num:02d}' + '_types.txt from scratch. Its source_files are now: ' + ', '.join(psf[num]) + '.
READ them in the project and write the real types/structs/invariants they define. Do not leave the file empty or invent content.'
for num in sorted(psf)]), and R equals '- Review engine_overview.txt and update any phase-numbered references so they match the
final phases.json after cleanup (' + cleanup_strs + ').' if (removed != [] or renum != {}) else '- engine_overview.txt does not
need updating; touch it only if it explicitly names a phase number that changed.', where cleanup_strs = '; '.join(filter(None,
[('removed old phase number(s): ' + ', '.join(str(p) for p in sorted(removed))) if removed else None, ('renumbered surviving
phase(s): ' + ', '.join(f'{o} -> {n}' for o,n in sorted(renum.items())) if renum else None])). The function raises no exception
and does not mutate any inputs."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_2b17b606da3e4c4bb7328325a78beee3_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_2b17b606da3e4c4bb7328325a78beee3_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_2b17b606da3e4c4bb7328325a78beee3_response.txt)

事件 2 — llm_a85bbf046f7f438cb3be696bf7872703

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input: phase_source_files={1: ['a.py']}, phase_cleanup={'renumbered': {None: 2}}. Code returns a prompt with
overview_rule = '- engine_overview.txt does not need updating; touch it only if it explicitly names a phase number that changed.'
The specification requires an instruction to update engine_overview.txt because after filtering out None-valued entries (the
entry's value is 2, not None) and identity renumberings, there remains a renumbered phase where old (None) differs from new (2).",
  "offending_statements": "Line 31:      renumbered_changes = {\nLine 32:          old: new for old, new in renumbered.items()\nLine
33:          if old is not None and new is not None and old != new\nLine 34:      }",
  "reason": "The specification says to filter out 'None-valued entries' from the renumbered mapping, which conventionally means
entries whose value is None. The code additionally filters out entries where the key (old) is None. For an input like {None: 2},
the specification would retain the entry and thus require an engine_overview.txt update instruction. The code discards it,
resulting in a different instruction string."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_a85bbf046f7f438cb3be696bf7872703_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_a85bbf046f7f438cb3be696bf7872703_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_a85bbf046f7f438cb3be696bf7872703_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-134 — src--pipeline_setup-py--_phase_plan_schema_errors

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: **confirmed**; attempts 1
- Phase / 模块：Phase 3 — Pipeline Planning & Domain Knowledge / **pipeline_setup**；源代码 **src/pipeline_setup.py**
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Returns a list of human-readable error strings. The returned list is empty (falsy) if and only if all of the following hold: (1) a regular file exists at phases_path and can be opened for reading, (2) the file content is syntactically valid JSON, (3) the top-level JSON value is an object, (4) that object contains a key named "phases" whose value is an array, (5) every element of the "phases" array is an object, (6) every phase object contains a key named "modules" whose value is an array, (7) every element of a "modules" array is an object, (8) every module object contains a key named "source_files" whose value is an array, and (9) every element of a "source_files" array is a string. The returned list is non-empty (truthy) otherwise, with at least one element describing the first encountered blocking failure or all accumulated non-blocking structural violations.

- **Actual behavior:** The function returns a list of strings describing schema errors found in the file at `phases_path`. The behavior is deterministic and covers all exceptions internally, never propagating them.

In natural language:

- If the file cannot be opened (`OSError`), the result is a single-element list with a message starting with `'phases.json could not be read: '` and the exception string.
- If the file is opened but contains invalid JSON (`JSONDecodeError`), the result is a single-element list with a message starting with `'phases.json is not valid JSON: '` and the exception string.
- Otherwise, the JSON is successfully decoded into a Python value `D`.
 - If `D` is not a dictionary, the result is `['the top-level value must be an object']`.
 - Else if `D` lacks a `'phases'` key or its value is not a list, the result is `['top-level field "phases" must be an array']`.
 - Else `D['phases']` is a list. The function initializes an empty errors list and iterates over each phase by index `i`:
 - If the phase is not a dict, appends `'phases[i] must be an object'` and continues to the next phase.
 - Else, checks the `'modules'` key. If missing or not a list, appends `'phases[i].modules must be an array'` and continues.
 - Else, for each module at index `j`:
 - If the module is not a dict, appends `'phases[i].modules[j] must be an object'` and continues to the next module.
 - Else, determines the context string: `module_path = 'phases[i].modules[j]'`; if `module.get('name')` is truthy, context becomes `"phases[i].modules[j] ('name')"`, else context is just `module_path`.
 - If `'source_files'` is missing from the module, appends `'{context}.source_files is missing'` and continues.
 - Else, let `source_files = module['source_files']`. If `source_files` is not a list, appends `'{context}.source_files must be an array'` and continues.
 - Else, for each source_file at index `k`, if it is not a string, appends `'{context}.source_files[k] must be a string'`.
 - Finally, the function returns the accumulated errors list (possibly empty).

Formal logic (using `D` for decoded JSON, `P` for `phases_path`): Let `file_readable_JSON(P,D)` mean the file at `P` is successfully opened and its contents decode to `D`. Let `exc` be the caught exception instance.

```
( OSError exc. open(P) raises exc ) return = [ 'phases.json could not be read: ' + str(exc) ] ( file is opened json.load(P) raises
json.JSONDecodeError exc ) return = [ 'phases.json is not valid JSON: ' + str(exc) ] ( file_readable_JSON(P, D) ) ( isinstance(D, dict) return = [
'the top-level value must be an object' ] ) ( isinstance(D, dict) ( 'phases' not in D isinstance(D['phases'], list) ) return = [ 'top-level field "phases"
must be an array' ] ) ( isinstance(D, dict) isinstance(D['phases'], list) ) let phases = D['phases']; errors = []; i [0, len(phases)-1]: phase = phases[i] (
isinstance(phase, dict) errors.append('phases['+str(i)+''] must be an object') ) ( isinstance(phase, dict) ( isinstance(phase.get('modules'), list)
errors.append('phases['+str(i)+''].modules must be an array') ) ( isinstance(phase.get('modules'), list) j [0, len(phase['modules'])-1]: module =
phase['modules'][j] ( isinstance(module, dict) errors.append('phases['+str(i)+''].modules['+str(j)+''] must be an object') ) ( isinstance(module, dict)
let c = 'phases['+str(i)+''].modules['+str(j)+'']; context = c if not module.get('name') else c + " (" + module['name'] + ")"; ( 'source_files' not in
module errors.append(context + '.source_files is missing') ) ( 'source_files' in module ( isinstance(module['source_files'], list)
errors.append(context + '.source_files must be an array') ) ( isinstance(module['source_files'], list) k [0, len(module['source_files'])-1]: (
isinstance(module['source_files'][k], str) errors.append(context + '.source_files['+str(k)+''] must be a string') ) ) ) ) ); return = errors
```

- **Code evidence:** Lines 3-9: The try-except block only handles `OSError` and `json.JSONDecodeError`, but does not handle `UnicodeDecodeError` that may be raised during file reading.
- **Trigger condition:** The specification requires the function to always return a list of human-readable error strings, but the code allows `UnicodeDecodeError` to propagate, causing the function to crash and not return a list. This violates the specification's requirement that the function returns a list in all cases.
- **Validator trigger:** A file at `phases_path` containing non-UTF-8 bytes causes `UnicodeDecodeError` to propagate uncaught, violating the spec guarantee that the function always returns a list.
- **Probe stdout:** CONFIRMED — actual: `UnicodeDecodeError` propagated uncaught | expected: a list of error strings (per spec)

► SPEC sidecar (完整)

```
{
  "signature": "_phase_plan_schema_errors(phases_path) -> list[str]",
  "pre_condition": "phases_path is a string representing a filesystem path. The file it refers to may or may not exist, may or may not be readable, and may or may not contain valid JSON or conform to any particular schema.",
  "post_condition": "Returns a list of human-readable error strings. The returned list is empty (falsy) if and only if all of the following hold: (1) a regular file exists at phases_path and can be opened for reading, (2) the file content is syntactically valid JSON, (3) the top-level JSON value is an object, (4) that object contains a key named \"phases\" whose value is an array, (5) every element of the \"phases\" array is an object, (6) every phase object contains a key named \"modules\" whose value is an array, (7) every element of a \"modules\" array is an object, (8) every module object contains a key named \"source_files\" whose value is an array, and (9) every element of a \"source_files\" array is a string. The returned list is non-empty (truthy) otherwise, with at least one element describing the first encountered blocking failure or all accumulated non-blocking structural violations."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-py/_phase_plan_schema_errors.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a list of human-readable error strings. The returned list is empty (falsy) if and only if all of the following hold: (1) a regular file exists at phases_path and can be opened for reading, (2) the file content is syntactically valid JSON, (3) the top-level JSON value is an object, (4) that object contains a key named \"phases\" whose value is an array, (5) every element of the \"phases\" array is an object, (6) every phase object contains a key named \"modules\" whose value is an array, (7) every element of a \"modules\" array is an object, (8) every module object contains a key named \"source_files\" whose value is an array, and (9) every element of a \"source_files\" array is a string. The returned list is non-empty (truthy) otherwise, with at least one element describing the first encountered blocking failure or all accumulated non-blocking structural violations.",
    "actual_behavior": "The function returns a list of strings describing schema errors found in the file at phases_path. The behavior is deterministic and covers all exceptions internally, never propagating them.\n\nIn natural language:\n- If the file cannot be opened (OSError), the result is a single-element list with a message starting with 'phases.json could not be read: ' and the exception string.\n- If the file is opened but contains invalid JSON (JSONDecodeError), the result is a single-element list with a message starting with 'phases.json is not valid JSON: ' and the exception string.\n- Otherwise, the JSON is successfully decoded into a Python value D.\n  - If D is not a dictionary, the result is ['the top-level value must be an object'].
  - Else if D lacks a 'phases' key or its value is not a list, the result is ['top-level field \"phases\" must be an array'].
  - Else D['phases'] is a list. The function initializes an empty errors list and iterates over each phase by index i:
    - If the phase is not a dict, appends 'phases[i] must be an object' and continues to the next phase.
    - Else, checks the 'modules' key. If missing or not a list, appends 'phases[i].modules must be an array' and continues.
    - Else, for each module at index j:
      - If the module is not a dict, appends 'phases[i].modules[j] must be an object' and continues to the next module.
      - Else, determines the context string: module_path = 'phases[i].modules[j]'; if module.get('name') is truthy, context becomes 'phases[i].modules[j] ('name')', else context is just module_path.
      - If 'source_files' is missing from the module, appends '{context}.source_files is missing' and continues.
      - Else, let source_files = module['source_files']. If source_files is not a list, appends '{context}.source_files must be an array' and continues.
      - Else, for each source_file at index k, if it is not a string, appends '{context}.source_files[k] must be a string'.
  - Finally, the function returns the accumulated errors list (possibly empty).\n\nFormal logic (using D for decoded JSON, P for phases_path):\nLet file_readable_JSON(P,D) mean the file at P is successfully opened and its contents decode to D.\nLet exc be the caught exception instance.\n( OSError exc. open(P) raises exc ) \n return = [ 'phases.json could not be read: ' + str(exc) ]\n( file is opened json.load(P) raises json.JSONDecodeError exc ) \n return = [ 'phases.json is not valid JSON: ' + str(exc) ]\n( file_readable_JSON(P, D) ) \n ( isinstance(D, dict) ) \n return = [ 'the top-level value must be an object' ]\n( isinstance(D, dict) ( 'phases' not in D isinstance(D['phases'], list) ) ) \n return = [ 'top-level field \"phases\" must be an array' ]\n( isinstance(D, dict) isinstance(D['phases'], list) ) \n let phases = D['phases'];\n errors = [];\n i [0, len(phases)-1]:\n phase = phases[i]\n ( isinstance(phase, dict) errors.append('phases['+str(i)+'] must be an object') )\n ( isinstance(phase, dict) \n ( isinstance(phase.get('modules'), list) errors.append('phases['+str(i)+'].modules must be an array') )\n ( isinstance(phase.get('modules'), list) \n j [0, len(phase['modules'])-1]:\n module = phase['modules'][j]\n ( isinstance(module, dict) errors.append('phases['+str(i)+'].modules['+str(j)+'] must be an object') )\n ( isinstance(module, dict) \n let c = 'phases['+str(i)+'].modules['+str(j)+']';\n context = c if not module.get('name') else c + ' (' + module['name'] + ')';\n ( 'source_files' not in module errors.append(context + '.source_files is missing') )\n ( 'source_files' in module \n ( isinstance(module['source_files'], list) errors.append(context + '.source_files must be an array') )\n ( isinstance(module['source_files'], list) \n k [0, len(module['source_files'])-1]:\n ( isinstance(module['source_files'][k], str) errors.append(context + '.source_files['+str(k)+'] must be a string') )\n )\n )\n )\n )\n return = errors",
    "code_evidence": "Lines 3-9: The try-except block only handles OSError and json.JSONDecodeError, but does not handle UnicodeDecodeError that may be raised during file reading.",
    "trigger_condition": "The specification requires the function to always return a list of human-readable error strings, but the code allows UnicodeDecodeError to propagate, causing the function to crash and not return a list. This violates the specification's requirement that the function returns a list in all cases."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--pipeline_setup-py--_phase_plan_schema_errors",
  "source_file": "/home/fancy/Projects/Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-py/_phase_plan_schema_errors.py",
  "function_name": "_phase_plan_schema_errors",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--pipeline_setup-py--_phase_plan_schema_errors.py",
  "detail_file": "fm_agent/bug_validation/src--pipeline_setup-py--_phase_plan_schema_errors.md",
  "probe_stdout": "CONFIRMED - actual: UnicodeDecodeError propagated uncaught | expected: a list of error strings (per spec)",
  "trigger_summary": "A file at phases_path containing non-UTF-8 bytes causes UnicodeDecodeError to propagate uncaught, violating the spec guarantee that the function always returns a list."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:39:17.849677Z	4562
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:40:05.953995Z	6760

事件 1 — [llm_7573e292cd374b19ae177c906ebe44dc](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a list of strings describing schema errors found in the file at phases_path. The behavior is deterministic and covers all exceptions internally, never propagating them.\n\nIn natural language:\n- If the file cannot be opened (OSError), the result is a single-element list with a message starting with 'phases.json could not be read: ' and the exception string.\n- If the file is opened but contains invalid JSON (JSONDecodeError), the result is a single-element list with a message starting with 'phases.json is not valid JSON: ' and the exception string.\n- Otherwise, the JSON is successfully decoded into a Python value D.\n  - If D is not a dictionary, the result is ['the top-level value must be an object'].\n  - Else if D lacks a 'phases' key or its value is not a list, the result is ['top-level field \"phases\" must be an array'].\n  - Else D['phases'] is a list. The function initializes an empty errors list and iterates over each phase by index i:\n    - If the phase is not a dict, appends 'phases[i] must be an object' and continues to the next phase.\n    - Else, checks the 'modules' key. If missing or not a list, appends 'phases[i].modules must be an array' and continues.\n    - Else, for each module at index j:\n      - If the module is not a dict, appends 'phases[i].modules[j] must be an object' and continues to the next module.\n      - Else, determines the context string: module_path = 'phases[i].modules[j]'; if module.get('name') is truthy, context becomes \"phases[i].modules[j] ('name')\", else context is just module_path.\n      - If 'source_files' is missing from the module, appends '{context}.source_files is missing' and continues.\n      - Else, let source_files = module['source_files']. If source_files is not a list, appends '{context}.source_files must be an array' and continues.\n      - Else, for each source_file at index k, if it is not a string, appends '{context}.source_files[k] must be a string'.\n  - Finally, the function returns the accumulated errors list (possibly empty).\n\nFormal logic (using D for decoded JSON, P for phases_path):\nLet file_readable_JSON(P,D) mean the file at P is successfully opened and its contents decode to D.\nLet exc be the caught exception instance.\n( ∃ OSError exc. open(P) raises exc ) ⇒ return = [ 'phases.json could not be read: ' + str(exc) ]\n( file is opened ∧ json.load(P) raises json.JSONDecodeError exc ) ⇒ return = [ 'phases.json is not valid JSON: ' + str(exc) ]\n( file_readable_JSON(P, D) ) ⇒ ( ¬ isinstance(D, dict) ⇒ return = [ 'the top-level value must be an object' ] )\n  ∧ ( isinstance(D, dict) ∧ ( 'phases' not in D ∨ ¬ isinstance(D['phases'], list) ) ⇒ return = [ 'top-level field \"phases\" must be an array' ] )\n  ∧ ( isinstance(D, dict) ∧ isinstance(D['phases'], list) ) ⇒ let phases = D['phases']; errors = [];\n    ∀ i ∈ [0, len(phases)-1]:\n      phase = phases[i]\n      ( ¬ isinstance(phase, dict) ⇒ errors.append('phases['+str(i)+'] must be an object') )\n      ∧ ( isinstance(phase, dict) ⇒ ( ¬ isinstance(phase.get('modules'), list) ⇒ errors.append('phases['+str(i)+'].modules must be an array') )\n        ∧ ( isinstance(phase.get('modules'), list) ⇒\n          ∀ j ∈ [0, len(phase['modules'])-1]:\n            module = phase['modules'][j]\n            ( ¬ isinstance(module, dict) ⇒ errors.append('phases['+str(i)+'].modules['+str(j)+'] must be an object') )\n            ∧ ( isinstance(module, dict) ⇒\n              let c = 'phases['+str(i)+'].modules['+str(j)+'];\n                context = c if not module.get('name') else c + \" ('\" + module['name'] + \"')\";\n                ( 'source_files' not in module ⇒ errors.append(context + '.source_files is missing') )\n                ∧ ( 'source_files' in module ⇒\n                  ( ¬ isinstance(module['source_files'], list) ⇒ errors.append(context + '.source_files must be an array') )\n                  ∧ ( isinstance(module['source_files'], list) ⇒\n                    ∀ k ∈ [0, len(module['source_files'])-1]:\n                      ( ¬ isinstance(module['source_files'][k], str) ⇒ errors.append(context + '.source_files['+str(k)+'] must be a string') )\n                    )\n                  )\n                )\n              return = errors\n            )\n          )\n        )\n      )\n    )\n  )\n}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7573e292cd374b19ae177c906ebe44dc_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_7573e292cd374b19ae177c906ebe44dc_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_7573e292cd374b19ae177c906ebe44dc_response.txt)

事件 2 — [llm_371b84bf75a54385a9638aa477e421ab](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "A file at '/tmp/invalid_utf8.json' containing the single byte 0xFF (an invalid UTF-8 sequence). The file exists and is readable, so open() succeeds. However, when json.load() reads the file, it raises a UnicodeDecodeError because the content cannot be decoded as UTF-8. The function does not catch this exception and crashes instead of returning a list of error strings.",
  "offending_statements": "Lines 3-9: The try-except block only handles OSError and json.JSONDecodeError, but does not handle UnicodeDecodeError that may be raised during file reading.",
  "reason": "The specification requires the function to always return a list of human-readable error strings, but the code allows UnicodeDecodeError to propagate, causing the function to crash and not return a list. This violates the specification's requirement that the function returns a list in all cases."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_371b84bf75a54385a9638aa477e421ab_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_371b84bf75a54385a9638aa477e421ab_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_371b84bf75a54385a9638aa477e421ab_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: plugins

FMA-MISMATCH-143 — [src--plugin-py--_resolve_command](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 3 — Pipeline Planning & Domain Knowledge / [plugins](#); 原源码 [src/plugin.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a string formed by processing each whitespace-delimited token of cmd independently. For each token that starts with `"/` or `"$"`: the token is preserved unchanged. For each remaining token: if `plugin_root / token` resolves to an existing regular file on the filesystem, the token is replaced by the string representation of its absolute path; otherwise, the token is preserved unchanged. The relative order of tokens in the returned string matches the order in cmd.
- **Actual behavior:** The function returns a string `r` such that, letting tokens be the list of non-empty strings obtained by splitting the input cmd on whitespace, for each index `i`, token `t = tokens[i]`, if `t` starts with `'/'` or `'$'`
'then r contains t at the corresponding position; otherwise, if the path plugin_root / t exists and is a file, r contains the absolute path to that file. Let tokens = cmd.split(); let resolved be a list of strings with length |tokens| such that for all i from 0 to |tokens| - 1, (tokens[i].startswith('/') tokens[i].startswith('\$') resolved[i] = tokens[i]) ((tokens[i].startswith('/') tokens[i].startswith('\$')) (plugin_root / tokens[i]).is_file() resolved[i] = str(plugin_root / tokens[i])) ((tokens[i].startswith('/') tokens[i].startswith('\$')) (plugin_root / tokens[i]).is_file() resolved[i] = tokens[i]); then r = ''.join(resolved).
- **Code evidence:** Line 10: if token.startswith("/") or token.startswith("\$"):
- **Trigger condition:** The specification (docstring) explicitly states that tokens starting with `'/'`, `'$'`, or `'-'` must be left unchanged. The code only checks for `'/'` and `'$'`, omitting the `'-'` case. Consequently, any token starting with `'-'` that happens to exist as a file under `plugin_root` will be replaced by its absolute path, directly violating the required behavior.
- **Validator trigger:** Token starting with `'-'` that exists as a file under `plugin_root` is replaced by its absolute path instead of being left unchanged
- **Probe stdout:** CONFIRMED - actual: `'/tmp/tmpg7ihe4nq/-testfile other_token'` | expected: `'-testfile other_token'`

► SPEC sidecar (完整)

```
{
  "signature": "_resolve_command(cmd: str, plugin_root: Path) -> str",
  "pre_condition": "cmd is a non-empty string containing a shell command; plugin_root is a pathlib.Path pointing to an existing directory.",
  "post_condition": "Returns a string formed by processing each whitespace-delimited token of cmd independently. For each token that starts with \"\\\"/\" or \"\\\"$\\\": the token is preserved unchanged. For each remaining token: if plugin_root / token resolves to an existing regular file on the filesystem, the token is replaced by the string representation of its absolute path; otherwise, the token is preserved unchanged. The relative order of tokens in the returned string matches the order in cmd."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/plugin-py/_resolve_command.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a string formed by processing each whitespace-delimited token of cmd independently. For each token that starts with \"\\\"/\" or \"\\\"$\": the token is preserved unchanged. For each remaining token: if plugin_root / token resolves to an existing regular file on the filesystem, the token is replaced by the string representation of its absolute path; otherwise, the token is preserved unchanged. The relative order of tokens in the returned string matches the order in cmd.",
    "actual_behavior": "The function returns a string r such that, letting tokens be the list of non-empty strings obtained by splitting the input cmd on whitespace, for each index i, token t = tokens[i], if t starts with '/' or '$' then r contains t at the corresponding position; otherwise, if the path plugin_root / t exists and is a file, r contains the absolute path string of that file; otherwise r contains t unchanged. The tokens are joined by a single space. Formally: Let tokens = cmd.split(); let resolved be a list of strings with length |tokens| such that for all i from 0 to |tokens|-1, (tokens[i].startswith('/') tokens[i].startswith('$') resolved[i] = tokens[i]) ((tokens[i].startswith('/') tokens[i].startswith('$')) (plugin_root / tokens[i]).is_file() resolved[i] = str(plugin_root / tokens[i])) ((tokens[i].startswith('/') tokens[i].startswith('$')) (plugin_root / tokens[i]).is_file() resolved[i] = tokens[i]); then r = ' '.join(resolved).",
    "code_evidence": "Line 10: if token.startswith(\"\\\"/\" or token.startswith(\"\\\"$\":",
    "trigger_condition": "The specification (docstring) explicitly states that tokens starting with '/', '$', or '-' must be left unchanged. The code only checks for '/' and '$', omitting the '-' case. Consequently, any token starting with '-' that happens to exist as a file under plugin_root will be replaced by its absolute path, directly violating the required behavior."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--plugin-py--_resolve_command",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/plugin-py/_resolve_command.py",
  "function_name": "_resolve_command",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--plugin-py--_resolve_command.py",
  "detail_file": "fm_agent/bug_validation/src--plugin-py--_resolve_command.md",
  "probe_stdout": "CONFIRMED - actual: '/tmp/tmpg7ihe4nq/-testfile other_token' | expected: '-testfile other_token'",
  "trigger_summary": "Token starting with '-' that exists as a file under plugin_root is replaced by its absolute path instead of being left unchanged"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:39:25.862075Z	1670
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:39:43.814631Z	2437

事件 1 — [llm_0427559cf3f34c01b75ea875b00a7a27](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string r such that, letting tokens be the list of non-empty strings obtained by splitting the input cmd on whitespace, for each index i, token t = tokens[i], if t starts with '/' or '$' then r contains t at the corresponding position; otherwise, if the path plugin_root / t exists and is a file, r contains the absolute path string of that file; otherwise r contains t unchanged. The tokens are joined by a single space. Formally: Let tokens = cmd.split(); let resolved be a list of strings with length |tokens| such that for all i from 0 to |tokens|-1, (tokens[i].startswith('/') v tokens[i].startswith('$') => resolved[i] = tokens[i]) ^ (~(tokens[i].startswith('/') v tokens[i].startswith('$')) ^ (plugin_root / tokens[i]).is_file() => resolved[i] = str(plugin_root / tokens[i])) ^ (~(tokens[i].startswith('/') v tokens[i].startswith('$')) ^ ~(plugin_root / tokens[i]).is_file() => resolved[i] = tokens[i]); then r = ' '.join(resolved)."
```

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Assume plugin_root contains a file named '-f'. Input cmd = '-f'. The specification requires leaving '-f' unchanged because it begins with '-', but the code will resolve it to the absolute path '/path/to/plugin_root/-f' and output that path instead of preserving the token.",
  "offending_statements": "Line 10: if token.startswith(\"/\") or token.startswith(\"$\"): ",
  "reason": "The specification (docstring) explicitly states that tokens starting with '/', '$', or '-' must be left unchanged. The code only checks for '/' and '$', omitting the '-' case. Consequently, any token starting with '-' that happens to exist as a file under plugin_root will be replaced by its absolute path, directly violating the required behavior."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1d7fe5d5fcd440aeb008fcbda0615ef6_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_1d7fe5d5fcd440aeb008fcbda0615ef6_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_1d7fe5d5fcd440aeb008fcbda0615ef6_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-144 — src--plugin-py--_validate_plugin_json_content

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： **confirmed**; attempts 1
- Phase / 模块： Phase 3 — Pipeline Planning & Domain Knowledge / **plugins**; 原源码 **src/plugin.py**
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim**: Returns None if any of the following validation conditions fail, after printing a diagnostic message to stdout identifying the plugin name and the specific failure condition: (1) the value of the key 'name' in data does not equal name, (2) the key 'version' in data is missing or its value is empty/falsy, (3) the key 'stages' in data is not a dict, (4) any value in stages is not a dict, (5) for any stage name in stages, a PluginStageConfig constructed from the stage's data produces a non-empty error list when validated, (6) any stage whose type field equals 'modify' references an input_md path that does not exist as a regular file within plugin_dir. Returns a PluginConfig when none of the failure conditions apply, where: the name attribute equals name (non-empty), the version attribute equals data['version'], the root attribute equals plugin_dir, and the stages attribute is a dict mapping each validated stage name (string) to its corresponding PluginStageConfig.
- Actual behavior**: The function returns either None or a PluginConfig object. If it returns a PluginConfig c, then all of the following hold: c.name equals the input argument name (c.name == name); c.version equals data.get('version') (and data.get('version') is truthy); c.root equals plugin_dir; and c.stages is a dictionary where for each key k in c.stages, data.get('stages', {}).get(k) is a dict, the PluginStageConfig object for k passes validation (stage.validated() returns an empty list), and if stage.type == 'modify' and stage.input_md is truthy, then (plugin_dir / stage.input_md).is_file() is True. If it returns None, then at least one of the following is true: 1) data.get('name', '') != name; 2) not data.get('version'); 3) not isinstance(data.get('stages', {}), dict); 4) there exists a key s in data.get('stages', {}) such that not isinstance(data.get('stages', {})[s], dict); 5) there exists a key s and corresponding PluginStageConfig object constructed from its data such that stage.validated() is non-empty; 6) there exists a key s with stage.type == 'modify' and stage.input_md truthy for which (plugin_dir / stage.input_md).is_file() is False. The inputs plugin_dir, name, and data are not modified.
- Code evidence**: Line 35: if stage.type == "modify" and stage.input_md:
- Trigger condition**: The code only checks the existence of input_md when stage.input_md is truthy. The specification requires rejecting any modify stage whose input_md does not exist as a regular file, regardless of whether input_md is truthy. An empty input_md produces a path that is not a regular file, so the function should return None, but the code returns a PluginConfig.
- Validator trigger**: Empty input_md on a modify stage skips the file-existence guard (stage.input_md is falsy), but plugin_dir / " is a directory, not a regular file, violating spec condition (6).
- Probe stdout**: CONFIRMED — actual: PluginConfig(name='probe_plugin_71kf98tp', version='1.0.0', root=PosixPath('/tmp/probe_plugin_71kf98tp'), stages={'mystage': PluginStageConfig(type='modify', replace_cmd=None, input_md="", output_process='echo hello')}) | expected: None Bug: empty input_md resolved to /tmp/probe_plugin_71kf98tp (a directory), which is not a regular file, but the function returned a PluginConfig.

► SPEC sidecar (完整)

```
{
  "signature": "_validate_plugin_json_content(plugin_dir: Path, name: str, data: dict) -> Optional[PluginConfig]",
```



```
"pre_condition": "plugin_dir is a pathlib.Path pointing to a directory containing plugin.json; name is the directory name of plugin_dir (non-empty string); data is a dict parsed from plugin.json.",
"post_condition": "Returns None if any of the following validation conditions fail, after printing a diagnostic message to stdout identifying the plugin name and the specific failure condition: (1) the value of the key 'name' in data does not equal name, (2) the key 'version' in data is missing or its value is empty/falsy, (3) the key 'stages' in data is not a dict, (4) any value in stages is not a dict, (5) for any stage name in stages, a PluginStageConfig constructed from the stage's data produces a non-empty error list when validated, (6) any stage whose type field equals 'modify' references an input_md path that does not exist as a regular file within plugin_dir. Returns a PluginConfig when none of the failure conditions apply, where: the name attribute equals name (non-empty), the version attribute equals data['version'], the root attribute equals plugin_dir, and the stages attribute is a dict mapping each validated stage name (string) to its corresponding PluginStageConfig."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/plugin-py/_validate_plugin_json_content.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns None if any of the following validation conditions fail, after printing a diagnostic message to stdout identifying the plugin name and the specific failure condition: (1) the value of the key 'name' in data does not equal name, (2) the key 'version' in data is missing or its value is empty/falsy, (3) the key 'stages' in data is not a dict, (4) any value in stages is not a dict, (5) for any stage name in stages, a PluginStageConfig constructed from the stage's data produces a non-empty error list when validated, (6) any stage whose type field equals 'modify' references an input_md path that does not exist as a regular file within plugin_dir. Returns a PluginConfig when none of the failure conditions apply, where: the name attribute equals name (non-empty), the version attribute equals data['version'], the root attribute equals plugin_dir, and the stages attribute is a dict mapping each validated stage name (string) to its corresponding PluginStageConfig.",
    "actual_behavior": "The function returns either None or a PluginConfig object. If it returns a PluginConfig c, then all of the following hold: c.name equals the input argument name (c.name == name); c.version equals data.get('version') (and data.get('version') is truthy); c.root equals plugin_dir; and c.stages is a dictionary where for each key k in c.stages, data.get('stages', {}).get(k) is a dict, the PluginStageConfig object for k passes validation (stage.validated() returns an empty list), and if stage.type == 'modify' and stage.input_md is truthy, then (plugin_dir / stage.input_md).is_file() is True. If it returns None, then at least one of the following is true: 1) data.get('name', '') != name; 2) not data.get('version'); 3) not isinstance(data.get('stages', {}), dict); 4) there exists a key s in data.get('stages', {}) such that not isinstance(data.get('stages', {})[s], dict); 5) there exists a key s and corresponding PluginStageConfig object constructed from its data such that stage.validated() is non-empty; 6) there exists a key s with stage.type == 'modify' and stage.input_md truthy for which (plugin_dir / stage.input_md).is_file() is False. The inputs plugin_dir, name, and data are not modified.",
    "code_evidence": "Line 35: if stage.type == \"modify\" and stage.input_md:",
    "trigger_condition": "The code only checks the existence of input_md when stage.input_md is truthy. The specification requires rejecting any modify stage whose input_md does not exist as a regular file, regardless of whether input_md is truthy. An empty input_md produces a path that is not a regular file, so the function should return None, but the code returns a PluginConfig."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--plugin-py--_validate_plugin_json_content",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/plugin-py/_validate_plugin_json_content.py",
  "function_name": "_validate_plugin_json_content",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--plugin-py--_validate_plugin_json_content.py",
  "detail_file": "fm_agent/bug_validation/src--plugin-py--_validate_plugin_json_content.md",
  "probe_stdout": "CONFIRMED - actual: PluginConfig(name='probe_plugin_71kf98tp', version='1.0.0', root=PosixPath('/tmp/probe_plugin_71kf98tp'), stages={'mystage': PluginStageConfig(type='modify', replace_cmd=None, input_md='', output_process='echo hello')}) | expected: None\n Bug: empty input_md resolved to /tmp/probe_plugin_71kf98tp (a directory), which is not a regular file, but the function returned a PluginConfig.",
  "trigger_summary": "Empty input_md on a modify stage skips the file-existence guard (stage.input_md is falsy), but plugin_dir / '' is a directory, not a regular file, violating spec condition (6)."
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:32:28.130778Z	2268

#	purpose	status	attempt	block	start	tokens
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:32:47.073710Z	4160

事件 1 — llm_d105e6d44d0e4cdeb2852cc1db639774

► parsed_json（完整）

```
{
  "post_condition": "The function returns either None or a PluginConfig object. If it returns a PluginConfig c, then all of the following hold: c.name equals the input argument name (c.name == name); c.version equals data.get('version') (and data.get('version') is truthy); c.root equals plugin_dir; and c.stages is a dictionary where for each key k in c.stages, data.get('stages', {}).get(k) is a dict, the PluginStageConfig object for k passes validation (stage.validated() returns an empty list), and if stage.type == 'modify' and stage.input_md is truthy, then (plugin_dir / stage.input_md).is_file() is True. If it returns None, then at least one of the following is true: 1) data.get('name', '') != name; 2) not data.get('version'); 3) not isinstance(data.get('stages', {}), dict); 4) there exists a key s in data.get('stages', {}) such that not isinstance(data.get('stages', {})[s], dict); 5) there exists a key s and corresponding PluginStageConfig object constructed from its data such that stage.validated() is non-empty; 6) there exists a key s with stage.type == 'modify' and stage.input_md truthy for which (plugin_dir / stage.input_md).is_file() is False. The inputs plugin_dir, name, and data are not modified."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d105e6d44d0e4cdeb2852cc1db639774_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d105e6d44d0e4cdeb2852cc1db639774_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d105e6d44d0e4cdeb2852cc1db639774_response.txt)

事件 2 — llm_673231522a454fd0beb1203fe4903abe

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "name = 'test', plugin_dir = Path('/tmp/plugin_dir') (exists), data = {'name': 'test', 'version': '1.0', 'stages': {'s1': {'type': 'modify', 'input_md': ''}}}, assuming PluginStageConfig.from_dict and validated() accept this data.",
  "offending_statements": "Line 35: if stage.type == \"modify\" and stage.input_md:",
  "reason": "The code only checks the existence of input_md when stage.input_md is truthy. The specification requires rejecting any modify stage whose input_md does not exist as a regular file, regardless of whether input_md is truthy. An empty input_md produces a path that is not a regular file, so the function should return None, but the code returns a PluginConfig."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_673231522a454fd0beb1203fe4903abe_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_673231522a454fd0beb1203fe4903abe_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_673231522a454fd0beb1203fe4903abe_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

Phase 4 — LLM Communication & Tracing

模块: llm_client

FMA-MISMATCH-157 — src--llm_client-py--_matches_inject_target

- 四类结论： 契约不确定
- 分类依据： 沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 4 — LLM Communication & Tracing / llm_client; 原源码 src/llm_client.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim:** Returns True when url is determined to belong to the address space of target; returns False otherwise. When target begins with 'http://' or 'https://' (case-insensitive), membership requires url to have target as a case-sensitive prefix. When target lacks a scheme, membership requires the hostname component extracted from url to exactly equal target, or to end with '.' followed by target (indicating a subdomain). Returns False when the hostname cannot be extracted from url.
- Actual behavior:** The function returns a boolean value. If the lowercase version of target starts with 'http://' or 'https://', the function returns True exactly when url.startswith(target) is true, False otherwise. Otherwise, the function attempts to parse the hostname from url using urllib.parse.urlparse. If an exception occurs during parsing, it returns False. If parsing succeeds, it extracts host

- `= urlparse(url).hostname or ''` (empty string if the hostname is `None`). It then returns `True` if `host == target` or `host.endswith('.' + target)`, and `False` otherwise. No other state is modified. Formally, let `low = target.lower()`; `scheme = low.startswith('http://')` or `low.startswith('https://')`; then the return value `r` satisfies: `r = ( scheme url.startswith(target) ) ( scheme [if no exception and host = (urlparse(url).hostname or '') then (host = target host.endsWith('.' + target)) else False] )`.
- **Code evidence:** Line 5: `host = urllib.parse.urlparse(url).hostname or ''` Line 8: `return host == target or host.endswith('.' + target)`
 - **Trigger condition:** Hostnames are case-insensitive, but the code performs a case-sensitive equality check. When `target` is "example.com" (no scheme) and `url` is "http://EXAMPLE.COM", `urllib.parse.urlparse(url).hostname` returns "EXAMPLE.COM". The code then compares "EXAMPLE.COM" == "example.com" (False) and "EXAMPLE.COM".endswith(".example.com") (False), returning False. According to the specification, the hostname "EXAMPLE.COM" belongs to the address space of "example.com", so the function should return True.
 - **Validator trigger:** Case-sensitive `host == target` comparison at line 8 fails when `target` contains uppercase ASCII letters (e.g., 'EXAMPLE.COM' vs hostname 'example.com' from `urlparse`), violating DNS case-insensitivity.
 - **Probe stdout:** CONFIRMED — `url='http://example.com/some/path'`, `target='EXAMPLE.COM'`, `actual=False`, `expected=True`

► SPEC sidecar (完整)

```
{
  "signature": "_matches_inject_target(url, target) -> bool",
  "pre_condition": "url is a non-None string (falsy values treated as empty) whose trailing slashes have been stripped. target is a non-empty string representing a host pattern to match against.",
  "post_condition": "Returns True when url is determined to belong to the address space of target; returns False otherwise. When target begins with 'http://' or 'https://' (case-insensitive), membership requires url to have target as a case-sensitive prefix. When target lacks a scheme, membership requires the hostname component extracted from url to exactly equal target, or to end with '.' followed by target (indicating a subdomain). Returns False when the hostname cannot be extracted from url."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/llm_client-py/_matches_inject_target.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns True when url is determined to belong to the address space of target; returns False otherwise. When target begins with 'http://' or 'https://' (case-insensitive), membership requires url to have target as a case-sensitive prefix. When target lacks a scheme, membership requires the hostname component extracted from url to exactly equal target, or to end with '.' followed by target (indicating a subdomain). Returns False when the hostname cannot be extracted from url.",
    "actual_behavior": "The function returns a boolean value. If the lowercase version of `target` starts with 'http://' or 'https://', the function returns `True` exactly when `url.startswith(target)` is true, `False` otherwise. Otherwise, the function attempts to parse the hostname from `url` using `urllib.parse.urlparse`. If an exception occurs during parsing, it returns `False`. If parsing succeeds, it extracts `host = urlparse(url).hostname or ''` (empty string if the hostname is `None`). It then returns `True` if `host == target` or `host.endswith('.' + target)`, and `False` otherwise. No other state is modified. Formally, let low = target.lower(); scheme = low.startswith('http://') or low.startswith('https://'); then the return value r satisfies: r = ( scheme url.startswith(target) ) ( scheme [if no exception and host = (urlparse(url).hostname or '') then (host = target host.endsWith('.' + target)) else False] ).",
    "code_evidence": "Line 5: host = urllib.parse.urlparse(url).hostname or ''\nLine 8: return host == target or host.endswith('.' + target)",
    "trigger_condition": "Hostnames are case-insensitive, but the code performs a case-sensitive equality check. When target is \"example.com\" (no scheme) and url is \"http://EXAMPLE.COM\", urllib.parse.urlparse(url).hostname returns \"EXAMPLE.COM\". The code then compares \"EXAMPLE.COM\" == \"example.com\" (False) and \"EXAMPLE.COM\".endswith(\".example.com\") (False), returning False. According to the specification, the hostname \"EXAMPLE.COM\" belongs to the address space of \"example.com\", so the function should return True."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--llm_client-py--_matches_inject_target",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/llm_client-py/_matches_inject_target.py",
  "function_name": "_matches_inject_target",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--llm_client-py--_matches_inject_target.py",
  "detail_file": "fm_agent/bug_validation/src--llm_client-py--_matches_inject_target.md",
}
```

```
"probe_stdout": "CONFIRMED -url='http://example.com/some/path', target='EXAMPLE.COM', actual=False, expected=True\n",
"trigger_summary": "Case-sensitive host == target comparison at line 8 fails when target contains uppercase ASCII letters (e.g., 'EXAMPLE.COM' vs hostname 'example.com' from urlparse), violating DNS case-insensitivity."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:19:31.041222Z	2408
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:20:10.164167Z	8048

事件 1 — [llm_29d4f31220fb4fd6a6d9681c82c2d8da](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a boolean value. If the lowercase version of `target` starts with 'http://' or 'https://', the function returns `True` exactly when `url.startswith(target)` is true, `False` otherwise. Otherwise, the function attempts to parse the hostname from `url` using `urllib.parse.urlparse`. If an exception occurs during parsing, it returns `False`. If parsing succeeds, it extracts `host = urlparse(url).hostname or ''` (empty string if the hostname is `None`). It then returns `True` if `host == target` or `host.endswith('.') + target`, and `False` otherwise. No other state is modified. Formally, let low = target.lower(); scheme = low.startswith('http://') or low.startswith('https://'); then the return value r satisfies: r = ( scheme ^ url.startswith(target) ) v ( ~scheme ^ [if no exception and host = (urlparse(url).hostname or '') then (host = target v host.endswith('.') + target)) else False] )."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_29d4f31220fb4fd6a6d9681c82c2d8da_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_29d4f31220fb4fd6a6d9681c82c2d8da_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_29d4f31220fb4fd6a6d9681c82c2d8da_response.txt)

事件 2 — [llm_ec6e7b3ef0014c1a87a2226b1a80e17e](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "url = \"http://EXAMPLE.COM\", target = \"example.com\"",
  "offending_statements": "Line 5: host = urllib.parse.urlparse(url).hostname or \"\"\nLine 8: return host == target or host.endswith(\".\" + target)",
  "reason": "Hostnames are case-insensitive, but the code performs a case-sensitive equality check. When target is \"example.com\" (no scheme) and url is \"http://EXAMPLE.COM\", urllib.parse.urlparse(url).hostname returns \"EXAMPLE.COM\". The code then compares \"EXAMPLE.COM\" == \"example.com\" (False) and \"EXAMPLE.COM\".endswith(\".example.com\") (False), returning False. According to the specification, the hostname \"EXAMPLE.COM\" belongs to the address space of \"example.com\", so the function should return True."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ec6e7b3ef0014c1a87a2226b1a80e17e_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ec6e7b3ef0014c1a87a2226b1a80e17e_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ec6e7b3ef0014c1a87a2226b1a80e17e_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-161 — [src--llm_client-py--_retry_create](#)

- 四类结论: 契约不确定
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 4 — LLM Communication & Tracing / llm_client; 原源码 src/llm_client.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim: Returns a 2-tuple (text, usage) where text is a non-empty string containing the LLM assistant response content and usage is a dict representing the token-usage metadata from the API response (an empty dict when no usage data is present in the response). On failure, raises an exception with the following error contract: (a) non-recoverable errors HTTP 400 responses and their SDK-equivalent

- exception types are raised immediately without any retry; (b) rate-limiting errors HTTP 429 responses and their SDK-equivalent exception types are retried with exponential backoff plus random jitter, bounded above and subject to a maximum retry count; upon exhausting the rate-limit retry budget, a `RuntimeError` is raised; (c) transient server errors HTTP 5xx responses and other unexpected exception types are retried with exponential backoff plus random jitter, subject to a distinct maximum retry count; upon exhausting the transient-error retry budget, a `RuntimeError` is raised. The backoff wait duration for each error category grows exponentially with each successive attempt and is clamped at a category-specific upper bound. When a CLI-based LLM backend is active, the request is delegated to the CLI agent subprocess and its return value is returned directly without further processing. When the model is identified as belonging to the Anthropic model family, the request is routed to the native Anthropic API endpoint rather than the OpenAI-compatible endpoint.
- **Actual behavior:** If `is_cli_backend_enabled()` evaluates to `True`, the function delegates to `run_agent_for_messages(model, messages)` and either returns its result a 2tuple `(text, usage_dict)` where `text` is a string and `usage_dict` is a dict (possibly empty) or propagates any exception raised by that call. Otherwise, the function enters a retry loop. In each iteration:
 - If the model is detected as an Anthropic model (`_is_anthropic_model(model)` is `True`), it calls `_anthropic_create(model, messages)`. For nonAnthropic models, it calls `client.chat.completions.create(model=model, messages=messages, **extra)` (with `extra` possibly containing an `extra_body` if URL injection is enabled).
 - On a successful call, the response content from the first choice (`response.choices[0].message.content`) becomes `text`; if `response.usage` is not `None`, `usage_dict` is obtained via `response.usage.model_dump()`, otherwise `usage_dict` is an empty dict. The tuple `(text, usage_dict)` is returned.
 - If `BadRequestError` is raised (by the API or client), it is immediately reraised without retry.
 - If `urllib.error.HTTPError` is raised:
 - If `exc.code == 400`, reraise the exception.
 - If `exc.code == 429`, increment a ratelimit attempt counter. If the counter reaches `_MAX_RATE_LIMIT_RETRIES`, raise `RuntimeError` with a descriptive message and the original exception as cause. Otherwise wait for `min(2 ** (attempts-1) * 5, 300) + random.uniform(1, 10)` seconds and continue.
 - For all other status codes (e.g., 5xx), increment a transient failure attempt counter. If that counter reaches `_MAX_LLM_RETRIES`, raise `RuntimeError` with a descriptive message and the original exception as cause. Otherwise wait for `min(2 ** (attempts-1) * 5, 60) + random.uniform(1, 3)` seconds and continue.
 - If `RateLimitError` is raised, apply the same logic as for HTTP 429 (increment the ratelimit counter, possibly raise `RuntimeError` after retries exhausted, else wait with the same backoff as 429).
 - Any other `Exception` is treated as a transient failure: increment the transient counter; if the counter reaches `_MAX_LLM_RETRIES`, raise `RuntimeError` with a descriptive message and the original exception as cause; otherwise wait with the same backoff as transient HTTP errors and continue.
 - Exceptions not derived from `Exception` (e.g., `KeyboardInterrupt`, `SystemExit`) are not caught and propagate out immediately.

Formally, let `Raise(e)` denote that the function raises exception `e`. Let `Return(t)` denote a normal return with value `t`. The postcondition is: `(is_cli_backend_enabled() (Return(run_agent_for_messages(model, messages)) Raise(e) where e was raised by run_agent_for_messages))`

`(is_cli_backend_enabled() ( (Return( (text, usage) ) text String usage Dict) (Raise(BadRequestError) Raise(RuntimeError)) )`, where the eventual outcome is governed by the retry logic above, and the function either returns a successful tuple or raises one of the specified exceptions after exhausting retries or encountering a permanent error.

- **Code evidence:** Line 21: `text = response.choices[0].message.content` Line 23: `return text, usage`
- **Trigger condition:** The specification requires the returned text to be a non-empty string. The code does not verify that the content is non-empty; it returns whatever the API provides. When the API responds with an empty string (e.g., `content=""`), the code returns an empty text, which violates the post-condition.
- **Validator trigger:** When the LLM API returns an empty response content, `_retry_create` returns an empty string instead of ensuring the returned text is non-empty as required by the specification.
- **Probe stdout:** CONFIRMED — actual: `" |` expected: `'non-empty string (spec requirement)'`

► SPEC sidecar (完整)

```
{
  "signature": "_retry_create(client, model, messages) -> tuple",
  "pre_condition": "client is a callable or object providing an OpenAI-compatible chat.completions.create method that accepts model and messages keyword arguments. model is a non-empty string naming the LLM model. messages is a non-empty list where each element is a dict with keys 'role' (non-empty string) and 'content' (string).",
  "post_condition": "Returns a 2-tuple (text, usage) where text is a non-empty string containing the LLM assistant response content and usage is a dict representing the token-usage metadata from the API response (an empty dict when no usage data is present in the response). On failure, raises an exception with the following error contract: (a) non-recoverable errors – HTTP 400 responses and their SDK-equivalent exception types – are raised immediately without any retry; (b) rate-limiting errors – HTTP 429 responses and their SDK-equivalent exception types – are retried with exponential backoff plus random jitter, bounded above and subject to a maximum retry count; upon exhausting the rate-limit retry budget, a RuntimeError is raised; (c) transient server errors – HTTP 5xx responses – and other unexpected exception types are retried with exponential backoff plus random jitter, subject to a distinct maximum retry count; upon exhausting the transient-error retry budget, a RuntimeError is raised. The backoff wait duration for each error category grows exponentially with each successive attempt and is clamped at a category-specific upper bound."
}
```



```

bound. When a CLI-based LLM backend is active, the request is delegated to the CLI agent subprocess and its return value is
returned directly without further processing. When the model is identified as belonging to the Anthropic model family, the request
is routed to the native Anthropic API endpoint rather than the OpenAI-compatible endpoint."
}

```

► INFO / callees sidecar (完整)

```

{
  "callees": [
    {
      "name": "is_cli_backend_enabled",
      "signature": "is_cli_backend_enabled() -> bool",
      "pre_condition": "None.",
      "post_condition": "Returns True when the configured LLM backend is a CLI-based backend (codex-cli or claude-cli); returns
False otherwise."
    },
    {
      "name": "run_agent_for_messages",
      "signature": "run_agent_for_messages(model, messages) -> tuple",
      "pre_condition": "model is a non-empty string naming the LLM model. messages is a non-empty list of message dicts with
'role' and 'content' keys.",
      "post_condition": "Returns a 2-tuple (text_response, usage_dict) where text_response is the string LLM output and usage_dict
is a dict with token usage information. Raises an exception on persistent failure."
    },
    {
      "name": "_is_anthropic_model",
      "signature": "_is_anthropic_model(model) -> bool",
      "pre_condition": "model is a non-empty string.",
      "post_condition": "Returns True when model identifies an Anthropic-family model (by name prefix); returns False otherwise."
    },
    {
      "name": "_should_inject_user_id",
      "signature": "_should_inject_user_id(base_url) -> bool",
      "pre_condition": "base_url is a string (may be empty) representing the LLM API base URL.",
      "post_condition": "Returns True when the base_url matches a configured host pattern that supports user-ID-based prompt-cache
routing affinity; returns False otherwise."
    },
    {
      "name": "_metadata_body",
      "signature": "_metadata_body() -> dict",
      "pre_condition": "None.",
      "post_condition": "Returns a dict containing metadata fields (including user_id) suitable for injection into the API request
body."
    },
    {
      "name": "_anthropic_create",
      "signature": "_anthropic_create(model, messages) -> tuple",
      "pre_condition": "model is a non-empty string identifying the Anthropic model. messages is a non-empty list of message dicts
with 'role' and 'content' keys.",
      "post_condition": "Returns a 2-tuple (text, usage) where text is the string LLM response content and usage is a dict with
token usage metadata. Raises an exception on API communication failure."
    },
    {
      "name": "_read_error_body",
      "signature": "_read_error_body(exc) -> str | None",
      "pre_condition": "exc is an urllib.error.HTTPError instance.",
      "post_condition": "Returns the raw response body string from the HTTP error when readable; returns None when the body cannot
be read."
    }
  ]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/llm_client-py/_retry_create.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a 2-tuple (text, usage) where text is a non-empty string containing the LLM assistant response content
and usage is a dict representing the token-usage metadata from the API response (an empty dict when no usage data is present in
the response). On failure, raises an exception with the following error contract: (a) non-recoverable errors HTTP 400 responses
and their SDK-equivalent exception types are raised immediately without any retry; (b) rate-limiting errors HTTP 429 responses
and their SDK-equivalent exception types are retried with exponential backoff plus random jitter, bounded above and subject to a
maximum retry count; upon exhausting the rate-limit retry budget, a RuntimeError is raised; (c) transient server errors HTTP 5xx
responses and other unexpected exception types are retried with exponential backoff plus random jitter, subject to a distinct
maximum retry count; upon exhausting the transient-error retry budget, a RuntimeError is raised. The backoff wait duration for
each error category grows exponentially with each successive attempt and is clamped at a category-specific upper bound. When a
CLI-based LLM backend is active, the request is delegated to the CLI agent subprocess and its return value is returned directly
without further processing. When the model is identified as belonging to the Anthropic model family, the request is routed to the
native Anthropic API endpoint rather than the OpenAI-compatible endpoint.",
    "actual_behavior": "If `is_cli_backend_enabled()` evaluates to True, the function delegates to `run_agent_for_messages(model,
messages)` and either returns its result a 2tuple `(text, usage_dict)` where `text` is a string and `usage_dict` is a dict

```



```
(possibly empty) or propagates any exception raised by that call. Otherwise, the function enters a retry loop. In each iteration:\n- If the model is detected as an Anthropic model (`_is_anthropic_model(model)` is True), it calls `_anthropic_create(model, messages)`. For nonAnthropic models, it calls `client.chat.completions.create(model=model, messages=messages, **extra)` (with `extra` possibly containing an `extra_body` if URL injection is enabled).\n- On a successful call, the response content from the first choice (`response.choices[0].message.content`) becomes `text`; if `response.usage` is not None, `usage_dict` is obtained via `response.usage.model_dump()`, otherwise `usage_dict` is an empty dict. The tuple `(text, usage_dict)` is returned.\n- If `BadRequestError` is raised (by the API or client), it is immediately reraised without retry.\n- If `urllib.error.HTTPError` is raised:\n    * If `exc.code == 400`, re-raise the exception.\n    * If `exc.code == 429`, increment a ratelimit attempt counter. If the counter reaches `_MAX_RATE_LIMIT_RETRIES`, raise `RuntimeError` with a descriptive message and the original exception as cause. Otherwise wait for `min(2 ** (attempts-1) * 5, 300) + random.uniform(1, 10)` seconds and continue.\n    * For all other status codes (e.g., 5xx), increment a transient failure attempt counter. If that counter reaches `_MAX_LLM_RETRIES`, raise `RuntimeError` with a descriptive message and the original exception as cause. Otherwise wait for `min(2 ** (attempts-1) * 5, 60) + random.uniform(1, 3)` seconds and continue.\n- If `RateLimitError` is raised, apply the same logic as for HTTP 429 (increment the ratelimit counter, possibly raise `RuntimeError` after retries exhausted, else wait with the same backoff as 429).\n- Any other `Exception` is treated as a transient failure: increment the transient counter; if the counter reaches `_MAX_LLM_RETRIES`, raise `RuntimeError` with a descriptive message and the original exception as cause; otherwise wait with the same backoff as transient HTTP errors and continue.\n- Exceptions not derived from `Exception` (e.g., `KeyboardInterrupt`, `SystemExit`) are not caught and propagate out immediately.\n\nFormally, let `Raise(e)` denote that the function raises exception `e`. Let `Return(t)` denote a normal return with value `t`. The postcondition is:\n(is_cli_backend_enabled() (Return(run_agent_for_messages(model, messages)) Raise(e) where e was raised by run_agent_for_messages()))\n\n(is_cli_backend_enabled() (Return((text, usage)) text String usage Dict) (Raise(BadRequestError) Raise(RuntimeError)) ),\n\nwhere the eventual outcome is governed by the retry logic above, and the function either returns a successful tuple or raises one of the specified exceptions after exhausting retries or encountering a permanent error.",\n    "code_evidence": "Line 21: text = response.choices[0].message.content\nLine 23: return text, usage",\n    "trigger_condition": "The specification requires the returned text to be a non-empty string. The code does not verify that the content is non-empty; it returns whatever the API provides. When the API responds with an empty string (e.g., content=''), the code returns an empty text, which violates the post-condition." }\n}
```

► Bug Validator result (完整)

```
{\n  "id": "src-llm_client-py--retry_create",\n  "source_file": "src/llm_client.py",\n  "function_name": "_retry_create",\n  "confirmation_status": "confirmed",\n  "attempts": 1,\n  "probe_script": "fm_agent/bug_validation/probe_src-llm_client-py--retry_create.py",\n  "detail_file": "fm_agent/bug_validation/src-llm_client-py--retry_create.md",\n  "probe_stdout": "CONFIRMED - actual: '' | expected: 'non-empty string (spec requirement)'",\n  "trigger_summary": "When the LLM API returns an empty response content, _retry_create returns an empty string instead of ensuring the returned text is non-empty as required by the specification." }\n}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:07:13.361229Z	4509
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:07:56.256726Z	6299

事件 1 — [llm_debf344e01f2404d93d1ff8780f1e593](#)

► parsed_json (完整)

```
{\n  "post_condition": "If `_is_cli_backend_enabled()` evaluates to True, the function delegates to `run_agent_for_messages(model, messages)` and either returns its result - a 2-tuple `(text, usage_dict)` where `text` is a string and `usage_dict` is a dict (possibly empty) - or propagates any exception raised by that call. Otherwise, the function enters a retry loop. In each iteration:\n- If the model is detected as an Anthropic model (`_is_anthropic_model(model)` is True), it calls `_anthropic_create(model, messages)`. For non-Anthropic models, it calls `client.chat.completions.create(model=model, messages=messages, **extra)` (with `extra` possibly containing an `extra_body` if URL injection is enabled).\n- On a successful call, the response content from the first choice (`response.choices[0].message.content`) becomes `text`; if `response.usage` is not None, `usage_dict` is obtained via `response.usage.model_dump()`, otherwise `usage_dict` is an empty dict. The tuple `(text, usage_dict)` is returned.\n- If `BadRequestError` is raised (by the API or client), it is immediately re-raised without retry.\n- If `urllib.error.HTTPError` is raised:\n    * If `exc.code == 400`, re-raise the exception.\n    * If `exc.code == 429`, increment a rate-limit attempt counter. If the counter reaches `_MAX_RATE_LIMIT_RETRIES`, raise `RuntimeError` with a descriptive message and the original exception as cause. Otherwise wait for `min(2 ** (attempts-1) * 5, 300) + random.uniform(1, 10)` seconds and continue.\n    * For all other status codes (e.g., 5xx), increment a transient failure attempt counter. If that counter reaches `_MAX_LLM_RETRIES`, raise `RuntimeError` with a descriptive message and the original exception as cause. Otherwise wait for `min(2 ** (attempts-1) * 5, 60) + random.uniform(1, 3)` seconds and continue.\n- If `RateLimitError` is raised, apply the same logic as for HTTP 429 (increment the rate-limit counter, possibly raise `RuntimeError` after retries exhausted, else wait with the same backoff as 429).\n- Any other `Exception` is treated as a transient failure: increment the transient counter; if the counter reaches `_MAX_LLM_RETRIES`, raise `RuntimeError` with a descriptive message and the original exception as cause; otherwise wait
```

```

with the same backoff as transient HTTP errors and continue.\n- Exceptions not derived from `Exception` (e.g.,
`KeyboardInterrupt`, `SystemExit`) are not caught and propagate out immediately.\n\nFormally, let `Raise(e)` denote that the
function raises exception `e`. Let `Return(t)` denote a normal return with value `t`. The post-condition is:\n
(is_cli_backend_enabled()  $\wedge$  (Return(run_agent_for_messages(model, messages))  $\vee$  Raise(e) where e was raised by
run_agent_for_messages))\n  $\vee$  \n ( $\neg$ is_cli_backend_enabled()  $\wedge$  \n ( $\square$  (Return( (text, usage) )  $\wedge$  text  $\in$  String  $\wedge$  usage  $\in$  Dict)  $\diamond$ 
(Raise(BadRequestError)  $\vee$  Raise(RuntimeError)) ),\n where the eventual outcome is governed by the retry logic above, and the
function either returns a successful tuple or raises one of the specified exceptions after exhausting retries or encountering a
permanent error."
}

```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_debf344e01f2404d93d1ff8780f1e593_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_debf344e01f2404d93d1ff8780f1e593_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_debf344e01f2404d93d1ff8780f1e593_response.txt)

事件 2 — llm_e93dc3714d934b35852d08625269d434

► parsed_json (完整)

```

{
  "verdict": "MISMATCH",
  "counterexample": "model='openai/gpt-4o', messages=[{'role':'user','content':'respond with empty string'}], client configured
for OpenAI-compatible endpoint, is_cli_backend_enabled()==False, _is_anthropic_model(model)==False. The API returns a successful
completion with choices[0].message.content == '' and response.usage is None.",
  "offending_statements": "Line 21: text = response.choices[0].message.content\nLine 23: return text, usage",
  "reason": "The specification requires the returned text to be a non-empty string. The code does not verify that the content is
non-empty; it returns whatever the API provides. When the API responds with an empty string (e.g., content=''), the code returns
an empty text, which violates the post-condition."
}

```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_e93dc3714d934b35852d08625269d434_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_e93dc3714d934b35852d08625269d434_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_e93dc3714d934b35852d08625269d434_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块：opencode_trace

FMA-MISMATCH-165 — src--opencode_trace-py--_opencode_env

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 4 — LLM Communication & Tracing / [opencode_trace](#)；源代码 [src/opencode_trace.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a dictionary of string keys to string values suitable as a subprocess environment. The returned dictionary contains every entry from the current process environment. It contains a TRACE_DIR entry whose value is an absolute filesystem path naming a directory that exists on the filesystem after the call and is located under the trace structure rooted at work_dir. It contains a TRACE_FILENAME entry whose value equals event_id. It contains a PWD entry whose value is the absolute path of the directory that directly contains work_dir. When all of the configured LLM API key, base URL, model name, and provider are available (i.e., the provider configuration is complete), the returned dictionary additionally contains an LLM_API_KEY entry whose value is the configured API key and an OPENCODE_CONFIG_CONTENT entry whose value is a valid JSON string encoding a dictionary whose top-level keys are the union of the top-level keys of any pre-existing OPENCODE_CONFIG_CONTENT value in the environment and the top-level keys of the resolved provider configuration, where provider-defined keys override pre-existing entries for the same key; when the provider configuration is incomplete, neither LLM_API_KEY nor a modified OPENCODE_CONFIG_CONTENT appears.
- **Actual behavior**: Upon normal completion (i.e. the function returns a dictionary without raising an exception), the following holds:

1. Filesystem sideeffect

The directory located at

```
td = os.path.abspath(os.path.join(_trace_dir(work_dir), 'opencode'))
```

exists. If it did not exist before the call, it is created.

2. Returned environment dictionary

Let `E0` be a snapshot of `os.environ` taken at function entry. The returned mapping `env` satisfies:

- `env['TRACE_DIR'] = td`
- `env['TRACE_FILENAME'] = event_id`
- `env['PWD'] = os.path.dirname(os.path.abspath(work_dir))`
- If `cfg = _opencode_provider_config()` is not `None` then:
 - `env['LLM_API_KEY'] = settings.llm.api_key`
 - Let `raw = E0.get('OPENCODE_CONFIG_CONTENT')`. If `raw` is a string that can be parsed by `json.loads` into a dictionary, then `base = that dictionary`; otherwise `base = {}`. Then `env['OPENCODE_CONFIG_CONTENT'] = json.dumps(_deep_merge(base, cfg))`. Otherwise (when `cfg` is `None`), the values of `'LLM_API_KEY'` and `'OPENCODE_CONFIG_CONTENT'` in `env` are exactly those that were present in `E0` (if any).
- For every other key `k` that existed in `E0`, `env[k] = E0[k]`. The set of keys of `env` is exactly `keys(E0) - {'TRACE_DIR', 'TRACE_FILENAME', 'PWD'} ∪ ({'LLM_API_KEY', 'OPENCODE_CONFIG_CONTENT'} if cfg is not None)`, with no additional keys.

3. No other observable state changes

Apart from the directory creation, the filesystem is unchanged; no network or other side effects occur.

Formal logic (expressed over the returned value and the filesystem):

Let `fs` be the filesystem state after the call. The postcondition `Post(env, fs)` holds iff:

```
td, base, raw, cfg :
  td = abspath(join(_trace_dir(work_dir), 'opencode'))
  directory_exists(td, fs)
  cfg = _opencode_provider_config()
  env = E0 {
    'TRACE_DIR': td,
    'TRACE_FILENAME': event_id,
    'PWD': dirname(abspath(work_dir))
  }
  ( if cfg is None then {
    'LLM_API_KEY': settings.llm.api_key,
    'OPENCODE_CONFIG_CONTENT': json.dumps(_deep_merge(base, cfg))
  } else {
    raw = E0.get('OPENCODE_CONFIG_CONTENT')
    ( if (raw is str & json.loads(raw) succeeds & the result is a dict) then
      base = that dict
    else
      base = {}
    )
  }
  )
  else true
)
```

(Here `U` denotes functional override of a mapping: for a given base mapping `M` and an update set `U`, `M ∪ U` maps each key `k` to `U[k]` if `k ∈ dom(U)`, otherwise to `M[k]`.)

- **Code evidence:** Line 25: `env["OPENCODE_CONFIG_CONTENT"] = json.dumps(_deep_merge(existing, provider_config))`
- **Trigger condition:** Specification states 'providerdefined keys override preexisting entries for the same key' (i.e., shallow override). The code uses `_deep_merge` which recursively merges nested dictionaries, causing preexisting nested keys to survive when they should be replaced. A concrete environment where `OPENCODE_CONFIG_CONTENT` contains a nested key that also appears in the provider config exposes the mismatch.
- **Validator trigger:** Pre-existing `OPENCODE_CONFIG_CONTENT` with nested provider keys: `deep_merge` preserves existing nested keys (e.g. 'plugins') that should be overridden by the provider config per the spec.
- **Probe stdout:** CONFIRMED -- `deep_merge` preserved existing nested key "plugins": [{"plugin-a", "plugin-b"}] Full provider block: {"npm": "@ai-sdk/openai-compatible", "plugins": [{"plugin-a", "plugin-b"}], "options": {"baseUrl": "https://test.example.com/api/v1", "apiKey": "env:LLM_API_KEY"}}, {"models": {"test-model": {}}}

► SPEC sidecar (完整)

```
{
  "signature": "_opencode_env(work_dir, event_id) -> dict[str, str]",
  "pre_condition": "work_dir is a valid filesystem directory path; event_id is a non-empty string",
  "post_condition": "Returns a dictionary of string keys to string values suitable as a subprocess environment. The returned dictionary contains every entry from the current process environment. It contains a TRACE_DIR entry whose value is an absolute filesystem path naming a directory that exists on the filesystem after the call and is located under the trace structure rooted at work_dir. It contains a TRACE_FILENAME entry whose value equals event_id. It contains a PWD entry whose value is the absolute path of the directory that directly contains work_dir. When all of the configured LLM API key, base URL, model name, and provider are available (i.e., the provider configuration is complete), the returned dictionary additionally contains an LLM_API_KEY entry whose value is the configured API key and an OPENCODE_CONFIG_CONTENT entry whose value is a valid JSON string encoding a dictionary
```

```

whose top-level keys are the union of the top-level keys of any pre-existing OPENCODE_CONFIG_CONTENT value in the environment and the top-level keys of the resolved provider configuration, where provider-defined keys override pre-existing entries for the same key; when the provider configuration is incomplete, neither LLM_API_KEY nor a modified OPENCODE_CONFIG_CONTENT appears."
}

```

► INFO / callees sidecar (完整)

```

{
  "callees": [
    {
      "name": "_trace_dir",
      "signature": "_trace_dir(work_dir) -> str",
      "pre_condition": "work_dir is a valid filesystem directory path",
      "post_condition": "Returns a filesystem path string that is a subdirectory named 'trace' under work_dir"
    },
    {
      "name": "_opencode_provider_config",
      "signature": "_opencode_provider_config() -> dict | None",
      "pre_condition": "None (takes no arguments)",
      "post_condition": "Returns a dictionary representing an OpenCode provider block when the configured LLM settings (api_key, base_url, name, and provider) are all present and non-empty; returns None when any required setting is missing or falsy, indicating no provider configuration should be injected"
    },
    {
      "name": "_deep_merge",
      "signature": "_deep_merge(base, overlay) -> dict",
      "pre_condition": "base and overlay are dictionaries",
      "post_condition": "Returns a dictionary whose top-level keys are the union of the top-level keys of base and overlay. For any key whose value is a dict in both base and overlay, the returned value is the recursive merge of those nested dicts. For any key whose value is not a dict in both dictionaries, or present in only one input, the returned value is the overlay value when the key exists in overlay, otherwise the base value"
    }
  ]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_opencode_env.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dictionary of string keys to string values suitable as a subprocess environment. The returned dictionary contains every entry from the current process environment. It contains a TRACE_DIR entry whose value is an absolute filesystem path naming a directory that exists on the filesystem after the call and is located under the trace structure rooted at work_dir. It contains a TRACE_FILENAME entry whose value equals event_id. It contains a PWD entry whose value is the absolute path of the directory that directly contains work_dir. When all of the configured LLM API key, base URL, model name, and provider are available (i.e., the provider configuration is complete), the returned dictionary additionally contains an LLM_API_KEY entry whose value is the configured API key and an OPENCODE_CONFIG_CONTENT entry whose value is a valid JSON string encoding a dictionary whose top-level keys are the union of the top-level keys of any pre-existing OPENCODE_CONFIG_CONTENT value in the environment and the top-level keys of the resolved provider configuration, where provider-defined keys override pre-existing entries for the same key; when the provider configuration is incomplete, neither LLM_API_KEY nor a modified OPENCODE_CONFIG_CONTENT appears.",
    "actual_behavior": "Upon normal completion (i.e. the function returns a dictionary without raising an exception), the following holds:\n\n1. **Filesystem sideeffect** \n The directory located at \n `td` = os.path.abspath(os.path.join(_trace_dir(work_dir), 'opencode'))` \n exists. If it did not exist before the call, it is created.\n\n2. **Returned environment dictionary** \n Let `E0` be a snapshot of `os.environ` taken at function entry. The returned mapping `env` satisfies:\n\n - `env['TRACE_DIR']` = `td`\n - `env['TRACE_FILENAME']` = `event_id`\n - `env['PWD']` = os.path.dirname(os.path.abspath(work_dir))\n\n - If `cfg = _opencode_provider_config()` is not `None` then:\n\n * `env['LLM_API_KEY']` = settings.llm.api_key\n * Let `raw = E0.get('OPENCODE_CONFIG_CONTENT')`. If `raw` is a string that can be parsed by `json.loads` into a dictionary, then `base = that dictionary`; otherwise `base = {}`. Then \n `env['OPENCODE_CONFIG_CONTENT']` = json.dumps(_deep_merge(base, cfg)).\n\n Otherwise (when `cfg` is `None`), the values of `LLM_API_KEY` and `OPENCODE_CONFIG_CONTENT` in `env` are exactly those that were present in `E0` (if any).\n\n - For every other key `k` that existed in `E0`, `env[k] = E0[k]`. The set of keys of `env` is exactly `keys(E0)`\n\n3. **No other observable state changes** \n Apart from the directory creation, the filesystem is unchanged; no network or other side effects occur.\n\n**Formal logic** (expressed over the returned value and the filesystem):\n\nLet `fs` be the filesystem state after the call. The postcondition `Post(env, fs)` holds iff:\n\n```\n\n td, base, raw, cfg : \n td = abspath(join(_trace_dir(work_dir), 'opencode'))\n directory_exists(td, fs)\n cfg = _opencode_provider_config()\n env = E0 { \n 'TRACE_DIR': td, \n 'TRACE_FILENAME': event_id, \n 'PWD': dirname(abspath(work_dir)) \n } \n ( if cfg None then { \n 'LLM_API_KEY': settings.llm.api_key, \n 'OPENCODE_CONFIG_CONTENT': json.dumps(_deep_merge(base, cfg)) \n } else { \n ( if cfg None then \n raw = E0.get('OPENCODE_CONFIG_CONTENT') \n ( if (raw is str json.loads(raw) succeeds the result is a dict) then \n base = that dict \n else \n base = {} \n ) \n else true \n ) \n```\n\n(Here `` denotes functional override of a mapping: for a given base mapping `M` and an update set `U`, `M U` maps each key `k` to `U[k]` if `k` dom(U), otherwise to `M[k]`.)",
    "code_evidence": "Line 25: env[\"OPENCODE_CONFIG_CONTENT\"] = json.dumps(_deep_merge(existing, provider_config))",
    "trigger_condition": "Specification states 'providerdefined keys override preexisting entries for the same key' (i.e., shallow override). The code uses _deep_merge which recursively merges nested dictionaries, causing preexisting nested keys to survive when they should be replaced. A concrete environment where OPENCODE_CONFIG_CONTENT contains a nested key that also appears in the provider config exposes the mismatch."
  }
}

```

► Bug Validator result (完整)

```
{
  "id": "src--opcode_trace-py--_opcode_env",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opcode_trace-py/_opcode_env.py",
  "function_name": "_opcode_env",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--opcode_trace-py--_opcode_env.py",
  "detail_file": "fm_agent/bug_validation/src--opcode_trace-py--_opcode_env.md",
  "probe_stdout": "CONFIRMED -- deep_merge preserved existing nested key \"plugins\": ['plugin-a', 'plugin-b']\n Full provider
block: {\n  \"npm\": \"@ai-sdk/openai-compatible\", \n  \"plugins\": [\n    \"plugin-a\", \n    \"plugin-b\"\n  ], \n  \"options\": {\n
{\n  \"baseURL\": \"https://test.example.com/api/v1\", \n  \"apiKey\": \"{env:LLM_API_KEY}\" \n  }, \n  \"models\": {\n
\"test-model\": {\n  }\n }\n }\n",
  "trigger_summary": "Pre-existing OPENCODE_CONFIG_CONTENT with nested provider keys: deep_merge preserves existing nested keys
(e.g. 'plugins') that should be overridden by the provider config per the spec."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:07:43.379185Z	4762
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:08:43.328018Z	10409

事件 1 — [llm_767bd496a4a84621adc4f684aa1e6e91](#)

► parsed_json (完整)

```
{
  "post_condition": "Upon normal completion (i.e. the function returns a dictionary without raising an exception), the following
holds:\n\n1. **Filesystem side-effect** \n  The directory located at \n  `td =
os.path.abspath(os.path.join(_trace_dir(work_dir), 'opcode'))` \n  exists. If it did not exist before the call, it is
created.\n\n2. **Returned environment dictionary** \n  Let `E0` be a snapshot of `os.environ` taken at function entry. The
returned mapping `env` satisfies:\n\n  - `env['TRACE_DIR'] = td`\n  - `env['TRACE_FILENAME'] = event_id`\n  - `env['PWD'] =
os.path.dirname(os.path.abspath(work_dir))`\n  - If `cfg = _opcode_provider_config()` is not `None` then:\n    *
`env['LLM_API_KEY'] = settings.llm.api_key`\n    * Let `raw = E0.get('OPENCODE_CONFIG_CONTENT')`. If `raw` is a string that
can be parsed by `json.loads` into a dictionary, then `base = that dictionary`; otherwise `base = {}`. Then \n
`env['OPENCODE_CONFIG_CONTENT'] = json.dumps(_deep_merge(base, cfg))`. \n    Otherwise (when `cfg` is `None`), the values of
`'LLM_API_KEY'` and `'OPENCODE_CONFIG_CONTENT'` in `env` are exactly those that were present in `E0` (if any). \n
  - For every
other key `k` that existed in `E0`, `env[k] = E0[k]`. The set of keys of `env` is exactly `keys(E0) u
{'TRACE_DIR', 'TRACE_FILENAME', 'PWD'} u ({'LLM_API_KEY', 'OPENCODE_CONFIG_CONTENT'} if cfg != None)`, with no additional keys.\n\n3.
**No other observable state changes** \n  Apart from the directory creation, the filesystem is unchanged; no network or other
side effects occur.\n\n**Formal logic** (expressed over the returned value and the filesystem):\n\nLet `fs` be the filesystem
state after the call. The post-condition `Post(env, fs)` holds iff:\n\n```\n∃ td, base, raw, cfg : \n  td =
abspath(join(_trace_dir(work_dir), 'opcode')) \n  ∧ directory_exists(td, fs) \n  ∧ cfg = _opcode_provider_config() \n  ∧ env = E0 ⊙
{\n    'TRACE_DIR': td, \n    'TRACE_FILENAME': event_id, \n    'PWD': dirname(abspath(work_dir)) \n  } \n  ⊙ ( if
cfg ≠ None then {\n    'LLM_API_KEY': settings.llm.api_key, \n    'OPENCODE_CONFIG_CONTENT':
json.dumps(_deep_merge(base, cfg)) \n  } else ∅ ) \n  ∧ ( if cfg ≠ None then \n    raw =
E0.get('OPENCODE_CONFIG_CONTENT') \n    ∧ ( if (raw is str ∧ json.loads(raw) succeeds ∧ the result is a dict) then \n
base = that dict \n    else \n      base = {} \n    ) \n    else true \n  ) \n  ```\n\n(Here `⊙` denotes
functional override of a mapping: for a given base mapping `M` and an update set `U`, `M ⊙ U` maps each key `k` to `U[k]` if `k ∈
dom(U)`, otherwise to `M[k]`.)"
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_767bd496a4a84621adc4f684aa1e6e91_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_767bd496a4a84621adc4f684aa1e6e91_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_767bd496a4a84621adc4f684aa1e6e91_response.txt)

事件 2 — [llm_be3ae1659dd04fe2a9664710df99f748](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Pre-existing environment variable OPENCODE_CONFIG_CONTENT = '{\"plugins\": {\"p1\": true}}' and
_opcode_provider_config() returns {\"plugins\": {\"p2\": true}}. The specification requires that provider-defined keys override
pre-existing entries, i.e., the resulting OPENCODE_CONFIG_CONTENT should be '{\"plugins\": {\"p2\": true}}'. The code deep-merges,
producing '{\"plugins\": {\"p1\": true, \"p2\": true}}' which violates the specification.",
  "offending_statements": "Line 25: env[\"OPENCODE_CONFIG_CONTENT\"] = json.dumps(_deep_merge(existing, provider_config))",
  "reason": "Specification states 'provider-defined keys override pre-existing entries for the same key' (i.e., shallow override).
The code uses _deep_merge which recursively merges nested dictionaries, causing pre-existing nested keys to survive when they
```



```
should be replaced. A concrete environment where OPENCODE_CONFIG_CONTENT contains a nested key that also appears in the provider config exposes the mismatch."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_be3ae1659dd04fe2a9664710df99f748_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_be3ae1659dd04fe2a9664710df99f748_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_be3ae1659dd04fe2a9664710df99f748_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-168 — src--opencode_trace-py--_payload_dir

- 四类结论： 契约不确定
- 分类依据： 沿用旧 fm_agent/bug_list.md 的逐条审计结论： 沿用旧清单的逐条审计结论。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 4 — LLM Communication & Tracing / opencode_trace； 源代码 src/opencode_trace.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Returns a filesystem path string that names a subdirectory called 'payloads' located directly under trace_dir; the directory named by the returned path exists on the filesystem after the call returns
- **Actual behavior**: After normal execution, the function returns the string path = os.path.join(trace_dir, 'payloads') and the directory at that path exists (os.path.isdir(path) is true). If an OSError is raised, the directory at path does not exist (os.path.isdir(path) is false). The argument trace_dir remains unchanged. Formally: (return_value = path isdir(path)) (OSError raised isdir(path)) where path = os.path.join(trace_dir, 'payloads').
- **Code evidence**: Line 2: path = os.path.join(trace_dir, "payloads") Line 3: os.makedirs(path, exist_ok=True)
- **Trigger condition**: The specification requires the function to return a path and ensure the directory exists, but when os.makedirs raises an OSError (e.g., FileExistsError because trace_dir is a file), the code raises an exception, returns nothing, and the directory does not exist. This violates the unconditional guarantee of a returned path and an existing directory.
- **Validator trigger**: When trace_dir is a file instead of a directory, os.makedirs raises NotADirectoryError which propagates unhandled, violating the spec guarantee that a directory exists after the call.
- **Probe stdout**: CONFIRMED — actual: "NotADirectoryError: [Errno 20] Not a directory: '/tmp/probe__payload_dir_qok5x53u/not_a_dir/payloads'" | expected: 'a valid path (dir exists)'

► SPEC sidecar (完整)

```
{
  "signature": "_payload_dir(trace_dir) -> str",
  "pre_condition": "trace_dir is a valid filesystem directory path",
  "post_condition": "Returns a filesystem path string that names a subdirectory called 'payloads' located directly under trace_dir; the directory named by the returned path exists on the filesystem after the call returns"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_payload_dir.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a filesystem path string that names a subdirectory called 'payloads' located directly under trace_dir; the directory named by the returned path exists on the filesystem after the call returns",
    "actual_behavior": "After normal execution, the function returns the string path = os.path.join(trace_dir, 'payloads') and the directory at that path exists (os.path.isdir(path) is true). If an OSError is raised, the directory at path does not exist (os.path.isdir(path) is false). The argument trace_dir remains unchanged. Formally: (return_value = path isdir(path)) (OSError raised isdir(path)) where path = os.path.join(trace_dir, 'payloads').",
    "code_evidence": "Line 2: path = os.path.join(trace_dir, \"payloads\")\nLine 3: os.makedirs(path, exist_ok=True)",
    "trigger_condition": "The specification requires the function to return a path and ensure the directory exists, but when os.makedirs raises an OSError (e.g., FileExistsError because trace_dir is a file), the code raises an exception, returns nothing, and the directory does not exist. This violates the unconditional guarantee of a returned path and an existing directory."
  }
}
```



```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--opcode_trace-py--_payload_dir",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opcode_trace-py/_payload_dir.py",
  "function_name": "_payload_dir",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--opcode_trace-py--_payload_dir.py",
  "detail_file": "fm_agent/bug_validation/src--opcode_trace-py--_payload_dir.md",
  "probe_stdout": "CONFIRMED - actual: \"NotADirectoryError: [Errno 20] Not a directory: '/tmp/probe__payload_dir_qok5x53u/not_a_dir/payloads'\" | expected: 'a valid path (dir exists)'\n",
  "trigger_summary": "When trace_dir is a file instead of a directory, os.makedirs raises NotADirectoryError which propagates unhandled, violating the spec guarantee that a directory exists after the call."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:07:45.381651Z	2334
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:08:21.897565Z	1833

事件 1 — [llm_98cb3f422d5b48ec8111a70db311afdd](#)

► parsed_json（完整）

```
{
  "post_condition": "After normal execution, the function returns the string path = os.path.join(trace_dir, 'payloads') and the directory at that path exists (os.path.isdir(path) is true). If an OSError is raised, the directory at path does not exist (os.path.isdir(path) is false). The argument trace_dir remains unchanged. Formally: (return_value = path ^ isdir(path)) v (OSError raised ^ ~isdir(path)) where path = os.path.join(trace_dir, 'payloads')."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_98cb3f422d5b48ec8111a70db311afdd_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_98cb3f422d5b48ec8111a70db311afdd_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_98cb3f422d5b48ec8111a70db311afdd_response.txt)

事件 2 — [llm_a45ab5f926b440fab1b99b1df2022f5d](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "trace_dir = '/bin/ls' (assuming /bin/ls exists as a file)",
  "offending_statements": "Line 2: path = os.path.join(trace_dir, \"payloads\")\nLine 3: os.makedirs(path, exist_ok=True)",
  "reason": "The specification requires the function to return a path and ensure the directory exists, but when os.makedirs raises an OSError (e.g., FileExistsError because trace_dir is a file), the code raises an exception, returns nothing, and the directory does not exist. This violates the unconditional guarantee of a returned path and an existing directory."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_a45ab5f926b440fab1b99b1df2022f5d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_a45ab5f926b440fab1b99b1df2022f5d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_a45ab5f926b440fab1b99b1df2022f5d_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-170 — [src--opcode_trace-py--_trace_dir](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: **confirmed**; attempts 1

- **Phase / 模块:** Phase 4 — LLM Communication & Tracing / [opcode_trace](#); 原源码 [src/opencode_trace.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a filesystem path string whose value names a subdirectory path under work_dir with the terminal component 'trace'. The result is deterministic identical work_dir inputs produce identical outputs. The function is pure: it computes and returns a path without creating, checking, or modifying any filesystem entry.
- **Actual behavior:** The function returns a string equal to os.path.join(work_dir, 'trace'); the return value is a non-empty string representing a filesystem path (the entry 'trace' inside work_dir). No exceptions are raised, and the program state is otherwise unchanged.
- **Code evidence:** Line 2: return os.path.join(work_dir, "trace")
- **Trigger condition:** The specification requires the returned path to be a subdirectory path under work_dir. When work_dir is an empty string, os.path.join("", 'trace') returns 'trace', which does not contain any directory separator and thus does not represent a subdirectory of the (unspecified) work_dir. Similarly, on Windows, an input like 'C:' (a drive letter without a trailing backslash) would yield 'C:trace', which is not a subdirectory of the root of drive C:. Therefore, the implementation does not guarantee the 'under work_dir' property for all valid inputs.
- **Validator trigger:** When work_dir is an empty string, os.path.join("", 'trace') returns 'trace' with no directory separator, violating the spec requirement that the result be a subdirectory path under work_dir.
- **Probe stdout:** CONFIRMED — actual: 'trace' (no directory separator in result, not a subdirectory path under work_dir="")

► SPEC sidecar (完整)

```
{
  "signature": "_trace_dir(work_dir) -> str",
  "pre_condition": "work_dir is a non-empty string representing a filesystem directory path.",
  "post_condition": "Returns a filesystem path string whose value names a subdirectory path under work_dir with the terminal component 'trace'. The result is deterministic – identical work_dir inputs produce identical outputs. The function is pure: it computes and returns a path without creating, checking, or modifying any filesystem entry."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_trace_dir.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a filesystem path string whose value names a subdirectory path under work_dir with the terminal component 'trace'. The result is deterministic identical work_dir inputs produce identical outputs. The function is pure: it computes and returns a path without creating, checking, or modifying any filesystem entry.",
    "actual_behavior": "The function returns a string equal to os.path.join(work_dir, 'trace'); the return value is a non-empty string representing a filesystem path (the entry 'trace' inside work_dir). No exceptions are raised, and the program state is otherwise unchanged.",
    "code_evidence": "Line 2: return os.path.join(work_dir, \"trace\")",
    "trigger_condition": "The specification requires the returned path to be a subdirectory path under work_dir. When work_dir is an empty string, os.path.join('', 'trace') returns 'trace', which does not contain any directory separator and thus does not represent a subdirectory of the (unspecified) work_dir. Similarly, on Windows, an input like 'C:' (a drive letter without a trailing backslash) would yield 'C:trace', which is not a subdirectory of the root of drive C:. Therefore, the implementation does not guarantee the 'under work_dir' property for all valid inputs."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--opencode_trace-py--_trace_dir",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_trace_dir.py",
  "function_name": "_trace_dir",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--opencode_trace-py--_trace_dir.py",
  "detail_file": "fm_agent/bug_validation/src--opencode_trace-py--_trace_dir.md",
  "probe_stdout": "CONFIRMED – actual: 'trace' (no directory separator in result, not a subdirectory path under work_dir=\\\"\\\")\\n",
  "trigger_summary": "When work_dir is an empty string, os.path.join('', 'trace') returns 'trace' with no directory separator,
```

```
violating the spec requirement that the result be a subdirectory path under work_dir."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:14:47.762201Z	1341
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:15:04.351238Z	3493

事件 1 — [llm_b016289c17a64ffc807b71d0b29f521c](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string equal to os.path.join(work_dir, 'trace'); the return value is a non-empty string representing a filesystem path (the entry 'trace' inside work_dir). No exceptions are raised, and the program state is otherwise unchanged."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b016289c17a64ffc807b71d0b29f521c_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_b016289c17a64ffc807b71d0b29f521c_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_b016289c17a64ffc807b71d0b29f521c_response.txt)

事件 2 — [llm_5a54f47806fd46b4b39ffbc7a8b1e2ed](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input: work_dir = \"\" (empty string). Output: \"trace\". This is not a subdirectory path under an empty base directory, violating the requirement that the result names a subdirectory path under work_dir.",
  "offending_statements": "Line 2: return os.path.join(work_dir, \"trace\")",
  "reason": "The specification requires the returned path to be a subdirectory path under work_dir. When work_dir is an empty string, os.path.join('', 'trace') returns 'trace', which does not contain any directory separator and thus does not represent a subdirectory of the (unspecified) work_dir. Similarly, on Windows, an input like 'C:' (a drive letter without a trailing backslash) would yield 'C:trace', which is not a subdirectory of the root of drive C:. Therefore, the implementation does not guarantee the 'under work_dir' property for all valid inputs."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5a54f47806fd46b4b39ffbc7a8b1e2ed_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5a54f47806fd46b4b39ffbc7a8b1e2ed_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5a54f47806fd46b4b39ffbc7a8b1e2ed_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

Phase 5 — Specification & Verification Utilities

模块: [batch_prompts](#)

FMA-MISMATCH-188 — [src--generate_batch_prompts-py--main](#)

- 四类结论: 契约不确定
- 分类依据: 沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: [not_confirmed](#); attempts 1
- Phase / 模块: Phase 5 — Specification & Verification Utilities / [batch_prompts](#); 原源码 [src/generate_batch_prompts.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim:** Returns 0. On dry-run: prints the batch plan (phase, layer range, function/batch counts with per-batch details) to stdout without writing any files. On non-dry-run: writes batch prompt .txt files under the output directory, one per batch of functions for the specified phase and layer range, each containing code, signatures, and callee specs for every function in that batch; writes a manifest.json

describing all batches (including already-specified functions on resume); removes stale batch files from the output directory that do not belong to the current run. On resume, functions whose .spec.json and .info.json are both ready are excluded from prompt generation.

Raises ValueError if batch_size is not strictly positive or if the requested layer range is outside [0, total_layers - 1].

- **Actual behavior:** If args.batch_size <= 0, a ValueError with message "--batch-size must be > 0" is raised and no subsequent statements execute. Otherwise, if after reading topdown_layers.json and parsing layers spec the condition start_layer < 0 or end_layer >= total_layers holds, a ValueError with message "layer range {args.layers} out of bounds [0, {total_layers - 1}]" is raised and no further assignments occur. Otherwise (normal termination): args is defined with all attributes reflecting the command line and args.batch_size > 0; work_dir is a Path object resolving to the fm_agent/ working directory (parent of the directory containing this script); repo_root is work_dir.parent; fm_agent_prefix is the string of the relative path from repo_root to work_dir followed by '/'; phases_json is the parsed dict of phases.json and project = phases_json['project']; languages and exts are list values from phases_json (defaulting to [] if missing); ext_to_lang is a dict mapping each extension from exts (lowercased and with leading dot removed) to the corresponding language from languages; topdown_path is the path to the requested phase's topdown_layers.json, topdown its parsed dict, layers = topdown.get('layers', []), and total_layers = len(layers); start_layer and end_layer are the inclusive bounds from parsing args.layers and satisfy 0 start_layer end_layer < total_layers; output_dir is a Path object, equal to Path(args.output_dir) if args.output_dir was truthy, else work_dir / 'spec_prompts' / f'batch_prompts_{project}_phase{args.phase:02d}'; func_to_layer maps each function name across all layers to its layer index (after stripping fm_agent_prefix from the file path if present); all_funcs maps the same names to their function metadata dicts (with the possibly modified file field); manifest_batches, total_functions, skipped_functions, batch_index, write_targets are initialized respectively as an empty list, 0, 0, 0, and an empty list. No filesystem side effects occur beyond the two JSON file reads.
- **Code evidence:** Line 1 - Line 40: The code only parses arguments, validates them, reads JSON configs, and initialises data structures. It never reaches the specification-required logic for generating batch prompts, writing output files, creating manifest.json, removing stale files, or handling resume/dry-run.
- **Trigger condition:** The code block ends after initialisation (line 40). For a valid input that passes all validation, the code does nothing further, violating the specification which mandates file generation, manifest creation, stale-file cleanup, and dry-run output. Any conforming input with batch_size > 0 and an in-range layer spec results in complete omission of the required behaviour.
- **Validator trigger:** Logic verification incorrectly truncated analysis at line 40; main() is 138 lines with full batch-generation logic present.
- **Probe stdout:** NOT CONFIRMED — main() source extends to 138 lines (beyond the claimed 40-line cutoff) and contains all required batch-generation patterns: build_prompt=True, output_dir.mkdir=True, manifest=True, dry_run=True, write_targets=True, stale=True, resume=True

► SPEC sidecar (完整)

```
{
  "signature": "main() -> int",
  "pre_condition": "The process is invoked from a valid FM-Agent working directory containing phases.json and spec_prompts/phase_NN_topdown_layers.json for the requested phase number.",
  "post_condition": "Returns 0. On dry-run: prints the batch plan (phase, layer range, function/batch counts with per-batch details) to stdout without writing any files. On non-dry-run: writes batch prompt .txt files under the output directory, one per batch of functions for the specified phase and layer range, each containing code, signatures, and callee specs for every function in that batch; writes a manifest.json describing all batches (including already-specced functions on resume); removes stale batch files from the output directory that do not belong to the current run. On resume, functions whose .spec.json and .info.json are both ready are excluded from prompt generation. Raises ValueError if batch_size is not strictly positive or if the requested layer range is outside [0, total_layers - 1]."
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "parse_args",
      "signature": "parse_args() -> Namespace",
      "pre_condition": "The process is invoked with command-line arguments including --phase, --layers, --batch-size, --output-dir, --resume, and --dry-run.",
      "post_condition": "Returns a Namespace object whose attributes include phase (int), layers (str), batch_size (int), output_dir (str or None), resume (bool), and dry_run (bool), each reflecting the corresponding command-line argument."
    },
    {
      "name": "read_json",
      "signature": "read_json(filepath) -> dict",
      "pre_condition": "filepath is a path to an existing, syntactically valid JSON file on the filesystem.",
      "post_condition": "Returns the fully parsed content of the JSON file as a Python dict. Raises an error if the file does not exist or contains invalid JSON."
    },
    {
      "name": "parse_layers_spec",
      "signature": "parse_layers_spec(layers_spec) -> tuple[int, int]",
      "pre_condition": "layers_spec is a string in one of two forms: a single non-negative integer N, or a range N-M where N and M are non-negative integers.",
      "post_condition": "Returns a (start, end) tuple of two ints representing the inclusive layer range. For a single integer N,
```

```

returns (N, N). Raises if the format is invalid or if start > end."
},
{
  "name": "is_file_ready",
  "signature": "is_file_ready(file_path) -> bool",
  "pre_condition": "file_path is a path to an extracted function file.",
  "post_condition": "Returns True when both an adjacent .spec.json file (with exactly the fields signature, pre_condition,
post_condition, all strings) and an adjacent .info.json file (with exactly the field callees, containing an array of dicts each
with name, signature, pre_condition, post_condition, all strings) exist; returns False otherwise."
},
{
  "name": "chunked",
  "signature": "chunked(iterable, chunk_size) -> Iterator[list]",
  "pre_condition": "iterable is a finite iterable. chunk_size is a positive integer.",
  "post_condition": "Yields consecutive, non-overlapping sub-lists of the input iterable, each of length exactly chunk_size
except the final sub-list which may be shorter. Element order within and across sub-lists matches the original iterable order."
},
{
  "name": "build_prompt",
  "signature": "build_prompt(phase, layer_idx, is_cycle, prompt_funcs, func_to_layer, all_funcs, work_dir, fm_agent_prefix,
ext_to_lang) -> str",
  "pre_condition": "phase is a positive integer. layer_idx is a non-negative integer. is_cycle is a boolean. prompt_funcs is a
list of function metadata dicts, each containing at least 'file' and 'name' keys. func_to_layer and all_funcs are dicts keyed by
function name. work_dir is a Path to the fm_agent/ root. fm_agent_prefix is a relative path string for prefix-stripping.
ext_to_lang maps lowercase file extensions to language names.",
  "post_condition": "Returns a string containing the full batch prompt text, including domain context (engine overview, phase
types), per-function extracted source code (read from work_dir), per-function signatures, and callee specifications drawn from
earlier-layer .info.json files for each function's callers. The prompt text is self-contained and ready for LLM consumption."
}
]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/generate_batch_prompts-py/main.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns 0. On dry-run: prints the batch plan (phase, layer range, function/batch counts with per-batch details)
to stdout without writing any files. On non-dry-run: writes batch prompt .txt files under the output directory, one per batch of
functions for the specified phase and layer range, each containing code, signatures, and callee specs for every function in that
batch; writes a manifest.json describing all batches (including already-specified functions on resume); removes stale batch files
from the output directory that do not belong to the current run. On resume, functions whose .spec.json and .info.json are both
ready are excluded from prompt generation. Raises ValueError if batch_size is not strictly positive or if the requested layer
range is outside [0, total_layers - 1].",
    "actual_behavior": "If args.batch_size <= 0, a ValueError with message \"--batch-size must be > 0\" is raised and no
subsequent statements execute. Otherwise, if after reading topdown_layers.json and parsing layers spec the condition start_layer <
0 or end_layer >= total_layers holds, a ValueError with message \"layer range {args.layers} out of bounds [0, {total_layers -
1}]\" is raised and no further assignments occur. Otherwise (normal termination): args is defined with all attributes reflecting
the command line and args.batch_size > 0; work_dir is a Path object resolving to the fm_agent/ working directory (parent of the
directory containing this script); repo_root is work_dir.parent; fm_agent_prefix is the string of the relative path from repo_root
to work_dir followed by '/'; phases_json is the parsed dict of phases.json and project = phases_json['project']; languages and
exts are list values from phases_json (defaulting to [] if missing); ext_to_lang is a dict mapping each extension from exts
(lowercased and with leading dot removed) to the corresponding language from languages; topdown_path is the path to the requested
phase's topdown_layers.json, topdown its parsed dict, layers = topdown.get('layers', []), and total_layers = len(layers);
start_layer and end_layer are the inclusive bounds from parsing args.layers and satisfy 0 <= start_layer <= end_layer < total_layers;
output_dir is a Path object, equal to Path(args.output_dir) if args.output_dir was truthy, else work_dir / 'spec_prompts' /
f'batch_prompts_{project}_phase{args.phase:02d}'; func_to_layer maps each function name across all layers to its layer index
(after stripping fm_agent_prefix from the file path if present); all_funcs maps the same names to their function metadata dicts
(with the possibly modified file field); manifest_batches, total_functions, skipped_functions, batch_index, write_targets are
initialized respectively as an empty list, 0, 0, 0, and an empty list. No filesystem side effects occur beyond the two JSON file
reads.",
    "code_evidence": "Line 1 - Line 40: The code only parses arguments, validates them, reads JSON configs, and initialises data
structures. It never reaches the specification-required logic for generating batch prompts, writing output files, creating
manifest.json, removing stale files, or handling resume/dry-run.",
    "trigger_condition": "The code block ends after initialisation (line 40). For a valid input that passes all validation, the
code does nothing further, violating the specification which mandates file generation, manifest creation, stale-file cleanup, and
dry-run output. Any conforming input with batch_size > 0 and an in-range layer spec results in complete omission of the required
behaviour."
  }
}

```

► Bug Validator result (完整)

```

{
  "id": "src--generate_batch_prompts-py--main",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/generate_batch_prompts.py:main",
  "function_name": "main",
  "confirmation_status": "not_confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--generate_batch_prompts-py--main.py",
  "detail_file": "fm_agent/bug_validation/src--generate_batch_prompts-py--main.md",

```

```
"probe_stdout": "NOT CONFIRMED - main() source extends to 138 lines (beyond the claimed 40-line cutoff) and contains all
required batch-generation patterns: build_prompt=True, output_dir.mkdir=True, manifest=True, dry_run=True, write_targets=True,
state=True, resume=True\n",
"trigger_summary": "Logic verification incorrectly truncated analysis at line 40; main() is 138 lines with full batch-generation
logic present."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/3	2026-07-28T03:29:35.658908Z	4096
2	check_post_implies_spec	mismatch	1	0/3	2026-07-28T03:30:18.503085Z	4175

事件 1 — [llm_1c66647475db438193b6da5cad612243](#)

► parsed_json（完整）

```
{
  "post_condition": "If args.batch_size <= 0, a ValueError with message \"--batch-size must be > 0\" is raised and no subsequent
statements execute. Otherwise, if after reading topdown_layers.json and parsing layers spec the condition start_layer < 0 or
end_layer >= total_layers holds, a ValueError with message \"layer range {args.layers} out of bounds [0, {total_layers - 1}]\" is
raised and no further assignments occur. Otherwise (normal termination): args is defined with all attributes reflecting the
command line and args.batch_size > 0; work_dir is a Path object resolving to the fm_agent/ working directory (parent of the
directory containing this script); repo_root is work_dir.parent; fm_agent_prefix is the string of the relative path from repo_root
to work_dir followed by '/'; phases_json is the parsed dict of phases.json and project = phases_json['project']; languages and
exts are list values from phases_json (defaulting to [] if missing); ext_to_lang is a dict mapping each extension from exts
(lowercased and with leading dot removed) to the corresponding language from languages; topdown_path is the path to the requested
phase's topdown_layers.json, topdown its parsed dict, layers = topdown.get('layers', []), and total_layers = len(layers);
start_layer and end_layer are the inclusive bounds from parsing args.layers and satisfy 0 <= start_layer <= end_layer <
total_layers; output_dir is a Path object, equal to Path(args.output_dir) if args.output_dir was truthy, else work_dir /
'spec_prompts' / f'batch_prompts_{project}_phase{args.phase:02d}'; func_to_layer maps each function name across all layers to its
layer index (after stripping fm_agent_prefix from the file path if present); all_funcs maps the same names to their function
metadata dicts (with the possibly modified file field); manifest_batches, total_functions, skipped_functions, batch_index,
write_targets are initialized respectively as an empty list, 0, 0, 0, and an empty list. No filesystem side effects occur beyond
the two JSON file reads."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1c66647475db438193b6da5cad612243_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_1c66647475db438193b6da5cad612243_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_1c66647475db438193b6da5cad612243_response.txt)

事件 2 — [llm_7166b8a0015d44948facac575d1aa501](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Command-line arguments: --phase 1 --layers 0-0 --batch-size 5 --output-dir not provided --resume false --dry-
run false. Filesystem state: work_dir/phases.json contains {\"project\": \"test\", \"languages\": [\"python\"],
\"file_extensions\": [\".py\"]} and work_dir/spec_prompts/phase_01_topdown_layers.json contains {\"layers\": [{\"layer\": 0,
\"functions\": [{\"name\": \"foo\", \"file\": \"src/foo.py\"}]}]. All other files exist as needed. The code will validate
batch_size > 0 and layer range [0,0] within bounds, complete initialisation up to line 40, then terminate without writing any
batch prompt files, manifest.json, or removing stale files, and without printing a dry-run plan.",
  "offending_statements": "Line 1 - Line 40: The code only parses arguments, validates them, reads JSON configs, and initialises
data structures. It never reaches the specification-required logic for generating batch prompts, writing output files, creating
manifest.json, removing stale files, or handling resume/dry-run.",
  "reason": "The code block ends after initialisation (line 40). For a valid input that passes all validation, the code does
nothing further, violating the specification which mandates file generation, manifest creation, stale-file cleanup, and dry-run
output. Any conforming input with batch_size > 0 and an in-range layer spec results in complete omission of the required
behaviour."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7166b8a0015d44948facac575d1aa501_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_7166b8a0015d44948facac575d1aa501_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_7166b8a0015d44948facac575d1aa501_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： `confirmed`; attempts `1`
- Phase / 模块： Phase 5 — Specification & Verification Utilities / `batch_prompts`; 原源码 `src/generate_batch_prompts.py`
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a Python dict that is the fully parsed representation of the JSON content in the file at path. Raises `FileNotFoundError` if no file exists at path. Raises an error if the file content is not syntactically valid JSON.
- **Actual behavior**: The function returns the Python value obtained by parsing the JSON content of the file at `path`. No exceptions are raised, the file remains unchanged, and the file is closed after reading. Formally, if `content = path.read_text()` before the call, then the return value `r` satisfies `r == json.loads(content)`.
- **Code evidence**: Line 4: `return json.loads(path.read_text())`
- **Trigger condition**: The specification requires the function to return a Python dict, but the code returns the raw output of `json.loads`, which can be any JSON value (list, string, number, boolean, null) if the file contains valid JSON not representing a JSON object. This violates the return type constraint.
- **Validator trigger**: Pass a file containing non-dict JSON (e.g. a JSON array) to `read_json`; `json.loads` returns the raw type (list) instead of dict, violating the return-type specification.
- **Probe stdout**: CONFIRMED — list file returned list: `[1, 2, 3]` | expected dict per spec

► SPEC sidecar (完整)

```
{
  "signature": "read_json(path) -> dict",
  "pre_condition": "path is a filesystem path to an existing file whose content is syntactically valid JSON.",
  "post_condition": "Returns a Python dict that is the fully parsed representation of the JSON content in the file at path. Raises
FileNotFoundError if no file exists at path. Raises an error if the file content is not syntactically valid JSON."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/generate_batch_prompts-py/read_json.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a Python dict that is the fully parsed representation of the JSON content in the file at path. Raises
FileNotFoundError if no file exists at path. Raises an error if the file content is not syntactically valid JSON.",
    "actual_behavior": "The function returns the Python value obtained by parsing the JSON content of the file at `path`. No
exceptions are raised, the file remains unchanged, and the file is closed after reading. Formally, if `content = path.read_text()`
before the call, then the return value `r` satisfies `r == json.loads(content)`.",
    "code_evidence": "Line 4: return json.loads(path.read_text())",
    "trigger_condition": "The specification requires the function to return a Python dict, but the code returns the raw output of
json.loads, which can be any JSON value (list, string, number, boolean, null) if the file contains valid JSON not representing a
JSON object. This violates the return type constraint."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--generate_batch_prompts-py--read_json",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/generate_batch_prompts-py/read_json.py",
  "function_name": "read_json",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--generate_batch_prompts-py--read_json.py",
  "detail_file": "fm_agent/bug_validation/src--generate_batch_prompts-py--read_json.md",
  "probe_stdout": "CONFIRMED — list file returned list: [1, 2, 3] | expected dict per spec",
  "trigger_summary": "Pass a file containing non-dict JSON (e.g. a JSON array) to read_json; json.loads returns the raw type"
```

```
(list) instead of dict, violating the return-type specification."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:36:16.061584Z	1829
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:36:43.322706Z	1621

事件 1 — [llm_2b17e736f7024493aabbd084358b9e4c](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns the Python value obtained by parsing the JSON content of the file at `path`. No exceptions are raised, the file remains unchanged, and the file is closed after reading. Formally, if `content = path.read_text()` before the call, then the return value `r` satisfies `r == json.loads(content)`."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_2b17e736f7024493aabbd084358b9e4c_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_2b17e736f7024493aabbd084358b9e4c_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_2b17e736f7024493aabbd084358b9e4c_response.txt)

事件 2 — [llm_2445d9fde0f34b78bd0722efd15d9ef6](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "A file containing a valid JSON array, e.g., '[1, 2, 3]'. The function returns a list, but the specification requires a dict.",
  "offending_statements": "Line 4: return json.loads(path.read_text())",
  "reason": "The specification requires the function to return a Python dict, but the code returns the raw output of json.loads, which can be any JSON value (list, string, number, boolean, null) if the file contains valid JSON not representing a JSON object. This violates the return type constraint."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_2445d9fde0f34b78bd0722efd15d9ef6_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_2445d9fde0f34b78bd0722efd15d9ef6_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_2445d9fde0f34b78bd0722efd15d9ef6_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: file_utils

FMA-MISMATCH-199 — [src--file_utils-py--collect_file_names](#)

- 四类结论: 契约不确定
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: not_confirmed; attempts 1
- Phase / 模块: Phase 5 — Specification & Verification Utilities / file_utils; 原源码 src/file_utils.py
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a sorted list of unique relative file paths for every file discovered under input_dir (recursive) whose filename does not match the metadata-sidecar naming pattern. A JSON array containing the same elements in the same sorted order is atomically written to output_path. The returned list and the JSON file contents are identical and sorted in ascending string order.
- **Actual behavior:** Upon normal termination, the function returns a sorted (ascending string order) list of unique relative file paths, and atomically writes the same list as a JSON array to output_path. The list contains exactly one entry for each file (regular file or symbolic link to a file) reachable by os.walk(input_dir) (with default arguments, followlinks=False) whose base name does not satisfy _is_metadata_sidecar. Duplicates cannot occur by construction because os.walk yields unique directory entries; nevertheless the

returned list is deduplicated. No other side effects occur. If an exception is raised during traversal or writing, the exception propagates, `output_path` remains unchanged from its prior state, and no value is returned.

Formally, let `entries = { (root, name) | (root, dirs, files) os.walk(input_dir), name files }` `is_sidecar(name)` `_is_metadata_sidecar(name) == True` `rel_path(e) = os.path.relpath(os.path.join(e.root, e.name), input_dir)` `S = { rel_path(e) | e entries is_sidecar(e.name) }` Then if the block completes successfully: (1) `result = sorted(S)` (ascending string order, duplicates already excluded) (2) `output_path` contains `json.dumps(result)` as an atomic write (3) the function returns `result`. If an exception occurs: (4) the exception is raised (5) `output_path` is unchanged (no partial write because `_write_file_names` writes atomically).

- **Code evidence:** Line 6-12: the loop iterates over `os.walk` entries but the resulting list (per Condition A) includes only regular files and symbolic links to files, thereby omitting other file types such as named pipes.
- **Trigger condition:** The specification requires every file discovered under `input_dir` to be included, which covers all non-directory entries (including special files like named pipes). The code's behavior as described restricts inclusion to regular files and symbolic links to files, missing those file types.
- **Validator trigger:** A directory containing a named pipe (FIFO) alongside a regular file — the code included the FIFO in results, contradicting the claim that only regular files and symlinks are returned.
- **Probe stdout:** Result: ['hello.txt', 'my_fifo'] Expected: ['hello.txt', 'my_fifo'] JSON file content: ['hello.txt', 'my_fifo'] NOT CONFIRMED — named pipe (FIFO) IS included in result; code does NOT omit special file types Result: ['hello.txt', 'my_fifo'] FIFO in JSON output: True

► SPEC sidecar (完整)

```
{
  "signature": "collect_file_names(input_dir, output_path='file_list.json') -> list[str]",
  "pre_condition": "input_dir is an existing directory on the filesystem. output_path is a writable file path string.",
  "post_condition": "Returns a sorted list of unique relative file paths for every file discovered under input_dir (recursive) whose filename does not match the metadata-sidecar naming pattern. A JSON array containing the same elements in the same sorted order is atomically written to output_path. The returned list and the JSON file contents are identical and sorted in ascending string order."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_is_metadata_sidecar",
      "signature": "_is_metadata_sidecar(fname) -> bool",
      "pre_condition": "fname is a filename string with no directory component.",
      "post_condition": "Returns True when fname matches the naming pattern of a specification or information sidecar file (e.g., .spec.json, .info.json); returns False for all other filenames."
    },
    {
      "name": "_write_file_names",
      "signature": "_write_file_names(file_names, output_path) -> list[str]",
      "pre_condition": "file_names is a list of file path strings. output_path is a writable file path.",
      "post_condition": "Returns a sorted list of unique strings derived from file_names, with duplicates removed. Atomically writes the same sorted, deduplicated list as a JSON array to output_path. The sort order is ascending string order."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-py/collect_file_names.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a sorted list of unique relative file paths for every file discovered under input_dir (recursive) whose filename does not match the metadata-sidecar naming pattern. A JSON array containing the same elements in the same sorted order is atomically written to output_path. The returned list and the JSON file contents are identical and sorted in ascending string order.",
    "actual_behavior": "Upon normal termination, the function returns a sorted (ascending string order) list of unique relative file paths, and atomically writes the same list as a JSON array to `output_path`. The list contains exactly one entry for each file (regular file or symbolic link to a file) reachable by `os.walk(input_dir)` (with default arguments, `followlinks=False`) whose base name does not satisfy `_is_metadata_sidecar`. Duplicates cannot occur by construction because `os.walk` yields unique directory entries; nevertheless the returned list is deduplicated. No other side effects occur. If an exception is raised during traversal or writing, the exception propagates, `output_path` remains unchanged from its prior state, and no value is returned.\n\nFormally, let\n  `entries = { (root, name) | (root, dirs, files) os.walk(input_dir), name files }`\n  `_is_sidecar(name)`\n  `_is_metadata_sidecar(name) == True`\n  `rel_path(e) = os.path.relpath(os.path.join(e.root, e.name), input_dir)`\n  `S = { rel_path(e) | e entries is_sidecar(e.name) }`\nThen if the block completes successfully:\n  (1) `result =`
```

```
sorted(S)` (ascending string order, duplicates already excluded)\n (2) `output_path` contains `json.dumps(result)` as an atomic
write\n (3) the function returns `result`. \nIf an exception occurs:\n (4) the exception is raised\n (5) `output_path` is
unchanged (no partial write because `_write_file_names` writes atomically).",
"code_evidence": "Line 6-12: the loop iterates over os.walk entries but the resulting list (per Condition A) includes only
regular files and symbolic links to files, thereby omitting other file types such as named pipes.",
"trigger_condition": "The specification requires every file discovered under input_dir to be included, which covers all non-
directory entries (including special files like named pipes). The code's behavior as described restricts inclusion to regular
files and symbolic links to files, missing those file types."
}
```

► Bug Validator result（完整）

```
{
  "id": "src--file_utils-py--collect_file_names",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-py/collect_file_names.py",
  "function_name": "collect_file_names",
  "confirmation_status": "not_confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--file_utils-py--collect_file_names.py",
  "detail_file": "fm_agent/bug_validation/src--file_utils-py--collect_file_names.md",
  "probe_stdout": "Result: ['hello.txt', 'my_fifo']\nExpected: ['hello.txt', 'my_fifo']\nJSON file content: ['hello.txt',
'my_fifo']\nNOT CONFIRMED - named pipe (FIFO) IS included in result; code does NOT omit special file types\n Result:
['hello.txt', 'my_fifo']\n FIFO in JSON output: True",
  "trigger_summary": "A directory containing a named pipe (FIFO) alongside a regular file - the code included the FIFO in results,
contradicting the claim that only regular files and symlinks are returned."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:29:21.637913Z	3426
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:30:20.216019Z	9018

事件 1 — [llm_22d5435a155e4038b793223e976a66f8](#)

► parsed_json（完整）

```
{
  "post_condition": "Upon normal termination, the function returns a sorted (ascending string order) list of unique relative file
paths, and atomically writes the same list as a JSON array to `output_path`. The list contains exactly one entry for each file
(regular file or symbolic link to a file) reachable by `os.walk(input_dir)` (with default arguments, `followlinks=False`) whose
base name does not satisfy `_is_metadata_sidecar`. Duplicates cannot occur by construction because `os.walk` yields unique
directory entries; nevertheless the returned list is deduplicated. No other side effects occur. If an exception is raised during
traversal or writing, the exception propagates, `output_path` remains unchanged from its prior state, and no value is
returned.\n\nFormally, let\n `entries = { (root, name) | (root, dirs, files) ∈ os.walk(input_dir), name ∈ files }`\n
`is_sidecar(name) ≡ _is_metadata_sidecar(name) == True`\n `rel_path(e) = os.path.relpath(os.path.join(e.root, e.name),
input_dir)`\n `S = { rel_path(e) | e ∈ entries ∧ ¬is_sidecar(e.name) }`\nThen if the block completes successfully:\n (1) `result
= sorted(S)` (ascending string order, duplicates already excluded)\n (2) `output_path` contains `json.dumps(result)` as an atomic
write\n (3) the function returns `result`. \nIf an exception occurs:\n (4) the exception is raised\n (5) `output_path` is
unchanged (no partial write because `_write_file_names` writes atomically)."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_22d5435a155e4038b793223e976a66f8_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_22d5435a155e4038b793223e976a66f8_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_22d5435a155e4038b793223e976a66f8_response.txt)

事件 2 — [llm_ac66215ec6df42dba63c073d74a67c40](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input directory contains a regular file 'a.txt' and a named pipe 'fifo'. The code returns ['a.txt'], but the
specification requires ['a.txt', 'fifo'].",
  "offending_statements": "Line 6-12: the loop iterates over os.walk entries but the resulting list (per Condition A) includes
only regular files and symbolic links to files, thereby omitting other file types such as named pipes.",
  "reason": "The specification requires every file discovered under input_dir to be included, which covers all non-directory
entries (including special files like named pipes). The code's behavior as described restricts inclusion to regular files and
```

```
symbolic links to files, missing those file types."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ac66215ec6df42dba63c073d74a67c40_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ac66215ec6df42dba63c073d74a67c40_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ac66215ec6df42dba63c073d74a67c40_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: parser

FMA-MISMATCH-202 — [src--parser-py--format_info_for_reasoner](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** [confirmed](#); attempts 1
- **Phase / 模块:** Phase 5 — Specification & Verification Utilities / [parser](#); 原源码 [src/parser.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a FunctionSpecMap containing one entry per callee in the input list. Each entry maps the callee's "name" value to a two-line string of the form "Pre-condition: <pre_condition>\nPost-condition: <post_condition>". The signatures attribute of the returned FunctionSpecMap is a dict mapping each callee's "name" value to its "signature" value. Returns an empty FunctionSpecMap (containing no entries and an empty signatures dict) when the input list is empty.
- **Actual behavior:** The function returns a newly created FunctionSpecMap object. Let callees = info['callees'] (a list of dicts, each with keys 'name', 'signature', 'pre_condition', 'post_condition'). For each d in callees, define name = d['name'], sig = d['signature'], spec_str = 'Pre-condition: ' + d['pre_condition'] + '\nPost-condition: ' + d['post_condition']. The resulting FunctionSpecMap stores: for each distinct name, the mappings name spec_str and name sig are taken from the last occurrence of that name in the iteration order of callees. If callees is empty, the map contains no entries. No exceptions are raised, and no external state is modified.
- **Code evidence:** Line 4: for callee in info.get("callees", []): Line 9: knowledge_map.add_entry(Line 10: callee.get("name", ""), Line 11: callee.get("signature", ""), Line 12: callee_spec, Line 13:)
- **Trigger condition:** The specification requires one entry per callee in the input list, but the code uses a mapping keyed by name and overwrites duplicates. For the counterexample with two callees both named 'foo', the resulting FunctionSpecMap has only one entry (the second callee's spec and signature) instead of preserving entries for each callee as required.
- **Validator trigger:** Two callees with the same name 'foo' cause the dict-based FunctionSpecMap to overwrite the first callee's entry, returning 1 entry instead of 2 as required by the spec.
- **Probe stdout:** CONFIRMED — actual entries: 1 (only last callee survives) | expected entries: 2 (one per callee) retained spec is from callee [#2](#): 'Pre-condition: x is not empty\nPost-condition: returns x.upper()'

► SPEC sidecar (完整)

```
{
  "signature": "format_info_for_reasoner(info) -> FunctionSpecMap",
  "pre_condition": "info is a dict containing a \"callees\" key whose value is a list of dicts, where each dict has exactly the string-typed keys \"name\", \"signature\", \"pre_condition\", and \"post_condition\"",
  "post_condition": "Returns a FunctionSpecMap containing one entry per callee in the input list. Each entry maps the callee's \"name\" value to a two-line string of the form \"Pre-condition: <pre_condition>\nPost-condition: <post_condition>\". The signatures attribute of the returned FunctionSpecMap is a dict mapping each callee's \"name\" value to its \"signature\" value. Returns an empty FunctionSpecMap (containing no entries and an empty signatures dict) when the input list is empty."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "FunctionSpecMap.add_entry",
      "signature": "add_entry(function_name, signature, spec) -> None",
      "pre_condition": "function_name and signature are strings; spec is a string containing formatted pre-condition and post-condition text",
      "post_condition": "Stores function_name as a key in the FunctionSpecMap with spec as its value, and maps function_name to signature in the signatures dict"
    }
  ]
}
```

```
}
]
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/parser-py/format_info_for_reasoner.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a FunctionSpecMap containing one entry per callee in the input list. Each entry maps the callee's\n\"name\" value to a two-line string of the form\n\"Pre-condition: <pre_condition>\nPost-condition: <post_condition>\". The\nsignatures attribute of the returned FunctionSpecMap is a dict mapping each callee's\n\"name\" value to its\n\"signature\" value.\nReturns an empty FunctionSpecMap (containing no entries and an empty signatures dict) when the input list is empty.",
    "actual_behavior": "The function returns a newly created FunctionSpecMap object. Let callees = info['callees'] (a list of\ndicts, each with keys 'name', 'signature', 'pre_condition', 'post_condition'). For each d in callees, define name = d['name'], sig\n= d['signature'], spec_str = 'Pre-condition: ' + d['pre_condition'] + '\nPost-condition: ' + d['post_condition']. The resulting\nFunctionSpecMap stores: for each distinct name, the mappings name spec_str and name sig are taken from the last occurrence of\nthat name in the iteration order of callees. If callees is empty, the map contains no entries. No exceptions are raised, and no\nexternal state is modified.",
    "code_evidence": "Line 4: for callee in info.get(\"callees\", []):\nLine 9: knowledge_map.add_entry(\nLine 10:\ncallee.get(\"name\", \"\"),\nLine 11:     callee.get(\"signature\", \"\"),\nLine 12:     callee_spec,\nLine 13: )",
    "trigger_condition": "The specification requires one entry per callee in the input list, but the code uses a mapping keyed by\nname and overwrites duplicates. For the counterexample with two callees both named 'foo', the resulting FunctionSpecMap has only\none entry (the second callee's spec and signature) instead of preserving entries for each callee as required."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--parser-py--format_info_for_reasoner",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/parser-py/format_info_for_reasoner.py",
  "function_name": "format_info_for_reasoner",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--parser-py--format_info_for_reasoner.py",
  "detail_file": "fm_agent/bug_validation/src--parser-py--format_info_for_reasoner.md",
  "probe_stdout": "CONFIRMED - actual entries: 1 (only last callee survives) | expected entries: 2 (one per callee)\n retained\nspec is from callee #2: 'Pre-condition: x is not empty\nPost-condition: returns x.upper()'",
  "trigger_summary": "Two callees with the same name 'foo' cause the dict-based FunctionSpecMap to overwrite the first callee's\nentry, returning 1 entry instead of 2 as required by the spec."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:35:54.023748Z	2032
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:36:17.776589Z	6839

事件 1 — [llm_adcb2f97416423cb2b4b455c4cfc82b](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a newly created FunctionSpecMap object. Let callees = info['callees'] (a list of dicts,\neach with keys 'name', 'signature', 'pre_condition', 'post_condition'). For each d in callees, define name = d['name'], sig =\nd['signature'], spec_str = 'Pre-condition: ' + d['pre_condition'] + '\nPost-condition: ' + d['post_condition']. The resulting\nFunctionSpecMap stores: for each distinct name, the mappings name spec_str and name sig are taken from the last occurrence of\nthat name in the iteration order of callees. If callees is empty, the map contains no entries. No exceptions are raised, and no\nexternal state is modified."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_adcb2f97416423cb2b4b455c4cfc82b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_adcb2f97416423cb2b4b455c4cfc82b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_adcb2f97416423cb2b4b455c4cfc82b_response.txt)

事件 2 — [llm_38c885f853444243b72f5aaf90db597c](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "info = {'callees': [{'name': 'foo', 'signature': '()', 'pre_condition': 'a', 'post_condition': 'b'}, {'name': 'foo', 'signature': '(int)', 'pre_condition': 'c', 'post_condition': 'd'}]}",
  "offending_statements": "Line 4: for callee in info.get(\"callees\", []):\nLine 9: knowledge_map.add_entry(\nLine 10: callee.get(\"name\", \"\"),\nLine 11:     callee.get(\"signature\", \"\"),\nLine 12:     callee_spec,\nLine 13: )",
  "reason": "The specification requires one entry per callee in the input list, but the code uses a mapping keyed by name and overwrites duplicates. For the counterexample with two callees both named 'foo', the resulting FunctionSpecMap has only one entry (the second callee's spec and signature) instead of preserving entries for each callee as required."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_38c885f853444243b72f5aaf90db597c_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_38c885f853444243b72f5aaf90db597c_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_38c885f853444243b72f5aaf90db597c_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-203 — [src--parser-py--format_spec_for_reasoner](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#); attempts [1](#)
- Phase / 模块： Phase 5 — Specification & Verification Utilities / [parser](#)； 原源码 [src/parser.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim:** Returns a string composed of four sections separated by blank lines: (1) the value of spec["signature"], (2) a line reading "Pre-condition:" followed by the value of spec["pre_condition"] on the next line, (3) a line reading "Post-condition:" followed by the value of spec["post_condition"] on the next line. The returned string always begins with the signature value and ends with the post_condition value. Every pair of adjacent sections is separated by exactly one blank line.
- Actual behavior:** The function returns a string composed as follows: newline-separated concatenation of the value of key "signature" from spec (or empty string if missing, though pre-condition guarantees its presence), followed by the literal "Pre-condition: ", the value of "pre_condition", then "

Post-condition: ", and finally the value of "post_condition". Formally: return_value = spec["signature"] + "

Recipients: " + "Pre-condition: " + spec["pre_condition"] + "

Post-condition: " + spec["post_condition"]. The function has no side effects, does not mutate spec, and does not raise exceptions.

- Code evidence:** Line 4: f'{spec.get('signature', "")}\n\n' Line 5: f'Pre-condition:\n{spec.get('pre_condition', '')}\n\n'
- Trigger condition:** When spec values end with a newline, the fixed '\n\n' separators produce two blank lines between sections instead of exactly one as required by the specification.
- Validator trigger:** When spec values end with a newline, fixed '\n\n' separators produce two blank lines between sections instead of exactly one as required by the specification.
- Probe stdout:** CONFIRMED — actual: 'def foo(x: int) -> int:\n\n\nPre-condition:\nx > 0\n\n\nPost-condition:\nReturns x + 1\n' | expected: 'def foo(x: int) -> int:\n\n\nPre-condition:\nx > 0\n\n\nPost-condition:\nReturns x + 1\n'

► [SPEC sidecar](#) (完整)

```
{
  "signature": "format_spec_for_reasoner(spec) -> str",
  "pre_condition": "spec is a dict with string-typed keys \"signature\", \"pre_condition\", and \"post_condition\"",
  "post_condition": "Returns a string composed of four sections separated by blank lines: (1) the value of spec[\"signature\"], (2) a line reading \"Pre-condition:\" followed by the value of spec[\"pre_condition\"] on the next line, (3) a line reading \"Post-condition:\" followed by the value of spec[\"post_condition\"] on the next line. The returned string always begins with the signature value and ends with the post_condition value. Every pair of adjacent sections is separated by exactly one blank line."
}
```

► [INFO / callees sidecar](#) (完整)

```
{
  "callees": []
}
```

```
}

```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/parser-py/format_spec_for_reasoner.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a string composed of four sections separated by blank lines: (1) the value of spec[\"signature\"], (2) a line reading \"Pre-condition:\" followed by the value of spec[\"pre_condition\"] on the next line, (3) a line reading \"Post-condition:\" followed by the value of spec[\"post_condition\"] on the next line. The returned string always begins with the signature value and ends with the post_condition value. Every pair of adjacent sections is separated by exactly one blank line.",
    "actual_behavior": "The function returns a string composed as follows: newline-separated concatenation of the value of key \"signature\" from spec (or empty string if missing, though pre-condition guarantees its presence), followed by the literal \"Pre-condition:\", the value of \"pre_condition\", then \"\\n\\nPost-condition:\", and finally the value of \"post_condition\". Formally: return_value = spec[\"signature\"] + \"\\n\\nRecipients: \" + \"Pre-condition:\" + spec[\"pre_condition\"] + \"\\n\\nPost-condition:\" + spec[\"post_condition\"]. The function has no side effects, does not mutate spec, and does not raise exceptions.",
    "code_evidence": "Line 4: f\"{spec.get('signature', '')}\\n\\n\"\\nLine 5: f\"Pre-condition:\\n{spec.get('pre_condition', '')}\\n\\n\\n\"",
    "trigger_condition": "When spec values end with a newline, the fixed '\\n\\n' separators produce two blank lines between sections instead of exactly one as required by the specification."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--parser-py--format_spec_for_reasoner",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/parser-py/format_spec_for_reasoner.py",
  "function_name": "format_spec_for_reasoner",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--parser-py--format_spec_for_reasoner.py",
  "detail_file": "fm_agent/bug_validation/src--parser-py--format_spec_for_reasoner.md",
  "probe_stdout": "CONFIRMED - actual: 'def foo(x: int) -> int:\\n\\n\\nPre-condition:\\nx > 0\\n\\n\\nPost-condition:\\nReturns x + 1\\n' | expected: 'def foo(x: int) -> int:\\n\\n\\nPre-condition:\\nx > 0\\n\\n\\nPost-condition:\\nReturns x + 1\\n'",
  "trigger_summary": "When spec values end with a newline, fixed '\\n\\n' separators produce two blank lines between sections instead of exactly one as required by the specification."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:35:58.032218Z	993
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:36:09.537123Z	3432

事件 1 — [llm_ffaef303bc904e9ea6ca83f2fa453fae](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string composed as follows: newline-separated concatenation of the value of key \"signature\" from spec (or empty string if missing, though pre-condition guarantees its presence), followed by the literal \"Pre-condition:\", the value of \"pre_condition\", then \"\\n\\nPost-condition:\", and finally the value of \"post_condition\". Formally: return_value = spec[\"signature\"] + \"\\n\\nRecipients: \" + \"Pre-condition:\" + spec[\"pre_condition\"] + \"\\n\\nPost-condition:\" + spec[\"post_condition\"]. The function has no side effects, does not mutate spec, and does not raise exceptions."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ffaef303bc904e9ea6ca83f2fa453fae_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ffaef303bc904e9ea6ca83f2fa453fae_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ffaef303bc904e9ea6ca83f2fa453fae_response.txt)

事件 2 — [llm_4aa0097ea1ae47ba8ad6bda6454c30b5](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "spec = {\n\"signature\": \"hello\\n\\n\", \"pre_condition\": \"x\\n\\n\", \"post_condition\": \"y\\n\\n\"},\n\"offending_statements\": \"Line 4: f\\n{spec.get('signature', '')}\\n\\n\\n\\nLine 5: f\\n\"Pre-condition:\\n{spec.get('pre_condition',\n''}\\n\\n\\n\\n\", \"reason\": \"When spec values end with a newline, the fixed '\\n\\n' separators produce two blank lines between sections instead of exactly one as required by the specification.\"}\n}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4aa0097ea1ae47ba8ad6bda6454c30b5_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4aa0097ea1ae47ba8ad6bda6454c30b5_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4aa0097ea1ae47ba8ad6bda6454c30b5_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

Phase 6 — Reasoning & Verification Engine

模块: [entry_pipeline](#)

FMA-MISMATCH-206 — [src--entry_reasoning_pipeline-py--_fqn_to_ident](#)

- 四类结论: [契约不确定](#)
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 6 — Reasoning & Verification Engine / [entry_pipeline](#); 原源码 [src/entry_reasoning_pipeline.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns the class-qualified identifier portion of fqn: the tail after the source-file component (the component whose rightmost hyphen-separated token is a known language file extension). If no source-file component is found, returns the last '::-'-delimited component of fqn.
- **Actual behavior:** Let parts = fqn.split('::-') and n = len(parts). The function returns a string result determined as follows: the code searches from i = n-1 down to 0 to find the first (i.e., rightmost) component where parts[i].rfind('-') returns a position h > 0 and the substring parts[i][h+1:] is a key in the global dictionary EXT_TO_LANG. If such an index i0 is found, the function immediately returns '::-'.join(parts[i0+1:]). Otherwise (if no component satisfies the condition), it returns parts[-1] (the last component). If n = 1, the loop over indices is empty and result equals parts[0].
- **Code evidence:** Line 14: if hyphen > 0 and comp[hyphen + 1:] in EXT_TO_LANG:
- **Trigger condition:** The code's condition requires hyphen > 0, so it rejects components like '-cpp' that start with a hyphen even though the spec treats the rightmost hyphen-separated token 'cpp' as a valid extension. For input 'src::-cpp::foo::bar', the spec expects 'foo::bar' (tail after the source-file component '-cpp'), but the code returns 'bar' because it does not recognize '-cpp' as a source-file component.
- **Validator trigger:** FQN with a source-file component starting with a hyphen (e.g. '-cpp') is not recognized because the condition hyphen > 0 excludes position 0.
- **Probe stdout:** CONFIRMED — actual: 'bar' | expected: 'foo::bar'

► [SPEC sidecar \(完整\)](#)

```
{
  "signature": "_fqn_to_ident(fqn) -> str",
  "pre_condition": "fqn is a non-empty string representing a fully-qualified function name whose components are separated by '::-'.",
  "post_condition": "Returns the class-qualified identifier portion of fqn: the tail after the source-file component (the component whose rightmost hyphen-separated token is a known language file extension). If no source-file component is found, returns the last '::-'-delimited component of fqn."
}
```

► [INFO / callees sidecar \(完整\)](#)

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/entry_reasoning_pipeline-py/_fqn_to_ident.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns the class-qualified identifier portion of fqn: the tail after the source-file component (the component whose rightmost hyphen-separated token is a known language file extension). If no source-file component is found, returns the last ':'-delimited component of fqn.",
    "actual_behavior": "Let parts = fqn.split(':') and n = len(parts). The function returns a string result determined as follows: the code searches from i = n-1 down to 0 to find the first (i.e., rightmost) component where parts[i].rfind('-') returns a position h > 0 and the substring parts[i][h+1:] is a key in the global dictionary EXT_TO_LANG. If such an index i0 is found, the function immediately returns ':'.join(parts[i0+1:]). Otherwise (if no component satisfies the condition), it returns parts[-1] (the last component). If n = 1, the loop over indices is empty and result equals parts[0].",
    "code_evidence": "Line 14: if hyphen > 0 and comp[hyphen + 1:] in EXT_TO_LANG:",
    "trigger_condition": "The code's condition requires hyphen > 0, so it rejects components like '-cpp' that start with a hyphen even though the spec treats the rightmost hyphen-separated token 'cpp' as a valid extension. For input 'src::-cpp::foo::bar', the spec expects 'foo::bar' (tail after the source-file component '-cpp'), but the code returns 'bar' because it does not recognize '-cpp' as a source-file component."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--entry_reasoning_pipeline-py--_fqn_to_ident",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/entry_reasoning_pipeline-py/_fqn_to_ident.py",
  "function_name": "_fqn_to_ident",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--entry_reasoning_pipeline-py--_fqn_to_ident.py",
  "detail_file": "fm_agent/bug_validation/src--entry_reasoning_pipeline-py--_fqn_to_ident.md",
  "probe_stdout": "CONFIRMED - actual: 'bar' | expected: 'foo::bar'",
  "trigger_summary": "FQN with a source-file component starting with a hyphen (e.g. '-cpp') is not recognized because the condition hyphen > 0 excludes position 0."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:43:15.159691Z	3261
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:44:02.709164Z	2877

事件 1 — [llm_bf42a06f3178448596f044dc3e2ab147](#)

► parsed_json（完整）

```
{
  "post_condition": "Let parts = fqn.split(':') and n = len(parts). The function returns a string result determined as follows: the code searches from i = n-1 down to 0 to find the first (i.e., rightmost) component where parts[i].rfind('-') returns a position h > 0 and the substring parts[i][h+1:] is a key in the global dictionary EXT_TO_LANG. If such an index i0 is found, the function immediately returns ':'.join(parts[i0+1:]). Otherwise (if no component satisfies the condition), it returns parts[-1] (the last component). If n = 1, the loop over indices is empty and result equals parts[0]."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_bf42a06f3178448596f044dc3e2ab147_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_bf42a06f3178448596f044dc3e2ab147_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_bf42a06f3178448596f044dc3e2ab147_response.txt)

事件 2 — [llm_048713f1cdd6430293b3802360b0f163](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "src::-cpp::foo::bar",
  "offending_statements": "Line 14: if hyphen > 0 and comp[hyphen + 1:] in EXT_TO_LANG:",
```

```
"reason": "The code's condition requires hyphen > 0, so it rejects components like '-cpp' that start with a hyphen even though the spec treats the rightmost hyphen-separated token 'cpp' as a valid extension. For input 'src::-cpp::foo::bar', the spec expects 'foo::bar' (tail after the source-file component '-cpp'), but the code returns 'bar' because it does not recognize '-cpp' as a source-file component."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_048713f1cdd6430293b3802360b0f163_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_048713f1cdd6430293b3802360b0f163_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_048713f1cdd6430293b3802360b0f163_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-211 — [src--entry_reasoning_pipeline-py--_trim_project_in_place](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [entry_pipeline](#); 原源码 [src/entry_reasoning_pipeline.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: For every source file path key in all_by_source: if keep_by_source contains a non-empty set of function identifiers for that path, the file under proj_dir has been modified to contain only the function bodies corresponding to identifiers in that set plus all non-function lines; all functions whose identifiers appear in all_by_source but not in keep_by_source for that file have been removed from the file. If keep_by_source does not contain the path or contains an empty set for it, the file at that path under proj_dir has been deleted. Files whose paths do not appear as keys in all_by_source are left unchanged. Returns None.
- **Actual behavior**: After execution, proj_dir's file system is updated: for each source_rel in sorted(all_by_source), let src_path = join(proj_dir, source_rel). If src_path was a regular file and keep_by_source contains a non-empty set for source_rel, then src_path remains as a regular file with modified content: only function definitions whose names are in keep_by_source[source_rel] are preserved, together with all non-function context lines (e.g., imports, comments, module-level code), maintaining original order; all other function definitions from all_by_source[source_rel] are removed. If src_path was a regular file and keep_by_source lacks source_rel or has an empty set, then src_path is deleted. If src_path was not a regular file, it is unchanged. Any file in proj_dir whose path is not a key in all_by_source is left untouched. total_kept and total_removed sum the kept and removed function counts from _trim_source_file calls, deleted_files counts the deleted files, and a summary message is printed.
- **Code evidence**: Line 12: if not os.path.isfile(src_path): Line 13: continue
- **Trigger condition**: The code only acts on entries in all_by_source if their path is a regular file (isfile). The specification unconditionally requires deletion when keep_by_source has no entry or an empty set for that key, and modification when it has a non-empty set. A dangling symlink is a valid file path but not a regular file, leading to a violation.
- **Validator trigger**: A dangling symlink listed in all_by_source is silently skipped instead of being deleted or modified because os.path.isfile() returns False for non-regular files.
- **Probe stdout**: [EntryPipeline] Trimmed /tmp/probe_trim_00xeou0x: kept 0 function(s), removed 0 function(s), deleted 1 source file(s). CONFIRMED — real.py was correctly deleted, but dangling symlink 'dangling.py' was NOT deleted. spec requires deletion for all all_by_source entries when keep_by_source lacks them; os.path.isfile() returned False for the dangling symlink and the function incorrectly continued past it.

► SPEC sidecar (完整)

```
{
  "signature": "_trim_project_in_place(proj_dir, all_by_source, keep_by_source) -> None",
  "pre_condition": "proj_dir is an existing directory containing a project copy. all_by_source is a dict mapping source file paths relative to proj_dir to sets of all function identifiers from those files. keep_by_source is a dict mapping source file paths relative to proj_dir to sets of function identifiers to preserve, where every key in keep_by_source is also a key in all_by_source.",
  "post_condition": "For every source file path key in all_by_source: if keep_by_source contains a non-empty set of function identifiers for that path, the file under proj_dir has been modified to contain only the function bodies corresponding to identifiers in that set plus all non-function lines; all functions whose identifiers appear in all_by_source but not in keep_by_source for that file have been removed from the file. If keep_by_source does not contain the path or contains an empty set for it, the file at that path under proj_dir has been deleted. Files whose paths do not appear as keys in all_by_source are left unchanged. Returns None."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/entry_reasoning_pipeline-
py/_trim_project_in_place.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "For every source file path key in all_by_source: if keep_by_source contains a non-empty set of function
identifiers for that path, the file under proj_dir has been modified to contain only the function bodies corresponding to
identifiers in that set plus all non-function lines; all functions whose identifiers appear in all_by_source but not in
keep_by_source for that file have been removed from the file. If keep_by_source does not contain the path or contains an empty set
for it, the file at that path under proj_dir has been deleted. Files whose paths do not appear as keys in all_by_source are left
unchanged. Returns None.",
    "actual_behavior": "After execution, proj_dir's file system is updated: for each source_rel in sorted(all_by_source), let
src_path = join(proj_dir, source_rel). If src_path was a regular file and keep_by_source contains a non-empty set for source_rel,
then src_path remains as a regular file with modified content: only function definitions whose names are in
keep_by_source[source_rel] are preserved, together with all non-function context lines (e.g., imports, comments, module-level
code), maintaining original order; all other function definitions from all_by_source[source_rel] are removed. If src_path was a
regular file and keep_by_source lacks source_rel or has an empty set, then src_path is deleted. If src_path was not a regular
file, it is unchanged. Any file in proj_dir whose path is not a key in all_by_source is left untouched. total_kept and
total_removed sum the kept and removed function counts from _trim_source_file calls, deleted_files counts the deleted files, and a
summary message is printed.",
    "code_evidence": "Line 12: if not os.path.isfile(src_path):\nLine 13:             continue",
    "trigger_condition": "The code only acts on entries in all_by_source if their path is a regular file (isfile). The
specification unconditionally requires deletion when keep_by_source has no entry or an empty set for that key, and modification
when it has a non-empty set. A dangling symlink is a valid file path but not a regular file, leading to a violation."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--entry_reasoning_pipeline-py--_trim_project_in_place",
  "source_file": "src/entry_reasoning_pipeline-py/_trim_project_in_place.py",
  "function_name": "_trim_project_in_place",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--entry_reasoning_pipeline-py--_trim_project_in_place.py",
  "detail_file": "fm_agent/bug_validation/src--entry_reasoning_pipeline-py--_trim_project_in_place.md",
  "probe_stdout": "[EntryPipeline] Trimmed /tmp/probe_trim_00xeou0x: kept 0 function(s), removed 0 function(s), deleted 1 source
file(s).\n\nCONFIRMED – real.py was correctly deleted, but dangling symlink 'dangling.py' was NOT deleted. spec requires deletion
for all all_by_source entries when keep_by_source lacks them; os.path.isfile() returned False for the dangling symlink and the
function incorrectly continued past it.",
  "trigger_summary": "A dangling symlink listed in all_by_source is silently skipped instead of being deleted or modified because
os.path.isfile() returns False for non-regular files."
}
```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-28T04:23:04.941213Z	4062
2	generate_block_post_condition	success	2	0/1	2026-07-28T04:24:06.386250Z	2285
3	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:24:24.056671Z	5205

事件 1 — [llm_91687dd0d2044658987a22eaa47775e5](#)

- parse_error: LLM response is not valid JSON: Invalid \\escape: line 2 column 2152 (char 2153)
- 载荷: [system_prompt](#) · [user_prompt](#) · [assistant_output](#)

事件 2 — [llm_89e70f629a134229a0400431380a2738](#)

► parsed_json（完整）


```
{
  "post_condition": "After execution, proj_dir's file system is updated: for each source_rel in sorted(all_by_source), let
src_path = join(proj_dir, source_rel). If src_path was a regular file and keep_by_source contains a non-empty set for source_rel,
then src_path remains as a regular file with modified content: only function definitions whose names are in
keep_by_source[source_rel] are preserved, together with all non-function context lines (e.g., imports, comments, module-level
code), maintaining original order; all other function definitions from all_by_source[source_rel] are removed. If src_path was a
regular file and keep_by_source lacks source_rel or has an empty set, then src_path is deleted. If src_path was not a regular
file, it is unchanged. Any file in proj_dir whose path is not a key in all_by_source is left untouched. total_kept and
total_removed sum the kept and removed function counts from _trim_source_file calls, deleted_files counts the deleted files, and a
summary message is printed."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_89e70f629a134229a0400431380a2738_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_89e70f629a134229a0400431380a2738_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_89e70f629a134229a0400431380a2738_message_02_assistant.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_89e70f629a134229a0400431380a2738_message_03_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_89e70f629a134229a0400431380a2738_response.txt)

事件 3 — **llm_9cb4fc1a756040a38df44b811f41589c**

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir contains a dangling symbolic link named 'link' (e.g., created by os.symlink('/nonexistent',
proj_dir+'/link')). all_by_source = {'link': ['func1']}, keep_by_source = {} (or keep_by_source={'link': set()}). The
specification requires the file 'link' to be deleted, but the code skips it because os.path.isfile() returns False for a broken
symlink, leaving it untouched.",
  "offending_statements": "Line 12: if not os.path.isfile(src_path):\nLine 13:             continue",
  "reason": "The code only acts on entries in all_by_source if their path is a regular file (isfile). The specification
unconditionally requires deletion when keep_by_source has no entry or an empty set for that key, and modification when it has a
non-empty set. A dangling symlink is a valid file path but not a regular file, leading to a violation."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_9cb4fc1a756040a38df44b811f41589c_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_9cb4fc1a756040a38df44b811f41589c_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_9cb4fc1a756040a38df44b811f41589c_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: incremental_pipeline

FMA-MISMATCH-214 — **src--incremental_reasoner-py--_StdoutTee::flush**

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** **confirmed**; attempts 1
- **Phase / 模块:** Phase 6 — Reasoning & Verification Engine / **incremental_pipeline**; 源代码 **src/incremental_reasoner.py**
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** All buffered data written to the console stream is flushed to the underlying device. All buffered data written to the log stream is flushed to the underlying file, unless the log stream is closed, in which case no flush is attempted and no exception is raised. Returns None.
- **Actual behavior:** If the method returns normally (no exception raised), then all buffered data in the console stream has been committed to the underlying device, and if the log stream was not closed at the time of the call, all buffered data in the log stream has been committed to the underlying file. If an exception is raised during the execution of either flush() call, the method does not complete normally, and the state of the stream buffers is unspecified: the console flush may have partially succeeded or not started, and the log flush may not have been called. Formal logic: For normal termination: (flush_call_console(succeed) (log_stream_not_closed_at_call_time flush_call_log(succeed))). For abnormal termination: raised(exception) {FlushError, IOError, ...}.
- **Code evidence:** Line 2: self._console.flush()
- **Trigger condition:** The specification requires that after the call, all buffered data in the log stream is flushed (unless the log stream is closed). The code does not flush the log stream when the console flush raises an exception, violating that post-condition.

- **Validator trigger:** When `self._console.flush()` raises an exception, `self._log_stream.flush()` is never called, violating the spec's post-condition that the log stream must be flushed unless closed.
- **Probe stdout:** CONFIRMED — exception propagated from console flush, but log stream flush was never called (spec requires it)

► SPEC sidecar (完整)

```
{
  "signature": "_StdoutTee.flush(self)",
  "pre_condition": "self._console and self._log_stream are file-like objects with a flush() method.",
  "post_condition": "All buffered data written to the console stream is flushed to the underlying device. All buffered data
written to the log stream is flushed to the underlying file, unless the log stream is closed, in which case no flush is attempted
and no exception is raised. Returns None."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "self._console.flush",
      "signature": "flush()",
      "pre_condition": "The console stream is open and may contain buffered data written by prior write calls.",
      "post_condition": "All buffered data in the console stream is committed to the underlying device."
    },
    {
      "name": "self._log_stream.flush",
      "signature": "flush()",
      "pre_condition": "The log stream is open (not closed) and may contain buffered data written by prior write calls.",
      "post_condition": "All buffered data in the log stream is committed to the underlying file."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_StdoutTee::flush.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "All buffered data written to the console stream is flushed to the underlying device. All buffered data written
to the log stream is flushed to the underlying file, unless the log stream is closed, in which case no flush is attempted and no
exception is raised. Returns None.",
    "actual_behavior": "If the method returns normally (no exception raised), then all buffered data in the console stream has
been committed to the underlying device, and if the log stream was not closed at the time of the call, all buffered data in the
log stream has been committed to the underlying file. If an exception is raised during the execution of either flush() call, the
method does not complete normally, and the state of the stream buffers is unspecified: the console flush may have partially
succeeded or not started, and the log flush may not have been called. Formal logic: For normal termination:
(flush_call_console(succeed) ( log_stream_not_closed_at_call_time flush_call_log(succeed) ) ). For abnormal termination:
raised(exception) {FlushError, IOError, ...}.",
    "code_evidence": "Line 2: self._console.flush()",
    "trigger_condition": "The specification requires that after the call, all buffered data in the log stream is flushed (unless
the log stream is closed). The code does not flush the log stream when the console flush raises an exception, violating that post-
condition."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--incremental_reasoner-py--_StdoutTee::flush",
  "source_file": "src/incremental_reasoner.py",
  "function_name": "_StdoutTee::flush",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_StdoutTee::flush.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_StdoutTee::flush.md",
  "probe_stdout": "CONFIRMED – exception propagated from console flush, but log stream flush was never called (spec requires it)",
  "trigger_summary": "When self._console.flush() raises an exception, self._log_stream.flush() is never called, violating the
spec's post-condition that the log stream must be flushed unless closed."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:58:07.400328Z	1400
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:58:24.923802Z	3532

事件 1 — [llm_0c32bf510ad6497ab09feb1cd09b1cbc](#)

► [parsed_json](#)（完整）

```
{
  "post_condition": "If the method returns normally (no exception raised), then all buffered data in the console stream has been committed to the underlying device, and if the log stream was not closed at the time of the call, all buffered data in the log stream has been committed to the underlying file. If an exception is raised during the execution of either flush() call, the method does not complete normally, and the state of the stream buffers is unspecified: the console flush may have partially succeeded or not started, and the log flush may not have been called. Formal logic: For normal termination: (flush_call_console(succeed) ^ ( log_stream_not_closed_at_call_time -> flush_call_log(succeed) ) ). For abnormal termination: raised(exception) ∈ {FlushError, IOError, ...}."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_0c32bf510ad6497ab09feb1cd09b1cbc_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_0c32bf510ad6497ab09feb1cd09b1cbc_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_0c32bf510ad6497ab09feb1cd09b1cbc_response.txt\)](#)

事件 2 — [llm_fceab38e52ae4159bd58ce08bbb538b0](#)

► [parsed_json](#)（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "An object where self._console.flush() raises an exception (e.g., IOError due to a broken pipe) and self._log_stream is open (not closed) and contains buffered data. Calling flush() on this object causes self._console.flush() to raise, preventing the execution of self._log_stream.flush(). The log stream data is never flushed.",
  "offending_statements": "Line 2: self._console.flush()",
  "reason": "The specification requires that after the call, all buffered data in the log stream is flushed (unless the log stream is closed). The code does not flush the log stream when the console flush raises an exception, violating that post-condition."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_fceab38e52ae4159bd58ce08bbb538b0_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_fceab38e52ae4159bd58ce08bbb538b0_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_fceab38e52ae4159bd58ce08bbb538b0_response.txt\)](#) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-215 — [src--incremental_reasoner-py--_StdoutTee::write](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#); 原源码 [src/incremental_reasoner.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: The string data is written to the console stream. The string data is written to the log stream unless the log stream is closed, in which case the log write is skipped and no exception is raised. Returns len(data), the number of characters in data.
- **Actual behavior**: If the method returns normally, then data has been successfully written to self._console, and if self._log_stream was not closed before the write attempt, data has also been written to self._log_stream; the return value equals len(data). Formally: s State such that pre(s) and method terminates normally with value r, r = len(data) written(s.self._console, data) (s.self._log_stream.closed written(s.self._log_stream, data)). If the method raises an exception (from either write call), the console and log stream may contain partial or no data, and no return value is produced.
- **Code evidence**: Line 2: self._console.write(data)
- **Trigger condition**: The specification requires that data is written to the console stream. The precondition states only that the console stream is open, not that it is open for writing. A valid input can provide an open console stream in read mode, causing

- self._console.write(data) to raise an exception. The code then does not write data to the console and does not return len(data), violating the specification.
- **Validator trigger:** Providing an open console stream in read mode causes self._console.write(data) to raise io.UnsupportedOperation instead of returning len(data).
 - **Probe stdout:** CONFIRMED — actual: 'UnsupportedOperation: not writable' | expected: 5

► SPEC sidecar (完整)

```
{
  "signature": "_StdoutTee.write(self, data) -> int",
  "pre_condition": "self._console and self._log_stream are file-like objects with a write(str) method. data is a str.",
  "post_condition": "The string data is written to the console stream. The string data is written to the log stream unless the log stream is closed, in which case the log write is skipped and no exception is raised. Returns len(data), the number of characters in data."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "self._console.write",
      "signature": "write(data: str) -> int",
      "pre_condition": "The console stream is open. data is a str.",
      "post_condition": "The string data is written to the console stream. Returns the number of characters written."
    },
    {
      "name": "self._log_stream.write",
      "signature": "write(data: str) -> int",
      "pre_condition": "The log stream is open (not closed). data is a str.",
      "post_condition": "The string data is written to the log stream. Returns the number of characters written."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_StdoutTee::write.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "The string data is written to the console stream. The string data is written to the log stream unless the log stream is closed, in which case the log write is skipped and no exception is raised. Returns len(data), the number of characters in data.",
    "actual_behavior": "If the method returns normally, then data has been successfully written to self._console, and if self._log_stream was not closed before the write attempt, data has also been written to self._log_stream; the return value equals len(data). Formally: s State such that pre(s) and method terminates normally with value r, r = len(data) written(s.self._console, data) (s.self._log_stream.closed written(s.self._log_stream, data)). If the method raises an exception (from either write call), the console and log stream may contain partial or no data, and no return value is produced.",
    "code_evidence": "Line 2: self._console.write(data)",
    "trigger_condition": "The specification requires that data is written to the console stream. The precondition states only that the console stream is open, not that it is open for writing. A valid input can provide an open console stream in read mode, causing self._console.write(data) to raise an exception. The code then does not write data to the console and does not return len(data), violating the specification."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--incremental_reasoner-py--_StdoutTee::write",
  "source_file": "src/incremental_reasoner.py",
  "function_name": "_StdoutTee::write",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_StdoutTee::write.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_StdoutTee::write.md",
  "probe_stdout": "CONFIRMED — actual: 'UnsupportedOperation: not writable' | expected: 5",
  "trigger_summary": "Providing an open console stream in read mode causes self._console.write(data) to raise io.UnsupportedOperation instead of returning len(data)."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:58:11.409653Z	1855
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:58:36.264061Z	14311

事件 1 — [llm_ea6f609fc1a94968aee3de48425f7452](#)

► [parsed_json](#)（完整）

```
{
  "post_condition": "If the method returns normally, then data has been successfully written to self._console, and if self._log_stream was not closed before the write attempt, data has also been written to self._log_stream; the return value equals len(data). Formally:  $\forall s \in \text{State such that pre}(s) \wedge \text{method terminates normally with value } r, r = \text{len}(data) \wedge \text{written}(s.\text{self}.\text{\_console}, data) \wedge (\neg s.\text{self}.\text{\_log\_stream}.\text{closed} \Rightarrow \text{written}(s.\text{self}.\text{\_log\_stream}, data))$ . If the method raises an exception (from either write call), the console and log stream may contain partial or no data, and no return value is produced."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ea6f609fc1a94968aee3de48425f7452_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ea6f609fc1a94968aee3de48425f7452_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ea6f609fc1a94968aee3de48425f7452_response.txt)

事件 2 — [llm_6ae73874c68540cd811121061d382623](#)

► [parsed_json](#)（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "data = 'hello', self._console = open('readonly.txt', 'r'), self._log_stream = open('log.txt', 'w')",
  "offending_statements": "Line 2: self._console.write(data)",
  "reason": "The specification requires that data is written to the console stream. The pre-condition states only that the console stream is open, not that it is open for writing. A valid input can provide an open console stream in read mode, causing self._console.write(data) to raise an exception. The code then does not write data to the console and does not return len(data), violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_6ae73874c68540cd811121061d382623_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_6ae73874c68540cd811121061d382623_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_6ae73874c68540cd811121061d382623_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-216 — [src--incremental_reasoner-py--_bare_function_name](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts 1
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#); 原源码 [src/incremental_reasoner.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

SPEC claim: Returns the rightmost segment of identifier after splitting on any occurrence of ':' or '.', with any suffix matching the pattern _ (underscore followed by one or more decimal digits at the end of that segment) removed. When identifier contains no ':' or '.' separators, the entire identifier is returned with any trailing _ suffix stripped. The returned string is non-empty (empty-return is possible only when the input contains no characters beyond the stripped suffix).

- Actual behavior:** The function returns a string formed by first splitting [identifier](#) with the regular expression `::|\.` and taking the last element, then removing any trailing substring that matches the pattern `_\d+$` (an underscore followed by one or more digits). Formally, if `parts = re.split(r'::|\.', identifier)`, then the returned value equals `re.sub(r'_\d+$', '', parts[-1])`.
- Code evidence:** Line 3
- Trigger condition:** The function returns an empty string for ':' because `re.split` yields `[ "", "" ]` and the last element is empty. This violates the specification, which states the returned string must be non-empty; empty is only permissible when the input contains no characters beyond the stripped suffix, but ':' includes non-stripped characters.

- Validator trigger:** Input identifier ':' causes re.split to yield ['', ''] and the last empty element passes through suffix stripping unchanged, returning '' which violates the spec's non-empty guarantee for inputs containing non-stripped characters.
- Probe stdout:** CONFIRMED — actual: '' | Function returns empty string for ':' which contains non-stripped characters, violating the spec claim that empty-return is only possible when the input has no characters beyond the stripped suffix
- SPEC sidecar (完整)

```
{
  "signature": "_bare_function_name(identifier) -> str",
  "pre_condition": "identifier is a non-empty string.",
  "post_condition": "Returns the rightmost segment of identifier after splitting on any occurrence of ':' or '.', with any suffix matching the pattern _<digits> (underscore followed by one or more decimal digits at the end of that segment) removed. When identifier contains no ':' or '.' separators, the entire identifier is returned with any trailing _<digits> suffix stripped. The returned string is non-empty (empty-return is possible only when the input contains no characters beyond the stripped suffix)."
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_bare_function_name.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns the rightmost segment of identifier after splitting on any occurrence of ':' or '.', with any suffix matching the pattern _<digits> (underscore followed by one or more decimal digits at the end of that segment) removed. When identifier contains no ':' or '.' separators, the entire identifier is returned with any trailing _<digits> suffix stripped. The returned string is non-empty (empty-return is possible only when the input contains no characters beyond the stripped suffix).",
    "actual_behavior": "The function returns a string formed by first splitting `identifier` with the regular expression `::|\\.` and taking the last element, then removing any trailing substring that matches the pattern `_\\d+$` (an underscore followed by one or more digits). Formally, if `parts = re.split(r '::|\\. ', identifier)`, then the returned value equals `re.sub(r '_\\d+$', '', parts[-1])`.",
    "code_evidence": "Line 3",
    "trigger_condition": "The function returns an empty string for ':' because re.split yields ['', ''] and the last element is empty. This violates the specification, which states the returned string must be non-empty; empty is only permissible when the input contains no characters beyond the stripped suffix, but ':' includes non-stripped characters."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--incremental_reasoner-py--_bare_function_name",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_bare_function_name.py",
  "function_name": "_bare_function_name",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_bare_function_name.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_bare_function_name.md",
  "probe_stdout": "CONFIRMED — actual: '' | Function returns empty string for ':' which contains non-stripped characters, violating the spec claim that empty-return is only possible when the input has no characters beyond the stripped suffix",
  "trigger_summary": "Input identifier ':' causes re.split to yield ['', ''] and the last empty element passes through suffix stripping unchanged, returning '' which violates the spec's non-empty guarantee for inputs containing non-stripped characters."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T05:13:36.556383Z	1438
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T05:13:56.639194Z	5275

► parsed_json (完整)

```
{
  "post_condition": "The function returns a string formed by first splitting `identifier` with the regular expression `::|\\.` and taking the last element, then removing any trailing substring that matches the pattern `_\\d+$` (an underscore followed by one or more digits). Formally, if `parts = re.split(r'::|\\.', identifier)`, then the returned value equals `re.sub(r'_\\d+$', '', parts[-1])`."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b24eb7801c9842f68531d75ba55256db_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_b24eb7801c9842f68531d75ba55256db_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_b24eb7801c9842f68531d75ba55256db_response.txt)

事件 2 — llm_fe377dacaabb4933a5335e8f32dfa131

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "::-",
  "offending_statements": "Line 3",
  "reason": "The function returns an empty string for ':' because re.split yields [ '', '' ] and the last element is empty. This violates the specification, which states the returned string must be non-empty; empty is only permissible when the input contains no characters beyond the stripped suffix, but ':' includes non-stripped characters."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_fe377dacaabb4933a5335e8f32dfa131_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_fe377dacaabb4933a5335e8f32dfa131_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_fe377dacaabb4933a5335e8f32dfa131_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-218 — src--incremental_reasoner-py--_codegraph_legacy_coverage

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: **confirmed**; attempts 1
- Phase / 模块： Phase 6 — Reasoning & Verification Engine / **incremental_pipeline**; 源代码 **src/incremental_reasoner.py**
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim**: Returns a dict whose keys are the normalized relative paths of the files listed in `file_languages` (paths relativized against `proj_dir` and case-normalized) and whose values are booleans. For a file whose corresponding disk path under `proj_dir` does not exist, the value is `True`. For an existing file, the value is `True` if and only if the ordered sequence of `(func_name, source)` tuples produced by the legacy extractor under the file's language key is an ordered subsequence of the `CodeGraph` entries for that file where a pair of entries match when they have the same unqualified function name and their source texts are identical after line-ending normalization and each matched `CodeGraph` entry appears at a strictly later index than the preceding match. When the ordered-subsequence condition fails meaning at least one legacy-discovered function lacks a matching `CodeGraph` entry in the required order the value is `False` and a warning identifying the unmatched function name and its file path is emitted via the logging system. The function makes no modifications to the filesystem.
- Actual behavior**: The function returns a dictionary `coverage` with one key-value pair for each `rel_path` in the input `file_languages`. For a given `rel_path` with associated language key `lang_key`, let `rel_key = _normalized_relative_path(proj_dir, rel_path)` and `abs_path = os.path.join(proj_dir, rel_path)`. If `os.path.exists(abs_path)` is `False`, then `coverage[rel_key]` is `True`. Otherwise, let `legacy_funcs` be the list of `(name, source)` tuples returned by `extract_functions_from_file(abs_path, lang_key)`, and let `codegraph_items` be the list of `(identifier, source)` pairs from `codegraph_functions.get(rel_key, {})` in dictionary item order. The value `coverage[rel_key]` is `True` if and only if there exists a strictly increasing integer sequence `j_0 < j_1 < ... < j_{L-1}` (where `L = len(legacy_funcs)`) with each `j_i` a valid index into `codegraph_items` such that for every `i`: `_bare_function_name(codegraph_items[j_i][0]) == _bare_function_name(legacy_funcs[i][0])` and `_normalized_function_source(codegraph_items[j_i][1]) == _normalized_function_source(legacy_funcs[i][1])`. If no such sequence exists, `coverage[rel_key]` is `False`. The iteration order over `file_languages` determines the order of insertion into `coverage`, but no further ordering of the returned dictionary is guaranteed beyond the natural Python dict insertion order (Python 3.7+).

- **Code evidence:** Line 28: `_bare_function_name(identifier) == legacy_bare_name`
- **Trigger condition:** The code uses `_bare_function_name`, which strips trailing dedup suffixes (e.g., `'_1'`), to compare identifiers. The specification requires that 'unqualified function name' refers to the name without only qualifiers, keeping any dedup suffix added by the legacy extractor. Consequently, a legacy name like `'foo_1'` and a CodeGraph name like `'foo()'` are considered equal by the code (both become `'foo'`) but should be considered unequal per the specification, leading to an incorrect True verdict when a False verdict is required.
- **Validator trigger:** `_bare_function_name` strips trailing dedup suffixes (e.g., `'_1'`) causing legacy name `'my_func_1'` to incorrectly match CodeGraph name `'my_func'` when the spec requires these to be treated as distinct.
- **Probe stdout:** CONFIRMED — actual: True | expected: False (legacy `'my_func_1'` should NOT match CodeGraph `'my_func'`)

► SPEC sidecar (完整)

```
{
  "signature": "_codegraph_legacy_coverage(proj_dir, codegraph_functions, file_languages) -> dict",
  "pre_condition": "proj_dir is a directory path. codegraph_functions is a dict mapping relative file paths to dicts of {function_name_string: source_text_string}. file_languages is a dict mapping relative file path strings to language key strings recognized by extract_functions_from_file.",
  "post_condition": "Returns a dict whose keys are the normalized relative paths of the files listed in file_languages (paths relativized against proj_dir and case-normalized) and whose values are booleans. For a file whose corresponding disk path under proj_dir does not exist, the value is True. For an existing file, the value is True if and only if the ordered sequence of (func_name, source) tuples produced by the legacy extractor under the file's language key is an ordered subsequence of the CodeGraph entries for that file – where a pair of entries match when they have the same unqualified function name and their source texts are identical after line-ending normalization – and each matched CodeGraph entry appears at a strictly later index than the preceding match. When the ordered-subsequence condition fails – meaning at least one legacy-discovered function lacks a matching CodeGraph entry in the required order – the value is False and a warning identifying the unmatched function name and its file path is emitted via the logging system. The function makes no modifications to the filesystem."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_normalized_relative_path",
      "signature": "_normalized_relative_path(proj_dir, path) -> str",
      "pre_condition": "proj_dir is a directory path. path is an absolute path or a path relative to proj_dir.",
      "post_condition": "Returns the path normalized relative to proj_dir: collapses . and .. segments, normalizes path separators, and applies case normalization for case-insensitive filesystems. The result is a string with no leading directory separator."
    },
    {
      "name": "extract_functions_from_file",
      "signature": "extract_functions_from_file(filepath, lang_key) -> list[tuple[str, str]]",
      "pre_condition": "filepath refers to an existing, readable file. lang_key is a language key present in the extraction configuration.",
      "post_condition": "Returns a list of (func_name, source_text) tuples, one per function found in the file via language-specific extraction rules, appearing in source-code order. Each func_name is canonicalized: unsafe characters are translated and duplicates are deduplicated by a numeric suffix in order of first occurrence. Each source_text includes the full function body from start to end inclusive, with line endings normalized to '\\n' and a trailing newline."
    },
    {
      "name": "_bare_function_name",
      "signature": "_bare_function_name(identifier) -> str",
      "pre_condition": "identifier is a string that may contain :: or . qualifier separators and a trailing _<digits> deduplication suffix.",
      "post_condition": "Returns the rightmost segment of identifier after the last :: or . separator, with any trailing underscore followed by digits removed. When the identifier contains no qualifier separators, the identifier itself is returned with any trailing dedup suffix stripped."
    },
    {
      "name": "_normalized_function_source",
      "signature": "_normalized_function_source(source) -> str",
      "pre_condition": "source is a string containing function source text.",
      "post_condition": "Returns the source text with all line endings normalized to a single \\n character. Carriage-return-line-feed pairs (\\r\\n) and standalone carriage returns (\\r) are each replaced with \\n."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_codegraph_legacy_coverage.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dict whose keys are the normalized relative paths of the files listed in file_languages (paths
```

relativized against proj_dir and case-normalized) and whose values are booleans. For a file whose corresponding disk path under proj_dir does not exist, the value is True. For an existing file, the value is True if and only if the ordered sequence of (func_name, source) tuples produced by the legacy extractor under the file's language key is an ordered subsequence of the CodeGraph entries for that file where a pair of entries match when they have the same unqualified function name and their source texts are identical after line-ending normalization and each matched CodeGraph entry appears at a strictly later index than the preceding match. When the ordered-subsequence condition fails meaning at least one legacy-discovered function lacks a matching CodeGraph entry in the required order the value is False and a warning identifying the unmatched function name and its file path is emitted via the logging system. The function makes no modifications to the filesystem.",

"actual_behavior": "The function returns a dictionary `coverage` with one key-value pair for each `rel\_path` in the input `file\_languages`. For a given `rel\_path` with associated language key `lang\_key`, let `rel\_key` = `_normalized_relative_path(proj_dir, rel_path)` and `abs_path = os.path.join(proj_dir, rel_path)`. If `os.path.exists(abs_path)` is False, then `coverage[rel_key]` is `True`. Otherwise, let `legacy_funcs` be the list of `(name, source)` tuples returned by `extract_functions_from_file(abs_path, lang_key)`, and let `codegraph_items` be the list of `(identifier, source)` pairs from `codegraph_functions.get(rel_key, {})` in dictionary item order. The value `coverage[rel_key]` is `True` if and only if there exists a strictly increasing integer sequence `j_0 < j_1 < ... < j_{L-1}` (where `L = len(legacy_funcs)`) with each `j_i` a valid index into `codegraph_items` such that for every `i`: `_bare_function_name(codegraph_items[j_i][0]) == _bare_function_name(legacy_funcs[i][0])` and `_normalized_function_source(codegraph_items[j_i][1]) == _normalized_function_source(legacy_funcs[i][1])`. If no such sequence exists, `coverage[rel_key]` is `False`. The iteration order over `file_languages` determines the order of insertion into `coverage`, but no further ordering of the returned dictionary is guaranteed beyond the natural Python dict insertion order (Python 3.7+).",

"code_evidence": "Line 28: `_bare_function_name(identifier) == legacy_bare_name`,

"trigger_condition": "The code uses `_bare_function_name`, which strips trailing dedup suffixes (e.g., `_1`), to compare identifiers. The specification requires that 'unqualified function name' refers to the name without only qualifiers, keeping any dedup suffix added by the legacy extractor. Consequently, a legacy name like `'foo_1'` and a CodeGraph name like `'foo()'` are considered equal by the code (both become `'foo'`) but should be considered unequal per the specification, leading to an incorrect True verdict when a False verdict is required."

```
}
}
```

► Bug Validator result (完整)

```
{
  "id": "src--incremental_reasoner-py--codegraph_legacy_coverage",
  "source_file": "/home/fancy/Projects/Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/codegraph_legacy_coverage.py",
  "function_name": "_codegraph_legacy_coverage",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--codegraph_legacy_coverage.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--codegraph_legacy_coverage.md",
  "probe_stdout": "CONFIRMED - actual: True | expected: False (legacy 'my_func_1' should NOT match CodeGraph 'my_func')\n",
  "trigger_summary": "_bare_function_name strips trailing dedup suffixes (e.g., '_1') causing legacy name 'my_func_1' to incorrectly match CodeGraph name 'my_func' when the spec requires these to be treated as distinct."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:48:14.474482Z	3174
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:48:46.457753Z	18858

事件 1 — [llm_95a1d0cbc6dc403e80f561f75f7464c2](#)

► parsed_json (完整)

```
{
  "post_condition": "The function returns a dictionary `coverage` with one key-value pair for each `rel_path` in the input `file_languages`. For a given `rel_path` with associated language key `lang_key`, let `rel_key` = _normalized_relative_path(proj_dir, rel_path) and abs_path = os.path.join(proj_dir, rel_path). If os.path.exists(abs_path) is False, then coverage[rel_key] is True. Otherwise, let legacy_funcs be the list of (name, source) tuples returned by extract_functions_from_file(abs_path, lang_key), and let codegraph_items be the list of (identifier, source) pairs from codegraph_functions.get(rel_key, {}) in dictionary item order. The value coverage[rel_key] is True if and only if there exists a strictly increasing integer sequence j_0 < j_1 < ... < j_{L-1} (where L = len(legacy_funcs)) with each j_i a valid index into codegraph_items such that for every i: _bare_function_name(codegraph_items[j_i][0]) == _bare_function_name(legacy_funcs[i][0]) and _normalized_function_source(codegraph_items[j_i][1]) == _normalized_function_source(legacy_funcs[i][1]). If no such sequence exists, coverage[rel_key] is False. The iteration order over file_languages determines the order of insertion into coverage, but no further ordering of the returned dictionary is guaranteed beyond the natural Python dict insertion order (Python 3.7+)."
```

(fm_agent/trace/payloads/llm_95a1d0cbc6dc403e80f561f75f7464c2_response.txt)

事件 2 — llm_b80a1c312ce142bfa13e3e66cded5627

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/proj', file_languages={'src/f.cpp': 'cpp'}, codegraph_functions={'src/f.cpp': {'foo()': 'int foo() { return 0; }'}}. The file exists and extract_functions_from_file returns [('foo_1', 'int foo() { return 0; }')] (dedup suffix added by legacy extractor). The code strips the '_1' via _bare_function_name, matching bare name 'foo' against codegraph's 'foo()', producing coverage True. The specification requires matching on the unqualified function name as produced by the extractor, i.e., 'foo_1' ≠ 'foo()', so correct result should be False.",
  "offending_statements": "Line 28:         _bare_function_name(identifier) == legacy_bare_name",
  "reason": "The code uses _bare_function_name, which strips trailing dedup suffixes (e.g., '_1'), to compare identifiers. The specification requires that 'unqualified function name' refers to the name without only qualifiers, keeping any dedup suffix added by the legacy extractor. Consequently, a legacy name like 'foo_1' and a CodeGraph name like 'foo()' are considered equal by the code (both become 'foo') but should be considered unequal per the specification, leading to an incorrect True verdict when a False verdict is required."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b80a1c312ce142bfa13e3e66cded5627_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_b80a1c312ce142bfa13e3e66cded5627_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_b80a1c312ce142bfa13e3e66cded5627_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-220 — src--incremental_reasoner-py--_collect_changed_functions::_funcs_from_commit

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#); 原源码 [src/incremental_reasoner.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a dictionary whose keys are function name strings extracted from the old_commit_id revision of the file at rel_path and whose values are the corresponding function source text strings. The dictionary is empty when the file at rel_path contains no functions extractable by the lang_key extractor. Any temporary file created on disk during extraction is deleted before the function returns, regardless of whether extraction succeeds or raises an exception. If the git show command fails (e.g., invalid commit, missing file), the subprocess error propagates to the caller after temporary-file cleanup.
- **Actual behavior**: If the code block completes normally, it returns a dictionary where each key is a non-empty function name and each value is the corresponding non-empty source code, extracted by [extract_functions_from_file](#) from a temporary file containing the content of [rel_path](#) at git revision [old_commit_id](#). The temporary file created (using [tempfile.NamedTemporaryFile](#) with suffix [ext](#)) is reliably deleted via [os.unlink](#) in the [finally](#) block, so no such file persists after the function returns. If any exception occurs (e.g., from [_git](#), file I/O, or [extract_functions_from_file](#)), that exception is propagated, but the [finally](#) block still executes, deleting the temporary file if it was successfully created and its path assigned to [tmp_path](#). Formally, for a normal return with no exception: let [c](#) = [_git\('show', f'{old_commit_id}:{rel_path}'\)](#); there exists a file at path [p](#) such that its content equals [c](#) and [p](#) ends with ['.' + ext](#); after the return, [os.path.exists\(p\)](#) is False; and the returned value satisfies [returned = dict\(extract_functions_from_file\(p, lang_key\)\)](#). For an exceptional return: if the temporary file at path [p](#) was created before the exception, then after the [finally](#) block [os.path.exists\(p\)](#) is False, and the exception is reraised; otherwise, no temporary file was created.
- **Code evidence**: Line 4: with [tempfile.NamedTemporaryFile\("w", suffix=f'{ext}", delete=False\)](#) as tmp: Line 5: tmp.write(text) Line 6: tmp_path = tmp.name
- **Trigger condition**: The specification requires that any temporary file created on disk during extraction is deleted before the function returns, regardless of whether extraction succeeds or raises an exception. In the code, if [tmp.write\(text\)](#) raises an exception (e.g., due to a full disk), the file is created but [tmp_path](#) is not assigned, so the [finally](#) block never executes, and the temporary file is left on disk.
- **Validator trigger**: When tmp.write(text) raises an exception, tmp_path is never assigned and the finally block never executes, leaving the temporary file on disk.
- **Probe stdout**: CONFIRMED — write failure caused temp file leak. File '/tmp/tmpgdrp9xnz.py' still on disk after exception. tmp_path was never assigned so cleanup never ran.

► SPEC sidecar (完整)

```
{
  "signature": "_funcs_from_commit(rel_path, lang_key, ext) -> dict",
  "pre_condition": "rel_path is a string path relative to the repository root, known to exist at the old_commit_id revision bound in the enclosing scope. old_commit_id is a valid, reachable commit identifier in the repository. lang_key is a string recognized by extract_functions_from_file as a language key. ext is a non-empty string.",
  "post_condition": "Returns a dictionary whose keys are function name strings extracted from the old_commit_id revision of the file at rel_path and whose values are the corresponding function source text strings. The dictionary is empty when the file at rel_path contains no functions extractable by the lang_key extractor. Any temporary file created on disk during extraction is deleted before the function returns, regardless of whether extraction succeeds or raises an exception. If the git show command fails (e.g., invalid commit, missing file), the subprocess error propagates to the caller after temporary-file cleanup."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_git",
      "signature": "_git(command, arg_string) -> str",
      "pre_condition": "command is a string naming a valid git subcommand. arg_string is a non-empty string of arguments for that subcommand. The current working directory is within a git repository.",
      "post_condition": "Executes git command arg_string and returns the captured stdout as a string. Raises subprocess.CalledProcessError when the git command exits with a non-zero status."
    },
    {
      "name": "extract_functions_from_file",
      "signature": "extract_functions_from_file(file_path, lang_key) -> Iterable[tuple[str, str]]",
      "pre_condition": "file_path is the absolute path to a source file that exists and is readable. lang_key is a string key recognized by the extraction backend for the language.",
      "post_condition": "Returns an iterable of (function_name, source_text) pairs extracted from the file at file_path. Each function_name is a non-empty string identifying a function defined in the file; each source_text is a non-empty string containing the function's source code."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_collect_changed_functions::_funcs_from_commit.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dictionary whose keys are function name strings extracted from the old_commit_id revision of the file at rel_path and whose values are the corresponding function source text strings. The dictionary is empty when the file at rel_path contains no functions extractable by the lang_key extractor. Any temporary file created on disk during extraction is deleted before the function returns, regardless of whether extraction succeeds or raises an exception. If the git show command fails (e.g., invalid commit, missing file), the subprocess error propagates to the caller after temporary-file cleanup.",
    "actual_behavior": "If the code block completes normally, it returns a dictionary where each key is a non-empty function name and each value is the corresponding non-empty source code, extracted by `extract_functions_from_file` from a temporary file containing the content of `rel_path` at git revision `old_commit_id`. The temporary file created (using `tempfile.NamedTemporaryFile` with suffix `ext`) is reliably deleted via `os.unlink` in the `finally` block, so no such file persists after the function returns. If any exception occurs (e.g., from `_git`, file I/O, or `extract_functions_from_file`), that exception is propagated, but the `finally` block still executes, deleting the temporary file if it was successfully created and its path assigned to `tmp_path`. Formally, for a normal return with no exception: let `c` = `_git('show', f'{old_commit_id}:{rel_path}');` there exists a file at path `p` such that its content equals `c` and `p` ends with `.` + `ext`; after the return, `os.path.exists(p)` is False; and the returned value satisfies `returned = dict(extract_functions_from_file(p, lang_key))`. For an exceptional return: if the temporary file at path `p` was created before the exception, then after the `finally` block `os.path.exists(p)` is False, and the exception is reraised; otherwise, no temporary file was created.",
    "code_evidence": "Line 4:         with tempfile.NamedTemporaryFile(\"w\", suffix=f\".{ext}\", delete=False) as tmp:\nLine 5: tmp.write(text)\nLine 6:         tmp_path = tmp.name",
    "trigger_condition": "The specification requires that any temporary file created on disk during extraction is deleted before the function returns, regardless of whether extraction succeeds or raises an exception. In the code, if `tmp.write(text)` raises an exception (e.g., due to a full disk), the file is created but `tmp_path` is not assigned, so the `finally` block never executes, and the temporary file is left on disk."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--incremental_reasoner-py--_collect_changed_functions::_funcs_from_commit",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_collect_changed_functions::_funcs_from_commit.py",
  "function_name": "_funcs_from_commit",
  "confirmation_status": "confirmed",
  "attempts": 1,
```



```
"probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_collect_changed_functions::_funcs_from_commit.py",
"detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_collect_changed_functions::_funcs_from_commit.md",
"probe_stdout": "CONFIRMED - write failure caused temp file leak. File '/tmp/tmpgdrp9xnz.py' still on disk after exception.
tmp_path was never assigned so cleanup never ran.\n",
"trigger_summary": "When tmp.write(text) raises an exception, tmp_path is never assigned and the finally block never executes,
leaving the temporary file on disk."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:22:49.407590Z	2131
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:23:16.909312Z	4745

事件 1 — [llm_b3df45a493204cd8ad441b3bdbbf84c6](#)

► parsed_json（完整）

```
{
  "post_condition": "If the code block completes normally, it returns a dictionary where each key is a non-empty function name and
each value is the corresponding non-empty source code, extracted by `extract_functions_from_file` from a temporary file containing
the content of `rel_path` at git revision `old_commit_id`. The temporary file created (using `tempfile.NamedTemporaryFile` with
suffix `ext`) is reliably deleted via `os.unlink` in the `finally` block, so no such file persists after the function returns. If
any exception occurs (e.g., from `_git`, file I/O, or `extract_functions_from_file`), that exception is propagated, but the
`finally` block still executes, deleting the temporary file if it was successfully created and its path assigned to `tmp_path`.
Formally, for a normal return with no exception: let `c = _git('show', f'{old_commit_id}:{rel_path}')`; there exists a file at
path `p` such that its content equals `c` and `p` ends with `.` + ext`; after the return, `os.path.exists(p)` is False; and the
returned value satisfies `returned = dict(extract_functions_from_file(p, lang_key))`. For an exceptional return: if the temporary
file at path `p` was created before the exception, then after the `finally` block `os.path.exists(p)` is False, and the exception
is reraised; otherwise, no temporary file was created."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b3df45a493204cd8ad441b3bdbbf84c6_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_b3df45a493204cd8ad441b3bdbbf84c6_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_b3df45a493204cd8ad441b3bdbbf84c6_response.txt)

事件 2 — [llm_1571fc5623c74514917d452d6403cc44](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "old_commit_id='abc123', rel_path='src/main.py', lang_key='python', ext='py', and the disk of the temporary
directory is full when tmp.write(text) executes.",
  "offending_statements": "Line 4:      with tempfile.NamedTemporaryFile(\"w\", suffix=f\".{ext}\", delete=False) as tmp:\nLine 5:
tmp.write(text)\nLine 6:      tmp_path = tmp.name",
  "reason": "The specification requires that any temporary file created on disk during extraction is deleted before the function
returns, regardless of whether extraction succeeds or raises an exception. In the code, if `tmp.write(text)` raises an exception
(e.g., due to a full disk), the file is created but `tmp_path` is not assigned, so the `finally` block never executes, and the
temporary file is left on disk."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1571fc5623c74514917d452d6403cc44_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_1571fc5623c74514917d452d6403cc44_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_1571fc5623c74514917d452d6403cc44_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-221 — [src--incremental_reasoner-py--_collect_changed_functions::_git](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#); attempts 1
- Phase / 模块： Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#); 原源码 [src/incremental_reasoner.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

SPEC claim: Executes the command `git -C proj_dir` in a subprocess and returns the captured standard output as a string. Raises `subprocess.CalledProcessError` when the git command exits with a non-zero status code; the captured standard error is available in the exception's `stderr` attribute.

Actual behavior: If the function returns normally, it returns the captured stdout (as a string) of the command `git -C proj_dir *args`. If the git subprocess terminates with a non-zero exit code, a `subprocess.CalledProcessError` is raised. If the `git` executable cannot be found, a `FileNotFoundError` (a subclass of `OSError`) is raised. No other side effects on the Python process state occur, although the git command may modify the repository and file system.

Formally: `post { normal_return res subprocess.CompletedProcess . res = subprocess.run(['git', '-C', proj_dir, *args], check=True, capture_output=True, text=True) res.returncode = 0 return = res.stdout } { exceptional (exception {subprocess.CalledProcessError, FileNotFoundError}) }`

- Code evidence:** Line 5: `text=True`
- Trigger condition:** The specification requires that when the git command exits with code 0, the function returns the captured standard output as a string. Using `text=True` without error handling can raise `UnicodeDecodeError` for binary stdout, violating the return-type guarantee.
- Validator trigger:** When `_funcs_from_commit` calls `_git('show', 'commit:binary_file')` on a file whose git blob is binary (non-UTF-8), `text=True` in `subprocess.run` raises `UnicodeDecodeError` instead of returning a string.
- Probe stdout:** WARNING:root:CodeGraph could not provide both revisions for crash.py; using legacy regex comparison. CONFIRMED - `UnicodeDecodeError: 'utf-8' codec can't decode byte 0x80 in position 0: invalid start byte`

► SPEC sidecar (完整)

```
{
  "signature": "_git(*args) -> str",
  "pre_condition": "proj_dir is bound in the enclosing scope to a directory path within a git repository. args is a non-empty tuple of strings forming a valid git subcommand and its arguments.",
  "post_condition": "Executes the command git -C proj_dir <args> in a subprocess and returns the captured standard output as a string. Raises subprocess.CalledProcessError when the git command exits with a non-zero status code; the captured standard error is available in the exception's stderr attribute."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_collect_changed_functions::_git.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Executes the command git -C proj_dir <args> in a subprocess and returns the captured standard output as a string. Raises subprocess.CalledProcessError when the git command exits with a non-zero status code; the captured standard error is available in the exception's stderr attribute.",
    "actual_behavior": "If the function returns normally, it returns the captured stdout (as a string) of the command `git -C proj_dir *args`. If the git subprocess terminates with a non-zero exit code, a `subprocess.CalledProcessError` is raised. If the `git` executable cannot be found, a `FileNotFoundError` (a subclass of `OSError`) is raised. No other side effects on the Python process state occur, although the git command may modify the repository and file system.\n\nFormally: post { normal_return res subprocess.CompletedProcess . res = subprocess.run(['git', '-C', proj_dir, *args], check=True, capture_output=True, text=True) res.returncode = 0 return = res.stdout } { exceptional (exception {subprocess.CalledProcessError, FileNotFoundError}) }",
    "code_evidence": "Line 5: text=True",
    "trigger_condition": "The specification requires that when the git command exits with code 0, the function returns the captured standard output as a string. Using text=True without error handling can raise UnicodeDecodeError for binary stdout, violating the return-type guarantee."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--incremental_reasoner-py--_collect_changed_functions::_git",
```

```

"source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_collect_changed_functions::_git.py",
"function_name": "_git",
"confirmation_status": "confirmed",
"attempts": 1,
"probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_collect_changed_functions::_git.py",
"detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_collect_changed_functions::_git.md",
"probe_stdout": "WARNING:root:CodeGraph could not provide both revisions for crash.py; using legacy regex comparison.\nCONFIRMED
- UnicodeDecodeError: 'utf-8' codec can't decode byte 0x80 in position 0: invalid start byte",
"trigger_summary": "When _funcs_from_commit calls _git('show', 'commit:binary_file') on a file whose git blob is binary (non-
UTF-8), text=True in subprocess.run raises UnicodeDecodeError instead of returning a string."
}

```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:48:34.100660Z	2739
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:49:19.300510Z	3678

事件 1 — [llm_2db1277c504847f2bc9ccb6e14f7ac5e](#)

► parsed_json（完整）

```

{
  "post_condition": "If the function returns normally, it returns the captured stdout (as a string) of the command `git -C proj_dir *args`. If the git subprocess terminates with a non-zero exit code, a `subprocess.CallledProcessError` is raised. If the `git` executable cannot be found, a `FileNotFoundError` (a subclass of `OSError`) is raised. No other side effects on the Python process state occur, although the git command may modify the repository and file system.\n\nFormally: post ∈ { normal_return ∧ ∃ res ∈ subprocess.CompletedProcess . res = subprocess.run(['git', '-C', proj_dir, *args], check=True, capture_output=True, text=True) ∧ res.returncode = 0 ∧ return = res.stdout } ⊕ { exceptional ∧ (exception ∈ {subprocess.CallledProcessError, FileNotFoundError}) }"
}

```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_2db1277c504847f2bc9ccb6e14f7ac5e_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_2db1277c504847f2bc9ccb6e14f7ac5e_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_2db1277c504847f2bc9ccb6e14f7ac5e_response.txt\)](#)

事件 2 — [llm_4ab1f64c763848589500e428a8cb5af0](#)

► parsed_json（完整）

```

{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/path/to/repo'; args = ['cat-file', '-p', 'HEAD:binaryfile'] where the file content contains a byte such as 0x80 (invalid UTF-8).",
  "offending_statements": "Line 5: text=True",
  "reason": "The specification requires that when the git command exits with code 0, the function returns the captured standard output as a string. Using text=True without error handling can raise UnicodeDecodeError for binary stdout, violating the return-type guarantee."
}

```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_4ab1f64c763848589500e428a8cb5af0_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_4ab1f64c763848589500e428a8cb5af0_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_4ab1f64c763848589500e428a8cb5af0_response.txt\)](#) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-222 — [src--incremental_reasoner-py--_collect_changed_functions::_is_workspace_file](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts 1
- Phase / 模块： Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#); 原源码 [src/incremental_reasoner.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

- **SPEC claim:** Returns True when `rel_path`, after replacing every backslash with a forward slash, equals the exact string `"fm_agent"` or has the prefix `"fm_agent/"`. Returns False in all other cases, including when `rel_path` is empty, contains only the `fm_agent` directory name but with additional path components that do not form a descendant path under `fm_agent/`, or represents a file whose normalized path falls outside the `fm_agent/` workspace tree.
- **Actual behavior:** The function `_is_workspace_file` returns True if the normalized version of `rel_path`, obtained by replacing every backslash `"` with a forward slash `/`, equals the string `'fm_agent'` or starts with `'fm_agent/'`; otherwise it returns False. The function has no side effects and does not modify `rel_path`. Formally: `rel_path Strings, _is_workspace_file(rel_path) (rel_path.replace('\', '/') = 'fm_agent') (rel_path.replace('\', '/').startswith('fm_agent/'))`.
- **Code evidence:** Line 3: `return norm == "fm_agent" or norm.startswith("fm_agent")`
- **Trigger condition:** The code uses a simple string prefix check after replacing backslashes. For the input `'fm_agent/..'`, normalization yields `'fm_agent/..'`, which starts with `'fm_agent/'`, so the code returns True. However, the specification requires False when the path does not form a descendant path under `fm_agent/`, and `'..'` escapes the directory, making it a nondescendant. The string prefix alone does not detect such escapes, violating the specification.
- **Validator trigger:** Paths starting with `fm_agent/` that contain `'..'` components (e.g. `fm_agent/..`) pass the naive `startswith` check but escape the workspace directory.
- **Probe stdout:** [PASS] `rel_path='fm_agent'` expected=True actual=True (exact match) [PASS] `rel_path='fm_agent/some_file.py'` expected=True actual=True (descendant inside workspace) [PASS] `rel_path='fm_agent/nested/deep/file.c'` expected=True actual=True (deeply nested descendant) [PASS] `rel_path='fm_agent\subdir\file.rs'` expected=True actual=True (Windows backslash normalization) [PASS] `rel_path='src/main.py'` expected=False actual=False (outside workspace entirely) [PASS] `rel_path=""` expected=False actual=False (empty path) [PASS] `rel_path='fm'` expected=False actual=False (partial prefix match, not `fm_agent`) [FAIL] `rel_path='fm_agent/..'` expected=False actual=True (`..` escapes the workspace directory) [FAIL] `rel_path='fm_agent/./escape.py'` expected=False actual=True (`..` escape with trailing file) [FAIL] `rel_path='fm_agent/./../outside'` expected=False actual=True (double `..` escape) [FAIL] `rel_path='fm_agent/subdir/./.././root.txt'` expected=False actual=True (nested then `..` escape) [FAIL] `rel_path='fm_agent\.\escape.rs'` expected=False actual=True (Windows backslash `..` escape)

CONFIRMED — 5 test case(s) expose the bug: `rel_path='fm_agent/..':` expected False, got True (`..` escapes the workspace directory) `rel_path='fm_agent/./escape.py':` expected False, got True (`..` escape with trailing file) `rel_path='fm_agent/./../outside':` expected False, got True (double `..` escape) `rel_path='fm_agent/subdir/./.././root.txt':` expected False, got True (nested then `..` escape) `rel_path='fm_agent\.\escape.rs':` expected False, got True (Windows backslash `..` escape)

► SPEC sidecar (完整)

```
{
  "signature": "_is_workspace_file(rel_path) -> bool",
  "pre_condition": "rel_path is a string representing a relative file path.",
  "post_condition": "Returns True when rel_path, after replacing every backslash with a forward slash, equals the exact string \"fm_agent\" or has the prefix \"fm_agent/\". Returns False in all other cases, including when rel_path is empty, contains only the fm_agent directory name but with additional path components that do not form a descendant path under fm_agent/, or represents a file whose normalized path falls outside the fm_agent/ workspace tree."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_collect_changed_functions::_is_workspace_file.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns True when rel_path, after replacing every backslash with a forward slash, equals the exact string \"fm_agent\" or has the prefix \"fm_agent/\". Returns False in all other cases, including when rel_path is empty, contains only the fm_agent directory name but with additional path components that do not form a descendant path under fm_agent/, or represents a file whose normalized path falls outside the fm_agent/ workspace tree.",
    "actual_behavior": "The function _is_workspace_file returns True if the normalized version of rel_path, obtained by replacing every backslash \"\" with a forward slash '/', equals the string 'fm_agent' or starts with 'fm_agent/'; otherwise it returns False. The function has no side effects and does not modify rel_path. Formally: rel_path Strings, _is_workspace_file(rel_path) (rel_path.replace('\\\\\\\\', '/') = 'fm_agent') (rel_path.replace('\\\\\\\\', '/').startswith('fm_agent/')).",
    "code_evidence": "Line 3: return norm == \"fm_agent\" or norm.startswith(\"fm_agent/\")",
    "trigger_condition": "The code uses a simple string prefix check after replacing backslashes. For the input 'fm_agent/..', normalization yields 'fm_agent/..', which starts with 'fm_agent/', so the code returns True. However, the specification requires False when the path does not form a descendant path under fm_agent/, and '..' escapes the directory, making it a nondescendant."
  }
}
```

```

    }
    }
}

```

► Bug Validator result（完整）

```

{
  "id": "src--incremental_reasoner-py--_collect_changed_functions::_is_workspace_file",
  "source_file": "src/incremental_reasoner.py",
  "function_name": "_is_workspace_file",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_collect_changed_functions::_is_workspace_file.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_collect_changed_functions::_is_workspace_file.md",
  "probe_stdout": "[PASS] rel_path='fm_agent' expected=True actual=True (exact match)\n[PASS] rel_path='fm_agent/some_file.py' expected=True actual=True (descendant inside workspace)\n[PASS] rel_path='fm_agent/nested/deep/file.c' expected=True actual=True (deeply nested descendant)\n[PASS] rel_path='fm_agent\\\\\\\\subdir\\\\\\\\file.rs' expected=True actual=True (Windows backslash normalization)\n[PASS] rel_path='src/main.py' expected=False actual=False (outside workspace entirely)\n[PASS] rel_path='' expected=False actual=False (empty path)\n[PASS] rel_path='fm' expected=False actual=False (partial prefix match, not fm_agent)\n[FAIL] rel_path='fm_agent/..' expected=False actual=True (.. escapes the workspace directory)\n[FAIL] rel_path='fm_agent/./escape.py' expected=False actual=True (.. escape with trailing file)\n[FAIL] rel_path='fm_agent/../../outside' expected=False actual=True (double .. escape)\n[FAIL] rel_path='fm_agent/subdir/../../root.txt' expected=False actual=True (nested then .. escape)\n[FAIL] rel_path='fm_agent\\\\\\\\.\\\\\\\\escape.rs' expected=False actual=True (Windows backslash .. escape)\n\nCONFIRMED - 5 test case(s) expose the bug:\n  rel_path='fm_agent/..': expected False, got True (.. escapes the workspace directory)\n  rel_path='fm_agent/./escape.py': expected False, got True (.. escape with trailing file)\n  rel_path='fm_agent/../../outside': expected False, got True (double .. escape)\n  rel_path='fm_agent/subdir/../../root.txt': expected False, got True (nested then .. escape)\n  rel_path='fm_agent\\\\\\\\.\\\\\\\\escape.rs': expected False, got True (Windows backslash .. escape)",
  "trigger_summary": "Paths starting with fm_agent/ that contain '..' components (e.g. fm_agent/..) pass the naive startswith check but escape the workspace directory."
}

```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:23:26.102254Z	1641
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:23:50.700100Z	2797

事件 1 — [llm_d5d4d398d71c4b98bddd614ddd4cfb98](#)

► parsed_json（完整）

```

{
  "post_condition": "The function _is_workspace_file returns True if the normalized version of rel_path, obtained by replacing every backslash '\\\\' with a forward slash '/', equals the string 'fm_agent' or starts with 'fm_agent/'; otherwise it returns False. The function has no side effects and does not modify rel_path. Formally: ∀ rel_path ∈ Strings, _is_workspace_file(rel_path) ⇔ (rel_path.replace('\\\\\\\\', '/') = 'fm_agent') ∨ (rel_path.replace('\\\\\\\\', '/').startswith('fm_agent/'))."
}

```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d5d4d398d71c4b98bddd614ddd4cfb98_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d5d4d398d71c4b98bddd614ddd4cfb98_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d5d4d398d71c4b98bddd614ddd4cfb98_response.txt)

事件 2 — [llm_de7c3366c9ff47c484567d05c09146e6](#)

► parsed_json（完整）

```

{
  "verdict": "MISMATCH",
  "counterexample": "rel_path = 'fm_agent/..'",
  "offending_statements": "Line 3: return norm == '\\fm_agent\\' or norm.startswith('\\fm_agent/\\')",
  "reason": "The code uses a simple string prefix check after replacing backslashes. For the input 'fm_agent/..', normalization yields 'fm_agent/..', which starts with 'fm_agent/', so the code returns True. However, the specification requires False when the path does not form a descendant path under fm_agent/, and '..' escapes the directory, making it a non-descendant. The string prefix alone does not detect such escapes, violating the specification."
}

```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_de7c3366c9ff47c484567d05c09146e6_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_de7c3366c9ff47c484567d05c09146e6_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_de7c3366c9ff47c484567d05c09146e6_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-223 — `src--incremental_reasoner.py--_collect_changed_functions::_path_exists_in_commit`

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: `confirmed`; attempts 2
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / `incremental_pipeline`; 源代码 `src/incremental_reasoner.py`
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Returns True when `rel_path` identifies an object present at `old_commit_id` in the repository at `proj_dir`. Returns False when no such object exists at that revision or when the git command fails for any other reason. Never raises an exception.
- **Actual behavior**: The function returns a boolean. It returns True if and only if the git command `git -C proj_dir cat-file -e old_commit_id:rel_path` exits with a return code of 0, which indicates that the object (file or directory) at the given relative path exists in the specified commit of the repository located at `proj_dir`. If the command fails for any reason (including the path not existing or external errors), the return code is non-zero and the function returns False. No exceptions are raised.

Formally: Let `cmd = ['git', '-C', proj_dir, 'cat-file', '-e', f'{old_commit_id}:{rel_path}']`. Then `_path_exists_in_commit(rel_path) == (subprocess.run(cmd, check=False, capture_output=True, text=True).returncode == 0)`.

- **Code evidence**: Line 3: `return subprocess.run(` Line 4: `['git', "-C", proj_dir, "cat-file", "-e", f'{old_commit_id}:{rel_path}']`, Line 5: `check=False`, Line 6: `capture_output=True`, Line 7: `text=True`, Line 8: `).returncode == 0`
- **Trigger condition**: The code runs a git command via `subprocess.run` without handling non-zero exit or `OSError`. If git is missing, `subprocess.run` raises `FileNotFoundError`, so the function raises an exception instead of returning False as required by the specification's 'Returns False when ... the git command fails for any other reason. Never raises an exception.'
- **Validator trigger**: `subprocess.run` raises `FileNotFoundError` when git is missing from `PATH`, propagating uncaught instead of returning False.
- **Probe stdout**: CONFIRMED — `FileNotFoundError` propagated: `[Errno 2] No such file or directory: 'git'`

► SPEC sidecar (完整)

```
{
  "signature": "_path_exists_in_commit(rel_path) -> bool",
  "pre_condition": "rel_path is a non-empty string representing a relative file path. proj_dir and old_commit_id are captured from the enclosing scope: proj_dir is a directory path within a git repository, and old_commit_id is a valid commit identifier reachable in that repository.",
  "post_condition": "Returns True when rel_path identifies an object present at old_commit_id in the repository at proj_dir. Returns False when no such object exists at that revision or when the git command fails for any other reason. Never raises an exception."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_collect_changed_functions::_path_exists_in_commit.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns True when rel_path identifies an object present at old_commit_id in the repository at proj_dir. Returns False when no such object exists at that revision or when the git command fails for any other reason. Never raises an exception.",
    "actual_behavior": "The function returns a boolean. It returns True if and only if the git command `git -C proj_dir cat-file -e old_commit_id:rel_path` exits with a return code of 0, which indicates that the object (file or directory) at the given relative path exists in the specified commit of the repository located at `proj_dir`. If the command fails for any reason (including the path not existing or external errors), the return code is non-zero and the function returns False. No exceptions are raised."
  }
}
```

```
\n\nFormally: Let cmd = ['git', '-C', proj_dir, 'cat-file', '-e', f\"{old_commit_id}:{rel_path}\"] . Then
_path_exists_in_commit(rel_path) == (subprocess.run(cmd, check=False, capture_output=True, text=True).returncode == 0).",
"code_evidence": "Line 3: return subprocess.run(\nLine 4:      [\"git\\", \"-C\\", proj_dir, \"cat-file\\", \"-e\\", f\"
{old_commit_id}:{rel_path}\"] ,\nLine 5:      check=False,\nLine 6:      capture_output=True,\nLine 7:      text=True,\nLine 8:
).returncode == 0",
"trigger_condition": "The code runs a git command via subprocess.run without handling non-zero exit or OSError. If git is
missing, subprocess.run raises FileNotFoundError, so the function raises an exception instead of returning False as required by
the specification's 'Returns False when ... the git command fails for any other reason. Never raises an exception.'"
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_collect_changed_functions::_path_exists_in_commit",
  "source_file": "src/incremental_reasoner.py",
  "function_name": "_path_exists_in_commit",
  "confirmation_status": "confirmed",
  "attempts": 2,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--
_collect_changed_functions::_path_exists_in_commit.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_collect_changed_functions::_path_exists_in_commit.md",
  "probe_stdout": "CONFIRMED - FileNotFoundError propagated: [Errno 2] No such file or directory: 'git'\n",
  "trigger_summary": "subprocess.run raises FileNotFoundError when git is missing from PATH, propagating uncaught instead of
returning False."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:22:39.375060Z	1613
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:23:02.865095Z	4777

事件 1 — [llm_fd8789144ae7440299d04ddb7f487a84](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a boolean. It returns True if and only if the git command `git -C proj_dir cat-file -e
old_commit_id:rel_path` exits with a return code of 0, which indicates that the object (file or directory) at the given relative
path exists in the specified commit of the repository located at `proj_dir`. If the command fails for any reason (including the
path not existing or external errors), the return code is non-zero and the function returns False. No exceptions are raised.
\n\nFormally: Let cmd = ['git', '-C', proj_dir, 'cat-file', '-e', f\"{old_commit_id}:{rel_path}\"] . Then
_path_exists_in_commit(rel_path) == (subprocess.run(cmd, check=False, capture_output=True, text=True).returncode == 0).\"
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_fd8789144ae7440299d04ddb7f487a84_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_fd8789144ae7440299d04ddb7f487a84_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_fd8789144ae7440299d04ddb7f487a84_response.txt\)](#)

事件 2 — [llm_1b5558947dc545d0bf9349cdf03739b9](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "rel_path = \"any_file\", environment where `git` is not installed or not found in PATH",
  "offending_statements": "Line 3: return subprocess.run(\nLine 4:      [\"git\\", \"-C\\", proj_dir, \"cat-file\\", \"-e\\", f\"
{old_commit_id}:{rel_path}\"] ,\nLine 5:      check=False,\nLine 6:      capture_output=True,\nLine 7:      text=True,\nLine 8:
).returncode == 0",
  "reason": "The code runs a git command via subprocess.run without handling non-zero exit or OSError. If git is missing,
subprocess.run raises FileNotFoundError, so the function raises an exception instead of returning False as required by the
specification's 'Returns False when ... the git command fails for any other reason. Never raises an exception.'"
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_1b5558947dc545d0bf9349cdf03739b9_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_1b5558947dc545d0bf9349cdf03739b9_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_1b5558947dc545d0bf9349cdf03739b9_message_01_user.txt\)](#)

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-224 — [src--incremental_reasoner-py--_extracted_files_by_method](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： [confirmed](#); attempts 2
- Phase / 模块： Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#); 原源码 [src/incremental_reasoner.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim:** Returns a dict whose keys are function identifier strings and whose values are non-empty lists of absolute file path strings. When func_dir exists as a directory: for every extracted function file found in a recursive traversal of func_dir (excluding metadata sidecar files), the file's absolute path is registered under its full file-stem identifier the filename without its extension, which may contain '::' class-qualifier separators; additionally, when a stem contains '::', the substring after the last '::' (the bare method tail) is registered as an additional key mapping to the same path list. A single bare method tail maps to all paths from different classes that share that tail. When func_dir does not exist or is not a directory: returns an empty dict.
- Actual behavior:** The function returns a collections.defaultdict instance `index`, whose default factory is `list`. If an exception (e.g., `FileNotFoundError`, `PermissionError`) is raised during the call to `os.path.isdir(func_dir)` or during `os.walk(func_dir)`, the function propagates that exception and no return occurs. If execution completes normally: If `os.path.isdir(func_dir)` is False, then `index` is empty (i.e., for all keys `k`, `index[k] == []`). Otherwise, let `D` be the set of absolute paths of all regular files under the directory tree rooted at `func_dir` (recursively) whose basename `fn` satisfies `_is_metadata_sidecar(fn) == False`. Let `stem(f)` for a file `f` in `D` be defined as: `fn[:fn.rfind('.')] if '.' is in the basename fn of f, else fn`. Let `bare(f) = stem(f).split('::')[-1]`. Then `index` satisfies: for every key `k`, `index[k]` is a list containing exactly the absolute paths `f` in `D` such that `k == stem(f)` or (`k == bare(f)` and `bare(f) != stem(f)`). No other keys exist. The relative order of elements in each `index[k]` matches the order files are encountered by `os.walk` (depth-first, top-down). If `func_dir` is a directory but `D` is empty, all `index` values are empty lists.
- Code evidence:** Line 14: for root, _dirs, fnames in os.walk(func_dir):
- Trigger condition:** Specification requires returning a dict when func_dir exists as a directory. However, os.walk('/root') raises `PermissionError` for an unprivileged user, which the code propagates. This violates the specification because a dict is never returned.
- Validator trigger:** os.walk raises `PermissionError` on an inaccessible directory and the exception propagates unhandled instead of returning a dict as the spec requires.
- Probe stdout:** CONFIRMED — `PermissionError` from os.walk propagated instead of returning a dict

► SPEC sidecar（完整）

```
{
  "signature": "_extracted_files_by_method(func_dir) -> dict[str, list[str]]",
  "pre_condition": "func_dir is a string representing a filesystem path.",
  "post_condition": "Returns a dict whose keys are function identifier strings and whose values are non-empty lists of absolute file path strings. When func_dir exists as a directory: for every extracted function file found in a recursive traversal of func_dir (excluding metadata sidecar files), the file's absolute path is registered under its full file-stem identifier – the filename without its extension, which may contain '::' class-qualifier separators; additionally, when a stem contains '::', the substring after the last '::' (the bare method tail) is registered as an additional key mapping to the same path list. A single bare method tail maps to all paths from different classes that share that tail. When func_dir does not exist or is not a directory: returns an empty dict."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_is_metadata_sidecar",
      "signature": "_is_metadata_sidecar(filename) -> bool",
      "pre_condition": "filename is a non-empty string representing a file basename found during directory traversal.",
      "post_condition": "Returns True when filename identifies a generated FM-Agent metadata sidecar file – a specification or callee-information file that must be excluded from the extracted function file listing; returns False when filename identifies an extracted function source file or any other non-sidecar file."
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_extracted_files_by_method.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dict whose keys are function identifier strings and whose values are non-empty lists of absolute file path strings. When func_dir exists as a directory: for every extracted function file found in a recursive traversal of func_dir (excluding metadata sidecar files), the file's absolute path is registered under its full file-stem identifier the filename without its extension, which may contain '::' class-qualifier separators; additionally, when a stem contains '::', the substring after the last '::' (the bare method tail) is registered as an additional key mapping to the same path list. A single bare method tail maps to all paths from different classes that share that tail. When func_dir does not exist or is not a directory: returns an empty dict.",
    "actual_behavior": "The function returns a collections.defaultdict instance `index`, whose default factory is `list`. If an exception (e.g., FileNotFoundError, PermissionError) is raised during the call to `os.path.isdir(func_dir)` or during `os.walk(func_dir)`, the function propagates that exception and no return occurs. If execution completes normally: If `os.path.isdir(func_dir)` is False, then `index` is empty (i.e., for all keys k, `index[k] == []`). Otherwise, let `D` be the set of absolute paths of all regular files under the directory tree rooted at `func_dir` (recursively) whose basename `fn` satisfies `_is_metadata_sidecar(fn) == False`. Let `stem(f)` for a file `f` in `D` be defined as: `fn[:fn.rfind('.')]` if `.` is in the basename `fn` of `f`, else `fn`. Let `bare(f) = stem(f).split(':')[0]`. Then `index` satisfies: for every key `k`, `index[k]` is a list containing exactly the absolute paths `f` in `D` such that `k == stem(f)` or (`k == bare(f)` and `bare(f) != stem(f)`). No other keys exist. The relative order of elements in each `index[k]` matches the order files are encountered by `os.walk` (depth-first, top-down). If `func_dir` is a directory but `D` is empty, all `index` values are empty lists.",
    "code_evidence": "Line 14: for root, _dirs, fnames in os.walk(func_dir):",
    "trigger_condition": "Specification requires returning a dict when func_dir exists as a directory. However, os.walk('/root') raises PermissionError for an unprivileged user, which the code propagates. This violates the specification because a dict is never returned."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_extracted_files_by_method",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_extracted_files_by_method.py",
  "function_name": "_extracted_files_by_method",
  "confirmation_status": "confirmed",
  "attempts": 2,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_extracted_files_by_method.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_extracted_files_by_method.md",
  "probe_stdout": "CONFIRMED - PermissionError from os.walk propagated instead of returning a dict",
  "trigger_summary": "os.walk raises PermissionError on an inaccessible directory and the exception propagates unhandled instead of returning a dict as the spec requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:47:00.534763Z	2142
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:47:28.027882Z	7559

事件 1 — [llm_aef28680b0a245b7b37d224deb811819](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a collections.defaultdict instance `index`, whose default factory is `list`. If an exception (e.g., FileNotFoundError, PermissionError) is raised during the call to `os.path.isdir(func_dir)` or during `os.walk(func_dir)`, the function propagates that exception and no return occurs. If execution completes normally: If `os.path.isdir(func_dir)` is False, then `index` is empty (i.e., for all keys k, `index[k] == []`). Otherwise, let `D` be the set of absolute paths of all regular files under the directory tree rooted at `func_dir` (recursively) whose basename `fn` satisfies `_is_metadata_sidecar(fn) == False`. Let `stem(f)` for a file `f` in `D` be defined as: `fn[:fn.rfind('.')]` if `.` is in the basename `fn` of `f`, else `fn`. Let `bare(f) = stem(f).split(':')[0]`. Then `index` satisfies: for every key `k`, `index[k]` is a list containing exactly the absolute paths `f` in `D` such that `k == stem(f)` or (`k == bare(f)` and `bare(f) != stem(f)`). No other keys exist. The relative order of elements in each `index[k]` matches the order files are encountered by `os.walk` (depth-first, top-down). If `func_dir` is a directory but `D` is empty, all `index` values are empty lists."
}
```

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "func_dir = '/root' on a Linux system where the current user is not root (so the directory exists, is a directory, but reading is denied).",
  "offending_statements": "Line 14: for root, _dirs, fnames in os.walk(func_dir):",
  "reason": "Specification requires returning a dict when func_dir exists as a directory. However, os.walk('/root') raises PermissionError for an unprivileged user, which the code propagates. This violates the specification because a dict is never returned."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_acc088e06c2e4d759f392d27cc114cfa_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_acc088e06c2e4d759f392d27cc114cfa_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_acc088e06c2e4d759f392d27cc114cfa_response.txt) ****复核动作****

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-227 — `src--incremental_reasoner-py--_llm_select_json`

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: `confirmed`; attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / `incremental_pipeline`; 源代码 `src/incremental_reasoner.py`
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Sends prompt_content to the configured LLM. Returns the parsed and validated result when the LLM produces output that can be parsed as JSON, conforms to schema_description, and passes validator. Returns None when the LLM produces no output, output that cannot be parsed as JSON, output that does not conform to schema_description, or output that fails validator. The exchange is traced under work_dir with stage and trace_meta as identifying labels. On return with None, an error-level log entry is produced noting the stage.
- **Actual behavior**: After execution of _llm_select_json, the function either terminates normally with a return value v, or raises an exception that propagates to the caller.

Normal termination post-condition: (1) (v is None) (v is not None validator(v)[0] = True v is a JSON-parsed object that conforms to schema_description). (2) The directory os.path.join(work_dir, 'trace') contains trace files from the LLM interaction(s), recorded with metadata {'stage': stage, 'summary': 'LLM ' + stage, **trace_meta}. (3) If v is None, a logging.error message containing stage is emitted.

Exceptional termination (exception raised by _llm_json_call): the exception propagates; there is no return value, and no guarantees are made about the completeness or existence of the trace data.

- **Code evidence**: Line 11: result = _llm_json_call(_llm_provider_client, LLM_MODEL, messages, validator, schema_description, trace_dir=os.path.join(work_dir, "trace"), trace_meta=meta,)
- **Trigger condition**: The specification states that the function returns None whenever the LLM does not produce a valid, conforming JSON result; there is no provision for raising exceptions. The code propagates any exception raised by _llm_json_call (e.g., a network or client error) directly to the caller, violating the expected contract.
- **Validator trigger**: Exceptions from _llm_json_call (e.g., network or client errors) propagate to the caller instead of being caught and returning None as the specification requires.
- **Probe stdout**: CONFIRMED — exception propagated to caller (expected: None returned)

► SPEC sidecar (完整)

```
{
  "signature": "_llm_select_json(work_dir, prompt_content, stage, validator, schema_description, trace_meta=None) -> Any | None",
  "pre_condition": "work_dir is a valid directory path. prompt_content is a non-empty string. stage is a non-empty string. validator is a callable that accepts a parsed value and returns a (bool, str | None) tuple where the bool indicates validity. schema_description is a non-empty string describing the expected output shape. If provided, trace_meta is a dict whose keys are strings and values are JSON-serializable.",
  "post_condition": "Sends prompt_content to the configured LLM. Returns the parsed and validated result when the LLM produces output that can be parsed as JSON, conforms to schema_description, and passes validator. Returns None when the LLM produces no output, output that cannot be parsed as JSON, output that does not conform to schema_description, or output that fails validator. The exchange is traced under work_dir with stage and trace_meta as identifying labels. On return with None, an error-level log
```

```
entry is produced noting the stage."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_llm_json_call",
      "signature": "_llm_json_call(client, model, messages, validator, schema_description, trace_dir, trace_meta) -> Any | None",
      "pre_condition": "client is a configured LLM provider client. model is a valid model identifier. messages is a non-empty list of message dicts each with 'role' and 'content' string keys. validator is a callable that accepts a parsed value and returns (bool, str | None). schema_description is a non-empty string. trace_dir is a writable directory path. trace_meta is a dict whose keys are strings and values are JSON-serializable.",
      "post_condition": "Sends messages to the LLM via client. Returns the parsed and validated result when the LLM produces output parseable as JSON that conforms to schema_description and passes validator. Returns None when the LLM produces no valid output or validation fails. Handles retries on protocol-level failures. The raw exchange is traced to trace_dir with trace_meta as identifying metadata."
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_llm_select_json.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Sends prompt_content to the configured LLM. Returns the parsed and validated result when the LLM produces output that can be parsed as JSON, conforms to schema_description, and passes validator. Returns None when the LLM produces no output, output that cannot be parsed as JSON, output that does not conform to schema_description, or output that fails validator. The exchange is traced under work_dir with stage and trace_meta as identifying labels. On return with None, an error-level log entry is produced noting the stage.",
    "actual_behavior": "After execution of _llm_select_json, the function either terminates normally with a return value v, or raises an exception that propagates to the caller. \n\nNormal termination post-condition: \n(1) (v is None) (v is not None validator(v)[0] = True v is a JSON-parsed object that conforms to schema_description). \n(2) The directory os.path.join(work_dir, 'trace') contains trace files from the LLM interaction(s), recorded with metadata {'stage': stage, 'summary': 'LLM ' + stage, **trace_meta}. \n(3) If v is None, a logging.error message containing stage is emitted. \n\nExceptional termination (exception raised by _llm_json_call): the exception propagates; there is no return value, and no guarantees are made about the completeness or existence of the trace data.",
    "code_evidence": "Line 11: result = _llm_json_call(\n          _llm_provider_client,\n          LLM_MODEL,\n          messages,\n          validator,\n          schema_description,\n          trace_dir=os.path.join(work_dir, \"trace\"),\n          trace_meta=meta,\n          )",
    "trigger_condition": "The specification states that the function returns None whenever the LLM does not produce a valid, conforming JSON result; there is no provision for raising exceptions. The code propagates any exception raised by _llm_json_call (e.g., a network or client error) directly to the caller, violating the expected contract."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_llm_select_json",
  "source_file": "src/incremental_reasoner-py/_llm_select_json.py",
  "function_name": "_llm_select_json",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_llm_select_json.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_llm_select_json.md",
  "probe_stdout": "CONFIRMED - exception propagated to caller (expected: None returned)",
  "trigger_summary": "Exceptions from _llm_json_call (e.g., network or client errors) propagate to the caller instead of being caught and returning None as the specification requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:24:06.066810Z	3286
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:25:00.772588Z	4268

事件 1 — [llm_cec4a2348ef744339713bbac7eeefdb5](#)

► `parsed_json` (完整)

```
{
  "post_condition": "After execution of _llm_select_json, the function either terminates normally with a return value v, or raises an exception that propagates to the caller. \n\nNormal termination post-condition: \n(1) (v is None) v (v is not None ^ validator(v)[0] = True ^ v is a JSON-parsed object that conforms to schema_description). \n(2) The directory os.path.join(work_dir, 'trace') contains trace files from the LLM interaction(s), recorded with metadata {'stage': stage, 'summary': 'LLM ' + stage, **trace_meta}. \n(3) If v is None, a logging.error message containing stage is emitted. \n\nExceptional termination (exception raised by _llm_json_call): the exception propagates; there is no return value, and no guarantees are made about the completeness or existence of the trace data."
}
```

- 载荷: `[system_prompt](fm_agent/trace/payloads/llm_cec4a2348ef744339713bbac7eeefdb5_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_cec4a2348ef744339713bbac7eeefdb5_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_cec4a2348ef744339713bbac7eeefdb5_response.txt)`

事件 2 — `llm_a9c42dbb33174a909370b4d8652293e3`

► `parsed_json` (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Set _llm_provider_client to a mock client whose chat.completions.create raises ValueError('invalid API key'). Call _llm_select_json(work_dir='/tmp', prompt_content='test', stage='test', validator=lambda x: (True,None), schema_description='desc'). The code executes _llm_json_call, which raises ValueError; it propagates out of _llm_select_json, causing an exception. The specification requires returning None and logging an error when the LLM produces no valid output (e.g., because of a client error).",
  "offending_statements": "Line 11: result = _llm_json_call(\n                _llm_provider_client,\n                LLM_MODEL,\n                messages,\n                validator,\n                schema_description,\n                trace_dir=os.path.join(work_dir, \"trace\"),\n                trace_meta=meta,\n                )",
  "reason": "The specification states that the function returns None whenever the LLM does not produce a valid, conforming JSON result; there is no provision for raising exceptions. The code propagates any exception raised by _llm_json_call (e.g., a network or client error) directly to the caller, violating the expected contract."
}
```

- 载荷: `[system_prompt](fm_agent/trace/payloads/llm_a9c42dbb33174a909370b4d8652293e3_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_a9c42dbb33174a909370b4d8652293e3_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_a9c42dbb33174a909370b4d8652293e3_response.txt)` **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-232 — `src--incremental_reasoner-py--reconcile_extracted_dir`

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator：confirmed; attempts 1
- Phase / 模块：Phase 6 — Reasoning & Verification Engine / `incremental_pipeline`；源代码 `src/incremental_reasoner.py`
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim**：Identifies the extracted-function directory under `extracted_functions/` corresponding to `abs_src`. When this directory does not exist on disk, returns immediately with no side effects. Otherwise, determines the set of function identifiers currently produced by the extraction backend for `abs_src`. For every file under the corresponding extracted-function directory whose base filename (with `.spec.json` or `.info.json` suffix stripped) does not match any such identifier, removes that file from disk including orphaned `.spec.json` and `.info.json` sidecars. When `abs_src` does not exist on disk, removes the entire corresponding extracted-function directory and all files within it. When `abs_src` exists but yields no function identifiers, removes all extracted function files and their sidecars from the corresponding directory. After removal, prunes every empty subdirectory under the extracted-function directory (deepest-first traversal). The extracted-function directory itself is preserved even when emptied. Does not modify or create any source file at `abs_src`. The set of function identifiers is computed using the same backend (codegraph or regex) that produced the original extracted-function files, ensuring identifier naming is consistent.
- Actual behavior**：After the function executes, if it returns normally (no exception), the state of the directory tree rooted at `func_dir` (derived from `proj_dir` and `abs_src`) is updated as follows.

Let `D`, `ext` = `_src_rel_to_func_dir(proj_dir, abs_src)`. If `D` is not a directory, the function returns immediately and the file system is unchanged.

Otherwise, let VALID be computed as: lang_key = EXT_TO_LANG.get(ext) if lang_key is not None and os.path.isfile(abs_src) in the initial state: spans = _function_spans(abs_src, lang_key, proj_dir)[0] For each (ident, _, _) in spans: function_path = os.path.abspath(os.path.join(D, ident) + ('.' + ext if ext else '')) VALID = {function_path, function_path+'.spec.json', function_path+'.info.json'} for all spans, unioned. else: VALID = empty set.

Then the function walks D recursively and deletes every regular file whose absolute path is not in VALID. After that, it walks D bottom-up and removes any subdirectory of D (excluding D itself) that is empty (i.e., os.listdir returns empty).

Upon normal termination, the final file system state F satisfies: For all absolute paths p under D: * If p was a regular file in the initial state: p exists in F iff p VALID. * If p was a directory in the initial state: p exists in F iff p = D or p contains at least one regular file or subdirectory that exists in F.

If an exception (e.g., OSError, PermissionError) is raised during deletion or pruning, the function terminates prematurely; in that case the file system may be partially modified (some files/directories deleted, others not) and the exception propagates.

- **Code evidence:** Line 25-33: The deletion and pruning logic removes files and empty subdirectories but never removes func_dir itself when abs_src does not exist.
- **Trigger condition:** The specification explicitly states: 'When abs_src does not exist on disk, removes the entire corresponding extracted-function directory and all files within it.' The code only empties the directory, leaving an empty directory behind, which violates this requirement.
- **Validator trigger:** abs_src does not exist on disk; _reconcile_extracted_dir empties func_dir but never removes the directory itself, violating the spec requirement of full removal.
- **Probe stdout:** CONFIRMED — func_dir still exists after reconcile (spec requires removal)

► SPEC sidecar (完整)

```
{
  "signature": "_reconcile_extracted_dir(proj_dir, abs_src) -> None",
  "pre_condition": "proj_dir is a valid directory path. abs_src is an absolute path to a source file path (may point to a file that does not exist on disk).",
  "post_condition": "Identifies the extracted-function directory under extracted_functions/ corresponding to abs_src. When this directory does not exist on disk, returns immediately with no side effects. Otherwise, determines the set of function identifiers currently produced by the extraction backend for abs_src. For every file under the corresponding extracted-function directory whose base filename (with .spec.json or .info.json suffix stripped) does not match any such identifier, removes that file from disk – including orphaned .spec.json and .info.json sidecars. When abs_src does not exist on disk, removes the entire corresponding extracted-function directory and all files within it. When abs_src exists but yields no function identifiers, removes all extracted function files and their sidecars from the corresponding directory. After removal, prunes every empty subdirectory under the extracted-function directory (deepest-first traversal). The extracted-function directory itself is preserved even when emptied. Does not modify or create any source file at abs_src. The set of function identifiers is computed using the same backend (codegraph or regex) that produced the original extracted-function files, ensuring identifier naming is consistent."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_src_rel_to_func_dir",
      "signature": "_src_rel_to_func_dir(proj_dir, abs_src)",
      "pre_condition": "proj_dir is a valid directory path. abs_src is an absolute path (may not exist on disk).",
      "post_condition": "Returns a (func_dir, ext) tuple where func_dir is the absolute directory path under extracted_functions/ corresponding to abs_src's position in the project tree (derived from the relative path of abs_src with respect to proj_dir), and ext is the file extension of abs_src (including the dot, e.g. '.py') or an empty string when abs_src has no extension."
    },
    {
      "name": "_function_spans",
      "signature": "_function_spans(abs_src, lang_key, proj_dir)",
      "pre_condition": "abs_src is an absolute path to an existing regular file on disk. lang_key is a non-empty string recognized by the extraction backend registry (e.g., 'python', 'rust'). proj_dir is a valid directory path.",
      "post_condition": "Returns (spans, raw) where raw is backend-specific parse data. spans is an iterable of (identifier, start_line, end_line) tuples, where each identifier is the class-qualified canonical function name produced by the extraction backend (using ':' as separator for nested scopes), start_line is the 1-based line number where the function definition begins, and end_line is the 1-based inclusive line number where the function definition ends. The identifiers returned are consistent with the naming convention used by run_extraction to write extracted-function files."
    }
  ]
}
```

► Logic verification result (完整)


```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_reconcile_extracted_dir.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Identifies the extracted-function directory under extracted_functions/ corresponding to abs_src. When this
directory does not exist on disk, returns immediately with no side effects. Otherwise, determines the set of function identifiers
currently produced by the extraction backend for abs_src. For every file under the corresponding extracted-function directory
whose base filename (with .spec.json or .info.json suffix stripped) does not match any such identifier, removes that file from
disk including orphaned .spec.json and .info.json sidecars. When abs_src does not exist on disk, removes the entire corresponding
extracted-function directory and all files within it. When abs_src exists but yields no function identifiers, removes all
extracted function files and their sidecars from the corresponding directory. After removal, prunes every empty subdirectory under
the extracted-function directory (deepest-first traversal). The extracted-function directory itself is preserved even when
emptied. Does not modify or create any source file at abs_src. The set of function identifiers is computed using the same backend
(codegraph or regex) that produced the original extracted-function files, ensuring identifier naming is consistent.",
    "actual_behavior": "After the function executes, if it returns normally (no exception), the state of the directory tree rooted
at func_dir (derived from proj_dir and abs_src) is updated as follows.\n\nLet D, ext = _src_rel_to_func_dir(proj_dir,
abs_src).\nIf D is not a directory, the function returns immediately and the file system is unchanged.\n\nOtherwise, let VALID be
computed as:\n  lang_key = EXT_TO_LANG.get(ext)\n  if lang_key is not None and os.path.isfile(abs_src) in the initial state:\n    spans = _function_spans(abs_src, lang_key, proj_dir)[0]\n    For each (ident, _, _) in spans:\n      function_path =
os.path.abspath(os.path.join(D, ident) + ('.' + ext if ext else ''))\n      VALID = {function_path,
function_path+'.spec.json', function_path+'.info.json'} for all spans, unioned.\n  else:\n    VALID = empty set.\n\nThen the
function walks D recursively and deletes every regular file whose absolute path is not in VALID. After that, it walks D bottom-up
and removes any subdirectory of D (excluding D itself) that is empty (i.e., os.listdir returns empty).\n\nUpon normal termination,
the final file system state F satisfies:\n  For all absolute paths p under D:\n    * If p was a regular file in the initial state:
p exists in F iff p  VALID.\n    * If p was a directory in the initial state: p exists in F iff p = D or p contains at least one
regular file or subdirectory that exists in F.\n\nIf an exception (e.g., OSError, PermissionError) is raised during deletion or
pruning, the function terminates prematurely; in that case the file system may be partially modified (some files/directories
deleted, others not) and the exception propagates.",
    "code_evidence": "Line 25-33: The deletion and pruning logic removes files and empty subdirectories but never removes func_dir
itself when abs_src does not exist.",
    "trigger_condition": "The specification explicitly states: 'When abs_src does not exist on disk, removes the entire
corresponding extracted-function directory and all files within it.' The code only empties the directory, leaving an empty
directory behind, which violates this requirement."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_reconcile_extracted_dir",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_reconcile_extracted_dir.py",
  "function_name": "_reconcile_extracted_dir",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_reconcile_extracted_dir.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_reconcile_extracted_dir.md",
  "probe_stdout": "CONFIRMED - func_dir still exists after reconcile (spec requires removal)",
  "trigger_summary": "abs_src does not exist on disk; _reconcile_extracted_dir empties func_dir but never removes the directory
itself, violating the spec requirement of full removal."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:27:38.390894Z	5225
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:28:54.392760Z	6188

事件 1 — [llm_9e1a4d11c99d47b3af7c3270bafbfc2a](#)

► parsed_json（完整）

```
{
  "post_condition": "After the function executes, if it returns normally (no exception), the state of the directory tree rooted at
func_dir (derived from proj_dir and abs_src) is updated as follows.\n\nLet D, ext = _src_rel_to_func_dir(proj_dir, abs_src).\nIf D
is not a directory, the function returns immediately and the file system is unchanged.\n\nOtherwise, let VALID be computed as:\n
lang_key = EXT_TO_LANG.get(ext)\n  if lang_key is not None and os.path.isfile(abs_src) in the initial state:\n    spans =
_function_spans(abs_src, lang_key, proj_dir)[0]\n    For each (ident, _, _) in spans:\n      function_path =
os.path.abspath(os.path.join(D, ident) + ('.' + ext if ext else ''))\n      VALID = {function_path,
function_path+'.spec.json', function_path+'.info.json'} for all spans, unioned.\n  else:\n    VALID = empty set.\n\nThen the
function walks D recursively and deletes every regular file whose absolute path is not in VALID. After that, it walks D bottom-up
and removes any subdirectory of D (excluding D itself) that is empty (i.e., os.listdir returns empty).\n\nUpon normal termination,
the final file system state F satisfies:\n  For all absolute paths p under D:\n    * If p was a regular file in the initial state:
```

```
p exists in F iff p ∈ VALID.\nexists * If p was a directory in the initial state: p exists in F iff p = D or p contains at least one regular file or subdirectory that exists in F.\n\nIf an exception (e.g., OSError, PermissionError) is raised during deletion or pruning, the function terminates prematurely; in that case the file system may be partially modified (some files/directories deleted, others not) and the exception propagates."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_9e1a4d11c99d47b3af7c3270bafbfc2a_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_9e1a4d11c99d47b3af7c3270bafbfc2a_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_9e1a4d11c99d47b3af7c3270bafbfc2a_response.txt)

事件 2 — [llm_c0f55b7093a34c68b6cb704ccd49061d](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/project', abs_src='/project/deleted.py'. The source file does not exist. The extracted-function directory (as returned by _src_rel_to_func_dir) exists and contains a file 'foo.py'. After _reconcile_extracted_dir runs, 'foo.py' is removed but the directory itself remains empty. The specification requires the entire directory to be removed.",
  "offending_statements": "Line 25-33: The deletion and pruning logic removes files and empty subdirectories but never removes func_dir itself when abs_src does not exist.",
  "reason": "The specification explicitly states: 'When abs_src does not exist on disk, removes the entire corresponding extracted-function directory and all files within it.' The code only empties the directory, leaving an empty directory behind, which violates this requirement."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c0f55b7093a34c68b6cb704ccd49061d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c0f55b7093a34c68b6cb704ccd49061d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c0f55b7093a34c68b6cb704ccd49061d_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-235 — [src--incremental_reasoner.py--_src_rel_to_func_dir](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** [confirmed](#); attempts 1
- **Phase / 模块:** Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#); 原源码 [src/incremental_reasoner.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a tuple (func_dir, ext) of two strings. func_dir is the absolute path formed by joining the prefix '<proj_dir>/fm_agent/extracted_functions/' with the directory portion of the path of abs_src relative to proj_dir and a transformed basename component. The basename transformation: when the basename of abs_src contains a period character, the portion before the last period is joined by a hyphen to the portion after the last period (e.g., 'loader.cpp' becomes 'loader-cpp'); when the basename contains no period, the basename appears unchanged. ext is the substring of the basename after the last period, excluding the period itself; ext is the empty string when the basename contains no period.
- **Actual behavior:** After execution, the function returns a tuple ((func_dir, ext)) with no side effects. Let (rel = os.path.relpath(abs_src, proj_dir)), (src_dir = os.path.dirname(rel)), (base = os.path.basename(rel)), and (dot = base.rfind(".")). If (dot > 0), then (dir_name = base[:dot] + "-" + base[dot+1:]) and (ext = base[dot+1:]); otherwise (dir_name = base) and (ext = ""). Let (extracted_base = os.path.join(proj_dir, "fm_agent", "extracted_functions")). Then (func_dir = os.path.join(extracted_base, src_dir, dir_name)) if (src_dir != ""), else (os.path.join(extracted_base, dir_name)). The returned (func_dir) is a string representing a filesystem path, and (ext) is a string (possibly empty) representing the derived file extension.
- **Code evidence:** Line 9: if last_dot > 0:
- **Trigger condition:** When the basename contains a period at the start (e.g., '.env'), last_dot equals 0, so the condition fails and the code takes the else branch, returning the original basename as dir_name and an empty ext. The specification, however, requires that any basename containing a period be transformed: the empty string before the last period and 'env' after should be joined with a hyphen, giving '-env', and ext should be 'env'. Thus the code's output violates the specification.
- **Validator trigger:** When the basename of abs_src starts with a period (e.g., '.env'), last_dot equals 0 so the condition 'last_dot > 0' fails, causing the else branch to return the original basename unchanged instead of transforming it per spec.
- **Probe stdout:** CONFIRMED — actual: dir_name='.env', ext="" | expected: dir_name='-env', ext='env'

► SPEC sidecar (完整)

```
{
  "signature": "_src_rel_to_func_dir(proj_dir, abs_src) -> tuple",
  "pre_condition": "proj_dir is a string naming a directory path. abs_src is a string naming an absolute file path.",
  "post_condition": "Returns a tuple (func_dir, ext) of two strings. func_dir is the absolute path formed by joining the prefix '<proj_dir>/fm_agent/extracted_functions/' with the directory portion of the path of abs_src relative to proj_dir and a transformed basename component. The basename transformation: when the basename of abs_src contains a period character, the portion before the last period is joined by a hyphen to the portion after the last period (e.g., 'loader.cpp' becomes 'loader-cpp'); when the basename contains no period, the basename appears unchanged. ext is the substring of the basename after the last period, excluding the period itself; ext is the empty string when the basename contains no period."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_src_rel_to_func_dir.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a tuple (func_dir, ext) of two strings. func_dir is the absolute path formed by joining the prefix '<proj_dir>/fm_agent/extracted_functions/' with the directory portion of the path of abs_src relative to proj_dir and a transformed basename component. The basename transformation: when the basename of abs_src contains a period character, the portion before the last period is joined by a hyphen to the portion after the last period (e.g., 'loader.cpp' becomes 'loader-cpp'); when the basename contains no period, the basename appears unchanged. ext is the substring of the basename after the last period, excluding the period itself; ext is the empty string when the basename contains no period.",
    "actual_behavior": "After execution, the function returns a tuple \\( (func\\_dir, ext) \\) with no side effects. Let \\( rel = os.path.relpath(abs\\_src, proj\\_dir) \\), \\( src\\_dir = os.path.dirname(rel) \\), \\( base = os.path.basename(rel) \\), and \\( dot = base.rfind('.') \\). If \\( dot > 0 \\), then \\( dir\\_name = base[:dot] + '-' + base[dot+1:] \\) and \\( ext = base[dot+1:] \\); otherwise \\( dir\\_name = base \\) and \\( ext = '' \\). Let \\( extracted\\_base = os.path.join(proj\\_dir, 'fm\\_agent', 'extracted\\_functions') \\). Then \\( func\\_dir = os.path.join(extracted\\_base, src\\_dir, dir\\_name) \\) if \\( src\\_dir != '' \\), else \\( os.path.join(extracted\\_base, dir\\_name) \\). The returned \\( func\\_dir \\) is a string representing a filesystem path, and \\( ext \\) is a string (possibly empty) representing the derived file extension.",
    "code_evidence": "Line 9: if last_dot > 0:",
    "trigger_condition": "When the basename contains a period at the start (e.g., '.env'), last_dot equals 0, so the condition fails and the code takes the else branch, returning the original basename as dir_name and an empty ext. The specification, however, requires that any basename containing a period be transformed: the empty string before the last period and 'env' after should be joined with a hyphen, giving '-env', and ext should be 'env'. Thus the code's output violates the specification."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_src_rel_to_func_dir",
  "source_file": "src/incremental_reasoner.py",
  "function_name": "_src_rel_to_func_dir",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_src_rel_to_func_dir.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_src_rel_to_func_dir.md",
  "probe_stdout": "CONFIRMED - actual: dir_name='.env', ext='' | expected: dir_name='-env', ext='env'",
  "trigger_summary": "When the basename of abs_src starts with a period (e.g., '.env'), last_dot equals 0 so the condition 'last_dot > 0' fails, causing the else branch to return the original basename unchanged instead of transforming it per spec."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:48:07.252291Z	2922
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:48:44.789402Z	3737

事件 1 — [llm_829215f07daf446e8edcbb1a658f611d](#)

► parsed_json（完整）

```
{
  "post_condition": "After execution, the function returns a tuple \\( (func\\_dir, ext) \\) with no side effects. Let \\( rel = os.path.relpath(abs\\_src, proj\\_dir) \\), \\( src\\_dir = os.path.dirname(rel) \\), \\( base = os.path.basename(rel) \\), and \\( dot = base.rfind(\".\") \\). If \\( dot > 0 \\), then \\( dir\\_name = base[:dot] + \"-\" + base[dot+1:] \\) and \\( ext = base[dot+1:] \\); otherwise \\( dir\\_name = base \\) and \\( ext = \"\" \\). Let \\( extracted\\_base = os.path.join(proj\\_dir, \"fm\\_agent\", \"extracted\\_functions\") \\). Then \\( func\\_dir = os.path.join(extracted\\_base, src\\_dir, dir\\_name) \\) if \\( src\\_dir \\neq \"\" \\), else \\( os.path.join(extracted\\_base, dir\\_name) \\). The returned \\( func\\_dir \\) is a string representing a filesystem path, and \\( ext \\) is a string (possibly empty) representing the derived file extension."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_829215f07daf446e8edcbb1a658f611d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_829215f07daf446e8edcbb1a658f611d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_829215f07daf446e8edcbb1a658f611d_response.txt)

事件 2 — llm_5d2e93b47dde4eb3b8bad8cec417f8ab

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/home/user', abs_src='/home/user/.env'",
  "offending_statements": "Line 9: if last_dot > 0:",
  "reason": "When the basename contains a period at the start (e.g., '.env'), last_dot equals 0, so the condition fails and the code takes the else branch, returning the original basename as dir_name and an empty ext. The specification, however, requires that any basename containing a period be transformed: the empty string before the last period and 'env' after should be joined with a hyphen, giving '-env', and ext should be 'env'. Thus the code's output violates the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5d2e93b47dde4eb3b8bad8cec417f8ab_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5d2e93b47dde4eb3b8bad8cec417f8ab_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5d2e93b47dde4eb3b8bad8cec417f8ab_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-238 — src--incremental_reasoner-py--_update_specs_for_intent::_plan_spec_update

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: confirmed; attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / incremental_pipeline; 源代码 src/incremental_reasoner.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Returns None when any of the following holds: (a) fq_n has no entry in file_map or the mapped file does not exist on disk, (b) the file extension corresponds to no recognized language in EXT_TO_LANG, or (c) the function already has a valid specification and that specification remains correct under the developer_intent. Otherwise returns a plan dict containing: 'fq_n' (the FQN unchanged), 'fpath' (the absolute file path), 'spec_dict' (a dict with 'signature', 'pre_condition', 'post_condition' keys describing the function's intended behavioral contract), 'info_dict' (a dict with a 'callees' key listing expected callee contracts), 'info_updated' (a boolean that is true when the callee-info dict was generated or changed from its prior version), and 'updated_callees' (a list of callee FQN strings whose expected contracts differ from the previous run). The returned dict is a pure data record no files are written or modified by this function.
- **Actual behavior**: After _plan_spec_update completes, one of the following holds:

1. The function returns None because:

- file_map.get(fq_n) is None or the corresponding path does not exist or is not a regular file, or
- the file extension cannot be mapped to a language key via EXT_TO_LANG, or
- after reading the source and possibly existing .spec.json/.info.json files, calling _opcode_generate_spec (when no prior spec exists) or _llm_check_spec_update (when prior spec and info exist) yields a falsy result or a result where result.get('spec_updated') is not True, or
- result.get('new_spec') is not a dictionary.

2. The function returns a dictionary plan with keys:

- 'fq_n': the input fq_n,
- 'fpath': the resolved source file path,
- 'spec_dict': result of _normalize_spec_dict(new_spec) where new_spec is the updated specification dict,

- `'info_dict': result of normalize_info_dict(new_info) where new_info is the (possibly newly generated) callee information dict, determined as:`
 - if `old_info` was `None`, `new_info = result.get('new_info')` if it is a dict, else `{'callees': []}`, and `info_updated = True`;
 - otherwise `info_updated = bool(result.get('info_updated'))`; if `info_updated` is true, `new_info = result.get('new_info')` if that is a dict, else `new_info = old_info`; if `info_updated` is false, `new_info = old_info`.
 - `'info_updated'`: boolean as determined above,
 - `'updated_callees': result.get('updated_callees')` or `[]`.
3. An exception (e.g., `OSError` when reading the source file, or any exception thrown by `_collect_caller_context`, the LLM functions, or the normalization helpers) is raised and not caught, propagating to the caller. In this case no return value is produced.

The function performs only read I/O; it does not write to the file system or modify the global mappings `file_map`, `callers_map`, `callees_map`, `edge_aliases_map`, `EXT_TO_LANG`, `proj_dir`, `work_dir`, or `developer_intent`.

- **Code evidence:** Line 42: if not result or not result.get("spec_updated"): Line 43: return None
- **Trigger condition:** Specification requires returning None only when conditions (a), (b), or (c) hold. When no prior specification exists, condition (c) does not apply, and (a) and (b) are false. Thus the function must return a plan dict. The code returns None because result is falsy, violating the specification.
- **Validator trigger:** When `_opencode_generate_spec` returns a falsy value for a function with no prior spec, `_plan_spec_update` returns None instead of a plan dict, silently skipping the function.
- **Probe stdout:** CONFIRMED — Both approaches reproduce the bug. Approach 1 (real code): CONFIRMED — `_update_specs_for_intent` returned an empty list, meaning `_plan_spec_update` returned None and the function was silently skipped. When (a) fpath exists, (b) extension is valid ('py'->'python'), and (c) no prior spec exists (old_spec is None), the spec requires returning a plan dict, not None. The bug is at line 1843-1844: 'if not result or not result.get("spec_updated"): return None' returns None even when `_opencode_generate_spec` returned None (falsy), violating the specification. Approach 2 (mirrored logic): CONFIRMED — The mirrored logic at lines 1843-1844 returns None when `_opencode_generate_spec` returns a falsy value (None), but the specification requires returning a plan dict because conditions (a), (b), and (c) are all false. The silent None return causes the function to be skipped in the caller's filtering at line 1961: 'applied = [p for p in plans if p]'.

► SPEC sidecar (完整)

```
{
  "signature": "_plan_spec_update(fqn, idx) -> dict | None",
  "pre_condition": "fqn is a fully-qualified function name string; idx is a non-negative integer; the module-level file_map maps FQNs to absolute file paths; callees_map maps FQNs to their callee FQNs; callers_map maps FQNs to their caller FQNs; developer_intent is a string describing the intended code modification; EXT_TO_LANG maps file extensions to recognized language keys; proj_dir and work_dir are valid directory paths.",
  "post_condition": "Returns None when any of the following holds: (a) fqn has no entry in file_map or the mapped file does not exist on disk, (b) the file extension corresponds to no recognized language in EXT_TO_LANG, or (c) the function already has a valid specification and that specification remains correct under the developer_intent. Otherwise returns a plan dict containing: 'fqn' (the FQN unchanged), 'fpath' (the absolute file path), 'spec_dict' (a dict with 'signature', 'pre_condition', 'post_condition' keys describing the function's intended behavioral contract), 'info_dict' (a dict with a 'callees' key listing expected callee contracts), 'info_updated' (a boolean that is true when the callee-info dict was generated or changed from its prior version), and 'updated_callees' (a list of callee FQN strings whose expected contracts differ from the previous run). The returned dict is a pure data record – no files are written or modified by this function."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_collect_caller_context",
      "signature": "_collect_caller_context(fqn, callers_map, file_map, edge_aliases_map)",
      "pre_condition": "fqn is a FQN present in file_map pointing to an existing file; callers_map maps FQNs to lists of their callers; file_map maps FQNs to file paths; edge_aliases_map maps callee names to alias entries.",
      "post_condition": "Returns a data structure describing what the callers of fqn expect from the function – the caller-side context used to guide behavioral specification generation."
    },
    {
      "name": "_opencode_generate_spec",
      "signature": "_opencode_generate_spec(proj_dir, work_dir, idx, fqn, lang_key, phase_label, developer_intent, callee_names, source, caller_context)",
      "pre_condition": "proj_dir and work_dir are valid directory paths; fqn identifies an existing function; lang_key is a recognized language identifier; source is the function's complete source code text; callee_names is a list of callee function name strings; caller_context describes caller expectations.",
      "post_condition": "Returns a dict. When 'spec_updated' is true: 'new_spec' contains a specification dict with 'signature', 'pre_condition', 'post_condition' keys, and 'new_info' contains a callee-spec dict with a 'callees' key. When generation is not"
    }
  ]
}
```



```

possible, returns a falsy or incomplete result."
},
{
  "name": "_llm_check_spec_update",
  "signature": "_llm_check_spec_update(proj_dir, work_dir, idx, fqn, lang_key, phase_label, developer_intent, old_spec,
old_info, callee_names, source)",
  "pre_condition": "old_spec is a dict with 'signature', 'pre_condition', 'post_condition' keys from a previous specification
run; old_info is a callee-spec dict from a previous run with a 'callees' key; developer_intent is a non-empty string describing
the code change; all other arguments are as in _opencode_generate_spec.",
  "post_condition": "Returns a dict. When 'spec_updated' is true: 'new_spec' contains the updated specification and
'info_updated' (boolean) and 'new_info' (dict) indicate whether callee expectations changed. When 'spec_updated' is false, the
existing specification remains correct under the developer_intent and no update is needed. The key 'updated_callees' lists the
FQNs of callees whose expected contracts changed."
},
{
  "name": "_normalize_spec_dict",
  "signature": "_normalize_spec_dict(spec_dict) -> dict",
  "pre_condition": "spec_dict is a dict containing at minimum the keys 'signature', 'pre_condition', and 'post_condition'.",
  "post_condition": "Returns a dict with all required specification keys present and values conforming to the canonical
.spec.json schema expected by downstream consumers."
},
{
  "name": "_normalize_info_dict",
  "signature": "_normalize_info_dict(info_dict) -> dict",
  "pre_condition": "info_dict is a dict containing a 'callees' key whose value is a list of callee-specification objects.",
  "post_condition": "Returns a dict conforming to the canonical .info.json schema with all callee entries normalized to the
expected structure."
}
]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_update_specs_for_intent::_plan_spec_update.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns None when any of the following holds: (a) fqn has no entry in file_map or the mapped file does not
exist on disk, (b) the file extension corresponds to no recognized language in EXT_TO_LANG, or (c) the function already has a
valid specification and that specification remains correct under the developer_intent. Otherwise returns a plan dict containing:
'fqn' (the FQN unchanged), 'fpath' (the absolute file path), 'spec_dict' (a dict with 'signature', 'pre_condition',
'post_condition' keys describing the function's intended behavioral contract), 'info_dict' (a dict with a 'callees' key listing
expected callee contracts), 'info_updated' (a boolean that is true when the callee-info dict was generated or changed from its
prior version), and 'updated_callees' (a list of callee FQN strings whose expected contracts differ from the previous run). The
returned dict is a pure data record no files are written or modified by this function.",
    "actual_behavior": "After `_plan_spec_update` completes, one of the following holds:\n1. The function returns `None`
because:\n - `file_map.get(fqn)` is `None` or the corresponding path does not exist or is not a regular file, or\n - the file
extension cannot be mapped to a language key via `EXT_TO_LANG`, or\n - after reading the source and possibly existing
`.spec.json`/`.info.json` files, calling `_opencode_generate_spec` (when no prior spec exists) or `_llm_check_spec_update` (when
prior spec and info exist) yields a falsy result or a result where `result.get('spec_updated')` is not `True`, or\n -
`result.get('new_spec')` is not a dictionary.\n2. The function returns a dictionary `plan` with keys:\n - `fqn`: the input
`fqn`,\n - `fpath`: the resolved source file path,\n - `spec_dict`: result of `_normalize_spec_dict(new_spec)` where
`new_spec` is the updated specification dict,\n - `info_dict`: result of `_normalize_info_dict(new_info)` where `new_info` is
the (possibly newly generated) callee information dict, determined as:\n      * if `old_info` was `None`, `new_info` =
result.get('new_info')` if it is a dict, else `{'callees': []}`, and `info_updated` = True;\n      * otherwise `info_updated` =
bool(result.get('info_updated'));\n if `info_updated` is true, `new_info` = result.get('new_info')` if that is a dict, else
`new_info` = old_info;\n if `info_updated` is false, `new_info` = old_info.\n - `info_updated`: boolean as determined above,\n
- `updated_callees`: `result.get('updated_callees')` or `[]`.\n3. An exception (e.g., `OSError` when reading the source file, or
any exception thrown by `_collect_caller_context`, the LLM functions, or the normalization helpers) is raised and not caught,
propagating to the caller. In this case no return value is produced.\n\nThe function performs only read I/O; it does not write to
the file system or modify the global mappings `file_map`, `callers_map`, `callees_map`, `edge_aliases_map`, `EXT_TO_LANG`,
`proj_dir`, `work_dir`, or `developer_intent`.",
    "code_evidence": "Line 42:         if not result or not result.get(\"spec_updated\"): \nLine 43:             return None",
    "trigger_condition": "Specification requires returning None only when conditions (a), (b), or (c) hold. When no prior
specification exists, condition (c) does not apply, and (a) and (b) are false. Thus the function must return a plan dict. The code
returns None because result is falsy, violating the specification."
  }
}

```

► Bug Validator result (完整)

```

{
  "id": "src--incremental_reasoner-py--_update_specs_for_intent::_plan_spec_update",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_update_specs_for_intent::_plan_spec_update.py",
  "function_name": "_plan_spec_update",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_update_specs_for_intent::_plan_spec_update.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_update_specs_for_intent::_plan_spec_update.md",
  "probe_stdout": "CONFIRMED - Both approaches reproduce the bug.\n Approach 1 (real code): CONFIRMED - _update_specs_for_intent

```


returned an empty list, meaning `_plan_spec_update` returned None and the function was silently skipped. When (a) `fpath` exists, (b) extension is valid ('py' -> 'python'), and (c) no prior spec exists (`old_spec` is None), the spec requires returning a plan dict, not None. The bug is at line 1843-1844: 'if not result or not result.get(\"spec_updated\"): return None' returns None even when `_opencode_generate_spec` returned None (falsy), violating the specification. Approach 2 (mirrored logic): CONFIRMED – The mirrored logic at lines 1843-1844 returns None when `_opencode_generate_spec` returns a falsy value (None), but the specification requires returning a plan dict because conditions (a), (b), and (c) are all false. The silent None return causes the function to be skipped in the caller's filtering at line 1961: 'applied = [p for p in plans if p]'.",

"trigger_summary": "When `_opencode_generate_spec` returns a falsy value for a function with no prior spec, `_plan_spec_update` returns None instead of a plan dict, silently skipping the function."

```
}

```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 `system/user/assistant` 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:59:32.082508Z	5713
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:00:25.097874Z	5555

事件 1 — [llm_102314e876aa4d7799022d9aa635f924](#)

► parsed_json（完整）

```
{
  "post_condition": "After `_plan_spec_update` completes, one of the following holds:\n1. The function returns `None` because:\n- `file_map.get(fqn)` is `None` or the corresponding path does not exist or is not a regular file, or\n- the file extension cannot be mapped to a language key via `EXT_TO_LANG`, or\n- after reading the source and possibly existing\n`.spec.json`/`.info.json` files, calling `_opencode_generate_spec` (when no prior spec exists) or `_llm_check_spec_update` (when prior spec and info exist) yields a falsy result or a result where `result.get('spec_updated')` is not `True`, or\n- `result.get('new_spec')` is not a dictionary.\n2. The function returns a dictionary `plan` with keys:\n- `fqn`: the input `fqn`,\n- `fpath`: the resolved source file path,\n- `spec_dict`: result of `_normalize_spec_dict(new_spec)` where `new_spec` is the updated specification dict,\n- `info_dict`: result of `_normalize_info_dict(new_info)` where `new_info` is the (possibly newly generated) callee information dict, determined as:\n    * if `old_info` was `None`, `new_info` = `result.get('new_info')` if it is a dict, else `{'callees': []}`, and `info_updated` = `True`;\n    * otherwise `info_updated` = `bool(result.get('info_updated'))`; if `info_updated` is true, `new_info` = `result.get('new_info')` if that is a dict, else `new_info` = `old_info`; if `info_updated` is false, `new_info` = `old_info`.\n- `info_updated`: boolean as determined above,\n- `updated_callees`: `result.get('updated_callees')` or `[]`.\n3. An exception (e.g., `OSError` when reading the source file, or any exception thrown by `_collect_caller_context`, the LLM functions, or the normalization helpers) is raised and not caught, propagating to the caller. In this case no return value is produced.\n\nThe function performs only read I/O; it does not write to the file system or modify the global mappings `file_map`, `callers_map`, `callees_map`, `edge_aliases_map`, `EXT_TO_LANG`, `proj_dir`, `work_dir`, or `developer_intent`."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_102314e876aa4d7799022d9aa635f924_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_102314e876aa4d7799022d9aa635f924_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_102314e876aa4d7799022d9aa635f924_response.txt)

事件 2 — [llm_43d761d32f5f46b5812bfeed94c01ee](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "fqn = 'mylib.myfunc'; file_map contains 'mylib.myfunc' -> '/path/to/myfunc.py' with file existing; extension 'py' maps to a recognized language in EXT_TO_LANG; the files myfunc.py.spec.json and myfunc.py.info.json do not exist (so old_spec and old_info are None); _opencode_generate_spec returns None (fails to generate).",
  "offending_statements": "Line 42:         if not result or not result.get(\"spec_updated\"):\\nLine 43:             return None",
  "reason": "Specification requires returning None only when conditions (a), (b), or (c) hold. When no prior specification exists, condition (c) does not apply, and (a) and (b) are false. Thus the function must return a plan dict. The code returns None because result is falsy, violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_43d761d32f5f46b5812bfeed94c01ee_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_43d761d32f5f46b5812bfeed94c01ee_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_43d761d32f5f46b5812bfeed94c01ee_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论：契约不确定

- **分类依据:** [probe](#) 已复现 SPEC 与实现的差异, 但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#); 原源码 [src/incremental_reasoner.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Each (callee_name, callee_new_spec) pair in updates is processed sequentially. For each pair, determines whether the caller's existing .info.json callee entries must be revised to remain consistent with callee_new_spec the evaluation considers the caller's source code, its current .info.json, and the callee's new specification. When a revision is required, the caller's entire .info.json is replaced with a reconciled version whose callee entries are consistent with callee_new_spec and all previously reconciled callee entries. When no revision is required for any pair, the .info.json is unmodified. Returns the caller's absolute source file path as a string if at least one .info.json replacement occurred; returns None when the caller source file is absent, its programming language cannot be determined, its .info.json is unreadable, or no replacements were applied.
- **Actual behavior:** After the function completes normally (i.e., without raising an exception), the following holds:

Let cpath = file_map.get(caller_fqn). If cpath is not a nonempty string that refers to an existing file, or if the language cannot be determined (clang falsy), the function returns None and the sidecar file cpath + '.info.json' is unmodified.

Otherwise, let n = len(updates). For each i from 0 to n-1, let (cname_i, cspec_i) = updates[i]. During iteration i:

- The source code is read from cpath. If an OSError occurs, the function terminates with that exception.
- The sidecar info is loaded. If OSError or JSONDecodeError, the iteration continues to i+1.
- cresult_i = _llm_check_caller_info_update(...). If cresult_i is None or cresult_i.get('info_updated') is not true, continue.
- c_new_info_i = cresult_i.get('new_info'); if not isinstance(..., dict), continue.
- The sidecar file is overwritten with _normalize_info_dict(c_new_info_i). If an OSError occurs, the function terminates with that exception.
- changed is set to True.

Define success_i iff no exception is raised during iteration i before the write AND cresult_i is not None AND cresult_i.info_updated is true AND cresult_i.new_info is a dict. If there exists any i with success_i, then changed = True; else changed = False.

Normal return value = cpath if changed else None.

State of the sidecar file after normal return:

- If changed = False: the file is unchanged (identical to its state before the call, except for temporary filesystem metadata).
- If changed = True: let j = max{ i | success_i }. The file contains the JSONserialized output of _normalize_info_dict(c_new_info_j). Its structure is a dictionary with a single key 'callees' whose value is a list of dictionaries, each containing exactly the keys 'name', 'signature', 'pre_condition', 'post_condition' in that order, with any missing fields filled by the empty string and extra keys removed.

If any exception (e.g., OSError, or from _llm_check_caller_info_update) is raised, the function does not return normally and the sidecar file may be in an arbitrary state (corrupted, partially written, or unchanged).

Formal postcondition (for normal termination): exists cpath = file_map.get(caller_fqn), cext, clang . ((cpath is not a valid file path not clang) return = None sidecar_unchanged(cpath)) (valid(cpath) clang let changed = (i 0..n-1 . success_i) in return = (cpath if changed else None) (changed j = max{ i | success_i } . sidecar_content(cpath) = serialize(_normalize_info_dict(c_new_info_j))) (changed sidecar_unchanged(cpath))) where success_i (no_exception_until_write(i) cresult_i None cresult_i.info_updated typeof(cresult_i.new_info) = dict)

- **Code evidence:** Line 35: with open(f"{cpath}.info.json", "w", encoding="utf-8") as f: Line 36: json.dump(_normalize_info_dict(c_new_info), f, indent=2, ensure_ascii=False)
- **Trigger condition:** The code replaces the entire sidecar file with the LLM's output verbatim, but the LLM's postcondition does not guarantee it retains all existing callee entries. This can cause loss of callee information that should remain unchanged according to the specification.
- **Validator trigger:** When _llm_check_caller_info_update returns a new_info dict that omits unrelated callee entries, _reconcile_caller overwrites the caller's .info.json with the LLM's output verbatim, permanently losing callee information that the specification requires be preserved.
- **Probe stdout:** [TopdownLayers] Phase 1 (test): 2 functions, 2 layers -> spec_prompts/phase_01_topdown_layers.json CONFIRMED — callee info lost: expected {'callee_func', 'other_func'}, got {'callee_func'}. The caller's .info.json was overwritten with only the reconciled callee, losing all other callee entries.

► SPEC sidecar（完整）

```
{
  "signature": "_reconcile_caller(caller_fqn, updates, base_idx) -> str | None",
  "pre_condition": "caller_fqn is a non-empty string identifying an extracted function. Its corresponding source file exists and
has a readable .info.json sidecar. updates is a non-empty list of (callee_name: str, callee_new_spec: dict) pairs where each
callee_new_spec dict contains the string keys 'signature', 'pre_condition', and 'post_condition'. base_idx is a non-negative
integer.",
  "post_condition": "Each (callee_name, callee_new_spec) pair in updates is processed sequentially. For each pair, determines
whether the caller's existing .info.json callee entries must be revised to remain consistent with callee_new_spec – the evaluation
considers the caller's source code, its current .info.json, and the callee's new specification. When a revision is required, the
caller's entire .info.json is replaced with a reconciled version whose callee entries are consistent with callee_new_spec and all
previously reconciled callee entries. When no revision is required for any pair, the .info.json is unmodified. Returns the
caller's absolute source file path as a string if at least one .info.json replacement occurred; returns None when the caller
source file is absent, its programming language cannot be determined, its .info.json is unreadable, or no replacements were
applied."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_llm_check_caller_info_update",
      "signature": "_llm_check_caller_info_update(proj_dir, work_dir, artifact_idx, caller_fqn, callee_name, callee_lang,
caller_path, callee_new_spec, caller_info, caller_source) -> dict | None",
      "pre_condition": "artifact_idx is a non-negative integer. caller_fqn and callee_name are non-empty strings. callee_lang is a
valid language identifier string. callee_new_spec is a dict with 'signature', 'pre_condition', and 'post_condition' string fields.
caller_info is a dict matching the .info.json schema. caller_source is the raw caller source code as a string.",
      "post_condition": "Returns a dict with key 'info_updated' (bool). When info_updated is true, also returns key 'new_info'
whose value is a dict conforming to the .info.json schema with callee entries consistent with callee_new_spec. Returns None when
the evaluation fails to produce a valid determination."
    },
    {
      "name": "_normalize_info_dict",
      "signature": "_normalize_info_dict(info: dict) -> dict",
      "pre_condition": "info is a dict optionally containing a 'callees' key whose value is a list of dicts with any subset of the
fields 'name', 'signature', 'pre_condition', 'post_condition'.",
      "post_condition": "Returns a dict with a single key 'callees' whose value is a list of dicts, each containing exactly the
keys 'name', 'signature', 'pre_condition', 'post_condition' in that order. Missing fields in input callee entries are filled with
empty strings. Extra keys in input callee entries are removed."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_update_specs_for_intent::_reconcile_caller.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Each (callee_name, callee_new_spec) pair in updates is processed sequentially. For each pair, determines
whether the caller's existing .info.json callee entries must be revised to remain consistent with callee_new_spec the evaluation
considers the caller's source code, its current .info.json, and the callee's new specification. When a revision is required, the
caller's entire .info.json is replaced with a reconciled version whose callee entries are consistent with callee_new_spec and all
previously reconciled callee entries. When no revision is required for any pair, the .info.json is unmodified. Returns the
caller's absolute source file path as a string if at least one .info.json replacement occurred; returns None when the caller
source file is absent, its programming language cannot be determined, its .info.json is unreadable, or no replacements were
applied.",
    "actual_behavior": "After the function completes normally (i.e., without raising an exception), the following holds:\n\nLet
cpath = file_map.get(caller_fqn). If cpath is not a nonempty string that refers to an existing file, or if the language cannot be
determined (clang falsy), the function returns None and the sidecar file cpath + '.info.json' is unmodified.\n\nOtherwise, let n =
len(updates). For each i from 0 to n-1, let (cname_i, cspec_i) = updates[i]. During iteration i:\n - The source code is read from
cpath. If an OSError occurs, the function terminates with that exception.\n - The sidecar info is loaded. If OSError or
JSONDecodeError, the iteration continues to i+1.\n - cresult_i = _llm_check_caller_info_update(...). If cresult_i is None or
cresult_i.get('info_updated') is not true, continue.\n - c_new_info_i = cresult_i.get('new_info'); if not isinstance(..., dict),
continue.\n - The sidecar file is overwritten with _normalize_info_dict(c_new_info_i). If an OSError occurs, the function
terminates with that exception.\n - changed is set to True.\n\nDefine success_i iff no exception is raised during iteration i
before the write AND cresult_i is not None AND cresult_i.info_updated is true AND cresult_i.new_info is a dict. If there exists
any i with success_i, then changed = True; else changed = False.\n\nNormal return value = cpath if changed else None.\n\nState of
the sidecar file after normal return:\n - If changed = False: the file is unchanged (identical to its state before the call,
except for temporary filesystem metadata).\n - If changed = True: let j = max{ i | success_i }. The file contains the
JSONSerialized output of _normalize_info_dict(c_new_info_j). Its structure is a dictionary with a single key 'callees' whose value
is a list of dictionaries, each containing exactly the keys 'name', 'signature', 'pre_condition', 'post_condition' in that order,
with any missing fields filled by the empty string and extra keys removed.\n\nIf any exception (e.g., OSError, or from
_llm_check_caller_info_update) is raised, the function does not return normally and the sidecar file may be in an arbitrary state
(corrupted, partially written, or unchanged).\n\nFormal postcondition (for normal termination):\n exists cpath =
file_map.get(caller_fqn), cext, clang .\n ( cpath is not a valid file path not clang) return = None
sidecar_unchanged(cpath) )\n ( valid(cpath) clang \n let changed = ( i 0..n-1 . success_i) in\n return =
(cpath if changed else None) \n ( changed j = max{ i | success_i } .\n sidecar_content(cpath) =
serialize(_normalize_info_dict(c_new_info_j)) )\n ( changed sidecar_unchanged(cpath) ) )\n where success_i
```

```
(no_exception_until_write(i) cresult_i None cresult_i.info_updated isinstance(cresult_i.new_info) = dict)",
"code_evidence": "Line 35: with open(f\"{cpath}.info.json\", \"w\", encoding=\"utf-8\") as f:\nLine 36:
json.dump(_normalize_info_dict(c_new_info), f, indent=2, ensure_ascii=False)",
"trigger_condition": "The code replaces the entire sidecar file with the LLM's output verbatim, but the LLM's postcondition
does not guarantee it retains all existing callee entries. This can cause loss of callee information that should remain unchanged
according to the specification."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--update_specs_for_intent::_reconcile_caller",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_update_specs_for_intent::_reconcile_caller.py",
  "function_name": "_update_specs_for_intent::_reconcile_caller",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--update_specs_for_intent::_reconcile_caller.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--update_specs_for_intent::_reconcile_caller.md",
  "probe_stdout": "[TopdownLayers] Phase 1 (test): 2 functions, 2 layers -> spec_prompts/phase_01_topdown_layers.json\nCONFIRMED -
callee info lost: expected {'callee_func', 'other_func'}, got {'callee_func'}. The caller's .info.json was overwritten with only
the reconciled callee, losing all other callee entries.\n",
  "trigger_summary": "When _llm_check_caller_info_update returns a new_info dict that omits unrelated callee entries,
_reconcile_caller overwrites the caller's .info.json with the LLM's output verbatim, permanently losing callee information that
the specification requires be preserved."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:59:08.100204Z	5120
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:00:08.981524Z	8070

事件 1 — [llm_a4c863bf222a4f98803c03c2703493ca](#)

► parsed_json（完整）

```
{
  "post_condition": "After the function completes normally (i.e., without raising an exception), the following holds:\n\nLet cpath = file_map.get(caller_fqn). If cpath is not a non-empty string that refers to an existing file, or if the language cannot be
determined (clang falsy), the function returns None and the sidecar file cpath + '.info.json' is unmodified.\n\nOtherwise, let n =
len(updates). For each i from 0 to n-1, let (cname_i, cspec_i) = updates[i]. During iteration i:\n  - The source code is read from
cpath. If an OSError occurs, the function terminates with that exception.\n  - The sidecar info is loaded. If OSError or
JSONDecodeError, the iteration continues to i+1.\n  - cresult_i = _llm_check_caller_info_update(...). If cresult_i is None or
cresult_i.get('info_updated') is not true, continue.\n  - c_new_info_i = cresult_i.get('new_info'); if not isinstance(..., dict),
continue.\n  - The sidecar file is overwritten with _normalize_info_dict(c_new_info_i). If an OSError occurs, the function
terminates with that exception.\n  - changed is set to True.\n\nDefine success_i iff no exception is raised during iteration i
before the write AND cresult_i is not None AND cresult_i.info_updated is true AND cresult_i.new_info is a dict. If there exists
any i with success_i, then changed = True; else changed = False.\n\nNormal return value = cpath if changed else None.\n\nState of
the sidecar file after normal return:\n  - If changed = False: the file is unchanged (identical to its state before the call,
except for temporary file-system metadata).\n  - If changed = True: let j = max{ i | success_i }. The file contains the JSON-
serialized output of _normalize_info_dict(c_new_info_j). Its structure is a dictionary with a single key 'callees' whose value is
a list of dictionaries, each containing exactly the keys 'name', 'signature', 'pre_condition', 'post_condition' in that order,
with any missing fields filled by the empty string and extra keys removed.\n\nIf any exception (e.g., OSError, or from
_llm_check_caller_info_update) is raised, the function does not return normally and the sidecar file may be in an arbitrary state
(corrupted, partially written, or unchanged).\n\nFormal post-condition (for normal termination):\n  exists cpath =
file_map.get(caller_fqn), cext, clang .\n    ( cpath is not a valid file path v not clang ) → return = None ∧
sidecar_unchanged(cpath) )\n    ∧ ( valid(cpath) ∧ clang →\n      let changed = ( ∃ i ∈ 0..n-1 . success_i ) in\n      return =
(cpath if changed else None) ∧\n      ( changed → ∃ j = max{ i | success_i } .\n        sidecar_content(cpath) =
serialize(_normalize_info_dict(c_new_info_j)) )\n    ∧ ( ¬changed → sidecar_unchanged(cpath) ) )\n  where success_i =
(no_exception_until_write(i) ∧ cresult_i ≠ None ∧ cresult_i.info_updated ∧ typeof(cresult_i.new_info) = dict)"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_a4c863bf222a4f98803c03c2703493ca_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_a4c863bf222a4f98803c03c2703493ca_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_a4c863bf222a4f98803c03c2703493ca_response.txt)

事件 2 — [llm_df656f0149554c69a20442be0089199d](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Suppose file_map.get('caller1') returns '/tmp/caller.py' which exists and has language determined. The
sidecar '/tmp/caller.py.info.json' initially contains {\"calleees\\\": [{\\\"name\\\": \\\"foo\\\", \\\"signature\\\": \\\"...\\\",
\\\"pre_condition\\\": \\\"...\\\", \\\"post_condition\\\": \\\"...\\\"}]}]. The updates list is [( \\\"bar\\\", {\\\"signature\\\": \\\"...\\\",
\\\"pre_condition\\\": \\\"...\\\", \\\"post_condition\\\": \\\"...\\\"} )]. The LLM call for bar returns {\\\"info_updated\\\": true, \\\"new_info\\\":
{\\\"calleees\\\": [{\\\"name\\\": \\\"bar\\\", ...}]} (i.e., it drops the existing entry for 'foo'). The specification requires that the
resulting .info.json should have callee entries consistent with bar and all previously reconciled entries, which includes the pre-
existing 'foo'; but the code overwrites with only bar, violating the spec.",
  "offending_statements": "Line 35: with open(f\\\"{cpath}.info.json\\\", \\\"w\\\", encoding=\\\"utf-8\\\") as f:\\nLine 36:
json.dump(_normalize_info_dict(c_new_info), f, indent=2, ensure_ascii=False)",
  "reason": "The code replaces the entire sidecar file with the LLM's output verbatim, but the LLM's postcondition does not
guarantee it retains all existing callee entries. This can cause loss of callee information that should remain unchanged according
to the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_df656f0149554c69a20442be0089199d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_df656f0149554c69a20442be0089199d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_df656f0149554c69a20442be0089199d_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-242 — [src--incremental_reasoner-py--_verify_incremental_functions::_validate](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#); 原源码 [src/incremental_reasoner.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns rel unchanged. Invokes the bug-validation machinery on the reasoning result whose JSON verdict is derived from rel by replacing its file extension with '.json' and resolving it relative to proj_dir. The validation process either produces a confirmed-bug report at <work_dir>/bug_validation/<bug_id>.md (containing specification claim, actual behavior, code evidence, trigger condition, reproduction steps, probe script, and probe output) or determines that no exploitable bug exists. Returns before the validation report is produced.
- **Actual behavior**: After _validate(rel) completes normally (i.e., does not raise an exception), the return value is exactly rel. The function has called _validate_single_bug(result_json_rel, proj_dir, work_dir, bug_validator_path=bug_validator_path), where result_json_rel = os.path.join(os.path.relpath(output_dir, proj_dir), os.path.splitext(rel)[0] + '.json'). The call to _validate_single_bug has one of the following effects: (1) if the reasoning verdict in the file identified by result_json_rel corresponds to a confirmed real bug, a bug report file has been created at <work_dir>/bug_validation/<bug_id>.md (for some bug_id) containing the specification claim, actual observed behavior, code evidence with line numbers, trigger condition, reproduction steps, a probe script, and its raw output; (2) otherwise, no such report file has been created. The variables output_dir, proj_dir, work_dir, and bug_validator_path remain unchanged relative to their enclosing scope. If _validate_single_bug raises an exception, _validate does not catch it, the exception propagates outward, and the function does not return; any side effects performed before the exception may be present in the file system.
- **Code evidence**: Line 6: _validate_single_bug(Line 7: result_json_rel, Line 8: proj_dir, Line 9: work_dir, Line 10: bug_validator_path=bug_validator_path, Line 11:) Line 12: return rel
- **Trigger condition**: The specification requires _validate to return before the validation report is produced. The code calls _validate_single_bug synchronously; the report is created during that call, so the return on line 12 occurs after the report has already been produced. This violates the timing contract.
- **Validator trigger**: _validate calls _validate_single_bug synchronously before return, so the report is produced before the function returns, violating the spec's 'returns before the validation report is produced' timing contract.
- **Probe stdout**: CONFIRMED — _validate_single_bug was called synchronously before _validate returned (mock flag set during execution). Spec states '_validate returns before the validation report is produced', but the synchronous call means the report is already produced by the time _validate returns rel. actual return: 'some_module/function.py' | expected return by spec: 'some_module/function.py'

► SPEC sidecar（完整）

```
{
  "signature": "_validate(rel) -> str",
  "pre_condition": "rel is a filesystem path (relative to the batch output directory) to a file whose reasoning verdict has been
produced. The enclosing scope provides output_dir (the directory tree containing reasoning result JSON files), proj_dir (the
project root directory), work_dir (the FM-Agent work directory), and bug_validator_path (an optional path to a custom bug-
validator prompt file; None when the built-in validator is used).",
```



```
"post_condition": "Returns rel unchanged. Invokes the bug-validation machinery on the reasoning result whose JSON verdict is derived from rel by replacing its file extension with '.json' and resolving it relative to proj_dir. The validation process either produces a confirmed-bug report at <work_dir>/bug_validation/<bug_id>.md (containing specification claim, actual behavior, code evidence, trigger condition, reproduction steps, probe script, and probe output) or determines that no exploitable bug exists. Returns before the validation report is produced."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_validate_single_bug",
      "signature": "_validate_single_bug(result_json_rel, proj_dir, work_dir, bug_validator_path=None) -> dict",
      "pre_condition": "result_json_rel is a relative path from proj_dir to a JSON file containing a reasoning verdict dict (with at minimum a 'verdict' field whose value is 'MISMATCH'). proj_dir is the project root directory; work_dir is the FM-Agent work directory; bug_validator_path is an optional path to a custom bug-validator prompt file.",
      "post_condition": "Returns a dict describing the validation outcome. Produces a bug report at <work_dir>/bug_validation/<bug_id>.md when the violation is confirmed as a real bug, or determines that the spec-code discrepancy is not exploitable as a bug and does not produce a report. The report contains: specification claim, actual observed behavior, code evidence with line numbers, trigger condition, reproduction steps, a probe script, and its raw output."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_verify_incremental_functions::_validate.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns rel unchanged. Invokes the bug-validation machinery on the reasoning result whose JSON verdict is derived from rel by replacing its file extension with '.json' and resolving it relative to proj_dir. The validation process either produces a confirmed-bug report at <work_dir>/bug_validation/<bug_id>.md (containing specification claim, actual behavior, code evidence, trigger condition, reproduction steps, probe script, and probe output) or determines that no exploitable bug exists. Returns before the validation report is produced.",
    "actual_behavior": "After _validate(rel) completes normally (i.e., does not raise an exception), the return value is exactly rel. The function has called _validate_single_bug(result_json_rel, proj_dir, work_dir, bug_validator_path=bug_validator_path), where result_json_rel = os.path.join(os.path.relpath(output_dir, proj_dir), os.path.splitext(rel)[0] + '.json'). The call to _validate_single_bug has one of the following effects: (1) if the reasoning verdict in the file identified by result_json_rel corresponds to a confirmed real bug, a bug report file has been created at <work_dir>/bug_validation/<bug_id>.md (for some bug_id) containing the specification claim, actual observed behavior, code evidence with line numbers, trigger condition, reproduction steps, a probe script, and its raw output; (2) otherwise, no such report file has been created. The variables output_dir, proj_dir, work_dir, and bug_validator_path remain unchanged relative to their enclosing scope. If _validate_single_bug raises an exception, _validate does not catch it, the exception propagates outward, and the function does not return; any side effects performed before the exception may be present in the file system.",
    "code_evidence": "Line 6:         _validate_single_bug(\nLine 7:             result_json_rel,\nLine 8:             proj_dir,\nLine 9:             work_dir,\nLine 10:             bug_validator_path=bug_validator_path,\nLine 11:             )\nLine 12:         return rel",
    "trigger_condition": "The specification requires _validate to return before the validation report is produced. The code calls _validate_single_bug synchronously; the report is created during that call, so the return on line 12 occurs after the report has already been produced. This violates the timing contract."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--incremental_reasoner-py--_verify_incremental_functions::_validate",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_verify_incremental_functions::_validate.py",
  "function_name": "_verify_incremental_functions::_validate",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_verify_incremental_functions::_validate.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_verify_incremental_functions::_validate.md",
  "probe_stdout": "CONFIRMED - _validate_single_bug was called synchronously before _validate returned (mock flag set during execution). Spec states '_validate returns before the validation report is produced', but the synchronous call means the report is already produced by the time _validate returns rel. actual return: 'some_module/function.py' | expected return by spec: 'some_module/function.py'\n",
  "trigger_summary": "_validate calls _validate_single_bug synchronously before return, so the report is produced before the function returns, violating the spec's 'returns before the validation report is produced' timing contract."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:58:39.268695Z	2854
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:59:13.035212Z	2305

事件 1 — llm_feb2735d58b040c6819a22355c96ec5b

► parsed_json（完整）

```
{
  "post_condition": "After _validate(rel) completes normally (i.e., does not raise an exception), the return value is exactly rel. The function has called _validate_single_bug(result_json_rel, proj_dir, work_dir, bug_validator_path=bug_validator_path), where result_json_rel = os.path.join(os.path.relpath(output_dir, proj_dir), os.path.splitext(rel)[0] + '.json'). The call to _validate_single_bug has one of the following effects: (1) if the reasoning verdict in the file identified by result_json_rel corresponds to a confirmed real bug, a bug report file has been created at <work_dir>/bug_validation/<bug_id>.md (for some bug_id) containing the specification claim, actual observed behavior, code evidence with line numbers, trigger condition, reproduction steps, a probe script, and its raw output; (2) otherwise, no such report file has been created. The variables output_dir, proj_dir, work_dir, and bug_validator_path remain unchanged relative to their enclosing scope. If _validate_single_bug raises an exception, _validate does not catch it, the exception propagates outward, and the function does not return; any side effects performed before the exception may be present in the file system."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_feb2735d58b040c6819a22355c96ec5b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_feb2735d58b040c6819a22355c96ec5b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_feb2735d58b040c6819a22355c96ec5b_response.txt)

事件 2 — llm_c14c6c9520e94add838f03849b413120

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "rel = 'verdict_123.txt' where the corresponding JSON file at os.path.join(os.path.relpath(output_dir, proj_dir), 'verdict_123.json') contains a MISMATCH verdict that results in a confirmed bug report being created by _validate_single_bug. In that case, the report <work_dir>/bug_validation/<bug_id>.md is produced during the call on line 6 before the function returns on line 12.",
  "offending_statements": "Line 6:      _validate_single_bug(\nLine 7:      result_json_rel,\nLine 8:      proj_dir,\nLine 9:      work_dir,\nLine 10:      bug_validator_path=bug_validator_path,\nLine 11:      )\nLine 12:      return rel",
  "reason": "The specification requires _validate to return before the validation report is produced. The code calls _validate_single_bug synchronously; the report is created during that call, so the return on line 12 occurs after the report has already been produced. This violates the timing contract."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c14c6c9520e94add838f03849b413120_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c14c6c9520e94add838f03849b413120_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c14c6c9520e94add838f03849b413120_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: orchestration

FMA-MISMATCH-251 — dashboard.py --State::elapsed

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / orchestration; 源代码 dashboard.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim: Returns None if no events have been ingested into the state. Otherwise returns a non-negative float representing the elapsed duration in seconds from the earliest recorded event's start time to the latest recorded event's end time, or to the current UTC timestamp if no end time has been recorded.

- **Actual behavior:** After execution, the instance `self` remains unchanged. If `self.first_event_time` is `None`, the method returns `None` (). Otherwise, it returns the elapsed seconds as a float computed by subtracting `self.first_event_time` from the effective end time, which is `self.last_event_time` if it is not `None`, else the current UTC time obtained from `datetime.now(timezone.utc)`.
Formally: Return value R satisfies: (R = None self.first_event_time = None)
(R None let end = (self.last_event_time if self.last_event_time None else datetime.now(timezone.utc)) in R = (end - self.first_event_time).total_seconds()). No exceptions are raised, and no side effects occur.
- **Code evidence:** Line 4: end = self.last_event_time or datetime.now(timezone.utc) Line 5: return (end - self.first_event_time).total_seconds()
- **Trigger condition:** When last_event_time is earlier than first_event_time, the code computes a negative elapsed seconds, violating the specification's requirement to return a non-negative float.
- **Validator trigger:** When last_event_time is earlier than first_event_time, the elapsed() method returns a negative float instead of clamping to a non-negative value.
- **Probe stdout:** CONFIRMED — actual: -172800.0 (negative) | expected: 172800.0 (non-negative) | bug: elapsed() returns negative when last_event_time < first_event_time

► SPEC sidecar (完整)

```
{
  "signature": "elapsed(self) -> float | None",
  "pre_condition": "self is an initialized State instance.",
  "post_condition": "Returns None if no events have been ingested into the state. Otherwise returns a non-negative float representing the elapsed duration in seconds from the earliest recorded event's start time to the latest recorded event's end time, or to the current UTC timestamp if no end time has been recorded."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/State::elapsed.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns None if no events have been ingested into the state. Otherwise returns a non-negative float representing the elapsed duration in seconds from the earliest recorded event's start time to the latest recorded event's end time, or to the current UTC timestamp if no end time has been recorded.",
    "actual_behavior": "After execution, the instance `self` remains unchanged. If `self.first_event_time` is `None`, the method returns `None` (). Otherwise, it returns the elapsed seconds as a float computed by subtracting `self.first_event_time` from the effective end time, which is `self.last_event_time` if it is not `None`, else the current UTC time obtained from `datetime.now(timezone.utc)`. Formally: \nReturn value R satisfies: \n(R = None self.first_event_time = None) \n(R None let end = (self.last_event_time if self.last_event_time None else datetime.now(timezone.utc)) in R = (end - self.first_event_time).total_seconds()).\nNo exceptions are raised, and no side effects occur.",
    "code_evidence": "Line 4: end = self.last_event_time or datetime.now(timezone.utc)\nLine 5: return (end - self.first_event_time).total_seconds()",
    "trigger_condition": "When last_event_time is earlier than first_event_time, the code computes a negative elapsed seconds, violating the specification's requirement to return a non-negative float."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "dashboard-py--State::elapsed",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/State::elapsed.py",
  "function_name": "elapsed",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_dashboard-py--State::elapsed.py",
  "detail_file": "fm_agent/bug_validation/dashboard-py--State::elapsed.md",
  "probe_stdout": "CONFIRMED — actual: -172800.0 (negative) | expected: 172800.0 (non-negative) | bug: elapsed() returns negative when last_event_time < first_event_time",
  "trigger_summary": "When last_event_time is earlier than first_event_time, the elapsed() method returns a negative float instead of clamping to a non-negative value."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:44:26.603191Z	825
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:44:37.381342Z	3044

事件 1 — [llm_cf38b73d37f149b7a495b08ea5139b13](#)

► parsed_json（完整）

```
{
  "post_condition": "After execution, the instance `self` remains unchanged. If `self.first_event_time` is `None`, the method returns `None` (∅). Otherwise, it returns the elapsed seconds as a float computed by subtracting `self.first_event_time` from the effective end time, which is `self.last_event_time` if it is not `None`, else the current UTC time obtained from `datetime.now(timezone.utc)`. Formally: \nReturn value R satisfies: \n(R = None ↔ self.first_event_time = None) ∧ \n(R ≠ None → let end = (self.last_event_time if self.last_event_time ≠ None else datetime.now(timezone.utc)) in R = (end - self.first_event_time).total_seconds()).\nNo exceptions are raised, and no side effects occur."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_cf38b73d37f149b7a495b08ea5139b13_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_cf38b73d37f149b7a495b08ea5139b13_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_cf38b73d37f149b7a495b08ea5139b13_response.txt)

事件 2 — [llm_55b9b9e0b5904011af2abf13b840fb6c](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "self.first_event_time = datetime(2023, 1, 2, tzinfo=timezone.utc), self.last_event_time = datetime(2023, 1, 1, tzinfo=timezone.utc)",
  "offending_statements": "Line 4: end = self.last_event_time or datetime.now(timezone.utc)\nLine 5: return (end - self.first_event_time).total_seconds()",
  "reason": "When last_event_time is earlier than first_event_time, the code computes a negative elapsed seconds, violating the specification's requirement to return a non-negative float."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_55b9b9e0b5904011af2abf13b840fb6c_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_55b9b9e0b5904011af2abf13b840fb6c_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_55b9b9e0b5904011af2abf13b840fb6c_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-253 — [dashboard-py--State::tail_events](#)

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [orchestration](#); 原源码 [dashboard.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: If self.events_path does not exist as a file, returns immediately with no side effects. Otherwise: reads all bytes appended to the file since self._events_offset. If file size is strictly less than self._events_offset (indicating truncation or rotation), reading begins from byte 0. Every non-empty line that parses as valid JSON is appended to the State's in-memory event list; empty lines and lines that fail to parse as JSON are skipped. The order of appended events matches the line order in the file. self._events_offset is updated to the current end-of-file byte position.
- **Actual behavior**: After a normal return from tail_events:
- If self.events_path does not exist, self._events_offset is unchanged and no events are ingested.
- Otherwise, let S_initial = initial self._events_offset, S_current = file size at entry. If S_initial > S_current (file truncated/rotated), self._events_offset is set to 0 before reading. Then if the offset equals the file size, the function returns immediately with no ingestion.

Otherwise, it reads all remaining lines from that offset to end-of-file (as determined during the open-read block). Each non-empty line that successfully parses as a JSON object is passed to `_ingest_event`, which appends it to the in-memory event list in the order read. After the loop, `self._events_offset` is updated to `f.tell()` (the file position at end of file). Thus on normal return, `self._events_offset` equals the final file size, and the event list has been extended by exactly the sequence of valid, non-empty JSON objects found from the (possibly reset) offset to end-of-file. No other state is modified. If an exception occurs during execution, the call does not return normally. In that case, `self._events_offset` may have been modified to 0 if the file was detected as truncated/rotated before the exception, and a prefix of the valid JSON lines (those processed before the exception) may have been ingested, but `self._events_offset` is not updated to `f.tell()`. The exact state is the result of executing a prefix of the normal execution path up to the point of the exception.

Formal logic (normal termination): Let `events` be the in-memory event list before the call, `off = self._events_offset`. After normal return: `events'` and `off'` denote post-state. `(exists(self.events_path)`
let `size = file_size(self.events_path)` in `(if off > size then off_reset = 0 else off_reset = off) (if off_reset = size then off' = off events' = events) else`
`lines = lines_from(off_reset, end)` such that `events' = events ++ [ parse!JSON(line) | line lines, line.strip() "", parse!JSON succeeds ] off' =`
`end_position_after_read)` `(exists(self.events_path) off' = off events' = events)`

- **Code evidence:** Line 2: `if not self.events_path.exists():`
- **Trigger condition:** The specification states: 'If `self.events_path` does not exist as a file, returns immediately with no side effects.' When `events_path` is a directory, it does not exist 'as a file', yet `Path.exists()` returns `True`, so the early return is skipped. Execution proceeds to `open()` which raises `IsADirectoryError`, causing an abnormal exit instead of the required normal return with no side effects.
- **Validator trigger:** When `events_path` is a directory, `Path.exists()` returns `True` (spec requires 'exists as a file' check), skipping the early return and causing `IsADirectoryError` on `open()` instead of a normal return with no side effects.
- **Probe stdout:** CONFIRMED — `tail_events()` raised `IsADirectoryError` when `events_path` is a directory; spec requires normal return with no side effects

► SPEC sidecar (完整)

```
{
  "signature": "tail_events(self)",
  "pre_condition": "self.events_path refers to a filesystem path that may or may not exist as a regular file. self._events_offset contains the byte position up to which previous tail_events calls have consumed the file.",
  "post_condition": "If self.events_path does not exist as a file, returns immediately with no side effects. Otherwise: reads all bytes appended to the file since self._events_offset. If file size is strictly less than self._events_offset (indicating truncation or rotation), reading begins from byte 0. Every non-empty line that parses as valid JSON is appended to the State's in-memory event list; empty lines and lines that fail to parse as JSON are skipped. The order of appended events matches the line order in the file. self._events_offset is updated to the current end-of-file byte position."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_ingest_event",
      "signature": "_ingest_event(self, ev)",
      "pre_condition": "ev is a dict parsed from a valid JSON object line in the events.jsonl trace file.",
      "post_condition": "ev is appended to the State's in-memory list of structured trace events. Event order across all tail_events calls is preserved as read from the file."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/State::tail_events.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "If self.events_path does not exist as a file, returns immediately with no side effects. Otherwise: reads all bytes appended to the file since self._events_offset. If file size is strictly less than self._events_offset (indicating truncation or rotation), reading begins from byte 0. Every non-empty line that parses as valid JSON is appended to the State's in-memory event list; empty lines and lines that fail to parse as JSON are skipped. The order of appended events matches the line order in the file. self._events_offset is updated to the current end-of-file byte position.",
    "actual_behavior": "After a normal return from tail_events:\n- If self.events_path does not exist, self._events_offset is unchanged and no events are ingested.\n- Otherwise, let S_initial = initial self._events_offset, S_current = file size at entry. If S_initial > S_current (file truncated/rotated), self._events_offset is set to 0 before reading. Then if the offset equals the file size, the function returns immediately with no ingestion. Otherwise, it reads all remaining lines from that offset to end-of-file (as determined during the open-read block). Each non-empty line that successfully parses as a JSON object is passed to _ingest_event, which appends it to the in-memory event list in the order read. After the loop, self._events_offset is updated to f.tell() (the file position at end of file). Thus on normal return, self._events_offset equals the final file size, and the event
```

```
list has been extended by exactly the sequence of valid, non-empty JSON objects found from the (possibly reset) offset to end-of-
file. No other state is modified.\nIf an exception occurs during execution, the call does not return normally. In that case,
self._events_offset may have been modified to 0 if the file was detected as truncated/rotated before the exception, and a prefix
of the valid JSON lines (those processed before the exception) may have been ingested, but self._events_offset is not updated to
f.tell(). The exact state is the result of executing a prefix of the normal execution path up to the point of the
exception.\n\nFormal logic (normal termination):\nLet events be the in-memory event list before the call, off =
self._events_offset.\nAfter normal return: events' and off' denote post-state.\n(exists(self.events_path) \n let size =
file_size(self.events_path) in\n (if off > size then off_reset = 0 else off_reset = off) \n (if off_reset = size then off' =
off events' = events)\n else lines = lines_from(off_reset, end) such that\n events' = events ++ [ parse!JSON(line) |
line lines, line.strip() \n\n, parse!JSON succeeds ] \n off' = end_position_after_read)\n (exists(self.events_path) off'
= off events' = events)",
"code_evidence": "Line 2: if not self.events_path.exists():",
"trigger_condition": "The specification states: 'If self.events_path does not exist as a file, returns immediately with no
side effects.' When events_path is a directory, it does not exist 'as a file', yet Path.exists() returns True, so the early return
is skipped. Execution proceeds to open() which raises IsADirectoryError, causing an abnormal exit instead of the required normal
return with no side effects."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "dashboard-py--State::tail_events",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/State::tail_events.py",
  "function_name": "State::tail_events",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_dashboard-py--State::tail_events.py",
  "detail_file": "fm_agent/bug_validation/dashboard-py--State::tail_events.md",
  "probe_stdout": "CONFIRMED - tail_events() raised IsADirectoryError when events_path is a directory; spec requires normal return
with no side effects\n",
  "trigger_summary": "When events_path is a directory, Path.exists() returns True (spec requires 'exists as a file' check),
skipping the early return and causing IsADirectoryError on open() instead of a normal return with no side effects."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:08:50.169792Z	3314
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:09:32.398425Z	4789

事件 1 — [llm_daa70a434f1449f1b937d821b9b855a0](#)

► parsed_json（完整）

```
{
  "post_condition": "After a normal return from tail_events:\n- If self.events_path does not exist, self._events_offset is
unchanged and no events are ingested.\n- Otherwise, let S_initial = initial self._events_offset, S_current = file size at entry.
If S_initial > S_current (file truncated/rotated), self._events_offset is set to 0 before reading. Then if the offset equals the
file size, the function returns immediately with no ingestion. Otherwise, it reads all remaining lines from that offset to end-of-
file (as determined during the open-read block). Each non-empty line that successfully parses as a JSON object is passed to
_ingest_event, which appends it to the in-memory event list in the order read. After the loop, self._events_offset is updated to
f.tell() (the file position at end of file). Thus on normal return, self._events_offset equals the final file size, and the event
list has been extended by exactly the sequence of valid, non-empty JSON objects found from the (possibly reset) offset to end-of-
file. No other state is modified.\nIf an exception occurs during execution, the call does not return normally. In that case,
self._events_offset may have been modified to 0 if the file was detected as truncated/rotated before the exception, and a prefix
of the valid JSON lines (those processed before the exception) may have been ingested, but self._events_offset is not updated to
f.tell(). The exact state is the result of executing a prefix of the normal execution path up to the point of the
exception.\n\nFormal logic (normal termination):\nLet events be the in-memory event list before the call, off =
self._events_offset.\nAfter normal return: events' and off' denote post-state.\n(exists(self.events_path) ^ \n let size =
file_size(self.events_path) in\n (if off > size then off_reset = 0 else off_reset = off) ^ \n (if off_reset = size then off'
= off ^ events' = events)\n else ∃ lines = lines_from(off_reset, end) such that\n events' = events ++ [ parse!JSON(line) |
line ← lines, line.strip() ≠ \n\n, parse!JSON succeeds ] ^ \n off' = end_position_after_read)\n v (¬exists(self.events_path)
→ off' = off ^ events' = events)"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_daa70a434f1449f1b937d821b9b855a0_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_daa70a434f1449f1b937d821b9b855a0_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_daa70a434f1449f1b937d821b9b855a0_response.txt)

事件 2 — [llm_029dfc62647740aa97fe159d54b4db4e](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "self.events_path is a directory (e.g., Path('/tmp/somedir') exists as a directory); self._events_offset is any valid integer (e.g., 0).",
  "offending_statements": "Line 2:         if not self.events_path.exists():",
  "reason": "The specification states: 'If self.events_path does not exist as a file, returns immediately with no side effects.' When events_path is a directory, it does not exist 'as a file', yet Path.exists() returns True, so the early return is skipped. Execution proceeds to open() which raises IsADirectoryError, causing an abnormal exit instead of the required normal return with no side effects."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_029dfc62647740aa97fe159d54b4db4e_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_029dfc62647740aa97fe159d54b4db4e_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_029dfc62647740aa97fe159d54b4db4e_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-254 — [dashboard-py](#) -- `State::tail_opencode`

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 2
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [orchestration](#); 源代码 [dashboard.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: If self.opencode_dir does not exist as a directory, returns immediately with no side effects. Otherwise: for each JSONL file in the directory (in ascending lexicographic filename order), reads all bytes appended to the file since its tracked offset in self._opencode_offsets. If a file's size is strictly less than its tracked offset (indicating truncation or rotation), reading begins from byte 0. Every non-empty line that parses as valid JSON is ingested into the State's in-memory raw trace collection; empty lines and lines that fail to parse as JSON are skipped. After processing all files, the aggregate cache_window collection in the State reflects the cumulative (cached_tokens, total_input_tokens) pairs across all raw trace records ingested so far. Each file's offset in self._opencode_offsets is updated to the current end-of-file byte position for that file.
- **Actual behavior**: If self.opencode_dir does not exist, self._opencode_offsets is unchanged and no ingestion occurs. Otherwise, for each path p in sorted(self.opencode_dir.glob("*.jsonl")): let name = p.name. If obtaining p.stat().st_size raises OSError, the file is skipped and self._opencode_offsets[name] remains unchanged. Else let size = st_size and old_off = self._opencode_offsets.get(name, 0). If size < old_off, set old_off = 0. If size == old_off, the file is skipped and self._opencode_offsets[name] remains old_off. If size > old_off, the file is opened, seeked to old_off, and all lines from that position to end-of-file are read. For each non-empty line l, if json.loads(l) succeeds, rec = loaded JSON; self._ingest_opencode(rec, name) is called. After processing all lines, self._opencode_offsets[name] is set to the file's current position (equal to its size at that moment). Consequently, after the method returns, for every *jsonl file in the directory that was accessible and had size > old_off at entry, self._opencode_offsets[name] equals its final size, all valid JSON records from the unconsumed portion have been ingested into the State's in-memory OpenCode trace collection, and the aggregate cache_window (pairs of (cached_tokens, total_input_tokens)) reflects cumulative token usage across all records ingested so far. Formally: ((not exists(self.opencode_dir)) => (self._opencode_offsets = old_offsets and no ingestion)) and (exists(self.opencode_dir) => or all path in sorted(glob(self.opencode_dir, '*.jsonl')): let name = path.name; if (OSError on stat) then self._opencode_offsets[name] = old_offsets[name] else let size = stat.st_size, old_off = old_offsets.get(name,0); if size < old_off then old_off := 0; if size == old_off then self._opencode_offsets[name] = old_off else (size > old_off) => (after open seek old_off, for all lines l from old_off to EOF: if l.strip() != "" and json.loads(l) succeeds with rec: _ingest_opencode(rec, name) called) and self._opencode_offsets[name] = new_end_pos))*
- **Code evidence**: Line 2: if not self.opencode_dir.exists():
- **Trigger condition**: Specification requires: if self.opencode_dir does not exist as a directory, return immediately with no side effects. The code only checks .exists(), which returns True for any path (file, symlink, etc.). If self.opencode_dir is a file, the code proceeds to .glob("*.jsonl"), which raises NotADirectoryError (or similar), thus not returning immediately and causing an exception side effect.
- **Validator trigger**: When self.opencode_dir is a regular file instead of a directory, Path.exists() returns True so the guard is bypassed and .glob("*.jsonl") is called instead of returning immediately as the spec requires.
- **Probe stdout**: CONFIRMED — guard bypassed: .glob() called on non-directory path | spec_claim: return immediately with no side effects if not a directory | actual: proceeded to .glob("*.jsonl") on a non-directory path | guard check: .exists() returned True for a file (should use .is_dir()) | offsets unchanged: True | root cause: line 341 uses .exists() instead of .is_dir()

► [SPEC sidecar](#) (完整)


```
{
  "signature": "tail_opencode(self)",
  "pre_condition": "self.opencode_dir refers to a filesystem path that may or may not exist as a directory. self._opencode_offsets is a dict mapping JSONL filenames (strings) to the byte position up to which the file has been consumed.",
  "post_condition": "If self.opencode_dir does not exist as a directory, returns immediately with no side effects. Otherwise: for each JSONL file in the directory (in ascending lexicographic filename order), reads all bytes appended to the file since its tracked offset in self._opencode_offsets. If a file's size is strictly less than its tracked offset (indicating truncation or rotation), reading begins from byte 0. Every non-empty line that parses as valid JSON is ingested into the State's in-memory raw trace collection; empty lines and lines that fail to parse as JSON are skipped. After processing all files, the aggregate cache_window collection in the State reflects the cumulative (cached_tokens, total_input_tokens) pairs across all raw trace records ingested so far. Each file's offset in self._opencode_offsets is updated to the current end-of-file byte position for that file."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_ingest_opencode",
      "signature": "_ingest_opencode(self, rec, name)",
      "pre_condition": "rec is a dict parsed from a valid JSON object line in a raw OpenCode trace file identified by name (a string filename).",
      "post_condition": "The raw trace record rec is incorporated into the State's in-memory OpenCode trace collection. The aggregate cache_window collection – pairs of (cached_tokens, total_input_tokens) – is updated to reflect cumulative token usage across all records ingested so far."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/State::tail_opencode.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "If self.opencode_dir does not exist as a directory, returns immediately with no side effects. Otherwise: for each JSONL file in the directory (in ascending lexicographic filename order), reads all bytes appended to the file since its tracked offset in self._opencode_offsets. If a file's size is strictly less than its tracked offset (indicating truncation or rotation), reading begins from byte 0. Every non-empty line that parses as valid JSON is ingested into the State's in-memory raw trace collection; empty lines and lines that fail to parse as JSON are skipped. After processing all files, the aggregate cache_window collection in the State reflects the cumulative (cached_tokens, total_input_tokens) pairs across all raw trace records ingested so far. Each file's offset in self._opencode_offsets is updated to the current end-of-file byte position for that file.",
    "actual_behavior": "If self.opencode_dir does not exist, self._opencode_offsets is unchanged and no ingestion occurs. Otherwise, for each path p in sorted(self.opencode_dir.glob('*.jsonl')): let name = p.name. If obtaining p.stat().st_size raises OSError, the file is skipped and self._opencode_offsets[name] remains unchanged. Else let size = st_size and old_off = self._opencode_offsets.get(name, 0). If size < old_off, set old_off = 0. If size == old_off, the file is skipped and self._opencode_offsets[name] remains old_off. If size > old_off, the file is opened, seeked to old_off, and all lines from that position to end-of-file are read. For each non-empty line l, if json.loads(l) succeeds, rec = loaded JSON; self._ingest_opencode(rec, name) is called. After processing all lines, self._opencode_offsets[name] is set to the file's current position (equal to its size at that moment). Consequently, after the method returns, for every *.jsonl file in the directory that was accessible and had size > old_off at entry, self._opencode_offsets[name] equals its final size, all valid JSON records from the unconsumed portion have been ingested into the State's in-memory OpenCode trace collection, and the aggregate cache_window (pairs of (cached_tokens, total_input_tokens)) reflects cumulative token usage across all records ingested so far. Formally: \n( ( not exists(self.opencode_dir) ) => ( self._opencode_offsets = old_offsets and no ingestion ) ) \nand\n( exists(self.opencode_dir) => \n  \forall path in sorted(glob(self.opencode_dir, '*.jsonl')): \n    let name = path.name; \n    if (OSError on stat) then self._opencode_offsets[name] = old_offsets[name] \n    else let size = stat.st_size, old_off = old_offsets.get(name,0); \n    if size < old_off then old_off := 0; \n    if size == old_off then self._opencode_offsets[name] = old_off \n    else ( size > old_off ) => \n      ( after open seek old_off, for all lines l from old_off to EOF: \n        if l.strip() != '' and json.loads(l) succeeds with rec: \n          _ingest_opencode(rec, name) called ) \n      and self._opencode_offsets[name] = new_end_pos )",
    "code_evidence": "Line 2: if not self.opencode_dir.exists():",
    "trigger_condition": "Specification requires: if self.opencode_dir does not exist as a directory, return immediately with no side effects. The code only checks .exists(), which returns True for any path (file, symlink, etc.). If self.opencode_dir is a file, the code proceeds to .glob('*.jsonl'), which raises NotADirectoryError (or similar), thus not returning immediately and causing an exception side effect."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "dashboard-py--State::tail_opencode",
  "source_file": "dashboard-py/State::tail_opencode.py",
  "function_name": "tail_opencode",
  "confirmation_status": "confirmed",
}
```

```
"attempts": "2",
"probe_script": "fm_agent/bug_validation/probe_dashboard-py--State::tail_opencode.py",
"detail_file": "fm_agent/bug_validation/dashboard-py--State::tail_opencode.md",
"probe_stdout": "CONFIRMED - guard bypassed: .glob() called on non-directory path\n| spec_claim: return immediately with no
side effects if not a directory\n| actual: proceeded to .glob('*.jsonl') on a non-directory path\n| guard check: .exists()
returned True for a file (should use .is_dir())\n| offsets unchanged: True\n| root cause: line 341 uses .exists() instead of
.is_dir()\n",
"trigger_summary": "When self.opencode_dir is a regular file instead of a directory, Path.exists() returns True so the guard is
bypassed and .glob('*.jsonl') is called instead of returning immediately as the spec requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:08:54.179128Z	2509
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:09:22.950709Z	2731

事件 1 — [llm_b946025f1e6540d1a06656a6da11c8bb](#)

► parsed_json（完整）

```
{
  "post_condition": "If self.opencode_dir does not exist, self._opencode_offsets is unchanged and no ingestion occurs. Otherwise,
for each path p in sorted(self.opencode_dir.glob('*.jsonl')): let name = p.name. If obtaining p.stat().st_size raises OSError, the
file is skipped and self._opencode_offsets[name] remains unchanged. Else let size = st_size and old_off =
self._opencode_offsets.get(name, 0). If size < old_off, set old_off = 0. If size == old_off, the file is skipped and
self._opencode_offsets[name] remains old_off. If size > old_off, the file is opened, seeked to old_off, and all lines from that
position to end-of-file are read. For each non-empty line l, if json.loads(l) succeeds, rec = loaded JSON;
self._ingest_opencode(rec, name) is called. After processing all lines, self._opencode_offsets[name] is set to the file's current
position (equal to its size at that moment). Consequently, after the method returns, for every *.jsonl file in the directory that
was accessible and had size > old_off at entry, self._opencode_offsets[name] equals its final size, all valid JSON records from
the unconsumed portion have been ingested into the State's in-memory OpenCode trace collection, and the aggregate cache_window
(pairs of (cached_tokens, total_input_tokens)) reflects cumulative token usage across all records ingested so far. Formally: \n( (
not exists(self.opencode_dir) ) => ( self._opencode_offsets = old_offsets and no ingestion ) ) \nand\n( exists(self.opencode_dir)
=> \n \forall path in sorted(glob(self.opencode_dir, '*.jsonl')): \n   let name = path.name; \n   if (OSError on stat) then
self._opencode_offsets[name] = old_offsets[name] \n   else let size = stat.st_size, old_off = old_offsets.get(name,0); \n   if
size < old_off then old_off := 0; \n   if size == old_off then self._opencode_offsets[name] = old_off \n   else ( size >
old_off ) => \n     ( after open seek old_off, for all lines l from old_off to EOF: \n       if l.strip() != '' and
json.loads(l) succeeds with rec: \n         _ingest_opencode(rec, name) called ) \n       and self._opencode_offsets[name] =
new_end_pos ) )"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b946025f1e6540d1a06656a6da11c8bb_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_b946025f1e6540d1a06656a6da11c8bb_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_b946025f1e6540d1a06656a6da11c8bb_response.txt)

事件 2 — [llm_7e277416dfb54f789ee4cd2c9a04510d](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "self.opencode_dir = Path('existing_file.txt') where 'existing_file.txt' exists and is a regular file (not a
directory).",
  "offending_statements": "Line 2: if not self.opencode_dir.exists():",
  "reason": "Specification requires: if self.opencode_dir does not exist as a directory, return immediately with no side effects.
The code only checks .exists(), which returns True for any path (file, symlink, etc.). If self.opencode_dir is a file, the code
proceeds to .glob('*.jsonl'), which raises NotADirectoryError (or similar), thus not returning immediately and causing an
exception side effect."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7e277416dfb54f789ee4cd2c9a04510d_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_7e277416dfb54f789ee4cd2c9a04510d_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_7e277416dfb54f789ee4cd2c9a04510d_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 6 — Reasoning & Verification Engine / orchestration; 原源码 dashboard.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Returns the result of successively indexing the nested dict structure with each key in order, starting from d. At each step, if the current value is not a dict or does not contain the next key, returns None. When keys is empty, returns d.
- **Actual behavior**: The function returns the value located at the nested key path given by keys within the dictionary d, provided that every intermediate object along the path is a dictionary that contains the next key; otherwise it returns None. If keys is empty, d itself is returned. More formally, let keys = (k_1, k_2, ..., k_n). Define a recursive function f(current, remaining):
 - if remaining is empty: return current
 - else if not isinstance(current, dict): return None
 - else: return f(current.get(remaining[0]), remaining[1:])Then _get_nested(d, *keys) evaluates to f(d, keys). Equivalently, an iterative description: initialize cur = d; for each k in keys in order, if not isinstance(cur, dict) return None, else set cur = cur.get(k); after the loop return cur. Thus the result is None whenever a key is missing or an intermediate value is not a mapping, and is the final value (which may itself be None) otherwise.
- **Code evidence**: Line 6: cur = cur.get(k)
- **Trigger condition**: The code uses cur.get(k) to retrieve the value for the next key, assuming that a missing key always yields None. However, a dict subclass can override get() to return a non-None value even when the key is missing (i.e., when contains returns False). The specification requires returning None whenever the current dict does not contain the key, which is violated by this input because the code returns the non-None get() value instead of None.
- **Validator trigger**: A dict subclass that overrides get() to return a non-None sentinel for missing keys causes _get_nested to return the sentinel instead of None, violating the spec that requires None when the key is absent.
- **Probe stdout**: CONFIRMED — _get_nested returned <object object at 0x7136acde3bc0> instead of None when dict subclass overrides get(). Key was in dict (contains): False, get() returned non-None sentinel: True

► SPEC sidecar (完整)

```
{
  "signature": "_get_nested(d, *keys)",
  "pre_condition": "d is a dict. keys is a sequence of zero or more hashable values (typically strings) forming a nested key-
access path.",
  "post_condition": "Returns the result of successively indexing the nested dict structure with each key in order, starting from
d. At each step, if the current value is not a dict or does not contain the next key, returns None. When keys is empty, returns
d."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/_get_nested.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns the result of successively indexing the nested dict structure with each key in order, starting from d.
At each step, if the current value is not a dict or does not contain the next key, returns None. When keys is empty, returns d.",
    "actual_behavior": "The function returns the value located at the nested key path given by `keys` within the dictionary `d`,
provided that every intermediate object along the path is a dictionary that contains the next key; otherwise it returns `None`. If
`keys` is empty, `d` itself is returned. More formally, let `keys = (k_1, k_2, ..., k_n)`. Define a recursive function `f(current,
remaining)`:
- if `remaining` is empty: return `current`
- else if `not isinstance(current, dict)`: return `None`
- else:
  return `f(current.get(remaining[0]), remaining[1:])`
Then `_get_nested(d, *keys)` evaluates to `f(d, keys)`. Equivalently, an
iterative description: initialize `cur = d`; for each `k` in `keys` in order, if `not isinstance(cur, dict)` return `None`, else
set `cur = cur.get(k)`; after the loop return `cur`. Thus the result is `None` whenever a key is missing or an intermediate value
is not a mapping, and is the final value (which may itself be `None`) otherwise.",
    "code_evidence": "Line 6: cur = cur.get(k)",
    "trigger_condition": "The code uses cur.get(k) to retrieve the value for the next key, assuming that a missing key always
yields None. However, a dict subclass can override get() to return a non-None value even when the key is missing (i.e., when
__contains__ returns False). The specification requires returning None whenever the current dict does not contain the key, which
is violated by this input because the code returns the non-None get() value instead of None."
  }
}
```

```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "dashboard-py--_get_nested",
  "source_file": "dashboard-py/_get_nested",
  "function_name": "_get_nested",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_dashboard-py--_get_nested.py",
  "detail_file": "fm_agent/bug_validation/dashboard-py--_get_nested.md",
  "probe_stdout": "CONFIRMED - _get_nested returned <object object at 0x7136acde3bc0> instead of None when dict subclass overrides get(). Key was in dict (__contains__): False, get() returned non-None sentinel: True",
  "trigger_summary": "A dict subclass that overrides get() to return a non-None sentinel for missing keys causes _get_nested to return the sentinel instead of None, violating the spec that requires None when the key is absent."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:43:27.210038Z	2357
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:44:05.673653Z	6663

事件 1 — [llm_8a2bb1ac9c81413da12aaef70ff7ce8d](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns the value located at the nested key path given by `keys` within the dictionary `d`, provided that every intermediate object along the path is a dictionary that contains the next key; otherwise it returns `None`. If `keys` is empty, `d` itself is returned. More formally, let `keys = (k_1, k_2, ..., k_n)`. Define a recursive function `f(current, remaining)`:\n- if `remaining` is empty: return `current`\n- else if `not isinstance(current, dict)`: return `None`\n- else: return `f(current.get(remaining[0]), remaining[1:])`\nThen `_get_nested(d, *keys)` evaluates to `f(d, keys)`. Equivalently, an iterative description: initialize `cur = d`; for each `k` in `keys` in order, if `not isinstance(cur, dict)` return `None`, else set `cur = cur.get(k)`; after the loop return `cur`. Thus the result is `None` whenever a key is missing or an intermediate value is not a mapping, and is the final value (which may itself be `None`) otherwise."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_8a2bb1ac9c81413da12aaef70ff7ce8d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_8a2bb1ac9c81413da12aaef70ff7ce8d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_8a2bb1ac9c81413da12aaef70ff7ce8d_response.txt)

事件 2 — [llm_865165c8d79e404d854905df3ac06742](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "d is an instance of a dict subclass where 'b' not in d is True but d.get('b') returns a non-None value (e.g., 42) and keys = ('b',). The call _get_nested(d, 'b') returns 42 instead of None.",
  "offending_statements": "Line 6: cur = cur.get(k)",
  "reason": "The code uses cur.get(k) to retrieve the value for the next key, assuming that a missing key always yields None. However, a dict subclass can override get() to return a non-None value even when the key is missing (i.e., when __contains__ returns False). The specification requires returning None whenever the current dict does not contain the key, which is violated by this input because the code returns the non-None get() value instead of None."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_865165c8d79e404d854905df3ac06742_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_865165c8d79e404d854905df3ac06742_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_865165c8d79e404d854905df3ac06742_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论： 契约不确定
- 分类依据： 沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论： 沿用旧清单的逐条审计结论。
- **Validator**: [confirmed](#); attempts [1](#)
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [orchestration](#); 原源码 [dashboard.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns None when ts is None or any falsy value. When ts is a string, interprets it as an ISO 8601 timestamp and returns a datetime object representing that moment if parsing succeeds. A trailing 'Z' suffix on the timestamp string denotes the UTC timezone offset. Returns None without raising an exception when ts is not a valid ISO 8601 timestamp string (including malformed strings, invalid date or time components, or otherwise unparseable input). The function never raises an exception to its caller.
- **Actual behavior**: If `ts` is falsy (e.g., `None`, empty string, or any other false value), the function returns `None`. If `ts` is a truthy string (a non-empty string), then before parsing, any trailing 'Z' is replaced by '+00:00'. The function then attempts to call `datetime.fromisoformat` on that resulting string. If the call succeeds without raising an exception, the function returns the created `datetime` object. If any exception is raised (for example, due to an invalid ISO-8601 format), the function catches it and returns `None`. Formally: `result = (None if not ts else (let s = ts[:-1] + '+00:00' if ts.endswith('Z') else ts in (datetime.fromisoformat(s) if no exception occurs else None)))`.
- **Code evidence**: Line 7: `return datetime.fromisoformat(ts)`
- **Trigger condition**: The specification requires that any valid ISO 8601 timestamp string be successfully parsed and returned as a datetime object. The code delegates parsing to `datetime.fromisoformat`, which does not support all ISO 8601 formats. For example, the ordinal date timestamp '2021-001T00:00:00Z' is valid per ISO 8601 but `fromisoformat` cannot parse it, causing the function to return `None` instead of a datetime, violating the specification.
- **Validator trigger**: Valid ISO 8601 ordinal date '2021-001T00:00:00Z' is rejected by `datetime.fromisoformat`, returning `None` instead of a datetime object.
- **Probe stdout**: 12:52:21 - LiteLLM:WARNING: get_model_cost_map.py:271 - LiteLLM: Failed to fetch remote model cost map from https://raw.githubusercontent.com/BerriAI/litellm/main/model_prices_and_context_window.json: [SSL: UNEXPECTED_EOF_WHILE_READING] EOF occurred in violation of protocol (_ssl.c:1000). Falling back to local backup. CONFIRMED — ordinal date '2021-001T00:00:00Z' returned None (expected a datetime object per spec)

► SPEC sidecar (完整)

```
{
  "signature": "_parse_iso(ts) -> datetime or None",
  "pre_condition": "ts is a value extracted from a timestamp field in a parsed JSON event record. It is either a string, or None (or any falsy value) when the source field was absent or empty.",
  "post_condition": "Returns None when ts is None or any falsy value. When ts is a string, interprets it as an ISO 8601 timestamp and returns a datetime object representing that moment if parsing succeeds. A trailing 'Z' suffix on the timestamp string denotes the UTC timezone offset. Returns None without raising an exception when ts is not a valid ISO 8601 timestamp string (including malformed strings, invalid date or time components, or otherwise unparseable input). The function never raises an exception to its caller."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/_parse_iso.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns None when ts is None or any falsy value. When ts is a string, interprets it as an ISO 8601 timestamp and returns a datetime object representing that moment if parsing succeeds. A trailing 'Z' suffix on the timestamp string denotes the UTC timezone offset. Returns None without raising an exception when ts is not a valid ISO 8601 timestamp string (including malformed strings, invalid date or time components, or otherwise unparseable input). The function never raises an exception to its caller.",
    "actual_behavior": "If `ts` is falsy (e.g., `None`, empty string, or any other false value), the function returns `None`. If `ts` is a truthy string (a non-empty string), then before parsing, any trailing 'Z' is replaced by '+00:00'. The function then attempts to call `datetime.fromisoformat` on that resulting string. If the call succeeds without raising an exception, the function returns the created `datetime` object. If any exception is raised (for example, due to an invalid ISO-8601 format), the function catches it and returns `None`. Formally: result = (None if not ts else (let s = ts[:-1] + '+00:00' if ts.endswith('Z') else ts in (datetime.fromisoformat(s) if no exception occurs else None))).",
  }
}
```

```
"code_evidence": "Line 7: return datetime.fromisoformat(ts)",
"trigger_condition": "The specification requires that any valid ISO 8601 timestamp string be successfully parsed and returned as a datetime object. The code delegates parsing to datetime.fromisoformat, which does not support all ISO 8601 formats. For example, the ordinal date timestamp '2021-001T00:00:00Z' is valid per ISO 8601 but fromisoformat cannot parse it, causing the function to return None instead of a datetime, violating the specification."
}
```

► Bug Validator result（完整）

```
{
  "id": "dashboard-py--_parse_iso",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/_parse_iso.py",
  "function_name": "_parse_iso",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_dashboard-py--_parse_iso.py",
  "detail_file": "fm_agent/bug_validation/dashboard-py--_parse_iso.md",
  "probe_stdout": "12:52:21 - LiteLLM:WARNING: get_model_cost_map.py:271 - LiteLLM: Failed to fetch remote model cost map from https://raw.githubusercontent.com/BerriAI/litellm/main/model_prices_and_context_window.json: [SSL: UNEXPECTED_EOF_WHILE_READING] EOF occurred in violation of protocol (_ssl.c:1000). Falling back to local backup.\nCONFIRMED - ordinal date '2021-001T00:00:00Z' returned None (expected a datetime object per spec)",
  "trigger_summary": "Valid ISO 8601 ordinal date '2021-001T00:00:00Z' is rejected by datetime.fromisoformat, returning None instead of a datetime object."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:45:06.297705Z	2051
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:45:38.141845Z	3836

事件 1 — [llm_846b1bc3e1594c669d02722eca70190d](#)

► parsed_json（完整）

```
{
  "post_condition": "If `ts` is falsy (e.g., `None`, empty string, or any other false value), the function returns `None`. If `ts` is a truthy string (a non-empty string), then before parsing, any trailing 'Z' is replaced by '+00:00'. The function then attempts to call `datetime.fromisoformat` on that resulting string. If the call succeeds without raising an exception, the function returns the created `datetime` object. If any exception is raised (for example, due to an invalid ISO-8601 format), the function catches it and returns `None`. Formally: result = (None if not ts else (let s = ts[:-1] + '+00:00' if ts.endswith('Z') else ts in (datetime.fromisoformat(s) if no exception occurs else None)))."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_846b1bc3e1594c669d02722eca70190d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_846b1bc3e1594c669d02722eca70190d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_846b1bc3e1594c669d02722eca70190d_response.txt)

事件 2 — [llm_5ba8370135ce4d669c2a0f4727c2d213](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "2021-001T00:00:00Z",
  "offending_statements": "Line 7: return datetime.fromisoformat(ts)",
  "reason": "The specification requires that any valid ISO 8601 timestamp string be successfully parsed and returned as a datetime object. The code delegates parsing to datetime.fromisoformat, which does not support all ISO 8601 formats. For example, the ordinal date timestamp '2021-001T00:00:00Z' is valid per ISO 8601 but fromisoformat cannot parse it, causing the function to return None instead of a datetime, violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5ba8370135ce4d669c2a0f4727c2d213_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5ba8370135ce4d669c2a0f4727c2d213_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5ba8370135ce4d669c2a0f4727c2d213_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 6 — Reasoning & Verification Engine / orchestration; 原源码 dashboard.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**： For each field name in keys, in order, probes d for both the plain field name and its decorator-prefixed variants. Returns the value of the first field that has a truthy value in d. Returns None if no field named by keys has a truthy value under any recognized key form.
- **Actual behavior**： After the function executes, the returned value ret satisfies: if d is not a dict, ret = None; if d is a dict, let C be the concatenated sequence of candidate strings obtained by iterating over the input keys in the given order and, for each key k, appending k, then '+' + k, then '*' + k. If no candidate in C is a key of d, ret = None; otherwise, let c be the first candidate in C that is a key of d, and ret = d[c].
- **Code evidence**： Line 7: if candidate in d: Line 8: return d[candidate]
- **Trigger condition**： The code checks key existence in d, but the specification requires the field to have a truthy value. A falsy value for an existing key causes code to return that falsy value, while the specification expects it to be skipped.
- **Validator trigger**： Passing a dict with a key holding a falsy value (e.g. 0) causes _trace_value to return that falsy value instead of skipping it as the spec requires.
- **Probe stdout**： CONFIRMED — actual: 0 | expected (spec-correct): None

► SPEC sidecar（完整）

```
{
  "signature": "_trace_value(d, *keys) -> Any",
  "pre_condition": "d is a dict representing a parsed trace record whose string keys may include streaming-delta decorator prefixes. keys is a sequence of string field names ordered by lookup priority.",
  "post_condition": "For each field name in keys, in order, probes d for both the plain field name and its decorator-prefixed variants. Returns the value of the first field that has a truthy value in d. Returns None if no field named by keys has a truthy value under any recognized key form."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/_trace_value.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "For each field name in keys, in order, probes d for both the plain field name and its decorator-prefixed variants. Returns the value of the first field that has a truthy value in d. Returns None if no field named by keys has a truthy value under any recognized key form.",
    "actual_behavior": "After the function executes, the returned value ret satisfies: if d is not a dict, ret = None; if d is a dict, let C be the concatenated sequence of candidate strings obtained by iterating over the input keys in the given order and, for each key k, appending k, then '+' + k, then '*' + k. If no candidate in C is a key of d, ret = None; otherwise, let c be the first candidate in C that is a key of d, and ret = d[c].",
    "code_evidence": "Line 7: if candidate in d:\nLine 8: return d[candidate]",
    "trigger_condition": "The code checks key existence in d, but the specification requires the field to have a truthy value. A falsy value for an existing key causes code to return that falsy value, while the specification expects it to be skipped."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "dashboard-py--_trace_value",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/_trace_value.py",
```

```
"function_name": "_trace_value",
"confirmation_status": "confirmed",
"attempts": "1",
"probe_script": "fm_agent/bug_validation/probe_dashboard-py--_trace_value.py",
"detail_file": "fm_agent/bug_validation/dashboard-py--_trace_value.md",
"probe_stdout": "CONFIRMED - actual: 0 | expected (spec-correct): None\n",
"trigger_summary": "Passing a dict with a key holding a falsy value (e.g. 0) causes _trace_value to return that falsy value
instead of skipping it as the spec requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:45:41.167084Z	2175
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:46:15.105573Z	1300

事件 1 — [llm_0db94343436649a59e77d10eaea34243](#)

► parsed_json（完整）

```
{
  "post_condition": "After the function executes, the returned value ret satisfies: if d is not a dict, ret = None; if d is a dict, let C be the concatenated sequence of candidate strings obtained by iterating over the input keys in the given order and, for each key k, appending k, then '+' + k, then '*' + k. If no candidate in C is a key of d, ret = None; otherwise, let c be the first candidate in C that is a key of d, and ret = d[c]."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_0db94343436649a59e77d10eaea34243_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_0db94343436649a59e77d10eaea34243_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_0db94343436649a59e77d10eaea34243_response.txt)

事件 2 — [llm_8b9a92bcd6ca4eadaf0bdd4bc6eb5774](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input: d={'a': 0}, keys=['a']; code returns 0 (because 'a' exists as key), specification requires None (because 0 is not truthy).",
  "offending_statements": "Line 7: if candidate in d:\nLine 8: return d[candidate]",
  "reason": "The code checks key existence in d, but the specification requires the field to have a truthy value. A falsy value for an existing key causes code to return that falsy value, while the specification expects it to be skipped."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_8b9a92bcd6ca4eadaf0bdd4bc6eb5774_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_8b9a92bcd6ca4eadaf0bdd4bc6eb5774_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_8b9a92bcd6ca4eadaf0bdd4bc6eb5774_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-269 — [dashboard-py--render_llm_status](#)

- 四类结论： 契约不确定
- 分类依据： 沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 6 — Reasoning & Verification Engine / orchestration; 原源码 dashboard.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim：** Returns a Rich Panel renderable with title 'LLM Calls' and cyan border-style. When state.llm_statuses is empty, the panel displays a vertically centered, dim-text message indicating that no LLM calls have occurred. When state.llm_statuses is non-empty, the panel displays three vertically stacked components: (a) a metrics line showing the number of recent status items (up to LLM_STATUS_WINDOW) and the percentage of those items whose resolved status label equals '200', formatted to one decimal place and

- colored green when 95%, yellow when 80%, and red when <80%; (b) a horizontal strip of colored symbols, one per status item from the trailing window of up to 50 items, where each symbol's color and shape are a deterministic function of that item's code and status values; (c) a table of the trailing up to 6 items in reverse chronological order, with columns for a timestamp, a source identifier, a color-coded status label, and a call description. The state argument is not mutated.
- **Actual behavior:** The function returns a rich.panel.Panel with title="LLM Calls" and border_style="cyan". Let S = list(state.llm_statuses)[-LLM_STATUS_WINDOW:]. If S is empty, the Panel encloses a center-aligned Text displaying "(no LLM calls yet)" in dim style. If S is non-empty, the Panel encloses a Table.grid (expand=True) with two rows: the first row is a Text consisting of a markup line "{|S|}/{LLM_STATUS_WINDOW} calls" followed by a sparkline (a Text strip) of the last up to 50 items of S, each mapped via _llm_status_style(code, status) to a color and symbol (if label=="200", if color=="yellow", else); the second row is a Table with columns time, src, code, call, showing the last up to 6 items of S in reverse order (timeitem.get("time", ""), srcitem.get("source", "?"), codecolored label, callitem.get("label", "")). The function does not mutate any external state and requires that _llm_status_style is deterministic. No exceptions are raised if state.llm_statuses items contain the expected keys. Formally: post_condition(state, result) isinstance(result, Panel) result.title = "LLM Calls" result.border_style = "cyan" (s S, s has 'code' and 'status' keys) ((|S|=0 result.renderable = Align.center(Text.from_markup("[dim](no LLM calls yet)[])", vertical="middle")) (|S|>0 result.renderable = Table.grid(expand=True) containing the sparkline Text and detail Table as described with exactly |S| and the specified derivations)).
 - **Code evidence:** Line 2: statuses = list(state.llm_statuses)[-LLM_STATUS_WINDOW:] Line 19: for item in list(reversed(statuses[-6:]))
 - **Trigger condition:** When state.llm_statuses has more than LLM_STATUS_WINDOW items, the detail table only draws from the windowed statuses, thus it can show fewer than 6 items even if the full list has 6 or more. The specification requires the table to display the trailing up to 6 items from the entire status list, not only the metrics window.
 - **Validator trigger:** Detail table slice ([-6:]) is applied to the already-windowed list ([-LLM_STATUS_WINDOW:]) instead of the full list, so when LLM_STATUS_WINDOW < 6 and the full list has >=6 items, the table shows fewer than 6 rows.
 - **Probe stdout:** CONFIRMED — detail table shows 5 rows, spec requires up to 6 rows from the full list (20 items). Window (LLM_STATUS_WINDOW=5) limited to 5 items, detail table draws from windowed tail 5 rows, full list tail would provide 6 rows.

► SPEC sidecar (完整)

```
{
  "signature": "render_llm_status(state: State) -> Rich-renderable",
  "pre_condition": "state is a State instance whose llm_statuses attribute is an iterable collection of dictionaries, each containing at minimum 'code' and 'status' keys. LLM_STATUS_WINDOW is a positive integer constant defined in the enclosing scope.",
  "post_condition": "Returns a Rich Panel renderable with title 'LLM Calls' and cyan border-style. When state.llm_statuses is empty, the panel displays a vertically centered, dim-text message indicating that no LLM calls have occurred. When state.llm_statuses is non-empty, the panel displays three vertically stacked components: (a) a metrics line showing the number of recent status items (up to LLM_STATUS_WINDOW) and the percentage of those items whose resolved status label equals '200', formatted to one decimal place and colored green when ≥95%, yellow when ≥80%, and red when <80%; (b) a horizontal strip of colored symbols, one per status item from the trailing window of up to 50 items, where each symbol's color and shape are a deterministic function of that item's code and status values; (c) a table of the trailing up to 6 items in reverse chronological order, with columns for a timestamp, a source identifier, a color-coded status label, and a call description. The state argument is not mutated."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_llm_status_style",
      "signature": "_llm_status_style(code, status) -> (color, label)",
      "pre_condition": "code and status are values obtained from an item in state.llm_statuses. Either or both may be None if the corresponding keys are absent from the item dictionary.",
      "post_condition": "Returns a tuple (color, label) where color is a Rich-compatible style name and label is a string. The mapping from (code, status) input pairs to (color, label) output pairs is deterministic: identical inputs always produce identical outputs."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/render_llm_status.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a Rich Panel renderable with title 'LLM Calls' and cyan border-style. When state.llm_statuses is empty, the panel displays a vertically centered, dim-text message indicating that no LLM calls have occurred. When state.llm_statuses is non-empty, the panel displays three vertically stacked components: (a) a metrics line showing the number of recent status items (up to LLM_STATUS_WINDOW) and the percentage of those items whose resolved status label equals '200', formatted to one decimal place and colored green when 95%, yellow when 80%, and red when <80%; (b) a horizontal strip of colored symbols, one per status
```

```
item from the trailing up of up to 50 items, where each symbol's color and shape are a deterministic function of that item's
code and status values; (c) a table of the trailing up to 6 items in reverse chronological order, with columns for a timestamp, a
source identifier, a color-coded status label, and a call description. The state argument is not mutated.",
    "actual_behavior": "The function returns a rich.panel.Panel with title=\"LLM Calls\" and border_style=\"cyan\". Let S =
list(state.llm_statuses)[-LLM_STATUS_WINDOW:]. If S is empty, the Panel encloses a center-aligned Text displaying \"(no LLM calls
yet)\" in dim style. If S is non-empty, the Panel encloses a Table.grid (expand=True) with two rows: the first row is a Text
consisting of a markup line \"recent {|S|}/{LLM_STATUS_WINDOW} calls\" followed by a sparkline (a Text strip) of the last up to 50
items of S, each mapped via _llm_status_style(code, status) to a color and symbol ( if label==\"200\", if color==\"yellow\", else
); the second row is a Table with columns time, src, code, call, showing the last up to 6 items of S in reverse order
(timeitem.get(\"time\", \"\"), srcitem.get(\"source\", \"?\"), codecolored label, callitem.get(\"label\", \"\")). The function does
not mutate any external state and requires that _llm_status_style is deterministic. No exceptions are raised if state.llm_statuses
items contain the expected keys. Formally: post_condition(state, result) isinstance(result, Panel) result.title = \"LLM Calls\"
result.border_style = \"cyan\" ( s S, s has 'code' and 'status' keys) ( {|S|=0 result.renderable =
Align.center(Text.from_markup(\"[dim](no LLM calls yet)[/]\"), vertical=\"middle\")) (|S|>0 result.renderable =
Table.grid(expand=True) containing the sparkline Text and detail Table as described with exactly |S| and the specified
derivations)).",
    "code_evidence": "Line 2: statuses = list(state.llm_statuses)[-LLM_STATUS_WINDOW:]\\nLine 19: for item in
list(reversed(statuses[-6:]))",
    "trigger_condition": "When state.llm_statuses has more than LLM_STATUS_WINDOW items, the detail table only draws from the
windowed statuses, thus it can show fewer than 6 items even if the full list has 6 or more. The specification requires the table
to display the trailing up to 6 items from the entire status list, not only the metrics window."
}
```

► Bug Validator result (完整)

```
{
  "id": "dashboard-py--render_llm_status",
  "source_file": "dashboard-py/render_llm_status.py",
  "function_name": "render_llm_status",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_dashboard-py--render_llm_status.py",
  "detail_file": "fm_agent/bug_validation/dashboard-py--render_llm_status.md",
  "probe_stdout": "CONFIRMED - detail table shows 5 rows, spec requires up to 6 rows from the full list (20 items). Window
(LLM_STATUS_WINDOW=5) limited to 5 items, detail table draws from windowed tail 5 rows, full list tail would provide 6 rows.\\n",
  "trigger_summary": "Detail table slice ([-6:]) is applied to the already-windowed list ([-LLM_STATUS_WINDOW:]) instead of the
full list, so when LLM_STATUS_WINDOW < 6 and the full list has >=6 items, the table shows fewer than 6 rows."
}
```

► Reasoner 流程 (3 条事件)

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:24:18.303182Z	3307
2	check_post_implies_spec	format_error	1	0/1	2026-07-28T04:25:00.872655Z	9428
3	check_post_implies_spec	mismatch	2	0/1	2026-07-28T04:27:02.599251Z	6431

事件 1 — [llm_b45a5e3aeb674f4e89fef9dde44e1c63](#)

► parsed_json (完整)

```
{
  "post_condition": "The function returns a rich.panel.Panel with title=\"LLM Calls\" and border_style=\"cyan\". Let S =
list(state.llm_statuses)[-LLM_STATUS_WINDOW:]. If S is empty, the Panel encloses a center-aligned Text displaying \"(no LLM calls
yet)\" in dim style. If S is non-empty, the Panel encloses a Table.grid (expand=True) with two rows: the first row is a Text
consisting of a markup line \"recent {|S|}/{LLM_STATUS_WINDOW} calls\" followed by a sparkline (a Text strip) of the last up to 50
items of S, each mapped via _llm_status_style(code, status) to a color and symbol (■ if label==\"200\", ▲ if color==\"yellow\",
else ■); the second row is a Table with columns time, src, code, call, showing the last up to 6 items of S in reverse order
(time-item.get(\"time\", \"\"), src-item.get(\"source\", \"?\"), code-colored label, call-item.get(\"label\", \"\")). The function
does not mutate any external state and requires that _llm_status_style is deterministic. No exceptions are raised if
state.llm_statuses items contain the expected keys. Formally: post_condition(state, result) = isinstance(result, Panel) ^
result.title = \"LLM Calls\" ^ result.border_style = \"cyan\" ^ (∀ s ∈ S, s has 'code' and 'status' keys) ^ ( {|S|=0 ⇒
result.renderable = Align.center(Text.from_markup(\"[dim](no LLM calls yet)[/]\"), vertical=\"middle\")) ^ (|S|>0 ⇒
result.renderable = Table.grid(expand=True) containing the sparkline Text and detail Table as described with exactly |S| and the
specified derivations))."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b45a5e3aeb674f4e89fef9dde44e1c63_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_b45a5e3aeb674f4e89fef9dde44e1c63_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_b45a5e3aeb674f4e89fef9dde44e1c63_response.txt)

事件 2 — [llm_308ef35dfddb4b8bbb67cf60ff4032de](#)

- **parse_error:** [spec-check response is not valid JSON: LLM response is not valid JSON: Invalid \\escape: line 3 column 44 \(char 70\): line 1 column 1 \(char 0\)](#)
- **载荷:** [system_prompt](#) · [user_prompt](#) · [assistant_output](#)

事件 3 — [llm_21d10ee3c4b949ac95eb5d9afd642bc9](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "state.llm_statuses = [{ 'time': '10:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 10'},
{ 'time': '09:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 9'}, { 'time': '08:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 8'}, { 'time': '07:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 7'},
{ 'time': '06:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 6'}, { 'time': '05:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 5'}, { 'time': '04:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 4'},
{ 'time': '03:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 3'}, { 'time': '02:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 2'}, { 'time': '01:00', 'source': 'test', 'code': 200, 'status': 'OK', 'label': 'call 1'}]
(length 10, most recent last); LLM_STATUS_WINDOW = 3.",
  "offending_statements": "Line 2: statuses = list(state.llm_statuses)[-LLM_STATUS_WINDOW:]\nLine 19: for item in
list(reversed(statuses[-6:]))",
  "reason": "When state.llm_statuses has more than LLM_STATUS_WINDOW items, the detail table only draws from the windowed
statuses, thus it can show fewer than 6 items even if the full list has 6 or more. The specification requires the table to display
the trailing up to 6 items from the entire status list, not only the metrics window."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_21d10ee3c4b949ac95eb5d9afd642bc9_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_21d10ee3c4b949ac95eb5d9afd642bc9_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_21d10ee3c4b949ac95eb5d9afd642bc9_message_02_assistant.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_21d10ee3c4b949ac95eb5d9afd642bc9_message_03_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_21d10ee3c4b949ac95eb5d9afd642bc9_response.txt\)](#) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-270 — [main.py--_clean_previous_run](#)

- **四类结论:** [契约不确定](#)
- **分类依据:** 沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论: 沿用旧清单的逐条审计结论。
- **Validator:** [confirmed](#); [attempts 1](#)
- **Phase / 模块:** [Phase 6 — Reasoning & Verification Engine / orchestration](#); 原源码 [main.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** The filesystem path denoted by `work_dir` no longer refers to an existing directory. If the path referred to a directory before the call, that directory and all files and subdirectories within it have been recursively removed from the filesystem.
- **Actual behavior:** If the function completes without raising an exception, then the following holds: if `os.path.isdir(work_dir)` was true before the call, then `os.path.exists(work_dir)` is false after the call (the directory and its contents have been removed); if `work_dir` was not a directory (or did not exist), the filesystem state at `work_dir` is unchanged. If the function raises an exception, the state of `work_dir` is unspecified (may be partially removed or unchanged). Formally: $(\text{normal_return} \rightarrow (\text{is_dir}(\text{work_dir}) \rightarrow \neg \text{exists}(\text{work_dir}) \rightarrow \text{unchanged}(\text{work_dir}))) \vee (\text{exceptional_return} \rightarrow \text{top})$
- **Code evidence:** Line 3: `if os.path.isdir(work_dir):` Line 4: `shutil.rmtree(work_dir)`
- **Trigger condition:** `os.path.isdir` follows symlinks, so a symlink to a directory is considered a directory and the condition is true. However, `shutil.rmtree` removes the symlink itself, not the target directory. The target directory and its contents remain, violating the requirement that the directory and all its contents are recursively removed.
- **Validator trigger:** When `work_dir` is a symlink to a directory, `os.path.isdir` returns True but `shutil.rmtree` raises `OSError` (Python 3.12+) or removes only the symlink, leaving the target directory untouched.
- **Probe stdout:** CONFIRMED `u2014` `shutil.rmtree` raised `OSError` on symlink (Cannot call `rmtree` on a symbolic link); target directory and contents untouched at: `'/tmp/tmpj6g9l18x/real_target'` `\n` marker file intact: `'/tmp/tmpj6g9l18x/real_target/important_data.txt'`

► [SPEC sidecar](#) (完整)

```
{
  "signature": "_clean_previous_run(work_dir)",
```

```
    "pre_condition": "work_dir is a string that denotes a path on the filesystem.",
    "post_condition": "The filesystem path denoted by work_dir no longer refers to an existing directory. If the path referred to a directory before the call, that directory and all files and subdirectories within it have been recursively removed from the filesystem."
  }
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/main-py/_clean_previous_run.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "The filesystem path denoted by work_dir no longer refers to an existing directory. If the path referred to a directory before the call, that directory and all files and subdirectories within it have been recursively removed from the filesystem.",
    "actual_behavior": "If the function completes without raising an exception, then the following holds: if `os.path.isdir(work_dir)` was true before the call, then `os.path.exists(work_dir)` is false after the call (the directory and its contents have been removed); if `work_dir` was not a directory (or did not exist), the filesystem state at `work_dir` is unchanged. If the function raises an exception, the state of `work_dir` is unspecified (may be partially removed or unchanged). Formally:  $(\\text{is\\_dir}(work\_dir) \\rightarrow \\neg \\text{exists}(work\_dir)) \\wedge (\\neg \\text{is\\_dir}(work\_dir) \\rightarrow \\text{unchanged}(work\_dir))$ ",
    "code_evidence": "Line 3:     if os.path.isdir(work_dir):\\nLine 4:         shutil.rmtree(work_dir)",
    "trigger_condition": "os.path.isdir follows symlinks, so a symlink to a directory is considered a directory and the condition is true. However, shutil.rmtree removes the symlink itself, not the target directory. The target directory and its contents remain, violating the requirement that the directory and all its contents are recursively removed."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "main-py--_clean_previous_run",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/main-py/_clean_previous_run.py",
  "function_name": "_clean_previous_run",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_main-py--_clean_previous_run.py",
  "detail_file": "fm_agent/bug_validation/main-py--_clean_previous_run.md",
  "probe_stdout": "CONFIRMED \\u2014\\n 14 shutil.rmtree raised OSError on symlink (Cannot call rmtree on a symbolic link); target directory and contents untouched at: '/tmp/tmpj6g9l18x/real_target'\\n  marker file intact: '/tmp/tmpj6g9l18x/real_target/important_data.txt'",
  "trigger_summary": "When work_dir is a symlink to a directory, os.path.isdir returns True but shutil.rmtree raises OSError (Python 3.12+) or removes only the symlink, leaving the target directory untouched."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:44:35.074310Z	1812
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:45:05.654801Z	2775

事件 1 — [llm_91080a4292e24638b63bbc8701746743](#)

► parsed_json（完整）

```
{
  "post_condition": "If the function completes without raising an exception, then the following holds: if `os.path.isdir(work_dir)` was true before the call, then `os.path.exists(work_dir)` is false after the call (the directory and its contents have been removed); if `work_dir` was not a directory (or did not exist), the filesystem state at `work_dir` is unchanged. If the function raises an exception, the state of `work_dir` is unspecified (may be partially removed or unchanged). Formally:  $(\\text{is\\_dir}(work\_dir) \\rightarrow \\neg \\text{exists}(work\_dir)) \\wedge (\\neg \\text{is\\_dir}(work\_dir) \\rightarrow \\text{unchanged}(work\_dir))$ "
}
```



```
\\wedge (\\text{exceptional}\\_return} \\rightarrow \\top ) \\)"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_91080a4292e24638b63bbc8701746743_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_91080a4292e24638b63bbc8701746743_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_91080a4292e24638b63bbc8701746743_response.txt)

事件 2 — llm_5e35e7b909674ee58d9a26d0cfc21bf5

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "work_dir is a symbolic link pointing to an existing directory (e.g., /tmp/link -> /tmp/readdir).",
  "offending_statements": "Line 3:     if os.path.isdir(work_dir):\\nLine 4:         shutil.rmtree(work_dir)",
  "reason": "os.path.isdir follows symlinks, so a symlink to a directory is considered a directory and the condition is true. However, shutil.rmtree removes the symlink itself, not the target directory. The target directory and its contents remain, violating the requirement that the directory and all its contents are recursively removed."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5e35e7b909674ee58d9a26d0cfc21bf5_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5e35e7b909674ee58d9a26d0cfc21bf5_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5e35e7b909674ee58d9a26d0cfc21bf5_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: **scope_analysis**

FMA-MISMATCH-291 — src--scope-py--_score_class

- 四类结论: 契约不确定
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- **Validator:** confirmed; attempts 1
- **Phase / 模块:** Phase 6 — Reasoning & Verification Engine / scope_analysis; 原源码 src/scope.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** Returns a non-negative float. The score is the sum of the following weighted components: (a) for each token in the class name's constituent parts that matches a token in signals['backtick_idents'], a fixed backtick-name overlap weight; (b) for each token in the class name's constituent parts that matches a token in signals['plain_idents'], a fixed class-name overlap weight; (c) for each token in the class name's constituent parts that matches a token in signals['all_words'], the same fixed class-name overlap weight; (d) for each token in the class name's constituent parts that matches a token in signals['dotted_classes'], a fixed dotted-reference overlap weight; (e) when cls['docstring'] is non-empty and truthy: for each alphabetic word of four or more characters in the docstring that is not a stop word and matches a token in signals['all_words'], a fixed class-doc overlap weight. When cls['docstring'] is empty or falsy, component (e) contributes zero. The returned value is exactly zero when no name-part token and no qualifying docstring-word token overlaps any of the specified signal sets.
- **Actual behavior:** The function returns a non-negative float score computed as: let name = cls['name']; doc = cls['docstring']; name_parts = _name_parts(name). score = |name_parts signals['backtick_idents']| * W_BACKTICK_NAME + |name_parts signals['plain_idents']| * W_CLASS_NAME_MATCH + |name_parts signals['all_words']| * W_CLASS_NAME_MATCH + |name_parts signals['dotted_classes']| * W_DOTTED_REF + (if doc " then |{w | w re.findall(r'\b([a-zA-Z]{4,})\b', doc.lower()) w _STOP} signals['all_words']| * W_CLASS_DOC_MATCH else 0). The inputs cls and signals are not modified; no exceptions are raised.
- **Code evidence:** Line 15: re.findall(r'\b([a-zA-Z]{4,})\b', cls['docstring'].lower())
- **Trigger condition:** The regex restricts word extraction to ASCII letters [a-zA-Z], but the specification requires extraction of all alphabetic words (including non-ASCII letters). For docstring 'nave', the code finds no matching words and scores 0, while the specification would score W_CLASS_DOC_MATCH because 'nave' is an alphabetic word of five characters and appears in signals['all_words'].
- **Validator trigger:** ASCII-only regex [a-zA-Z] fails to match non-ASCII alphabetic words in class docstrings, causing _score_class to miss qualifying docstring-word overlaps with signals['all_words'].
- **Probe stdout:** CONFIRMED — actual: 0.0 | expected: 2.0

► SPEC sidecar (完整)

```
{
  "signature": "_score_class(cls: dict, signals: dict[str, set[str]]) -> float",
  "pre_condition": "cls is a dict containing at minimum the keys 'name' (string, a class identifier) and 'docstring' (string, the class's docstring or the empty string). signals is a dict keyed by signal-category strings, each value being a set of lowercase string tokens. The dict contains at minimum the keys 'backtick_idsents', 'plain_idsents', 'all_words', and 'dotted_classes'.",
  "post_condition": "Returns a non-negative float. The score is the sum of the following weighted components: (a) for each token in the class name's constituent parts that matches a token in signals['backtick_idsents'], a fixed backtick-name overlap weight; (b) for each token in the class name's constituent parts that matches a token in signals['plain_idsents'], a fixed class-name overlap weight; (c) for each token in the class name's constituent parts that matches a token in signals['all_words'], the same fixed class-name overlap weight; (d) for each token in the class name's constituent parts that matches a token in signals['dotted_classes'], a fixed dotted-reference overlap weight; (e) when cls['docstring'] is non-empty and truthy: for each alphabetic word of four or more characters in the docstring that is not a stop word and matches a token in signals['all_words'], a fixed class-doc overlap weight. When cls['docstring'] is empty or falsy, component (e) contributes zero. The returned value is exactly zero when no name-part token and no qualifying docstring-word token overlaps any of the specified signal sets."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_name_parts",
      "signature": "_name_parts(name: str) -> set[str]",
      "pre_condition": "name is a non-empty string representing a class identifier.",
      "post_condition": "Returns a set of lowercase string tokens obtained by splitting the class name on word boundaries – including underscore separators and transitions between lowercase and uppercase letters – such that each returned token is a semantic constituent part of the original identifier."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/scope-py/_score_class.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a non-negative float. The score is the sum of the following weighted components: (a) for each token in the class name's constituent parts that matches a token in signals['backtick_idsents'], a fixed backtick-name overlap weight; (b) for each token in the class name's constituent parts that matches a token in signals['plain_idsents'], a fixed class-name overlap weight; (c) for each token in the class name's constituent parts that matches a token in signals['all_words'], the same fixed class-name overlap weight; (d) for each token in the class name's constituent parts that matches a token in signals['dotted_classes'], a fixed dotted-reference overlap weight; (e) when cls['docstring'] is non-empty and truthy: for each alphabetic word of four or more characters in the docstring that is not a stop word and matches a token in signals['all_words'], a fixed class-doc overlap weight. When cls['docstring'] is empty or falsy, component (e) contributes zero. The returned value is exactly zero when no name-part token and no qualifying docstring-word token overlaps any of the specified signal sets.",
    "actual_behavior": "The function returns a non-negative float score computed as: let name = cls['name']; doc = cls['docstring']; name_parts = _name_parts(name). score = |name_parts signals['backtick_idsents']| * W_BACKTICK_NAME + |name_parts signals['plain_idsents']| * W_CLASS_NAME_MATCH + |name_parts signals['all_words']| * W_CLASS_NAME_MATCH + |name_parts signals['dotted_classes']| * W_DOTTED_REF + ( if doc '' then |{w | w re.findall(r'\\b([a-zA-Z]{4,})\\b', doc.lower()) w _STOP} signals['all_words']| * W_CLASS_DOC_MATCH else 0 ). The inputs cls and signals are not modified; no exceptions are raised.",
    "code_evidence": "Line 15: re.findall(r'\\b([a-zA-Z]{4,})\\b', cls['docstring'].lower()),
    "trigger_condition": "The regex restricts word extraction to ASCII letters [a-zA-Z], but the specification requires extraction of all alphabetic words (including non-ASCII letters). For docstring 'nave', the code finds no matching words and scores 0, while the specification would score W_CLASS_DOC_MATCH because 'nave' is an alphabetic word of five characters and appears in signals['all_words']."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--scope-py--_score_class",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/scope-py/_score_class.py",
  "function_name": "_score_class",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--scope-py--_score_class.py",
  "detail_file": "fm_agent/bug_validation/src--scope-py--_score_class.md",
  "probe_stdout": "CONFIRMED – actual: 0.0 | expected: 2.0\n",
  "trigger_summary": "ASCII-only regex [a-zA-Z] fails to match non-ASCII alphabetic words in class docstrings, causing _score_class to miss qualifying docstring-word overlaps with signals['all_words'].",
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T05:13:15.432429Z	2193
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T05:13:37.303103Z	7998

事件 1 — [llm_0d94d1ab4b01478d9143a0cf9738e02f](#)

► [parsed_json](#)（完整）

```
{
  "post_condition": "The function returns a non-negative float score computed as: let name = cls['name']; doc = cls['docstring'];
name_parts = _name_parts(name). score = |name_parts ∩ signals['backtick_idsents']| * W_BACKTICK_NAME + |name_parts ∩
signals['plain_idsents']| * W_CLASS_NAME_MATCH + |name_parts ∩ signals['all_words']| * W_CLASS_NAME_MATCH + |name_parts ∩
signals['dotted_classes']| * W_DOTTED_REF + ( if doc ≠ '' then |{w | w ∈ re.findall(r'\b([a-zA-Z]{4,})\b', doc.lower()) ∧ w ∉
_STOP} ∩ signals['all_words']| * W_CLASS_DOC_MATCH else 0 ). The inputs cls and signals are not modified; no exceptions are
raised."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_0d94d1ab4b01478d9143a0cf9738e02f_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_0d94d1ab4b01478d9143a0cf9738e02f_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_0d94d1ab4b01478d9143a0cf9738e02f_response.txt\)](#)

事件 2 — [llm_bb457c53155d4cdeb07208b28d856343](#)

► [parsed_json](#)（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "cls = {'name': 'MyClass', 'docstring': 'naïve'}; signals = {'backtick_idsents': set(), 'plain_idsents': set(),
'all_words': {'naïve'}, 'dotted_classes': set()}; All weights positive, e.g., W_CLASS_DOC_MATCH = 1.0.",
  "offending_statements": "Line 15: re.findall(r'\b([a-zA-Z]{4,})\b', cls['docstring'].lower())",
  "reason": "The regex restricts word extraction to ASCII letters [a-zA-Z], but the specification requires extraction of all
alphabetic words (including non-ASCII letters). For docstring 'naïve', the code finds no matching words and scores 0, while the
specification would score W_CLASS_DOC_MATCH because 'naïve' is an alphabetic word of five characters and appears in
signals['all_words']."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_bb457c53155d4cdeb07208b28d856343_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_bb457c53155d4cdeb07208b28d856343_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_bb457c53155d4cdeb07208b28d856343_response.txt\)](#) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

模块: [spec_orchestrator](#)

FMA-MISMATCH-293 — [src--spec_generation_and_verification-py--_run_spec_generation_batch](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts 1
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / [spec_orchestrator](#); 原源码 [src/spec_generation_and_verification.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC claim:** Dispatches an OpenCode subprocess instructed to generate a .spec.json and .info.json sidecar for every function listed in batch_info['functions'] by following fm_agent/workflow_spec_step4_batch.md and fm_agent/spec_prompts/system_prompt.md. Records a structured trace event under stage='spec_generation' that identifies the function IDs derived from the function file paths and catalogs all specified input and output files. Returns an integer: 0 when the subprocess exits successfully, or the non-zero returncode when the subprocess exits with an error or crashes with a CalledProcessError. When attempt equals 1, the subprocess is instructed to generate all sidecars from scratch. When attempt is greater than 1, the subprocess is instructed to inspect each function's existing .spec.json and

.info.json sidecars, preserve those whose content matches the schema defined in fm_agent/spec_prompts/system_prompt.md, and regenerate only those that are missing, unparseable, or schema-nonconforming.

- **Actual behavior:** The function returns an integer r. If no exception other than subprocess.CalledProcessError occurs, r is the return code of the subprocess launched to generate spec files. If run_opencode_traced raises CalledProcessError, it is caught and r is that exception's returncode; otherwise r is the CompletedProcess.returncode (0 for success, non-zero for failure). The subprocess is built with build_llm_cli_command(model=OPENCODE_SPEC_MODEL, prompt=p, cwd=proj_dir, files=[prompt_file]) where p is a prompt string that depends on attempt: if attempt == 1, p is an initial generation instruction; otherwise p is a continuation instruction that asks to skip functions only when both .spec.json and .info.json already exist and are valid. The executed command receives context from fm_agent/workflow_spec_step4_batch.md, the batch prompt file (batch_rel_dir/batch_file), fm_agent/spec_prompts/system_prompt.md, and any staged domain knowledge files from work_dir. It is expected to produce .spec.json and .info.json files for each function in batch_info['functions']. The function itself does not guarantee these output files exist or are valid. It does not modify any existing project files. If run_opencode_traced executes without raising CalledProcessError, a structured trace event is recorded. Any exception not of type subprocess.CalledProcessError propagates out of the function.
- **Code evidence:** Line 36: f"is missing, malformed, or schema-invalid, rewrite the complete " Line 37: f".spec.json and .info.json files for that function. " Line 38: f"Do not modify the function source files. {fm_reminder}"
- **Trigger condition:** For attempt > 1, the specification states: "preserve those whose content matches the schema ... and regenerate only those that are missing, unparseable, or schema-nonconforming." This requires preserving each individually valid sidecar file. The prompt in the code, however, instructs the subprocess to rewrite both .spec.json and .info.json for a function whenever either sidecar is missing/invalid, failing to preserve a still-valid partner file.
- **Validator trigger:** For attempt > 1, the prompt tells the OpenCode subprocess to rewrite BOTH .spec.json and .info.json when either sidecar is invalid, rather than individually preserving each valid sidecar as the spec requires.
- **Probe stdout:** CONFIRMED — prompt instructs rewriting BOTH sidecars when only one may be invalid Matched buggy keywords in prompt: ['either sidecar', 'rewrite the complete', '.spec.json and .info.json files'] Spec requires: individually preserve valid sidecars, regenerate only invalid ones Actual prompt: instructs rewriting the complete .spec.json and .info.json when either is bad

► SPEC sidecar (完整)

```
{
  "signature": "_run_spec_generation_batch(proj_dir, work_dir, attempt, phase_num, layer_idx, batch_rel_dir, batch_info) -> int",
  "pre_condition": "proj_dir is a valid filesystem path to the project root directory. work_dir is a valid path to the FM-Agent workspace directory. batch_info is a dict containing the key 'file' whose value is a string naming a batch prompt file, and the key 'functions' whose value is a non-empty list of strings naming extracted function files relative to proj_dir. batch_rel_dir is a relative path within proj_dir that, joined with batch_info['file'], resolves to an existing batch prompt file. fm_agent/workflow_spec_step4_batch.md exists at proj_dir and contains the spec-generation workflow instructions. fm_agent/spec_prompts/system_prompt.md exists at proj_dir and contains mandatory spec format rules.",
  "post_condition": "Dispatches an OpenCode subprocess instructed to generate a .spec.json and .info.json sidecar for every function listed in batch_info['functions'] by following fm_agent/workflow_spec_step4_batch.md and fm_agent/spec_prompts/system_prompt.md. Records a structured trace event under stage='spec_generation' that identifies the function IDs derived from the function file paths and catalogs all specified input and output files. Returns an integer: 0 when the subprocess exits successfully, or the non-zero returncode when the subprocess exits with an error or crashes with a CalledProcessError. When attempt equals 1, the subprocess is instructed to generate all sidecars from scratch. When attempt is greater than 1, the subprocess is instructed to inspect each function's existing .spec.json and .info.json sidecars, preserve those whose content matches the schema defined in fm_agent/spec_prompts/system_prompt.md, and regenerate only those that are missing, unparseable, or schema-nonconforming."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "function_id_from_extracted_path",
      "signature": "function_id_from_extracted_path(func_rel: str) -> str",
      "pre_condition": "func_rel is a string representing a relative path to an extracted function file within the project.",
      "post_condition": "Returns a string identifier that uniquely corresponds to the extracted function referenced by func_rel."
    },
    {
      "name": "build_llm_cli_command",
      "signature": "build_llm_cli_command(model, prompt, cwd, files) -> list[str]",
      "pre_condition": "model is a string identifying the LLM model. prompt is a non-empty string containing the instruction text. cwd is a valid filesystem path. files is a list of file path strings that must be provided as context to the LLM invocation.",
      "post_condition": "Returns an ordered list of command-line tokens that, when executed as a subprocess, invoke the configured LLM CLI backend with the given model, the given prompt as an instruction, the given cwd as working directory, and the given file paths as input context."
    },
    {
      "name": "run_opencode_traced",
      "signature": "run_opencode_traced(proj_dir, work_dir, command, stage, function_ids, input_files, output_files, summary, metadata) -> CompletedProcess",
      "pre_condition": "proj_dir and work_dir are valid filesystem paths. command is a non-empty list of CLI tokens. stage is a string identifying the pipeline stage. function_ids is a list of string identifiers for the functions being processed. input_files
```

```

and output_files are lists of relative file paths.",
    "post_condition": "Executes the given command as a subprocess. Records a structured trace event to the workspace trace directory with timestamp, event_id, the given stage label, the function_ids, the input_files and output_files references, the summary string, and the metadata dict. Returns a CompletedProcess whose returncode is 0 on success and non-zero on failure. On subprocess failure, raises CalledProcessError containing the exit code, stdout, and stderr."
  },
  {
    "name": "list_staged_domain_knowledge_relpaths",
    "signature": "list_staged_domain_knowledge_relpaths(work_dir) -> list[str]",
    "pre_condition": "work_dir is a valid path to the FM-Agent workspace directory where domain knowledge files may have been staged.",
    "post_condition": "Returns a list of relative path strings for every domain knowledge file that has been staged under the workspace directory. Returns an empty list when no domain knowledge files are staged."
  }
]
}

```

► Logic verification result（完整）

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/spec_generation_and_verification-py/_run_spec_generation_batch.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Dispatches an OpenCode subprocess instructed to generate a .spec.json and .info.json sidecar for every function listed in batch_info['functions'] by following fm_agent/workflow_spec_step4_batch.md and fm_agent/spec_prompts/system_prompt.md. Records a structured trace event under stage='spec_generation' that identifies the function IDs derived from the function file paths and catalogs all specified input and output files. Returns an integer: 0 when the subprocess exits successfully, or the non-zero returncode when the subprocess exits with an error or crashes with a CalledProcessError. When attempt equals 1, the subprocess is instructed to generate all sidecars from scratch. When attempt is greater than 1, the subprocess is instructed to inspect each function's existing .spec.json and .info.json sidecars, preserve those whose content matches the schema defined in fm_agent/spec_prompts/system_prompt.md, and regenerate only those that are missing, unparseable, or schema-nonconforming.",
    "actual_behavior": "The function returns an integer r. If no exception other than subprocess.CalledProcessError occurs, r is the return code of the subprocess launched to generate spec files. If run_opencode_traced raises CalledProcessError, it is caught and r is that exception's returncode; otherwise r is the CompletedProcess.returncode (0 for success, non-zero for failure). The subprocess is built with build_llm_cli_command(model=OPENCODE_SPEC_MODEL, prompt=p, cwd=proj_dir, files=[prompt_file]) where p is a prompt string that depends on attempt: if attempt == 1, p is an initial generation instruction; otherwise p is a continuation instruction that asks to skip functions only when both .spec.json and .info.json already exist and are valid. The executed command receives context from fm_agent/workflow_spec_step4_batch.md, the batch prompt file (batch_re_dir/batch_file), fm_agent/spec_prompts/system_prompt.md, and any staged domain knowledge files from work_dir. It is expected to produce .spec.json and .info.json files for each function in batch_info['functions']. The function itself does not guarantee these output files exist or are valid. It does not modify any existing project files. If run_opencode_traced executes without raising CalledProcessError, a structured trace event is recorded. Any exception not of type subprocess.CalledProcessError propagates out of the function.",
    "code_evidence": "Line 36:          f\"is missing, malformed, or schema-invalid, rewrite the complete \\\"\\\"\\nLine 37: f\".spec.json and .info.json files for that function. \\\"\\\"\\nLine 38:          f\"Do not modify the function source files. {fm_reminder}\\\"\\\"\\n",
    "trigger_condition": "For attempt > 1, the specification states: \\\"preserve those whose content matches the schema ... and regenerate only those that are missing, unparseable, or schema-nonconforming.\\\" This requires preserving each individually valid sidecar file. The prompt in the code, however, instructs the subprocess to rewrite both .spec.json and .info.json for a function whenever either sidecar is missing/invalid, failing to preserve a still-valid partner file."
  }
}

```

► Bug Validator result（完整）

```

{
  "id": "src--spec_generation_and_verification-py--_run_spec_generation_batch",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/spec_generation_and_verification-py/_run_spec_generation_batch.py",
  "function_name": "_run_spec_generation_batch",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--spec_generation_and_verification-py--_run_spec_generation_batch.py",
  "detail_file": "fm_agent/bug_validation/src--spec_generation_and_verification-py--_run_spec_generation_batch.md",
  "probe_stdout": "CONFIRMED - prompt instructs rewriting BOTH sidecars when only one may be invalid\\n Matched buggy keywords in prompt: ['either sidecar', 'rewrite the complete', '.spec.json and .info.json files']\\n Spec requires: individually preserve valid sidecars, regenerate only invalid ones\\n Actual prompt: instructs rewriting the complete .spec.json and .info.json when either is bad",
  "trigger_summary": "For attempt > 1, the prompt tells the OpenCode subprocess to rewrite BOTH .spec.json and .info.json when either sidecar is invalid, rather than individually preserving each valid sidecar as the spec requires."
}

```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:58:15.427187Z	3968
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:58:48.799532Z	11660

事件 1 — llm_750f6ad118724af58f83bb7e170f14c0

► parsed_json（完整）

```
{
  "post_condition": "The function returns an integer r. If no exception other than subprocess.CalledProcessError occurs, r is the return code of the subprocess launched to generate spec files. If run_opencode_traced raises CalledProcessError, it is caught and r is that exception's returncode; otherwise r is the CompletedProcess.returncode (0 for success, non-zero for failure). The subprocess is built with build_llm_cli_command(model=OPENCODE_SPEC_MODEL, prompt=p, cwd=proj_dir, files=[prompt_file]) where p is a prompt string that depends on attempt: if attempt == 1, p is an initial generation instruction; otherwise p is a continuation instruction that asks to skip functions only when both .spec.json and .info.json already exist and are valid. The executed command receives context from fm_agent/workflow_spec_step4_batch.md, the batch prompt file (batch_rel_dir/batch_file), fm_agent/spec_prompts/system_prompt.md, and any staged domain knowledge files from work_dir. It is expected to produce .spec.json and .info.json files for each function in batch_info['functions']. The function itself does not guarantee these output files exist or are valid. It does not modify any existing project files. If run_opencode_traced executes without raising CalledProcessError, a structured trace event is recorded. Any exception not of type subprocess.CalledProcessError propagates out of the function."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_750f6ad118724af58f83bb7e170f14c0_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_750f6ad118724af58f83bb7e170f14c0_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_750f6ad118724af58f83bb7e170f14c0_response.txt)

事件 2 — llm_471c39a1fc57415fa8a6e9bd5c40e094

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "attempt = 2, batch_info = {\"file\": \"batch1.txt\", \"functions\": [\"path/to/func.py\"]}. The project contains a valid .spec.json for that function (matches the schema) but no .info.json. The code's prompt tells the subprocess: \"rewrite the complete .spec.json and .info.json files for that function.\" This discards the valid .spec.json. The specification requires preserving the valid .spec.json and only regenerating the missing .info.json.\",
  \"offending_statements\": \"Line 36: f\\\"is missing, malformed, or schema-invalid, rewrite the complete \\\"\\nLine 37: f\\\".spec.json and .info.json files for that function. \\\"\\nLine 38: f\\\"Do not modify the function source files. {fm_reminder}\\\"\",
  \"reason\": \"For attempt > 1, the specification states: \\\"preserve those whose content matches the schema ... and regenerate only those that are missing, unparseable, or schema-nonconforming.\\\" This requires preserving each individually valid sidecar file. The prompt in the code, however, instructs the subprocess to rewrite both .spec.json and .info.json for a function whenever either sidecar is missing/invalid, failing to preserve a still-valid partner file.\"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_471c39a1fc57415fa8a6e9bd5c40e094_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_471c39a1fc57415fa8a6e9bd5c40e094_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_471c39a1fc57415fa8a6e9bd5c40e094_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

可能有 Bug（51 项）

Phase 1 — Initialization & Configuration

模块: configuration

FMA-MISMATCH-007 — src--configure_llm-py--_private_backup_root

- 四类结论: 可能有 Bug
- 分类依据: probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 1 — Initialization & Configuration / configuration; 原源码 src/configure_llm.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

- **SPEC claim:** Returns a Path within the platform's temporary-file directory whose final component encodes the calling process's user identity. The returned path is a directory location, not a file. The user-identity component contains only the characters AZ, az, 09, '.', '_', and '-'. The same user identity always produces the same path; distinct user identities produce distinct paths.
- **Actual behavior:** The function returns a `pathlib.Path` object. Let `temp_dir = tempfile.gettempdir()`. The user component `user_component` is determined as: if `hasattr(os, 'getuid')` then `user_component = 'uid-' + str(os.getuid())`; else if `os.environ.get('USERNAME')` is a non-empty string then that value; else if `os.environ.get('USER')` is a non-empty string then that value; else `'user'`. Then `safe = re.sub(r'^A-Za-z0-9._-]+', '_', user_component).strip('._-')` or `'user'`. The returned value is `Path(temp_dir) / f'fm-agent-config-backups-{safe}'`. No filesystem changes occur. Formal: `$result = Path(tempfile.gettempdir()) / 'fm-agent-config-backups-' + safe`, where `safe` is derived from the user identity as described, and `safe` is a non-empty string matching `[A-Za-z0-9._-]+` with no leading/trailing `._-` (or `'user'` as fallback).
- **Code evidence:** Line 5: `user_component = os.environ.get("USERNAME") or os.environ.get("USER") or "user"` Line 6: `safe = re.sub(r"^[A-Za-z0-9.-]+", "", user_component).strip("-._") or "user"`
- **Trigger condition:** The mapping from `user_component` to `safe` is not injective; different inputs (e.g., containing backslash vs underscore) can yield the same `safe` string, causing two distinct user identities to produce identical paths.
- **Validator trigger:** The regex `re.sub(r"^[A-Za-z0-9.-]+", "", ...)` collapses sequences of disallowed chars into a single underscore, making distinct usernames (e.g. `DOMAIN\user123` vs `DOMAIN/user123`) map to the same `safe` component.
- **Probe stdout:** CONFIRMED — `_private_backup_root` produces identical paths for distinct usernames because the sanitisation regex is non-injective. username `r'DOMAIN\user123' -> safe: 'DOMAIN_user123'` username `'DOMAIN/user123' -> safe: 'DOMAIN_user123'` username `'DOMAIN_user123' -> safe: 'DOMAIN_user123'` Result path: `/tmp/fm-agent-config-backups-DOMAIN_user123` Specification requires distinct user identities to produce distinct paths.

► SPEC sidecar (完整)

```
{
  "signature": "_private_backup_root() -> Path",
  "pre_condition": "The calling process has an OS-level user identity (numeric UID on POSIX platforms; the USERNAME or USER environment variable on other platforms).",
  "post_condition": "Returns a Path within the platform's temporary-file directory whose final component encodes the calling process's user identity. The returned path is a directory location, not a file. The user-identity component contains only the characters A-Z, a-z, 0-9, '.', '_', and '-'. The same user identity always produces the same path; distinct user identities produce distinct paths."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/_private_backup_root.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a Path within the platform's temporary-file directory whose final component encodes the calling process's user identity. The returned path is a directory location, not a file. The user-identity component contains only the characters AZ, az, 09, '.', '_', and '-'. The same user identity always produces the same path; distinct user identities produce distinct paths.",
    "actual_behavior": "The function returns a `pathlib.Path` object. Let `temp_dir = tempfile.gettempdir()`. The user component `user_component` is determined as: if `hasattr(os, 'getuid')` then `user_component = 'uid-' + str(os.getuid())`; else if `os.environ.get('USERNAME')` is a non-empty string then that value; else if `os.environ.get('USER')` is a non-empty string then that value; else `user`. Then `safe = re.sub(r'^A-Za-z0-9._-]+', '_', user_component).strip('._-') or 'user'`. The returned value is `Path(temp_dir) / f'fm-agent-config-backups-{safe}'`. No filesystem changes occur. Formal: $result = Path(tempfile.gettempdir()) / 'fm-agent-config-backups-' + safe, where safe is derived from the user identity as described, and safe is a non-empty string matching [A-Za-z0-9._-]+ with no leading/trailing ._- (or 'user' as fallback).",
    "code_evidence": "Line 5: user_component = os.environ.get(\"USERNAME\") or os.environ.get(\"USER\") or \"user\"\nLine 6: safe = re.sub(r\"^[A-Za-z0-9.-]+\", \"\", user_component).strip(\".-_\") or \"user\"",
    "trigger_condition": "The mapping from user_component to safe is not injective; different inputs (e.g., containing backslash vs underscore) can yield the same safe string, causing two distinct user identities to produce identical paths."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--configure_llm-py--_private_backup_root",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/_private_backup_root.py",
  "function_name": "_private_backup_root",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--_private_backup_root.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--_private_backup_root.md",
  "probe_stdout": "CONFIRMED - _private_backup_root produces identical paths for distinct usernames because the sanitisation regex
is non-injective.\n username r'DOMAIN\\\\user123' -> safe: 'DOMAIN_user123'\n username 'DOMAIN/user123' -> safe:
'DOMAIN_user123'\n username 'DOMAIN_user123' -> safe: 'DOMAIN_user123'\n Result path: /tmp/fm-agent-config-backups-
DOMAIN_user123\n Specification requires distinct user identities to produce distinct paths.\n",
  "trigger_summary": "The regex re.sub(r'[^A-Za-z0-9._-]+', '_', ...) collapses sequences of disallowed chars into a single
underscore, making distinct usernames (e.g. DOMAIN\\user123 vs DOMAIN/user123) map to the same safe component."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:15:18.194814Z	4166
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:16:15.655508Z	4257

事件 1 — [llm_94e33904683a4082929eb6cdb8aeabc1](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a `pathlib.Path` object. Let `temp_dir = tempfile.gettempdir()`. The user component
`user_component` is determined as: if `hasattr(os, 'getuid')` then `user_component = 'uid-' + str(os.getuid())`; else if
`os.environ.get('USERNAME')` is a non-empty string then that value; else if `os.environ.get('USER')` is a non-empty string then
that value; else `user`. Then `safe = re.sub(r'[^A-Za-z0-9._-]+', '_', user_component).strip('.')` or `user`. The returned
value is `Path(temp_dir) / f'fm-agent-config-backups-{safe}'`. No filesystem changes occur. Formal: $result =
Path(tempfile.gettempdir()) / 'fm-agent-config-backups-' + safe, where safe is derived from the user identity as described, and
safe is a non-empty string matching `[A-Za-z0-9._-]+` with no leading/trailing `.` (or `user` as fallback)."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_94e33904683a4082929eb6cdb8aeabc1_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_94e33904683a4082929eb6cdb8aeabc1_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_94e33904683a4082929eb6cdb8aeabc1_response.txt)

事件 2 — [llm_933d6cff483e4106bb999b3c4c24219a](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "On a Windows system (without os.getuid), two different user identities: one is a domain user with
USERNAME='DOMAIN\\john', the other a local user with USERNAME='DOMAIN_john'. The code produces safe='DOMAIN_john' for both,
resulting in the same path '.../fm-agent-config-backups-DOMAIN_john', violating the requirement that distinct user identities
produce distinct paths.",
  "offending_statements": "Line 5: user_component = os.environ.get(\"USERNAME\") or os.environ.get(\"USER\") or \"user\"\\nLine 6:
safe = re.sub(r\"[^A-Za-z0-9._-]+\", \"_\", user_component).strip(\".\") or \"user\"\\n",
  "reason": "The mapping from user_component to safe is not injective; different inputs (e.g., containing backslash vs underscore)
can yield the same safe string, causing two distinct user identities to produce identical paths."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_933d6cff483e4106bb999b3c4c24219a_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_933d6cff483e4106bb999b3c4c24219a_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_933d6cff483e4106bb999b3c4c24219a_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-030 — [src--configure_llm-py--update_llm_settings_toml_text](#)

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。

- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 1 — Initialization & Configuration / [configuration](#); 原源码 [src/configure_llm.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Raises ConfigWizardError when updates is empty. Raises ConfigWizardError when any key-value pair in updates fails validation (the key is not a recognized LLM configuration field name, or the value does not conform to the expected format for that field). Raises ConfigWizardError when text is not parseable as valid TOML by the standard library parser, or when text decodes to a non-table TOML value. Otherwise returns a string containing valid TOML in which: for every key in updates, the corresponding field under the [llm] section is set to updates[key]; every other TOML section, every field outside the updated keys, every comment, and all whitespace outside the updated field values is preserved unchanged from text. When text does not contain an [llm] section, a new [llm] section is appended containing all fields from updates.
- **Actual behavior:** The function update_llm_settings_toml_text either returns a modified TOML string that integrates all desired LLM settings from updates into the [llm] section of the input text, or raises an error if preconditions are violated. More precisely:
- If updates is falsy (e.g., empty dict), it raises ConfigWizardError('Provide at least one LLM setting to update.').
- If any (key, value) pair in updates fails validate_llm_setting(key, value), the error raised by that function propagates (this error indicates that key is unrecognized or value does not meet the field's type/format constraints).
- If text cannot be parsed as TOML for any reason (i.e., tomlib.loads(text) raises a tomlib.TOMLDecodeError), it raises ConfigWizardError('Existing fm-agent.toml is invalid TOML; refusing to overwrite it.').
- If text decodes successfully but the result is a truthy value that is not a dict (e.g., a string, integer, array), it raises ConfigWizardError('Existing fm-agent.toml must decode to a table/object.').

Otherwise (all checks pass), the function returns a new string r that is a valid TOML document, satisfying:

1. r contains a [llm] section (top-level table).
2. In that [llm] section, for every key k in updates, there is exactly one keyvalue line of the form {k:<9} = {_quote_toml_string(updates[k])} (where _quote_toml_string produces a TOML-valid quoted string). Any preexisting value for such a key in the original [llm] section is replaced; if a key was absent, it is added at the end of the section.
3. All other content of text (other sections, comments, formatting, blank lines outside [llm]) is preserved in r, except that if the original text was empty, r consists solely of [llm]\n followed by the required keyvalue lines (each terminated by a newline).
4. When the original text contained an [llm] section, any existing LLM keys not mentioned in updates remain unchanged in r.

Symbolically, given precondition P(text, updates): (updates is empty) function raises ConfigWizardError return value undefined (k,v updates such that validate_llm_setting(k,v) fails) function raises an error return undefined (tomlib.loads(text) fails OR returns a truthy nondict) function raises ConfigWizardError return undefined otherwise function returns r, where r is a string meeting properties 14 above.

- **Code evidence:** Line 37: kv_match = _KV_RE.match(line) Line 38: if kv_match and kv_match.group(2) in target:
- **Trigger condition:** The code uses a regex (_KV_RE) to identify key-value lines. When a key is quoted (e.g., "api_key"), the regex may not match, causing the original line to be treated as a non-key line. The key is not recognized, so the original line is preserved unchanged and the new key-value pair is appended later (lines 44-47). The output contains duplicate keys, which is invalid TOML and does not replace the field as required by the specification.
- **Validator trigger:** When the input TOML uses quoted keys (e.g. "name"), the _KV_RE regex fails to match, causing the old key-value line to be preserved and a new bare-key line to be appended, producing duplicate keys.
- **Probe stdout:** CONFIRMED — duplicate "name" keys found in output (2 occurrences) Actual output: '[llm]\n"name" = "old-model"\nprovider = "old-provider"\nname = "new-model"\n' Expected: single "name" key with value "new-model"

► SPEC sidecar (完整)

```
{
  "signature": "update_llm_settings_toml_text(text: str, updates: dict[str, str]) -> str",
  "pre_condition": "text is a string; updates is a dict mapping dot-separated LLM section field path strings to their desired string values.",
  "post_condition": "Raises ConfigWizardError when updates is empty. Raises ConfigWizardError when any key-value pair in updates fails validation (the key is not a recognized LLM configuration field name, or the value does not conform to the expected format for that field). Raises ConfigWizardError when text is not parseable as valid TOML by the standard library parser, or when text decodes to a non-table TOML value. Otherwise returns a string containing valid TOML in which: for every key in updates, the corresponding field under the [llm] section is set to updates[key]; every other TOML section, every field outside the updated keys, every comment, and all whitespace outside the updated field values is preserved unchanged from text. When text does not
```

```
contain an [llm] section, a new [llm] section is appended containing all fields from updates."
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "validate_llm_setting",
      "signature": "validate_llm_setting(key: str, value: str) -> None",
      "pre_condition": "key is a string identifying an LLM configuration field name; value is a string representing the desired new value for that field.",
      "post_condition": "Completes without error when key is a recognized LLM configuration field name and value conforms to the expected type and format for that field. Raises an error when key is not a recognized field name or value does not satisfy the field's type and format constraints."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/update_llm_settings_toml_text.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Raises ConfigWizardError when updates is empty. Raises ConfigWizardError when any key-value pair in updates fails validation (the key is not a recognized LLM configuration field name, or the value does not conform to the expected format for that field). Raises ConfigWizardError when text is not parseable as valid TOML by the standard library parser, or when text decodes to a non-table TOML value. Otherwise returns a string containing valid TOML in which: for every key in updates, the corresponding field under the [llm] section is set to updates[key]; every other TOML section, every field outside the updated keys, every comment, and all whitespace outside the updated field values is preserved unchanged from text. When text does not contain an [llm] section, a new [llm] section is appended containing all fields from updates.",
    "actual_behavior": "The function update_llm_settings_toml_text either returns a modified TOML string that integrates all desired LLM settings from `updates` into the `[llm]` section of the input `text`, or raises an error if preconditions are violated. More precisely:\n\n- If `updates` is falsy (e.g., empty dict), it raises ConfigWizardError('Provide at least one LLM setting to update.').\n- If any (key, value) pair in `updates` fails `validate_llm_setting(key, value)`, the error raised by that function propagates (this error indicates that `key` is unrecognized or `value` does not meet the field's type/format constraints).\n- If `text` cannot be parsed as TOML for any reason (i.e., `tomllib.loads(text)` raises a `tomllib.TOMLDecodeError`), it raises ConfigWizardError('Existing fm-agent.toml is invalid TOML; refusing to overwrite it.').\n- If `text` decodes successfully but the result is a truthy value that is not a dict (e.g., a string, integer, array), it raises ConfigWizardError('Existing fm-agent.toml must decode to a table/object.').\n\nOtherwise (all checks pass), the function returns a new string `r` that is a valid TOML document, satisfying:\n1. `r` contains a `[llm]` section (top-level table).\n2. In that `[llm]` section, for every key `k` in `updates`, there is exactly one keyvalue line of the form `{k:<9} = {_quote_toml_string(updates[k])}` (where `_quote_toml_string` produces a TOML-valid quoted string). Any preexisting value for such a key in the original `[llm]` section is replaced; if a key was absent, it is added at the end of the section.\n3. All other content of `text` (other sections, comments, formatting, blank lines outside `[llm]`) is preserved in `r`, except that if the original `text` was empty, `r` consists solely of `[llm]\n` followed by the required keyvalue lines (each terminated by a newline).\n4. When the original `text` contained an `[llm]` section, any existing LLM keys not mentioned in `updates` remain unchanged in `r`.\n\nSymbolically, given precondition P(text, updates):\n (updates is empty)  function raises ConfigWizardError\n return value undefined\n ( k, v updates such that validate_llm_setting(k,v) fails)  function raises an error  return undefined\n (tomllib.loads(text) fails OR returns a truthy nondict)  function raises ConfigWizardError  return undefined\n otherwise  function returns r, where r is a string meeting properties 14 above.",
    "code_evidence": "Line 37: kv_match = _KV_RE.match(line)\nLine 38: if kv_match and kv_match.group(2) in target:",
    "trigger_condition": "The code uses a regex (_KV_RE) to identify key-value lines. When a key is quoted (e.g., `\"api_key\"`), the regex may not match, causing the original line to be treated as a non-key line. The key is not recognized, so the original line is preserved unchanged and the new key-value pair is appended later (lines 44-47). The output contains duplicate keys, which is invalid TOML and does not replace the field as required by the specification."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--configure_llm-py--update_llm_settings_toml_text",
  "source_file": "src/configure_llm.py",
  "function_name": "update_llm_settings_toml_text",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--update_llm_settings_toml_text.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--update_llm_settings_toml_text.md",
  "probe_stdout": "CONFIRMED - duplicate \"name\" keys found in output (2 occurrences)\n Actual output: '[llm]\n\n\"name\" = \"old-model\"\n\nprovider = \"old-provider\"\n\nname = \"new-model\"\n\n' Expected: single \"name\" key with value \"new-model\"",
  "trigger_summary": "When the input TOML uses quoted keys (e.g. `\"name\"`), the _KV_RE regex fails to match, causing the old key-value line to be preserved and a new bare-key line to be appended, producing duplicate keys."
}
```

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:09:27.922585Z	4158
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:10:14.358311Z	5243

事件 1 — [llm_0069fc6457f94443a0677b8f44de88e3](#)

► [parsed_json](#)（完整）

```
{
  "post_condition": "The function update_llm_settings_toml_text either returns a modified TOML string that integrates all desired LLM settings from `updates` into the `[llm]` section of the input `text`, or raises an error if preconditions are violated. More precisely:\n\n- If `updates` is falsy (e.g., empty dict), it raises ConfigWizardError('Provide at least one LLM setting to update.').\n- If any (key, value) pair in `updates` fails `validate_llm_setting(key, value)`, the error raised by that function propagates (this error indicates that `key` is unrecognized or `value` does not meet the field's type/format constraints).\n- If `text` cannot be parsed as TOML for any reason (i.e., `tomllib.loads(text)` raises a `tomllib.TOMLDecodeError`), it raises ConfigWizardError('Existing fm-agent.toml is invalid TOML; refusing to overwrite it.').\n- If `text` decodes successfully but the result is a truthy value that is not a dict (e.g., a string, integer, array), it raises ConfigWizardError('Existing fm-agent.toml must decode to a table/object.').\n\nOtherwise (all checks pass), the function returns a new string `r` that is a valid TOML document, satisfying:\n1. `r` contains a `[llm]` section (top-level table).\n2. In that `[llm]` section, for every key `k` in `updates`, there is exactly one key-value line of the form `{k:<9} = {_quote_toml_string(updates[k])}` (where `_quote_toml_string` produces a TOML-valid quoted string). Any pre-existing value for such a key in the original `[llm]` section is replaced; if a key was absent, it is added at the end of the section.\n3. All other content of `text` (other sections, comments, formatting, blank lines outside `[llm]`) is preserved in `r`, except that if the original `text` was empty, `r` consists solely of `[llm]\n` followed by the required key-value lines (each terminated by a newline).\n4. When the original `text` contained an `[llm]` section, any existing LLM keys not mentioned in `updates` remain unchanged in `r`.\n\nSymbolically, given pre-condition P(text, updates):\n (updates is empty) => function raises ConfigWizardError ^ return value undefined\n (exists k,v in updates such that validate_llm_setting(k,v) fails) => function raises an error ^ return undefined\n (tomllib.loads(text) fails OR returns a truthy non-dict) => function raises ConfigWizardError ^ return undefined\n otherwise => function returns r, where r is a string meeting properties 1-4 above."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_0069fc6457f94443a0677b8f44de88e3_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_0069fc6457f94443a0677b8f44de88e3_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_0069fc6457f94443a0677b8f44de88e3_response.txt)

事件 2 — [llm_52b8a00569bf4cc8a311e54afd8e485c](#)

► [parsed_json](#)（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "text = '[llm]\n\n\"api_key\" = \"old_value\"\n\n', updates = {'api_key': 'new_value'}",
  "offending_statements": "Line 37: kv_match = _KV_RE.match(line)\nLine 38: if kv_match and kv_match.group(2) in target:",
  "reason": "The code uses a regex (_KV_RE) to identify key-value lines. When a key is quoted (e.g., \"api_key\"), the regex may not match, causing the original line to be treated as a non-key line. The key is not recognized, so the original line is preserved unchanged and the new key-value pair is appended later (lines 44-47). The output contains duplicate keys, which is invalid TOML and does not replace the field as required by the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_52b8a00569bf4cc8a311e54afd8e485c_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_52b8a00569bf4cc8a311e54afd8e485c_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_52b8a00569bf4cc8a311e54afd8e485c_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: environment

FMA-MISMATCH-038 — [src--env_check.py--_check_oh_my_openagent](#)

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 1 — Initialization & Configuration / environment; 原源码 src/env_check.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

- **SPEC claim:** Returns (True, None) when oh-my-openagent is reachable via bunx and responds to its version flag within 10 seconds, indicating the tool is installed and executable. Returns (False, "oh-my-openagent is not installed (bunx unavailable or timed out)") when oh-my-openagent cannot be invoked via bunx for any reason, including bunx itself being unavailable, the tool not being installed, or the invocation exceeding 10 seconds. The function produces no side effects it does not create, modify, or delete any files.
- **Actual behavior:** After the function call, one of the following holds:
 1. If the import of `subprocess` failed, an `ImportError` (or subclass) was raised and the function did not return. No tuple was produced.
 2. Otherwise, the function returned a 2-tuple (`result`, `message`) where:
 - If the subprocess run completed without raising any exception, then `result` is `True` and `message` is `None`.
 - If any exception occurred during `subprocess.run(...)` (including `CalledProcessError`, `FileNotFoundError`, `TimeoutExpired`, etc.), then `result` is `False` and `message` equals the string "oh-my-openagent is not installed (bunx unavailable or timed out)".

Formally, let **S** be the state after the call, **R** the return value, and **E** any uncaught exception.

- (**E** = none) (import subprocess succeeded)
- (**E** = none) (**R** is a 2-tuple) (**R**[0] {True, False})
- (**E** = none **R**[0] = True) (**R**[1] is None) (subprocess.run completed without exception)
- (**E** = none **R**[0] = False) (**R**[1] = "oh-my-openagent is not installed (bunx unavailable or timed out)") (subprocess.run raised an exception)
- If import subprocess raised an exception, then **E** is that exception and no return value exists.
- **Code evidence:** Line 4: subprocess.run(...); Line 8: return True, None
- **Trigger condition:** The code does not check the subprocess return code or output. A non-zero exit status (e.g., when oh-my-openagent is missing) causes the function to return True, violating the specification which requires False for any failure to invoke the tool.
- **Validator trigger:** subprocess.run() without check=True does not raise on non-zero exit codes; when bunx exits non-zero, the function returns (True, None) instead of (False, error_message).
- **Probe stdout:** CONFIRMED — Bug reproduced: function returned (True, None) even though subprocess exited with code 1. Spec requires (False, 'oh-my-openagent is not installed (bunx unavailable or timed out)') when the tool cannot be invoked.

► SPEC sidecar (完整)

```
{
  "signature": "_check_oh_my_openagent() -> tuple[bool, str | None]",
  "pre_condition": "None required – the function accepts no arguments and has no external state dependencies.",
  "post_condition": "Returns (True, None) when oh-my-openagent is reachable via bunx and responds to its version flag within 10 seconds, indicating the tool is installed and executable. Returns (False, \"oh-my-openagent is not installed (bunx unavailable or timed out)\") when oh-my-openagent cannot be invoked via bunx for any reason, including bunx itself being unavailable, the tool not being installed, or the invocation exceeding 10 seconds. The function produces no side effects – it does not create, modify, or delete any files."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/env_check-py/_check_oh_my_openagent.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns (True, None) when oh-my-openagent is reachable via bunx and responds to its version flag within 10 seconds, indicating the tool is installed and executable. Returns (False, \"oh-my-openagent is not installed (bunx unavailable or timed out)\") when oh-my-openagent cannot be invoked via bunx for any reason, including bunx itself being unavailable, the tool not being installed, or the invocation exceeding 10 seconds. The function produces no side effects it does not create, modify, or delete any files.",
    "actual_behavior": "After the function call, one of the following holds:\n1. If the import of `subprocess` failed, an `ImportError` (or subclass) was raised and the function did not return. No tuple was produced.\n2. Otherwise, the function returned a 2-tuple `(result, message)` where:\n   - If the subprocess run completed without raising any exception, then `result` is `True` and `message` is `None`.\n   - If any exception occurred during `subprocess.run(...)` (including `CalledProcessError`, `FileNotFoundError`, `TimeoutExpired`, etc.), then `result` is `False` and `message` equals the string `\"oh-my-openagent is not installed (bunx unavailable or timed out)\"`.\n\nFormally, let `S` be the state after the call, `R` the return value, and `E` any uncaught exception.\n- `(E = none) (import subprocess succeeded)`\n- `(E = none) (R is a 2-tuple) (R[0] {True, False})`"
  }
}
```



```
= None R[0] = True) (R[1] is None) (subprocess.run completed without exception)`\n- `(E = None R[0] = False) (R[1] = \"oh-my-openagent is not installed (bunx unavailable or timed out)\") (subprocess.run raised an exception)`\n- If `import subprocess` raised an exception, then `E` is that exception and no return value exists.",
    "code_evidence": "Line 4: subprocess.run(...); Line 8: return True, None",
    "trigger_condition": "The code does not check the subprocess return code or output. A non-zero exit status (e.g., when oh-my-openagent is missing) causes the function to return True, violating the specification which requires False for any failure to invoke the tool."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--env_check-py--_check_oh_my_openagent",
  "source_file": "src/env_check.py",
  "function_name": "_check_oh_my_openagent",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--env_check-py--_check_oh_my_openagent.py",
  "detail_file": "fm_agent/bug_validation/src--env_check-py--_check_oh_my_openagent.md",
  "probe_stdout": "CONFIRMED - Bug reproduced: function returned (True, None) even though subprocess exited with code 1. Spec requires (False, 'oh-my-openagent is not installed (bunx unavailable or timed out)') when the tool cannot be invoked.\n",
  "trigger_summary": "subprocess.run() without check=True does not raise on non-zero exit codes; when bunx exits non-zero, the function returns (True, None) instead of (False, error_message).",
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:37:00.390795Z	1347
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:37:15.579456Z	3518

事件 1 — [llm_a61c737b8b544a3fb490f9e744b6c302](#)

► parsed_json（完整）

```
{
  "post_condition": "After the function call, one of the following holds:\n1. If the import of `subprocess` failed, an `ImportError` (or subclass) was raised and the function did not return. No tuple was produced.\n2. Otherwise, the function returned a 2-tuple `(result, message)` where:\n   - If the subprocess run completed without raising any exception, then `result` is `True` and `message` is `None`.\n   - If any exception occurred during `subprocess.run(...)` (including `CalledProcessError`, `FileNotFoundError`, `TimeoutExpired`, etc.), then `result` is `False` and `message` equals the string `\"oh-my-openagent is not installed (bunx unavailable or timed out)\"`.\n\nFormally, let `S` be the state after the call, `R` the return value, and `E` any uncaught exception.\n- `(E = None) → (import subprocess succeeded)`\n- `(E = None) → (R is a 2-tuple) ∧ (R[0] ∈ {True, False})`\n- `(E = None ∧ R[0] = True) → (R[1] is None) ∧ (subprocess.run completed without exception)`\n- `(E = None ∧ R[0] = False) → (R[1] = \"oh-my-openagent is not installed (bunx unavailable or timed out)\") ∧ (subprocess.run raised an exception)`\n- If `import subprocess` raised an exception, then `E` is that exception and no return value exists."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_a61c737b8b544a3fb490f9e744b6c302_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_a61c737b8b544a3fb490f9e744b6c302_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_a61c737b8b544a3fb490f9e744b6c302_response.txt)

事件 2 — [llm_ab7199e380844587bfbd9b2478895214](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "The system has bunx installed, but oh-my-openagent is not installed. Running 'bunx oh-my-openagent --version' exits with code 1 and an error message without raising a Python exception. The function returns (True, None) instead of (False, \"oh-my-openagent is not installed (bunx unavailable or timed out)\").",
  "offending_statements": "Line 4: subprocess.run(...); Line 8: return True, None",
  "reason": "The code does not check the subprocess return code or output. A non-zero exit status (e.g., when oh-my-openagent is missing) causes the function to return True, violating the specification which requires False for any failure to invoke the tool."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ab7199e380844587bfbd9b2478895214_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ab7199e380844587bfbd9b2478895214_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ab7199e380844587bfbd9b2478895214_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修实现。

FMA-MISMATCH-040 — [src--env_check-py--_save_ignored](#)

- 四类结论: 可能有 Bug
- 分类依据: probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险, 列为实现缺陷候选。
- **Validator:** [confirmed](#); attempts 1
- **Phase / 模块:** Phase 1 — Initialization & Configuration / [environment](#); 原源码 [src/env_check.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Persists every check_id in ignored to the ignored-checks file under work_dir, one check_id per line. Creates the file if it does not exist; truncates and overwrites any previous content if it does. When ignored is an empty set, the file is created or truncated to empty. The function does not signal errors to the caller for any failure condition.
- **Actual behavior:** After successful execution, the function creates or overwrites the file at path = `_memory_path(work_dir)`. The file is closed, and its contents are the lines formed by each element of the input set `ignored` sorted in ascending order, each followed by a newline character, with no additional characters. Formally: let path = `_memory_path(work_dir)`; then `file_exists(path)` line (`readlines(path)`) line = x + "\n" for some x sorted(`ignored`) `length(readlines(path)) = |ignored|` closed(`file_descriptor`). If `ignored` is empty, the file is empty. The set `ignored` and the directory `work_dir` remain unchanged.
- **Code evidence:** Line 3: with `open(path, "w")` as f:
- **Trigger condition:** The code lacks exception handling; any I/O failure (e.g., permission denied) will raise an exception that propagates to the caller, signaling an error, while the specification requires that no errors are signaled to the caller for any failure condition.
- **Validator trigger:** Pass a non-existent work_dir path; `_save_ignored` propagates `FileNotFoundError` from `open()` instead of silently handling the failure.
- **Probe stdout:** CONFIRMED — `_save_ignored` propagated `FileNotFoundError`: [Errno 2] No such file or directory: '/tmp/tmpyp3o7war/nonexistent/.env_check_memory'

► SPEC sidecar (完整)

```
{
  "signature": "_save_ignored(work_dir, ignored) -> None",
  "pre_condition": "work_dir is a path to an existing directory. ignored is a set of check_id strings (possibly empty).",
  "post_condition": "Persists every check_id in ignored to the ignored-checks file under work_dir, one check_id per line. Creates the file if it does not exist; truncates and overwrites any previous content if it does. When ignored is an empty set, the file is created or truncated to empty. The function does not signal errors to the caller for any failure condition."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_memory_path",
      "signature": "_memory_path(work_dir) -> str",
      "pre_condition": "work_dir is a path to an existing directory.",
      "post_condition": "Returns the path identifying the ignored-checks memory file under work_dir. The returned path resides under the directory work_dir."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/env_check-py/_save_ignored.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Persists every check_id in ignored to the ignored-checks file under work_dir, one check_id per line. Creates the file if it does not exist; truncates and overwrites any previous content if it does. When ignored is an empty set, the file is created or truncated to empty. The function does not signal errors to the caller for any failure condition.",
    "actual_behavior": "After successful execution, the function creates or overwrites the file at path = _memory_path(work_dir)."
  }
}
```

```
The file is closed, and its contents are the lines formed by each element of the input set `ignored` sorted in ascending order,
each followed by a newline character, with no additional characters. Formally: let path = _memory_path(work_dir); then
file_exists(path)  line  (readlines(path))  line = x + `\\n\\n` for some x  sorted(ignored)  length(readlines(path)) = |ignored|
closed(file_descriptor). If `ignored` is empty, the file is empty. The set `ignored` and the directory `work_dir` remain
unchanged.",
  "code_evidence": "Line 3: with open(path, `w`) as f:",
  "trigger_condition": "The code lacks exception handling; any I/O failure (e.g., permission denied) will raise an exception
that propagates to the caller, signaling an error, while the specification requires that no errors are signaled to the caller for
any failure condition."
}
```

► Bug Validator result（完整）

```
{
  "id": "src--env_check-py--_save_ignored",
  "source_file": "src/env_check-py/_save_ignored.py",
  "function_name": "_save_ignored",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--env_check-py--_save_ignored.py",
  "detail_file": "fm_agent/bug_validation/src--env_check-py--_save_ignored.md",
  "probe_stdout": "CONFIRMED - _save_ignored propagated FileNotFoundError: [Errno 2] No such file or directory:
'/tmp/tmpyp3o7war/nonexistent/.env_check_memory'\n",
  "trigger_summary": "Pass a non-existent work_dir path; _save_ignored propagates FileNotFoundError from open() instead of
silently handling the failure."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:44:38.737493Z	1567
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:44:59.094816Z	2272

事件 1 — [llm_69bccfe176a544789c13888dea104f45](#)

► parsed_json（完整）

```
{
  "post_condition": "After successful execution, the function creates or overwrites the file at path = _memory_path(work_dir). The
file is closed, and its contents are the lines formed by each element of the input set `ignored` sorted in ascending order, each
followed by a newline character, with no additional characters. Formally: let path = _memory_path(work_dir); then
file_exists(path) ^ V line E (readlines(path)) -> line = x + `\\n\\n` for some x E sorted(ignored) ^ length(readlines(path)) =
|ignored| ^ closed(file_descriptor). If `ignored` is empty, the file is empty. The set `ignored` and the directory `work_dir`
remain unchanged."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_69bccfe176a544789c13888dea104f45_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_69bccfe176a544789c13888dea104f45_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_69bccfe176a544789c13888dea104f45_response.txt)

事件 2 — [llm_49046a10706a49da9d598997bae97991](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "work_dir='/readonly_existing_dir', ignored=set()",
  "offending_statements": "Line 3: with open(path, `w`) as f:",
  "reason": "The code lacks exception handling; any I/O failure (e.g., permission denied) will raise an exception that propagates
to the caller, signaling an error, while the specification requires that no errors are signaled to the caller for any failure
condition."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_49046a10706a49da9d598997bae97991_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_49046a10706a49da9d598997bae97991_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_49046a10706a49da9d598997bae97991_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块：git_utils

FMA-MISMATCH-042 — src--git-py--_get_head_commit

- 四类结论： 可能有 Bug
- 分类依据： 沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 1 — Initialization & Configuration / git_utils; 原源码 src/git.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: If proj_dir is a valid git repository with a HEAD commit reachable via rev-parse, returns the full hexadecimal SHA identifier of that commit as a non-empty string. If proj_dir is not a git repository or the git command fails for any reason (including a missing or corrupt HEAD), returns None.
- **Actual behavior**: After execution, exactly one of the following holds: (1) the subprocess.run(...) call succeeds without raising CalledProcessError, returns a CompletedProcess with some stdout string s, and the function returns s.strip(); no side effect occurs. (2) the call raises subprocess.CalledProcessError, a logging.info message is emitted containing proj_dir, and the function returns None. (3) the call raises any other exception (e.g., OSError), that exception is propagated and the function does not return normally. proj_dir is not modified in any case.
- **Code evidence**: Line 8: except subprocess.CalledProcessError:
- **Trigger condition**: If git is not found, subprocess.run raises FileNotFoundError (or OSError), which is not caught. The exception propagates instead of returning None as required by the specification for any git command failure.
- **Validator trigger**: When git binary is not found on the system, subprocess.run raises FileNotFoundError (subclass of OSError) which is not caught by the except subprocess.CalledProcessError handler, causing the exception to propagate instead of returning None.
- **Probe stdout**: CONFIRMED — FileNotFoundError raised instead of returning None (spec: 'returns None' for any git command failure)

► SPEC sidecar（完整）

```
{
  "signature": "_get_head_commit(proj_dir) -> str | None",
  "pre_condition": "proj_dir is a filesystem path string",
  "post_condition": "If proj_dir is a valid git repository with a HEAD commit reachable via rev-parse, returns the full hexadecimal SHA identifier of that commit as a non-empty string. If proj_dir is not a git repository or the git command fails for any reason (including a missing or corrupt HEAD), returns None."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/git-py/_get_head_commit.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "If proj_dir is a valid git repository with a HEAD commit reachable via rev-parse, returns the full hexadecimal SHA identifier of that commit as a non-empty string. If proj_dir is not a git repository or the git command fails for any reason (including a missing or corrupt HEAD), returns None.",
    "actual_behavior": "After execution, exactly one of the following holds: (1) the subprocess.run(...) call succeeds without raising CalledProcessError, returns a CompletedProcess with some stdout string s, and the function returns s.strip(); no side effect occurs. (2) the call raises subprocess.CalledProcessError, a logging.info message is emitted containing proj_dir, and the function returns None. (3) the call raises any other exception (e.g., OSError), that exception is propagated and the function does not return normally. proj_dir is not modified in any case.",
    "code_evidence": "Line 8: except subprocess.CalledProcessError:",
    "trigger_condition": "If git is not found, subprocess.run raises FileNotFoundError (or OSError), which is not caught. The exception propagates instead of returning None as required by the specification for any git command failure."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--git-py--_get_head_commit",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/git-py/_get_head_commit.py",
  "function_name": "_get_head_commit",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--git-py--_get_head_commit.py",
  "detail_file": "fm_agent/bug_validation/src--git-py--_get_head_commit.md",
  "probe_stdout": "CONFIRMED - FileNotFoundError raised instead of returning None (spec: 'returns None' for any git command failure)\n",
  "trigger_summary": "When git binary is not found on the system, subprocess.run raises FileNotFoundError (subclass of OSError) which is not caught by the except subprocess.CalledProcessError handler, causing the exception to propagate instead of returning None."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:36:20.352722Z	1480
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:36:39.322826Z	2180

事件 1 — [llm_7aab79c54ece49e5a5b0646e4aaf699b](#)

► parsed_json（完整）

```
{
  "post_condition": "After execution, exactly one of the following holds: (1) the subprocess.run(...) call succeeds without raising CalledProcessError, returns a CompletedProcess with some stdout string s, and the function returns s.strip(); no side effect occurs. (2) the call raises subprocess.CalledProcessError, a logging.info message is emitted containing proj_dir, and the function returns None. (3) the call raises any other exception (e.g., OSError), that exception is propagated and the function does not return normally. proj_dir is not modified in any case."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7aab79c54ece49e5a5b0646e4aaf699b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_7aab79c54ece49e5a5b0646e4aaf699b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_7aab79c54ece49e5a5b0646e4aaf699b_response.txt)

事件 2 — [llm_60d798b450274f82853e840f129466ee](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/tmp/some_dir', with git not installed or not on PATH.",
  "offending_statements": "Line 8: except subprocess.CalledProcessError:",
  "reason": "If git is not found, subprocess.run raises FileNotFoundError (or OSError), which is not caught. The exception propagates instead of returning None as required by the specification for any git command failure."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_60d798b450274f82853e840f129466ee_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_60d798b450274f82853e840f129466ee_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_60d798b450274f82853e840f129466ee_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-043 — [src--git-py--_is_git_repo](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论；沿用旧清单的逐条审计结论。
- Validator：confirmed；attempts 1
- Phase / 模块：Phase 1 — Initialization & Configuration / [git_utils](#)；原源码 [src/git.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns True if and only if proj_dir is an existing git repository whose HEAD reference resolves to at least one commit.
Returns False if proj_dir is not a git repository, does not have a valid or reachable HEAD, or the verification fails for any other reason. The function does not modify proj_dir or any filesystem state.
- **Actual behavior:** If the function returns normally (no uncaught exception), then:
 - the return value is True if and only if executing `git -C proj_dir rev-parse --verify HEAD` via subprocess.run succeeds (return code 0, no CalledProcessError);
 - the return value is False if and only if that subprocess.run call raises a subprocess.CalledProcessError (nonzero exit code, e.g., because proj_dir is not a git repository or has no commits). If any other exception occurs (e.g., FileNotFoundError from missing git, OSError, TimeoutExpired, etc.), it propagates uncaught and the function exits without returning a value. In both normal and exceptional exits, proj_dir is not modified and no persistent side effects are produced; the only observable effect is the execution of the subprocess. In formal logic: (exception return = True) (subprocess.run(...) completes with returncode = 0 no CalledProcessError) (exception return = False) (subprocess.run(...) raises CalledProcessError) (exception) (return is undefined proj_dir = proj_dir_pre)
- **Code evidence:** Line 9: except subprocess.CalledProcessError:
- **Trigger condition:** The code only catches subprocess.CalledProcessError. If the 'git' executable is missing, subprocess.run raises FileNotFoundError (or another OSError), which is not caught, so the function propagates an uncaught exception instead of returning False as required by the specification for any verification failure.
- **Validator trigger:** The function only catches subprocess.CalledProcessError; when the 'git' executable is missing, subprocess.run raises FileNotFoundError, which is not caught, causing the function to propagate an exception instead of returning False.
- **Probe stdout:** CONFIRMED — bug reproduced: function crashed with FileNotFoundError: [Errno 2] No such file or directory: 'git'\n Expected: False\n Actual: uncaught exception propagated

► SPEC sidecar (完整)

```
{
  "signature": "_is_git_repo(proj_dir) -> bool",
  "pre_condition": "proj_dir is a filesystem path string",
  "post_condition": "Returns True if and only if proj_dir is an existing git repository whose HEAD reference resolves to at least one commit. Returns False if proj_dir is not a git repository, does not have a valid or reachable HEAD, or the verification fails for any other reason. The function does not modify proj_dir or any filesystem state."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/git-py/_is_git_repo.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns True if and only if proj_dir is an existing git repository whose HEAD reference resolves to at least one commit. Returns False if proj_dir is not a git repository, does not have a valid or reachable HEAD, or the verification fails for any other reason. The function does not modify proj_dir or any filesystem state.",
    "actual_behavior": "If the function returns normally (no uncaught exception), then:\n- the return value is True if and only if executing `git -C proj_dir rev-parse --verify HEAD` via subprocess.run succeeds (return code 0, no CalledProcessError);\n- the return value is False if and only if that subprocess.run call raises a subprocess.CalledProcessError (nonzero exit code, e.g., because proj_dir is not a git repository or has no commits).\nIf any other exception occurs (e.g., FileNotFoundError from missing git, OSError, TimeoutExpired, etc.), it propagates uncaught and the function exits without returning a value.\nIn both normal and exceptional exits, proj_dir is not modified and no persistent side effects are produced; the only observable effect is the execution of the subprocess. In formal logic:\n(exception return = True) (subprocess.run(...) completes with returncode = 0 no CalledProcessError)\n(exception return = False) (subprocess.run(...) raises CalledProcessError)\n(exception) (return is undefined proj_dir = proj_dir_pre)",
    "code_evidence": "Line 9: except subprocess.CalledProcessError:",
    "trigger_condition": "The code only catches subprocess.CalledProcessError. If the 'git' executable is missing, subprocess.run raises FileNotFoundError (or another OSError), which is not caught, so the function propagates an uncaught exception instead of returning False as required by the specification for any verification failure."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src-git-py--_is_git_repo",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/git-py/_is_git_repo.py",
}
```



```
"function_name": "_is_git_repo",
"confirmation_status": "confirmed",
"attempts": 1,
"probe_script": "fm_agent/bug_validation/probe_src--git-py--_is_git_repo.py",
"detail_file": "fm_agent/bug_validation/src--git-py--_is_git_repo.md",
"probe_stdout": "CONFIRMED - bug reproduced: function crashed with FileNotFoundError: [Errno 2] No such file or directory:
'git'\\n Expected: False\\n Actual:   uncaught exception propagated",
"trigger_summary": "The function only catches subprocess.CalledProcessError; when the 'git' executable is missing,
subprocess.run raises FileNotFoundError, which is not caught, causing the function to propagate an exception instead of returning
False."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:36:22.355136Z	1743
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:36:45.609333Z	1933

事件 1 — [llm_2a520ccf56e2423e917a7777bfa771cc](#)

► parsed_json（完整）

```
{
  "post_condition": "If the function returns normally (no uncaught exception), then:\\n- the return value is True if and only if
executing `git -C proj_dir rev-parse --verify HEAD` via subprocess.run succeeds (return code 0, no CalledProcessError);\\n- the
return value is False if and only if that subprocess.run call raises a subprocess.CalledProcessError (non-zero exit code, e.g.,
because proj_dir is not a git repository or has no commits).\\nIf any other exception occurs (e.g., FileNotFoundError from missing
git, OSError, TimeoutExpired, etc.), it propagates uncaught and the function exits without returning a value.\\nIn both normal and
exceptional exits, proj_dir is not modified and no persistent side effects are produced; the only observable effect is the
execution of the subprocess. In formal logic:\\n(¬exception ∧ return = True) ↔ (subprocess.run(...) completes with returncode = 0
∧ no CalledProcessError)\\n(¬exception ∧ return = False) ↔ (subprocess.run(...) raises CalledProcessError)\\n(exception) → (return
is undefined ∧ proj_dir = proj_dir_pre)"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_2a520ccf56e2423e917a7777bfa771cc_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_2a520ccf56e2423e917a7777bfa771cc_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_2a520ccf56e2423e917a7777bfa771cc_response.txt)

事件 2 — [llm_bc410c2f18fc4be198a3a1d9a351b118](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp/example' on a system where 'git' is not installed or not in the PATH.",
  "offending_statements": "Line 9: except subprocess.CalledProcessError:",
  "reason": "The code only catches subprocess.CalledProcessError. If the 'git' executable is missing, subprocess.run raises
FileNotFoundError (or another OSError), which is not caught, so the function propagates an uncaught exception instead of returning
False as required by the specification for any verification failure."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_bc410c2f18fc4be198a3a1d9a351b118_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_bc410c2f18fc4be198a3a1d9a351b118_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_bc410c2f18fc4be198a3a1d9a351b118_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-046 — [src--git-py--frozen_worktree::_git](#)

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: [confirmed](#); attempts 1
- Phase / 模块：Phase 1 — Initialization & Configuration / [git_utils](#); 原源码 [src/git.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

- **SPEC claim:** Executes `git -C proj_dir` with the arguments provided in `args` as a subprocess and returns the captured stdout with surrounding whitespace removed. By default, raises `subprocess.CalledProcessError` when the `git` subprocess exits with a non-zero status; this error behavior is overridable via `kwargs`. By default, `stdout` and `stderr` are captured and decoded as text; both defaults are overridable via `kwargs`.
- **Actual behavior:** If the system `git` executable is available in the `PATH` and the subprocess command `git -C <proj_dir> <args>` executes successfully (exit code 0), the function returns a string containing the stripped standard output of that command. If the command executes but exits with a non-zero code, a `subprocess.CalledProcessError` is raised. If the `git` executable cannot be found or executed (e.g., not installed), an `OSError` (specifically `FileNotFoundError` on POSIX) is raised. The function performs no other side effects on the calling program's state. Formal logic: Let `cmd = ["git", "-C", proj_dir] + list(args)` and `env` denote the execution environment. The post-condition is: (out: subprocess.CompletedProcess such that out.returncode = 0 return = out.stdout.strip()) (exc: CalledProcessError such that exc.returncode 0 exc.cmd = cmd raised(exc)) (exc: FileNotFoundError (or OSError) such that raised(exc)).
- **Code evidence:** Line 2-5: `subprocess.run(...).stdout.strip()`
- **Trigger condition:** The specification states that the default for capturing `stdout` is overridable via `kwargs`. If a caller overrides `capture_output=False`, the function attempts to access `.stdout.strip()` on the returned `CompletedProcess`, but `.stdout` is `None`, raising an `AttributeError`. This violates the specification because the function does not properly handle the overridden default.
- **Validator trigger:** `_git()` hardcodes `capture_output=True` in `subprocess.run()`, preventing any override via `kwargs` — passing `capture_output=False` raises `TypeError` for duplicate keyword arguments, contradicting the specification which states both defaults are overridable.
- **Probe stdout:** CONFIRMED — `TypeError` raised when passing `capture_output=False` via `kwargs`: `subprocess.run()` got multiple values for keyword argument 'capture_output' — spec says `capture_output` default is overridable, but the hardcoded `capture_output=True` prevents any override

► SPEC sidecar（完整）

```
{
  "signature": "_git(*args, **kwargs) -> str",
  "pre_condition": "proj_dir is a string path to an existing directory. Each element of args is a string.",
  "post_condition": "Executes git -C proj_dir with the arguments provided in args as a subprocess and returns the captured stdout with surrounding whitespace removed. By default, raises subprocess.CalledProcessError when the git subprocess exits with a non-zero status; this error behavior is overridable via kwargs. By default, stdout and stderr are captured and decoded as text; both defaults are overridable via kwargs."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/git-py/frozen_worktree:_git.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Executes git -C proj_dir with the arguments provided in args as a subprocess and returns the captured stdout with surrounding whitespace removed. By default, raises subprocess.CalledProcessError when the git subprocess exits with a non-zero status; this error behavior is overridable via kwargs. By default, stdout and stderr are captured and decoded as text; both defaults are overridable via kwargs.",
    "actual_behavior": "If the system git executable is available in the PATH and the subprocess command `git -C <proj_dir> <args>` executes successfully (exit code 0), the function returns a string containing the stripped standard output of that command. If the command executes but exits with a non-zero code, a `subprocess.CalledProcessError` is raised. If the git executable cannot be found or executed (e.g., not installed), an `OSError` (specifically `FileNotFoundError` on POSIX) is raised. The function performs no other side effects on the calling program's state. Formal logic: Let `cmd = [\"git\", \"-C\", proj_dir] + list(args)` and `env` denote the execution environment. The post-condition is: ( out: subprocess.CompletedProcess such that out.returncode = 0 return = out.stdout.strip()) ( exc: CalledProcessError such that exc.returncode 0 exc.cmd = cmd raised(exc)) ( exc: FileNotFoundError (or OSError) such that raised(exc)).",
    "code_evidence": "Line 2-5: subprocess.run(...).stdout.strip()",
    "trigger_condition": "The specification states that the default for capturing stdout is overridable via kwargs. If a caller overrides capture_output=False, the function attempts to access .stdout.strip() on the returned CompletedProcess, but .stdout is None, raising an AttributeError. This violates the specification because the function does not properly handle the overridden default."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--git-py--frozen_worktree::_git",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/git-py/frozen_worktree::_git.py",
  "function_name": "_git",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--git-py--frozen_worktree::_git.py",
  "detail_file": "fm_agent/bug_validation/src--git-py--frozen_worktree::_git.md",
  "probe_stdout": "CONFIRMED - TypeError raised when passing capture_output=False via kwargs: subprocess.run() got multiple values
for keyword argument 'capture_output' - spec says capture_output default is overridable, but the hardcoded capture_output=True
prevents any override",
  "trigger_summary": "_git() hardcodes capture_output=True in subprocess.run(), preventing any override via kwargs - passing
capture_output=False raises TypeError for duplicate keyword arguments, contradicting the specification which states both defaults
are overridable."
}
```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-27T15:44:20.722591Z	2838
2	generate_block_post_condition	success	2	0/1	2026-07-27T15:44:59.032679Z	1463
3	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:45:10.366541Z	6405

事件 1 — [llm_eadf9fcff2dd46779189c1c6216533f4](#)

- parse_error: LLM response is not valid JSON: Expecting ',' delimiter: line 2 column 599 (char 600)
- 载荷: [system_prompt](#) · [user_prompt](#) · [assistant_output](#)

事件 2 — [llm_e0c36b90ab794ed2880ad1cf7a0e4ef9](#)

► parsed_json（完整）

```
{
  "post_condition": "If the system git executable is available in the PATH and the subprocess command `git -C <proj_dir> <args>`
executes successfully (exit code 0), the function returns a string containing the stripped standard output of that command. If the
command executes but exits with a non-zero code, a `subprocess.CalledProcessError` is raised. If the git executable cannot be
found or executed (e.g., not installed), an `OSError` (specifically `FileNotFoundError` on POSIX) is raised. The function performs
no other side effects on the calling program's state. Formal logic: Let `cmd = [\"git\", \"-C\", proj_dir] + list(args)` and `env`
denote the execution environment. The post-condition is: (∃ out: subprocess.CompletedProcess such that out.returncode = 0 ⇒ return
= out.stdout.strip()) ∨ (∃ exc: CalledProcessError such that exc.returncode ≠ 0 ∧ exc.cmd = cmd ∧ raised(exc)) ∨ (∃ exc:
FileNotFoundError (or OSError) such that raised(exc))."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_e0c36b90ab794ed2880ad1cf7a0e4ef9_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_e0c36b90ab794ed2880ad1cf7a0e4ef9_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_e0c36b90ab794ed2880ad1cf7a0e4ef9_message_02_assistant.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_e0c36b90ab794ed2880ad1cf7a0e4ef9_message_03_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_e0c36b90ab794ed2880ad1cf7a0e4ef9_response.txt\)](#)

事件 3 — [llm_44811a63f0c94f929d6b604d3f07ddbd](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "args=(), kwargs={'capture_output': False} (with git command exiting successfully)",
  "offending_statements": "Line 2-5: subprocess.run(...).stdout.strip()",
  "reason": "The specification states that the default for capturing stdout is overridable via kwargs. If a caller overrides
capture_output=False, the function attempts to access .stdout.strip() on the returned CompletedProcess, but .stdout is None,
raising an AttributeError. This violates the specification because the function does not properly handle the overridden default."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_44811a63f0c94f929d6b604d3f07ddbd_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_44811a63f0c94f929d6b604d3f07ddbd_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_44811a63f0c94f929d6b604d3f07ddbd_response.txt\)](#) **复核动作**

- 核对 `probe` 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

Phase 2 — Language Backend & Function Extraction

模块：`call_graph_edges`

FMA-MISMATCH-054 — `src--call_graph_edges-py--normalize_endpoint_label`

- 四类结论：可能有 **Bug**
- 分类依据：沿用旧 `fm_agent/bug_list.md` 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**：`confirmed`；attempts `1`
- **Phase / 模块**：Phase 2 — Language Backend & Function Extraction / `call_graph_edges`；原源码 `src/call_graph_edges.py`
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**：When label contains at least one `::` and the substring before the last `::` consists of POSIX-path-like components, returns a canonical FQN where: (1) any leading `./` prefix in the path portion is removed; (2) the last `.` in the path's final filename component is replaced by `-`; (3) all parent-directory path components excluding `.` and the empty string are joined with `::` to form the FQN prefix; and (4) the function name is appended as the final `::`-separated segment. When label does not match this pattern, returns label unchanged after cleaning.
- **Actual behavior**：The function returns a new string with no side effects. Let `cl = _clean_label(label)` (the label with leading and trailing whitespace removed). If `_is_path_function_label(cl)` returns True, let `(path_str, func) = cl.rsplit(':', 1)`; let `stripped_path = path_str.lstrip('./')` (removing any leading occurrences of `.` and `/`); let `pp = PurePosixPath(stripped_path)`; let `base = pp.name`; let `last_dot = base.rfind('.')`; let `func_dir = (base[:last_dot] + '-' + base[last_dot+1:])` if `last_dot > 0` else `base`; let `parts = [p for p in pp.parent.parts if p not in {'', '.'}]`; then the return value is `::'.join(parts + [func_dir, func])`. Otherwise, the return value is `cl`. Formally: `result = ((lambda cl: (lambda path_str, func: (lambda stripped_path: (lambda pp: (lambda base: (lambda last_dot: (lambda func_dir: (lambda parts: '::'.join(parts + [func_dir, func]))([p for p in pp.parent.parts if p not in {'', '.'}])(base[:last_dot] + '-' + base[last_dot+1:]) if last_dot > 0 else base))(base.rfind('.')))(pp.name))(PurePosixPath(stripped_path))(path_str.lstrip('./'))(*cl.rsplit(':', 1)) if _is_path_function_label(cl) else cl)(_clean_label(label))`
- **Code evidence**：Line 5: `path = path.lstrip('./')`
- **Trigger condition**：The specification requires removing only a leading `./` prefix, but `lstrip('./')` removes all leading `.` and `/` characters, so `../foo::func` incorrectly becomes `foo::func` instead of `...:foo::func`.
- **Validator trigger**：`lstrip('./')` strips all leading `.` and `/` characters, so `../foo.c::func` loses the `..` parent component and becomes `foo-c::func` instead of `...:foo-c::func`.
- **Probe stdout**：CONFIRMED — actual: `'foo-c::func'` | expected: `'...:foo-c::func'`

► SPEC sidecar（完整）

```
{
  "signature": "_normalize_endpoint_label(label: str) -> str",
  "pre_condition": "label is a non-empty string. If label represents a file-path-qualified function name, the path separators have been normalized to the FQN separator '::' (i.e., the label contains no '/' characters).",
  "post_condition": "When label contains at least one '::' and the substring before the last '::' consists of POSIX-path-like components, returns a canonical FQN where: (1) any leading './' prefix in the path portion is removed; (2) the last '.' in the path's final filename component is replaced by '-'; (3) all parent-directory path components excluding '.' and the empty string are joined with '::' to form the FQN prefix; and (4) the function name is appended as the final '::'-separated segment. When label does not match this pattern, returns label unchanged after cleaning."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_clean_label",
      "signature": "_clean_label(label: str) -> str",
      "pre_condition": "label is a string.",
      "post_condition": "Returns label with any leading and trailing whitespace stripped."
    },
    {
      "name": "_is_path_function_label",
      "signature": "_is_path_function_label(label: str) -> bool",
      "pre_condition": "label is a cleaned string.",
      "post_condition": "Returns True when label contains '::' and the substring before the last '::' consists of one or more POSIX-path-like components; returns False otherwise."
    }
  ]
}
```

```
}
]
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-
py/_normalize_endpoint_label.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When label contains at least one ':' and the substring before the last ':' consists of POSIX-path-like
components, returns a canonical FQN where: (1) any leading './' prefix in the path portion is removed; (2) the last '.' in the
path's final filename component is replaced by '-'; (3) all parent-directory path components excluding '.' and the empty string
are joined with ':' to form the FQN prefix; and (4) the function name is appended as the final ':'-separated segment. When label
does not match this pattern, returns label unchanged after cleaning.",
    "actual_behavior": "The function returns a new string with no side effects. Let cl = _clean_label(label) (the label with
leading and trailing whitespace removed). If _is_path_function_label(cl) returns True, let (path_str, func) = cl.rsplit(':', 1);
let stripped_path = path_str.lstrip('./') (removing any leading occurrences of '.' and '/'); let pp =
PurePosixPath(stripped_path); let base = pp.name; let last_dot = base.rfind('.'); let func_dir = (base[:last_dot] + '-' +
base[last_dot+1:]) if last_dot > 0 else base; let parts = [p for p in pp.parent.parts if p not in {'', '.'}]; then the return
value is ':'.join(parts + [func_dir, func]). Otherwise, the return value is cl. Formally: \nresult = ( (lambda cl: (lambda
path_str, func: (lambda stripped_path: (lambda pp: (lambda base: (lambda last_dot: (lambda func_dir: (lambda parts:
':'.join(parts + [func_dir, func]))([p for p in pp.parent.parts if p not in {'', '.'}]))((base[:last_dot] + '-' +
base[last_dot+1:]) if last_dot > 0 else base))(base.rfind('.')))(pp.name))(PurePosixPath(stripped_path)))(path_str.lstrip('./'))
(*cl.rsplit(':', 1)) if _is_path_function_label(cl) else cl )(_clean_label(label))",
    "code_evidence": "Line 5: path = path.lstrip(\"./\")",
    "trigger_condition": "The specification requires removing only a leading './' prefix, but lstrip('./') removes all leading '.'
and '/' characters, so './foo::func' incorrectly becomes 'foo::func' instead of '...:foo::func'."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--call_graph_edges-py--_normalize_endpoint_label",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-
py/_normalize_endpoint_label.py",
  "function_name": "_normalize_endpoint_label",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--call_graph_edges-py--_normalize_endpoint_label.py",
  "detail_file": "fm_agent/bug_validation/src--call_graph_edges-py--_normalize_endpoint_label.md",
  "probe_stdout": "CONFIRMED - actual: 'foo-c::func' | expected: '...:foo-c::func'",
  "trigger_summary": "lstrip(\"./\") strips all leading '.' and '/' characters, so './foo-c::func' loses the '.' parent
component and becomes 'foo-c::func' instead of '...:foo-c::func'."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:48:57.381680Z	3754
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:49:47.872550Z	4778

事件 1 — [llm_4a39c0868104484da399a60a3113a519](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a new string with no side effects. Let cl = _clean_label(label) (the label with leading
and trailing whitespace removed). If _is_path_function_label(cl) returns True, let (path_str, func) = cl.rsplit(':', 1); let
stripped_path = path_str.lstrip('./') (removing any leading occurrences of '.' and '/'); let pp = PurePosixPath(stripped_path);
let base = pp.name; let last_dot = base.rfind('.'); let func_dir = (base[:last_dot] + '-' + base[last_dot+1:]) if last_dot > 0
else base; let parts = [p for p in pp.parent.parts if p not in {'', '.'}]; then the return value is ':'.join(parts + [func_dir,
func]). Otherwise, the return value is cl. Formally: \nresult = ( (lambda cl: (lambda path_str, func: (lambda stripped_path:
(lambda pp: (lambda base: (lambda last_dot: (lambda func_dir: (lambda parts: ':'.join(parts + [func_dir, func]))([p for p in
pp.parent.parts if p not in {'', '.'}]))((base[:last_dot] + '-' + base[last_dot+1:]) if last_dot > 0 else base))(base.rfind('.'))
(pp.name))(PurePosixPath(stripped_path)))(path_str.lstrip('./')))(*cl.rsplit(':', 1)) if _is_path_function_label(cl) else cl )
(_clean_label(label))"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4a39c0868104484da399a60a3113a519_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_4a39c0868104484da399a60a3113a519_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_4a39c0868104484da399a60a3113a519_response.txt)

事件 2 — llm 4ccf150d496a4cc3b9477004fddae6f7

► parsed json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "../foo::func",
  "offending_statements": "Line 5: path = path.rstrip(\".\").",
  "reason": "The specification requires removing only a leading './' prefix, but lstrip('./') removes all leading '.' and '/' characters, so '../foo::func' incorrectly becomes 'foo::func' instead of '...:foo::func'."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4ccf150d496a4cc3b9477004fddae6f7_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4ccf150d496a4cc3b9477004fddae6f7_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4ccf150d496a4cc3b9477004fddae6f7_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-058 — `src--call_graph` edges-py--load_call edges

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator：confirmed；attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / call_graph_edges；原源码 src/call_graph_edges.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** When path is None, returns an empty list. When path identifies a single file, returns the CallEdge objects parsed from that file with all duplicate edges removed. When path identifies a directory, returns the CallEdge objects from every qualifying edge file found recursively under that directory, combined into a single list with all duplicate edges removed. In all non-None cases, each returned element is a CallEdge, and for any distinct pair of caller and callee specifications at most one CallEdge appears. Results derived from a directory walk are returned in a stable order consistent with the filesystem walk ordering.
- **Actual behavior:** Natural language: If path is None, the return value is an empty list. Otherwise, let p = Path(path). If p is a directory, the function recursively collects files via p.rglob('*'), selects those for which is_file() and _is_edge_file() return True, sorts them lexicographically, loads the CallEdge list from each via _load_call_edge_file, concatenates them in that order, deduplicates by caller/callee specification while preserving first occurrence, and returns the resulting list. If p is a file, the function loads the CallEdge list from that file via _load_call_edge_file, deduplicates it, and returns the result. If any step raises an exception (e.g., Path construction, filesystem traversal, file reading, or JSON parsing), that exception propagates and no value is returned.

Formal logic: Let **ret** denote the return value after successful execution.

- If `path` is `None`: `ret = []`.
- Else define `p = Path(path)`.
 - If `p.is_dir()`: `files = [f for f in sorted(p.rglob('*')) if f.is_file() and _is_edge_file(f)]` `edges = concatenation of [(_load_call_edge_file(f)) for f in files]` `ret = dedupe(edges)` where `dedupe` ensures `i < j`, (`ret[i].caller == ret[j].caller` `ret[i].callee == ret[j].callee`) and each element appears at its earliest occurrence index in `edges`.
 - If `p.is_file()`: `ret = dedupe(_load_call_edge_file(p))`. If any operation above raises an exception `E`, the function exits with exception `E` and `ret` is not produced.
- **Code evidence:** Line 8: `for file_path in sorted(edge_path.rglob('*')):`
- **Trigger condition:** The specification requires the order of results to be 'consistent with the filesystem walk ordering', i.e., the order in which `rglob` yields the entries. The code invokes `sorted()` on the `rglob` result, which imposes an alphabetical order that may differ from the walk ordering, thereby violating the specification.
- **Validator trigger:** The code calls `sorted()` on `rglob` results, imposing alphabetical order instead of filesystem walk ordering as the spec requires; observable when two files with the same edge key are processed in the wrong order.
- **Probe stdout:** `rglob` walk order: `['z.json', 'a.json']` `sorted(rglob)` order: `['a.json', 'z.json']` `load_call_edges` merged edge source: `'SOURCE FROM a'` **CONFIRMED** — `load_call_edges` uses `sorted` order (`source='SOURCE FROM a'`) instead of walk order

(expected='SOURCE_FROM_z')

► SPEC sidecar (完整)

```
{
  "signature": "load_call_edges(path: str | os.PathLike | None) -> list[CallEdge]",
  "pre_condition": "path is None, or identifies a file containing supplemental call edges in JSON format, or identifies a directory whose recursive contents may include JSON call-edge files.",
  "post_condition": "When path is None, returns an empty list. When path identifies a single file, returns the CallEdge objects parsed from that file with all duplicate edges removed. When path identifies a directory, returns the CallEdge objects from every qualifying edge file found recursively under that directory, combined into a single list with all duplicate edges removed. In all non-None cases, each returned element is a CallEdge, and for any distinct pair of caller and callee specifications at most one CallEdge appears. Results derived from a directory walk are returned in a stable order consistent with the filesystem walk ordering."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_is_edge_file",
      "signature": "_is_edge_file(path: Path) -> bool",
      "pre_condition": "path refers to a file on disk.",
      "post_condition": "Returns True when the file is recognized as containing supplemental call edge data; False otherwise."
    },
    {
      "name": "_load_call_edge_file",
      "signature": "_load_call_edge_file(path: Path) -> list[CallEdge]",
      "pre_condition": "path refers to a file containing valid supplemental call edge data matching the expected JSON schema.",
      "post_condition": "Returns the list of CallEdge objects parsed from the file, in the order they appear in the file."
    },
    {
      "name": "_dedupe_edges",
      "signature": "_dedupe_edges(edges: list[CallEdge]) -> list[CallEdge]",
      "pre_condition": "edges is a list of CallEdge objects, possibly containing entries with identical caller and callee specifications.",
      "post_condition": "Returns a list of CallEdge objects where no two entries share the same caller specification and callee specification. The relative order of first occurrences is preserved."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-py/load_call_edges.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When path is None, returns an empty list. When path identifies a single file, returns the CallEdge objects parsed from that file with all duplicate edges removed. When path identifies a directory, returns the CallEdge objects from every qualifying edge file found recursively under that directory, combined into a single list with all duplicate edges removed. In all non-None cases, each returned element is a CallEdge, and for any distinct pair of caller and callee specifications at most one CallEdge appears. Results derived from a directory walk are returned in a stable order consistent with the filesystem walk ordering.",
    "actual_behavior": "Natural language: If `path` is `None`, the return value is an empty list. Otherwise, let `p = Path(path)`. If `p` is a directory, the function recursively collects files via `p.rglob('*')`, selects those for which `is_file()` and `_is_edge_file()` return `True`, sorts them lexicographically, loads the `CallEdge` list from each via `_load_call_edge_file`, concatenates them in that order, deduplicates by caller/callee specification while preserving first occurrence, and returns the resulting list. If `p` is a file, the function loads the `CallEdge` list from that file via `_load_call_edge_file`, deduplicates it, and returns the result. If any step raises an exception (e.g., `Path` construction, filesystem traversal, file reading, or JSON parsing), that exception propagates and no value is returned.\n\nFormal logic:\nLet `ret` denote the return value after successful execution.\n- If `path` is `None`: `ret = []`.\n- Else define `p = Path(path)`.\n  - If `p.is_dir()`:\n    `files = [f for f in sorted(p.rglob('*')) if f.is_file() and _is_edge_file(f)]`\n    `edges = concatenation of [_load_call_edge_file(f) for f in files]`\n    `ret = dedupe(edges)` where `dedupe` ensures `i < j, (ret[i].caller == ret[j].caller & ret[i].callee == ret[j].callee)` and each element appears at its earliest occurrence index in `edges`.\n  - If `p.is_file()`:\n    `ret = dedupe(_load_call_edge_file(p))`.\nIf any operation above raises an exception `E`, the function exits with exception `E` and `ret` is not produced.",
    "code_evidence": "Line 8:         for file_path in sorted(edge_path.rglob(\"*\")):",
    "trigger_condition": "The specification requires the order of results to be 'consistent with the filesystem walk ordering', i.e., the order in which rglob yields the entries. The code invokes sorted() on the rglob result, which imposes an alphabetical order that may differ from the walk ordering, thereby violating the specification."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--call_graph_edges-py--load_call_edges",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-py/load_call_edges.py",
  "function_name": "load_call_edges",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--call_graph_edges-py--load_call_edges.py",
  "detail_file": "fm_agent/bug_validation/src--call_graph_edges-py--load_call_edges.md",
  "probe_stdout": "rglob walk order:  ['z.json', 'a.json']\nsorted(rglob) order: ['a.json', 'z.json']\nload_call_edges merged
edge source: 'SOURCE_FROM_a'\n\nCONFIRMED - load_call_edges uses sorted order (source='SOURCE_FROM_a') instead of walk order
(expected='SOURCE_FROM_z')\n",
  "trigger_summary": "The code calls sorted() on rglob results, imposing alphabetical order instead of filesystem walk ordering as
the spec requires; observable when two files with the same edge key are processed in the wrong order."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:35:00.362325Z	1880
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:35:21.966251Z	4865

事件 1 — [llm_1a2626db48f7472281e7578a242bfc8b](#)

► parsed_json（完整）

```
{
  "post_condition": "Natural language: If `path` is `None`, the return value is an empty list. Otherwise, let `p = Path(path)`. If
`p` is a directory, the function recursively collects files via `p.rglob('*')`, selects those for which `is_file()` and
`_is_edge_file()` return `True`, sorts them lexicographically, loads the `CallEdge` list from each via `_load_call_edge_file`,
concatenates them in that order, deduplicates by caller/callee specification while preserving first occurrence, and returns the
resulting list. If `p` is a file, the function loads the `CallEdge` list from that file via `_load_call_edge_file`, deduplicates
it, and returns the result. If any step raises an exception (e.g., `Path` construction, filesystem traversal, file reading, or
JSON parsing), that exception propagates and no value is returned.\n\nFormal logic:\nLet `ret` denote the return value after
successful execution.\n- If `path` is None: `ret = []`. \n- Else define `p = Path(path)`. \n  - If `p.is_dir()`: \n    `files = [f
for f in sorted(p.rglob('*')) if f.is_file() and _is_edge_file(f)]` \n    `edges = concatenation of [_load_call_edge_file(f)) for
f in files]` \n    `ret = dedupe(edges)` where `dedupe` ensures `∀ i<j, ¬(ret[i].caller == ret[j].caller ∧ ret[i].callee ==
ret[j].callee)` and each element appears at its earliest occurrence index in `edges`. \n  - If `p.is_file()`: \n    `ret =
dedupe(_load_call_edge_file(p))`. \nIf any operation above raises an exception `E`, the function exits with exception `E` and `ret`
is not produced."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1a2626db48f7472281e7578a242bfc8b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_1a2626db48f7472281e7578a242bfc8b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_1a2626db48f7472281e7578a242bfc8b_response.txt)

事件 2 — [llm_b9b8e8906e5a4e5fb67f20d4f1e8fbbb](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "A directory containing two edge files: 'b_edges.json' and 'a_edges.json', where the natural filesystem
ordering (as returned by rglob) places 'b_edges.json' first (e.g., because it was created first). The sorting in the code reorders
them to 'a_edges.json' then 'b_edges.json', violating the required order.",
  "offending_statements": "Line 8:          for file_path in sorted(edge_path.rglob(\"*\")): ",
  "reason": "The specification requires the order of results to be 'consistent with the filesystem walk ordering', i.e., the order
in which rglob yields the entries. The code invokes sorted() on the rglob result, which imposes an alphabetical order that may
differ from the walk ordering, thereby violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b9b8e8906e5a4e5fb67f20d4f1e8fbbb_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_b9b8e8906e5a4e5fb67f20d4f1e8fbbb_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_b9b8e8906e5a4e5fb67f20d4f1e8fbbb_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块：codegraph

- **四类结论：** 可能有 **Bug**
- **分类依据：** 沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator：** [confirmed](#)；attempts 2
- **Phase / 模块：** Phase 2 — Language Backend & Function Extraction / [codegraph](#)；原源码 [src/languages/codegraph.py](#)
- **证据：** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim：** Returns the bare unqualified function or method name with all tree-sitter decorations removed. The result is empty exactly when the input is empty or whitespace-only. For names containing the 'operator' keyword, the result is the complete operator specifier: the keyword 'operator' followed by the operator symbol or keyword suffix, with any separating whitespace collapsed. For all other names, the result is the first alphabetic or alphanumeric identifier token extracted after stripping prefix decorations and before any trailing parameter lists, template bodies, or type annotations. The returned string contains no '::' or '.' qualifier separators. If no identifier token can be extracted from a non-empty input, the stripped input is returned unchanged.
- **Actual behavior：** The function returns a string r. Let s = name.strip(). If s is empty, r = ". Otherwise, define tail = (s.rsplitt(':', 1)[1].lstrip() if ':' in s else s.rsplitt('.', 1)[1].lstrip()) if '.' in s else s. If tail.startswith('operator'): let rest = tail[8:].lstrip(); if rest.startswith('['): r = 'operator['; elif rest.startswith('('): r = 'operator('; elif rest is either exactly 'new' or consists of 'new' followed by optional whitespace, then '[', optional whitespace, ']' with nothing else, then r = 'operator new[' if '[' in rest else 'operator new'; elif rest is either exactly 'delete' or consists of 'delete' followed by optional whitespace, then '[', optional whitespace, ']' with nothing else, then r = 'operator delete[' if '[' in rest else 'operator delete'; else: let prefix be the longest initial substring of rest containing only characters from {+, -, , /, %, &, |, ^, ~, !, =, <, >, .}; if prefix is not empty, r = 'operator' + prefix. If r is still unassigned, then examine s: if s ends with a sequence of word characters (alphanumeric or underscore) that is immediately preceded by either the start of the string, '::', or '.', r is that word sequence; else if s matches the pattern '(' followed by optional whitespace, ", optional whitespace, a word, optional whitespace, ')', r is that word; else if s matches the pattern '*' followed by optional whitespace, a word, r is that word; else if s starts with a word, r is that word; else r = s.
- **Code evidence：** Line 43: m = re.search(r'(?::~:|\.)(\w+)\$', name) Line 45: return m.group(1)
- **Trigger condition：** For input 'func -> int', the code returns 'int' (the last word) but the specification requires returning the first identifier token before trailing type annotations, i.e., 'func'.
- **Validator trigger：** Input 'foo::bar -> int': qualifier '::' is stripped to get 'bar', but the regex on line 49 searches the original name and fails, so the fallback incorrectly returns 'foo'.
- **Probe stdout：** CONFIRMED — actual: 'foo' | expected: 'bar'

► SPEC sidecar（完整）

```
{
  "signature": "_bare_function_name(name: str) -> str",
  "pre_condition": "name is a string that may contain tree-sitter decorations – including qualified name prefixes (separated by '::' or '.'), pointer/receiver syntax, type annotations, parameter lists, template bodies, and full function signatures.",
  "post_condition": "Returns the bare unqualified function or method name with all tree-sitter decorations removed. The result is empty exactly when the input is empty or whitespace-only. For names containing the 'operator' keyword, the result is the complete operator specifier: the keyword 'operator' followed by the operator symbol or keyword suffix, with any separating whitespace collapsed. For all other names, the result is the first alphabetic or alphanumeric identifier token extracted after stripping prefix decorations and before any trailing parameter lists, template bodies, or type annotations. The returned string contains no '::' or '.' qualifier separators. If no identifier token can be extracted from a non-empty input, the stripped input is returned unchanged."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/_bare_function_name.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns the bare unqualified function or method name with all tree-sitter decorations removed. The result is empty exactly when the input is empty or whitespace-only. For names containing the 'operator' keyword, the result is the complete operator specifier: the keyword 'operator' followed by the operator symbol or keyword suffix, with any separating whitespace collapsed. For all other names, the result is the first alphabetic or alphanumeric identifier token extracted after stripping
```

```
prefix decorations and before any trailing parameter lists, template bodies, or type annotations. The returned string contains no
'::' or '.' qualifier separators. If no identifier token can be extracted from a non-empty input, the stripped input is returned
unchanged.",
    "actual_behavior": "The function returns a string r. Let s = name.strip(). If s is empty, r = ''. Otherwise, define tail =
(s.rsplit(':', 1)[1].rstrip() if ':' in s else s.rsplit('.', 1)[1].rstrip()) if '.' in s else s. If tail.startswith('operator'):
let rest = tail[8:].rstrip(); if rest.startswith('[]'): r = 'operator[]'; elif rest.startswith('('): r = 'operator()'; elif rest
is either exactly 'new' or consists of 'new' followed by optional whitespace, then '[' , optional whitespace, ']' with nothing
else, then r = 'operator new[]' if '[' in rest else 'operator new'; elif rest is either exactly 'delete' or consists of 'delete'
followed by optional whitespace, then '[' , optional whitespace, ']' with nothing else, then r = 'operator delete[]' if '[' in rest
else 'operator delete'; else: let prefix be the longest initial substring of rest containing only characters from {+, -, *, /, %,
&, |, ^, ~, !, =, <, >, ,}; if prefix is not empty, r = 'operator' + prefix. If r is still unassigned, then examine s: if s ends
with a sequence of word characters (alphanumeric or underscore) that is immediately preceded by either the start of the string,
'::', or '.', r is that word sequence; else if s matches the pattern '(' followed by optional whitespace, '*', optional
whitespace, a word, optional whitespace, ')', r is that word; else if s matches the pattern '*' followed by optional whitespace, a
word, r is that word; else if s starts with a word, r is that word; else r = s.",
    "code_evidence": "Line 43: m = re.search(r'(?^[:~\.\.)(\\w+)$', name)\nLine 45: return m.group(1)",
    "trigger_condition": "For input 'func -> int', the code returns 'int' (the last word) but the specification requires returning
the first identifier token before trailing type annotations, i.e., 'func'."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--codegraph-py--_bare_function_name",
  "source_file": "src/languages/codegraph.py",
  "function_name": "_bare_function_name",
  "confirmation_status": "confirmed",
  "attempts": 2,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--_bare_function_name.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--_bare_function_name.md",
  "probe_stdout": "CONFIRMED - actual: 'foo' | expected: 'bar'\n",
  "trigger_summary": "Input 'foo:bar -> int': qualifier ':' is stripped to get 'bar', but the regex on line 49 searches the
original name and fails, so the fallback incorrectly returns 'foo'."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:40:14.002946Z	8126
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:41:59.630850Z	5440

事件 1 — [llm_75ba04ec2f57465d95353dc0da67795e](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string r. Let s = name.strip(). If s is empty, r = ''. Otherwise, define tail =
(s.rsplit(':', 1)[1].rstrip() if ':' in s else s.rsplit('.', 1)[1].rstrip()) if '.' in s else s. If tail.startswith('operator'):
let rest = tail[8:].rstrip(); if rest.startswith('[]'): r = 'operator[]'; elif rest.startswith('('): r = 'operator()'; elif rest
is either exactly 'new' or consists of 'new' followed by optional whitespace, then '[' , optional whitespace, ']' with nothing
else, then r = 'operator new[]' if '[' in rest else 'operator new'; elif rest is either exactly 'delete' or consists of 'delete'
followed by optional whitespace, then '[' , optional whitespace, ']' with nothing else, then r = 'operator delete[]' if '[' in rest
else 'operator delete'; else: let prefix be the longest initial substring of rest containing only characters from {+, -, *, /, %,
&, |, ^, ~, !, =, <, >, ,}; if prefix is not empty, r = 'operator' + prefix. If r is still unassigned, then examine s: if s ends
with a sequence of word characters (alphanumeric or underscore) that is immediately preceded by either the start of the string,
'::', or '.', r is that word sequence; else if s matches the pattern '(' followed by optional whitespace, '*', optional
whitespace, a word, optional whitespace, ')', r is that word; else if s matches the pattern '*' followed by optional whitespace, a
word, r is that word; else if s starts with a word, r is that word; else r = s."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_75ba04ec2f57465d95353dc0da67795e_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_75ba04ec2f57465d95353dc0da67795e_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_75ba04ec2f57465d95353dc0da67795e_response.txt)

事件 2 — [llm_60d15a7ae6ac4798b5037da4886789fb](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "func -> int",
  "offending_statements": "Line 43: m = re.search(r'(?::^|::|\\.)((\\w+)$', name)\\nLine 45: return m.group(1)",
  "reason": "For input 'func -> int', the code returns 'int' (the last word) but the specification requires returning the first identifier token before trailing type annotations, i.e., 'func'."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_60d15a7ae6ac4798b5037da4886789fb_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_60d15a7ae6ac4798b5037da4886789fb_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_60d15a7ae6ac4798b5037da4886789fb_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-066 — [src](#) - [languages](#) - [codegraph.py](#) - [_fqn_for](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / [codegraph](#); 原源码 [src/languages/codegraph.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a string composed of: (1) the non-empty segments of the `file_path` directory when split by '/', (2) the basename of `file_path` with its last occurrence of '.' replaced by '-' (unmodified when the basename contains no '.'), and (3) name. All components are joined by '::'. The returned string contains at least two '::' separators and no component in the returned string is empty.
- **Actual behavior**: The function returns a string FQN formed by normalizing `file_path` (replacing `os.sep` with '/'), extracting its directory and base name via `os.path.dirname` and `os.path.basename`, then replacing the last '.' in the base name (if any) with '-' to produce a dashed base name. The final result is `:::'.join([comp for comp in dirname(normalized).split('/') if comp] + [dashed_basename, name])`, where `normalized = file_path.replace(os.sep, '/')`, `dirname = os.path.dirname(normalized)`, `base = os.path.basename(normalized)`, and `dashed_basename = base[:last_dot] + '-' + base[last_dot+1:] if (last_dot := base.rfind('.')) > 0 else base`. No side effects occur, and no exceptions are raised for the given valid pre-conditions.
- **Code evidence**: Line 14: `dashed = base[:last_dot] + "-" + base[last_dot + 1:] if last_dot > 0 else base`
- **Trigger condition**: The specification requires that the last occurrence of '.' in the basename is replaced by '-' regardless of its position. The code only performs the replacement when `last_dot > 0`, excluding the case where the dot is the first character (e.g., basename `'.hidden'`). For input `file_path='dir/.hidden'`, `name='func'`, the code returns `dir::.hidden::func` while the spec would require `dir::-hidden::func`, violating the specification.
- **Validator trigger**: When basename starts with '.' (e.g. `'.hidden'`), `last_dot=0` fails the `> 0` guard, so the dot is not replaced with '-' as the spec requires.
- **Probe stdout**: CONFIRMED — actual: `'dir::.hidden::func'` | expected: `'dir::-hidden::func'`

► SPEC sidecar (完整)

```
{
  "signature": "_fqn_for(file_path: str, name: str) -> str",
  "pre_condition": "file_path is a non-empty relative source file path string with '/' separating directory and basename components. name is a non-empty canonicalized function identifier string.",
  "post_condition": "Returns a string composed of: (1) the non-empty segments of the file_path directory when split by '/', (2) the basename of file_path with its last occurrence of '.' replaced by '-' (unmodified when the basename contains no '.'), and (3) name. All components are joined by '::'. The returned string contains at least two '::' separators and no component in the returned string is empty."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/_fqn_for.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a string composed of: (1) the non-empty segments of the file_path directory when split by '/', (2) the
    basename of file_path with its last occurrence of '.' replaced by '-' (unmodified when the basename contains no '.'), and (3)
    name. All components are joined by ':'. The returned string contains at least two ':' separators and no component in the
    returned string is empty.",
    "actual_behavior": "The function returns a string FQN formed by normalizing `file_path` (replacing `os.sep` with '/'),
    extracting its directory and base name via `os.path.dirname` and `os.path.basename`, then replacing the last '.' in the base name
    (if any) with '-' to produce a dashed base name. The final result is `::'.join([comp for comp in dirname(normalized).split('/')
    if comp] + [dashed_basename, name])`, where `normalized = file_path.replace(os.sep, '/')`, `dirname =
    os.path.dirname(normalized)`, `base = os.path.basename(normalized)`, and `dashed_basename = base[:last_dot] + '-' +
    base[last_dot+1:] if (last_dot := base.rfind('.')) > 0 else base`. No side effects occur, and no exceptions are raised for the
    given valid pre-conditions.",
    "code_evidence": "Line 14: dashed = base[:last_dot] + \"-\" + base[last_dot + 1:] if last_dot > 0 else base",
    "trigger_condition": "The specification requires that the last occurrence of '.' in the basename is replaced by '-' regardless
    of its position. The code only performs the replacement when last_dot > 0, excluding the case where the dot is the first character
    (e.g., basename '.hidden'). For input file_path='dir/.hidden', name='func', the code returns 'dir::.hidden::func' while the spec
    would require 'dir::-hidden::func', violating the specification."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--codegraph-py--_fqn_for",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/_fqn_for.py",
  "function_name": "_fqn_for",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--_fqn_for.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--_fqn_for.md",
  "probe_stdout": "CONFIRMED — actual: 'dir::.hidden::func' | expected: 'dir::-hidden::func'",
  "trigger_summary": "When basename starts with '.' (e.g. '.hidden'), last_dot=0 fails the > 0 guard, so the dot is not replaced
  with '-' as the spec requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:27:42.645827Z	1924
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:28:05.721293Z	4705

事件 1 — [llm_393516c9e10e427a9f1c40794339f324](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string FQN formed by normalizing `file_path` (replacing `os.sep` with '/'), extracting
  its directory and base name via `os.path.dirname` and `os.path.basename`, then replacing the last '.' in the base name (if any)
  with '-' to produce a dashed base name. The final result is `::'.join([comp for comp in dirname(normalized).split('/') if comp] +
  [dashed_basename, name])`, where `normalized = file_path.replace(os.sep, '/')`, `dirname = os.path.dirname(normalized)`, `base =
  os.path.basename(normalized)`, and `dashed_basename = base[:last_dot] + '-' + base[last_dot+1:] if (last_dot := base.rfind('.')) >
  0 else base`. No side effects occur, and no exceptions are raised for the given valid pre-conditions."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_393516c9e10e427a9f1c40794339f324_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_393516c9e10e427a9f1c40794339f324_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_393516c9e10e427a9f1c40794339f324_response.txt)

事件 2 — [llm_4ab1d5ae30ee4617839830ca7e25c73c](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "file_path = 'dir/.hidden', name = 'func'",
  "offending_statements": "Line 14: dashed = base[:last_dot] + \"-\" + base[last_dot + 1:] if last_dot > 0 else base",
}
```



```
"reason": "The specification requires that the last occurrence of '.' in the basename is replaced by '-' regardless of its position. The code only performs the replacement when last_dot > 0, excluding the case where the dot is the first character (e.g., basename '.hidden'). For input file_path='dir/.hidden', name='func', the code returns 'dir::.hidden::func' while the spec would require 'dir::-hidden::func', violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4ab1d5ae30ee4617839830ca7e25c73c_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4ab1d5ae30ee4617839830ca7e25c73c_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4ab1d5ae30ee4617839830ca7e25c73c_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: **extraction**

FMA-MISMATCH-071 — **src--extract-py--_extract_func_name_brace**

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 **fm_agent/bug_list.md** 的逐条审计结论: 沿用旧清单的逐条审计结论。
- **Validator**: **confirmed**; attempts 1
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / **extraction**; 原源码 **src/extract.py**
- 证据: **抽取源码** · **SPEC** · **INFO** · **Logic** · **Validator result** · **probe** · **详细报告**

冲突摘要

- **SPEC claim**: Returns the bare function name as a string when signature_text contains a recognizable function declaration, or None when it does not. For declarations matching the pattern of an operator overload 'operator' followed by operator tokens or keywords (e.g., '[]', '()'), 'new[]', arithmetic/bitwise symbols) immediately preceding '(' returns the full operator specifier string. For all other declarations, after removing template/generic angle-bracket regions from signature_text, returns the first word token immediately preceding '(' whose text is not a member of lang_cfg['keywords']. Returns None when no such token exists after keyword filtering.
- **Actual behavior**: The function returns a string representing the extracted function name, or None if no name could be extracted. Specifically:
 - If signature_text contains a substring matching the regular expression **\b(operator\s*(?:\[|\(|\)|\+|-|*|/|%&|^~!=<>]+|new(?:\s*\[|\s*\])?|delete(?:\s*\[|\s*\])?))\s*(**, return the captured group (the operator keyword followed by its overload specification, e.g., 'operator+', 'operator()', 'operator new[]').
 - Otherwise, let cleaned_text be the result of _strip_angle_brackets(signature_text). For each match of the regular expression **\b(\w+)\s*** (in cleaned_text (in left-to-right order), if the captured group (the identifier before the parenthesis) is not a member of lang_cfg['keywords'], return that identifier.
 - If no such identifier is found, return None.

Formally: Let op_pattern = **\b(operator\s*(?:\[|\(|\)|\+|-|*|/|%&|^~!=<>]+|new(?:\s*\[|\s*\])?|delete(?:\s*\[|\s*\])?))\s*(** Let id_pattern = **\b(\w+)\s*** Let keywords = lang_cfg['keywords'] Let remove_brackets(s) = _strip_angle_brackets(s)

If match m = search(op_pattern, signature_text), then return m.group(1). Else let cleaned = remove_brackets(signature_text). If match m in finditer(id_pattern, cleaned) such that m.group(1) keywords, then return the first such m.group(1). Else return None.

- **Code evidence**: Line 4: m = re.search(...) Line 9: if m: Line 10: return m.group(1)
- **Trigger condition**: The code searches for the operator pattern in the entire signature_text before stripping angle brackets. This causes a false match on 'operator+' inside the template parameter list, even though the overall declaration is not an operator overload. The specification requires that operator overload patterns apply only to operator declarations; for all other declarations the name should be extracted after removing template/generic anglebracket regions, yielding 'bar' here.
- **Validator trigger**: operator() inside angle-bracket template parameters is matched by re.search before angle brackets are stripped, causing false-positive operator overload detection
- **Probe stdout**: CONFIRMED — actual: 'operator()' | expected: 'bar'

► SPEC sidecar (完整)

```
{
  "signature": "_extract_func_name_brace(signature_text: str, lang_cfg: dict) -> str | None",
  "pre_condition": "signature_text is a string containing a candidate function signature from a brace-delimited language, possibly spanning concatenated lines. lang_cfg is a dict whose 'keywords' key maps to a collection of language reserved words that must be excluded from returned function names.",
  "post_condition": "Returns the bare function name as a string when signature_text contains a recognizable function declaration, or None when it does not. For declarations matching the pattern of an operator overload – 'operator' followed by operator tokens or keywords (e.g., '[]', '()', 'new[]', arithmetic/bitwise symbols) immediately preceding '(' – returns the full operator specifier string. For all other declarations, after removing template/generic angle-bracket regions from signature_text, returns
```

```
the first word token immediately preceding '(' whose text is not a member of lang_cfg['keywords']. Returns None when no such token exists after keyword filtering."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_strip_angle_brackets",
      "signature": "_strip_angle_brackets(text: str) -> str",
      "pre_condition": "text is a string that may contain angle-bracket-delimited regions such as template or generic parameter lists.",
      "post_condition": "Returns text with each outermost balanced '<...>' region removed: both the delimiting angle brackets and all content between them are stripped. Nesting is resolved – a '<' encountered between an outer '<' and its matching '>' belongs to the inner region and does not terminate the outer region. All characters outside any balanced '<...>' region are preserved in their original relative order."
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/_extract_func_name_brace.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns the bare function name as a string when signature_text contains a recognizable function declaration, or None when it does not. For declarations matching the pattern of an operator overload 'operator' followed by operator tokens or keywords (e.g., '[]', '()', 'new[]', arithmetic/bitwise symbols) immediately preceding '(' returns the full operator specifier string. For all other declarations, after removing template/generic angle-bracket regions from signature_text, returns the first word token immediately preceding '(' whose text is not a member of lang_cfg['keywords']. Returns None when no such token exists after keyword filtering.",
    "actual_behavior": "The function returns a string representing the extracted function name, or None if no name could be extracted. Specifically:\n- If signature_text contains a substring matching the regular expression '\\b(operator\\s*(?:\\[\\]|\\(|\\)|\\+|\\-|\\*|\\/|&|!|=|<|>|new(?:\\s*\\[\\]|\\(|\\)|\\+|\\-|\\*|\\/|&|!|=|<|>)+|delete(?:\\s*\\[\\]|\\(|\\)|\\+|\\-|\\*|\\/|&|!|=|<|>))\\s*\\(', return the captured group (the operator keyword followed by its overload specification, e.g., 'operator+', 'operator()', 'operator new[]').\n- Otherwise, let cleaned_text be the result of _strip_angle_brackets(signature_text). For each match of the regular expression '\\b(\\w+)\\s*\\(' in cleaned_text (in left-to-right order), if the captured group (the identifier before the parenthesis) is not a member of lang_cfg['keywords'], return that identifier.\n- If no such identifier is found, return None.\n\nFormally:\nLet op_pattern = '\\b(operator\\s*(?:\\[\\]|\\(|\\)|\\+|\\-|\\*|\\/|&|!|=|<|>)+|new(?:\\s*\\[\\]|\\(|\\)|\\+|\\-|\\*|\\/|&|!|=|<|>))\\s*\\(' and id_pattern = '\\b(\\w+)\\s*\\('.\nLet keywords = lang_cfg['keywords'].\nLet remove_brackets(s) = _strip_angle_brackets(s).\n\nIf match m = search(op_pattern, signature_text), then return m.group(1).\nElse let cleaned = remove_brackets(signature_text).\nIf match m in finditer(id_pattern, cleaned) such that m.group(1) in keywords, then return the first such m.group(1).\nElse return None.",
    "code_evidence": "Line 4: m = re.search(...)\nLine 9: if m:\nLine 10: return m.group(1)",
    "trigger_condition": "The code searches for the operator pattern in the entire signature_text before stripping angle brackets. This causes a false match on 'operator+' inside the template parameter list, even though the overall declaration is not an operator overload. The specification requires that operator overload patterns apply only to operator declarations; for all other declarations the name should be extracted after removing template/generic anglebracket regions, yielding 'bar' here."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--extract-py--_extract_func_name_brace",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/_extract_func_name_brace.py",
  "function_name": "_extract_func_name_brace",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--extract-py--_extract_func_name_brace.py",
  "detail_file": "fm_agent/bug_validation/src--extract-py--_extract_func_name_brace.md",
  "probe_stdout": "CONFIRMED – actual: 'operator()' | expected: 'bar'\n",
  "trigger_summary": "operator() inside angle-bracket template parameters is matched by re.search before angle brackets are stripped, causing false-positive operator overload detection"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:27:40.136546Z	2345

#	purpose	status	attempt	block	start	tokens
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:28:06.608460Z	9948

事件 1 — llm_f63868ecf26f4c5c86a7d24faebfce0e

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string representing the extracted function name, or None if no name could be
extracted. Specifically:\n- If signature_text contains a substring matching the regular expression ``b(operator\\s*(?:\\[\\]|\\
(\\)|[+\\-*/%&|^~!=<>]+|new(?:\\s*\\[\\s*\\])?|delete(?:\\s*\\[\\s*\\])?)\\s*\\(`, return the captured group (the operator
keyword followed by its overload specification, e.g., 'operator+', 'operator()', 'operator new[]').\n- Otherwise, let cleaned_text
be the result of _strip_angle_brackets(signature_text). For each match of the regular expression ``b(\\w+)\\s*\\(` in
cleaned_text (in left-to-right order), if the captured group (the identifier before the parenthesis) is not a member of
lang_cfg['keywords'], return that identifier.\n- If no such identifier is found, return None.\n\nFormally:\n\nLet op_pattern =
/\\b(operator\\s*(?:\\[\\]|\\(\\)|[+\\-*/%&|^~!=<>]+|new(?:\\s*\\[\\s*\\])?|delete(?:\\s*\\[\\s*\\])?)\\s*\\(/\\nLet id_pattern =
/\\b(\\w+)\\s*\\(/\\nLet keywords = lang_cfg['keywords']\n\nLet remove_brackets(s) = _strip_angle_brackets(s)\n\nIf ∃ match m =
search(op_pattern, signature_text), then return m.group(1).\nElse let cleaned = remove_brackets(signature_text).\nIf ∃ match m in
finditer(id_pattern, cleaned) such that m.group(1) ∉ keywords, then return the first such m.group(1).\nElse return None."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f63868ecf26f4c5c86a7d24faebfce0e_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f63868ecf26f4c5c86a7d24faebfce0e_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f63868ecf26f4c5c86a7d24faebfce0e_response.txt)

事件 2 — llm_008aa78483d5445f9caf4c250f580b2d

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input: signature_text = \"template<class T, T operator+(T,T)> void bar(int)\\", lang_cfg = {\"keywords\":
[\"void\", \"int\\\", \"template\\\"]}. Actual output: \\\"operator+\\\". Expected output according to specification: \\\"bar\\\".",
  "offending_statements": "Line 4: m = re.search(...)\\nLine 9: if m:\\nLine 10: return m.group(1)",
  "reason": "The code searches for the operator pattern in the entire signature_text before stripping angle brackets. This causes
a false match on 'operator+' inside the template parameter list, even though the overall declaration is not an operator overload.
The specification requires that operator overload patterns apply only to operator declarations; for all other declarations the
name should be extracted after removing template/generic angle-bracket regions, yielding 'bar' here."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_008aa78483d5445f9caf4c250f580b2d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_008aa78483d5445f9caf4c250f580b2d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_008aa78483d5445f9caf4c250f580b2d_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-073 — src--extract-py--_extract_functions_indent

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / extraction; 原源码 src/extract.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim：Returns a list of (name, start_idx, end_idx) tuples, one per top-level function definition found in lines. start_idx and end_idx are 0-based inclusive line indices. The span from start_idx to end_idx covers the entire function: any preceding decorator lines, the function definition header (which may span multiple lines), and the function body but excludes trailing blank lines that follow the last non-blank body line. name is the identifier token appearing immediately after the function-definition keyword on the definition line. Functions are returned in the order they appear in lines. Returns an empty list when no function definitions are found.
- Actual behavior：The function returns a list functions of tuples (name, start, end), where each tuple corresponds to a top-level function definition found in lines. A function definition is a line matching the pattern def name((with any indentation). start is the index of the first line of the function, including any preceding decorator lines (lines starting with '@'). end is the index of the last non-blank line of the function body. The body consists of lines after the definition with indentation greater than the definition's indentation, blank lines, and lines that have exactly the definition's indentation but start with a closing parenthesis followed by ';' or '->' (continuation of a multi-line

signature). The body ends just before the first line that is not blank, has indentation less than or equal to the definition's indentation, and is not a signature continuation. Functions are extracted in the order their `def` lines appear, skipping any `def` lines that lie within the body of a previously extracted function (i.e., only outermost functions are extracted). The input `lines` and `lang_cfg` are not modified.

Formally, let `n = len(lines)` and define:

- `is_blank(l)` `l.strip() == ''`
- `is_decorator(l)` `l.strip().startswith('@')`
- `is_func_def(l)` `re.match(r'^(\s*)def\s+(\w+)\s*(?:|>)', l)` is not `None`; if true, let `name(l)` be the captured name and `indent(l) = len(leading_whitespace)`.
- `is_sig_cont(l, ind)` `len(l) - len(l.lstrip()) == ind` and `re.match(r'\s*(:|->)', l.lstrip())` is not `None`.

Then `functions = [(N_0, S_0, E_0), ..., (N_{m-1}, S_{m-1}, E_{m-1})]` for some `m` `0`, with strictly increasing `def` positions `D_0 < D_1 < ... < D_{m-1}` such that:

- `is_func_def(lines[D_k])` holds, `N_k = name(lines[D_k])`, `I_k = indent(lines[D_k])`.
- `S_k` is the smallest index `s` `D_k` where every line in `[s, D_k-1]` is a decorator, and either `s=0` or `lines[s-1]` is not a decorator.
- Let `B_k` be the smallest index `b` `D_k+1` such that `b == n` or `(not is_blank(lines[b]) and indent(lines[b]) < I_k and not is_sig_cont(lines[b], I_k))`. Then `E_k` is the largest index `e` with `D_k < e < B_k` where `e == D_k` or `not is_blank(lines[e])`.
- For each `k`, `D_{k+1} > E_k` (if `k < m-1`), and for any index `d` with `is_func_def(lines[d])` not among `{D_k}`, either `d` lies in some interval `[S_k, E_k]` or `d` would produce a `B_k` `d` but that case is excluded by the scanning order.
- No other tuples are in `functions`.

- Code evidence:** Line 28: `if line_indent == indent and re.match(r'\s*(:|->)', l.lstrip()):`
- Trigger condition:** The code only recognizes a multi-line function signature when a line with the definition's indentation starts with `)` followed by `:` or `->`. When a continuation line at the same indentation does not start with `)`, such as an argument line or the closing line without the prefix `)`, the code incorrectly treats it as the end of the function, truncating the span. In the counterexample, line `'b):'` has indentation `0`, equal to the `def`'s indent, but does not match the signature-continuation regex, so the inner loop breaks, yielding a span of `[0,0]` instead of covering the whole function.
- Validator trigger:** Multi-line function signature continuation lines not starting with `)` cause premature function span termination at line 562 of `src/extract.py`.
- Probe stdout:** CONFIRMED — function body truncated. Extracted source: `'def foo(a,\n'`

► SPEC sidecar (完整)

```
{
  "signature": "_extract_functions_indent(lines: list[str], lang_cfg: dict) -> list[tuple[str, int, int]]",
  "pre_condition": "lines is a list of strings representing source code lines of an indentation-delimited language, each with normalized line endings. lang_cfg is a language configuration dictionary for a language whose body type is 'indent'.",
  "post_condition": "Returns a list of (name, start_idx, end_idx) tuples, one per top-level function definition found in lines. start_idx and end_idx are 0-based inclusive line indices. The span from start_idx to end_idx covers the entire function: any preceding decorator lines, the function definition header (which may span multiple lines), and the function body – but excludes trailing blank lines that follow the last non-blank body line. name is the identifier token appearing immediately after the function-definition keyword on the definition line. Functions are returned in the order they appear in lines. Returns an empty list when no function definitions are found."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/_extract_functions_indent.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a list of (name, start_idx, end_idx) tuples, one per top-level function definition found in lines. start_idx and end_idx are 0-based inclusive line indices. The span from start_idx to end_idx covers the entire function: any preceding decorator lines, the function definition header (which may span multiple lines), and the function body but excludes trailing blank lines that follow the last non-blank body line. name is the identifier token appearing immediately after the"
  }
}
```

```
function-definition keyword on the definition line. Returns an empty list when no function definitions are found.",
    "actual_behavior": "The function returns a list `functions` of tuples `(name, start, end)`, where each tuple corresponds to a top-level function definition found in `lines`. A function definition is a line matching the pattern `def name(` (with any indentation). `start` is the index of the first line of the function, including any preceding decorator lines (lines starting with `@`). `end` is the index of the last non-blank line of the function body. The body consists of lines after the definition with indentation greater than the definition's indentation, blank lines, and lines that have exactly the definition's indentation but start with a closing parenthesis followed by `:` or `->` (continuation of a multi-line signature). The body ends just before the first line that is not blank, has indentation less than or equal to the definition's indentation, and is not a signature continuation. Functions are extracted in the order their `def` lines appear, skipping any `def` lines that lie within the body of a previously extracted function (i.e., only outermost functions are extracted). The input `lines` and `lang_cfg` are not modified.\n\nFormally, let `n = len(lines)` and define:\n- `is_blank(l) = l.strip() == ''`\n- `is_decorator(l) = l.strip().startswith('@')`\n- `is_func_def(l) = re.match(r'^(\s*)def\s+(\w+)\s*(\s*\(|,|l) is not None'; if true, let `name(l)` be the captured name and `indent(l) = len(leading_whitespace)`.\n- `is_sig_cont(l, ind) = len(l) - len(l.lstrip()) == ind` and `re.match(r'\s*(\s*\(|,|l) is not None'.\n\nThen `functions` = `[(N_0, S_0, E_0), ..., (N_{m-1}, S_{m-1}, E_{m-1})]` for some `m ≥ 0`, with strictly increasing `def` positions `D_0 < D_1 < ... < D_{m-1}` such that:\n1. `is_func_def(lines[D_k])` holds, `N_k = name(lines[D_k])`, `I_k = indent(lines[D_k])`.\n2. `S_k` is the smallest index `s ≤ D_k` where every line in `[s, D_k-1]` is a decorator, and either `s=0` or `lines[s-1]` is not a decorator.\n3. Let `B_k` be the smallest index `b ≥ D_{k+1}` such that `b == n` or `(not is_blank(lines[b]))` and `indent(lines[b]) ≤ I_k` and `not is_sig_cont(lines[b], I_k)`. Then `E_k` is the largest index `e` with `D_k ≤ e < B_k` where `e == D_k` or `not is_blank(lines[e])`.\n4. For each `k`, `D_{k+1} > E_k` (if `k < m-1`), and for any index `d` with `is_func_def(lines[d])` not among `{D_k}`, either `d` lies in some interval `[S_k, E_k]` or `d` would produce a `B_k` but that case is excluded by the scanning order.\n5. No other tuples are in `functions`.",
    "code_evidence": "Line 28: if line_indent == indent and re.match(r'\s*(\s*\(|,|l) is not None', l.lstrip()):",
    "trigger_condition": "The code only recognizes a multi-line function signature when a line with the definition's indentation starts with `)` followed by `:` or `->`. When a continuation line at the same indentation does not start with `)`, such as an argument line or the closing line without the prefix `)`, the code incorrectly treats it as the end of the function, truncating the span. In the counterexample, line `b):` has indentation 0, equal to the def's indent, but does not match the signature-continuation regex, so the inner loop breaks, yielding a span of [0,0] instead of covering the whole function."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--extract-py--_extract_functions_indent",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/extract-py/_extract_functions_indent.py",
  "function_name": "_extract_functions_indent",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--extract-py--_extract_functions_indent.py",
  "detail_file": "fm_agent/bug_validation/src--extract-py--_extract_functions_indent.md",
  "probe_stdout": "CONFIRMED - function body truncated. Extracted source: 'def foo(a,\n",
  "trigger_summary": "Multi-line function signature continuation lines not starting with `)` cause premature function span termination at line 562 of src/extract.py."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:10:45.335775Z	4892
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:11:46.016526Z	6502

事件 1 — [llm_3e27da8d4e9d46a9a93e0233410593d8](#)

► parsed_json (完整)

```
{
  "post_condition": "The function returns a list `functions` of tuples `(name, start, end)`, where each tuple corresponds to a top-level function definition found in `lines`. A function definition is a line matching the pattern `def name(` (with any indentation). `start` is the index of the first line of the function, including any preceding decorator lines (lines starting with `@`). `end` is the index of the last non-blank line of the function body. The body consists of lines after the definition with indentation greater than the definition's indentation, blank lines, and lines that have exactly the definition's indentation but start with a closing parenthesis followed by `:` or `->` (continuation of a multi-line signature). The body ends just before the first line that is not blank, has indentation less than or equal to the definition's indentation, and is not a signature continuation. Functions are extracted in the order their `def` lines appear, skipping any `def` lines that lie within the body of a previously extracted function (i.e., only outermost functions are extracted). The input `lines` and `lang_cfg` are not modified.\n\nFormally, let `n = len(lines)` and define:\n- `is_blank(l) = l.strip() == ''`\n- `is_decorator(l) = l.strip().startswith('@')`\n- `is_func_def(l) = re.match(r'^(\s*)def\s+(\w+)\s*(\s*\(|,|l) is not None'; if true, let `name(l)` be the captured name and `indent(l) = len(leading_whitespace)`.\n- `is_sig_cont(l, ind) = len(l) - len(l.lstrip()) == ind` and `re.match(r'\s*(\s*\(|,|l) is not None'.\n\nThen `functions` = `[(N_0, S_0, E_0), ..., (N_{m-1}, S_{m-1}, E_{m-1})]` for some `m ≥ 0`, with strictly increasing `def` positions `D_0 < D_1 < ... < D_{m-1}` such that:\n1. `is_func_def(lines[D_k])` holds, `N_k = name(lines[D_k])`, `I_k = indent(lines[D_k])`.\n2. `S_k` is the smallest index `s ≤ D_k` where every line in `[s, D_k-1]` is a decorator, and either `s=0` or `lines[s-1]` is not a decorator.\n3. Let `B_k` be the smallest index `b ≥ D_{k+1}` such that `b == n` or `(not is_blank(lines[b]))` and `indent(lines[b]) ≤ I_k` and `not is_sig_cont(lines[b], I_k)`. Then `E_k` is the largest index `e` with `D_k ≤ e < B_k` where `e == D_k` or `not is_blank(lines[e])`.\n4. For each `k`, `D_{k+1} > E_k` (if `k < m-
```



```
1'), and for any index `d` with is_func_def(lines[d])` not among `{D_k}`, either `d` lies in some interval `[S_k, E_k)` or `d` would produce a `B_k ≤ d` but that case is excluded by the scanning order.\n5. No other tuples are in `functions`."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_3e27da8d4e9d46a9a93e0233410593d8_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_3e27da8d4e9d46a9a93e0233410593d8_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_3e27da8d4e9d46a9a93e0233410593d8_response.txt)

事件 2 — llm_376d274c0a68424795c32186864702ef

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "lines = [\"def foo(a,\\\", \\\"b):\\\", \\\"    return a + b\\\"]\",
  \"offending_statements\": \"Line 28: if line_indent == indent and re.match(r'\\\\\\\\\\\\s*(|>)', l.lstrip()):\",
  \"reason\": \"The code only recognizes a multi-line function signature when a line with the definition's indentation starts with ')\" followed by ':' or '>'. When a continuation line at the same indentation does not start with ')\", such as an argument line or the closing line without the prefix ')\", the code incorrectly treats it as the end of the function, truncating the span. In the counterexample, line 'b):' has indentation 0, equal to the def's indent, but does not match the signature-continuation regex, so the inner loop breaks, yielding a span of [0,0] instead of covering the whole function.\"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_376d274c0a68424795c32186864702ef_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_376d274c0a68424795c32186864702ef_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_376d274c0a68424795c32186864702ef_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-076 — src--extract-py--_strip_angle_brackets

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / extraction; 原源码 src/extract.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** Returns a string formed from text by removing every outermost balanced '<...>' region: both the delimiting '<' and '>' characters and all content between them are stripped. Nesting is resolved a '<' encountered inside an active '<...>' region belongs to the inner region and does not terminate the outer. Characters that are not part of any balanced '<...>' region are preserved in their original relative order. A '>' character that does not close any active '<...>' region is omitted from the result.
- **Actual behavior:** The function returns a new string that omits all occurrences of '<' and '>', and also omits every character that appears inside a balanced pair of angle brackets (following standard nesting). Formally, define the pre-character depth d(i) for index i (0 ≤ i < len(text)) as d(0)=0, and for i>0: d(i+1) = d(i)+1 if text[i]='<'; d(i+1) = max(0, d(i)-1) if text[i]='>'; d(i+1) = d(i) otherwise. Then the output string is the concatenation of all characters text[i] such that text[i] ∈ {'<','>'} and d(i)=0. Unmatched '<' cause all subsequent characters up to the end of the string (or until a matching '>') to be treated as inside a region and thus removed; any unmatched '>' when depth is zero are also removed.
- **Code evidence:** Line 6: if ch == '<': Line 7: depth += 1
- **Trigger condition:** The code unconditionally removes '<' characters, even when they are unmatched and not part of any balanced region. The specification (Condition B) requires preservation of unmatched '<', as only balanced outermost '<...>' regions (and unmatched '>') should be removed. For the input '<', the code returns "" but the specification requires '<'.
- **Validator trigger:** Input '<' (unmatched angle bracket) returns empty string instead of '<' — the '<' character is unconditionally dropped rather than preserved when unbalanced.
- **Probe stdout:** CONFIRMED — actual: "" | expected: '<'

► SPEC sidecar (完整)

```
{
  "signature": "_strip_angle_brackets(text) -> str",
  "pre_condition": "text is a string that may contain angle-bracket-delimited regions such as template or generic parameter lists.",
  "post_condition": "Returns a string formed from text by removing every outermost balanced '<...>' region: both the delimiting '<' and '>' characters and all content between them are stripped. Nesting is resolved – a '<' encountered inside an active '<...>' region belongs to the inner region and does not terminate the outer. Characters that are not part of any balanced '<...>' region are preserved in their original relative order. A '>' character that does not close any active '<...>' region is omitted from the"
```



```
result."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/_strip_angle_brackets.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a string formed from text by removing every outermost balanced '<...>' region: both the delimiting '<' and '>' characters and all content between them are stripped. Nesting is resolved a '<' encountered inside an active '<...>' region belongs to the inner region and does not terminate the outer. Characters that are not part of any balanced '<...>' region are preserved in their original relative order. A '>' character that does not close any active '<...>' region is omitted from the result.",
    "actual_behavior": "The function returns a new string that omits all occurrences of '<' and '>', and also omits every character that appears inside a balanced pair of angle brackets (following standard nesting). Formally, define the pre-character depth d(i) for index i (0 ≤ i ≤ len(text)) as d(0)=0, and for i>0: d(i+1) = d(i)+1 if text[i]='<'; d(i+1) = max(0, d(i)-1) if text[i]='>'; d(i+1) = d(i) otherwise. Then the output string is the concatenation of all characters text[i] such that text[i] ∈ {'<', '>'} and d(i)=0. Unmatched '<' cause all subsequent characters up to the end of the string (or until a matching '>') to be treated as inside a region and thus removed; any unmatched '>' when depth is zero are also removed.",
    "code_evidence": "Line 6: if ch == '<':\nLine 7: depth += 1",
    "trigger_condition": "The code unconditionally removes '<' characters, even when they are unmatched and not part of any balanced region. The specification (Condition B) requires preservation of unmatched '<', as only balanced outermost '<...>' regions (and unmatched '>') should be removed. For the input '<', the code returns '' but the specification requires '<'."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--extract-py--_strip_angle_brackets",
  "source_file": "fm_agent/extracted_functions/src/extract-py/_strip_angle_brackets.py",
  "function_name": "_strip_angle_brackets",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--extract-py--_strip_angle_brackets.py",
  "detail_file": "fm_agent/bug_validation/src--extract-py--_strip_angle_brackets.md",
  "probe_stdout": "CONFIRMED - actual: '' | expected: '<\\n'",
  "trigger_summary": "Input '<' (unmatched angle bracket) returns empty string instead of '<' - the '<' character is unconditionally dropped rather than preserved when unbalanced."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:40:12.173233Z	2954
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:41:01.106134Z	3650

事件 1 — [llm_b5e785664e2d4205b6a4416d453edff9](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a new string that omits all occurrences of '<' and '>', and also omits every character that appears inside a balanced pair of angle brackets (following standard nesting). Formally, define the pre-character depth d(i) for index i (0 ≤ i ≤ len(text)) as d(0)=0, and for i≥0: d(i+1) = d(i)+1 if text[i]='<'; d(i+1) = max(0, d(i)-1) if text[i]='>'; d(i+1) = d(i) otherwise. Then the output string is the concatenation of all characters text[i] such that text[i] ∉ {'<', '>'} and d(i)=0. Unmatched '<' cause all subsequent characters up to the end of the string (or until a matching '>') to be treated as inside a region and thus removed; any unmatched '>' when depth is zero are also removed."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b5e785664e2d4205b6a4416d453edff9_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_b5e785664e2d4205b6a4416d453edff9_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_b5e785664e2d4205b6a4416d453edff9_response.txt)

事件 2 — llm_fd5bafbbd20c47eb879e553e1c1046e8

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "<",
  "offending_statements": "Line 6: if ch == '<':\nLine 7: depth += 1",
  "reason": "The code unconditionally removes '<' characters, even when they are unmatched and not part of any balanced region. The specification (Condition B) requires preservation of unmatched '<', as only balanced outermost '<...>' regions (and unmatched '>') should be removed. For the input '<', the code returns '' but the specification requires '<'."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_fd5bafbbd20c47eb879e553e1c1046e8_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_fd5bafbbd20c47eb879e553e1c1046e8_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_fd5bafbbd20c47eb879e553e1c1046e8_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-077 — src--extract-py--extract_functions_from_file

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: **confirmed**; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / **extraction**; 原源码 [src/extract.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a list of (func_name, source_text) tuples, one per function found in the file via language-specific extraction rules. Each func_name is canonicalized: unsafe characters are translated and duplicates in the file are deduplicated by appending a numeric suffix in order of first occurrence. source_text includes the full function body from start to end inclusive, line endings normalized to '\n', and the body ends with a trailing newline. Functions appear in source order. Returns an empty list when the language has no file-local regex extraction capability.
- **Actual behavior**: On normal execution, the file is read and closed without side effects; the return value is a list of (deduped_name, source_text) tuples. If lang_cfg["body"] is neither "brace" nor "indent", the list is empty. Otherwise, for each top-level function detected by languagespecific extraction (brace matching or indentation), a tuple is included. deduped_name is the canonicalized form of the functions raw name (with '/' translated to ') plus a numeric suffix '{count}' when the same canonical name appears more than once, counting from 1 for the second occurrence. source_text contains the joined source lines of that function (from start index inclusive to end index inclusive, with lines separated by newlines and a trailing newline). The order of tuples matches the order of functions in the file. Formally, let lines = [l.rstrip('\n').rstrip('\r') for l in readlines(filepath)]; let lang_cfg = LANG_CONFIG[lang_key]. If lang_cfg["body"] {"brace", "indent"}: result = []. Else let P = (if lang_cfg["body"] = "brace" then _extract_functions_brace(lines, lang_key, lang_cfg) else extract_functions_indent(lines, lang_cfg)), where P is a list of (name_i, start_i, end_i) for i = 0..m-1. For each i, let cname_i = canonicalize(name_i) and count_i = |{j < i : canonicalize(P[j][0]) = cname_i}|. Then deduped_name_i = cname_i if count_i = 0 else cname_i + " + count_i; source_i = ''.join(lines[start_i .. end_i]) + '\n'; and result = [(deduped_name_i, source_i) | i = 0..m-1].
- **Code evidence**: Line 9: lines = [l.rstrip('\n').rstrip('\r') for l in lines]
- **Trigger condition**: The code strips trailing carriage returns from every line after removing newlines, which removes a literal \r that is part of the source content (e.g., inside a string). This corrupts the function body, violating the specification's requirement that the source text include the full function body with only line endings normalized.
- **Validator trigger**: Literal CR byte (0x0D) inside source content is treated as line ending and stripped, corrupting the extracted function body.
- **Probe stdout**: CONFIRMED — CR byte stripped from function body. actual: 'def func_with_cr():\n return "prefix\n'

► SPEC sidecar (完整)

```
{
  "signature": "extract_functions_from_file(filepath, lang_key) -> list[tuple[str, str]]",
  "pre_condition": "filepath refers to an existing, readable file. lang_key is a language key present in LANG_CONFIG.",
  "post_condition": "Returns a list of (func_name, source_text) tuples, one per function found in the file via language-specific extraction rules. Each func_name is canonicalized: unsafe characters are translated and duplicates in the file are deduplicated by
```

```
appending a numeric suffix in order of first occurrence. source_text includes the full function body from start to end inclusive, line endings normalized to '\n', and the body ends with a trailing newline. Functions appear in source order. Returns an empty list when the language has no file-local regex extraction capability."
}
```

► INFO / callees sidebar (完整)

```
{
  "callees": [
    {
      "name": "_extract_functions_brace",
      "signature": "_extract_functions_brace(lines: list[str], lang_key: str, lang_cfg: dict) -> list[tuple[str, int, int]]",
      "pre_condition": "lines is a non-empty list of strings with normalized line endings. lang_key is a recognized language key. lang_cfg is the LANG_CONFIG entry for that language with body type 'brace'.",
      "post_condition": "Returns a list of (func_name, start_index, end_index) tuples identifying each top-level function in the file by brace matching, using language-specific keyword and prefix filtering from lang_cfg. Indices are 0-based and inclusive."
    },
    {
      "name": "_extract_functions_indent",
      "signature": "_extract_functions_indent(lines: list[str], lang_cfg: dict) -> list[tuple[str, int, int]]",
      "pre_condition": "lines is a non-empty list of strings with normalized line endings. lang_cfg is the LANG_CONFIG entry for a language with body type 'indent'.",
      "post_condition": "Returns a list of (func_name, start_index, end_index) tuples identifying each top-level function in the file by indentation boundaries, using language-specific keyword and prefix filtering from lang_cfg. Indices are 0-based and inclusive."
    },
    {
      "name": "canonicalize",
      "signature": "canonicalize(func_name: str) -> str",
      "pre_condition": "func_name is a raw function name string that may contain characters unsafe for filesystem paths or FQN separators.",
      "post_condition": "Returns a filesystem-safe, FQN-compatible string with '/' characters translated to '_'. The result is suitable for use in file paths and FQN components."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/extract_functions_from_file.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a list of (func_name, source_text) tuples, one per function found in the file via language-specific extraction rules. Each func_name is canonicalized: unsafe characters are translated and duplicates in the file are deduplicated by appending a numeric suffix in order of first occurrence. source_text includes the full function body from start to end inclusive, line endings normalized to '\n', and the body ends with a trailing newline. Functions appear in source order. Returns an empty list when the language has no file-local regex extraction capability.",
    "actual_behavior": "On normal execution, the file is read and closed without side effects; the return value is a list of (deduped_name, source_text) tuples. If lang_cfg['body'] is neither \"brace\" nor \"indent\", the list is empty. Otherwise, for each top-level function detected by languagespecific extraction (brace matching or indentation), a tuple is included. deduped_name is the canonicalized form of the functions raw name (with '/' translated to '_') plus a numeric suffix '_{count}' when the same canonical name appears more than once, counting from 1 for the second occurrence. source_text contains the joined source lines of that function (from start index inclusive to end index inclusive, with lines separated by newlines and a trailing newline). The order of tuples matches the order of functions in the file. Formally, let lines = [l.rstrip('\n').rstrip('\r') for l in readlines(filepath)]; let lang_cfg = LANG_CONFIG[lang_key]. If lang_cfg['body'] {\"brace\", \"indent\"}: result = []. Else let P = (if lang_cfg['body'] = \"brace\" then _extract_functions_brace(lines, lang_key, lang_cfg) else _extract_functions_indent(lines, lang_cfg)), where P is a list of (name_i, start_i, end_i) for i = 0..m-1. For each i, let cname_i = canonicalize(name_i) and count_i = |{j < i : canonicalize(P[j][0]) = cname_i}|. Then deduped_name_i = cname_i if count_i = 0 else cname_i + '_' + count_i; source_i = '\n'.join(lines[start_i .. end_i]) + '\n'; and result = [(deduped_name_i, source_i) | i = 0..m-1].",
    "code_evidence": "Line 9: lines = [l.rstrip('\n').rstrip('\r') for l in lines]",
    "trigger_condition": "The code strips trailing carriage returns from every line after removing newlines, which removes a literal \r that is part of the source content (e.g., inside a string). This corrupts the function body, violating the specification's requirement that the source text include the full function body with only line endings normalized."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--extract-py--extract_functions_from_file",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/extract_functions_from_file.py",
  "function_name": "extract_functions_from_file",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--extract-py--extract_functions_from_file.py",
  "detail_file": "fm_agent/bug_validation/src--extract-py--extract_functions_from_file.md",
  "probe_stdout": "CONFIRMED - CR byte stripped from function body. actual: 'def func_with_cr():\n    return \"prefix\\n\"',"
```

```
"trigger_summary": "Literal CR byte (0x0D) inside source content is treated as line ending and stripped, corrupting the
extracted function body."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:04:41.014808Z	3494
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:05:18.751119Z	7255

事件 1 — [llm_c05ba7db55c640aba1890d9b4e83b81a](#)

► parsed_json（完整）

```
{
  "post_condition": "On normal execution, the file is read and closed without side effects; the return value is a list of
(deduped_name, source_text) tuples. If lang_cfg[\"body\"] is neither \"brace\" nor \"indent\", the list is empty. Otherwise, for
each top-level function detected by language-specific extraction (brace matching or indentation), a tuple is included.
deduped_name is the canonicalized form of the function's raw name (with '/' translated to '_') plus a numeric suffix '_{count}'
when the same canonical name appears more than once, counting from 1 for the second occurrence. source_text contains the joined
source lines of that function (from start index inclusive to end index inclusive, with lines separated by newlines and a trailing
newline). The order of tuples matches the order of functions in the file. Formally, let lines = [l.rstrip('\\n').rstrip('\\r') for
l in readlines(filepath)]; let lang_cfg = LANG_CONFIG[lang_key]. If lang_cfg[\"body\"] \u2208 {\"brace\", \"indent\"}: result = [].
Else let P = (if lang_cfg[\"body\"] = \"brace\" then _extract_functions_brace(lines, lang_key, lang_cfg) else
_extract_functions_indent(lines, lang_cfg)), where P is a list of (name_i, start_i, end_i) for i = 0..m-1. For each i, let cname_i
= canonicalize(name_i) and count_i = |\{j < i : canonicalize(P[j][0]) = cname_i\}|. Then deduped_name_i = cname_i if count_i = 0
else cname_i + '_' + count_i; source_i = '\\n'.join(lines[start_i .. end_i]) + '\\n'; and result = [(deduped_name_i, source_i) | i =
0..m-1].\"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c05ba7db55c640aba1890d9b4e83b81a_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c05ba7db55c640aba1890d9b4e83b81a_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c05ba7db55c640aba1890d9b4e83b81a_response.txt)

事件 2 — [llm_d0f6a08847f94dd8b45a00f52b4d2975](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "A Python source file whose content includes a line with a trailing carriage return inside a string literal,
e.g.:\\n\\ndef foo():\\n    print(\"hello\\r\")\\n\\n(where the line literally ends with \\r before the newline). The language key
corresponds to a language using indent-based extraction (e.g., 'python').\",
  "offending_statements": \"Line 9: lines = [l.rstrip('\\n').rstrip('\\r') for l in lines]\",
  "reason": \"The code strips trailing carriage returns from every line after removing newlines, which removes a literal \\r that
is part of the source content (e.g., inside a string). This corrupts the function body, violating the specification's requirement
that the source text include the full function body with only line endings normalized.\"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d0f6a08847f94dd8b45a00f52b4d2975_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d0f6a08847f94dd8b45a00f52b4d2975_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d0f6a08847f94dd8b45a00f52b4d2975_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: language_handlers

FMA-MISMATCH-097 — [src--languages--erlang.py--elp_argv](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / language_handlers；原源码 src/languages/erlang.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

- **SPEC claim:** Returns a non-empty list of strings whose final element is "server", suitable for invoking ELP as a subprocess that reads JSON-RPC requests from standard input and writes JSON-RPC responses to standard output. The preceding elements form a platform-appropriate invocation of the configured ELP binary. The return value is stable: two calls return equal lists if the ELP backend configuration has not changed, and return unequal lists if the ELP backend has changed in a way that could produce different analysis output for the same input project.
- **Actual behavior:** The function returns a list of strings representing the argument vector to launch the ELP server. Let `cmd = settings.erlang.command.strip()` and `tokens = shlex.split(cmd, posix=(os.name != 'nt'))`. If `tokens` is empty, let `base = ['elp']`; otherwise `base = tokens`. The returned list is `base + ['server']`. Consequently, the result is never empty, always has 'server' as its last element, and its first element is either the stripped nonempty content of `settings.erlang.command` split into tokens or 'elp' if no valid command was configured.
- **Code evidence:** Line 6: `return [*argv, "server"]`
- **Trigger condition:** The returned list becomes `['elp', 'server', 'server']`, which does not form a platform-appropriate invocation because the preceding elements now contain a duplicate 'server' that could confuse ELP. The specification requires that the arguments before the final 'server' constitute a correct invocation; the duplicate 'server' violates this requirement.
- **Validator trigger:** When `settings.erlang.command` is 'elp server' (or any command already containing 'server'), the unconditional append produces a duplicate 'server' in the argument vector.
- **Probe stdout:** CONFIRMED — `_elp_argv()` returns a list with duplicate 'server': actual: `['elp', 'server', 'server']` | expected (no duplicate): `['elp', 'server']`

► SPEC sidecar (完整)

```
{
  "signature": "_elp_argv() -> list[str]",
  "pre_condition": "The ELP backend binary exists at the configured location or is discoverable via the system path.",
  "post_condition": "Returns a non-empty list of strings whose final element is \"server\", suitable for invoking ELP as a
subprocess that reads JSON-RPC requests from standard input and writes JSON-RPC responses to standard output. The preceding
elements form a platform-appropriate invocation of the configured ELP binary. The return value is stable: two calls return equal
lists if the ELP backend configuration has not changed, and return unequal lists if the ELP backend has changed in a way that
could produce different analysis output for the same input project."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_elp_argv.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a non-empty list of strings whose final element is \"server\", suitable for invoking ELP as a
subprocess that reads JSON-RPC requests from standard input and writes JSON-RPC responses to standard output. The preceding
elements form a platform-appropriate invocation of the configured ELP binary. The return value is stable: two calls return equal
lists if the ELP backend configuration has not changed, and return unequal lists if the ELP backend has changed in a way that
could produce different analysis output for the same input project.",
    "actual_behavior": "The function returns a list of strings representing the argument vector to launch the ELP server. Let `cmd
= settings.erlang.command.strip()` and `tokens = shlex.split(cmd, posix=(os.name != 'nt'))`. If `tokens` is empty, let `base =
['elp']`; otherwise `base = tokens`. The returned list is `base + ['server']`. Consequently, the result is never empty, always has
`server` as its last element, and its first element is either the stripped nonempty content of `settings.erlang.command` split
into tokens or `elp` if no valid command was configured.",
    "code_evidence": "Line 6: return [*argv, \"server\"]",
    "trigger_condition": "The returned list becomes ['elp', 'server', 'server'], which does not form a platform-appropriate
invocation because the preceding elements now contain a duplicate 'server' that could confuse ELP. The specification requires that
the arguments before the final 'server' constitute a correct invocation; the duplicate 'server' violates this requirement."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--_elp_argv",
  "source_file": "src/languages/erlang.py",
}
```

```
    "function_name": "_elp_argv",
    "confirmation_status": "confirmed",
    "attempts": 1,
    "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_elp_argv.py",
    "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_elp_argv.md",
    "probe_stdout": "CONFIRMED - _elp_argv() returns a list with duplicate 'server': actual: ['elp', 'server', 'server'] | expected (no duplicate): ['elp', 'server']",
    "trigger_summary": "When settings.erlang.command is 'elp server' (or any command already containing 'server'), the unconditional append produces a duplicate 'server' in the argument vector."
  }
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:27:04.054694Z	1261
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:27:20.774869Z	4907

事件 1 — [llm_72c4938f0da547428ef5f0631328abe6](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a list of strings representing the argument vector to launch the ELP server. Let `cmd = settings.erlang.command.strip()` and `tokens = shlex.split(cmd, posix=(os.name != 'nt'))`. If `tokens` is empty, let `base = ['elp']`; otherwise `base = tokens`. The returned list is `base + ['server']`. Consequently, the result is never empty, always has `server` as its last element, and its first element is either the stripped non-empty content of `settings.erlang.command` split into tokens or `elp` if no valid command was configured."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_72c4938f0da547428ef5f0631328abe6_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_72c4938f0da547428ef5f0631328abe6_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_72c4938f0da547428ef5f0631328abe6_response.txt)

事件 2 — [llm_d1505f4ad70848ab9bc33f0fe6408870](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "When the configuration string contains the word 'server' as a separate argument, e.g., `settings.erlang.command = 'elp server'` on a POSIX system.",
  "offending_statements": "Line 6: return [*argv, \"server\"]",
  "reason": "The returned list becomes ['elp', 'server', 'server'], which does not form a platform-appropriate invocation because the preceding elements now contain a duplicate 'server' that could confuse ELP. The specification requires that the arguments before the final 'server' constitute a correct invocation; the duplicate 'server' violates this requirement."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d1505f4ad70848ab9bc33f0fe6408870_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d1505f4ad70848ab9bc33f0fe6408870_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d1505f4ad70848ab9bc33f0fe6408870_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-098 — [src--languages--erlang-py--_escape_component](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / language_handlers; 源代码 src/languages/erlang.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- SPEC claim: Returns a string where every character in value that is an ASCII alphanumeric (a-z, A-Z, 0-9) or underscore is preserved at its original relative position, and every other character is replaced by an underscore followed by its Unicode code point expressed as a

zero-padded two-digit hexadecimal number. The output is deterministic: identical input strings always produce identical output strings.

Every character in the output belongs to the set [a-zA-Z0-9_].

- **Actual behavior:** The function returns a string obtained by iterating over each character `c` in `value` (in order) and appending either `c` itself (if `c` is an ASCII character and either `c.isalnum()` is `True` or `c` equals `'_'`) or the string `_` followed by the lowercase zeropadded (minimum width 2) hexadecimal representation of `ord(c)`. Formally: let `process(c) = c` if `c.isascii()` and `(c.isalnum() or c == '_')` else `'_' + f'{ord(c):02x}'`. Then the returned string equals `''.join(process(c) for c in value)`. The output is always nonempty because at least one character contributes at least one character.
- **Code evidence:** Line 7: `result.append(f'_{ord(char):02x}')`
- **Trigger condition:** For the input string `"(U+03C0)`, the code returns `'_3c0'`, which uses three hexadecimal digits. The specification requires an underscore followed by a zero-padded two-digit hexadecimal number for every non-alphanumeric-underscore character. Because `0x3C0` needs three digits, the output violates the 'two-digit' requirement.
- **Validator trigger:** Input `'π'` (`U+03C0`, `ord 960=0x3C0`) produces `'_3c0'` with 3 hex digits, but the spec requires exactly 2 zero-padded hex digits.
- **Probe stdout:** CONFIRMED — actual: `'_3c0'` | expected (spec two-digit): `'_c0'` | hex digits in output: 3 (spec requires exactly 2)

► SPEC sidecar (完整)

```
{
  "signature": "_escape_component(value: str) -> str",
  "pre_condition": "value is a non-empty string.",
  "post_condition": "Returns a string where every character in value that is an ASCII alphanumeric (a-z, A-Z, 0-9) or underscore is preserved at its original relative position, and every other character is replaced by an underscore followed by its Unicode code point expressed as a zero-padded two-digit hexadecimal number. The output is deterministic: identical input strings always produce identical output strings. Every character in the output belongs to the set [a-zA-Z0-9_]."
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_escape_component.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a string where every character in value that is an ASCII alphanumeric (a-z, A-Z, 0-9) or underscore is preserved at its original relative position, and every other character is replaced by an underscore followed by its Unicode code point expressed as a zero-padded two-digit hexadecimal number. The output is deterministic: identical input strings always produce identical output strings. Every character in the output belongs to the set [a-zA-Z0-9_].",
    "actual_behavior": "The function returns a string obtained by iterating over each character `c` in `value` (in order) and appending either `c` itself (if `c` is an ASCII character and either `c.isalnum()` is `True` or `c` equals `_`) or the string _ followed by the lowercase zeropadded (minimum width 2) hexadecimal representation of `ord(c)`. Formally: let `process(c) = c` if `c.isascii()` and `(c.isalnum() or c == '_')` else `_` + f'{ord(c):02x}'. Then the returned string equals `''.join(process(c) for c in value)`. The output is always nonempty because at least one character contributes at least one character.",
    "code_evidence": "Line 7: result.append(f'_{ord(char):02x}'))",
    "trigger_condition": "For the input string '(U+03C0), the code returns '_3c0', which uses three hexadecimal digits. The specification requires an underscore followed by a zero-padded two-digit hexadecimal number for every non-alphanumeric-underscore character. Because 0x3C0 needs three digits, the output violates the 'two-digit' requirement."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--_escape_component",
  "source_file": "src/languages/erlang.py",
  "function_name": "_escape_component",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_escape_component.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_escape_component.md",
  "probe_stdout": "CONFIRMED - actual: '_3c0' | expected (spec two-digit): '_c0' | hex digits in output: 3 (spec requires exactly 2)\n",
  "trigger_summary": "Input 'π' (U+03C0, ord 960=0x3C0) produces '_3c0' with 3 hex digits, but the spec requires exactly 2 zero-padded hex digits."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:39:08.871476Z	1566
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:39:34.308838Z	3201

事件 1 — [llm_bef5cf913de2469cb7140325cc4821d6](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string obtained by iterating over each character `c` in `value` (in order) and appending either `c` itself (if `c` is an ASCII character and either `c.isalnum()` is `True` or `c` equals `_`) or the string `\"_\"` followed by the lowercase zero-padded (minimum width 2) hexadecimal representation of `ord(c)`. Formally: let `process(c) = c` if `c.isascii()` and `(c.isalnum() or c == '_')` else `\"_\" + f'{ord(c):02x}`. Then the returned string equals `\"\".join(process(c) for c in value)`. The output is always non-empty because at least one character contributes at least one character."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_bef5cf913de2469cb7140325cc4821d6_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_bef5cf913de2469cb7140325cc4821d6_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_bef5cf913de2469cb7140325cc4821d6_response.txt)

事件 2 — [llm_36d7fee00d1b42ff86689555a63107d4](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "π",
  "offending_statements": "Line 7: result.append(f\"_{ord(char):02x}\")",
  "reason": "For the input string 'π' (U+03C0), the code returns '_3c0', which uses three hexadecimal digits. The specification requires an underscore followed by a zero-padded two-digit hexadecimal number for every non-alphanumeric-underscore character. Because 0x3C0 needs three digits, the output violates the 'two-digit' requirement."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_36d7fee00d1b42ff86689555a63107d4_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_36d7fee00d1b42ff86689555a63107d4_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_36d7fee00d1b42ff86689555a63107d4_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-099 — [src--languages--erlang-py--_function_id](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论；沿用旧清单的逐条审计结论。
- Validator：confirmed; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / language_handlers; 原源码 src/languages/erlang.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

- SPEC claim:** Returns a canonical function identifier string formed from the module identifier derived from uri, the unqualified function name extracted from label, and the arity extracted from label, joined with double underscores in the form . Each component (,) has any characters that would conflict with the double-underscore separator deterministically replaced. The unqualified function name is the portion of label before the final forward slash; when that portion contains a colon and does not begin with a single quote character, only the substring after the last colon is used as the unqualified name. Raises ValueError when label is not a string, label does not contain a forward slash separator, or the portion of label after the final forward slash is not parseable as a non-negative integer.
- **Actual behavior:** If the label string contains a slash '/' and the substring after the last slash is a valid nonnegative integer, the function returns a string formed by (1) computing a module identifier from uri via `_module_from_uri(uri)`, (2) extracting a function name from the part before the last slash: if that part contains a colon ':' and does not start with a single quote, the function name is taken as the substring after the last colon; otherwise it is the entire part before the slash, (3) applying `_escape_component` to the module identifier and to the function name, and (4) concatenating them with double underscores and the integer arity: `f"{escaped_module}__{escaped_name}__{arity}"`. If the label does not contain a slash or the substring after the slash cannot be converted to a nonnegative integer, a `ValueError` is raised with the message `"ELP function label has no valid arity: {label!r}"`, chained from the original exception. The functions `_module_from_uri` and `_escape_component` are assumed to always produce a deterministic result under the given preconditions; any exceptions they raise propagate directly. Formally, for all `uri`, `label` satisfying the precondition: (name, arity_s : label = name + '/' + arity_s arity_s) result = concat(_escape_component(_module_from_uri(uri)), "", _escape_component(name_without_module), "", arity_s) where name_without_module = if (':' name name[0] "") then rsplit(name, ':', 1)[1] else name; otherwise the function raises `ValueError` with the stated message.
 - **Code evidence:** Line 4: `int(arity)`
 - **Trigger condition:** The code does not validate that the arity substring represents a non-negative integer. `int('-1')` succeeds, so `label='foo/-1'` returns a string containing `'-1'` instead of raising `ValueError` as required. Additionally, the code does not handle the case where label is not a string (e.g., `label=42`) and raises `AttributeError` instead of `ValueError`.
 - **Validator trigger:** Negative arity passes `int()` check unchecked; non-string label raises `AttributeError` instead of `ValueError`
 - **Probe stdout:** `[negative_arity] CONFIRMED` — Negative arity not rejected. `label='foo/-1'` returned `'module__foo__-1'` instead of raising `ValueError`
`[non_string_label] CONFIRMED` — Non-string label raises `AttributeError` instead of `ValueError` (spec requires `ValueError`)
`CONFIRMED` — All bug aspects reproduced
- SPEC sidecar (完整)

```
{
  "signature": "_function_id(uri: str, label: str) -> str",
  "pre_condition": "uri is a non-empty string representing an absolute file path for an Erlang source file. label is a non-empty string from an ELP symbol that encodes a function name and an arity, where the arity follows the final forward slash and is parseable as a non-negative integer.",
  "post_condition": "Returns a canonical function identifier string formed from the module identifier derived from uri, the unqualified function name extracted from label, and the arity extracted from label, joined with double underscores in the form <module>__<name>__<arity>. Each component (<module>, <name>) has any characters that would conflict with the double-underscore separator deterministically replaced. The unqualified function name is the portion of label before the final forward slash; when that portion contains a colon and does not begin with a single quote character, only the substring after the last colon is used as the unqualified name. Raises ValueError when label is not a string, label does not contain a forward slash separator, or the portion of label after the final forward slash is not parseable as a non-negative integer."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_module_from_uri",
      "signature": "_module_from_uri(uri: str) -> str",
      "pre_condition": "uri is a non-empty string representing an absolute file path for an Erlang source file.",
      "post_condition": "Returns a deterministic string identifier for the module containing the source file at uri. The same uri always produces the same module identifier."
    },
    {
      "name": "_escape_component",
      "signature": "_escape_component(component: str) -> str",
      "pre_condition": "component is a non-empty string (a module identifier or a function name).",
      "post_condition": "Returns a transformed version of component where any characters that would clash with the double-underscore separator in the function identifier format are deterministically replaced. Identical component values always produce identical results."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_function_id.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a canonical function identifier string formed from the module identifier derived from uri, the
unqualified function name extracted from label, and the arity extracted from label, joined with double underscores in the form
<module>__<name>__<arity>. Each component (<module>, <name>) has any characters that would conflict with the double-underscore
separator deterministically replaced. The unqualified function name is the portion of label before the final forward slash; when
that portion contains a colon and does not begin with a single quote character, only the substring after the last colon is used as
the unqualified name. Raises ValueError when label is not a string, label does not contain a forward slash separator, or the
portion of label after the final forward slash is not parseable as a non-negative integer.",
    "actual_behavior": "If the label string contains a slash '/' and the substring after the last slash is a valid nonnegative
integer, the function returns a string formed by (1) computing a module identifier from uri via `_module_from_uri(uri)`, (2)
extracting a function name from the part before the last slash: if that part contains a colon ':' and does not start with a single
quote, the function name is taken as the substring after the last colon; otherwise it is the entire part before the slash, (3)
applying `_escape_component` to the module identifier and to the function name, and (4) concatenating them with double underscores
and the integer arity: `f\"{escaped_module}__{escaped_name}__{arity}\"`. If the label does not contain a slash or the substring
after the slash cannot be converted to a nonnegative integer, a `ValueError` is raised with the message `\"ELP function label has
no valid arity: {label!r}\"`, chained from the original exception. The functions `_module_from_uri` and `_escape_component` are
assumed to always produce a deterministic result under the given preconditions; any exceptions they raise propagate directly.
Formally, for all `uri`, `label` satisfying the precondition: ( name, arity_s : label = name + '/' + arity_s  arity_s ) result =
concat(_escape_component(_module_from_uri(uri)), \"__\", _escape_component(name_without_module), \"__\", arity_s) where
name_without_module = if (':' in name  name[0] != '') then rsplit(name, ':', 1)[1] else name; otherwise the function raises
ValueError with the stated message.",
    "code_evidence": "Line 4: int(arity)",
    "trigger_condition": "The code does not validate that the arity substring represents a non-negative integer. int('-1')
succeeds, so label='foo/-1' returns a string containing '__-1__' instead of raising ValueError as required. Additionally, the code
does not handle the case where label is not a string (e.g., label=42) and raises AttributeError instead of ValueError."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--_function_id",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/languages/erlang.py",
  "function_name": "_function_id",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_function_id.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_function_id.md",
  "probe_stdout": "[negative_arity] CONFIRMED – Negative arity not rejected. label='foo/-1' returned 'module__foo__-1' instead of
raising ValueError\\n[non_string_label] CONFIRMED – Non-string label raises AttributeError instead of ValueError (spec requires
ValueError)\\n\\nCONFIRMED – All bug aspects reproduced\\n",
  "trigger_summary": "Negative arity passes int() check unchecked; non-string label raises AttributeError instead of ValueError"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:27:22.086677Z	2215
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:27:52.051626Z	3916

事件 1 — [llm_570efaec3d5548ff9503cc01167504de](#)

► parsed_json（完整）

```
{
  "post_condition": "If the label string contains a slash '/' and the substring after the last slash is a valid non-negative
integer, the function returns a string formed by (1) computing a module identifier from uri via `_module_from_uri(uri)`, (2)
extracting a function name from the part before the last slash: if that part contains a colon ':' and does not start with a single
quote, the function name is taken as the substring after the last colon; otherwise it is the entire part before the slash, (3)
applying `_escape_component` to the module identifier and to the function name, and (4) concatenating them with double underscores
and the integer arity: `f\"{escaped_module}__{escaped_name}__{arity}\"`. If the label does not contain a slash or the substring
after the slash cannot be converted to a non-negative integer, a `ValueError` is raised with the message `\"ELP function label has
no valid arity: {label!r}\"`, chained from the original exception. The functions `_module_from_uri` and `_escape_component` are
assumed to always produce a deterministic result under the given pre-conditions; any exceptions they raise propagate directly.
Formally, for all `uri`, `label` satisfying the pre-condition: (∃ name, arity_s : label = name + '/' + arity_s ∧ arity_s ∈ ℕ) ⇒
result = concat(_escape_component(_module_from_uri(uri)), \"__\", _escape_component(name_without_module), \"__\", arity_s) where
name_without_module = if (':' ∈ name ∧ name[0] ≠ '') then rsplit(name, ':', 1)[1] else name; otherwise the function raises
ValueError with the stated message."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_570efaec3d5548ff9503cc01167504de_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_570efaec3d5548ff9503cc01167504de_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_570efaec3d5548ff9503cc01167504de_response.txt)

事件 2 — llm_9299b3156a8f44dfa01dcd2e085ef876

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "uri='dummy.erl', label='foo/-1'",
  "offending_statements": "Line 4: int(arity)",
  "reason": "The code does not validate that the arity substring represents a non-negative integer. int('-1') succeeds, so label='foo/-1' returns a string containing '__-1__' instead of raising ValueError as required. Additionally, the code does not handle the case where label is not a string (e.g., label=42) and raises AttributeError instead of ValueError."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_9299b3156a8f44dfa01dcd2e085ef876_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_9299b3156a8f44dfa01dcd2e085ef876_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_9299b3156a8f44dfa01dcd2e085ef876_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-102 — src--languages--erlang-py--_project_fingerprint

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / language_handlers; 源代码 src/languages/erlang.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** Returns a tuple that deterministically and uniquely identifies the current state of the Erlang project located at proj_dir. Two calls to _project_fingerprint with the same proj_dir return equal tuples if and only if no file relevant to Erlang analysis under proj_dir has changed between the two calls. A file change is any modification to the file's content, size in bytes, or last-modification timestamp. The returned tuple does not depend on filesystem enumeration order or other non-deterministic factors.
- **Actual behavior:** If the function terminates normally, the return value is a tuple (backend, records) such that:
- backend = _elp_argv(), identifying the current ELP backend version and configuration.
- records = tuple(sorted(set((os.path.relpath(p, root), os.stat(p).st_size, os.stat(p).st_mtime_ns) for p in paths))) where root = os.path.abspath(proj_dir), paths = [p for p in _iter_project_files(root, {'.erl', '.hrl'})] + [os.path.join(root, name) for name in _PROJECT_CONFIG_FILES if os.path.isfile(os.path.join(root, name))], and sorted() orders by absolute path lexicographically. Each record is a 3-tuple (rel_path, size, mtime_ns). The records are sorted, distinct, and include every .erl/.hrl source file recursively under root and every existing project configuration file named in _PROJECT_CONFIG_FILES located directly in root (non-recursive). The file metadata (size, modification time) reflects the state at the moment os.stat() is called. Under the given precondition, records is nonempty. If any os.stat(), os.path.isfile(), filesystem iteration, or os.path.abspath() call raises an OSError (e.g., FileNotFoundError, PermissionError), the exception propagates and the function produces no return value; all local state is discarded.
- **Code evidence:** Line 12
- **Trigger condition:** The fingerprint is built only from file size and modification timestamp, ignoring file content. Thus a content modification that preserves size and mtime_ns goes undetected, violating the requirement that the fingerprint must change whenever any file's content changes (a file change is defined as any modification to content, size, or mtime).
- **Validator trigger:** Fingerprint uses only (size, mtime_ns) and ignores file content; same-length content change with preserved mtime goes undetected.
- **Probe stdout:** CONFIRMED — fingerprints equal despite content change: fp1fp2, size=4949, mtime_ns=1785172396729580385==1785172396729580385, but content differs

► SPEC sidecar (完整)

```
{
  "signature": "_project_fingerprint(proj_dir: str) -> tuple",
  "pre_condition": "proj_dir is a non-empty string that, when resolved to an absolute path, refers to an existing directory containing an Erlang project structure with at least one source file present.",
  "post_condition": "Returns a tuple that deterministically and uniquely identifies the current state of the Erlang project located at proj_dir. Two calls to _project_fingerprint with the same proj_dir return equal tuples if and only if no file relevant"
```

```
to Erlang analysis under proj_dir has changed between the two calls. A file change is any modification to the file's content, size in bytes, or last-modification timestamp. The returned tuple does not depend on filesystem enumeration order or other non-deterministic factors."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_iter_project_files",
      "signature": "_iter_project_files(root: str, extensions: set[str]) -> Iterator[str]",
      "pre_condition": "root is an absolute path to an existing directory; extensions is a non-empty set of file extension strings, each beginning with a period and containing only ASCII alphanumeric characters.",
      "post_condition": "Returns an iterator that yields the absolute path of every regular file whose extension is in extensions and that is located under root or any of its subdirectories, traversed recursively. Each matching file is yielded exactly once. Directories, symbolic links, and files with extensions not in extensions are excluded from the yielded output."
    },
    {
      "name": "_elp_argv",
      "signature": "_elp_argv() -> tuple",
      "pre_condition": "The ELP backend binary is installed and reachable via the configured command path.",
      "post_condition": "Returns a value that identifies the current ELP backend version and configuration. The returned value remains equal across successive calls when the ELP backend has not changed, and differs when the ELP backend changes in a way that could produce different analysis output for the same input project."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_project_fingerprint.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a tuple that deterministically and uniquely identifies the current state of the Erlang project located at proj_dir. Two calls to _project_fingerprint with the same proj_dir return equal tuples if and only if no file relevant to Erlang analysis under proj_dir has changed between the two calls. A file change is any modification to the file's content, size in bytes, or last-modification timestamp. The returned tuple does not depend on filesystem enumeration order or other non-deterministic factors.",
    "actual_behavior": "If the function terminates normally, the return value is a tuple (backend, records) such that:\n- backend = _elp_argv(), identifying the current ELP backend version and configuration.\n- records = tuple(sorted(set( (os.path.relpath(p, root), os.stat(p).st_size, os.stat(p).st_mtime_ns) for p in paths )))\n  where root = os.path.abspath(proj_dir),\n  paths = [p for p in _iter_project_files(root, ['.erl', '.hrl'])] + [os.path.join(root, name) for name in _PROJECT_CONFIG_FILES if os.path.isfile(os.path.join(root, name))],\n  and sorted() orders by absolute path lexicographically.\nEach record is a 3-tuple (rel_path, size, mtime_ns). The records are sorted, distinct, and include every .erl/.hrl source file recursively under root and every existing project configuration file named in _PROJECT_CONFIG_FILES located directly in root (non-recursive). The file metadata (size, modification time) reflects the state at the moment os.stat() is called. Under the given precondition, records is nonempty.\nIf any os.stat(), os.path.isfile(), filesystem iteration, or os.path.abspath() call raises an OSError (e.g., FileNotFoundError, PermissionError), the exception propagates and the function produces no return value; all local state is discarded.",
    "code_evidence": "Line 12",
    "trigger_condition": "The fingerprint is built only from file size and modification timestamp, ignoring file content. Thus a content modification that preserves size and mtime_ns goes undetected, violating the requirement that the fingerprint must change whenever any file's content changes (a file change is defined as any modification to content, size, or mtime)."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--_project_fingerprint",
  "source_file": "src/languages/erlang-py/_project_fingerprint.py",
  "function_name": "_project_fingerprint",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_project_fingerprint.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_project_fingerprint.md",
  "probe_stdout": "CONFIRMED - fingerprints equal despite content change: fp1==fp2, size=49==49, mtime_ns=1785172396729580385==1785172396729580385, but content differs\n",
  "trigger_summary": "Fingerprint uses only (size, mtime_ns) and ignores file content; same-length content change with preserved mtime goes undetected."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:10:35.326233Z	3410
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:11:21.359453Z	3324

事件 1 — [llm_bbab17cb593b4a00a2529943d04743f9](#)

► [parsed_json](#)（完整）

```
{
  "post_condition": "If the function terminates normally, the return value is a tuple (backend, records) such that:\n- backend = _elp_argv(), identifying the current ELP backend version and configuration.\n- records = tuple(sorted(set( (os.path.relpath(p, root), os.stat(p).st_size, os.stat(p).st_mtime_ns) for p in paths )))\n  where root = os.path.abspath(proj_dir),\n  paths = [p for p in _iter_project_files(root, {'.erl', '.hrl'})] + [os.path.join(root, name) for name in _PROJECT_CONFIG_FILES if os.path.isfile(os.path.join(root, name))],\n  and sorted() orders by absolute path lexicographically.\nEach record is a 3-tuple (rel_path, size, mtime_ns). The records are sorted, distinct, and include every .erl/.hrl source file recursively under root and every existing project configuration file named in _PROJECT_CONFIG_FILES located directly in root (non-recursive). The file metadata (size, modification time) reflects the state at the moment os.stat() is called. Under the given pre-condition, records is non-empty.\nIf any os.stat(), os.path.isfile(), filesystem iteration, or os.path.abspath() call raises an OSError (e.g., FileNotFoundError, PermissionError), the exception propagates and the function produces no return value; all local state is discarded."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_bbab17cb593b4a00a2529943d04743f9_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_bbab17cb593b4a00a2529943d04743f9_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_bbab17cb593b4a00a2529943d04743f9_response.txt\)](#)

事件 2 — [llm_e16e55040a9c493db14bb421df1e888f](#)

► [parsed_json](#)（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/tmp/erl' with a single file 'src/mod.erl' containing 'a' (size=1). Let mtime_ns = 1000000000. After first call, modify file content to 'b' (size=1) and set mtime_ns to exactly 1000000000 (e.g., via os.uptime). When _project_fingerprint is called again, it returns the same tuple because size and mtime_ns are identical, but a file content change occurred.",
  "offending_statements": "Line 12",
  "reason": "The fingerprint is built only from file size and modification timestamp, ignoring file content. Thus a content modification that preserves size and mtime_ns goes undetected, violating the requirement that the fingerprint must change whenever any file's content changes (a file change is defined as any modification to content, size, or mtime)."
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_e16e55040a9c493db14bb421df1e888f_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_e16e55040a9c493db14bb421df1e888f_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_e16e55040a9c493db14bb421df1e888f_response.txt\)](#) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-106 — [src--languages--erlang-py--batch_extract](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论；沿用旧清单的逐条审计结论。
- Validator: [not_confirmed](#)；attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / [language_handlers](#)；原源码 [src/languages/erlang.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a dictionary whose keys are absolute file paths (strings) of Erlang source files and whose values are lists of (function_id: str, body: str) pairs. Each function_id is a canonicalized, module-qualified name usable as an FQN. Each body is the verbatim source text of that function as extracted from the source file. Returns an empty dict when the Erlang Language Platform backend is unavailable or when proj_dir contains no extractable Erlang functions.
- **Actual behavior**: If the function returns normally, the returned value is a dictionary where each key is an absolute file path string referring to an Erlang source file within the directory specified by proj_dir, and each value is a list of tuples (function_id, body) extracted by the

- analysis. If an exception occurs (for example, from `_analysis_or_empty` due to invalid `proj_dir`, I/O errors, or because the result of `_analysis_or_empty` lacks a `functions` attribute), the exception propagates and the function does not return a value.
- **Code evidence:** Line 3: `return _analysis_or_empty(proj_dir).functions`
 - **Trigger condition:** The specification requires that the function returns an empty dict when the Erlang Language Platform backend is unavailable. However, the code directly returns the `.functions` attribute of `_analysis_or_empty` without any exception handling. If the backend is unavailable, `_analysis_or_empty` may raise an exception (as indicated by the example in Condition A), which would propagate and prevent the function from returning an empty dict. This violates the specification's requirement for that scenario.
 - **Validator trigger:** Call `batch_extract` on a directory with `.erl` files when ELP is not installed; the backend-unavailable exception is caught by `_analysis_or_empty`, which returns an empty `ErlangAnalysis(functions={}, edges={})`, so `batch_extract` returns `{}` as specified.
 - **Probe stdout:** `WARNING:root:ELP Erlang analysis unavailable for /tmp/erlang_probe_ov_1w_vt: [Errno 2] No such file or directory: 'elp' NOT CONFIRMED — batch_extract(...) returned: {} (no exception raised)`

► SPEC sidecar (完整)

```
{
  "signature": "batch_extract(proj_dir: str) -> dict",
  "pre_condition": "proj_dir is a string referring to a filesystem directory that may contain Erlang source files",
  "post_condition": "Returns a dictionary whose keys are absolute file paths (strings) of Erlang source files and whose values are lists of (function_id: str, body: str) pairs. Each function_id is a canonicalized, module-qualified name usable as an FQN. Each body is the verbatim source text of that function as extracted from the source file. Returns an empty dict when the Erlang Language Platform backend is unavailable or when proj_dir contains no extractable Erlang functions."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/batch_extract.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dictionary whose keys are absolute file paths (strings) of Erlang source files and whose values are lists of (function_id: str, body: str) pairs. Each function_id is a canonicalized, module-qualified name usable as an FQN. Each body is the verbatim source text of that function as extracted from the source file. Returns an empty dict when the Erlang Language Platform backend is unavailable or when proj_dir contains no extractable Erlang functions.",
    "actual_behavior": "If the function returns normally, the returned value is a dictionary where each key is an absolute file path string referring to an Erlang source file within the directory specified by proj_dir, and each value is a list of tuples (function_id, body) extracted by the analysis. If an exception occurs (for example, from _analysis_or_empty due to invalid proj_dir, I/O errors, or because the result of _analysis_or_empty lacks a `functions` attribute), the exception propagates and the function does not return a value.",
    "code_evidence": "Line 3: return _analysis_or_empty(proj_dir).functions",
    "trigger_condition": "The specification requires that the function returns an empty dict when the Erlang Language Platform backend is unavailable. However, the code directly returns the .functions attribute of _analysis_or_empty without any exception handling. If the backend is unavailable, _analysis_or_empty may raise an exception (as indicated by the example in Condition A), which would propagate and prevent the function from returning an empty dict. This violates the specification's requirement for that scenario."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--batch_extract",
  "source_file": "src/languages/erlang.py",
  "function_name": "batch_extract",
  "confirmation_status": "not_confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--batch_extract.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--batch_extract.md",
  "probe_stdout": "WARNING:root:ELP Erlang analysis unavailable for /tmp/erlang_probe_ov_1w_vt: [Errno 2] No such file or directory: 'elp'\nNOT CONFIRMED — batch_extract(...) returned: {} (no exception raised)",
  "trigger_summary": "Call batch_extract on a directory with .erl files when ELP is not installed; the backend-unavailable exception is caught by _analysis_or_empty, which returns an empty ErlangAnalysis(functions={}, edges={}), so batch_extract returns {} as specified."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:33:55.023783Z	1251
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:34:10.627515Z	3951

事件 1 — [llm_f22fbf4007b94f2da81b378a9a33557c](#)

► [parsed_json](#)（完整）

```
{
  "post_condition": "If the function returns normally, the returned value is a dictionary where each key is an absolute file path string referring to an Erlang source file within the directory specified by proj_dir, and each value is a list of tuples (function_id, body) extracted by the analysis. If an exception occurs (for example, from _analysis_or_empty due to invalid proj_dir, I/O errors, or because the result of _analysis_or_empty lacks a `functions` attribute), the exception propagates and the function does not return a value."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_f22fbf4007b94f2da81b378a9a33557c_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_f22fbf4007b94f2da81b378a9a33557c_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_f22fbf4007b94f2da81b378a9a33557c_response.txt\)](#)

事件 2 — [llm_d1a4b0e9886444e59add6e8247b3fb57](#)

► [parsed_json](#)（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = \"./valid/dir\" (a directory that exists and contains .erl files) while the Erlang Language Platform backend is unavailable. The code calls _analysis_or_empty, which raises an exception (e.g., BackendUnavailableError) and the exception propagates, so the function does not return an empty dictionary as required by the specification.",
  "offending_statements": "Line 3: return _analysis_or_empty(proj_dir).functions",
  "reason": "The specification requires that the function returns an empty dict when the Erlang Language Platform backend is unavailable. However, the code directly returns the .functions attribute of _analysis_or_empty without any exception handling. If the backend is unavailable, _analysis_or_empty may raise an exception (as indicated by the example in Condition A), which would propagate and prevent the function from returning an empty dict. This violates the specification's requirement for that scenario."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_d1a4b0e9886444e59add6e8247b3fb57_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_d1a4b0e9886444e59add6e8247b3fb57_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_d1a4b0e9886444e59add6e8247b3fb57_response.txt\)](#) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-107 — [src--languages--erlang-py--call_edges](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: [confirmed](#); attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / [language_handlers](#)；原源码 [src/languages/erlang.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a dictionary mapping each caller FQN string to a set of callee FQN strings for all Erlang functions in the project. Both caller and callee FQNs follow the same canonicalized, module-qualified naming convention used throughout the pipeline. Returns an empty dict when the Erlang Language Platform backend is unavailable or when no call edges can be extracted from the project.
- **Actual behavior:** The function returns a dictionary containing module-qualified Erlang call edges in registry format derived from the project directory [proj_dir](#). If [proj_dir](#) does not exist, contains no analyzable Erlang sources, or an internal error occurs, an empty dictionary is returned. No exceptions propagate and no mutable global state is modified. Formally: `proj_dir str, let result = call_edges(proj_dir); then isinstance(result, dict) (filesystem_contains_analyzable_erlang(proj_dir) result = edges(analysis_or_empty(callgraph_project_root(proj_dir)))) (filesystem_contains_analyzable_erlang(proj_dir) result = {})` execution terminates normally.

- **Code evidence:** Line 3
- **Trigger condition:** The code unconditionally returns an empty dictionary when any internal error occurs (via `_analysis_or_empty`), but the specification restricts returning an empty dictionary only to the specific cases where the backend is unavailable or no call edges can be extracted. For a valid project with callable functions and an available backend, an internal error causes the code to violate the specification by returning `{}` instead of the expected call-edge dictionary.
- **Validator trigger:** `_analysis_or_empty()` catches all exceptions via bare `'except Exception'`, causing `call_edges()` to return `{}` for any internal error — not just backend-unavailable or no-call-edges cases as the spec requires.
- **Probe stdout:** `WARNING:root:ELP Erlang analysis unavailable for /tmp/probe_erlang_2yqvwx7y: [Errno 2] No such file or directory: 'elp'`
`WARNING:root:ELP Erlang analysis unavailable for /tmp/probe_erlang_2yqvwx7y: Simulated mid-analysis ELP internal error CONFIRMED` — `call_edges()` silently returned `{}` when an internal error (`RuntimeError`) was raised during analysis. The spec allows `{}` only for backend-unavailable or no-call-edges cases, but `_analysis_or_empty` swallows ALL exceptions. `result_normal={}` `result_bug={}`

► SPEC sidecar (完整)

```
{
  "signature": "call_edges(proj_dir: str) -> dict",
  "pre_condition": "proj_dir is a string referring to a filesystem directory that may contain Erlang source files with statically analyzable call relationships",
  "post_condition": "Returns a dictionary mapping each caller FQN string to a set of callee FQN strings for all Erlang functions in the project. Both caller and callee FQNs follow the same canonicalized, module-qualified naming convention used throughout the pipeline. Returns an empty dict when the Erlang Language Platform backend is unavailable or when no call edges can be extracted from the project."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/call_edges.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dictionary mapping each caller FQN string to a set of callee FQN strings for all Erlang functions in the project. Both caller and callee FQNs follow the same canonicalized, module-qualified naming convention used throughout the pipeline. Returns an empty dict when the Erlang Language Platform backend is unavailable or when no call edges can be extracted from the project.",
    "actual_behavior": "The function returns a dictionary containing module-qualified Erlang call edges in registry format derived from the project directory `proj_dir`. If `proj_dir` does not exist, contains no analyzable Erlang sources, or an internal error occurs, an empty dictionary is returned. No exceptions propagate and no mutable global state is modified. Formally: proj_dir str, let result = call_edges(proj_dir); then isinstance(result, dict) (filesystem_contains_analyzable_erlang(proj_dir) result = edges(analysis_or_empty(callgraph_project_root(proj_dir)))) (filesystem_contains_analyzable_erlang(proj_dir) result = {})  
execution terminates normally.",
    "code_evidence": "Line 3",
    "trigger_condition": "The code unconditionally returns an empty dictionary when any internal error occurs (via _analysis_or_empty), but the specification restricts returning an empty dictionary only to the specific cases where the backend is unavailable or no call edges can be extracted. For a valid project with callable functions and an available backend, an internal error causes the code to violate the specification by returning {} instead of the expected call-edge dictionary."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--call_edges",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/call_edges.py",
  "function_name": "call_edges",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--call_edges.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--call_edges.md",
  "probe_stdout": "WARNING:root:ELP Erlang analysis unavailable for /tmp/probe_erlang_2yqvwx7y: [Errno 2] No such file or directory: 'elp'\nWARNING:root:ELP Erlang analysis unavailable for /tmp/probe_erlang_2yqvwx7y: Simulated mid-analysis ELP internal error\nCONFIRMED - call_edges() silently returned {} when an internal error (RuntimeError) was raised during analysis. The spec allows {} only for backend-unavailable or no-call-edges cases, but _analysis_or_empty swallows ALL exceptions. result_normal={} result_bug={}",
  "trigger_summary": "_analysis_or_empty() catches all exceptions via bare 'except Exception', causing call_edges() to return {}"
```

```
for any internal error – not just backend-unavailable or no-call-edges cases as the spec requires."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:34:15.248595Z	1457
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:34:34.369771Z	4930

事件 1 — [llm_ea6ec95ca2f04996aa6c9031e46b57c3](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a dictionary containing module-qualified Erlang call edges in registry format derived
from the project directory `proj_dir`. If `proj_dir` does not exist, contains no analyzable Erlang sources, or an internal error
occurs, an empty dictionary is returned. No exceptions propagate and no mutable global state is modified. Formally:  $\forall \text{proj\_dir} \in \text{str}, \text{let result} = \text{call\_edges}(\text{proj\_dir}); \text{then } \text{isinstance}(\text{result}, \text{dict}) \wedge (\text{filesystem\_contains\_analyzable\_erlang}(\text{proj\_dir}) \Rightarrow \text{result} = \text{edges}(\text{analysis\_or\_empty}(\text{callgraph\_project\_root}(\text{proj\_dir})))) \wedge (\neg \text{filesystem\_contains\_analyzable\_erlang}(\text{proj\_dir}) \Rightarrow \text{result} = \{\}) \wedge \text{execution terminates normally.}"
}$ 
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ea6ec95ca2f04996aa6c9031e46b57c3_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ea6ec95ca2f04996aa6c9031e46b57c3_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ea6ec95ca2f04996aa6c9031e46b57c3_response.txt)

事件 2 — [llm_4acab196eb104490b017828277346985](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/tmp/valid_erlang_project' containing a simple Erlang module with function calls (e.g., module1 calling module2), where the Erlang Language Platform backend is available but a transient internal error (e.g., an unexpected exception in the underlying analysis library) occurs.",
  "offending_statements": "Line 3",
  "reason": "The code unconditionally returns an empty dictionary when any internal error occurs (via _analysis_or_empty), but the specification restricts returning an empty dictionary only to the specific cases where the backend is unavailable or no call edges can be extracted. For a valid project with callable functions and an available backend, an internal error causes the code to violate the specification by returning {} instead of the expected call-edge dictionary."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4acab196eb104490b017828277346985_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4acab196eb104490b017828277346985_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4acab196eb104490b017828277346985_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-114 — [src](#) - [languages](#) - [python](#) - [py](#) - [batch_extract](#)

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: [confirmed](#); attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 原源码 [src/languages/python.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a dictionary whose keys are absolute file paths (str) for every Python source file discovered in proj_dir. The value for each key is a list of (func_name: str, func_body: str) 2-tuples, one tuple per function defined in that file. A file containing no function definitions yields an empty list. When the static-analysis backend cannot initialize, returns an empty dictionary.
- **Actual behavior:** After execution, if no exception occurs, the function returns a dictionary. Let cg = CodeGraphExtractor.from_proj_dir(proj_dir). If cg is not None, the returned dictionary R equals cg.get_functions_by_file('python', proj_dir);

i.e., for each absolute file path `p` of a Python source file under `proj_dir`, `R[p]` is a list of `(func_name, func_body)` tuples for all functions in that file. If `cg` is `None`, then `R = {}`. If an exception is raised during `from_proj_dir` or `get_functions_by_file`, the exception propagates and `batch_extract` returns no value.

Formal logic: Normal termination: $(\text{cg} \{ \text{CodeGraphExtractor}, \text{None} \}. \text{cg} = \text{CodeGraphExtractor.from_proj_dir}(\text{proj_dir}) \mid (\text{cg} \text{ None } R = \text{cg.get_functions_by_file}(\text{'python'}, \text{proj_dir})) \mid (\text{cg} = \text{None } R = \{ \}))$. Abnormal termination: An exception `E` is raised, and the function stack is unwound.

- **Code evidence:** Line 4: `return cg.get_functions_by_file("python", proj_dir) if cg else {}`
- **Trigger condition:** The specification mandates that every Python file in the project is a key, with an empty list if no functions are present. The implementation relies on `get_functions_by_file`, which may only include files that contain at least one function, thus omitting files without functions.
- **Validator trigger:** `batch_extract` relies on `get_functions_by_file` which queries `codegraph` for function/method nodes; files with no functions yield no rows and are silently omitted from the result dictionary, violating the spec that every Python file must be a key (with an empty list if no functions exist).
- **Probe stdout:** CONFIRMED — 1 Python file(s) missing from result dict: `['no_funcs.py']`. Present: `['has_funcs.py']`. Spec requires every Python file to be a key.

► SPEC sidecar (完整)

```
{
  "signature": "batch_extract(proj_dir: str) -> dict",
  "pre_condition": "proj_dir is a path to an existing directory",
  "post_condition": "Returns a dictionary whose keys are absolute file paths (str) for every Python source file discovered in proj_dir. The value for each key is a list of (func_name: str, func_body: str) 2-tuples, one tuple per function defined in that file. A file containing no function definitions yields an empty list. When the static-analysis backend cannot initialize, returns an empty dictionary."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "CodeGraphExtractor.from_proj_dir",
      "signature": "from_proj_dir(proj_dir: str) -> CodeGraphExtractor | None",
      "pre_condition": "proj_dir is a path to an existing directory",
      "post_condition": "Returns a CodeGraphExtractor instance initialized with the project source index, or None if initialization fails"
    },
    {
      "name": "get_functions_by_file",
      "signature": "get_functions_by_file(language: str, proj_dir: str) -> dict",
      "pre_condition": "language is a recognized language key; proj_dir is a path to an existing directory",
      "post_condition": "Returns a dict whose keys are absolute file paths (str) of source files matching the language and whose values are lists of (func_name: str, func_body: str) tuples for every function in that file"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/python-py/batch_extract.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dictionary whose keys are absolute file paths (str) for every Python source file discovered in proj_dir. The value for each key is a list of (func_name: str, func_body: str) 2-tuples, one tuple per function defined in that file. A file containing no function definitions yields an empty list. When the static-analysis backend cannot initialize, returns an empty dictionary.",
    "actual_behavior": "After execution, if no exception occurs, the function returns a dictionary. Let cg = CodeGraphExtractor.from_proj_dir(proj_dir). If cg is not None, the returned dictionary R equals cg.get_functions_by_file('python', proj_dir); i.e., for each absolute file path p of a Python source file under proj_dir, R[p] is a list of (func_name, func_body) tuples for all functions in that file. If cg is None, then R = {}. If an exception is raised during from_proj_dir or get_functions_by_file, the exception propagates and batch_extract returns no value.\n\nFormal logic:\nNormal termination: ( cg {CodeGraphExtractor, None}. cg = CodeGraphExtractor.from_proj_dir(proj_dir) ( cg None R = cg.get_functions_by_file('python', proj_dir)) (cg = None R = { }) ) .\nAbnormal termination: An exception E is raised, and the function stack is unwound.",
    "code_evidence": "Line 4: return cg.get_functions_by_file(\"python\", proj_dir) if cg else {}",
    "trigger_condition": "The specification mandates that every Python file in the project is a key, with an empty list if no functions are present. The implementation relies on get_functions_by_file, which may only include files that contain at least one function, thus omitting files without functions."
  }
}
```



```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--python-py--batch_extract",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/python-py/batch_extract.py",
  "function_name": "batch_extract",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--python-py--batch_extract.py",
  "detail_file": "fm_agent/bug_validation/src--languages--python-py--batch_extract.md",
  "probe_stdout": "CONFIRMED - 1 Python file(s) missing from result dict: ['no_funcs.py']. Present: ['has_funcs.py']. Spec requires every Python file to be a key.",
  "trigger_summary": "batch_extract relies on get_functions_by_file which queries codegraph for function/method nodes; files with no functions yield no rows and are silently omitted from the result dictionary, violating the spec that every Python file must be a key (with an empty list if no functions exist)."
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:38:18.348307Z	2609
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:38:48.209212Z	2710

事件 1 — [llm_db8fba5fb8a745eba9efe6c8ea52c338](#)

► parsed_json（完整）

```
{
  "post_condition": "After execution, if no exception occurs, the function returns a dictionary. Let cg = CodeGraphExtractor.from_proj_dir(proj_dir). If cg is not None, the returned dictionary R equals cg.get_functions_by_file('python', proj_dir); i.e., for each absolute file path p of a Python source file under proj_dir, R[p] is a list of (func_name, func_body) tuples for all functions in that file. If cg is None, then R = {}. If an exception is raised during from_proj_dir or get_functions_by_file, the exception propagates and batch_extract returns no value.\n\nFormal logic:\nNormal termination: (∃ cg ∈ {CodeGraphExtractor, None}. cg = CodeGraphExtractor.from_proj_dir(proj_dir) ∧ ( cg ≠ None ∧ R = cg.get_functions_by_file('python', proj_dir)) ∨ (cg = None ∧ R = {}) ) .\nAbnormal termination: An exception E is raised, and the function stack is unwound."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_db8fba5fb8a745eba9efe6c8ea52c338_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_db8fba5fb8a745eba9efe6c8ea52c338_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_db8fba5fb8a745eba9efe6c8ea52c338_response.txt)

事件 2 — [llm_310362618f694a21a98333ad2ed7f083](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp/test_project' containing a single Python file 'empty.py' with content '# just a comment' (no function definitions). If from_proj_dir succeeds and returns a non-None CodeGraphExtractor, then cg.get_functions_by_file('python', proj_dir) returns {} (an empty dict with no keys) because no functions were found. The code returns {}. However, the specification requires a dictionary with key '/tmp/test_project/empty.py' and value [].",
  "offending_statements": "Line 4: return cg.get_functions_by_file('python', proj_dir) if cg else {}",
  "reason": "The specification mandates that every Python file in the project is a key, with an empty list if no functions are present. The implementation relies on get_functions_by_file, which may only include files that contain at least one function, thus omitting files without functions."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_310362618f694a21a98333ad2ed7f083_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_310362618f694a21a98333ad2ed7f083_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_310362618f694a21a98333ad2ed7f083_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

Phase 3 — Pipeline Planning & Domain Knowledge

模块：pipeline_setup

FMA-MISMATCH-128 — src--pipeline_setup-py--_collapse_phases_to_one

- 四类结论： 可能有 Bug
- 分类依据： 沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 3 — Pipeline Planning & Domain Knowledge / pipeline_setup; 源代码 src/pipeline_setup.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim：** If phases.json contained at least one phase, the file under work_dir is rewritten such that: (1) the 'phases' array contains exactly one phase element; (2) the single phase's 'modules' array is the concatenation, in ascending 'phase' order, of every 'modules' array from all original phases, with the relative ordering of modules within each original phase's 'modules' array preserved; (3) the single phase's 'description' is a string formed by joining the stripped, non-empty descriptions of all original phases in ascending 'phase' order with a blank-line separator; (4) the single phase's 'depends_on_phases' is an empty list. Additionally, if one or more 'phase_types.txt' files exist under '<work_dir>/spec_prompts/domain_context/', their stripped contents are concatenated in ascending phase order (separated by blank lines) and written to 'phase_01_types.txt' in the same directory, then all other 'phase_types.txt' files in that directory are deleted. If phases.json contained zero phases, no file modifications occur.
- **Actual behavior：** If the 'phases' list is empty, the file system remains unchanged and the function returns immediately. Otherwise, after normal execution (no exceptions):
 1. The file at phases_path (work_dir/phases.json) is overwritten with a JSON object whose 'phases' key holds a single-element list containing a merged phase object M. M has the same keys as the first phase in the original sorted list (including its original 'phase' number, which may not necessarily be 1), with the following modifications:
 - 'name' is set to "Unified Analysis Phase".
 - 'description' is the concatenation of all nonempty, stripped 'description' strings from every original phase, in sorted order, separated by "\n\n". If none are nonempty, this field is present but empty.
 - 'modules' is a list containing every module dictionary from every original phase, preserving the order of phases and the order of modules within each phase.
 - 'depends_on_phases' is set to an empty list [].
 2. In the directory work_dir/spec_prompts/domain_context: a. Let T be the set of paths phase_{:02d}types.txt for each original phase (using its 'phase' number, zero-padded to two digits). b. If at least one such file existed, a single file phase_01_types.txt is created (or overwritten) containing the stripped contents of all existing files in T, concatenated in order with "\n\n" separators and a trailing newline. All other files matching the glob pattern "phase*_types.txt" in that directory (including any that were not in T) are deleted, except the newly written phase_01_types.txt. After this step, the only file matching that pattern is phase_01_types.txt with the merged content. c. If no file from T existed, no new file is created and no files are deleted; the directory remains as is.

All operations assume existing files and directories required by the precondition are readable/writable. If an I/O error occurs (e.g., inability to write phases.json or create the merged types file), the function raises an exception and the file system may be left in a partially modified state.

- **Code evidence：** Line 10: f"Phase {phase['phase']} ({phase['name']}): {phase_description}"
- **Trigger condition：** The specification requires the merged description to be just the stripped, non-empty descriptions of all phases joined by a blank line. The code incorrectly inserts a prefix (e.g., 'Phase 1 (Test): ') before each description, changing the content.
- **Validator trigger：** The code prepends a 'Phase N (Name): ' prefix before each description instead of joining just the stripped non-empty descriptions with blank lines.
- **Probe stdout：** CONFIRMED — description has unwanted phase prefix: actual: 'Phase 1 (Analysis Phase): First phase handles initial setup\n\nPhase 2 (Verification Phase): Second phase verifies the results\n\nPhase 3 (Cleanup Phase): Final phase does cleanup' | expected: 'First phase handles initial setup\n\nSecond phase verifies the results\n\nFinal phase does cleanup'

► SPEC sidecar （完整）

```
{
  "signature": "_collapse_phases_to_one(work_dir)",
  "pre_condition": "work_dir is a string path to a directory containing a readable phases.json file whose content conforms to the phases.json schema:the top-level object has a 'phases' key whose value is a list of phase objects, each containing at minimum 'phase' (int, 1-indexed) and optionally 'name' (str), 'description' (str), 'modules' (list of module dicts), and
```

```
    'depends_on_phases' (list of ints)",
    "post_condition": "If phases.json contained at least one phase, the file under work_dir is rewritten such that: (1) the 'phases' array contains exactly one phase element; (2) the single phase's 'modules' array is the concatenation, in ascending 'phase' order, of every 'modules' array from all original phases, with the relative ordering of modules within each original phase's 'modules' array preserved; (3) the single phase's 'description' is a string formed by joining the stripped, non-empty descriptions of all original phases in ascending 'phase' order with a blank-line separator; (4) the single phase's 'depends_on_phases' is an empty list. Additionally, if one or more 'phase_*_types.txt' files exist under '<work_dir>/spec_prompts/domain_context/', their stripped contents are concatenated in ascending phase order (separated by blank lines) and written to 'phase_01_types.txt' in the same directory, then all other 'phase_*_types.txt' files in that directory are deleted. If phases.json contained zero phases, no file modifications occur."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-py/_collapse_phases_to_one.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "If phases.json contained at least one phase, the file under work_dir is rewritten such that: (1) the 'phases' array contains exactly one phase element; (2) the single phase's 'modules' array is the concatenation, in ascending 'phase' order, of every 'modules' array from all original phases, with the relative ordering of modules within each original phase's 'modules' array preserved; (3) the single phase's 'description' is a string formed by joining the stripped, non-empty descriptions of all original phases in ascending 'phase' order with a blank-line separator; (4) the single phase's 'depends_on_phases' is an empty list. Additionally, if one or more 'phase_*_types.txt' files exist under '<work_dir>/spec_prompts/domain_context/', their stripped contents are concatenated in ascending phase order (separated by blank lines) and written to 'phase_01_types.txt' in the same directory, then all other 'phase_*_types.txt' files in that directory are deleted. If phases.json contained zero phases, no file modifications occur.",
    "actual_behavior": "If the 'phases' list is empty, the file system remains unchanged and the function returns immediately. Otherwise, after normal execution (no exceptions):\n\n1. The file at phases_path (work_dir/phases.json) is overwritten with a JSON object whose 'phases' key holds a single-element list containing a merged phase object M. M has the same keys as the first phase in the original sorted list (including its original 'phase' number, which may not necessarily be 1), with the following modifications:\n   - 'name' is set to \"Unified Analysis Phase\".\n   - 'description' is the concatenation of all nonempty, stripped 'description' strings from every original phase, in sorted order, separated by \"\\n\\n\". If none are nonempty, this field is present but empty.\n   - 'modules' is a list containing every module dictionary from every original phase, preserving the order of phases and the order of modules within each phase.\n   - 'depends_on_phases' is set to an empty list [].\n\n2. In the directory work_dir/spec_prompts/domain_context:\n   a. Let T be the set of paths phase_{:02d}_types.txt for each original phase (using its 'phase' number, zero-padded to two digits).\n   b. If at least one such file existed, a single file phase_01_types.txt is created (or overwritten) containing the stripped contents of all existing files in T, concatenated in order with \"\\n\\n\" separators and a trailing newline. All other files matching the glob pattern \"phase_*_types.txt\" in that directory (including any that were not in T) are deleted, except the newly written phase_01_types.txt. After this step, the only file matching that pattern is phase_01_types.txt with the merged content.\n   c. If no file from T existed, no new file is created and no files are deleted; the directory remains as is.\n\nAll operations assume existing files and directories required by the precondition are readable/writable. If an I/O error occurs (e.g., inability to write phases.json or create the merged types file), the function raises an exception and the file system may be left in a partially modified state.",
    "code_evidence": "Line 10: f\"Phase {phase['phase']} ({phase['name']}): {phase['description']}\"",
    "trigger_condition": "The specification requires the merged description to be just the stripped, non-empty descriptions of all phases joined by a blank line. The code incorrectly inserts a prefix (e.g., 'Phase 1 (Test): ') before each description, changing the content."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--pipeline_setup-py--collapse_phases_to_one",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-py/_collapse_phases_to_one.py",
  "function_name": "_collapse_phases_to_one",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--pipeline_setup-py--collapse_phases_to_one.py",
  "detail_file": "fm_agent/bug_validation/src--pipeline_setup-py--collapse_phases_to_one.md",
  "probe_stdout": "CONFIRMED - description has unwanted phase prefix: actual: 'Phase 1 (Analysis Phase): First phase handles initial setup\\n\\nPhase 2 (Verification Phase): Second phase verifies the results\\n\\nPhase 3 (Cleanup Phase): Final phase does cleanup' | expected: 'First phase handles initial setup\\n\\nSecond phase verifies the results\\n\\nFinal phase does cleanup'",
  "trigger_summary": "The code prepends a 'Phase N (Name): ' prefix before each description instead of joining just the stripped non-empty descriptions with blank lines."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:31:37.357893Z	5332
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:32:42.560868Z	4702

事件 1 — [llm_30322ce94dc34ad8b2b58a76cafb1c65](#)

► [parsed_json](#)（完整）

```
{
  "post_condition": "If the 'phases' list is empty, the file system remains unchanged and the function returns immediately. Otherwise, after normal execution (no exceptions):\n\n1. The file at phases_path (work_dir/phases.json) is overwritten with a JSON object whose 'phases' key holds a single-element list containing a merged phase object M. M has the same keys as the first phase in the original sorted list (including its original 'phase' number, which may not necessarily be 1), with the following modifications:\n  - 'name' is set to \"Unified Analysis Phase\".\n  - 'description' is the concatenation of all non-empty, stripped 'description' strings from every original phase, in sorted order, separated by \"\\n\\n\". If none are non-empty, this field is present but empty.\n  - 'modules' is a list containing every module dictionary from every original phase, preserving the order of phases and the order of modules within each phase.\n  - 'depends_on_phases' is set to an empty list [].\n\n2. In the directory work_dir/spec_prompts/domain_context:\n  a. Let T be the set of paths phase_{:02d}_types.txt for each original phase (using its 'phase' number, zero-padded to two digits).\n  b. If at least one such file existed, a single file phase_01_types.txt is created (or overwritten) containing the stripped contents of all existing files in T, concatenated in order with \"\\n\\n\\n\" separators and a trailing newline. All other files matching the glob pattern \"phase*_types.txt\" in that directory (including any that were not in T) are deleted, except the newly written phase_01_types.txt. After this step, the only file matching that pattern is phase_01_types.txt with the merged content.\n  c. If no file from T existed, no new file is created and no files are deleted; the directory remains as is.\n\nAll operations assume existing files and directories required by the pre-condition are readable/writable. If an I/O error occurs (e.g., inability to write phases.json or create the merged types file), the function raises an exception and the file system may be left in a partially modified state."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_30322ce94dc34ad8b2b58a76cafb1c65_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_30322ce94dc34ad8b2b58a76cafb1c65_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_30322ce94dc34ad8b2b58a76cafb1c65_response.txt\)](#)

事件 2 — [llm_7277412f28514647ae25196e276739fb](#)

► [parsed_json](#)（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "phases.json contains a single phase with a non-empty description: {\"phases\": [{\"phase\": 1, \"name\": \"Test\", \"description\": \"A description.\", \"modules\": []}]}.",
  "offending_statements": "Line 10: f\"Phase {phase['phase']} ({phase['name']}): {phase_description}\"",
  "reason": "The specification requires the merged description to be just the stripped, non-empty descriptions of all phases joined by a blank line. The code incorrectly inserts a prefix (e.g., 'Phase 1 (Test): ') before each description, changing the content."
}
```

- 载荷: [\[system_prompt\]\(fm_agent/trace/payloads/llm_7277412f28514647ae25196e276739fb_message_00_system.txt\)](#) · [\[user_prompt\]\(fm_agent/trace/payloads/llm_7277412f28514647ae25196e276739fb_message_01_user.txt\)](#) · [\[assistant_output\]\(fm_agent/trace/payloads/llm_7277412f28514647ae25196e276739fb_response.txt\)](#) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-131 — [src--pipeline_setup-py--_domain_context_complete](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: [confirmed](#); attempts 1
- Phase / 模块：Phase 3 — Pipeline Planning & Domain Knowledge / [pipeline_setup](#)；源代码 [src/pipeline_setup.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns True if and only if all of the following hold: (1) work_dir contains a valid phases.json file a readable, parseable JSON file whose top-level object has a 'phases' array where each element has a non-None integer 'phase' key; (2) the file work_dir/spec_prompts/domain_context/engine_overview.txt exists; (3) for every phase number P present in the 'phases' array, the file work_dir/spec_prompts/domain_context/phase_P_types.txt exists, where P is formatted as a zero-padded two-digit integer. Returns False if

- any of these conditions is not met, including when phases.json is missing, unreadable, not valid JSON, or has a 'phases' array containing an element whose 'phase' key is missing or None. The function does not modify any files on the filesystem.
- **Actual behavior:** The filesystem state is preserved exactly as it was before the call; no files are created, modified, or deleted, and no external resources are altered. The function either returns True or False without raising any exceptions (all internal exceptions are caught). The function returns True if and only if all of the following conditions are met, otherwise it returns False: (1) The file at path `os.path.join(work_dir, 'phases.json')` exists, is readable, and contains syntactically valid JSON (as determined by `json_file_is_valid`). (2) The file at path `os.path.join(work_dir, 'spec_prompts', 'domain_context', 'engine_overview.txt')` exists. (3) After successfully opening and parsing the `phases.json` file, the resulting JSON object contains a key 'phases' whose value is iterable, and for every element in that sequence, the element has a non-None value under the key 'phase' (typically an integer), and the file at path `os.path.join(work_dir, 'spec_prompts', 'domain_context', f'phase{phase:02d}_types.txt')` exists. Formally: `(return_value = True) (json_file_is_valid(phases_path) os.path.exists(engine_overview_path) phase parse(json.load(open(phases_path))).get('phases', []), phase.get('phase') None os.path.exists(phase_types_path(phase.get('phase'))))), where phases_path = os.path.join(work_dir, 'phases.json'), engine_overview_path = os.path.join(domain_dir, 'engine_overview.txt'), domain_dir = os.path.join(work_dir, 'spec_prompts', 'domain_context'), and phase_types_path(n) = os.path.join(domain_dir, f'phase{n:02d}_types.txt')`. If any of these conditions is not satisfied, the function returns False; for example, missing files, invalid JSON, a missing 'phases' key, a phase element missing the 'phase' key, or a missing phase types file all yield False. The return statement at line 21 is reached only when all checks pass.
 - **Code evidence:** Line 14: `for phase in phases_data.get("phases", []):`
 - **Trigger condition:** If the top-level JSON value is not a dict (e.g., an array), `_json_file_is_valid` returns True, but `json.load` returns a list, and calling `.get` on that list raises an uncaught `AttributeError`, causing the function to propagate an exception instead of returning False, violating the specification.
 - **Validator trigger:** When `phases.json` contains a top-level JSON array, `_json_file_is_valid` returns True but `phases_data.get()` fails with `AttributeError`, propagating an exception instead of returning False.
 - **Probe stdout:** CONFIRMED — actual: `AttributeError` raised | expected: return False

► SPEC sidecar (完整)

```
{
  "signature": "_domain_context_complete(work_dir)",
  "pre_condition": "work_dir is a directory path",
  "post_condition": "Returns True if and only if all of the following hold: (1) work_dir contains a valid phases.json file – a
readable, parseable JSON file whose top-level object has a 'phases' array where each element has a non-None integer 'phase' key;
(2) the file work_dir/spec_prompts/domain_context/engine_overview.txt exists; (3) for every phase number P present in the 'phases'
array, the file work_dir/spec_prompts/domain_context/phase_P_types.txt exists, where P is formatted as a zero-padded two-digit
integer. Returns False if any of these conditions is not met, including when phases.json is missing, unreadable, not valid JSON,
or has a 'phases' array containing an element whose 'phase' key is missing or None. The function does not modify any files on the
filesystem."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_json_file_is_valid",
      "signature": "_json_file_is_valid(path)",
      "pre_condition": "path is a string representing a filesystem path which may or may not correspond to an existing, readable
file",
      "post_condition": "Returns True if the file at path exists, is readable, and contains syntactically valid JSON. Returns
False if the file does not exist, cannot be read, or its contents are not valid JSON. Does not modify the file or the filesystem."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-
py/_domain_context_complete.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns True if and only if all of the following hold: (1) work_dir contains a valid phases.json file a
readable, parseable JSON file whose top-level object has a 'phases' array where each element has a non-None integer 'phase' key;
(2) the file work_dir/spec_prompts/domain_context/engine_overview.txt exists; (3) for every phase number P present in the 'phases'
array, the file work_dir/spec_prompts/domain_context/phase_P_types.txt exists, where P is formatted as a zero-padded two-digit
integer. Returns False if any of these conditions is not met, including when phases.json is missing, unreadable, not valid JSON,
or has a 'phases' array containing an element whose 'phase' key is missing or None. The function does not modify any files on the
filesystem.",
    "actual_behavior": "The filesystem state is preserved exactly as it was before the call; no files are created, modified, or
```



```
deleted, and no external resources are altered. The function either returns True or False without raising any exceptions (all internal exceptions are caught). The function returns True if and only if all of the following conditions are met, otherwise it returns False: (1) The file at path os.path.join(work_dir, 'phases.json') exists, is readable, and contains syntactically valid JSON (as determined by _json_file_is_valid). (2) The file at path os.path.join(work_dir, 'spec_prompts', 'domain_context', 'engine_overview.txt') exists. (3) After successfully opening and parsing the phases.json file, the resulting JSON object contains a key 'phases' whose value is iterable, and for every element in that sequence, the element has a non-None value under the key 'phase' (typically an integer), and the file at path os.path.join(work_dir, 'spec_prompts', 'domain_context', f'phase_{phase:02d}_types.txt') exists. Formally: (return_value = True)  $\leftrightarrow$  ( _json_file_is_valid(phases_path)  $\wedge$  os.path.exists(engine_overview_path)  $\wedge$   $\forall$  phase  $\in$  parse(json.load(open(phases_path))).get('phases', []), phase.get('phase')  $\neq$  None  $\wedge$  os.path.exists(phase_types_path(phase.get('phase')))) ), where phases_path = os.path.join(work_dir, 'phases.json'), engine_overview_path = os.path.join(domain_dir, 'engine_overview.txt'), domain_dir = os.path.join(work_dir, 'spec_prompts', 'domain_context'), and phase_types_path(n) = os.path.join(domain_dir, f'phase_{n:02d}_types.txt'). If any of these conditions is not satisfied, the function returns False; for example, missing files, invalid JSON, a missing 'phases' key, a phase element missing the 'phase' key, or a missing phase types file all yield False. The return statement at line 21 is reached only when all checks pass.",
    "code_evidence": "Line 14: for phase in phases_data.get(\"phases\", []):",
    "trigger_condition": "If the top-level JSON value is not a dict (e.g., an array), _json_file_is_valid returns True, but json.load returns a list, and calling .get on that list raises an uncaught AttributeError, causing the function to propagate an exception instead of returning False, violating the specification."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--pipeline_setup-py--_domain_context_complete",
  "source_file": "src/pipeline_setup-py/_domain_context_complete.py",
  "function_name": "_domain_context_complete",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--pipeline_setup-py--_domain_context_complete.py",
  "detail_file": "fm_agent/bug_validation/src--pipeline_setup-py--_domain_context_complete.md",
  "probe_stdout": "CONFIRMED - actual: AttributeError raised | expected: return False",
  "trigger_summary": "When phases.json contains a top-level JSON array, _json_file_is_valid returns True but phases_data.get() fails with AttributeError, propagating an exception instead of returning False."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:31:35.352913Z	2354
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:31:57.581934Z	3884

事件 1 — [llm_a63d6a6abf9d4b96972d9ccfc81e2d68](#)

► parsed_json（完整）

```
{
  "post_condition": "The filesystem state is preserved exactly as it was before the call; no files are created, modified, or deleted, and no external resources are altered. The function either returns True or False without raising any exceptions (all internal exceptions are caught). The function returns True if and only if all of the following conditions are met, otherwise it returns False: (1) The file at path os.path.join(work_dir, 'phases.json') exists, is readable, and contains syntactically valid JSON (as determined by _json_file_is_valid). (2) The file at path os.path.join(work_dir, 'spec_prompts', 'domain_context', 'engine_overview.txt') exists. (3) After successfully opening and parsing the phases.json file, the resulting JSON object contains a key 'phases' whose value is iterable, and for every element in that sequence, the element has a non-None value under the key 'phase' (typically an integer), and the file at path os.path.join(work_dir, 'spec_prompts', 'domain_context', f'phase_{phase:02d}_types.txt') exists. Formally: (return_value = True)  $\leftrightarrow$  ( _json_file_is_valid(phases_path)  $\wedge$  os.path.exists(engine_overview_path)  $\wedge$   $\forall$  phase  $\in$  parse(json.load(open(phases_path))).get('phases', []), phase.get('phase')  $\neq$  None  $\wedge$  os.path.exists(phase_types_path(phase.get('phase')))) ), where phases_path = os.path.join(work_dir, 'phases.json'), engine_overview_path = os.path.join(domain_dir, 'engine_overview.txt'), domain_dir = os.path.join(work_dir, 'spec_prompts', 'domain_context'), and phase_types_path(n) = os.path.join(domain_dir, f'phase_{n:02d}_types.txt'). If any of these conditions is not satisfied, the function returns False; for example, missing files, invalid JSON, a missing 'phases' key, a phase element missing the 'phase' key, or a missing phase types file all yield False. The return statement at line 21 is reached only when all checks pass."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_a63d6a6abf9d4b96972d9ccfc81e2d68_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_a63d6a6abf9d4b96972d9ccfc81e2d68_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_a63d6a6abf9d4b96972d9ccfc81e2d68_response.txt)

事件 2 — [llm_805360204eed43559bea09193928ca25](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "work_dir where phases.json contains the valid JSON array `[]`, and
`work_dir/spec_prompts/domain_context/engine_overview.txt` exists.",
  "offending_statements": "Line 14: for phase in phases_data.get(\"phases\", []):",
  "reason": "If the top-level JSON value is not a dict (e.g., an array), _json_file_is_valid returns True, but json.load returns a
list, and calling .get on that list raises an uncaught AttributeError, causing the function to propagate an exception instead of
returning False, violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_805360204eed43559bea09193928ca25_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_805360204eed43559bea09193928ca25_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_805360204eed43559bea09193928ca25_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-132 — src--pipeline_setup-py--ensure_source_files_in_phases

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 3 — Pipeline Planning & Domain Knowledge / pipeline_setup; 原源码 src/pipeline_setup.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** When required_source_files is None or empty, returns {'forced': [], 'augmented': {}, 'augmented_modules': []} without reading or modifying phases_json. Otherwise, guarantees that after return every path in required_source_files (with backslash directory separators normalized to forward slash) appears in the source_files list of exactly one module in the phases.json file. Returns a dict where: 'forced' is the list of required paths that were not already present in any phase module's source_files before the call (empty when all paths were already present); 'augmented' maps the numeric phase identifier of the receiving phase (the phase with the smallest phase number) to the list of paths added to it; 'augmented_modules' lists each module that received new files, each entry containing the module's phase number, module name, and the added file paths. If the phases.json file originally contained no phases, a single phase numbered 1 is created containing one module whose source_files list contains all paths from required_source_files. The receiving module's description field is extended to include a note referencing the added source files (without duplicating any description content already present). No existing phases are removed, no existing module loses any source files, and phase numbering of pre-existing phases is unchanged.
- **Actual behavior:** If the function completes without raising an exception, the following holds: Let F_before be the JSON contents of the file at phases_json before the call.
- If required_source_files is falsy (None or empty sequence): the function returns {'forced': [], 'augmented': {}, 'augmented_modules': []} and the file is unchanged (F_after = F_before).
- Otherwise, let required_norm = { p with backslashes replaced by forward slashes for p in required_source_files } and existing_norm = { p with backslashes replaced by forward slashes for p in all source_files listed in any module of any phase in F_before }.
 - If required_norm existing_norm, the function returns the same empty dictionary and the file is unchanged.
 - Otherwise, missing = [sf for sf in required_source_files if sf with backslashes replaced not in existing_norm] (preserving order). The file is transformed: the earliest phase (by numeric 'phase' key) in F_before is selected; if no phases exist, a new phase with 'phase': 1, name 'Entry Points', description 'Entry-point source files.', empty modules, empty depends_on_phases is created. In that phase's 'modules' list, if empty, a new module with name 'entry_points', empty description, and empty source_files is appended. The first module in that list has its 'source_files' extended by appending all paths from missing (as given, without normalization) and its 'description' updated to _merge_descriptions(old_description, 'Includes required entry-point source file(s): ' + ', '.join(missing) + '.'). No other phases or modules are modified. The file at phases_json is overwritten with the resulting JSON object (F_after). The function returns: {'forced': missing, 'augmented': {earliest_phase_number: list(missing)}, 'augmented_modules': [{'phase': earliest_phase_number, 'module': name_of_first_module, 'added_files': list(missing)}]} where earliest_phase_number is the 'phase' value of the selected earliest phase. If an exception occurs (e.g., file I/O error, invalid JSON), the function raises an exception and no guarantees about the return value or file state are made.

Formally, for all normal executions: r such that ReturnValue = r ((required_source_files = None required_source_files = []) (r = {'forced': [], augmented: {}, augmented_modules: []} file_unchanged))

(required_source_files None required_source_files [] let req_norm = { normalize(p) | p required_source_files } in let exist_norm = { normalize(q) | q all_file_source_files(F_before) } in (req_norm exist_norm r = {'forced': [], augmented: {}, augmented_modules: []} file_unchanged)

```

    (req_norm exist_norm
      let missing = [ p required_source_files | normalize(p) exist_norm ] in
      let phases_before = F_before.phases (or [] if absent) in
      let earliest_phase = (if phases_before = [] then new_phase(1) else argmin_{pphases_before} p.phase) in
      let modules_after = (if earliest_phase.modules = [] then [new_module('entry_points', '', [])] else earliest_phase.modules) in
      let mod = modules_after[0] in
      let mod_desc_before = mod.description (or '' if absent) in
      let mod_files_before = mod.source_files (or [] if absent) in
      file_after = file_before with earliest_phase replaced by:
        earliest_phase[modules modules_after[0 (mod_files_before ++ missing, description _merge_descriptions(mod_desc_before,
note))]]
      where note = 'Includes required entry-point source file(s): ' + join(missing, ', ') + '.'
      r = {forced: missing, augmented: {earliest_phase.phase: missing}, augmented_modules: [{phase: earliest_phase.phase, module:
mod.name, added_files: missing}]}
      file(phases_json) = file_after
    )

```

) where normalize(s) = replace all backslash characters in s with forward slash.

- **Code evidence:** Line 27: missing = [sf for sf in required_source_files if sf.replace("\", "/") not in listed] Line 28: if not missing: Line 29: return {"forced": [], "augmented": {}, "augmented_modules": []}
- **Trigger condition:** The specification guarantees that after return each required source file appears in exactly one module. When a required file already exists in multiple modules, the code does not detect or fix this duplication; it simply returns early without modification. This leaves the file in a state that violates the post-condition.
- **Validator trigger:** When a required source file already exists in multiple modules, the code returns early without deduplicating, leaving the file in a state that violates the 'exactly one module' post-condition.
- **Probe stdout:** CONFIRMED — actual: forced=[], foo.py appears in 2 modules after call | expected (spec): exactly 1 module

► SPEC sidecar (完整)

```

{
  "signature": "_ensure_source_files_in_phases(phases_json, required_source_files) -> dict",
  "pre_condition": "phases_json is a path to a valid phases.json file whose schema conforms to the phase_03_types specification with phases as an ordered list of objects each containing a numeric 'phase' key and a 'modules' list; required_source_files is None or a sequence of file path strings",
  "post_condition": "When required_source_files is None or empty, returns {'forced': [], 'augmented': {}, 'augmented_modules': []} without reading or modifying phases_json. Otherwise, guarantees that after return every path in required_source_files (with backslash directory separators normalized to forward slash) appears in the source_files list of exactly one module in the phases.json file. Returns a dict where: 'forced' is the list of required paths that were not already present in any phase module's source_files before the call (empty when all paths were already present); 'augmented' maps the numeric phase identifier of the receiving phase (the phase with the smallest phase number) to the list of paths added to it; 'augmented_modules' lists each module that received new files, each entry containing the module's phase number, module name, and the added file paths. If the phases.json file originally contained no phases, a single phase numbered 1 is created containing one module whose source_files list contains all paths from required_source_files. The receiving module's description field is extended to include a note referencing the added source files (without duplicating any description content already present). No existing phases are removed, no existing module loses any source files, and phase numbering of pre-existing phases is unchanged."
}

```

► INFO / callees sidecar (完整)

```

{
  "callees": [
    {
      "name": "_merge_descriptions",
      "signature": "_merge_descriptions(target_desc, source_desc) -> str",
      "pre_condition": "target_desc is a string (an existing description); source_desc is a non-empty string (description text to append)",
      "post_condition": "Returns a string formed by appending source_desc to target_desc. If both are non-empty, a single space separates them. If target_desc already contains the full text of source_desc, source_desc is not appended and target_desc is returned unchanged."
    }
  ]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-
py/_ensure_source_files_in_phases.py",
  "verdict": "MISMATCH",
}

```

```
"gaps": {
  "spec_claim": "When required_source_files is None or empty, returns {'forced': [], 'augmented': {}, 'augmented_modules': []}
without reading or modifying phases_json. Otherwise, guarantees that after return every path in required_source_files (with
backslash directory separators normalized to forward slash) appears in the source_files list of exactly one module in the
phases.json file. Returns a dict where: 'forced' is the list of required paths that were not already present in any phase module's
source_files before the call (empty when all paths were already present); 'augmented' maps the numeric phase identifier of the
receiving phase (the phase with the smallest phase number) to the list of paths added to it; 'augmented_modules' lists each module
that received new files, each entry containing the module's phase number, module name, and the added file paths. If the
phases.json file originally contained no phases, a single phase numbered 1 is created containing one module whose source_files
list contains all paths from required_source_files. The receiving module's description field is extended to include a note
referencing the added source files (without duplicating any description content already present). No existing phases are removed,
no existing module loses any source files, and phase numbering of pre-existing phases is unchanged.",
  "actual_behavior": "If the function completes without raising an exception, the following holds:\nLet F_before be the JSON
contents of the file at phases_json before the call.\n- If required_source_files is falsy (None or empty sequence): the function
returns {'forced': [], 'augmented': {}, 'augmented_modules': []} and the file is unchanged (F_after = F_before).\n- Otherwise, let
required_norm = { p with backslashes replaced by forward slashes for p in required_source_files } and existing_norm = { p with
backslashes replaced by forward slashes for p in all source_files listed in any module of any phase in F_before }.\n - If
required_norm == existing_norm, the function returns the same empty dictionary and the file is unchanged.\n - Otherwise, missing =
[ sf for sf in required_source_files if sf with backslashes replaced not in existing_norm ] (preserving order). The file is
transformed: the earliest phase (by numeric 'phase' key) in F_before is selected; if no phases exist, a new phase with 'phase': 1,
name 'Entry Points', description 'Entry-point source files.', empty modules, empty depends_on_phases is created. In that phase's
'modules' list, if empty, a new module with name 'entry_points', empty description, and empty source_files is appended. The first
module in that list has its 'source_files' extended by appending all paths from missing (as given, without normalization) and its
'description' updated to _merge_descriptions(old_description, 'Includes required entry-point source file(s): ' + ',
'.join(missing) + '.'). No other phases or modules are modified. The file at phases_json is overwritten with the resulting JSON
object (F_after). The function returns:\n {'forced': missing,\n   'augmented': {earliest_phase_number: list(missing)},\n   'augmented_modules': [{ 'phase': earliest_phase_number, 'module': name_of_first_module, 'added_files': list(missing)}]}\n where
earliest_phase_number is the 'phase' value of the selected earliest phase.\nIf an exception occurs (e.g., file I/O error, invalid
JSON), the function raises an exception and no guarantees about the return value or file state are made.\n\nFormally, for all
normal executions:\n r such that ReturnValue = r\n ( (required_source_files = None required_source_files = [])\n (r =
{'forced': [], augmented: {}, augmented_modules: []} file_unchanged) )\n\n ( required_source_files = None required_source_files
= [])\n\n let req_norm = { normalize(p) | p required_source_files } in\n\n let exist_norm = { normalize(q) | q
all_file_source_files(F_before) } in\n\n (req_norm == exist_norm r = {'forced': [], augmented: {}, augmented_modules: []}
file_unchanged)\n\n (req_norm == exist_norm\n\n let missing = [ p required_source_files | normalize(p)
exist_norm ] in\n\n let phases_before = F_before.phases (or [] if absent) in\n\n let earliest_phase = (if
phases_before = [] then new_phase(1) else argmin_{p in phases_before} p.phase) in\n\n let modules_after = (if
earliest_phase.modules = [] then [new_module('entry_points', '', [])] else earliest_phase.modules) in\n\n let mod =
modules_after[0] in\n\n let mod_desc_before = mod.description (or '' if absent) in\n\n let mod_files_before =
mod.source_files (or [] if absent) in\n\n file_after = file_before with earliest_phase replaced by:\n
earliest_phase[modules modules_after[0] (mod_files_before ++ missing, description _merge_descriptions(mod_desc_before, note))]\n
where note = 'Includes required entry-point source file(s): ' + join(missing, ', ') + '.'\n\n r = {'forced': missing,
augmented: {earliest_phase.phase: missing}, augmented_modules: [{phase: earliest_phase.phase, module: mod.name, added_files:
missing}]]}\n\n file(phases_json) = file_after\n\n )\n\n )\n\nwhere normalize(s) = replace all backslash characters in s
with forward slash.",
  "code_evidence": "Line 27: missing = [sf for sf in required_source_files if sf.replace(\"\\\\\\\\\", \"/\") not in listed]\nLine
28: if not missing:\nLine 29:     return {'forced': [], 'augmented': {}, 'augmented_modules': []}",
  "trigger_condition": "The specification guarantees that after return each required source file appears in exactly one module.
When a required file already exists in multiple modules, the code does not detect or fix this duplication; it simply returns early
without modification. This leaves the file in a state that violates the post-condition."
}
```

► Bug Validator result（完整）

```
{
  "id": "src--pipeline_setup-py--ensure_source_files_in_phases",
  "source_file": "src/pipeline_setup-py/_ensure_source_files_in_phases.py",
  "function_name": "_ensure_source_files_in_phases",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--pipeline_setup-py--ensure_source_files_in_phases.py",
  "detail_file": "fm_agent/bug_validation/src--pipeline_setup-py--ensure_source_files_in_phases.md",
  "probe_stdout": "CONFIRMED - actual: forced=[], foo.py appears in 2 modules after call | expected (spec): exactly 1 module",
  "trigger_summary": "When a required source file already exists in multiple modules, the code returns early without
deduplicating, leaving the file in a state that violates the 'exactly one module' post-condition."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:31:29.334546Z	5440
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:32:22.833094Z	7181

事件 1 — [llm_5acae9f41b23475181f8f7ce3856979a](#)

► parsed_json（完整）

```
{
  "post_condition": "If the function completes without raising an exception, the following holds:\nLet F_before be the JSON
contents of the file at phases_json before the call.\n- If required_source_files is falsy (None or empty sequence): the function
returns {'forced': [], 'augmented': {}, 'augmented_modules': []} and the file is unchanged (F_after = F_before).\n- Otherwise, let
required_norm = { p with backslashes replaced by forward slashes for p in required_source_files } and existing_norm = { p with
backslashes replaced by forward slashes for p in all source_files listed in any module of any phase in F_before }.\n  - If
required_norm ⊆ existing_norm, the function returns the same empty dictionary and the file is unchanged.\n  - Otherwise, missing =
[ sf for sf in required_source_files if sf with backslashes replaced not in existing_norm ] (preserving order). The file is
transformed: the earliest phase (by numeric 'phase' key) in F_before is selected; if no phases exist, a new phase with 'phase': 1,
name 'Entry Points', description 'Entry-point source files.', empty modules, empty depends_on_phases is created. In that phase's
'modules' list, if empty, a new module with name 'entry_points', empty description, and empty source_files is appended. The first
module in that list has its 'source_files' extended by appending all paths from missing (as given, without normalization) and its
'description' updated to _merge_descriptions(old_description, 'Includes required entry-point source file(s): ' + ',
'.join(missing) + '.'). No other phases or modules are modified. The file at phases_json is overwritten with the resulting JSON
object (F_after). The function returns:\n  {'forced': missing,\n    'augmented': {earliest_phase_number: list(missing)},\n    'augmented_modules': [{'phase': earliest_phase_number, 'module': name_of_first_module, 'added_files': list(missing)}]}\n  where
earliest_phase_number is the 'phase' value of the selected earliest phase.\nIf an exception occurs (e.g., file I/O error, invalid
JSON), the function raises an exception and no guarantees about the return value or file state are made.\n\nFormally, for all
normal executions:\n∃ r such that ReturnValue = r ∧\n  ( (required_source_files = None ∨ required_source_files = []) →\n    (r =
{forced: [], augmented: {}, augmented_modules: []} ∧ file_unchanged) )\n  ∧\n  ( required_source_files ≠ None ∧
required_source_files ≠ [] →\n    let req_norm = { normalize(p) | p ∈ required_source_files } in\n    let exist_norm = {
normalize(q) | q ∈ all_file_source_files(F_before) } in\n    (req_norm ⊆ exist_norm → r = {forced: [], augmented: {},
augmented_modules: []} ∧ file_unchanged)\n    ∧\n    (req_norm ⊄ exist_norm →\n      let missing = [ p ∈
required_source_files | normalize(p) ∉ exist_norm ] in\n      let phases_before = F_before.phases (or [] if absent) in\n
let earliest_phase = (if phases_before = [] then new_phase(1) else argmin_{p∈phases_before} p.phase) in\n      let
modules_after = (if earliest_phase.modules = [] then [new_module('entry_points', '', [])] else earliest_phase.modules) in\
let mod = modules_after[0] in\n      let mod_desc_before = mod.description (or '' if absent) in\n      let
mod_files_before = mod.source_files (or [] if absent) in\n      file_after = file_before with earliest_phase replaced by:\n
earliest_phase[modules → modules_after[0→ (mod_files_before ++ missing, description → _merge_descriptions(mod_desc_before,
note))]]\n      where note = 'Includes required entry-point source file(s): ' + join(missing, ', ') + '.\n      ∧ r =
{forced: missing, augmented: {earliest_phase.phase: missing}, augmented_modules: [{phase: earliest_phase.phase, module: mod.name,
added_files: missing}]]\n      ∧ file(phases_json) = file_after\n    )\n  )\n\nwhere normalize(s) = replace all backslash
characters in s with forward slash."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5acae9f41b23475181f8f7ce3856979a_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5acae9f41b23475181f8f7ce3856979a_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5acae9f41b23475181f8f7ce3856979a_response.txt)

事件 2 — llm_d43f66766f9349fbbb1078f9455ad60c

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input: phases_json file contains a JSON object with two phases, each containing a module that lists the same
source file 'main.py' in their source_files arrays. Example content: {\"phases\": [{\"phase\": 1, \"modules\": [{\"name\":
\\\"mod1\\\", \\\"source_files\\\": [\\\"main.py\\\"]}]}, {\"phase\": 2, \"modules\": [{\"name\": \\\"mod2\\\", \\\"source_files\\\":
[\\\"main.py\\\"]}]}]. required_source_files is ['main.py']. The function will find 'main.py' already listed (normalized), compute
missing as empty, return {'forced': [], 'augmented': {}, 'augmented_modules': []}, and leave the file unchanged. After the call,
'main.py' still appears in two different modules, violating the specification's requirement that every path in
required_source_files appears in exactly one module.",
  "offending_statements": "Line 27: missing = [sf for sf in required_source_files if sf.replace(\"\\\\\\\\\\\\\\\\\", \"\\\\/\") not in
listed]\nLine 28: if not missing:\nLine 29:     return {\"forced\": [], \"augmented\": {}, \"augmented_modules\": []}",
  "reason": "The specification guarantees that after return each required source file appears in exactly one module. When a
required file already exists in multiple modules, the code does not detect or fix this duplication; it simply returns early
without modification. This leaves the file in a state that violates the post-condition."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d43f66766f9349fbbb1078f9455ad60c_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d43f66766f9349fbbb1078f9455ad60c_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d43f66766f9349fbbb1078f9455ad60c_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-139 — src--pipeline_setup-py--_run_generate_phases

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator：confirmed；attempts 1
- Phase / 模块：Phase 3 — Pipeline Planning & Domain Knowledge / pipeline_setup；原源码 src/pipeline_setup.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** If the function returns normally (without exiting the process): either `plugin_stage.type` is 'pass' or 'replace' (the plugin assumes full responsibility for `phases.json` output), or `phases.json` exists at the expected path under `work_dir` containing a list of phases where each phase has a phase number (integer), name (string), description (string), modules (a list of objects each with name, description, and `source_files`), and `depends_on_phases` (a list of integers referencing other phase numbers). In incremental mode, the existing `phases.json` is updated to reflect the current source code state by adding new modules and source files that have appeared, removing entries whose files no longer exist, and adjusting phase assignments as needed while preserving still-accurate entries. In submodule-filtered mode, `source_files` entries reference only files within the specified subdirectory paths. If after `OPENCODE_MAX_RETRIES` attempts `phases.json` is missing, has schema validation errors, or in submodule mode fails to cover current sources under the specified subdirectories, the process exits with code 1 and emits a diagnostic message describing the failure.

- **Actual behavior:** After the code block executes, one of the following holds:

1. (Termination) If `phase_plan_ready` (as assigned in lines 121130) is `False` and `attempt >= OPENCODE_MAX_RETRIES`, the program calls `sys.exit(1)` and terminates; no further statements are executed.
2. (Exception) If `phase_plan_ready` is `True` and the `break` is taken, and then the condition `plugin_stage is not None and plugin_stage.type == "modify" and plugin_stage.output_process` holds, and the command executed by `run_plugin_command` exits with a nonzero code, a `CalledProcessError` is raised; the block does not complete normally.
3. (Normal completion) Otherwise the block finishes normally without terminating the program or raising an exception. In this case:
 - If `phase_plan_ready` is `True`: the enclosing loop has been exited (via `break`). Afterwards, if `plugin_stage is not None and plugin_stage.type == "modify" and plugin_stage.output_process` is true, the shell command `plugin_stage.output_process` was executed by `run_plugin_command` with `cwd=plugin_root`, environment variable `FM_AGENT_PLUGIN_ROOT` set to `plugin_root`, and label "`generate_phase_plan post-process`", and completed successfully (exit code 0). If that condition is false, no plugin postprocess ran.
 - If `phase_plan_ready` is `False` and `attempt < OPENCODE_MAX_RETRIES`: a warning message containing the computed `missing` reason was logged via `logging.warning` and printed to stdout; `time.sleep(10)` completed. The loop was not broken, so control will return to the top of the outer loop for the next iteration (outside this block).
 - All other program state brought into the block remains unchanged: `prompt`, `command`, `attempt`, `proj_dir`, `submodules` (if present), `is_incremental`, `prev_mtime`, `phases_json`, `phase_plan_errors`, `OPENCODE_MAX_RETRIES`, `plugin_root`, `plugin_stage`. The trace directory still contains an event for stage '`generate_phases_json`' with metadata `{'attempt': attempt}`.

Formally, let `ready_before = False`, and let `ready_after` be defined as:

- `submodules` and `_phases_cover_current_sources(phases_json, proj_dir, submodules)` OR
- `(not submodules)` and `is_incremental` and `(os.path.getmtime(phases_json) != prev_mtime or _phases_cover_current_sources(phases_json, proj_dir))` OR
- `(not submodules)` and `(not is_incremental)`. Define `exit_attempt = not ready_after and attempt >= OPENCODE_MAX_RETRIES`. Define `plugin_condition = (plugin_stage is not None and plugin_stage.type == "modify" and plugin_stage.output_process)`. Then the postcondition for normal completion is: `exit_attempt (ready_after (loop_broken (plugin_condition (run_plugin_command_succeeded no_exception))) ) (ready_after (attempt < OPENCODE_MAX_RETRIES logged_warning slept_10s loop_broken))` with all other variables unchanged as described.
- **Code evidence:** Line 125: `phase_plan_ready = (` Line 126: `os.path.getmtime(phases_json) != prev_mtime` Line 127: `or _phases_cover_current_sources(phases_json, proj_dir)` Line 128: `)`
- **Trigger condition:** In incremental mode the specification demands that the updated `phases.json` reflect the current source state. The code uses an mtime-based check as an alternative to the coverage check, so a mere rewrite of the file (even with incomplete coverage) is accepted as ready, violating the required guarantee.
- **Validator trigger:** In incremental mode, the mtime-based OR check at lines 1034-1037 accepts a `phases.json` rewrite even when coverage is incomplete, bypassing the required `_phases_cover_current_sources` verification.
- **Probe stdout:** CONFIRMED — `mtime_changed=True`, `coverage_ok=False`, `buggy_phase_plan_ready=True`, `expected_phase_plan_ready=False`

► SPEC sidecar (完整)

```
{
  "signature": "_run_generate_phases(proj_dir, work_dir, script_dir, is_incremental=False, resume=False, submodules=None,
plugin_stage=None, plugin_root=None)",
  "pre_condition": "proj_dir is an existing directory containing the project source tree; work_dir is a writable directory for
pipeline outputs; script_dir is an existing directory containing FM-Agent scripts; is_incremental and resume are booleans;
submodules, if not None, is a sequence of subdirectory paths within proj_dir; plugin_stage is a PluginStageConfig or None;
plugin_root is a Path or None",
  "post_condition": "If the function returns normally (without exiting the process): either plugin_stage.type is 'pass' or
'replace' (the plugin assumes full responsibility for phases.json output), or phases.json exists at the expected path under
work_dir containing a list of phases where each phase has a phase number (integer), name (string), description (string), modules
(a list of objects each with name, description, and source_files), and depends_on_phases (a list of integers referencing other
```


phase numbers). In incremental mode, the existing `phases.json` is updated to reflect the current source code state by adding new modules and source files that have appeared, removing entries whose files no longer exist, and adjusting phase assignments as needed while preserving still-accurate entries. In submodule-filtered mode, `source_files` entries reference only files within the specified subdirectory paths. If after `OPENCODE_MAX_RETRIES` attempts `phases.json` is missing, has schema validation errors, or in submodule mode fails to cover current sources under the specified subdirectories, the process exits with code 1 and emits a diagnostic message describing the failure."

```
}

```

```
{
  "callees": [
    {
      "name": "build_llm_cli_command",
      "signature": "build_llm_cli_command(model, prompt, cwd, files) -> str",
      "pre_condition": "model is a string identifying the LLM model to use; prompt is a string containing the natural-language instruction; cwd is a valid directory path; files is a list of absolute file paths that exist and are readable",
      "post_condition": "Returns a CLI command string that, when executed in the environment, invokes the configured LLM backend with the given model, prompt, and file attachments"
    },
    {
      "name": "run_opencode_traced",
      "signature": "run_opencode_traced(proj_dir, work_dir, command, stage, input_files, output_files, summary, metadata)",
      "pre_condition": "proj_dir is an existing project directory; work_dir is a writable directory; command is a valid CLI command string; stage is a string identifying the pipeline stage; input_files is a list of file paths relative to proj_dir that exist and are readable; output_files is a list of file paths relative to proj_dir that the invoked process is expected to produce",
      "post_condition": "Returns normally when the command executes. Upon return, output_files are expected to exist at their specified paths under proj_dir if the LLM-based process completed successfully; structured trace events are written to the trace directory"
    },
    {
      "name": "run_plugin_command",
      "signature": "run_plugin_command(cmd, plugin_root, proj_dir, label)",
      "pre_condition": "cmd is a non-empty string specifying a shell command; plugin_root is a valid directory path; proj_dir is an existing directory; label is a string identifying the operation for log messages",
      "post_condition": "Executes the command with the working directory set to plugin_root and FM_AGENT_PLUGIN_ROOT set to plugin_root in the subprocess environment. If the command exits with a non-zero code, raises CalledProcessError"
    }
  ]
}
```

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-py/_run_generate_phases.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "If the function returns normally (without exiting the process): either plugin_stage.type is 'pass' or 'replace' (the plugin assumes full responsibility for phases.json output), or phases.json exists at the expected path under work_dir containing a list of phases where each phase has a phase number (integer), name (string), description (string), modules (a list of objects each with name, description, and source_files), and depends_on_phases (a list of integers referencing other phase numbers). In incremental mode, the existing phases.json is updated to reflect the current source code state by adding new modules and source files that have appeared, removing entries whose files no longer exist, and adjusting phase assignments as needed while preserving still-accurate entries. In submodule-filtered mode, source_files entries reference only files within the specified subdirectory paths. If after OPENCODE_MAX_RETRIES attempts phases.json is missing, has schema validation errors, or in submodule mode fails to cover current sources under the specified subdirectories, the process exits with code 1 and emits a diagnostic message describing the failure.",
    "actual_behavior": "After the code block executes, one of the following holds:\n1. (Termination) If `phase_plan_ready` (as assigned in lines 121130) is `False` and `attempt >= OPENCODE_MAX_RETRIES`, the program calls `sys.exit(1)` and terminates; no further statements are executed.\n2. (Exception) If `phase_plan_ready` is `True` and the `break` is taken, and then the condition `plugin_stage` is not None and plugin_stage.type == `\"modify\"` and plugin_stage.output_process` holds, and the command executed by `run_plugin_command` exits with a nonzero code, a `CalledProcessError` is raised; the block does not complete normally.\n3. (Normal completion) Otherwise the block finishes normally without terminating the program or raising an exception. In this case:\n- If `phase_plan_ready` is `True`: the enclosing loop has been exited (via `break`). Afterwards, if `plugin_stage` is not None and plugin_stage.type == `\"modify\"` and plugin_stage.output_process` is true, the shell command `plugin_stage.output_process` was executed by `run_plugin_command` with `cwd=plugin_root`, environment variable `FM_AGENT_PLUGIN_ROOT` set to `plugin_root`, and label `\"generate_phase_plan post-process\"`, and completed successfully (exit code 0). If that condition is false, no plugin postprocess ran.\n- If `phase_plan_ready` is `False` and `attempt < OPENCODE_MAX_RETRIES`: a warning message containing the computed `missing` reason was logged via `logging.warning` and printed to stdout; `time.sleep(10)` completed. The loop was not broken, so control will return to the top of the outer loop for the next iteration (outside this block).\n- All other program state brought into the block remains unchanged: `prompt`, `command`, `attempt`, `proj_dir`, `submodules` (if present), `is_incremental`, `prev_mtime`, `phases_json`, `phase_plan_errors`, `OPENCODE_MAX_RETRIES`, `plugin_root`, `plugin_stage`. The trace directory still contains an event for stage `generate_phases_json` with metadata `{ 'attempt': attempt }`.\n\nFormally, let `ready_before` = False, and let `ready_after` be defined as:\n- `submodules` and `_phases_cover_current_sources(phases_json, proj_dir, submodules)` OR\n- `(not submodules)` and `is_incremental` and `(os.path.getmtime(phases_json) != prev_mtime or _phases_cover_current_sources(phases_json, proj_dir))` OR\n- `(not submodules)` and `(not is_incremental)`.\n\nDefine `exit_attempt` = not ready_after and attempt >= OPENCODE_MAX_RETRIES.\n\nDefine `plugin_condition` = (plugin_stage is not None and plugin_stage.type == `\"modify\"` and plugin_stage.output_process).\n\nThen the postcondition for normal completion is:\nnext_attempt (ready_after (loop_broken (plugin_condition (run_plugin_command_succeeded no_exception))) ) (ready_after (attempt < OPENCODE_MAX_RETRIES logged_warning slept_10s loop_broken)))\n\nwith all other variables unchanged as described.",
    "code_evidence": "Line 125: phase_plan_ready = (\\nLine 126: os.path.getmtime(phases_json)"
  }
}
```



```

    "post_condition": "If any operation in the block raises an exception (e.g., import error, file I/O error, CalledProcessError
from run_plugin_command), the function does not complete normally and the exception propagates. Otherwise, the block executes
without exceptions, and one of the following scenarios holds after its completion (return or end-of-block):\n\n1. Early return
(pass): If plugin_stage \u2260 None \u2217 plugin_stage.type = 'pass', a message is printed and the function returns immediately. No
variables phases_json, prev_mtime, phase_plan_errors, _resume_skip are bound; no file modifications occur.\n\n2. Early return
(replace): If plugin_stage \u2260 None \u2217 plugin_stage.type = 'replace', a message is printed, then
run_plugin_command(plugin_stage.replace_cmd, plugin_root, proj_dir, label='generate_phase_plan') is invoked. Provided the command
exits with code 0, the function returns. If the command fails, CalledProcessError propagates.\n\n3. Flow-through (all other
cases): The function does not return inside the block; execution reaches line 40 and continues with the following local state:\n
- phases_json = os.path.join(work_dir, 'phases.json')\n
- prev_mtime = mtime of phases_json if exists else None\n
- phase_plan_errors = _phase_plan_schema_errors(phases_json) if exists(phases_json) else []\n
- _resume_skip = (resume \u2217
_phase_plan_complete(work_dir))\n
- If _resume_skip is true, a resume message is printed.\n
- If plugin_stage \u2260 None \u2217
plugin_stage.type = 'modify' \u2217 bool(plugin_stage.input_md):\n
    workflow_src = str(plugin_root / plugin_stage.input_md)\n
workflow_dst = os.path.join(work_dir, 'workflow_generate_phases.md')\n
    shutil.copy2(workflow_src, workflow_dst)
(successful copy)\n
    user_knowledge_paths = list_staged_domain_knowledge_relpaths(work_dir)\n
    If user_knowledge_paths
\u2260 []:\n
    The file workflow_dst is appended with a section containing:\n
    - a separator '\n\n---\n\n'\n
- a heading '\n\n## User-Provided Domain Knowledge\n\n'\n
- instructional text\n
- the formatted bullet
list of user_knowledge_paths (via format_domain_knowledge_bullets).\n
- Otherwise (plugin_stage is None, type \u2260 'modify', or
input_md is falsy), no copy or augmentation occurs.\n
- All above local variables are defined, and no other filesystem side
effects are produced.\n\n4. Normally:\n\npost-condition \u2260\n
    ( plugin_stage \u2260 None \u2217 plugin_stage.type =
'pass' \u2217 printed('...skip...') \u2217 return() )\n
    \u2217 (plugin_stage \u2260 None \u2217 plugin_stage.type = 'replace' \u2217 printed('...running...') \u2217
run_plugin_command_success(plugin_stage.replace_cmd, plugin_root, proj_dir) \u2217 return() )\n
    \u2217 ( \u2217(plugin_stage \u2260 None \u2217
plugin_stage.type = 'pass') \u2217 (plugin_stage \u2260 None \u2217 plugin_stage.type = 'replace') ) \u2217
    phases_json = join(work_dir,
'phases.json') \u2217
    prev_mtime = (exists(phases_json) ? mtime(phases_json) : None) \u2217
    phase_plan_errors =
(exist(phases_json) ? schema_errors(phases_json) : []) \u2217
    _resume_skip = (resume \u2217 phase_plan_complete(work_dir)) \u2217\n
    (_resume_skip \u2217 printed('RESUME ...')) \u2217
    [ (plugin_stage \u2260 None \u2217 plugin_stage.type = 'modify' \u2217
truthy(plugin_stage.input_md)) \u2217
    workflow_src = str(plugin_root / plugin_stage.input_md) \u2217
    workflow_dst
= join(work_dir, 'workflow_generate_phases.md') \u2217
    copy2_success(workflow_src, workflow_dst) \u2217\n
user_knowledge_paths = list_staged_domain_knowledge_relpaths(work_dir) \u2217
    (user_knowledge_paths \u2260 [] \u2217
file_appended(workflow_dst, knowledge_markdown(user_knowledge_paths)) )\n
    ; else True\n
    ]\n
    )\n

```

```
)\n\n(Where truthy(x) means x is not None and not empty string, etc.)"
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_cc1db75bff17408da24e55fbae7555ea_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_cc1db75bff17408da24e55fbae7555ea_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_cc1db75bff17408da24e55fbae7555ea_response.txt)

事件 2 — llm_6da3a5fa7b6a424f9aed0aeaa1e7d8ac

► parsed_json (完整)

```
{
  "verdict": "MATCH",
  "counterexample": null,
  "offending_statements": null,
  "reason": ""
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_6da3a5fa7b6a424f9aed0aeaa1e7d8ac_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_6da3a5fa7b6a424f9aed0aeaa1e7d8ac_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_6da3a5fa7b6a424f9aed0aeaa1e7d8ac_response.txt)

事件 3 — llm_f7c282e3e0b84015bdab97be24ef1541

► parsed_json (完整)

```
{
  "post_condition": "The code block executes without raising an exception. The else branch is active.
_prepare_workflow_file(proj_dir, work_dir, script_dir, \"workflow_generate_phases.md\") is called and completes successfully,
placing or updating the workflow file in work_dir. Three string variables are bound: fm_reminder = \"IMPORTANT: The fm_agent/
directory is NOT part of the project source code. It is a workspace for storing your output files only. Do NOT include fm_agent/
paths in phases.json. Do NOT modify any existing project files.\"; incremental_reminder = \"IMPORTANT: An existing
fm_agent/phases.json from a previous run is already present. Do NOT regenerate it from scratch. Instead, inspect the current state
of the source code and UPDATE the existing fm_agent/phases.json so it reflects the current version of the code: add modules and
source files that are new, remove entries whose files no longer exist, and adjust phases as needed. Preserve entries that are
still accurate.\"; submodule_reminder is \"\" if submodules is falsy, otherwise it contains a list of allowed subdirectories as
per lines 55-60. A for loop over attempt = 1 .. OPENCODE_MAX_RETRIES begins. If _resume_skip is true, break is executed
immediately; control passes to the next statement after the loop. Otherwise, the loop runs; inside each iteration a prompt is
built according to the rules: for the first attempt when resume is false a specific initial prompt is used, else a retry prompt;
is_incremental extends it with incremental_reminder; if phase_plan_errors is non-empty, schema repair instructions are included
(the construction of schema_repair_prompt starts at line 79 and is assumed to be completed in the full loop body). Subsequently, a
CLI command is built from that prompt (via build_llm_cli_command) and executed via run_opencode_traced, which invokes the LLM to
generate or update phases.json and possibly other files in work_dir. After the loop (either by break or exhaustion of attempts),
the function returns or continues to its next statement. No other persistent side-effects occur within this block."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f7c282e3e0b84015bdab97be24ef1541_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f7c282e3e0b84015bdab97be24ef1541_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f7c282e3e0b84015bdab97be24ef1541_response.txt)

事件 4 — llm_4ecf75fd81f745bc96fa5d6c8a123536

► parsed_json (完整)

```
{
  "post_condition": "After the code block executes, the program state satisfies: (a) The variable `prompt` contains the full
string that includes the schema repair prompt iff the previous value of `phase_plan_errors` was non-empty. (b) `command` is a CLI
command string built by `build_llm_cli_command` with the given model, prompt, cwd, and files (including `prompt_file`). (c)
`run_opencode_traced` has been invoked; if the command exited with a non-zero code, a warning was logged via `logging.warning` and
no exception propagated. (d) The trace directory contains an event for the stage `generate_phases_json` with metadata
`{'attempt': attempt}`. (e) The file `fm_agent/phases.json` may or may not exist; `phase_plan_errors` is bound to the list
returned by `_phase_plan_schema_errors(phases_json)` if the file exists, otherwise to `['phases.json is missing']`. (f)
`phase_plan_ready` is bound to `False`. (g) If `phase_plan_errors` is empty and `submodules` is truthy, control flow has entered
the body of the `if` statement at line 120; otherwise, that body has been skipped. (h) The loop variable `attempt` has not been
modified."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4ecf75fd81f745bc96fa5d6c8a123536_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4ecf75fd81f745bc96fa5d6c8a123536_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4ecf75fd81f745bc96fa5d6c8a123536_response.txt)

► [parsed_json](#) (完整)

```
{
  "post_condition": "After the code block executes, one of the following holds:\n1. (Termination) If `phase_plan_ready` (as assigned in lines 121-130) is `False` and `attempt >= OPENCODE_MAX_RETRIES`, the program calls `sys.exit(1)` and terminates; no further statements are executed.\n2. (Exception) If `phase_plan_ready` is `True` and the `break` is taken, and then the condition `plugin_stage` is not None and `plugin_stage.type == \"modify\"` and `plugin_stage.output_process` holds, and the command executed by `run_plugin_command` exits with a non-zero code, a `CalledProcessError` is raised; the block does not complete normally.\n3. (Normal completion) Otherwise the block finishes normally without terminating the program or raising an exception. In this case:\n- If `phase_plan_ready` is `True`: the enclosing loop has been exited (via `break`). Afterwards, if `plugin_stage` is not None and `plugin_stage.type == \"modify\"` and `plugin_stage.output_process` is true, the shell command `plugin_stage.output_process` was executed by `run_plugin_command` with `cwd=plugin_root`, environment variable `FM_AGENT_PLUGIN_ROOT` set to `plugin_root`, and label `\"generate_phase_plan post-process\"`, and completed successfully (exit code 0). If that condition is false, no plugin post-process ran.\n- If `phase_plan_ready` is `False` and `attempt < OPENCODE_MAX_RETRIES`: a warning message containing the computed `missing` reason was logged via `logging.warning` and printed to stdout; `time.sleep(10)` completed. The loop was not broken, so control will return to the top of the outer loop for the next iteration (outside this block).\n- All other program state brought into the block remains unchanged: `prompt`, `command`, `attempt`, `proj_dir`, `submodules` (if present), `is_incremental`, `prev_mtime`, `phases_json`, `phase_plan_errors`, `OPENCODE_MAX_RETRIES`, `plugin_root`, `plugin_stage`. The trace directory still contains an event for stage `\"generate_phases_json\"` with metadata `{\"attempt\": attempt}`.\n\nFormally, let `ready_before = False`, and let `ready_after` be defined as:\n- `submodules` and `_phases_cover_current_sources(phases_json, proj_dir, submodules)` OR\n- `(not submodules) and is_incremental and (os.path.getmtime(phases_json) != prev_mtime or _phases_cover_current_sources(phases_json, proj_dir))` OR\n- `(not submodules) and (not is_incremental)`.\n\nDefine `exit_attempt = not ready_after and attempt >= OPENCODE_MAX_RETRIES`. Define `plugin_condition = (plugin_stage is not None and plugin_stage.type == \"modify\" and plugin_stage.output_process)`.\n\nThen the post-condition for normal completion is:\n- `exit_attempt` ( `ready_after` => `(loop_broken` ( `plugin_condition` => `(run_plugin_command_succeeded` ( `no_exception`))) ) ( `ready_after` => `(attempt < OPENCODE_MAX_RETRIES` ( `logged_warning` ( `slept_10s` ( `loop_broken`)))`)\n\nwith all other variables unchanged as described."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_9fc18aae2bbc4c33ac4798500f5ae073_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_9fc18aae2bbc4c33ac4798500f5ae073_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_9fc18aae2bbc4c33ac4798500f5ae073_response.txt)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Set is_incremental=True, submodules=False, prev_mtime to an old timestamp, and provide a phases.json file that was just rewritten by the LLM (so its mtime differs from prev_mtime) but whose content fails _phases_cover_current_sources (e.g., a new source file appeared that is not referenced). Ensure phase_plan_errors is empty so that the readiness block is reached. The code will then set phase_plan_ready = True because os.path.getmtime(phases_json) != prev_mtime, even though _phases_cover_current_sources returns False. The loop breaks, and the stage finishes without requiring an up-to-date phases.json.",
  "offending_statements": "Line 125:           phase_plan_ready = (\nLine 126: os.path.getmtime(phases_json) != prev_mtime\nLine 127:           or _phases_cover_current_sources(phases_json, proj_dir)\nLine 128:           )",
  "reason": "In incremental mode the specification demands that the updated phases.json reflect the current source state. The code uses an mtime-based check as an alternative to the coverage check, so a mere rewrite of the file (even with incomplete coverage) is accepted as ready, violating the required guarantee."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_006a771e44824bb1984db30e0954f619_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_006a771e44824bb1984db30e0954f619_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_006a771e44824bb1984db30e0954f619_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: [plugins](#)

- 四类结论: 可能有 Bug
- 分类依据: probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: [confirmed](#); attempts 1
- Phase / 模块: Phase 3 — Pipeline Planning & Domain Knowledge / [plugins](#); 原源码 [src/plugin.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

- **SPEC claim:** Executes cmd as a shell subprocess. Relative file paths in cmd are interpreted relative to plugin_root. The subprocess runs with working directory set to plugin_root and the environment variable FM_AGENT_PLUGIN_ROOT set to the string representation of plugin_root. If the command exits with a non-zero code, a diagnostic message identifying the label, the command string, and the exit code is printed to stdout, then subprocess.CalledProcessError is raised. If the command exits with code zero, the function returns without error.
- **Actual behavior:** The function either returns None after the resolved shell command (with relative paths resolved under plugin_root) ran successfully (exit code 0) in directory proj_dir with FM_AGENT_PLUGIN_ROOT set to plugin_root and no error message was printed, or it raises a subprocess.CalledProcessError exception after printing an error message containing the label, return code, and resolved command to stdout. In both cases no other program state is modified. Formal: (result = None resolved_cmd = _resolve_command(cmd, plugin_root) subprocess_success(resolved, proj_dir, plugin_root) printed_error) (exception = CalledProcessError resolved_cmd = _resolve_command(cmd, plugin_root) subprocess_failed(resolved, proj_dir, plugin_root, exit_code) printed_error(resolved, exit_code, label) exit_code 0)
- **Code evidence:** Line 11: subprocess.run(resolved, shell=True, check=True, cwd=proj_dir, env=env)
- **Trigger condition:** The specification requires the subprocess to run with working directory set to plugin_root, but the code sets cwd=proj_dir. For any input where proj_dir != plugin_root, the working directory differs, violating the specification. For example, with the given input, the command 'pwd' would output '/home/user' instead of '/tmp/plugin'.
- **Validator trigger:** The bug is triggered when proj_dir != plugin_root; subprocess.run() receives cwd=proj_dir instead of the spec-required cwd=plugin_root.
- **Probe stdout:** hello CONFIRMED — actual cwd: '/tmp/probe_p41_nvvx/proj_dir' | expected cwd: '/tmp/probe_p41_nvvx/plugin_root'

► SPEC sidecar（完整）

```
{
  "signature": "run_plugin_command(cmd: str, plugin_root: Path, proj_dir: str, label: str = '') -> None",
  "pre_condition": "cmd is a non-empty string specifying a shell command; plugin_root is a valid directory path; proj_dir is an existing directory; label is a string identifying the operation for log messages.",
  "post_condition": "Executes cmd as a shell subprocess. Relative file paths in cmd are interpreted relative to plugin_root. The subprocess runs with working directory set to plugin_root and the environment variable FM_AGENT_PLUGIN_ROOT set to the string representation of plugin_root. If the command exits with a non-zero code, a diagnostic message identifying the label, the command string, and the exit code is printed to stdout, then subprocess.CalledProcessError is raised. If the command exits with code zero, the function returns without error."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/plugin-py/run_plugin_command.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Executes cmd as a shell subprocess. Relative file paths in cmd are interpreted relative to plugin_root. The subprocess runs with working directory set to plugin_root and the environment variable FM_AGENT_PLUGIN_ROOT set to the string representation of plugin_root. If the command exits with a non-zero code, a diagnostic message identifying the label, the command string, and the exit code is printed to stdout, then subprocess.CalledProcessError is raised. If the command exits with code zero, the function returns without error.",
    "actual_behavior": "The function either returns None after the resolved shell command (with relative paths resolved under plugin_root) ran successfully (exit code 0) in directory proj_dir with FM_AGENT_PLUGIN_ROOT set to plugin_root and no error message was printed, or it raises a subprocess.CalledProcessError exception after printing an error message containing the label, return code, and resolved command to stdout. In both cases no other program state is modified. Formal: (result = None resolved_cmd = _resolve_command(cmd, plugin_root) subprocess_success(resolved, proj_dir, plugin_root) printed_error) (exception = CalledProcessError resolved_cmd = _resolve_command(cmd, plugin_root) subprocess_failed(resolved, proj_dir, plugin_root, exit_code) printed_error(resolved, exit_code, label) exit_code 0)",
    "code_evidence": "Line 11: subprocess.run(resolved, shell=True, check=True, cwd=proj_dir, env=env)",
    "trigger_condition": "The specification requires the subprocess to run with working directory set to plugin_root, but the code sets cwd=proj_dir. For any input where proj_dir != plugin_root, the working directory differs, violating the specification. For example, with the given input, the command 'pwd' would output '/home/user' instead of '/tmp/plugin'."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--plugin-py--run_plugin_command",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/plugin-py/run_plugin_command.py",
```

```
"function_name": "run_plugin_command",
"confirmation_status": "confirmed",
"attempts": "1",
"probe_script": "fm_agent/bug_validation/probe_src--plugin-py--run_plugin_command.py",
"detail_file": "fm_agent/bug_validation/src--plugin-py--run_plugin_command.md",
"probe_stdout": "hello\nCONFIRMED - actual cwd: '/tmp/probe_p41_nvvx/proj_dir' | expected cwd:
'/tmp/probe_p41_nvvx/plugin_root'",
"trigger_summary": "The bug is triggered when proj_dir != plugin_root; subprocess.run() receives cwd=proj_dir instead of the
spec-required cwd=plugin_root."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:32:29.839334Z	2121
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:32:53.053572Z	2546

事件 1 — [llm_814565bf127348fea85a2f95ff326b31](#)

► parsed_json（完整）

```
{
  "post_condition": "The function either returns None after the resolved shell command (with relative paths resolved under
plugin_root) ran successfully (exit code 0) in directory proj_dir with FM_AGENT_PLUGIN_ROOT set to plugin_root and no error
message was printed, or it raises a subprocess.CalledProcessError exception after printing an error message containing the label,
return code, and resolved command to stdout. In both cases no other program state is modified. Formal: (result = None ^
resolved_cmd = _resolve_command(cmd, plugin_root) ^ subprocess_success(resolved, proj_dir, plugin_root) ^ ~printed_error) v
(exception = CalledProcessError ^ resolved_cmd = _resolve_command(cmd, plugin_root) ^ subprocess_failed(resolved, proj_dir,
plugin_root, exit_code) ^ printed_error(resolved, exit_code, label) ^ exit_code != 0)"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_814565bf127348fea85a2f95ff326b31_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_814565bf127348fea85a2f95ff326b31_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_814565bf127348fea85a2f95ff326b31_response.txt)

事件 2 — [llm_687f0f8731bc45e583ba66828a409bc6](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "cmd='pwd', plugin_root=Path('/tmp/plugin'), proj_dir='/home/user', label='',
  "offending_statements": "Line 11: subprocess.run(resolved, shell=True, check=True, cwd=proj_dir, env=env)",
  "reason": "The specification requires the subprocess to run with working directory set to plugin_root, but the code sets
cwd=proj_dir. For any input where proj_dir != plugin_root, the working directory differs, violating the specification. For
example, with the given input, the command 'pwd' would output '/home/user' instead of '/tmp/plugin'."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_687f0f8731bc45e583ba66828a409bc6_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_687f0f8731bc45e583ba66828a409bc6_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_687f0f8731bc45e583ba66828a409bc6_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

Phase 4 — LLM Communication & Tracing

模块: llm_client

FMA-MISMATCH-159 — [src--llm_client-py--_parse_json_response](#)

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: **confirmed**; attempts 1
- Phase / 模块：Phase 4 — LLM Communication & Tracing / [llm_client](#); 原源码 [src/llm_client.py](#)

冲突摘要

- **SPEC claim:** When response contains exactly one JSON object or array determined by structural parsing that recognizes balanced JSON bracket pairs at any nesting depth returns that value as a dict or list respectively. Raises ValueError when: (a) response is not a string; (b) no JSON object or array exists anywhere in the text; (c) more than one JSON object or array exists in the text; (d) a direct full-string parse succeeds but produces a non-structured JSON scalar (string, number, boolean, or null) instead of an object or array. The extraction scans the full text left-to-right; structural parsing means that non-JSON text containing '{' or '[' characters without balanced JSON structure does not produce a match.
- **Actual behavior:** After execution, if `response` is not a string, a `ValueError` with message 'LLM response must be a JSON string' is raised. Otherwise, let `text = response.strip()`. If `json.loads(text)` succeeds and returns a value `data` that is a `dict` or `list`, the function returns `data`. If `json.loads(text)` succeeds but `data` is not a `dict` or `list`, a `ValueError` with message 'LLM response must contain a JSON object or array' is raised. If `json.loads(text)` raises a `JSONDecodeError` (call it `exc`), the function scans `text` using a JSON decoder to find all valid JSON objects and arrays in sequential order. Let `values` be the list of these found values. If `len(values)` is exactly 1, returns `values[0]`. If `len(values) > 1`, raises `ValueError` with message 'LLM response contains multiple JSON values'. If `len(values) == 0`, raises `ValueError` with message 'LLM response is not valid JSON: {exc}' chained from `exc`.

Formally, where `is_str(x)` holds if `x` is a string, `full_parse(s)` returns the parsed value if `json.loads(s)` succeeds, or an error otherwise, `is_dict_or_list(v)` holds if `v` is a dict or list, and `scan(s)` returns the ordered list of dict/list values extracted by the scanning algorithm:

```
post_condition ( is_str(response) Raise(ValueError("LLM response must be a JSON string")) ) ( is_str(response) let t = strip(response) in (
(full_parse(t) = v is_dict_or_list(v)) Return(v) ) (full_parse(t) = v is_dict_or_list(v)) Raise(ValueError("LLM response must contain a JSON object
or array")) ) (full_parse(t) = (error) let exc = error in ( |scan(t)| = 1 Return(scan(t)[0]) ) ( |scan(t)| > 1 Raise(ValueError("LLM response contains
multiple JSON values")) ) ( |scan(t)| = 0 Raise(ValueError(f"LLM response is not valid JSON: {exc}")) from exc ) )
```

- **Code evidence:** Line 17: `object_start = text.find("{", index)` Line 18: `array_start = text.find("[", index)` Line 22: `start = min(starts)` Line 24: `data, end = decoder.raw_decode(text, start)` Line 28: `if isinstance(data, (dict, list)): values.append(data)`
- **Trigger condition:** The scanning logic finds any '{' or '[' character without considering whether it is inside a JSON string, causing it to extract a valid JSON object that appears inside a quoted string. The specification requires structural parsing that respects string boundaries, so a JSON object embedded in a string should not be considered a match. In this counterexample, the code returns `{'a': 1}` while the specification would raise a `ValueError` because no standalone JSON object/array exists.
- **Validator trigger:** The scanning logic finds '{' or '[' without checking if inside a JSON string, so a JSON object embedded in a quoted string is incorrectly extracted as a standalone value.
- **Probe stdout:** CONFIRMED — bug reproduced: `actual={'a': 1} | expected='ValueError (no standalone JSON object/array)'`

► SPEC sidecar (完整)

```
{
  "signature": "_parse_json_response(response) -> dict | list",
  "pre_condition": "response is a string that may contain zero, one, or more JSON objects or arrays, possibly embedded within
markdown code fences or interleaved with explanatory prose.",
  "post_condition": "When response contains exactly one JSON object or array – determined by structural parsing that recognizes
balanced JSON bracket pairs at any nesting depth – returns that value as a dict or list respectively. Raises ValueError when: (a)
response is not a string; (b) no JSON object or array exists anywhere in the text; (c) more than one JSON object or array exists
in the text; (d) a direct full-string parse succeeds but produces a non-structured JSON scalar (string, number, boolean, or null)
instead of an object or array. The extraction scans the full text left-to-right; structural parsing means that non-JSON text
containing '{' or '[' characters without balanced JSON structure does not produce a match."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/llm_client-py/_parse_json_response.py",
  "verdict": "MISMATCH",
  "gaps": {
```



```
"spec_claim": "When response contains exactly one JSON object or array determined by structural parsing that recognizes balanced JSON bracket pairs at any nesting depth returns that value as a dict or list respectively. Raises ValueError when: (a) response is not a string; (b) no JSON object or array exists anywhere in the text; (c) more than one JSON object or array exists in the text; (d) a direct no-string parse succeeds but produces a non-structured JSON scalar (string, number, boolean, or null) instead of an object or array. The extraction scans the full text left-to-right; structural parsing means that non-JSON text containing '{' or '[' characters without balanced JSON structure does not produce a match.",
"actual_behavior": "After execution, if `response` is not a string, a `ValueError` with message 'LLM response must be a JSON string' is raised. Otherwise, let `text` = response.strip(). If `json.loads(text)` succeeds and returns a value `data` that is a `dict` or `list`, the function returns `data`. If `json.loads(text)` succeeds but `data` is not a `dict` or `list`, a `ValueError` with message 'LLM response must contain a JSON object or array' is raised. If `json.loads(text)` raises a `JSONDecodeError` (call it `exc`), the function scans `text` using a JSON decoder to find all valid JSON objects and arrays in sequential order. Let `values` be the list of these found values. If `len(values)` is exactly 1, returns `values[0]`. If `len(values) > 1`, raises `ValueError` with message 'LLM response contains multiple JSON values'. If `len(values) == 0`, raises `ValueError` with message 'LLM response is not valid JSON: {exc}' chained from `exc`.
\n\nFormally, where `is_str(x)` holds if `x` is a string, `full_parse(s)` returns the parsed value if `json.loads(s)` succeeds, or an error otherwise, `is_dict_or_list(v)` holds if `v` is a dict or list, and `scan(s)` returns the ordered list of dict/list values extracted by the scanning algorithm:
\n\npost_condition
\n ( is_str(response) → Raise(ValueError("LLM response must be a JSON string")) ) ∧ ( is_str(response) → let t = strip(response) in
\n      ( (full_parse(t) = v ∧ is_dict_or_list(v)) → Return(v) ) ∧ (full_parse(t) = v ∧ is_dict_or_list(v))
\n      Raise(ValueError("LLM response must contain a JSON object or array")) ) ∧ (full_parse(t) = (error) → let exc = error in
\n      ( |scan(t)| = 1 → Return(scan(t)[0]) ) ∧ ( |scan(t)| > 1 → Raise(ValueError("LLM response contains multiple JSON values")) ) ) ∧ ( |scan(t)| = 0 → Raise(ValueError(f"LLM response is not valid JSON: {exc}")) from exc )
\n      )
\n      )",
"code_evidence": "Line 17: object_start = text.find('{', index)\nLine 18: array_start = text.find('[', index)\nLine 22: start = min(starts)\nLine 24: data, end = decoder.raw_decode(text, start)\nLine 28: if isinstance(data, (dict, list)): values.append(data)",
"trigger_condition": "The scanning logic finds any '{' or '[' character without considering whether it is inside a JSON string, causing it to extract a valid JSON object that appears inside a quoted string. The specification requires structural parsing that respects string boundaries, so a JSON object embedded in a string should not be considered a match. In this counterexample, the code returns {'a': 1} while the specification would raise a ValueError because no standalone JSON object/array exists."
}
```

► Bug Validator result（完整）

```
{
  "id": "src--llm_client-py--_parse_json_response",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/llm_client-py/_parse_json_response.py",
  "function_name": "_parse_json_response",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--llm_client-py--_parse_json_response.py",
  "detail_file": "fm_agent/bug_validation/src--llm_client-py--_parse_json_response.md",
  "probe_stdout": "CONFIRMED – bug reproduced: actual={'a': 1} | expected='ValueError (no standalone JSON object/array)'\n",
  "trigger_summary": "The scanning logic finds '{' or '[' without checking if inside a JSON string, so a JSON object embedded in a quoted string is incorrectly extracted as a standalone value."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:15:23.822349Z	4319
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:16:11.024595Z	9049

事件 1 — [llm_d2bb1195cc514cec88c68b5bd4a5e032](#)

► parsed_json（完整）

```
{
  "post_condition": "After execution, if `response` is not a string, a `ValueError` with message 'LLM response must be a JSON string' is raised. Otherwise, let `text` = response.strip(). If `json.loads(text)` succeeds and returns a value `data` that is a `dict` or `list`, the function returns `data`. If `json.loads(text)` succeeds but `data` is not a `dict` or `list`, a `ValueError` with message 'LLM response must contain a JSON object or array' is raised. If `json.loads(text)` raises a `JSONDecodeError` (call it `exc`), the function scans `text` using a JSON decoder to find all valid JSON objects and arrays in sequential order. Let `values` be the list of these found values. If `len(values)` is exactly 1, returns `values[0]`. If `len(values) > 1`, raises `ValueError` with message 'LLM response contains multiple JSON values'. If `len(values) == 0`, raises `ValueError` with message 'LLM response is not valid JSON: {exc}' chained from `exc`.
\n\nFormally, where `is_str(x)` holds if `x` is a string, `full_parse(s)` returns the parsed value if `json.loads(s)` succeeds, or an error otherwise, `is_dict_or_list(v)` holds if `v` is a dict or list, and `scan(s)` returns the ordered list of dict/list values extracted by the scanning algorithm:
\n\npost_condition
≡\n ( ¬is_str(response) → Raise(ValueError("LLM response must be a JSON string")) ) ∧ ( is_str(response) → let t = strip(response) in
\n      ( (full_parse(t) = v ∧ is_dict_or_list(v)) → Return(v) ) ∧ (full_parse(t) = v ∧
\n      ¬is_dict_or_list(v)) → Raise(ValueError("LLM response must contain a JSON object or array")) ) ∧ (full_parse(t) = (error) → let exc = error in
\n      ( |scan(t)| = 1 → Return(scan(t)[0]) ) ∧ ( |scan(t)| > 1 →
\n      Raise(ValueError("LLM response contains multiple JSON values")) ) ) ∧ ( |scan(t)| = 0 → Raise(ValueError(f"LLM
```

```
response is not valid JSON: {exc}\") from exc )\n )\n )"
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d2bb1195cc514cec88c68b5bd4a5e032_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d2bb1195cc514cec88c68b5bd4a5e032_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d2bb1195cc514cec88c68b5bd4a5e032_response.txt)

事件 2 — llm_1c8e778079634522a7544e8fd7c44a2f

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "response = 'He said, \"{\\\"a\\\": 1}\"'",
  "offending_statements": "Line 17: object_start = text.find(\"{\", index)\nLine 18: array_start = text.find(\"[\", index)\nLine 22: start = min(starts)\nLine 24: data, end = decoder.raw_decode(text, start)\nLine 28: if isinstance(data, (dict, list)):\n    values.append(data)",
  "reason": "The scanning logic finds any '{' or '[' character without considering whether it is inside a JSON string, causing it to extract a valid JSON object that appears inside a quoted string. The specification requires structural parsing that respects string boundaries, so a JSON object embedded in a string should not be considered a match. In this counterexample, the code returns {'a': 1} while the specification would raise a ValueError because no standalone JSON object/array exists."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1c8e778079634522a7544e8fd7c44a2f_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_1c8e778079634522a7544e8fd7c44a2f_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_1c8e778079634522a7544e8fd7c44a2f_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: **opcode_trace**

FMA-MISMATCH-164 — **src--opcode_trace-py--_copy_opcode_output**

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: **confirmed**; attempts 1
- Phase / 模块: Phase 4 — LLM Communication & Tracing / **opcode_trace**; 原源码 **src/opcode_trace.py**
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: stream is closed; if trace_log_path is not None, a file exists at trace_log_path containing the concatenation of every byte read from stream before exhaustion, the file is closed, and its content is UTF-8 encoded with unencodable characters replaced; if trace_log_path is None, no file is created; stream is closed and (if opened) the trace log file is closed under all execution paths, including when an exception is raised during reading or writing
- **Actual behavior**: After the function completes (whether normally or via an unhandled exception), the following holds: (1) The **stream** object is closed. (2) If **trace_log_path** was not **None** and the attempt to open the file succeeded, then a file at **trace_log_path** exists, is closed, and its content equals the concatenation of a finite prefix of the strings returned by **stream.read(4096)** specifically, all chunks successfully read before the stream was exhausted or an error occurred. (3) If **trace_log_path** was not **None** but the file could not be opened (the **open()** call raised an exception), then the file at that path remains unchanged and no content was written. (4) If **trace_log_path** was **None**, no file was opened or written. Formally:

```
stream.closed = True
```

```
( (trace_log_path None)
```

```
( ( prefix P of stream_chunks : file_content(trace_log_path) = concat(P) file_closed(trace_log_path) ) ( file_unchanged(trace_log_path)
opened(trace_log_path) ) ) )
```

where **stream_chunks** is the sequence of strings obtained by repeatedly calling **stream.read(4096)** until exhaustion or an exception.

- **Code evidence**: Line 10: trace_log.write(chunk)
- **Trigger condition**: When the stream is a binary stream, stream.read(4096) returns bytes (e.g., b'\xff'). At line 10, the code attempts to write these bytes directly to a text file opened with encoding='utf-8'. This raises TypeError because write() expects a string, not bytes. The specification requires that every byte read from the stream be written to the file as UTF-8 encoded text with unencodable characters

- replaced (i.e., decoding the bytes with 'replace' and encoding to UTF-8). The code does not perform this conversion, causing it to fail to produce the required file content and instead raise an exception.
- **Validator trigger:** Passing a binary stream (BytesIO) to `_copy_opencode_output` causes `trace_log.write()` to raise `TypeError` because the text-mode file expects `str` but receives `bytes`.
 - **Probe stdout:** CONFIRMED — `TypeError` raised when writing bytes to text-mode trace log: `write()` argument must be `str`, not `bytes`

► SPEC sidecar (完整)

```
{
  "signature": "_copy_opencode_output(stream, trace_log_path=None)",
  "pre_condition": "stream is a readable file-like object whose repeated read() calls return chunks until exhaustion;
trace_log_path is either None or a filesystem path at which a file may be opened for writing in UTF-8 encoding",
  "post_condition": "stream is closed; if trace_log_path is not None, a file exists at trace_log_path containing the concatenation
of every byte read from stream before exhaustion, the file is closed, and its content is UTF-8 encoded with unencodable characters
replaced; if trace_log_path is None, no file is created; stream is closed and (if opened) the trace log file is closed under all
execution paths, including when an exception is raised during reading or writing"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_copy_opencode_output.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "stream is closed; if trace_log_path is not None, a file exists at trace_log_path containing the concatenation
of every byte read from stream before exhaustion, the file is closed, and its content is UTF-8 encoded with unencodable characters
replaced; if trace_log_path is None, no file is created; stream is closed and (if opened) the trace log file is closed under all
execution paths, including when an exception is raised during reading or writing",
    "actual_behavior": "After the function completes (whether normally or via an unhandled exception), the following holds: (1)
The `stream` object is closed. (2) If `trace_log_path` was not `None` and the attempt to open the file succeeded, then a file at
`trace_log_path` exists, is closed, and its content equals the concatenation of a finite prefix of the strings returned by
`stream.read(4096)` specifically, all chunks successfully read before the stream was exhausted or an error occurred. (3) If
`trace_log_path` was not `None` but the file could not be opened (the `open()` call raised an exception), then the file at that
path remains unchanged and no content was written. (4) If `trace_log_path` was `None`, no file was opened or written.
Formally:\n\nstream.closed = True \n( (trace_log_path None) \n    ( ( prefix P of stream_chunks : file_content(trace_log_path)
= concat(P)    file_closed(trace_log_path) )\n        ( file_unchanged(trace_log_path)    opened(trace_log_path) )\n    )\n)\n\nwhere `stream_chunks` is the sequence of strings obtained by repeatedly calling `stream.read(4096)` until exhaustion or an
exception.",
    "code_evidence": "Line 10: trace_log.write(chunk)",
    "trigger_condition": "When the stream is a binary stream, stream.read(4096) returns bytes (e.g., b'\\xff'). At line 10, the
code attempts to write these bytes directly to a text file opened with encoding='utf-8'. This raises TypeError because write()
expects a string, not bytes. The specification requires that every byte read from the stream be written to the file as UTF-8
encoded text with unencodable characters replaced (i.e., decoding the bytes with 'replace' and encoding to UTF-8). The code does
not perform this conversion, causing it to fail to produce the required file content and instead raise an exception."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--opencode_trace-py--_copy_opencode_output",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_copy_opencode_output.py",
  "function_name": "_copy_opencode_output",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--opencode_trace-py--_copy_opencode_output.py",
  "detail_file": "fm_agent/bug_validation/src--opencode_trace-py--_copy_opencode_output.md",
  "probe_stdout": "CONFIRMED – TypeError raised when writing bytes to text-mode trace log: write() argument must be str, not
bytes",
  "trigger_summary": "Passing a binary stream (BytesIO) to _copy_opencode_output causes trace_log.write() to raise TypeError
because the text-mode file expects str but receives bytes."
}
```

► Reasoner 流程 (2 条事件)

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:46:54.280072Z	6264
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:48:34.068676Z	8842

事件 1 — llm_63e1cc5ad01240cea0e6c394ec244bc9

► parsed_json (完整)

```
{  "post_condition": "After the function completes (whether normally or via an unhandled exception), the following holds: (1) The `stream` object is closed. (2) If `trace_log_path` was not `None` and the attempt to open the file succeeded, then a file at `trace_log_path` exists, is closed, and its content equals the concatenation of a finite prefix of the strings returned by `stream.read(4096)` – specifically, all chunks successfully read before the stream was exhausted or an error occurred. (3) If `trace_log_path` was not `None` but the file could not be opened (the `open()` call raised an exception), then the file at that path remains unchanged and no content was written. (4) If `trace_log_path` was `None`, no file was opened or written. Formally:\n\nstream.closed = True  ^\n( (trace_log_path != None) ->\n  ( (exists prefix P of stream_chunks : file_content(trace_log_path) = concat(P)  ^  file_closed(trace_log_path) )\n    v  ( file_unchanged(trace_log_path) ^ ~opened(trace_log_path) )\n  )\n)\n\nwhere `stream_chunks` is the sequence of strings obtained by repeatedly calling `stream.read(4096)` until exhaustion or an exception."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_63e1cc5ad01240cea0e6c394ec244bc9_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_63e1cc5ad01240cea0e6c394ec244bc9_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_63e1cc5ad01240cea0e6c394ec244bc9_response.txt)

事件 2 — llm_4f1e6634533b4e20b9bf6caa6d6e481f

► parsed_json (完整)

```
{  "verdict": "MISMATCH",  "counterexample": "stream = io.BytesIO(b'\\xff'), trace_log_path = 'output.log'",  "offending_statements": "Line 10: trace_log.write(chunk)",  "reason": "When the stream is a binary stream, stream.read(4096) returns bytes (e.g., b'\\xff'). At line 10, the code attempts to write these bytes directly to a text file opened with encoding='utf-8'. This raises TypeError because write() expects a string, not bytes. The specification requires that every byte read from the stream be written to the file as UTF-8 encoded text with unencodable characters replaced (i.e., decoding the bytes with 'replace' and encoding to UTF-8). The code does not perform this conversion, causing it to fail to produce the required file content and instead raise an exception."}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4f1e6634533b4e20b9bf6caa6d6e481f_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4f1e6634533b4e20b9bf6caa6d6e481f_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4f1e6634533b4e20b9bf6caa6d6e481f_response.txt) **复核动作**

• 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-166 — src--opcode_trace-py--_opcode_log_path

• 四类结论：可能有 Bug

• 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。

• Validator：confirmed; attempts 1

• Phase / 模块：Phase 4 — LLM Communication & Tracing / opcode_trace；原源码 src/opcode_trace.py

• 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

• SPEC claim：returns a filesystem path string within the trace directory structure derived from work_dir that uniquely identifies a log file for the given event_id; the returned path is derivable solely from work_dir and event_id and is suitable for writing subprocess stdout log content

• Actual behavior：The function returns a string representing a file path. Let trace_dir = _trace_dir(work_dir) and payload_dir = _payload_dir(trace_dir). The return value ret satisfies: ret == os.path.join(payload_dir, f'{event_id}_opcode.log'). The returned string is a concatenation of the payload directory path and the filename composed of the event_id and the suffix '_opcode.log', using the OS-specific path separator.

• Code evidence：Line 2: return os.path.join(_payload_dir(_trace_dir(work_dir)), f'{event_id}_opcode.log')

- **SPEC claim:** returns a filesystem path string within the trace directory structure derived from `work_dir` that uniquely identifies a log file for the given `event_id`; the returned path is derivable solely from `work_dir` and `event_id` and is suitable for writing subprocess stdout log content
- **Actual behavior:** The function returns a string representing a file path. Let `trace_dir = _trace_dir(work_dir)` and `payload_dir = _payload_dir(trace_dir)`. The return value `ret` satisfies: `ret == os.path.join(payload_dir, f"{event_id}_opcode.log")`. The returned string is a concatenation of the payload directory path and the filename composed of the `event_id` and the suffix `'_opcode.log'`, using the OS-specific path separator.
- **Code evidence:** Line 2: `return os.path.join( payload_dir( trace_dir(work_dir)), f"{event_id}_opcode.log")`

- **Trigger condition:** The specification requires the returned path to be within the trace directory structure. With event_id='.././malicious', the returned path escapes the trace directory, violating the requirement.
- **Validator trigger:** event_id='.././malicious' causes os.path.join to traverse out of the trace directory, writing the log outside the intended sandbox.
- **Probe stdout:** CONFIRMED — path traversal successful work_dir: /tmp/tmpbeyck_mv/run trace_dir: /tmp/tmpbeyck_mv/run/trace expected_under: /tmp/tmpbeyck_mv/run/trace actual_path: /tmp/tmpbeyck_mv/run/malicious_opencode.log event_id: '.././malicious' escaped trace directory

► SPEC sidecar (完整)

```
{
  "signature": "_opencode_log_path(work_dir, event_id) -> str",
  "pre_condition": "work_dir is a valid directory path; event_id is a non-empty string",
  "post_condition": "returns a filesystem path string within the trace directory structure derived from work_dir that uniquely identifies a log file for the given event_id; the returned path is derivable solely from work_dir and event_id and is suitable for writing subprocess stdout log content"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_trace_dir",
      "signature": "_trace_dir(work_dir) -> str",
      "pre_condition": "work_dir is a valid directory path",
      "post_condition": "returns the trace subdirectory path within work_dir"
    },
    {
      "name": "_payload_dir",
      "signature": "_payload_dir(trace_dir) -> str",
      "pre_condition": "trace_dir is a valid directory path",
      "post_condition": "returns the payload subdirectory path within trace_dir"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_opencode_log_path.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "returns a filesystem path string within the trace directory structure derived from work_dir that uniquely identifies a log file for the given event_id; the returned path is derivable solely from work_dir and event_id and is suitable for writing subprocess stdout log content",
    "actual_behavior": "The function returns a string representing a file path. Let trace_dir = _trace_dir(work_dir) and payload_dir = _payload_dir(trace_dir). The return value ret satisfies: ret == os.path.join(payload_dir, f\"{event_id}_opencode.log\"). The returned string is a concatenation of the payload directory path and the filename composed of the event_id and the suffix '_opencode.log', using the OS-specific path separator.",
    "code_evidence": "Line 2: return os.path.join(_payload_dir(_trace_dir(work_dir)), f\"{event_id}_opencode.log\")",
    "trigger_condition": "The specification requires the returned path to be within the trace directory structure. With event_id='.././malicious', the returned path escapes the trace directory, violating the requirement."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--opencode_trace-py--_opencode_log_path",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_opencode_log_path.py",
  "function_name": "_opencode_log_path",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--opencode_trace-py--_opencode_log_path.py",
  "detail_file": "fm_agent/bug_validation/src--opencode_trace-py--_opencode_log_path.md",
  "probe_stdout": "CONFIRMED - path traversal successful\n work_dir:      /tmp/tmpbeyck_mv/run\n trace_dir: /tmp/tmpbeyck_mv/run/trace\n expected_under: /tmp/tmpbeyck_mv/run/trace\n actual_path: /tmp/tmpbeyck_mv/run/malicious_opencode.log\n event_id:      '.././malicious'\n  escaped trace directory",
  "trigger_summary": "event_id='.././malicious' causes os.path.join to traverse out of the trace directory, writing the log outside the intended sandbox."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:01:41.018641Z	875
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:01:50.483045Z	2027

事件 1 — llm_445e5183a4094482811a532d1c0367cb

► parsed_json（完整）

```
{
  "post_condition": "The function returns a string representing a file path. Let trace_dir = _trace_dir(work_dir) and payload_dir = _payload_dir(trace_dir). The return value ret satisfies: ret == os.path.join(payload_dir, f\"{event_id}_opcode.log\"). The returned string is a concatenation of the payload directory path and the filename composed of the event_id and the suffix '_opcode.log', using the OS-specific path separator."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_445e5183a4094482811a532d1c0367cb_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_445e5183a4094482811a532d1c0367cb_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_445e5183a4094482811a532d1c0367cb_response.txt)

事件 2 — llm_4fe094cbd5c0465197983bd23d6d652f

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "work_dir='/tmp', event_id='.././malicious'",
  "offending_statements": "Line 2: return os.path.join(_payload_dir(_trace_dir(work_dir)), f\"{event_id}_opcode.log\")",
  "reason": "The specification requires the returned path to be within the trace directory structure. With event_id='.././malicious', the returned path escapes the trace directory, violating the requirement."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_4fe094cbd5c0465197983bd23d6d652f_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_4fe094cbd5c0465197983bd23d6d652f_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_4fe094cbd5c0465197983bd23d6d652f_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-167 — src--opcode_trace-py--_opcode_trace_path

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 4 — LLM Communication & Tracing / opcode_trace；原源码 src/opcode_trace.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** Returns a filesystem path string that uniquely identifies a trace file for the given event_id within the trace directory hierarchy derived from work_dir. The returned path is suitable for writing raw LLM request/response trace data.
- **Actual behavior:** The function returns a string equal to the filesystem path formed by joining the trace directory path of work_dir (obtained from _trace_dir(work_dir)), the literal 'opcode' folder name, and the filename {event_id}.jsonl. Formally: let r be the return value; then r = os.path.join(_trace_dir(work_dir), 'opcode', event_id + '.jsonl').
- **Code evidence:** Line 2: return os.path.join(_trace_dir(work_dir), "opcode", f"{event_id}.jsonl")
- **Trigger condition:** The code does not sanitize event_id, allowing path traversal characters. For example, if event_id is '../other', the returned path resolves outside the 'opcode' subdirectory, and different event_ids can map to the same file (e.g., '../other' and 'opcode/../other' both yield trace_dir/other.jsonl). This violates the specification's requirement that the returned path uniquely identifies a trace file for the given event_id within the intended hierarchy.
- **Validator trigger:** The code does not sanitize event_id, allowing path traversal characters (e.g., '../other') to produce a path that escapes the intended opcode/ subdirectory, violating the uniqueness and containment guarantees required by the specification.

- **Probe stdout:** CONFIRMED — path traversal: event_id='../other' → /tmp/tmpkvvgwabw/trace/other.jsonl escapes /tmp/tmpkvvgwabw/trace/opencode

► SPEC sidecar（完整）

```
{
  "signature": "_opencode_trace_path(work_dir, event_id) -> str",
  "pre_condition": "work_dir is an existing directory path and event_id is a non-empty string",
  "post_condition": "Returns a filesystem path string that uniquely identifies a trace file for the given event_id within the trace directory hierarchy derived from work_dir. The returned path is suitable for writing raw LLM request/response trace data."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_trace_dir",
      "signature": "_trace_dir(work_dir) -> str",
      "pre_condition": "work_dir is a valid directory path",
      "post_condition": "Returns a filesystem path string to the trace directory under work_dir"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_opencode_trace_path.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a filesystem path string that uniquely identifies a trace file for the given event_id within the trace directory hierarchy derived from work_dir. The returned path is suitable for writing raw LLM request/response trace data.",
    "actual_behavior": "The function returns a string equal to the filesystem path formed by joining the trace directory path of `work_dir` (obtained from `_trace_dir(work_dir)`), the literal `opencode` folder name, and the filename `{event_id}.jsonl`. Formally: let `r` be the return value; then `r = os.path.join(_trace_dir(work_dir), 'opencode', event_id + '.jsonl')`.",
    "code_evidence": "Line 2: return os.path.join(_trace_dir(work_dir), `opencode`, f`{event_id}.jsonl`)",
    "trigger_condition": "The code does not sanitize event_id, allowing path traversal characters. For example, if event_id is '../other', the returned path resolves outside the 'opencode' subdirectory, and different event_ids can map to the same file (e.g., '../other' and 'opencode/../other' both yield trace_dir/other.jsonl). This violates the specification's requirement that the returned path uniquely identifies a trace file for the given event_id within the intended hierarchy."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--opencode_trace-py--_opencode_trace_path",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_opencode_trace_path.py",
  "function_name": "opencode_trace_path",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--opencode_trace-py--_opencode_trace_path.py",
  "detail_file": "fm_agent/bug_validation/src--opencode_trace-py--_opencode_trace_path.md",
  "probe_stdout": "CONFIRMED — path traversal: event_id='../other' → /tmp/tmpkvvgwabw/trace/other.jsonl escapes /tmp/tmpkvvgwabw/trace/opencode",
  "trigger_summary": "The code does not sanitize event_id, allowing path traversal characters (e.g., '../other') to produce a path that escapes the intended opencode/ subdirectory, violating the uniqueness and containment guarantees required by the specification."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:01:00.985390Z	1071
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:01:11.789548Z	3214

事件 1 — [llm_f9acc373be164d69950d6dbe5d3577d1](#)

► parsed_json (完整)

```
{
  "post_condition": "The function returns a string equal to the filesystem path formed by joining the trace directory path of
`work_dir` (obtained from `_trace_dir(work_dir)`), the literal `opencode` folder name, and the filename `{event_id}.jsonl`.
Formally: let `r` be the return value; then `r` = os.path.join(_trace_dir(work_dir), 'opencode', event_id + '.jsonl')."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f9acc373be164d69950d6dbe5d3577d1_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_f9acc373be164d69950d6dbe5d3577d1_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_f9acc373be164d69950d6dbe5d3577d1_response.txt)

事件 2 — llm_15c0c88d711840e8b1963e8879d3aa3a

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "event_id = '../other'",
  "offending_statements": "Line 2: return os.path.join(_trace_dir(work_dir), \"opencode\", f\"{event_id}.jsonl\")",
  "reason": "The code does not sanitize event_id, allowing path traversal characters. For example, if event_id is '../other', the
returned path resolves outside the 'opencode' subdirectory, and different event_ids can map to the same file (e.g., '../other' and
'opencode/../other' both yield trace_dir/other.jsonl). This violates the specification's requirement that the returned path
uniquely identifies a trace file for the given event_id within the intended hierarchy."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_15c0c88d711840e8b1963e8879d3aa3a_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_15c0c88d711840e8b1963e8879d3aa3a_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_15c0c88d711840e8b1963e8879d3aa3a_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-171 — src--opencode_trace-py--_wait_opencode_process

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: **confirmed**; attempts 1
- Phase / 模块：Phase 4 — LLM Communication & Tracing / **opencode_trace**；原源码 **src/opencode_trace.py**
- 证据：抽取源码 · **SPEC** · **INFO** · **Logic** · **Validator result** · **probe** · 详细报告

冲突摘要

- SPEC claim**: Blocks until the process in proc completes or the timeout expires. Returns a 2-tuple (exit_code, error). When the process exits within timeout_seconds: exit_code is the integer process exit code and error is None. When the process does not exit within timeout_seconds: proc is terminated (SIGTERM), then force-killed (SIGKILL) if termination does not complete within a 10-second grace period; exit_code is the process exit code after forced termination, mapped to -15 when that exit code is falsy (0 or None); error is a descriptive string indicating the timeout condition.
- Actual behavior**: Upon normal return (no unhandled exception), the function returns a 2-tuple (ec, em) where ec, em {None} {s | s == f'timeout after {timeout_seconds}s'}. The subprocess.Popen instance proc has been reaped: proc.wait() was called (possibly after terminate/kill), so proc.returncode and the child process is not running. If the initial proc.wait(timeout=timeout_seconds) completed without raising TimeoutExpired (the no-timeout case), then ec = proc.returncode and em = None. If it raised TimeoutExpired (the timeout case), then em = f'timeout after {timeout_seconds}s'. In the timeout case, proc.terminate() was called; if the subsequent proc.wait(timeout=10) raised TimeoutExpired, proc.kill() was called and then proc.wait() without timeout. Finally, ec = proc.returncode unless proc.returncode == 0, in which case ec = -15 (so a killed-on-timeout success is never reported as success). The attribute proc.returncode equals the actual exit status from the last successful wait, which may be 0 even when ec is -15.
- Code evidence**: Line 18: if not exit_code: Line 19: exit_code = -15 # killed-on-timeout must never record as success
- Trigger condition**: The specification maps the exit code to -15 only after forced termination (SIGKILL). The code unconditionally maps any falsy exit code in the timeout path, including when the process exits cleanly via SIGTERM with code 0. This causes a mismatch.
- Validator trigger**: A subprocess that times out and exits cleanly via SIGTERM (exit code 0) within the 10s grace period has its exit code incorrectly remapped to -15; the -15 mapping should only apply after SIGKILL.
- Probe stdout**: WARNING:root:opencode test timed out after 1s, killing: echo hello CONFIRMED — actual: -15 | expected: 0

► SPEC sidecar (完整)

```
{
  "signature": "_wait_opencode_process(proc, command, stage, timeout_seconds) -> (int, str | None)",
  "pre_condition": "proc is a running subprocess.Popen instance whose process has not yet been waited on. command is a non-empty sequence of strings. stage is a non-empty string. timeout_seconds is a positive integer.",
  "post_condition": "Blocks until the process in proc completes or the timeout expires. Returns a 2-tuple (exit_code, error). When the process exits within timeout_seconds: exit_code is the integer process exit code and error is None. When the process does not exit within timeout_seconds: proc is terminated (SIGTERM), then force-killed (SIGKILL) if termination does not complete within a 10-second grace period; exit_code is the process exit code after forced termination, mapped to -15 when that exit code is falsy (0 or None); error is a descriptive string indicating the timeout condition."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_wait_opencode_process.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Blocks until the process in proc completes or the timeout expires. Returns a 2-tuple (exit_code, error). When the process exits within timeout_seconds: exit_code is the integer process exit code and error is None. When the process does not exit within timeout_seconds: proc is terminated (SIGTERM), then force-killed (SIGKILL) if termination does not complete within a 10-second grace period; exit_code is the process exit code after forced termination, mapped to -15 when that exit code is falsy (0 or None); error is a descriptive string indicating the timeout condition.",
    "actual_behavior": "Upon normal return (no unhandled exception), the function returns a 2-tuple (ec, em) where ec , em {None} {s | s == f'timeout after {timeout_seconds}s'}. The subprocess.Popen instance proc has been reaped: proc.wait() was called (possibly after terminate/kill), so proc.returncode and the child process is not running. If the initial proc.wait(timeout=timeout_seconds) completed without raising TimeoutExpired (the no-timeout case), then ec = proc.returncode and em = None. If it raised TimeoutExpired (the timeout case), then em = f'timeout after {timeout_seconds}s'. In the timeout case, proc.terminate() was called; if the subsequent proc.wait(timeout=10) raised TimeoutExpired, proc.kill() was called and then proc.wait() without timeout. Finally, ec = proc.returncode unless proc.returncode == 0, in which case ec = -15 (so a killed-on-timeout success is never reported as success). The attribute proc.returncode equals the actual exit status from the last successful wait, which may be 0 even when ec is -15.",
    "code_evidence": "Line 18: if not exit_code:\nLine 19:         exit_code = -15 # killed-on-timeout must never record as success",
    "trigger_condition": "The specification maps the exit code to -15 only after forced termination (SIGKILL). The code unconditionally maps any falsy exit code in the timeout path, including when the process exits cleanly via SIGTERM with code 0. This causes a mismatch."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--opencode_trace-py--_wait_opencode_process",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/opencode_trace.py",
  "function_name": "_wait_opencode_process",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--opencode_trace-py--_wait_opencode_process.py",
  "detail_file": "fm_agent/bug_validation/src--opencode_trace-py--_wait_opencode_process.md",
  "probe_stdout": "WARNING:root:opencode test timed out after 1s, killing: echo hello\nCONFIRMED – actual: -15 | expected: 0\n",
  "trigger_summary": "A subprocess that times out and exits cleanly via SIGTERM (exit code 0) within the 10s grace period has its exit code incorrectly remapped to -15; the -15 mapping should only apply after SIGKILL."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:01:47.026720Z	2995
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:02:25.328498Z	4445

事件 1 — [llm_2accfe7642f14f8b99eaeac544099338](#)

► parsed_json（完整）

```
{
  "post_condition": "Upon normal return (no unhandled exception), the function returns a 2-tuple (ec, em) where ec ∈ ℤ, em ∈ {None} ∪ {s | s == f'timeout after {timeout_seconds}s'}. The subprocess.Popen instance proc has been reaped: proc.wait() was called (possibly after terminate/kill), so proc.returncode ∈ ℤ and the child process is not running. If the initial proc.wait(timeout=timeout_seconds) completed without raising TimeoutExpired (the no-timeout case), then ec = proc.returncode and em = None. If it raised TimeoutExpired (the timeout case), then em = f'timeout after {timeout_seconds}s'. In the timeout case, proc.terminate() was called; if the subsequent proc.wait(timeout=10) raised TimeoutExpired, proc.kill() was called and then proc.wait() without timeout. Finally, ec = proc.returncode unless proc.returncode == 0, in which case ec = -15 (so a killed-on-timeout success is never reported as success). The attribute proc.returncode equals the actual exit status from the last successful wait, which may be 0 even when ec is -15."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_2accfe7642f14f8b99eaeac544099338_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_2accfe7642f14f8b99eaeac544099338_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_2accfe7642f14f8b99eaeac544099338_response.txt)

事件 2 — llm_9dfeb9e0abb14f9ba9f23a2a2f4d6ad7

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "A subprocess that, upon receiving SIGTERM, exits with exit code 0 within the 10-second grace period. For example, a Python script:\nimport signal\nimport sys\nsignal.signal(signal.SIGTERM, lambda s, f: sys.exit(0))\nwhile True:\n    pass\nWhen _wait_opencode_process is called with timeout_seconds=1, the initial wait times out; terminate() sends SIGTERM; the process catches it and exits with 0. The code sets exit_code to -15, but the specification requires exit_code 0 because no forced kill occurred.",
  "offending_statements": "Line 18: if not exit_code:\nLine 19:         exit_code = -15 # killed-on-timeout must never record as success",
  "reason": "The specification maps the exit code to -15 only after forced termination (SIGKILL). The code unconditionally maps any falsy exit code in the timeout path, including when the process exits cleanly via SIGTERM with code 0. This causes a mismatch."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_9dfeb9e0abb14f9ba9f23a2a2f4d6ad7_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_9dfeb9e0abb14f9ba9f23a2a2f4d6ad7_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_9dfeb9e0abb14f9ba9f23a2a2f4d6ad7_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-177 — src--opencode_trace-py--run_opencode_traced

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 4 — LLM Communication & Tracing / opencode_trace；源代码 src/opencode_trace.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: When the subprocess exits with code 0 and no error: returns a subprocess.CompletedProcess with returncode=0. When the subprocess exits with a non-zero code or an error occurs: raises subprocess.CallProcessError whose returncode equals the subprocess exit code. In every execution path (success, failure, or exception), exactly one structured event record is appended atomically to events.jsonl under the trace directory derived from work_dir. The recorded event contains: a unique event_id, the stage label, ISO 8601 start and end timestamps covering the full execution window, the full command list, the process exit code, a status of "success" (exit_code=0 and no error) or "error" (exit_code0 or error present), the provided function_ids, input_files, output_files, summary, error, and metadata values, and paths to the corresponding opencode log and trace files. All background threads (log streaming and stdin feeding) are joined and terminated before this function returns normally or propagates an exception.
- **Actual behavior**: After the execution of run_opencode_traced, regardless of whether it returns normally or raises an exception, the following holds. An event identifier event_id of the form "opencode_<hex_uuid>" has been generated and two ISO 8601 UTC timestamps started and ended have been captured with ended >= started. The call record_opencode_call(work_dir=work_dir, event_id=event_id, stage=stage, status='success' if exit_code == 0 else 'error', started=started, ended=ended, command=command, function_ids=function_ids, input_files=input_files, output_files=output_files, exit_code=exit_code, summary=summary, error=error, metadata=metadata, opencode_log_path=_opencode_log_path(work_dir, event_id), opencode_trace_path=_opencode_trace_path(work_dir, event_id)) has been executed exactly once, atomically appending the corresponding JSON event record to events.jsonl in the trace directory derived from work_dir. The values of exit_code and error reflect the final outcome: if _wait_opencode_process set exit_code and error, exit_code is the

integer process exit code and error is a string or None; if a subprocess.CalledProcessError was raised, exit_code is exc.returncode and error is error or str(exc); otherwise exit_code remains 0 and error None. Any non-None log_thread or stdin_thread has been joined (if alive in the finally block, they are waited for completion), ensuring the log file at opencode_log_path is fully written and closed. If _start_opencode_process completed without raising an exception, the subprocess proc has terminated and its exit code is reflected. The function either returns normally with a subprocess.CompletedProcess instance whose args are command_argv(command) and returncode is exit_code (always 0 in this case), or it propagates an exception, which is always a subprocess.CalledProcessError when error or nonzero exit_code was detected, or any other exception produced by the subprocess management. In all cases, the work_dir and proj_dir are not modified except for the files explicitly created (log, trace, and event record).

- **Code evidence:** Line 45: status="success" if exit_code == 0 else "error",
- **Trigger condition:** The code determines the event status solely by comparing exit_code to 0. The specification requires status 'error' whenever exit_code 0 or an error is present. Here the process exits with code 0 but _wait_opencode_process reports an error string, so status should be 'error', not 'success'.
- **Validator trigger:** When the subprocess exits with code 0 but _wait_opencode_process reports an error string, the event status is recorded as 'success' instead of 'error' because the status logic only checks exit_code==0, ignoring the error parameter entirely.
- **Probe stdout:** CONFIRMED — record_opencode_call received status='success', expected='error'. Bug: status='success' despite error='mock error: something went wrong during execution' and exit_code=0. The status logic only checks exit_code==0, ignoring the error parameter.

► SPEC sidecar (完整)

```
{
  "signature": "run_opencode_traced(*, proj_dir, work_dir, command, stage, function_ids=None, input_files=None, output_files=None,
summary=None, metadata=None) -> CompletedProcess",
  "pre_condition": "proj_dir, work_dir, command, and stage are provided. command is a non-empty sequence of strings. work_dir is a
writable directory path.",
  "post_condition": "When the subprocess exits with code 0 and no error: returns a subprocess.CompletedProcess with returncode=0.
When the subprocess exits with a non-zero code or an error occurs: raises subprocess.CalledProcessError whose returncode equals
the subprocess exit code. In every execution path (success, failure, or exception), exactly one structured event record is
appended atomically to events.jsonl under the trace directory derived from work_dir. The recorded event contains: a unique
event_id, the stage label, ISO 8601 start and end timestamps covering the full execution window, the full command list, the
process exit code, a status of \"success\" (exit_code=0 and no error) or \"error\" (exit_code≠0 or error present), the provided
function_ids, input_files, output_files, summary, error, and metadata values, and paths to the corresponding opencode log and
trace files. All background threads (log streaming and stdin feeding) are joined and terminated before this function returns
normally or propagates an exception."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "new_event_id",
      "signature": "new_event_id(prefix) -> str",
      "pre_condition": "prefix is the string \"opencode\"",
      "post_condition": "Returns a unique string identifier in the format \"<prefix>_<hex_uuid>\" that is distinct from all
identifiers returned by prior calls"
    },
    {
      "name": "utc_now_iso",
      "signature": "utc_now_iso() -> str",
      "pre_condition": "None",
      "post_condition": "Returns the current date and time in UTC as an ISO 8601 formatted string"
    },
    {
      "name": "_opencode_log_path",
      "signature": "_opencode_log_path(work_dir, event_id) -> str",
      "pre_condition": "work_dir is an existing directory path and event_id is a non-empty string",
      "post_condition": "Returns a file path within work_dir that is unique to event_id and suitable for writing the subprocess
stdout log"
    },
    {
      "name": "_opencode_trace_path",
      "signature": "_opencode_trace_path(work_dir, event_id) -> str",
      "pre_condition": "work_dir is an existing directory path and event_id is a non-empty string",
      "post_condition": "Returns a file path within work_dir that is unique to event_id and suitable for writing raw LLM
request/response traces"
    },
    {
      "name": "_start_opencode_process",
      "signature": "_start_opencode_process(proj_dir, work_dir, event_id, command, opencode_log_path) -> (Popen, Thread | None,
Thread | None)",
      "pre_condition": "proj_dir and work_dir are valid directory paths; command is a non-empty list of strings; opencode_log_path
is a writable file path",
      "post_condition": "Returns a 3-tuple of (a running subprocess.Popen instance, an optional thread streaming subprocess stdout
```

```

opencode_log_path, an optional thread feeding stdin to the subprocess). The subprocess environment includes
OPENCODE_CONFIG_CONTENT, TRACE_DIR, and LLM_API_KEY injections."
},
{
  "name": "_wait_opencode_process",
  "signature": "_wait_opencode_process(proc, command, stage) -> (int, str | None)",
  "pre_condition": "proc is a running subprocess. Popen returned by _start_opencode_process",
  "post_condition": "Blocks until the subprocess terminates, then returns (exit_code, error) where exit_code is the integer
process exit code and error is an error description string if the process terminated abnormally, or None if it exited normally"
},
{
  "name": "command_argv",
  "signature": "command_argv(command) -> list[str]",
  "pre_condition": "command is a non-empty sequence of strings",
  "post_condition": "Returns a list of strings with the executable as the first element, suitable for display in error
messages and event records"
},
{
  "name": "record_opencode_call",
  "signature": "record_opencode_call(*, work_dir, event_id, stage, status, started, ended, command, function_ids, input_files,
output_files, exit_code, summary, error, metadata, opencode_log_path, opencode_trace_path) -> None",
  "pre_condition": "started and ended are ISO 8601 UTC timestamp strings where ended is not earlier than started; status is
one of \"success\" or \"error\"; exit_code is an integer",
  "post_condition": "Atomically appends one structured JSON event record to the events.jsonl file in the trace directory
derived from work_dir. The record includes event_id, stage, status, started, ended, command, exit_code, and all optional metadata
fields. On success the caller observes exactly one new line in events.jsonl containing the complete event."
}
]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/run_opencode_traced.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When the subprocess exits with code 0 and no error: returns a subprocess.CompletedProcess with returncode=0.
When the subprocess exits with a non-zero code or an error occurs: raises subprocess.CallledProcessError whose returncode equals
the subprocess exit code. In every execution path (success, failure, or exception), exactly one structured event record is
appended atomically to events.jsonl under the trace directory derived from work_dir. The recorded event contains: a unique
event_id, the stage label, ISO 8601 start and end timestamps covering the full execution window, the full command list, the
process exit code, a status of \"success\" (exit_code=0 and no error) or \"error\" (exit_code!=0 or error present), the provided
function_ids, input_files, output_files, summary, error, and metadata values, and paths to the corresponding opencode log and
trace files. All background threads (log streaming and stdin feeding) are joined and terminated before this function returns
normally or propagates an exception.",
    "actual_behavior": "After the execution of run_opencode_traced, regardless of whether it returns normally or raises an
exception, the following holds. An event identifier event_id of the form \"opencode-<hex_uuid>\" has been generated and two ISO
8601 UTC timestamps started and ended have been captured with ended >= started. The call record_opencode_call(work_dir=work_dir,
event_id=event_id, stage=stage, status='success' if exit_code == 0 else 'error', started=started, ended=ended, command=command,
function_ids=function_ids, input_files=input_files, output_files=output_files, exit_code=exit_code, summary=summary, error=error,
metadata=metadata, opencode_log_path=opencode_log_path(work_dir, event_id), opencode_trace_path=opencode_trace_path(work_dir,
event_id)) has been executed exactly once, atomically appending the corresponding JSON event record to events.jsonl in the trace
directory derived from work_dir. The values of exit_code and error reflect the final outcome: if _wait_opencode_process set
exit_code and error, exit_code is the integer process exit code and error is a string or None; if a subprocess.CallledProcessError
was raised, exit_code is exc.returncode and error is error or str(exc); otherwise exit_code remains 0 and error None. Any non-None
log_thread or stdin_thread has been joined (if alive in the finally block, they are waited for completion), ensuring the log file
at opencode_log_path is fully written and closed. If _start_opencode_process completed without raising an exception, the
subprocess proc has terminated and its exit code is reflected. The function either returns normally with a
subprocess.CompletedProcess instance whose args are command_argv(command) and returncode is exit_code (always 0 in this case), or
it propagates an exception, which is always a subprocess.CallledProcessError when error or nonzero exit_code was detected, or any
other exception produced by the subprocess management. In all cases, the work_dir and proj_dir are not modified except for the
files explicitly created (log, trace, and event record).",
    "code_evidence": "Line 45:                 status=\"success\" if exit_code == 0 else \"error\",",
    "trigger_condition": "The code determines the event status solely by comparing exit_code to 0. The specification requires
status 'error' whenever exit_code!=0 or an error is present. Here the process exits with code 0 but _wait_opencode_process reports
an error string, so status should be 'error', not 'success'."
  }
}

```

► Bug Validator result (完整)

```

{
  "id": "src--opencode_trace-py--run_opencode_traced",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/run_opencode_traced.py",
  "function_name": "run_opencode_traced",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--opencode_trace-py--run_opencode_traced.py",
  "detail_file": "fm_agent/bug_validation/src--opencode_trace-py--run_opencode_traced.md",
  "probe_stdout": "CONFIRMED - record_opencode_call received status='success', expected='error'. Bug: status='success' despite
error='mock error: something went wrong during execution' and exit_code=0. The status logic only checks exit_code==0, ignoring the
error parameter."
}

```



```
"trigger_summary": "When the subprocess exits with code 0 but _wait_opencode_process reports an error string, the event status is recorded as 'success' instead of 'error' because the status logic only checks exit_code==0, ignoring the error parameter entirely."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:47:44.333132Z	4178
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:48:27.897344Z	6538

事件 1 — [llm_7dd79333e84c48a9a91ea0dfb85529aa](#)

► parsed_json（完整）

```
{
  "post_condition": "After the execution of run_opencode_traced, regardless of whether it returns normally or raises an exception, the following holds. An event identifier event_id of the form \"opencode_<hex_uuid>\" has been generated and two ISO 8601 UTC timestamps started and ended have been captured with ended >= started. The call record_opencode_call(work_dir=work_dir, event_id=event_id, stage=stage, status='success' if exit_code == 0 else 'error', started=started, ended=ended, command=command, function_ids=function_ids, input_files=input_files, output_files=output_files, exit_code=exit_code, summary=summary, error=error, metadata=metadata, opencode_log_path=opencode_log_path(work_dir, event_id), opencode_trace_path=opencode_trace_path(work_dir, event_id)) has been executed exactly once, atomically appending the corresponding JSON event record to events.jsonl in the trace directory derived from work_dir. The values of exit_code and error reflect the final outcome: if _wait_opencode_process set exit_code and error, exit_code is the integer process exit code and error is a string or None; if a subprocess.CalledProcessError was raised, exit_code is exc.returncode and error is error or str(exc); otherwise exit_code remains 0 and error None. Any non-None log_thread or stdin_thread has been joined (if alive in the finally block, they are waited for completion), ensuring the log file at opencode_log_path is fully written and closed. If _start_opencode_process completed without raising an exception, the subprocess proc has terminated and its exit code is reflected. The function either returns normally with a subprocess.CompletedProcess instance whose args are command_argv(command) and returncode is exit_code (always 0 in this case), or it propagates an exception, which is always a subprocess.CalledProcessError when error or non-zero exit_code was detected, or any other exception produced by the subprocess management. In all cases, the work_dir and proj_dir are not modified except for the files explicitly created (log, trace, and event record)."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7dd79333e84c48a9a91ea0dfb85529aa_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_7dd79333e84c48a9a91ea0dfb85529aa_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_7dd79333e84c48a9a91ea0dfb85529aa_response.txt)

事件 2 — [llm_e2f5ae65131e43beb1f3e877b60255e1](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp/proj', work_dir='/tmp/work', command=['true'], stage='test', input_files=['dummy.txt'], other optional arguments None. The subprocess 'true' exits with code 0 very quickly, causing the stdin feeding thread to hit a BrokenPipeError. _wait_opencode_process detects this abnormal condition and returns (0, 'Broken pipe'). The code raises subprocess.CalledProcessError(returncode=0), and in the finally block records an event with status='success' because exit_code == 0. According to the specification, when an error occurs the status must be 'error' even if the exit code is 0.",
  "offending_statements": "Line 45: status=\"success\" if exit_code == 0 else \"error\",",
  "reason": "The code determines the event status solely by comparing exit_code to 0. The specification requires status 'error' whenever exit_code!=0 or an error is present. Here the process exits with code 0 but _wait_opencode_process reports an error string, so status should be 'error', not 'success'."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_e2f5ae65131e43beb1f3e877b60255e1_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_e2f5ae65131e43beb1f3e877b60255e1_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_e2f5ae65131e43beb1f3e877b60255e1_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: trace_writer

FMA-MISMATCH-182 — [src--trace_writer-py--append_event](#)

- 四类结论：可能有 Bug

- **分类依据:** 沿用旧 `fm_agent/bug_list.md` 的逐条审计结论: 沿用旧清单的逐条审计结论。
- **Validator:** `confirmed`; attempts `1`
- **Phase / 模块:** Phase 4 — LLM Communication & Tracing / `trace_writer`; 原源码 `src/trace_writer.py`
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Thread-safely appends the event dict as a single, self-contained JSON line (terminated by a newline character) to the `events.jsonl` file located within `trace_dir`. The written JSON preserves every top-level key from the provided event dict. The function does not modify the event dict argument. If the `trace_dir` directory hierarchy does not exist, it is created before the write. On filesystem write failure (e.g., disk full, permission denied), an `OSError` propagates to the caller.
- **Actual behavior:** Normal execution: The function returns `None`. The lock `_LOCK` is acquired and released. The file `os.path.join(trace_dir, 'events.jsonl')` has a new line appended containing `json.dumps(event, ensure_ascii=False) + '\n'`. The directory `trace_dir` exists (created if necessary by `_ensure_trace_dirs`).

Exceptional paths:

- `_ensure_trace_dirs` raises `OSError`: The function exits with `OSError`. No lock is acquired. No file write occurs. No side effects on the event file.
- `json.dumps` raises `TypeError`: The function exits with `TypeError`. The trace directory may have been created (if `_ensure_trace_dirs` succeeded), but no lock is acquired and no file write occurs.
- File operations (`open` or `write`) raise `OSError` while the lock is held: The lock is released (guaranteed by the `with` statement). The trace directory exists (created if needed). The file may be partially written or unchanged; no atomicity guarantee. The `OSError` propagates.

Formal: post

(returns `None`)

`directory_exists(trace_dir)`

`appended_line(file_path, json.dumps(event, ensure_ascii=False) + '\n')`

`lock_released(_LOCK)` raises `OSError` from `_ensure_trace_dirs` (no file modifications) raises `TypeError` from `json.dumps` (directory may exist, no file modifications) raises `OSError` from file I/O (directory exists, lock released, file state undefined)

- **Code evidence:** Line 4: `line = json.dumps(event, ensure_ascii=False)`
- **Trigger condition:** The code calls `json.dumps(event)` on the input dict unconditionally. If the event dict contains a top-level key that is not a valid JSON key (e.g., a tuple), `json.dumps` raises `TypeError`. The specification (B) only allows for `OSError` propagation on filesystem write failure and does not account for `TypeError`. The input `event={1,2: 'value'}` is a valid dict, yet the codes behavior (`TypeError`) violates the specified post-condition, which does not include this exceptional outcome.
- **Validator trigger:** Passing an event dict with a non-string key (e.g., a tuple) causes `json.dumps` to raise `TypeError`, which is not documented in the specification.
- **Probe stdout:** `CONFIRMED` — `TypeError` raised when `json.dumps` encounters non-serializable dict key (1, 2); spec only permits `OSError`

► SPEC sidecar (完整)

```
{
  "signature": "append_event(trace_dir, event) -> None",
  "pre_condition": "trace_dir is a string path. event is a dict containing at minimum an 'event_id' field with a non-empty string value and a 'timestamp' field with an ISO 8601 formatted string value.",
  "post_condition": "Thread-safely appends the event dict as a single, self-contained JSON line (terminated by a newline character) to the events.jsonl file located within trace_dir. The written JSON preserves every top-level key from the provided event dict. The function does not modify the event dict argument. If the trace_dir directory hierarchy does not exist, it is created before the write. On filesystem write failure (e.g., disk full, permission denied), an OSError propagates to the caller."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_ensure_trace_dirs",
      "signature": "_ensure_trace_dirs(trace_dir) -> None",
      "pre_condition": "trace_dir is a string path.",
      "post_condition": "If trace_dir does not exist, creates trace_dir and any missing parent directories. If trace_dir already exists, no side effects occur. On directory creation failure (e.g., permission denied), an OSError propagates to the caller."
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/trace_writer-py/append_event.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Thread-safely appends the event dict as a single, self-contained JSON line (terminated by a newline character) to the events.jsonl file located within trace_dir. The written JSON preserves every top-level key from the provided event dict. The function does not modify the event dict argument. If the trace_dir directory hierarchy does not exist, it is created before the write. On filesystem write failure (e.g., disk full, permission denied), an OSError propagates to the caller.",
    "actual_behavior": "Normal execution: The function returns None. The lock `_LOCK` is acquired and released. The file `os.path.join(trace_dir, 'events.jsonl')` has a new line appended containing `json.dumps(event, ensure_ascii=False) + '\\n'`. The directory `trace_dir` exists (created if necessary by `_ensure_trace_dirs`).\\n\\nExceptional paths:\\n- `_ensure_trace_dirs` raises `OSError`: The function exits with `OSError`. No lock is acquired. No file write occurs. No side effects on the event file.\\n- `json.dumps` raises `TypeError`: The function exits with `TypeError`. The trace directory may have been created (if `_ensure_trace_dirs` succeeded), but no lock is acquired and no file write occurs.\\n- File operations (`open` or `write`) raise `OSError` while the lock is held: The lock is released (guaranteed by the `with` statement). The trace directory exists (created if needed). The file may be partially written or unchanged; no atomicity guarantee. The `OSError` propagates.\\n\\nFormal:\\npost `\\n` (returns None `\\n` directory_exists(trace_dir) `\\n` appended_line(file_path, json.dumps(event, ensure_ascii=False) + '\\n') `\\n` lock_released(_LOCK))\\n `\\n` raises OSError from _ensure_trace_dirs (no file modifications)\\n `\\n` raises TypeError from json.dumps (directory may exist, no file modifications)\\n `\\n` raises OSError from file I/O (directory exists, lock released, file state undefined)",
    "code_evidence": "Line 4: line = json.dumps(event, ensure_ascii=False)",
    "trigger_condition": "The code calls json.dumps(event) on the input dict unconditionally. If the event dict contains a top-level key that is not a valid JSON key (e.g., a tuple), json.dumps raises TypeError. The specification (B) only allows for OSError propagation on filesystem write failure and does not account for TypeError. The input event=({1,2}: 'value') is a valid dict, yet the codes behavior (TypeError) violates the specified post-condition, which does not include this exceptional outcome."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--trace_writer-py--append_event",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/trace_writer-py/append_event.py",
  "function_name": "append_event",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--trace_writer-py--append_event.py",
  "detail_file": "fm_agent/bug_validation/src--trace_writer-py--append_event.md",
  "error_stdout": "CONFIRMED – TypeError raised when json.dumps encounters non-serializable dict key (1, 2); spec only permits OSError\\n",
  "trigger_summary": "Passing an event dict with a non-string key (e.g., a tuple) causes json.dumps to raise TypeError, which is not documented in the specification."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:13:06.689906Z	2919
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:13:48.217586Z	7545

事件 1 — [llm_c36e6cc765274ceaa86be26bbb74cf48](#)

► parsed_json（完整）

```
{
  "post_condition": "Normal execution: The function returns None. The lock `_LOCK` is acquired and released. The file `os.path.join(trace_dir, 'events.jsonl')` has a new line appended containing `json.dumps(event, ensure_ascii=False) + '\\n'`. The directory `trace_dir` exists (created if necessary by `_ensure_trace_dirs`).\\n\\nExceptional paths:\\n- `_ensure_trace_dirs` raises `OSError`: The function exits with `OSError`. No lock is acquired. No file write occurs. No side effects on the event file.\\n- `json.dumps` raises `TypeError`: The function exits with `TypeError`. The trace directory may have been created (if `_ensure_trace_dirs` succeeded), but no lock is acquired and no file write occurs.\\n- File operations (`open` or `write`) raise `OSError` while the lock is held: The lock is released (guaranteed by the `with` statement). The trace directory exists (created if needed). The file may be partially written or unchanged; no atomicity guarantee. The `OSError` propagates.\\n\\nFormal:\\npost = `\\n` (returns None `\\n` directory_exists(trace_dir) `\\n` appended_line(file_path, json.dumps(event, ensure_ascii=False) + '\\n') `\\n` lock_released(_LOCK))\\n `\\n` v raises OSError from _ensure_trace_dirs (no file modifications)\\n `\\n` v raises TypeError from json.dumps (directory may exist, no file modifications)\\n `\\n` v raises OSError from file I/O (directory exists, lock released, file state undefined)"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c36e6cc765274ceaa86be26bbb74cf48_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_c36e6cc765274ceaa86be26bbb74cf48_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_c36e6cc765274ceaa86be26bbb74cf48_response.txt)

事件 2 — [llm_47e6e11b57454141b87c3ea7d5016927](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "trace_dir='/tmp/test', event={(1,2): 'value'}",
  "offending_statements": "Line 4: line = json.dumps(event, ensure_ascii=False)",
  "reason": "The code calls json.dumps(event) on the input dict unconditionally. If the event dict contains a top-level key that is not a valid JSON key (e.g., a tuple), json.dumps raises TypeError. The specification (B) only allows for OSError propagation on filesystem write failure and does not account for TypeError. The input event={(1,2): 'value'} is a valid dict, yet the code's behavior (TypeError) violates the specified post-condition, which does not include this exceptional outcome."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_47e6e11b57454141b87c3ea7d5016927_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_47e6e11b57454141b87c3ea7d5016927_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_47e6e11b57454141b87c3ea7d5016927_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

Phase 5 — Specification & Verification Utilities

模块: [file_utils](#)

FMA-MISMATCH-192 — [src--file_utils-py--_get_incomplete_verification_files](#)

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: [confirmed](#); attempts 1
- Phase / 模块: Phase 5 — Specification & Verification Utilities / [file_utils](#); 原源码 [src/file_utils.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a list of file paths from `layer_files` that require further processing, preserving the original input order. A file requires further processing if (a) its corresponding verification result file at `<output_dir>/<file_without_extension>.json` cannot be read as valid JSON (file missing, unreadable, or not valid JSON), or (b) the verification result has a top-level 'verdict' key equal to the string 'MISMATCH' AND the corresponding bug validation result file at `<work_dir>/bug_validation/<bug_id>.result.json` is not a valid JSON file as determined by `_json_file_is_valid`, where `bug_id` is derived from the relative path by replacing every occurrence of '/' and `os.sep` with '--'. Files with a readable verification result whose 'verdict' is not 'MISMATCH' are excluded from the returned list.
- **Actual behavior:** After normal execution, the function returns a list `incomplete` that is a subsequence (in iteration order) of the elements from `layer_files`. For each relative path `rel` in `layer_files`, `rel` is included in `incomplete` if and only if either: (a) the file at `os.path.join(output_dir, os.path.splitext(rel)[0] + '.json')` does not exist, cannot be opened for reading, or contains syntactically invalid JSON; or (b) that file exists, is readable, contains valid JSON, the loaded object has `result.get('verdict') == 'MISMATCH'`, and the bug validation file at path `os.path.join(work_dir, 'bug_validation', (os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')) + '.result.json')` is not valid according to `_json_file_is_valid` (i.e., it does not exist, is not readable, or contains invalid JSON). All other elements from `layer_files` are omitted. If any unhandled exception occurs (e.g., from the `os` operations), the function may raise an exception and not return a value. Formal logic: Let `L` be the sequence of elements obtained by iterating over `layer_files`. Then the returned list `incomplete` satisfies: `incomplete = [rel for rel in L | (let P = os.path.join(output_dir, os.path.splitext(rel)[0] + '.json') in ((P exists P readable P contains valid JSON)) (P exists P readable P contains valid JSON let result = load_json(P) in result.get('verdict') = 'MISMATCH' let bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--') in _json_file_is_valid(os.path.join(work_dir, 'bug_validation', bug_id + '.result.json')))]`
- **Code evidence:** Line 14: `bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')`
- **Trigger condition:** The specification requires `bug_id` to be derived from the relative path by replacing every '/' and `os.sep` with '--' without removing the file extension. The code uses `os.path.splitext` to strip the extension before replacement, causing a different bug validation file

path to be checked. When the spec's path exists and is valid but the code's path is missing, the code erroneously includes the file in the incomplete list, violating the specification.

- **Validator trigger:** Code uses `os.path.splitext` to strip the file extension before building `bug_id`, but the spec requires including the extension; when a bug validation result exists at the spec path but not at the code's `splitext`-derived path, the function incorrectly marks the entry as incomplete.
- **Probe stdout:** Probe workspace: `/tmp/probe_9vajms6m` CONFIRMED — actual: `['foo/bar.py']` | expected: `[]` | `spec_validation_path` exists: True

► SPEC sidecar (完整)

```
{
  "signature": "_get_incomplete_verification_files(layer_files, input_dir, output_dir, work_dir) -> list",
  "pre_condition": "layer_files is an iterable of relative file path strings. output_dir is a string path to a directory containing verification result JSON files. work_dir is a string path to a directory potentially containing a bug_validation/ subdirectory.",
  "post_condition": "Returns a list of file paths from layer_files that require further processing, preserving the original input order. A file requires further processing if (a) its corresponding verification result file at '<output_dir>/<file_without_extension>.json' cannot be read as valid JSON (file missing, unreadable, or not valid JSON), or (b) the verification result has a top-level 'verdict' key equal to the string 'MISMATCH' AND the corresponding bug validation result file at '<work_dir>/bug_validation/<bug_id>.result.json' is not a valid JSON file as determined by _json_file_is_valid, where bug_id is derived from the relative path by replacing every occurrence of '/' and os.sep with '--'. Files with a readable verification result whose 'verdict' is not 'MISMATCH' are excluded from the returned list."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_json_file_is_valid",
      "signature": "_json_file_is_valid(file_path) -> bool",
      "pre_condition": "file_path is a string path to a file whose existence and readability are not yet established.",
      "post_condition": "Returns True if the file at file_path exists, is readable, and contains syntactically valid JSON; returns False otherwise. Does not raise an exception for a missing file, a permission-denied read, or malformed JSON content."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-py/_get_incomplete_verification_files.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a list of file paths from layer_files that require further processing, preserving the original input order. A file requires further processing if (a) its corresponding verification result file at '<output_dir>/<file_without_extension>.json' cannot be read as valid JSON (file missing, unreadable, or not valid JSON), or (b) the verification result has a top-level 'verdict' key equal to the string 'MISMATCH' AND the corresponding bug validation result file at '<work_dir>/bug_validation/<bug_id>.result.json' is not a valid JSON file as determined by _json_file_is_valid, where bug_id is derived from the relative path by replacing every occurrence of '/' and os.sep with '--'. Files with a readable verification result whose 'verdict' is not 'MISMATCH' are excluded from the returned list.",
    "actual_behavior": "After normal execution, the function returns a list `incomplete` that is a subsequence (in iteration order) of the elements from `layer_files`. For each relative path `rel` in `layer_files`, `rel` is included in `incomplete` if and only if either: (a) the file at `os.path.join(output_dir, os.path.splitext(rel)[0] + '.json')` does not exist, cannot be opened for reading, or contains syntactically invalid JSON; or (b) that file exists, is readable, contains valid JSON, the loaded object has `result.get('verdict') == 'MISMATCH'`, and the bug validation file at path `os.path.join(work_dir, 'bug_validation', (os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')) + '.result.json')` is not valid according to `_json_file_is_valid` (i.e., it does not exist, is not readable, or contains invalid JSON). All other elements from `layer_files` are omitted. If any unhandled exception occurs (e.g., from the `os` operations), the function may raise an exception and not return a value. Formal logic: Let L be the sequence of elements obtained by iterating over `layer_files`. Then the returned list `incomplete` satisfies: `incomplete = [ rel for rel in L | ( let P = os.path.join(output_dir, os.path.splitext(rel)[0] + '.json') in ( (P exists P readable P contains valid JSON) ) ) ( P exists P readable P contains valid JSON let result = load_json(P) in result.get('verdict') = 'MISMATCH' let bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--') in _json_file_is_valid(os.path.join(work_dir, 'bug_validation', bug_id + '.result.json')) ) ]`",
    "code_evidence": "Line 14: bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')",
    "trigger_condition": "The specification requires bug_id to be derived from the relative path by replacing every '/' and os.sep with '--' without removing the file extension. The code uses os.path.splitext to strip the extension before replacement, causing a different bug validation file path to be checked. When the spec's path exists and is valid but the code's path is missing, the code erroneously includes the file in the incomplete list, violating the specification."
  }
}
```

► Bug Validator result (完整)


```
{
  "id": "src--file_utils-py--_get_incomplete_verification_files",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-
py/_get_incomplete_verification_files.py",
  "function_name": "_get_incomplete_verification_files",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--file_utils-py--_get_incomplete_verification_files.py",
  "detail_file": "fm_agent/bug_validation/src--file_utils-py--_get_incomplete_verification_files.md",
  "probe_stdout": "Probe workspace: /tmp/probe_9vajms6m\n\nCONFIRMED - actual: ['foo/bar.py'] | expected: [] | spec_validation_path
exists: True",
  "trigger_summary": "Code uses os.path.splitext to strip the file extension before building bug_id, but the spec requires
including the extension; when a bug validation result exists at the spec path but not at the code's splitext-derived path, the
function incorrectly marks the entry as incomplete."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:29:25.645570Z	2602
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:30:01.773481Z	4450

事件 1 — [llm_747b2d5dac8c45b484265daec26809af](#)

► parsed_json（完整）

```
{
  "post_condition": "After normal execution, the function returns a list `incomplete` that is a subsequence (in iteration order)
of the elements from `layer_files`. For each relative path `rel` in `layer_files`, `rel` is included in `incomplete` if and only
if either: (a) the file at `os.path.join(output_dir, os.path.splitext(rel)[0] + '.json')` does not exist, cannot be opened for
reading, or contains syntactically invalid JSON; or (b) that file exists, is readable, contains valid JSON, the loaded object has
`result.get('verdict') == 'MISMATCH'`, and the bug validation file at path `os.path.join(work_dir, 'bug_validation',
(os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')) + '.result.json')` is not valid according to
`_json_file_is_valid` (i.e., it does not exist, is not readable, or contains invalid JSON). All other elements from `layer_files`
are omitted. If any unhandled exception occurs (e.g., from the `os` operations), the function may raise an exception and not
return a value. Formal logic: Let L be the sequence of elements obtained by iterating over `layer_files`. Then the returned list
`incomplete` satisfies: `incomplete = [ rel for rel in L | ( let P = os.path.join(output_dir, os.path.splitext(rel)[0] + '.json')
in ( ~(P exists ^ P readable ^ P contains valid JSON) ) ) v ( P exists ^ P readable ^ P contains valid JSON ^ let result =
load_json(P) in result.get('verdict') = 'MISMATCH' ^ let bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')
in ~_json_file_is_valid(os.path.join(work_dir, 'bug_validation', bug_id + '.result.json')) ) ]`"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_747b2d5dac8c45b484265daec26809af_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_747b2d5dac8c45b484265daec26809af_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_747b2d5dac8c45b484265daec26809af_response.txt)

事件 2 — [llm_be126ba68545437aad50f27a13c1778c](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "layer_files = ['app.py'], output_dir = '/out', work_dir = '/work', with filesystem state: /out/app.json
exists and contains {\"verdict\": \"MISMATCH\"}; /work/bug_validation/app.py.result.json exists and is valid JSON;
/work/bug_validation/app.result.json does not exist.",
  "offending_statements": "Line 14: bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--)",
  "reason": "The specification requires bug_id to be derived from the relative path by replacing every '/' and os.sep with '--'
without removing the file extension. The code uses os.path.splitext to strip the extension before replacement, causing a different
bug validation file path to be checked. When the spec's path exists and is valid but the code's path is missing, the code
erroneously includes the file in the incomplete list, violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_be126ba68545437aad50f27a13c1778c_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_be126ba68545437aad50f27a13c1778c_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_be126ba68545437aad50f27a13c1778c_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

- 四类结论： 可能有 Bug
- 分类依据： probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险， 列为实现缺陷候选。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 5 — Specification & Verification Utilities / [file_utils](#); 原源码 [src/file_utils.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a list of file identifiers (relative paths from input_dir) for all extracted function files belonging to the specified phase. Each identifier is a relative path to a regular file that is not a metadata sidecar file. No file identifier appears more than once in the returned list. Files within each extracted directory are returned in lexicographically ascending order of their names. Source files whose derived extracted directory does not exist on disk are skipped without raising an error.
- **Actual behavior**: If the pre-condition holds and phases_data is well-formed, the function returns a list of relative file paths (relative to input_dir) of regular non-metadata files found within extracted directories corresponding to source files of the chosen phase. For each module in the phase and each source file in the module, build extracted_dir = os.path.join(input_dir, os.path.dirname(src_file), subdir) where subdir is the base name with the last dot replaced by '-' if present, else unchanged. If that directory exists, recursively walk it via os.walk, collecting paths of regular files (excluding those where *is_metadata_sidecar* returns True) in alphabetical filename order; append their relative paths to the result. The order preserves module/source-file iteration and, within each directory, alphabetical filenames. Violations of structural assumptions (missing keys, non-iterable values, wrong types) result in a *KeyError*, *TypeError*, *AttributeError*, or *StopIteration*. The returned list may contain duplicate relative paths if the same file is encountered in multiple directories. Formal logic: $R = \text{concatenation}\{m \text{ in modules}\} \text{ concatenation}_{\{s \text{ in } m[\text{'source_files'}]\}} [\text{os.path.relpath}(\text{os.path.join}(\text{root}, \text{fname}), \text{input_dir}) \text{ for } (\text{root}, _, \text{fnames}) \text{ in sorted}(\text{os.walk}(\text{extracted_dir}(s))) \text{ for } \text{fname} \text{ in sorted}(\text{fnames}) \text{ if } \text{os.path.isfile}(\text{os.path.join}(\text{root}, \text{fname})) \text{ and not } _is_metadata_sidecar(\text{fname})]$ where extracted_dir(s) is defined as above.
- **Code evidence**: Line 24: phase_files.append(os.path.relpath(fpath, input_dir))
- **Trigger condition**: The code never removes duplicates. If the same extracted directory is visited multiple times (e.g., because source_files contains duplicate entries), the same relative path is appended multiple times. The specification forbids any duplicate file identifiers.
- **Validator trigger**: Duplicate entries in source_files cause the same extracted directory to be visited multiple times, appending identical relative paths to the result list without deduplication.
- **Probe stdout**: CONFIRMED — actual: ['foo-py/some_func.py', 'foo-py/some_func.py'] | expected: non-duplicate list of length 1 total entries=2, unique=1, duplicates=1

► SPEC sidecar (完整)

```
{
  "signature": "_get_phase_files(phases_data, phase_num, input_dir) -> list",
  "pre_condition": "phases_data is the full phases configuration dict whose key 'phases' maps to a list of dicts, at least one of which has a 'phase' key equal to phase_num. phase_num is an integer identifying a phase. input_dir is a string path to the extracted functions directory.",
  "post_condition": "Returns a list of file identifiers (relative paths from input_dir) for all extracted function files belonging to the specified phase. Each identifier is a relative path to a regular file that is not a metadata sidecar file. No file identifier appears more than once in the returned list. Files within each extracted directory are returned in lexicographically ascending order of their names. Source files whose derived extracted directory does not exist on disk are skipped without raising an error."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_is_metadata_sidecar",
      "signature": "_is_metadata_sidecar(fname) -> bool",
      "pre_condition": "fname is a string representing a filename (the base name component of a file path).",
      "post_condition": "Returns True when fname identifies a metadata sidecar file (a .spec.json or .info.json file accompanying an extracted function source file); returns False otherwise."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-py/_get_phase_files.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a list of file identifiers (relative paths from input_dir) for all extracted function files belonging to the specified phase. Each identifier is a relative path to a regular file that is not a metadata sidecar file. No file identifier appears more than once in the returned list. Files within each extracted directory are returned in lexicographically ascending order of their names. Source files whose derived extracted directory does not exist on disk are skipped without raising an error.",
    "actual_behavior": "If the pre-condition holds and phases_data is well-formed, the function returns a list of relative file paths (relative to input_dir) of regular non-metadata files found within extracted directories corresponding to source files of the chosen phase. For each module in the phase and each source file in the module, build extracted_dir = os.path.join(input_dir, os.path.dirname(src_file), subdir) where subdir is the base name with the last dot replaced by '-' if present, else unchanged. If that directory exists, recursively walk it via os.walk, collecting paths of regular files (excluding those where _is_metadata_sidecar returns True) in alphabetical filename order; append their relative paths to the result. The order preserves module/source-file iteration and, within each directory, alphabetical filenames. Violations of structural assumptions (missing keys, non-iterable values, wrong types) result in a KeyError, TypeError, AttributeError, or StopIteration. The returned list may contain duplicate relative paths if the same file is encountered in multiple directories. Formal logic: R = concatenation_{m in modules} concatenation_{s in m['source_files']} [ os.path.relpath(os.path.join(root, fname), input_dir) for (root, _, fnames) in sorted(os.walk(extracted_dir(s))) for fname in sorted(fnames) if os.path.isfile(os.path.join(root, fname)) and not _is_metadata_sidecar(fname) ] where extracted_dir(s) is defined as above.",
    "code_evidence": "Line 24: phase_files.append(os.path.relpath(fpath, input_dir))",
    "trigger_condition": "The code never removes duplicates. If the same extracted directory is visited multiple times (e.g., because source_files contains duplicate entries), the same relative path is appended multiple times. The specification forbids any duplicate file identifiers."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--file_utils-py--_get_phase_files",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-py/_get_phase_files.py",
  "function_name": "_get_phase_files",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--file_utils-py--_get_phase_files.py",
  "detail_file": "fm_agent/bug_validation/src--file_utils-py--_get_phase_files.md",
  "probe_stdout": "CONFIRMED - actual: ['foo-py/some_func.py', 'foo-py/some_func.py'] | expected: non-duplicate list of length 1\n total entries=2, unique=1, duplicates=1\n",
  "trigger_summary": "Duplicate entries in source_files cause the same extracted directory to be visited multiple times, appending identical relative paths to the result list without deduplication."
}
```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-28T03:36:34.081205Z	4307
2	generate_block_post_condition	success	2	0/1	2026-07-28T03:37:31.521792Z	2455
3	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:37:43.691716Z	4097

事件 1 — [llm_0b61691dcbfd423080865af2fae6cff4](#)

- parse_error: [LLM response contains multiple JSON values](#)
- 载荷: [system_prompt](#) · [user_prompt](#) · [assistant_output](#)

事件 2 — [llm_074b998bcf9a40099c77604c9157ecfd](#)

► parsed_json（完整）

```
{
  "post_condition": "If the pre-condition holds and phases_data is well-formed, the function returns a list of relative file paths (relative to input_dir) of regular non-metadata files found within extracted directories corresponding to source files of the chosen phase. For each module in the phase and each source file in the module, build extracted_dir = os.path.join(input_dir, os.path.dirname(src_file), subdir) where subdir is the base name with the last dot replaced by '-' if present, else unchanged. If that directory exists, recursively walk it via os.walk, collecting paths of regular files (excluding those where _is_metadata_sidecar returns True) in alphabetical filename order; append their relative paths to the result. The order preserves module/source-file iteration and, within each directory, alphabetical filenames. Violations of structural assumptions (missing keys, non-iterable values, wrong types) result in a KeyError, TypeError, AttributeError, or StopIteration. The returned list may contain duplicate relative paths if the same file is encountered in multiple directories. Formal logic: R = concatenation_{m in modules} concatenation_{s in m['source_files']} [ os.path.relpath(os.path.join(root, fname), input_dir) for (root, _, fnames) in
```

```
sorted(os.walk(extracted_dir(s))) for fname in sorted(fnames) if os.path.isfile(os.path.join(root, fname)) and not
_is_metadata_sidecar(fname) ] where extracted_dir(s) is defined as above."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_message_02_assistant.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_message_03_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_response.txt)

事件 3 — [llm_7dac04c8afb243c6805e53d4d5b172f8](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Suppose phases_data = {'phases': [{'phase': 1, 'modules': [{'source_files': ['dir/file.cpp',
'dir/file.cpp']}]}}], phase_num = 1, input_dir = '/tmp'. The directory '/tmp/dir/file-cpp' exists and contains a regular file
'func.cpp' that is not a metadata sidecar. The code processes 'dir/file.cpp' twice, each time walking the same extracted
directory, resulting in phase_files = ['dir/file-cpp/func.cpp', 'dir/file-cpp/func.cpp']. This list contains duplicate
identifiers, which violates the specification's requirement that no file identifier appears more than once.",
  "offending_statements": "Line 24: phase_files.append(os.path.relpath(fpath, input_dir))",
  "reason": "The code never removes duplicates. If the same extracted directory is visited multiple times (e.g., because
source_files contains duplicate entries), the same relative path is appended multiple times. The specification forbids any
duplicate file identifiers."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_7dac04c8afb243c6805e53d4d5b172f8_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_7dac04c8afb243c6805e53d4d5b172f8_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_7dac04c8afb243c6805e53d4d5b172f8_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-198 — [src--file_utils-py--_json_file_is_valid](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 5 — Specification & Verification Utilities / [file_utils](#); 原源码 [src/file_utils.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns True exactly when the file at `file_path` exists in the filesystem, can be opened for reading, and its entire contents constitute syntactically valid JSON. Returns False when the file does not exist, cannot be opened for reading, or its contents are not syntactically valid JSON. Never raises an exception; every error condition including missing file, permission-denied read, and malformed JSON is caught and results in a False return value.
- **Actual behavior**: After execution, the function returns a boolean. If True, the file at `path` existed at call time, was readable, and contained syntactically valid JSON; if False, either the file did not exist, was not readable, did not contain valid JSON, or another OS/JSON error occurred. The file is closed and unmodified. Formally: Let `result = _json_file_is_valid(p)`. Then `(result = True) (exists_file(p) readable(p) valid_json(contents(p)))`, and `(result = False) (exists_file(p) readable(p) valid_json(contents(p))) raised(SError JSONDecodeError)`. The valuation is snapshotted at the time of the call.
- **Code evidence**: Line 6: `except (SError, json.JSONDecodeError):`
- **Trigger condition**: The code only catches `SError` and `json.JSONDecodeError`. Opening the file in text mode (line 3) may raise a `UnicodeDecodeError` if the file contents cannot be decoded by the default encoding. This exception is not caught, so the function raises an exception instead of returning False as required by the specification ('Never raises an exception; every error condition ... is caught and results in a False return value').
- **Validator trigger**: Opening a file with non-UTF-8 bytes in text mode raises `UnicodeDecodeError`, which is not caught by the `except (SError, json.JSONDecodeError)` clause.
- **Probe stdout**: CONFIRMED — `_json_file_is_valid` raised `UnicodeDecodeError` instead of returning False

► SPEC sidecar (完整)

```
{
  "signature": "_json_file_is_valid(file_path) -> bool",
  "pre_condition": "file_path is a string path to a file whose existence and readability are not yet established.",
  "post_condition": "Returns True exactly when the file at file_path exists in the filesystem, can be opened for reading, and its entire contents constitute syntactically valid JSON. Returns False when the file does not exist, cannot be opened for reading, or its contents are not syntactically valid JSON. Never raises an exception; every error condition – including missing file, permission-denied read, and malformed JSON – is caught and results in a False return value."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-py/_json_file_is_valid.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns True exactly when the file at file_path exists in the filesystem, can be opened for reading, and its entire contents constitute syntactically valid JSON. Returns False when the file does not exist, cannot be opened for reading, or its contents are not syntactically valid JSON. Never raises an exception; every error condition including missing file, permission-denied read, and malformed JSON is caught and results in a False return value.",
    "actual_behavior": "After execution, the function returns a boolean. If True, the file at `path` existed at call time, was readable, and contained syntactically valid JSON; if False, either the file did not exist, was not readable, did not contain valid JSON, or another OS/JSON error occurred. The file is closed and unmodified. Formally: Let result = _json_file_is_valid(p). Then (result = True) => (exists_file(p) & readable(p) & valid_json(contents(p))), and (result = False) => (exists_file(p) & readable(p) & valid_json(contents(p))) raised(OSError JSONDecodeError). The valuation is snapshotted at the time of the call.",
    "code_evidence": "Line 6:     except (OSError, json.JSONDecodeError):",
    "trigger_condition": "The code only catches OSError and json.JSONDecodeError. Opening the file in text mode (line 3) may raise a UnicodeDecodeError if the file contents cannot be decoded by the default encoding. This exception is not caught, so the function raises an exception instead of returning False as required by the specification ('Never raises an exception; every error condition ... is caught and results in a False return value')."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--file_utils-py--_json_file_is_valid",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-py/_json_file_is_valid.py",
  "function_name": "_json_file_is_valid",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--file_utils-py--_json_file_is_valid.py",
  "detail_file": "fm_agent/bug_validation/src--file_utils-py--_json_file_is_valid.md",
  "probe_stdout": "CONFIRMED - _json_file_is_valid raised UnicodeDecodeError instead of returning False\n",
  "trigger_summary": "Opening a file with non-UTF-8 bytes in text mode raises UnicodeDecodeError, which is not caught by the except (OSError, json.JSONDecodeError) clause."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:35:46.013343Z	1432
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:36:06.857865Z	4895

事件 1 — [llm_49646d1fbacb499695413fa041172583](#)

► parsed_json（完整）

```
{
  "post_condition": "After execution, the function returns a boolean. If True, the file at `path` existed at call time, was readable, and contained syntactically valid JSON; if False, either the file did not exist, was not readable, did not contain valid JSON, or another OS/JSON error occurred. The file is closed and unmodified. Formally: Let result = _json_file_is_valid(p). Then (result = True) => (exists_file(p) & readable(p) & valid_json(contents(p))), and (result = False) => ~(exists_file(p) & readable(p) & valid_json(contents(p)))"
}
```

```
    if not valid_json(contents(p))) v raised(OSError v JSONDecodeError). The valuation is snapshotted at the time of the call."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_49646d1fbacb499695413fa041172583_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_49646d1fbacb499695413fa041172583_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_49646d1fbacb499695413fa041172583_response.txt)

事件 2 — llm_5e69c08b444847259541f838c7eaf5f3

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "path points to a file that exists and is readable by the OS but contains bytes that are not valid for the default text encoding (e.g., a file named 'broken.txt' containing the byte 0xFF when the locale encoding is UTF-8).",
  "offending_statements": "Line 6:         except (OSError, json.JSONDecodeError):",
  "reason": "The code only catches OSErrors and json.JSONDecodeErrors. Opening the file in text mode (line 3) may raise a UnicodeDecodeError if the file contents cannot be decoded by the default encoding. This exception is not caught, so the function raises an exception instead of returning False as required by the specification ('Never raises an exception; every error condition ... is caught and results in a False return value')."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5e69c08b444847259541f838c7eaf5f3_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5e69c08b444847259541f838c7eaf5f3_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5e69c08b444847259541f838c7eaf5f3_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

Phase 6 — Reasoning & Verification Engine

模块: entry_pipeline

FMA-MISMATCH-205 — src--entry_reasoning_pipeline-py--_count_mismatches

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: not_confirmed; attempts 2
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / entry_pipeline; 原源码 src/entry_reasoning_pipeline.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** Returns a non-negative integer equal to the count of JSON files reachable by recursively walking the directory tree rooted at results_dir whose deserialized 'verdict' field equals the literal string 'MISMATCH'. A JSON file that cannot be opened for reading or whose content is not valid JSON does not contribute to the count. When results_dir does not exist as a directory, returns 0.
- **Actual behavior:** If the function terminates normally (returns), the return value is an integer count such that count = |{ f ∈ F | f.name.endswith('.json') ∧ readable_json_object(f) ∧ decoded(f).get('verdict') == 'MISMATCH' }|, where F is the set of all regular files discovered by os.walk(results_dir), readable_json_object(f) holds iff opening and reading f succeeds and its content is valid JSON representing a mapping object. Files that cause OSErrors or ValueError during opening, reading, or JSON decoding are excluded from the count. If the function terminates with an uncaught exception, that exception is an OSErrors (or subclass) raised by os.walk(results_dir) (e.g., because results_dir does not exist, is not a directory, or cannot be accessed); in this case no value is returned.
- **Code evidence:** Line 8: for root, _dirs, files in os.walk(results_dir):
- **Trigger condition:** Specification requires returning 0 when results_dir does not exist as a directory, but the code allows os.walk to raise an uncaught OSErrors (e.g., FileNotFoundError) instead of returning 0.
- **Validator trigger:** os.walk in Python 3.12+ silently yields nothing for non-existent/non-directory paths, so the function returns 0 as specified; bug only reproducible on Python <3.12.
- **Probe stdout:** NOT CONFIRMED — function returns 0 for all trigger conditions Details: non-existent directory: returned 0 (no exception); regular file (not a directory): returned 0 (no exception); unreadable directory: returned 0 (no exception) Note: Python 3.12.3 os.walk silently handles these cases.

► SPEC sidecar (完整)

```
{
  "signature": "_count_mismatches(results_dir)",
  "pre_condition": "results_dir is a filesystem path.",
  "post_condition": "Returns a non-negative integer equal to the count of JSON files reachable by recursively walking the
directory tree rooted at results_dir whose deserialized 'verdict' field equals the literal string 'MISMATCH'. A JSON file that
cannot be opened for reading or whose content is not valid JSON does not contribute to the count. When results_dir does not exist
as a directory, returns 0."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/entry_reasoning_pipeline-
py/_count_mismatches.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a non-negative integer equal to the count of JSON files reachable by recursively walking the directory
tree rooted at results_dir whose deserialized 'verdict' field equals the literal string 'MISMATCH'. A JSON file that cannot be
opened for reading or whose content is not valid JSON does not contribute to the count. When results_dir does not exist as a
directory, returns 0.",
    "actual_behavior": "If the function terminates normally (returns), the return value is an integer `count` such that `count = |
{ f | F | f.name.endswith('.json') readable_json_object(f) decoded(f).get('verdict') == 'MISMATCH' }|`, where `F` is the set of
all regular files discovered by `os.walk(results_dir)`, `readable_json_object(f)` holds iff opening and reading `f` succeeds and
its content is valid JSON representing a mapping object. Files that cause `OSError` or `ValueError` during opening, reading, or
JSON decoding are excluded from the count. If the function terminates with an uncaught exception, that exception is an `OSError`
(or subclass) raised by `os.walk(results_dir)` (e.g., because `results_dir` does not exist, is not a directory, or cannot be
accessed); in this case no value is returned.",
    "code_evidence": "Line 8: for root, _dirs, files in os.walk(results_dir):",
    "trigger_condition": "Specification requires returning 0 when results_dir does not exist as a directory, but the code allows
os.walk to raise an uncaught OSError (e.g., FileNotFoundError) instead of returning 0."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--entry_reasoning_pipeline-py--_count_mismatches",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/entry_reasoning_pipeline-
py/_count_mismatches.py",
  "function_name": "_count_mismatches",
  "confirmation_status": "not_confirmed",
  "attempts": 2,
  "probe_script": "fm_agent/bug_validation/probe_src--entry_reasoning_pipeline-py--_count_mismatches.py",
  "detail_file": "fm_agent/bug_validation/src--entry_reasoning_pipeline-py--_count_mismatches.md",
  "probe_stdout": "NOT CONFIRMED – function returns 0 for all trigger conditions\nDetails: non-existent directory: returned 0 (no
exception); regular file (not a directory): returned 0 (no exception); unreadable directory: returned 0 (no exception)\nNote:
Python 3.12.3 os.walk silently handles these cases.",
  "trigger_summary": "os.walk in Python 3.12+ silently yields nothing for non-existent/non-directory paths, so the function
returns 0 as specified; bug only reproducible on Python <3.12."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:24:20.971121Z	2564
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:25:02.156929Z	1742

事件 1 — [llm_5640de0826664977bd331a06e7e998d3](#)

► parsed_json（完整）


```
- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5640de0826664977bd331a06e7e998d3_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_5640de0826664977bd331a06e7e998d3_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_5640de0826664977bd331a06e7e998d3_response.txt)
```

事件 2 — llm_388bb329d276462a84b7b13387b79c5e

► parsed_json (完整)

```
- 载荷: [system_prompt](fm_agent/trace/payloads/llm_388bb329d276462a84b7b13387b79c5e_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_388bb329d276462a84b7b13387b79c5e_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_388bb329d276462a84b7b13387b79c5e_response.txt) **复核动作**
```

- 核对 `probe` 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: incremental_pipeline

FMA-MISMATCH-225 — src--incremental_reasoner-py--_llm_check_caller_info_update

- 四类结论：可能有 Bug
- 分类依据：沿用旧 `fm_agent/bug_list.md` 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**: `confirmed`; attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / `incremental_pipeline`; 原源码 `src/incremental_reasoner.py`
- 证据：抽取源码 · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a dict when a consistency determination is made. The returned dict has key 'info_updated' (bool). When info_updated is true, the dict also has key 'new_info' whose value is a dict conforming to the .info.json schema. In that new_info dict, every callee entry whose name is not equal to callee_name has the same 'signature', 'pre_condition', and 'post_condition' values as its counterpart in caller_info_block; the entry whose name equals callee_name has a pre-condition and post-condition that do not contradict any assertion in callee_new_spec's pre-condition or post-condition. When info_updated is false, the entry for callee_name in caller_info_block is already consistent with callee_new_spec and no revision is required. Returns None when no determination can be made.
- **Actual behavior:** Upon successful execution, the function delegates to _llm_select_json and returns its result. No side-effects other than those of _llm_select_json (which may include an LLM call) are produced. If all implicit pre-conditions on the parameters work_dir, comment_prefix, and proj_dir are satisfied (work_dir is a valid directory path, comment_prefix is a string, etc.), no exceptions are raised.

```
Let result = _llm_check_caller_info_update(proj_dir, work_dir, idx, caller_fqn, callee_name, lang_key, comment_prefix, callee_new_spec,
caller_info_block, caller_source).
```

Natural language: The function returns either None or a dictionary. If the configured LLM generates a response that is valid JSON, conforms to the schema {"info_updated": boolean, "new_info": object|null}, and passes the _validate_caller_info_update validator, then result is that parsed dictionary; otherwise, result is None.

Formal logic: (result = None)

(result Dict hasKeys(result, {"info_updated", "new_info"}) result["info_updated"] Bool (result["new_info"] = None result["new_info"] Dict)
alt_result (alt_result = _llm_select_json(...) (alt_result = None result = None) (alt_result None result = alt_result))) (The final conjunct captures
that result is exactly the return of _llm_select_json.)

- **Code evidence:** Line 43: return _llm_select_json(Line 44: work_dir, Line 45: prompt_content, Line 46: stage="update_caller_info", Line 47: validator=_validate_caller_info_update, Line 48: schema_description={"info_updated": boolean, "new_info": object|null}, Line 49: trace_meta={"caller_fqn": caller_fqn, "callee_name": callee_name, "idx": idx}, Line 50:)
- **Trigger condition:** The function unconditionally returns the result of _llm_select_json after only structural validation. It does not verify that the returned dict satisfies the semantic constraints required by the specification (e.g., that when info_updated is false the existing entry is actually consistent, or when info_updated is true the new_info preserves other callee entries and makes the named entry consistent). An LLM could produce a structurally valid response that violates these constraints, and the function would deliver it, breaking Condition B.
- **Validator trigger:** _llm_select_json result passes through _validate_caller_info_update with only structural checks, no semantic verification that other callee entries are preserved or that the named callee entry is consistent with the new spec.
- **Probe stdout:** CONFIRMED — function returned the structurally valid but semantically wrong response: other_callee entry was dropped from new_info

► SPEC sidecar (完整)

```
{
  "signature": "_llm_check_caller_info_update(proj_dir, work_dir, idx, caller_fqn, callee_name, lang_key, comment_prefix,
callee_new_spec, caller_info_block, caller_source) -> dict | None",
  "pre_condition": "idx is a non-negative integer. caller_fqn and callee_name are non-empty strings. lang_key is a string
identifying a source language in the system's recognized language set. callee_new_spec is a dict with exactly the string keys
'signature', 'pre_condition', and 'post_condition'. caller_info_block is a dict matching the .info.json schema: it has a 'callees'
key whose value is a list of dicts, each with 'name', 'signature', 'pre_condition', and 'post_condition' string keys.
caller_source is a non-empty string.",
  "post_condition": "Returns a dict when a consistency determination is made. The returned dict has key 'info_updated' (bool).
When info_updated is true, the dict also has key 'new_info' whose value is a dict conforming to the .info.json schema. In that
new_info dict, every callee entry whose name is not equal to callee_name has the same 'signature', 'pre_condition', and
'post_condition' values as its counterpart in caller_info_block; the entry whose name equals callee_name has a pre-condition and
post-condition that do not contradict any assertion in callee_new_spec's pre-condition or post-condition. When info_updated is
false, the entry for callee_name in caller_info_block is already consistent with callee_new_spec and no revision is required.
Returns None when no determination can be made."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_domain_knowledge_prompt_section",
      "signature": "_domain_knowledge_prompt_section(work_dir) -> str",
      "pre_condition": "work_dir is a path to a directory that may contain domain-knowledge markdown files staged from the --
domain-knowledge pipeline argument.",
      "post_condition": "Returns a string that is empty when work_dir contains no domain-knowledge files. When domain-knowledge
files are present, returns a formatted string suitable for inclusion in an LLM prompt, prefixed with a header line followed by the
contents of each file."
    },
    {
      "name": "_llm_select_json",
      "signature": "_llm_select_json(work_dir, prompt_content, stage, validator, schema_description, trace_meta) -> dict | None",
      "pre_condition": "work_dir is a valid directory path. prompt_content is a non-empty string. stage is a non-empty string
identifying a pipeline stage. validator is a callable that accepts a dict and returns a (bool, str | None) tuple where the bool
indicates validity and the second element is an error description when false. schema_description is a non-empty string describing
the expected JSON output shape. trace_meta is a dict whose keys are strings and values are JSON-serializable.",
      "post_condition": "Sends prompt_content to the configured LLM. When the LLM returns output that is parseable as JSON,
conforms to schema_description, and passes validator, returns the parsed dict. When the LLM returns no output, unparseable output,
or output that is not valid JSON, does not conform to schema_description, or fails validator, returns None. The exchange is traced
with stage and trace_meta as identifying labels."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_llm_check_caller_info_update.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dict when a consistency determination is made. The returned dict has key 'info_updated' (bool). When
info_updated is true, the dict also has key 'new_info' whose value is a dict conforming to the .info.json schema. In that new_info
dict, every callee entry whose name is not equal to callee_name has the same 'signature', 'pre_condition', and 'post_condition'
```

```
values as its counterpart in caller_info_block; the entry whose name equals callee_name has a pre-condition and post-condition
that do not contradict any assertion in callee_new_spec's pre-condition or post-condition. When info_updated is false, the entry
for callee_name in caller_info_block is already consistent with callee_new_spec and no revision is required. Returns None when no
determination can be made.",
    "actual_behavior": "Upon successful execution, the function delegates to _llm_select_json and returns its result. No side-
effects other than those of _llm_select_json (which may include an LLM call) are produced. If all implicit pre-conditions on the
parameters work_dir, comment_prefix, and proj_dir are satisfied (work_dir is a valid directory path, comment_prefix is a string,
etc.), no exceptions are raised.\n\nLet result = _llm_check_caller_info_update(proj_dir, work_dir, idx, caller_fqn, callee_name,
lang_key, comment_prefix, callee_new_spec, caller_info_block, caller_source).\n\nNatural language: The function returns either
None or a dictionary. If the configured LLM generates a response that is valid JSON, conforms to the schema {"info_updated":
boolean, "new_info": object|null}, and passes the _validate_caller_info_update validator, then result is that parsed dictionary;
otherwise, result is None.\n\nFormal logic:\n( result = None )\n\n( result Dict hasKeys(result, {"info_updated",
"new_info"}) \n result[\"info_updated\"] Bool \n ( result[\"new_info\"] = None result[\"new_info\"] Dict )\n alt_result
( alt_result = _llm_select_json(...) (alt_result = None result = None) (alt_result None result = alt_result) )\n)\n(The final
conjoint captures that result is exactly the return of _llm_select_json.)",
    "code_evidence": "Line 43: return _llm_select_json(\nLine 44:     work_dir,\nLine 45:     prompt_content,\nLine 46:
stage=\"update_caller_info\", \nLine 47:     validator=_validate_caller_info_update, \nLine 48:
schema_description='{\"info_updated\": boolean, \"new_info\": object|null}', \nLine 49:     trace_meta={\"caller_fqn\": caller_fqn,
\"callee_name\": callee_name, \"idx\": idx}, \nLine 50: )",
    "trigger_condition": "The function unconditionally returns the result of _llm_select_json after only structural validation. It
does not verify that the returned dict satisfies the semantic constraints required by the specification (e.g., that when
info_updated is false the existing entry is actually consistent, or when info_updated is true the new_info preserves other callee
entries and makes the named entry consistent). An LLM could produce a structurally valid response that violates these constraints,
and the function would deliver it, breaking Condition B."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_llm_check_caller_info_update",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_llm_check_caller_info_update.py",
  "function_name": "_llm_check_caller_info_update",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_llm_check_caller_info_update.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_llm_check_caller_info_update.md",
  "probe_stdout": "CONFIRMED - function returned the structurally valid but semantically wrong response: other_callee entry was
dropped from new_info\n",
  "trigger_summary": "_llm_select_json result passes through _validate_caller_info_update with only structural checks, no semantic
verification that other callee entries are preserved or that the named callee entry is consistent with the new spec."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:10:52.307695Z	4000
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:11:29.910903Z	5426

事件 1 — [llm_9e3d1aee841a4b11ab5b1774ba059a6e](#)

► parsed_json（完整）

```
{
  "post_condition": "Upon successful execution, the function delegates to _llm_select_json and returns its result. No side-effects
other than those of _llm_select_json (which may include an LLM call) are produced. If all implicit pre-conditions on the
parameters work_dir, comment_prefix, and proj_dir are satisfied (work_dir is a valid directory path, comment_prefix is a string,
etc.), no exceptions are raised.\n\nLet result = _llm_check_caller_info_update(proj_dir, work_dir, idx, caller_fqn, callee_name,
lang_key, comment_prefix, callee_new_spec, caller_info_block, caller_source).\n\nNatural language: The function returns either
None or a dictionary. If the configured LLM generates a response that is valid JSON, conforms to the schema {"info_updated":
boolean, "new_info": object|null}, and passes the _validate_caller_info_update validator, then result is that parsed dictionary;
otherwise, result is None.\n\nFormal logic:\n( result = None )\n\n( result ∈ Dict ∧ hasKeys(result, {"info_updated",
"new_info"}) ) ∧\n result[\"info_updated\"] ∈ Bool ∧\n ( result[\"new_info\"] = None ∨ result[\"new_info\"] ∈ Dict )\n\n ∧ ∇
alt_result ( alt_result = _llm_select_json(...) → (alt_result = None ↔ result = None) ∧ (alt_result ≠ None → result = alt_result)
)\n)\n(The final conjoint captures that result is exactly the return of _llm_select_json.)"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_9e3d1aee841a4b11ab5b1774ba059a6e_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_9e3d1aee841a4b11ab5b1774ba059a6e_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_9e3d1aee841a4b11ab5b1774ba059a6e_response.txt)

► `parsed_json` (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp', work_dir='/tmp', idx=1, caller_fqn='mod.A', callee_name='my_callee', lang_key='python',
  comment_prefix='#', callee_new_spec={'pre_condition': 'x <= 0', 'post_condition': 'y == 0'}, caller_info_block={'my_callee':
  {'signature': 'def foo(x: int) -> int', 'pre_condition': 'x > 0', 'post_condition': 'y == 0'}}, caller_source='def A(): B(). The
  LLM, through the _llm_select_json call, returns the dict {"info_updated": false, "new_info": null}. The validator
  _validate_caller_info_update accepts this because it conforms to the expected schema. The function returns the same dict. The
  specification requires that when info_updated is false, the caller's entry for callee_name must already be consistent with the
  callee's new spec, but here the pre-conditions 'x > 0' and 'x <= 0' are contradictory, so the entry is not consistent.",
  "offending_statements": "Line 43: return _llm_select_json(\nLine 44:     work_dir,\nLine 45:     prompt_content,\nLine 46:
  stage=\"update_caller_info\", \nLine 47:     validator=_validate_caller_info_update, \nLine 48:
  schema_description='{\"info_updated\": boolean, \"new_info\": object|null}', \nLine 49:     trace_meta={\"caller_fqn\": caller_fqn,
  \"callee_name\": callee_name, \"idx\": idx}, \nLine 50: )",
  "reason": "The function unconditionally returns the result of _llm_select_json after only structural validation. It does not
  verify that the returned dict satisfies the semantic constraints required by the specification (e.g., that when info_updated is
  false the existing entry is actually consistent, or when info_updated is true the new_info preserves other callee entries and
  makes the named entry consistent). An LLM could produce a structurally valid response that violates these constraints, and the
  function would deliver it, breaking Condition B."
}
```

- 载荷: `[system_prompt](fm_agent/trace/payloads/llm_e1363a618dda47d8905cd563898a8eed_message_00_system.txt)` · `[user_prompt](fm_agent/trace/payloads/llm_e1363a618dda47d8905cd563898a8eed_message_01_user.txt)` · `[assistant_output](fm_agent/trace/payloads/llm_e1363a618dda47d8905cd563898a8eed_response.txt)` **复核动作**

- 核对 `probe` 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-229 — `src--incremental_reasoner-py--_opcode_generate_spec`

- 四类结论：可能有 Bug
- 分类依据：沿用旧 `fm_agent/bug_list.md` 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**: `not_confirmed`；attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / `incremental_pipeline`；原源码 `src/incremental_reasoner.py`
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: Returns None when the opcode subprocess fails to produce a valid result (subprocess exits with error, produces no output file, or the output fails structural validation). When opcode succeeds, returns a dict with exactly these keys: 'spec_updated' (boolean, true), 'new_spec' (dict with 'signature', 'pre_condition', 'post_condition' keys describing the function's intended behavioral contract generated by opcode from the function source, developer intent, and all provided caller context such that the resulting spec is consistent with every caller's stated expectations), 'info_updated' (boolean, true), 'new_info' (dict with a 'callees' key whose value is a list of callee contract dicts, each containing 'name', 'signature', 'pre_condition', and 'post_condition'), and 'updated_callees' (list of callee name strings recorded in new_info empty list when callee_names is empty). The returned dict is a pure data record: no .spec.json or .info.json sidecar files are written by this function. Side effects: writes exactly one prompt markdown file to a path under `fm_agent/` within the project directory whose name incorporates the `idx` value; opcode, when successful, writes exactly one result JSON file to a path under `fm_agent/` whose name also incorporates `idx`.
- **Actual behavior**: Natural language: The code block (lines 41107) is never executed because the Python interpreter raises a `SyntaxError` before reaching it (at line 33) due to an incomplete f-string literal. Consequently, the function `_opcode_generate_spec` is not defined, no assignments from the block take place, and no side effects (like file writes or `_opcode_select_json` calls) occur. The program terminates in an exceptional state with a `SyntaxError`. Formal logic: For any initial state , executing the module (including the def statement for `_opcode_generate_spec` and all subsequent lines) yields a final state ' where a `SyntaxError` exception is raised, the name `_opcode_generate_spec` is not bound, and all variables introduced in lines 41107 are unbound. Symbolically: `. exec(module, )` `SyntaxError, ' '_opcode_generate_spec' dom()` `v {callee_hint, user_knowledge_paths, user_knowledge_step, step_offset, info_step_number, info_step, prompt_content, result} . v dom()`.
- **Code evidence**: Line 41-107: The code block is part of a function definition that is invalid due to a `SyntaxError` at line 33, so the function is never defined and the block is never executed.
- **Trigger condition**: The specification requires that calling `_opcode_generate_spec` with valid arguments returns a dict or None. However, the Python module fails to compile because of a `SyntaxError` at line 33, preventing the function definition. Any valid input results in a `NameError` or `AttributeError`, violating the specification.
- **Validator trigger**: Claimed `SyntaxError` at line 33 of `src/incremental_reasoner.py` prevents compilation; probe confirms the module compiles, imports, and the function is callable with no `SyntaxError` present.

- **Probe stdout:** Check 1: Compiling src/incremental_reasoner.py... OK Check 2: Importing src.incremental_reasoner... OK Check 3: `_opcode_generate_spec` defined and callable?... OK — callable function NOT CONFIRMED — Module compiles and imports successfully; `_opcode_generate_spec` is defined and callable. No `SyntaxError` at line 33 or anywhere else.

► SPEC sidecar (完整)

```
{
  "signature": "_opcode_generate_spec(proj_dir, work_dir, idx, fn, lang_key, comment_prefix, developer_intent, callee_names,
source, caller_context)",
  "pre_condition": "proj_dir and work_dir are valid directory paths; fn is a non-empty fully-qualified function name string;
lang_key is a recognized language identifier; source is the complete source code text of the function (non-empty string);
callee_names is a list of callee function name strings (may be empty); caller_context is an iterable of (caller_fn,
caller_spec_dict_or_None, caller_expectation_or_None) tuples where caller_spec_dict_or_None is either a dict with 'signature',
'pre_condition', 'post_condition' keys or None, and caller_expectation_or_None is either a string describing what the caller
expects from this function or None; developer_intent is a non-empty string describing the intended code modification;
comment_prefix is the language's comment prefix string; idx is a non-negative integer unique to this invocation.",
  "post_condition": "Returns None when the opcode subprocess fails to produce a valid result (subprocess exits with error,
produces no output file, or the output fails structural validation). When opcode succeeds, returns a dict with exactly these
keys: 'spec_updated' (boolean, true), 'new_spec' (dict with 'signature', 'pre_condition', 'post_condition' keys describing the
function's intended behavioral contract – generated by opcode from the function source, developer intent, and all provided
caller context such that the resulting spec is consistent with every caller's stated expectations), 'info_updated' (boolean,
true), 'new_info' (dict with a 'callees' key whose value is a list of callee contract dicts, each containing 'name', 'signature',
'pre_condition', and 'post_condition'), and 'updated_callees' (list of callee name strings recorded in new_info – empty list when
callee_names is empty). The returned dict is a pure data record: no .spec.json or .info.json sidecar files are written by this
function. Side effects: writes exactly one prompt markdown file to a path under fm_agent/ within the project directory whose name
incorporates the idx value; opcode, when successful, writes exactly one result JSON file to a path under fm_agent/ whose name
also incorporates idx."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "list_staged_domain_knowledge_relpaths",
      "signature": "list_staged_domain_knowledge_relpaths(work_dir)",
      "pre_condition": "work_dir is a valid directory path.",
      "post_condition": "Returns a list of relative file path strings to domain knowledge markdown files that have been staged
under work_dir for the current run. Returns an empty list when no domain knowledge files are staged."
    },
    {
      "name": "format_domain_knowledge_bullets",
      "signature": "format_domain_knowledge_bullets(paths)",
      "pre_condition": "paths is a list of file path strings.",
      "post_condition": "Returns a string where each path is formatted as a markdown bullet item line, suitable for embedding as a
numbered reading step in a prompt."
    },
    {
      "name": "_opcode_select_json",
      "signature": "_opcode_select_json(proj_dir, work_dir, prompt_relpath, prompt_content, result_relpath, stage,
input_files)",
      "pre_condition": "proj_dir and work_dir are valid directory paths; prompt_relpath and result_relpath are relative file paths
under work_dir; prompt_content is a non-empty string; stage is a recognized pipeline stage identifier; input_files is a list of
relative file paths to contextual files that opcode may read.",
      "post_condition": "Writes prompt_content to prompt_relpath on disk, then invokes opcode with that prompt and the listed
input_files as readable context. When opcode succeeds, reads and returns the parsed JSON content from result_relpath. Returns
None when opcode exits with a non-zero code, result_relpath does not exist or is empty after execution, or the file content is
not valid JSON."
    },
    {
      "name": "_validate_spec_update",
      "signature": "_validate_spec_update(result)",
      "pre_condition": "result is a dict potentially produced by an opcode spec generation or spec-update run.",
      "post_condition": "When result contains the expected keys with the expected types (spec_updated is bool; new_spec, when
spec_updated is true, is a dict with 'signature', 'pre_condition', 'post_condition' string values; info_updated is bool; new_info
is a dict with a 'callees' key; updated_callees is a list of strings), returns result unchanged. Returns None when any key is
missing, has an unexpected type, or the value of a boolean key is not a boolean."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_opcode_generate_spec.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns None when the opcode subprocess fails to produce a valid result (subprocess exits with error,
```


produces no output file, or the output fails structural validation). When `opencode` succeeds, returns a dict with exactly these keys: 'spec_updated' (boolean, true), 'new_spec' (dict with 'signature', 'pre_condition', 'post_condition' keys describing the function's intended behavioral contract generated by `opencode` from the function source, developer intent, and all provided caller context such that the resulting spec is consistent with every caller's stated expectations), 'info_updated' (boolean, true), 'new_info' (dict with a 'callees' key whose value is a list of callee contract dicts, each containing 'name', 'signature', 'pre_condition', and 'post_condition'), and 'updated_callees' (list of callee name strings recorded in `new_info` empty list when `callee_names` is empty). The returned dict is a pure data record: no `.spec.json` or `.info.json` sidecar files are written by this function. Side effects: writes exactly one prompt markdown file to a path under `fm_agent/` within the project directory whose name incorporates the `idx` value; `opencode`, when successful, writes exactly one result JSON file to a path under `fm_agent/` whose name also incorporates `idx`.",

"actual_behavior": "Natural language: The code block (lines 41107) is never executed because the Python interpreter raises a `SyntaxError` before reaching it (at line 33) due to an incomplete f-string literal. Consequently, the function ``_opencode_generate_spec`` is not defined, no assignments from the block take place, and no side effects (like file writes or ``_opencode_select_json`` calls) occur. The program terminates in an exceptional state with a `SyntaxError`. Formal logic: For any initial state , executing the module (including the `def` statement for ``_opencode_generate_spec`` and all subsequent lines) yields a final state ' where a `SyntaxError` exception is raised, the name ``_opencode_generate_spec`` is not bound, and all variables introduced in lines 41107 are unbound. Symbolically: `. exec(module, )` `SyntaxError, ' '_opencode_generate_spec' dom(') v {callee_hint, user_knowledge_paths, user_knowledge_step, step_offset, info_step_number, info_step, prompt_content, result} . v dom(').`",

"code_evidence": "Line 41-107: The code block is part of a function definition that is invalid due to a `SyntaxError` at line 33, so the function is never defined and the block is never executed.",

"trigger_condition": "The specification requires that calling `_opencode_generate_spec` with valid arguments returns a dict or `None`. However, the Python module fails to compile because of a `SyntaxError` at line 33, preventing the function definition. Any valid input results in a `NameError` or `AttributeError`, violating the specification."

```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_opencode_generate_spec",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_opencode_generate_spec.py",
  "function_name": "_opencode_generate_spec",
  "confirmation_status": "not_confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_opencode_generate_spec.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_opencode_generate_spec.md",
  "probe_stdout": "Check 1: Compiling src/incremental_reasoner.py... OK\nCheck 2: Importing src.incremental_reasoner... OK\nCheck 3: _opencode_generate_spec defined and callable?... OK – callable function\n\nNOT CONFIRMED – Module compiles and imports successfully; _opencode_generate_spec is defined and callable. No SyntaxError at line 33 or anywhere else.",
  "trigger_summary": "Claimed SyntaxError at line 33 of src/incremental_reasoner.py prevents compilation; probe confirms the module compiles, imports, and the function is callable with no SyntaxError present."
}
```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 `system/user/assistant` 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/2	2026-07-28T04:09:28.267847Z	3630
2	generate_block_post_condition	success	1	1/2	2026-07-28T04:10:01.191665Z	3466
3	check_post_implies_spec	mismatch	1	1/2	2026-07-28T04:10:26.003599Z	4134

事件 1 — [llm_27e8f8e7f4ad469ab3264c93b312778f](#)

► parsed_json（完整）

```
{
  "post_condition": "After the code block is executed, the interpreter raises a SyntaxError exception because line 33 contains an incomplete f-string literal. The result is that the function _opencode_generate_spec is not defined in the current namespace, and no further code in the block is executed. Formally: for any initial state satisfying the pre-condition, the execution leads to an exceptional state where a SyntaxError is raised and the def statement fails to create the function."
}
```

- 载荷: `[system_prompt](fm_agent/trace/payloads/llm_27e8f8e7f4ad469ab3264c93b312778f_message_00_system.txt)` · `[user_prompt](fm_agent/trace/payloads/llm_27e8f8e7f4ad469ab3264c93b312778f_message_01_user.txt)` · `[assistant_output](fm_agent/trace/payloads/llm_27e8f8e7f4ad469ab3264c93b312778f_response.txt)`

事件 2 — [llm_f01392e479ed433db84494a6e6d0cc20](#)

► parsed_json（完整）


```
{
  "post_condition": "Natural language: The code block (lines 41-107) is never executed because the Python interpreter raises a
  SyntaxError before reaching it (at line 33) due to an incomplete f-string literal. Consequently, the function
  `_opcode_generate_spec` is not defined, no assignments from the block take place, and no side effects (like file writes or
  `_opcode_select_json` calls) occur. The program terminates in an exceptional state with a SyntaxError. Formal logic: For any
  initial state  $\sigma$ , executing the module (including the def statement for `_opcode_generate_spec` and all subsequent lines) yields
  a final state  $\sigma'$  where a SyntaxError exception is raised, the name `_opcode_generate_spec` is not bound, and all variables
  introduced in lines 41-107 are unbound. Symbolically:  $\forall \sigma. \text{exec}(\text{module}, \sigma) \rightarrow (\text{SyntaxError}, \sigma') \wedge \text{'_opcode\_generate\_spec'} \notin \text{dom}(\sigma') \wedge \forall v \in \{\text{callee\_hint}, \text{user\_knowledge\_paths}, \text{user\_knowledge\_step}, \text{step\_offset}, \text{info\_step\_number}, \text{info\_step}, \text{prompt\_content}, \text{result}\} . v \notin \text{dom}(\sigma')."$ 
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f01392e479ed433db84494a6e6d0cc20_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_f01392e479ed433db84494a6e6d0cc20_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_f01392e479ed433db84494a6e6d0cc20_response.txt)

事件 3 — llm_e5bc21ed79de48b782e5b76ff16a813f

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Calling _opcode_generate_spec('/tmp/proj', '/tmp/work', 'mod.func', 'python', 'def func(): pass', 'do something', [], [], 0)",
  "offending_statements": "Line 41-107: The code block is part of a function definition that is invalid due to a SyntaxError at line 33, so the function is never defined and the block is never executed.",
  "reason": "The specification requires that calling _opcode_generate_spec with valid arguments returns a dict or None. However, the Python module fails to compile because of a SyntaxError at line 33, preventing the function definition. Any valid input results in a NameError or AttributeError, violating the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_e5bc21ed79de48b782e5b76ff16a813f_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_e5bc21ed79de48b782e5b76ff16a813f_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_e5bc21ed79de48b782e5b76ff16a813f_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-230 — src--incremental_reasoner-py--_opcode_select_json

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 6 — Reasoning & Verification Engine / incremental_pipeline; 原源码 src/incremental_reasoner.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** Atomically writes prompt_content to proj_dir/prompt_relpath via a temp-file rename, ensuring the previous content of that path (if any) is fully replaced. If a file exists at proj_dir/result_relpath before invocation, it is removed. Invokes the configured LLM CLI with the prompt file as the sole file argument, retrying up to OPENCODE_MAX_RETRIES times with a fixed 10-second delay between attempts. Each invocation is traced producing a structured trace event with the stage identifier, attempt number, and any input/output file references. On the earliest attempt where the LLM invocation completes and the file at proj_dir/result_relpath exists afterward, reads and returns its content parsed as JSON. Returns None when: the LLM process exits with a non-zero code on every attempt, result_relpath does not exist after the last attempt, or result_relpath exists but contains content that is not valid JSON or is unreadable. The returned value is either a dict, a list, or None depending on the JSON type produced by the LLM and whether generation succeeded. The original prompt_content, result_relpath, stage, and input_files arguments are not modified. No .spec.json or .info.json sidecar files are written by this function.
- **Actual behavior:** After normal termination of _opcode_select_json (i.e., the function returns without raising an exception), the following holds:
 - The file at os.path.join(proj_dir, prompt_relpath) exists and its content exactly equals prompt_content.
 - Let result_path = os.path.join(proj_dir, result_relpath).
 - If the return value is not None, then result_path exists, its content is valid JSON, and the parsed JSON value equals the return value.

- If the return value is `None`, then either `result_path` does not exist (the agent never wrote the file after `OPENCODE_MAX_RETRIES` attempts) OR `result_path` exists but either its content could not be read due to an `OSError` or its content is not valid JSON.
- The function may have invoked `run_opencode_traced` multiple times (between 1 and `OPENCODE_MAX_RETRIES` inclusive), but the loop ensures that if a successful attempt produced `result_relpath`, the file remains on disk and is not removed by subsequent attempts.

Formal logic (in state after normal return):

```

Let result_path  join(proj_dir, result_relpath)
Let prompt_path  join(proj_dir, prompt_relpath)
Returned_value  (JSON_value {None})
(
  (exists(prompt_path)  file_content(prompt_path) = prompt_content)

  (Returned_value  None
    (exists(result_path)  is_valid_json(file_content(result_path))  json_parse(file_content(result_path)) = Returned_value)
  )

  (Returned_value = None
    (exists(result_path)
      (exists(result_path)  (read_error_occurred(result_path)  is_valid_json(file_content(result_path)))) )
  )
)

```

Here `read_error_occurred(result_path)` is true if an `OSError` prevented reading the file; `is_valid_json` and `json_parse` are standard JSON operations.

- **Code evidence:** Line 61: `return json.load(f)`
- **Trigger condition:** The code returns the raw parsed JSON value from the file, which can be a string, number, or boolean, violating the specification's requirement that the return value be only a dict, a list, or `None`.
- **Validator trigger:** The function returns raw `json.load()` result, which can be a string, number, or boolean — not only dict, list, or `None` as the spec claims.
- **Probe stdout:** CONFIRMED — string: 'hello world' (type=str); integer: 42 (type=int); boolean: True (type=bool) | expected: must be dict, list, or `None`

► SPEC sidecar (完整)

```

{
  "signature": "_opencode_select_json(proj_dir, work_dir, prompt_relpath, prompt_content, result_relpath, stage, input_files) -> dict | list | None",
  "pre_condition": "proj_dir and work_dir are valid, existing directory paths. prompt_relpath and result_relpath are relative file paths resolvable under proj_dir. prompt_content is a non-empty string containing the instructions for the LLM agent. stage is a non-empty string identifier used for logging and tracing. input_files is an iterable of file paths (relative to proj_dir or absolute) that the LLM agent may read as context. The globally configured LLM model (OPENCODE_SETUP_MODEL) and maximum retry count (OPENCODE_MAX_RETRIES) are available.",
  "post_condition": "Atomically writes prompt_content to proj_dir/prompt_relpath via a temp-file rename, ensuring the previous content of that path (if any) is fully replaced. If a file exists at proj_dir/result_relpath before invocation, it is removed. Invokes the configured LLM CLI with the prompt file as the sole file argument, retrying up to OPENCODE_MAX_RETRIES times with a fixed 10-second delay between attempts. Each invocation is traced – producing a structured trace event with the stage identifier, attempt number, and any input/output file references. On the earliest attempt where the LLM invocation completes and the file at proj_dir/result_relpath exists afterward, reads and returns its content parsed as JSON. Returns None when: the LLM process exits with a non-zero code on every attempt, result_relpath does not exist after the last attempt, or result_relpath exists but contains content that is not valid JSON or is unreadable. The returned value is either a dict, a list, or None – depending on the JSON type produced by the LLM and whether generation succeeded. The original prompt_content, result_relpath, stage, and input_files arguments are not modified. No .spec.json or .info.json sidecar files are written by this function."
}

```

► INFO / callees sidecar (完整)

```

{
  "callees": [
    {
      "name": "build_llm_cli_command",
      "signature": "build_llm_cli_command(model, prompt, cwd, files) -> list[str]",
      "pre_condition": "model is a non-empty string identifying the LLM model. prompt is a non-empty string containing the instructions for the LLM. cwd is an existing directory path to use as the working directory. files is an iterable of absolute or relative file paths that the LLM should have access to.",
      "post_condition": "Returns a list of command-line argument strings that, when executed, invoke the configured LLM CLI backend with the specified model, prompt text, working directory, and file access list. The returned command is suitable for execution via subprocess.run or an equivalent process invocation."
    },
    {
      "name": "run_opencode_traced",
      "signature": "run_opencode_traced(proj_dir, work_dir, command, stage, input_files, output_files, summary, metadata) ->

```

```
None",
    "pre_condition": "proj_dir and work_dir are valid directory paths. command is a list of strings forming a valid CLI invocation. stage is a non-empty stage identifier string. input_files is an iterable of file paths available for reading. output_files is an iterable of relative file paths that the process is expected to produce. summary is a non-empty string describing the invocation. metadata is a dict of additional key-value pairs to attach to the trace event.",
    "post_condition": "Executes command as a subprocess with working directory proj_dir. Produces a structured trace event – recorded under proj_dir/fm_agent/trace/ – with the stage, input/output file references, summary, metadata, process exit code, and a unique event identifier. If the subprocess exits with a non-zero code, raises subprocess.CallProcessError. On success, any output files listed in output_files that were produced by the subprocess exist on disk after return."
  }
]
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_opencode_select_json.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Atomically writes prompt_content to proj_dir/prompt_relpath via a temp-file rename, ensuring the previous content of that path (if any) is fully replaced. If a file exists at proj_dir/result_relpath before invocation, it is removed. Invokes the configured LLM CLI with the prompt file as the sole file argument, retrying up to OPENCODE_MAX_RETRIES times with a fixed 10-second delay between attempts. Each invocation is traced producing a structured trace event with the stage identifier, attempt number, and any input/output file references. On the earliest attempt where the LLM invocation completes and the file at proj_dir/result_relpath exists afterward, reads and returns its content parsed as JSON. Returns None when: the LLM process exits with a non-zero code on every attempt, result_relpath does not exist after the last attempt, or result_relpath exists but contains content that is not valid JSON or is unreadable. The returned value is either a dict, a list, or None depending on the JSON type produced by the LLM and whether generation succeeded. The original prompt_content, result_relpath, stage, and input_files arguments are not modified. No .spec.json or .info.json sidecar files are written by this function.",
    "actual_behavior": "After normal termination of _opencode_select_json (i.e., the function returns without raising an exception), the following holds:\n\n- The file at `os.path.join(proj_dir, prompt_relpath)` exists and its content exactly equals `prompt_content`.\n- Let `result_path = os.path.join(proj_dir, result_relpath)`.\n- If the return value is not `None`, then `result_path` exists, its content is valid JSON, and the parsed JSON value equals the return value.\n- If the return value is `None`, then either `result_path` does not exist (the agent never wrote the file after `OPENCODE_MAX_RETRIES` attempts) OR `result_path` exists but either its content could not be read due to an `OSError` or its content is not valid JSON.\n- The function may have invoked `run_opencode_traced` multiple times (between 1 and `OPENCODE_MAX_RETRIES` inclusive), but the loop ensures that if a successful attempt produced `result_relpath`, the file remains on disk and is not removed by subsequent attempts.\n\nFormal logic (in state after normal return):\n\n```\nLet result_path join(proj_dir, result_relpath)\nLet prompt_path join(proj_dir, prompt_relpath)\nReturned_value (JSON_value {None})\n\n(exists(prompt_path) file_content(prompt_path) = prompt_content)\n\n\n(Returned_value None \n  (exists(result_path) is_valid_json(file_content(result_path))\n  json_parse(file_content(result_path)) = Returned_value)\n )\n\n\n(Returned_value = None \n  (exists(result_path) \n  (exists(result_path) (read_error_occurred(result_path) is_valid_json(file_content(result_path)))) )\n )\n\n```\n\nHere `read_error_occurred(result_path)` is true if an `OSError` prevented reading the file; `is_valid_json` and `json_parse` are standard JSON operations.",
    "code_evidence": "Line 61: return json.load(f)",
    "trigger_condition": "The code returns the raw parsed JSON value from the file, which can be a string, number, or boolean, violating the specification's requirement that the return value be only a dict, a list, or None."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_opencode_select_json",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_opencode_select_json.py",
  "function_name": "_opencode_select_json",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_opencode_select_json.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_opencode_select_json.md",
  "probe_stdout": "CONFIRMED – string: 'hello world' (type=str); integer: 42 (type=int); boolean: True (type=bool) | expected: must be dict, list, or None",
  "trigger_summary": "The function returns raw json.load() result, which can be a string, number, or boolean – not only dict, list, or None as the spec claims."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:28:06.909915Z	4685
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:29:10.782236Z	5914

事件 1 — `llm_03c43257fbf64c92a171561df3b11dbc`

► parsed_json (完整)

```
{
  "post_condition": "After normal termination of _opencode_select_json (i.e., the function returns without raising an exception),
the following holds:\n\n- The file at `os.path.join(proj_dir, prompt_relpath)` exists and its content exactly equals
`prompt_content`.\n- Let `result_path = os.path.join(proj_dir, result_relpath)`.\n- If the return value is not `None`, then
`result_path` exists, its content is valid JSON, and the parsed JSON value equals the return value.\n- If the return value is
`None`, then either `result_path` does not exist (the agent never wrote the file after `OPENCODE_MAX_RETRIES` attempts) OR
`result_path` exists but either its content could not be read due to an `OSError` or its content is not valid JSON.\n- The
function may have invoked `run_opencode_traced` multiple times (between 1 and `OPENCODE_MAX_RETRIES` inclusive), but the loop
ensures that if a successful attempt produced `result_relpath`, the file remains on disk and is not removed by subsequent
attempts.\n\nFormal logic (in state after normal return):\n```\nLet result_path ≡ join(proj_dir, result_relpath)\nLet prompt_path
≡ join(proj_dir, prompt_relpath)\nReturned_value ∈ (JSON_value ∪ {None})\n(\n  (exists(prompt_path) ∧ file_content(prompt_path) =
prompt_content)\n  ∧\n  (Returned_value ≠ None →\n    (exists(result_path) ∧ is_valid_json(file_content(result_path)) ∧
json_parse(file_content(result_path)) = Returned_value)\n  )\n  ∧\n  (Returned_value = None →\n    (¬exists(result_path) ∨\n    (exists(result_path) ∧ (read_error_occurred(result_path) ∨ ¬is_valid_json(file_content(result_path)))) )\n  )\n)\n```\nHere
`read_error_occurred(result_path)` is true if an `OSError` prevented reading the file; `is_valid_json` and `json_parse` are
standard JSON operations."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_03c43257fbf64c92a171561df3b11dbc_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_03c43257fbf64c92a171561df3b11dbc_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_03c43257fbf64c92a171561df3b11dbc_response.txt)

事件 2 — `llm_f4852cdb144a4c9daa102b9e00593bf9`

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp', work_dir='/tmp/w', prompt_relpath='prompt.txt', prompt_content='Output exactly the JSON
string \"hello\" without quotes. Write it to result.json.', result_relpath='result.json', stage='test', input_files=[]. The LLM
subprocess (via run_opencode_traced) writes result.json containing the JSON string 'hello' and exits successfully. The function
returns the Python string 'hello', which is neither dict, list, nor None.",
  "offending_statements": "Line 61: return json.load(f)",
  "reason": "The code returns the raw parsed JSON value from the file, which can be a string, number, or boolean, violating the
specification's requirement that the return value be only a dict, a list, or None."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f4852cdb144a4c9daa102b9e00593bf9_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f4852cdb144a4c9daa102b9e00593bf9_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f4852cdb144a4c9daa102b9e00593bf9_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-244 — `src--incremental_reasoner-py--check_last_run_existence`

- 四类结论：可能有 Bug
- 分类依据：沿用旧 `fm_agent/bug_list.md` 的逐条审计结论；沿用旧清单的逐条审计结论。
- **Validator**: `confirmed`; attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / `incremental_pipeline`; 源代码 `src/incremental_reasoner.py`
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns True iff all of the following hold simultaneously: (a) `proj_dir/fm_agent/phases.json` exists as a regular file; (b) `proj_dir/fm_agent/extracted_functions/` exists as a directory and contains at least one non-sidecar function file; (c) every non-sidecar function file found under `extracted_functions/` (when `submodules` is provided, restricted to those whose relative path begins with at least one submodule path followed by a separator) has both a valid `.spec.json` and `.info.json` sidecar. Returns False if any condition (a), (b), or (c) is not met, indicating the previous run is incomplete and not a sound basis for incremental analysis. Files whose names match the metadata sidecar naming pattern (`.spec.json` or `.info.json` suffix) are excluded from the function-file count and readiness check. The function has no side effects: it reads from disk only.
- **Actual behavior**: The function returns True iff (1) the file `<proj_dir>/fm_agent/phases.json` exists, (2) the directory `<proj_dir>/fm_agent/extracted_functions` exists, (3) there exists at least one regular file within the `extracted_functions` directory tree that is not a metadata sidecar (i.e., does not end in `.spec.json` or `.info.json`) and, when `submodules` is provided, whose relative path from `extracted_functions` starts with one of the submodule paths immediately followed by a forward slash, and (4) every such file `f` satisfies

- `is_file_ready(f)` (i.e., both `f.spec.json` and `f.info.json` exist as reachable, valid JSON files with the required schemas). Otherwise the function returns `False`. Formally: let `work_dir = proj_dir + '/fm_agent'`, `extracted_dir = work_dir + '/extracted_functions'`. Let $E = \{ f \mid f \text{ is a file under } extracted_dir \text{ (recursively) and not } sidecar(f) \}$. Define $E' = \text{if } submodules \text{ is None then } E \text{ else } \{ f \text{ in } E \mid is_under_submodules(relpath(f, extracted_dir), submodules) \}$. The return value V satisfies $V (is_file(work_dir + '/phases.json') is_dir(extracted_dir) E' f E' : ready(f))$.
- **Code evidence:** Line 31: `if submodules and not _is_under_submodules(rel, submodules):`
 - **Trigger condition:** The specification (Condition B) separates condition (b) (unrestricted existence of at least one non-sidecar function file) from the submodule-based restriction in condition (c). The code, through Lines 31-35, requires at least one ready function file within the selected submodules, which is stricter. When submodules are provided but no file matches them while a ready file exists elsewhere, the code returns `False` while the specification expects `True`.
 - **Validator trigger:** When submodules are provided but no extracted function file resides under any of the specified submodule paths while files exist elsewhere, the code returns `False` (submodule filter applied before `saw_function` count) but the spec expects `True` (condition b only requires any file to exist unconditionally).
 - **Probe stdout:** CONFIRMED — actual: `False` | expected: `True` (`submodules=[subdir_a]` but file is under `subdir_b`)

► SPEC sidecar (完整)

```
{
  "signature": "check_last_run_existence(proj_dir: str, submodules: list | None) -> bool",
  "pre_condition": "proj_dir is an existing directory that was previously the target of a full pipeline run (run_pipeline). If provided, submodules is a non-empty list of project-relative subdirectory paths.",
  "post_condition": "Returns True iff all of the following hold simultaneously: (a) proj_dir/fm_agent/phases.json exists as a regular file; (b) proj_dir/fm_agent/extracted_functions/ exists as a directory and contains at least one non-sidecar function file; (c) every non-sidecar function file found under extracted_functions/ (when submodules is provided, restricted to those whose relative path begins with at least one submodule path followed by a separator) has both a valid .spec.json and .info.json sidecar. Returns False if any condition (a), (b), or (c) is not met, indicating the previous run is incomplete and not a sound basis for incremental analysis. Files whose names match the metadata sidecar naming pattern (.spec.json or .info.json suffix) are excluded from the function-file count and readiness check. The function has no side effects: it reads from disk only."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "is_file_ready",
      "signature": "is_file_ready(fpath: str) -> bool",
      "pre_condition": "fpath is an absolute path to an extracted function file within the extracted_functions/ directory.",
      "post_condition": "Returns True iff both {{fpath}}.spec.json and {{fpath}}.info.json exist as valid JSON files containing the required spec and info schemas respectively. Returns False if either sidecar is missing, unparseable, or does not contain the expected structure."
    },
    {
      "name": "_is_metadata_sidecar",
      "signature": "_is_metadata_sidecar(fpath: str) -> bool",
      "pre_condition": "fpath is a file path within the extracted_functions/ directory tree.",
      "post_condition": "Returns True if the filename ends with .spec.json or .info.json; returns False otherwise."
    },
    {
      "name": "_is_under_submodules",
      "signature": "_is_under_submodules(rel: str, submodules: list[str]) -> bool",
      "pre_condition": "rel is a forward-slash-separated relative path from the extracted_functions/ root. submodules is a list of non-empty project-relative subdirectory paths, none containing a trailing separator.",
      "post_condition": "Returns True iff rel starts with one of the paths in submodules immediately followed by a path separator (e.g., '{submodule_path}/'). Returns False if rel does not begin with any submodule path in this way."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/check_last_run_existence.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns True iff all of the following hold simultaneously: (a) proj_dir/fm_agent/phases.json exists as a regular file; (b) proj_dir/fm_agent/extracted_functions/ exists as a directory and contains at least one non-sidecar function file; (c) every non-sidecar function file found under extracted_functions/ (when submodules is provided, restricted to those whose relative path begins with at least one submodule path followed by a separator) has both a valid .spec.json and .info.json sidecar. Returns False if any condition (a), (b), or (c) is not met, indicating the previous run is incomplete and not a sound basis for incremental analysis. Files whose names match the metadata sidecar naming pattern (.spec.json or .info.json suffix) are excluded from the function-file count and readiness check. The function has no side effects: it reads from disk only.",
    "actual_behavior": "The function returns True iff (1) the file <proj_dir>/fm_agent/phases.json exists, (2) the directory
```



```
<proj_dir>/fm_agent/extracted_functions exists, (3) there exists at least one regular file within the extracted_functions
directory tree that is not a metadata sidecar (i.e., does not end in .spec.json or .info.json) and, when submodules is provided,
whose relative path from extracted_functions starts with one of the submodule paths immediately followed by a forward slash, and
(4) every such file f satisfies is_file_ready(f) (i.e., both f.spec.json and f.info.json exist as reachable, valid JSON files with
the required schemas). Otherwise the function returns False. Formally: let work_dir = proj_dir + '/fm_agent', extracted_dir =
work_dir + '/extracted_functions'. Let E = { f | f is a file under extracted_dir (recursively) and not sidecar(f) }. Define E' =
if submodules is None then E else { f in E | is_under_submodules(relpath(f, extracted_dir), submodules) }. The return value V
satisfies V ( is_file(work_dir + '/phases.json') is_dir(extracted_dir) E' f E' : ready(f) ).",
"code_evidence": "Line 31: if submodules and not _is_under_submodules(rel, submodules):",
"trigger_condition": "The specification (Condition B) separates condition (b) (unrestricted existence of at least one non-
sidecar function file) from the submodule-based restriction in condition (c). The code, through Lines 31-35, requires at least one
ready function file within the selected submodules, which is stricter. When submodules are provided but no file matches them while
a ready file exists elsewhere, the code returns False while the specification expects True."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--check_last_run_existence",
  "source_file": "fm_agent/extracted_functions/src/incremental_reasoner-py/check_last_run_existence.py",
  "function_name": "check_last_run_existence",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--check_last_run_existence.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--check_last_run_existence.md",
  "probe_stdout": "CONFIRMED - actual: False | expected: True (submodules=[subdir_a] but file is under subdir_b)",
  "trigger_summary": "When submodules are provided but no extracted function file resides under any of the specified submodule
paths while files exist elsewhere, the code returns False (submodule filter applied before saw_function count) but the spec
expects True (condition b only requires any file to exist unconditionally)."
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:09:14.228843Z	3132
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:09:43.030393Z	9501

事件 1 — [llm_de9a5f56a7c44eb2bafcd686a5aa6504](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns True iff (1) the file <proj_dir>/fm_agent/phases.json exists, (2) the directory
<proj_dir>/fm_agent/extracted_functions exists, (3) there exists at least one regular file within the extracted_functions
directory tree that is not a metadata sidecar (i.e., does not end in .spec.json or .info.json) and, when submodules is provided,
whose relative path from extracted_functions starts with one of the submodule paths immediately followed by a forward slash, and
(4) every such file f satisfies is_file_ready(f) (i.e., both f.spec.json and f.info.json exist as reachable, valid JSON files with
the required schemas). Otherwise the function returns False. Formally: let work_dir = proj_dir + '/fm_agent', extracted_dir =
work_dir + '/extracted_functions'. Let E = { f | f is a file under extracted_dir (recursively) and not sidecar(f) }. Define E' =
if submodules is None then E else { f in E | is_under_submodules(relpath(f, extracted_dir), submodules) }. The return value V
satisfies V = ( is_file(work_dir + '/phases.json') ^ is_dir(extracted_dir) ^ E' != 0 ^ V f in E' : ready(f) )."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_de9a5f56a7c44eb2bafcd686a5aa6504_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_de9a5f56a7c44eb2bafcd686a5aa6504_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_de9a5f56a7c44eb2bafcd686a5aa6504_response.txt)

事件 2 — [llm_f47c10a610de45488c89b7c256b79f8a](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "Create a project directory 'proj' containing 'proj/fm_agent/phases.json' (regular file) and
'proj/fm_agent/extracted_functions/other.py' (non-sidecar file) with valid 'other.py.spec.json' and 'other.py.info.json'. No files
under a 'some_submodule' subdirectory exist. Call check_last_run_existence(proj_dir='proj', submodules=['some_submodule']). The
code returns False because no function file falls under 'some_submodule/', but the specification returns True: condition (b) is
satisfied by the existence of 'other.py', and condition (c) is vacuously true as no file matches the submodule restriction.",
  "offending_statements": "Line 31: if submodules and not _is_under_submodules(rel, submodules):",
```



```
"reason": "The specification (Condition B) separates condition (b) (unrestricted existence of at least one non-sidecar function file) from the submodule-based restriction in condition (c). The code, through Lines 31-35, requires at least one ready function file within the selected submodules, which is stricter. When submodules are provided but no file matches them while a ready file exists elsewhere, the code returns False while the specification expects True."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f47c10a610de45488c89b7c256b79f8a_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f47c10a610de45488c89b7c256b79f8a_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f47c10a610de45488c89b7c256b79f8a_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: **orchestration**

FMA-MISMATCH-248 — **dashboard.py** - **State::_ingest_event**

- 四类结论: 可能有 Bug
- 分类依据: probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- **Validator**: **confirmed**; attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / **orchestration**; 原源码 **dashboard.py**
- 证据: 抽取源码 · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: self reflects the cumulative contents of all events ingested in order, now including ev. self.first_event_time equals the earliest parseable start_time across all ingested events (None if none). self.last_event_time equals the latest parseable end_time across all ingested events (None if none). self.stage_counts[stage][status] equals the count of events with that (stage, status) pair across all ingested events. For each event where ev['type'] is 'llm_call': self.totals reflects the sum of input, output, cache read, and cache write token counts from all LLM call events; self.cost_native reflects the cumulative cost computed per LLM call event; self.cache_window contains exactly one (cache_read, total_input) pair for each LLM call event whose total input tokens exceeded zero, appended in ingestion order; at most one of self.verification_success, self.verification_mismatch, or self.verification_error is incremented per event, determined by the event's status and whether its purpose is 'check_post_implies_spec'; a recent-event entry and an LLM-status entry are appended carrying timestamp, stage, status, and purpose for the event. For each event where ev['type'] is 'opcode_call': a recent-event entry is appended carrying timestamp, stage, status, and summary for the event. Appended entries preserve occurrence order across successive ingestions.
- **Actual behavior**: After the normal execution of **_ingest_event** (no unhandled exceptions), the following holds for the object **self**.

Let **pre** denote the object state immediately before the call, **post** the state immediately after. Let **stage** = ev.get("stage", "?"), **status** = ev.get("status", "?"), **et** = ev.get("type"). Let **start** = **_parse_iso**(ev.get("start_time")) and **end** = **_parse_iso**(ev.get("end_time")).

1. First/Last time updates:

- **post.first_event_time** = **start** if **start** is not None and (**pre.first_event_time** is None or **start** < **pre.first_event_time**), else **pre.first_event_time**.
- **post.last_event_time** = **end** if **end** is not None and (**pre.last_event_time** is None or **end** > **pre.last_event_time**), else **pre.last_event_time**.

2. Stage counter:

- **post.stage_counts[stage][status]** = **pre.stage_counts[stage][status]** + 1.

3. Conditional on event type:

(A) If **et** == "llm_call": Let **md** = ev.get("metadata", {}), **model** = md.get("model"). a. **post.model_seen** = **model** if **model** is truthy and not **pre.model_seen** is truthy, else **pre.model_seen**. b. Let **usage** = md.get("usage") or {}. If **usage** is non-empty: Compute token counts via the Anthropic/OpenAI shape detection: - **inp0** = usage.get("input_tokens", 0) or 0 - **out0** = usage.get("output_tokens", 0) or 0 - **cr0** = usage.get("cache_read_input_tokens", 0) or 0 - **cw0** = usage.get("cache_creation_input_tokens", 0) or 0 If none of **inp0**, **out0**, **cr0**, **cw0** are truthy (all zero/falsy): - **inp1** = usage.get("prompt_tokens", 0) or 0 - **out1** = usage.get("completion_tokens", 0) or 0 - **hit** = usage.get("prompt_cache_hit_tokens") - If **hit** is not None: **cr1** = **hit**; **inp1** = usage.get("prompt_cache_miss_tokens", **inp1** - **hit**). - Else: **cr1** = 0; **inp1** = **inp1**. - Set **inp** = **inp1**, **out** = **out1**, **cr** = **cr1**, **cw** = 0. Else: - Set **inp** = **inp0**, **out** = **out0**, **cr** = **cr0**, **cw** = **cw0**. Then update totals: - **post.totals["input"]** = **pre.totals["input"]** + **inp** - **post.totals["output"]** = **pre.totals["output"]** + **out** - **post.totals["cache_read"]** = **pre.totals["cache_read"]** + **cr** - **post.totals["cache_write"]** =

`pre.totals["cache_write"] + cw - post.cost_native = pre.cost_native + _cost_from_usage(model, usage)` - Let `total_in = inp + cr + cw`. If `total_in > 0`: `post.cache_window = pre.cache_window + [(cr, total_in)]` Else `post.cache_window` unchanged. Else (`usage empty/falsy`): `totals`, `cost_native`, and `cache_window` unchanged. c. Verification counters: If `status == "success"` and `md.get("purpose") == "check_post_implies_spec"`: `post.verification_success = pre.verification_success + 1` Else if `status == "mismatch"`: `post.verification_mismatch = pre.verification_mismatch + 1` Else if `status == "error"`: `post.verification_error = pre.verification_error + 1` Else: verification counters unchanged. d. Summary: `summary = ev.get("summary")` or `md.get("purpose")` or `"llm_call"`. e. Timeline appends: - `post._push_recent` is called with timestamp `end` or `start`, stage, status, and summary. This appends a new entry to the recent-event timeline. - `post._push_llm_status` is called with timestamp `end` or `start`, source `"direct"`, label `md.get("purpose")` or `summary`, status, model, code `_llm_code_for_event(status)`, and detail `md.get("error")`. This appends a new entry to the LLM-status timeline.

(B) If `et == "opencode_call"`: a. `summary = ev.get("summary")` or `"opencode_call"`. b. `post._push_recent` is called with timestamp `end` or `start`, stage, status, and summary, appending to the recent-event timeline. c. No other attribute modifications (`model_seen`, `totals`, verification counters, etc. remain as pre).

(C) Otherwise (`et` not in `{"llm_call", "opencode_call"}`): Only the first/last time and stage-counter changes described in (1)(2) apply; no further modifications.

All timeline sequences (recent-events and LLM-status) are extended by one element each when the corresponding push methods are called, preserving previous entries.

- **Code evidence:** Line 44: `elif status == "mismatch"`: Line 46: `elif status == "error"`:
- **Trigger condition:** The specification requires verification counters to be incremented only when purpose is `'check_post_implies_spec'`, but these branches increment `self.verification_mismatch` and `self.verification_error` unconditionally, so a non-verification `llm_call` event with status `'mismatch'` or `'error'` incorrectly increments the respective counter.
- **Validator trigger:** Non-verification `llm_call` event with status `'mismatch'` or `'error'` incorrectly increments verification counters because the mismatch and error branches lack a purpose check (only the success branch checks `purpose == 'check_post_implies_spec'`).
- **Probe stdout:** CONFIRMED — `verification_mismatch`: 0 -> 1 (incorrectly incremented for non-verification event) | `verification_error`: 0 -> 1 (incorrectly incremented for non-verification event)

► SPEC sidecar (完整)

```
{
  "signature": "_ingest_event(self, ev)",
  "pre_condition": "ev is a dict, parsed from a valid JSON object line in the trace events file. ev maps string keys to JSON-typed values.",
  "post_condition": "self reflects the cumulative contents of all events ingested in order, now including ev.
self.first_event_time equals the earliest parseable start_time across all ingested events (None if none). self.last_event_time equals the latest parseable end_time across all ingested events (None if none). self.stage_counts[stage][status] equals the count of events with that (stage, status) pair across all ingested events. For each event where ev['type'] is 'llm_call': self.totals reflects the sum of input, output, cache read, and cache write token counts from all LLM call events; self.cost_native reflects the cumulative cost computed per LLM call event; self.cache_window contains exactly one (cache_read, total_input) pair for each LLM call event whose total input tokens exceeded zero, appended in ingestion order; at most one of self.verification_success, self.verification_mismatch, or self.verification_error is incremented per event, determined by the event's status and whether its purpose is 'check_post_implies_spec'; a recent-event entry and an LLM-status entry are appended carrying timestamp, stage, status, and purpose for the event. For each event where ev['type'] is 'opencode_call': a recent-event entry is appended carrying timestamp, stage, status, and summary for the event. Appended entries preserve occurrence order across successive ingestions."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_parse_iso",
      "signature": "_parse_iso(ts)",
      "pre_condition": "ts is a string value extracted from the ev dict's 'start_time' or 'end_time' field, or None if the field was absent or falsy.",
      "post_condition": "Returns a parsed datetime value if ts is a valid ISO 8601 timestamp string. Returns None if ts is None or cannot be parsed as a valid timestamp."
    },
    {
      "name": "_cost_from_usage",
      "signature": "_cost_from_usage(model, usage)",
      "pre_condition": "model is a string model identifier or None. usage is a dict containing token-count fields reflecting a single LLM request's consumption.",
      "post_condition": "Returns the monetary cost in dollars for the given model and token counts. Returns 0.0 if model is None or unrecognized."
    }
  ],
}
```

```

{
  "name": "_push_recent",
  "signature": "_push_recent(self, ts, stage, status, summary)",
  "pre_condition": "ts is a datetime or None. stage, status, and summary are strings describing the event.",
  "post_condition": "A new entry is appended to self's in-memory recent-event timeline, recording the timestamp, stage, status, and summary. Entries are appended in call order."
},
{
  "name": "_push_llm_status",
  "signature": "_push_llm_status(self, ts, source, label, status, model, code, detail)",
  "pre_condition": "ts is a datetime or None. source is a string identifying the origin (e.g., 'direct' or 'opencode'). label, status, model, code, and detail are strings describing the LLM call outcome.",
  "post_condition": "A new entry is appended to self's in-memory LLM-status timeline, recording the timestamp, source, label, status, model, status code, and detail. Entries are appended in call order."
},
{
  "name": "_llm_code_for_event",
  "signature": "_llm_code_for_event(status)",
  "pre_condition": "status is a string representing an event status value.",
  "post_condition": "Returns a short display code string corresponding to the status value. Returns a default code for status values that have no specific mapping."
}
]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/State::_ingest_event.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "self reflects the cumulative contents of all events ingested in order, now including ev. self.first_event_time equals the earliest parseable start_time across all ingested events (None if none). self.last_event_time equals the latest parseable end_time across all ingested events (None if none). self.stage_counts[stage][status] equals the count of events with that (stage, status) pair across all ingested events. For each event where ev['type'] is 'llm_call': self.totals reflects the sum of input, output, cache read, and cache write token counts from all LLM call events; self.cost_native reflects the cumulative cost computed per LLM call event; self.cache_window contains exactly one (cache_read, total_input) pair for each LLM call event whose total input tokens exceeded zero, appended in ingestion order; at most one of self.verification_success, self.verification_mismatch, or self.verification_error is incremented per event, determined by the event's status and whether its purpose is 'check_post_implies_spec'; a recent-event entry and an LLM-status entry are appended carrying timestamp, stage, status, and purpose for the event. For each event where ev['type'] is 'opencode_call': a recent-event entry is appended carrying timestamp, stage, status, and summary for the event. Appended entries preserve occurrence order across successive ingestions.",
    "actual_behavior": "After the normal execution of `_ingest_event` (no unhandled exceptions), the following holds for the object `self`.\n\nLet `pre` denote the object state immediately before the call, `post` the state immediately after. Let `stage = ev.get("stage", "")`, `status = ev.get("status", "")`, `et = ev.get("type")`. Let `start = _parse_iso(ev.get("start_time"))` and `end = _parse_iso(ev.get("end_time"))`. \n\n1. First/Last time updates:\n - `post.first_event_time = start` if `start is not None` and (`pre.first_event_time is None` or `start < pre.first_event_time`), else `pre.first_event_time`. \n - `post.last_event_time = end` if `end is not None` and (`pre.last_event_time is None` or `end > pre.last_event_time`), else `pre.last_event_time`. \n\n2. Stage counter:\n - `post.stage_counts[stage][status] = pre.stage_counts[stage][status] + 1`. \n\n3. Conditional on event type:\n\n(A) If `et == "llm_call"`:\n Let `md = ev.get("metadata", {})`, `model = md.get("model")`. \n a. `post.model_seen = model` if `model` is truthy and `not pre.model_seen` is truthy, else `pre.model_seen`. \n b. Let `usage = md.get("usage")` or `{}`. If `usage` is non-empty:\n Compute token counts via the Anthropic/OpenAI shape detection:\n - `inp0 = usage.get("input_tokens", 0)` or `0` \n - `cr0 = usage.get("cache_read_input_tokens", 0)` or `0` \n - `cw0 = usage.get("cache_creation_input_tokens", 0)` or `0` \n If none of `inp0, out0, cr0, cw0` are truthy (all zero/falsy):\n - `inp1 = usage.get("prompt_tokens", 0)` or `0` \n - `out1 = usage.get("completion_tokens", 0)` or `0` \n - `hit = usage.get("prompt_cache_hit_tokens")` \n If `hit is not None`: `cr1 = hit`; `inp1 = usage.get("prompt_cache_miss_tokens", inp1 - hit)` \n Else: `cr1 = 0` \n `inp1 = inp1` \n Set `inp = inp1`, `out = out1`, `cr = cr1`, `cw = 0` \n Else:\n Set `inp = inp0`, `out = out0`, `cr = cr0`, `cw = cw0` \n Then update totals:\n - `post.totals["input"] = pre.totals["input"] + inp` \n - `post.totals["output"] = pre.totals["output"] + out` \n - `post.totals["cache_read"] = pre.totals["cache_read"] + cr` \n - `post.totals["cache_write"] = pre.totals["cache_write"] + cw` \n - `post.cost_native = pre.cost_native + _cost_from_usage(model, usage)` \n Let `total_in = inp + cr + cw`. If `total_in > 0`:\n `post.cache_window = pre.cache_window + [(cr, total_in)]` \n Else `post.cache_window` unchanged. \n c. Verification counters:\n `md.get("purpose") == "check_post_implies_spec"`:\n `post.verification_success = pre.verification_success + 1` \n Else if `status == "mismatch"`:\n `post.verification_mismatch = pre.verification_mismatch + 1` \n Else if `status == "error"`:\n `post.verification_error = pre.verification_error + 1` \n Else: verification counters unchanged. \n\n d. Summary: `summary = ev.get("summary")` or `md.get("purpose")` or `"llm_call"` \n\n e. Timeline appends:\n - `post._push_recent` is called with timestamp `end` or `start`, stage, status, and summary. This appends a new entry to the recent-event timeline. \n - `post._push_llm_status` is called with timestamp `end` or `start`, source `"direct"`, label `md.get("purpose")` or `summary`, status, model, code `_llm_code_for_event(status)`, and detail `md.get("error")`. This appends a new entry to the LLM-status timeline. \n\n(B) If `et == "opencode_call"`:\n a. `summary = ev.get("summary")` or `"opencode_call"` \n b. `post._push_recent` is called with timestamp `end` or `start`, stage, status, and summary, appending to the recent-event timeline. \n c. No other attribute modifications (model_seen, totals, verification counters, etc. remain as pre). \n\n(C) Otherwise (et not in {"llm_call", "opencode_call"}):\n Only the first/last time and stage-counter changes described in (1)(2) apply; no further modifications. \n\nAll timeline sequences (recent-events and LLM-status) are extended by one element each when the corresponding push methods are called, preserving previous entries.",
    "code_evidence": "Line 44: elif status == \"mismatch\":\nLine 46: elif status == \"error\":",
    "trigger_condition": "The specification requires verification counters to be incremented only when purpose is 'check_post_implies_spec', but these branches increment self.verification_mismatch and self.verification_error unconditionally, so a non-verification llm_call event with status 'mismatch' or 'error' incorrectly increments the respective counter."
  }
}

```

```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "dashboard-py--State::_ingest_event",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/dashboard.py",
  "function_name": "State::_ingest_event",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_dashboard-py--State::_ingest_event.py",
  "detail_file": "fm_agent/bug_validation/dashboard-py--State::_ingest_event.md",
  "probe_stdout": "CONFIRMED - verification_mismatch: 0 -> 1 (incorrectly incremented for non-verification event) |
verification_error: 0 -> 1 (incorrectly incremented for non-verification event)",
  "trigger_summary": "Non-verification llm_call event with status 'mismatch' or 'error' incorrectly increments verification
counters because the mismatch and error branches lack a purpose check (only the success branch checks purpose ==
'check_post_implies_spec')."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:25:02.405913Z	8976
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:26:46.925125Z	8397

事件 1 — [llm_9b11d7b9daa74b43ae2a38f40450e17a](#)

► parsed_json（完整）

```
{
  "post_condition": "After the normal execution of `_ingest_event` (no unhandled exceptions), the following holds for the object
`self`.\\n\\nLet `pre` denote the object state immediately before the call, `post` the state immediately after. Let `stage` =
ev.get(`stage`, `\"?\"`), `status` = ev.get(`status`, `\"?\"`), `et` = ev.get(`type`), `start` =
_parse_iso(ev.get(`start_time`)) and `end` = _parse_iso(ev.get(`end_time`)).\\n\\n1. First/Last time updates:\\n  -
`post.first_event_time` = `start` if `start` is not None and (`pre.first_event_time` is None or `start` < `pre.first_event_time`),
else `pre.first_event_time`.\\n  - `post.last_event_time` = `end` if `end` is not None and (`pre.last_event_time` is None or `end` >
`pre.last_event_time`), else `pre.last_event_time`.\\n\\n2. Stage counter:\\n  - `post.stage_counts[stage][status]` =
pre.stage_counts[stage][status] + 1.\\n\\n3. Conditional on event type:\\n\\n  (A) If `et` == `llm_call`:\\n    Let `md` =
ev.get(`metadata`, {}), `model` = md.get(`model`).\\n    a. `post.model_seen` = `model` if `model` is truthy and `not
pre.model_seen` is truthy, else `pre.model_seen`.\\n    b. Let `usage` = md.get(`usage`) or {}. If `usage` is non-empty:\\n
Compute token counts via the Anthropic/OpenAI shape detection:\\n    - `inp0` = usage.get(`input_tokens`, 0) or 0\\n
- `out0` = usage.get(`output_tokens`, 0) or 0\\n
- `cr0` = usage.get(`cache_read_input_tokens`, 0) or 0\\n
- `cw0` = usage.get(`cache_creation_input_tokens`, 0) or 0\\n
If none of `inp0`, `out0`, `cr0`, `cw0` are truthy (all
zero/falsy):\\n    - `inp1` = usage.get(`prompt_tokens`, 0) or 0\\n
- `out1` =
usage.get(`completion_tokens`, 0) or 0\\n
- `hit` = usage.get(`prompt_cache_hit_tokens`)\\n
- If
`hit` is not None: `cr1` = `hit`; `inp1` = usage.get(`prompt_cache_miss_tokens`, `inp1` - `hit`).\\n
- Else: `cr1` = 0;
`inp1` = `inp1`.\\n
- Set `inp` = `inp1`, `out` = `out1`, `cr` = `cr1`, `cw` = 0.\\n
Else:\\n
Set `inp` = `inp0`, `out` = `out0`, `cr` = `cr0`, `cw` = `cw0`.\\n
Then update totals:\\n
- `post.totals[\"input\"]`
= pre.totals[\"input\"] + `inp`\\n
- `post.totals[\"output\"]` = pre.totals[\"output\"] + `out`\\n
- `post.totals[\"cache_read\"]` = pre.totals[\"cache_read\"] + `cr`\\n
- `post.totals[\"cache_write\"]` = pre.totals[\"cache_write\"] + `cw`\\n
- `post.cost_native` = pre.cost_native + _cost_from_usage(model, usage)\\n
- Let `total_in` = `inp` + `cr` + `cw`. If `total_in` > 0:\\n
`post.cache_window` = pre.cache_window + [(cr,
total_in)]\\n
Else `post.cache_window` unchanged.\\n
and cache_window unchanged.\\n
c. Verification counters:\\n
`md.get(`purpose`) == `check_post_implies_spec`:\\n
+ 1\\n
Else if `status` == `mismatch`:\\n
pre.verification_mismatch + 1\\n
Else if `status` == `error`:\\n
`post.verification_error` =
pre.verification_error + 1\\n
Else: verification counters unchanged.\\n
d. Summary: `summary` =
ev.get(`summary`) or md.get(`purpose`) or `llm_call`.\\n
e. Timeline appends:\\n
- `post._push_recent`
is called with timestamp `end` or `start`, stage, status, and summary. This appends a new entry to the recent-event timeline.\\n
- `post._push_llm_status` is called with timestamp `end` or `start`, source `direct`, label `md.get(`purpose`) or summary`,
status, model, code _llm_code_for_event(status), and detail `md.get(`error`)`. This appends a new entry to the LLM-status
timeline.\\n\\n  (B) If `et` == `opencode_call`:\\n    a. `summary` = ev.get(`summary`) or `opencode_call`.\\n    b.
`post._push_recent` is called with timestamp `end` or `start`, stage, status, and summary, appending to the recent-event timeline.\\n
c. No other attribute modifications (model_seen, totals, verification counters, etc. remain as pre).\\n\\n  (C) Otherwise (et not
in {`llm_call`, `opencode_call`}):\\n    Only the first/last time and stage-counter changes described in (1)-(2) apply; no
further modifications.\\n\\nAll timeline sequences (recent-events and LLM-status) are extended by one element each when the
corresponding push methods are called, preserving previous entries."
}
```

事件 2 — [llm_d2fd29c382f3492591d801d4e2d6b5ed](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "{\n\"type\": \"llm_call\", \"status\": \"mismatch\", \"metadata\": {\n\"purpose\": \"other_thing\"},\n\"start_time\": \"2023-01-01T00:00:00Z\", \"end_time\": \"2023-01-01T00:01:00Z\"},\n\"offending_statements\": \"Line 44:           elif status == \"mismatch\":\nLine 46:           elif status == \"error\":\", \"reason\": \"The specification requires verification counters to be incremented only when purpose is 'check_post_implies_spec', but these branches increment self.verification_mismatch and self.verification_error unconditionally, so a non-verification llm_call event with status 'mismatch' or 'error' incorrectly increments the respective counter.\"",
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d2fd29c382f3492591d801d4e2d6b5ed_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d2fd29c382f3492591d801d4e2d6b5ed_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d2fd29c382f3492591d801d4e2d6b5ed_response.txt) **复核动作**

- 核对 `probe` 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-252 — [dashboard-py--State::scan_bugs](#)

- 四类结论： 可能有 Bug
- 分类依据: `probe` 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [orchestration](#); 原源码 [dashboard.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: `self.bugs_confirmed`, `self.bugs_not_confirmed`, and `self.bugs_pending` are updated to reflect the current state of `self.bug_dir`. When the directory exists, every `.result.json` file is classified: a file whose `confirmation_status` (case-insensitive) contains the substring 'confirm' without containing 'not' increments `bugs_confirmed`; any other file increments `bugs_not_confirmed`. `self.bugs_pending` is `max(0, completed_bug_validation_stages (bugs_confirmed + bugs_not_confirmed))`. When `self.bug_dir` does not exist, all three counters are zero.
- **Actual behavior**: After execution, the attributes `self.bugs_confirmed`, `self.bugs_not_confirmed`, and `self.bugs_pending` are updated according to the following rules, and no other attributes are modified. Let `E` be `self.bug_dir.exists()` before the method. If `E`, then `self.bugs_confirmed = 0`, `self.bugs_not_confirmed = 0`, and `self.bugs_pending = max(0, bv_done)` where `bv_done = sum(self.stage_counts.get('bug_validation', {}).values())` (with original value). If `E`, define:
 - `F = { p : p self.bug_dir.glob('*.result.json') }` (the set of file paths matching the pattern)
 - `S = { p F : opening and parsing p as JSON succeeds without raising any exception }`
 - For each `p S`, let `status(p) = (d.get('confirmation_status') or '').lower()` where `d` is the parsed JSON object
 - `C = { p S : 'confirm' status(p) 'not' status(p) }`
 - `N = S \ C`

Then: `self.bugs_confirmed = |C|` `self.bugs_not_confirmed = |N|` `bv_done = sum of values in self.stage_counts.get('bug_validation', {})` (as it was at method entry) `self.bugs_pending = max(0, bv_done - |S|)`.

That is, `bugs_confirmed` counts parsed files whose lowercased confirmation status contains "confirm" but not "not"; `bugs_not_confirmed` counts the remaining parsed files (those containing "not" or neither string); `bugs_pending` is the maximum of zero and the difference between the sum of count values for the 'bug_validation' stage (or 0 if absent) and the total number of successfully parsed result files.

- **Code evidence**: Line 4: if not `self.bug_dir.exists()`: Line 5: return
- **Trigger condition**: When the directory does not exist, the code returns early without setting `self.bugs_pending` to zero. Condition A (the actual behaviour) results in `self.bugs_pending` being `max(0, bv_done)` (or retains its previous value), whereas Condition B explicitly states that all three counters must be zero.
- **Validator trigger**: When `self.bug_dir` does not exist, `scan_bugs()` returns early without resetting `self.bugs_pending` to zero, leaving a stale value.

- **Probe stdout:** CONFIRMED — bugs_pending: 5 (should be 0), bugs_confirmed: 0, bugs_not_confirmed: 0

► SPEC sidecar（完整）

```
{
  "signature": "State.scan_bugs(self)",
  "pre_condition": "self.bug_dir is a Path attribute. The directory self.bug_dir, when present, may contain zero or more .result.json files. Each such file, when valid JSON, has a 'confirmation_status' key whose value is a string.",
  "post_condition": "self.bugs_confirmed, self.bugs_not_confirmed, and self.bugs_pending are updated to reflect the current state of self.bug_dir. When the directory exists, every .result.json file is classified: a file whose confirmation_status (case-insensitive) contains the substring 'confirm' without containing 'not' increments bugs_confirmed; any other file increments bugs_not_confirmed. self.bugs_pending is max(0, completed_bug_validation_stages - (bugs_confirmed + bugs_not_confirmed)). When self.bug_dir does not exist, all three counters are zero."
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/State::scan_bugs.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "self.bugs_confirmed, self.bugs_not_confirmed, and self.bugs_pending are updated to reflect the current state of self.bug_dir. When the directory exists, every .result.json file is classified: a file whose confirmation_status (case-insensitive) contains the substring 'confirm' without containing 'not' increments bugs_confirmed; any other file increments bugs_not_confirmed. self.bugs_pending is max(0, completed_bug_validation_stages - (bugs_confirmed + bugs_not_confirmed)). When self.bug_dir does not exist, all three counters are zero.",
    "actual_behavior": "After execution, the attributes `self.bugs_confirmed`, `self.bugs_not_confirmed`, and `self.bugs_pending` are updated according to the following rules, and no other attributes are modified. Let `E` be `self.bug_dir.exists()` before the method. If `E`, then `self.bugs_confirmed = 0`, `self.bugs_not_confirmed = 0`, and `self.bugs_pending = max(0, bv_done)` where `bv_done = sum(self.stage_counts.get('bug_validation', {})).values()` (with original value). If `!E`, define: `F` = { p : p self.bug_dir.glob('*.result.json') } (the set of file paths matching the pattern)\n- `S` = { p F : opening and parsing `p` as JSON succeeds without raising any exception }\n- For each `p S`, let `status(p)` = `(d.get('confirmation_status') or '').lower()` where `d` is the parsed JSON object\n- `C` = { p S : 'confirm' status(p) 'not' status(p) }\n- `N` = `S \ C`\n\nThen:\n`self.bugs_confirmed` = |C|\n`self.bugs_not_confirmed` = |N|\n`bv_done` = sum of values in `self.stage_counts.get('bug_validation', {})` (as it was at method entry)\n`self.bugs_pending` = max(0, bv_done - |S|)\n\nThat is, `bugs_confirmed` counts parsed files whose lowercased confirmation status contains `"confirm"` but not `"not"`; `bugs_not_confirmed` counts the remaining parsed files (those containing `"not"` or neither string); `bugs_pending` is the maximum of zero and the difference between the sum of count values for the 'bug_validation' stage (or 0 if absent) and the total number of successfully parsed result files.",
    "code_evidence": "Line 4:         if not self.bug_dir.exists():\nLine 5:             return",
    "trigger_condition": "When the directory does not exist, the code returns early without setting self.bugs_pending to zero. Condition A (the actual behaviour) results in self.bugs_pending being max(0, bv_done) (or retains its previous value), whereas Condition B explicitly states that all three counters must be zero."
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "dashboard-py--State::scan_bugs",
  "source_file": "dashboard.py",
  "function_name": "scan_bugs",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_dashboard-py--State::scan_bugs.py",
  "detail_file": "fm_agent/bug_validation/dashboard-py--State::scan_bugs.md",
  "probe_stdout": "CONFIRMED — bugs_pending: 5 (should be 0), bugs_confirmed: 0, bugs_not_confirmed: 0\n",
  "trigger_summary": "When self.bug_dir does not exist, scan_bugs() returns early without resetting self.bugs_pending to zero, leaving a stale value."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:09:39.859966Z	3376
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:10:19.094956Z	5904

事件 1 — llm_497b467ee7954a80b5c1d17290c2a96a

► parsed_json（完整）

```
{
  "post_condition": "After execution, the attributes `self.bugs_confirmed`, `self.bugs_not_confirmed`, and `self.bugs_pending` are updated according to the following rules, and no other attributes are modified. Let `E` be `self.bug_dir.exists()` before the method. If `¬E`, then `self.bugs_confirmed = 0`, `self.bugs_not_confirmed = 0`, and `self.bugs_pending = max(0, bv_done)` where `bv_done = sum(self.stage_counts.get('bug_validation', {})).values()` (with original value). If `E`, define:\n\n- `F` = { p : p ∈ self.bug_dir.glob('*.result.json') } (the set of file paths matching the pattern)\n- `S` = { p ∈ F : opening and parsing `p` as JSON succeeds without raising any exception }\n- For each `p ∈ S`, let `status(p)` = `(d.get('confirmation_status') or '').lower()` where `d` is the parsed JSON object\n- `C` = { p ∈ S : 'confirm' ∈ status(p) ∧ 'not' ∉ status(p) }\n- `N` = `S \\ C`\n\nThen:\n`self.bugs_confirmed` = |C| \n `self.bugs_not_confirmed` = |N| \n `bv_done` = sum of values in `self.stage_counts.get('bug_validation', {})` (as it was at method entry) \n `self.bugs_pending` = max(0, bv_done - |S|) . \n\nThat is, `bugs_confirmed` counts parsed files whose lowercased confirmation status contains \"confirm\" but not \"not\"; `bugs_not_confirmed` counts the remaining parsed files (those containing \"not\" or neither string); `bugs_pending` is the maximum of zero and the difference between the sum of count values for the 'bug_validation' stage (or 0 if absent) and the total number of successfully parsed result files."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_497b467ee7954a80b5c1d17290c2a96a_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_497b467ee7954a80b5c1d17290c2a96a_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_497b467ee7954a80b5c1d17290c2a96a_response.txt)

事件 2 — llm_21cc2d66f96d4c0dbe1bc5044fc1923b

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "self.bug_dir does not exist (self.bug_dir.exists() returns False), self.stage_counts = {'bug_validation': {'completed': 3}}, bv_done = 3, and self.bugs_pending initially holds a value like 10. After scan_bugs() the specification requires self.bugs_pending to be 0, but the code (as described by A) sets it to max(0, bv_done) = 3 (or leaves it unchanged from its original value if the code is taken literally), which is non-zero and violates the requirement.",
  "offending_statements": "Line 4:         if not self.bug_dir.exists():\nLine 5:             return",
  "reason": "When the directory does not exist, the code returns early without setting self.bugs_pending to zero. Condition A (the actual behaviour) results in self.bugs_pending being max(0, bv_done) (or retains its previous value), whereas Condition B explicitly states that all three counters must be zero."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_21cc2d66f96d4c0dbe1bc5044fc1923b_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_21cc2d66f96d4c0dbe1bc5044fc1923b_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_21cc2d66f96d4c0dbe1bc5044fc1923b_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块：reasoner

FMA-MISMATCH-272 — src--reasoner-py--_compute_brace_depth_per_line

- 四类结论： 可能有 Bug
- 分类依据： 沿用旧 fm_agent/bug_list.md 的逐条审计结论： 沿用旧清单的逐条审计结论。
- Validator： confirmed; attempts 1
- Phase / 模块： Phase 6 — Reasoning & Verification Engine / reasoner; 原源码 src/reasoner.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim：** Returns a list of integers of the same length as lines. For each index i, the element at position i equals the net count of unmatched '{' characters minus unmatched '}' characters in the combined text of lines[0] through lines[i], where '{' and '}' occurring inside any double-quoted string literal, single-quoted character literal, line comment (from '/' to end of line), or block comment (between '/' and '/') are excluded from counting regardless of whether the comment spans multiple lines. The initial depth before the first line is 0.

- **Actual behavior:** The function returns a list 'depths' such that len(depths) == len(lines) and for each index i from 0 to len(lines)-1, depths[i] is the brace depth after processing lines[0..i] respecting the following rules: initial depth is 0; for each line, characters are scanned left to right, and any character that is part of a double-quoted string literal (starting and ending with unescaped '"', with backslash escape skipping the next character), a single-quoted character literal (similarly), a line comment (starting with '/' until end of line), or a block comment (starting with '/' and ending with '/' on the same line, or if not closed, the rest of the line) is ignored. Non-ignored '{' increments depth by 1, non-ignored '}' decrements depth by 1. depths[i] is the depth at the end of line i. Block comments that span lines are not recognized; each line is processed independently.
- **Code evidence:** Line 40: if ch == '/' and i + 1 < len(line) and line[i + 1] == ': Line 41: i += 2 Line 42: while i < len(line): Line 43: if line[i] == " and i + 1 < len(line) and line[i + 1] == ': Line 44: i += 2 Line 45: break Line 46: i += 1 Line 47: # If block comment spans lines, we ignore braces inside it (simplified) Line 48: continue
- **Trigger condition:** The code processes each line independently without carrying over the block comment state, so braces inside a multi-line block comment are not ignored, violating the specification that requires all braces inside block comments to be excluded regardless of spanning multiple lines.
- **Validator trigger:** Multi-line block comments are not tracked across lines, so braces inside /* ... */ spanning multiple lines are counted as real braces instead of being excluded.
- **Probe stdout:** CONFIRMED — actual: [0, 1, 1] | expected: [0, 0, 0]

► SPEC sidecar (完整)

```
{
  "signature": "_compute_brace_depth_per_line(lines: list[str]) -> list[int]",
  "pre_condition": "lines is a non-empty list of strings, each representing one line of source code.",
  "post_condition": "Returns a list of integers of the same length as lines. For each index i, the element at position i equals the net count of unmatched '{' characters minus unmatched '}' characters in the combined text of lines[0] through lines[i], where '{' and '}' occurring inside any double-quoted string literal, single-quoted character literal, line comment (from '/' to end of line), or block comment (between '/*' and '*/') are excluded from counting regardless of whether the comment spans multiple lines. The initial depth before the first line is 0."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/reasoner-py/_compute_brace_depth_per_line.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a list of integers of the same length as lines. For each index i, the element at position i equals the net count of unmatched '{' characters minus unmatched '}' characters in the combined text of lines[0] through lines[i], where '{' and '}' occurring inside any double-quoted string literal, single-quoted character literal, line comment (from '/' to end of line), or block comment (between '/*' and '*/') are excluded from counting regardless of whether the comment spans multiple lines. The initial depth before the first line is 0.",
    "actual_behavior": "The function returns a list 'depths' such that len(depths) == len(lines) and for each index i from 0 to len(lines)-1, depths[i] is the brace depth after processing lines[0..i] respecting the following rules: initial depth is 0; for each line, characters are scanned left to right, and any character that is part of a double-quoted string literal (starting and ending with unescaped '\\'', with backslash escape skipping the next character), a single-quoted character literal (similarly), a line comment (starting with '/' until end of line), or a block comment (starting with '/*' and ending with '*/' on the same line, or if not closed, the rest of the line) is ignored. Non-ignored '{' increments depth by 1, non-ignored '}' decrements depth by 1. depths[i] is the depth at the end of line i. Block comments that span lines are not recognized; each line is processed independently.",
    "code_evidence": "Line 40:           if ch == '/' and i + 1 < len(line) and line[i + 1] == '*':\nLine 41:                                     i\n+= 2\nLine 42:           while i < len(line):\nLine 43:                                     if line[i] == '*' and i + 1 < len(line) and\nline[i + 1] == '':\nLine 44:                                     i += 2\nLine 45:                                     break\nLine 46:                                     i += 1\nLine 47:                                     # If block comment spans lines, we ignore braces inside it (simplified)\nLine 48:                                     continue",
    "trigger_condition": "The code processes each line independently without carrying over the block comment state, so braces inside a multi-line block comment are not ignored, violating the specification that requires all braces inside block comments to be excluded regardless of spanning multiple lines."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--reasoner-py--_compute_brace_depth_per_line",
}
```

```
"source_file": "/home/fancy/Projects/Vault/FM-Agent/fm_agent/extracted_functions/src/reasoner-py/_compute_brace_depth_per_line.py",
"function_name": "_compute_brace_depth_per_line",
"confirmation_status": "confirmed",
"attempts": 1,
"probe_script": "fm_agent/bug_validation/probe_src--reasoner-py--_compute_brace_depth_per_line.py",
"detail_file": "fm_agent/bug_validation/src--reasoner-py--_compute_brace_depth_per_line.md",
"probe_stdout": "CONFIRMED - actual: [0, 1, 1] | expected: [0, 0, 0]\n",
"trigger_summary": "Multi-line block comments are not tracked across lines, so braces inside /* ... */ spanning multiple lines are counted as real braces instead of being excluded."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T05:12:59.374572Z	1981
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T05:13:20.996312Z	3037

事件 1 — [llm_dc3af6ed2a6b411a9d723bb546d1cb77](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a list 'depths' such that len(depths) == len(lines) and for each index i from 0 to len(lines)-1, depths[i] is the brace depth after processing lines[0..i] respecting the following rules: initial depth is 0; for each line, characters are scanned left to right, and any character that is part of a double-quoted string literal (starting and ending with unescaped '\"', with backslash escape skipping the next character), a single-quoted character literal (similarly), a line comment (starting with '//' until end of line), or a block comment (starting with '/*' and ending with '*/' on the same line, or if not closed, the rest of the line) is ignored. Non-ignored '{' increments depth by 1, non-ignored '}' decrements depth by 1. depths[i] is the depth at the end of line i. Block comments that span lines are not recognized; each line is processed independently."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_dc3af6ed2a6b411a9d723bb546d1cb77_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_dc3af6ed2a6b411a9d723bb546d1cb77_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_dc3af6ed2a6b411a9d723bb546d1cb77_response.txt)

事件 2 — [llm_ed01aca097ca4f3f836a0444a48d5190](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "lines = ['{', '/* {', '}', '*/', '}"]',
  "offending_statements": "Line 40:           if ch == '/' and i + 1 < len(line) and line[i + 1] == '*':\nLine 41:\ni += 2\nLine 42:           while i < len(line):\nLine 43:           if line[i] == '*' and i + 1 < len(line) and\nline[i + 1] == '/*':\nLine 44:           i += 2\nLine 45:           break\nLine 46:\ni += 1\nLine 47:           # If block comment spans lines, we ignore braces inside it (simplified)\nLine 48:           continue",
  "reason": "The code processes each line independently without carrying over the block comment state, so braces inside a multi-line block comment are not ignored, violating the specification that requires all braces inside block comments to be excluded regardless of spanning multiple lines."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ed01aca097ca4f3f836a0444a48d5190_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_ed01aca097ca4f3f836a0444a48d5190_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_ed01aca097ca4f3f836a0444a48d5190_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: scope_analysis

FMA-MISMATCH-278 — [src--scope-py--_collect_calls](#)

- 四类结论: 可能有 Bug
- 分类依据: probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: confirmed; attempts 1

• **Phase / 模块:** Phase 6 — Reasoning & Verification Engine / [scope_analysis](#); 原源码 [src/scope.py](#)

• **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** Returns a set of zero or more strings, each representing the syntactic name used at a call site within the function body. For a call to `f()`, the name `'f'` is collected. For a call to `x.f()`, the attribute name `'f'` is collected. Each unique call-site name appears at most once in the returned set.
- **Actual behavior:** The returned set `calls` contains exactly the strings gathered from the `func` attribute of every `ast.Call` node encountered during a walk of the AST rooted at `node` (including `node` itself and all sub-nodes). For each such call `c`, if `c.func` is an `ast.Name` node, its `id` string is added; if `c.func` is an `ast.Attribute` node, its `attr` string is added. No other call forms (e.g., `ast.Subscript`, `ast.Lambda`, `ast.Call`, etc.) contribute to the set. The set has no side effects, does not modify `node`, and contains no duplicates. Formally: `\result = { n.func.id | n \in ast.walk(node) \land isinstance(n, ast.Call) \land isinstance(n.func, ast.Name) } \cup { n.func.attr | n \in ast.walk(node) \land isinstance(n, ast.Call) \land isinstance(n.func, ast.Attribute) }`.
- **Code evidence:** Line 3: for `n` in `ast.walk(node)`:
- **Trigger condition:** The specification requires collecting only call-site names that occur within the function body. The code walks the entire `ast.FunctionDef/AsyncFunctionDef` node, which includes decorators (`decorator_list`). Calls inside decorators are not part of the function body but are nevertheless collected, adding extra names to the returned set.
- **Validator trigger:** `ast.walk()` traverses `decorator_list` of `FunctionDef` nodes, causing call-site names from decorator expressions to be incorrectly collected as if they were part of the function body.
- **Probe stdout:** CONFIRMED — actual: `{'bar', 'mydecorator'}` | expected: `{'bar'}`

► SPEC sidecar (完整)

```
{
  "signature": "_collect_calls(node: ast.FunctionDef | ast.AsyncFunctionDef) -> set[str]",
  "pre_condition": "node is a Python AST FunctionDef or AsyncFunctionDef node with a non-empty body subtree that can be traversed by ast.walk.",
  "post_condition": "Returns a set of zero or more strings, each representing the syntactic name used at a call site within the function body. For a call to f(), the name 'f' is collected. For a call to x.f(), the attribute name 'f' is collected. Each unique call-site name appears at most once in the returned set."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/scope-py/_collect_calls.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a set of zero or more strings, each representing the syntactic name used at a call site within the function body. For a call to f(), the name 'f' is collected. For a call to x.f(), the attribute name 'f' is collected. Each unique call-site name appears at most once in the returned set.",
    "actual_behavior": "The returned set `calls` contains exactly the strings gathered from the `func` attribute of every `ast.Call` node encountered during a walk of the AST rooted at `node` (including `node` itself and all sub-nodes). For each such call `c`, if `c.func` is an `ast.Name` node, its `id` string is added; if `c.func` is an `ast.Attribute` node, its `attr` string is added. No other call forms (e.g., `ast.Subscript`, `ast.Lambda`, `ast.Call`, etc.) contribute to the set. The set has no side effects, does not modify `node`, and contains no duplicates. Formally: `\\result = { n.func.id | n \\in ast.walk(node) \\land isinstance(n, ast.Call) \\land isinstance(n.func, ast.Name) } \\cup { n.func.attr | n \\in ast.walk(node) \\land isinstance(n, ast.Call) \\land isinstance(n.func, ast.Attribute) }`.",
    "code_evidence": "Line 3: for n in ast.walk(node):",
    "trigger_condition": "The specification requires collecting only call-site names that occur within the function body. The code walks the entire ast.FunctionDef/AsyncFunctionDef node, which includes decorators (decorator_list). Calls inside decorators are not part of the function body but are nevertheless collected, adding extra names to the returned set."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--scope-py--_collect_calls",
}
```

```
"source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/scope-py/_collect_calls.py",
"function_name": "_collect_calls",
"confirmation_status": "confirmed",
"attempts": 1,
"probe_script": "fm_agent/bug_validation/probe_src--scope-py--_collect_calls.py",
"detail_file": "fm_agent/bug_validation/src--scope-py--_collect_calls.md",
"probe_stdout": "CONFIRMED - actual: {'bar', 'mydecorator'} | expected: {'bar'} \n",
"trigger_summary": "`ast.walk()` traverses `decorator_list` of FunctionDef nodes, causing call-site names from decorator expressions to be incorrectly collected as if they were part of the function body."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T05:39:16.971257Z	1197
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T05:39:33.724717Z	3655

事件 1 — [llm_9a5b67e1358646818b84d380aca31957](#)

► parsed_json（完整）

```
{
  "post_condition": "The returned set `calls` contains exactly the strings gathered from the `func` attribute of every `ast.Call` node encountered during a walk of the AST rooted at `node` (including `node` itself and all sub-nodes). For each such call `c`, if `c.func` is an `ast.Name` node, its `id` string is added; if `c.func` is an `ast.Attribute` node, its `attr` string is added. No other call forms (e.g., `ast.Subscript`, `ast.Lambda`, `ast.Call`, etc.) contribute to the set. The set has no side effects, does not modify `node`, and contains no duplicates. Formally: `\\result = { n.func.id | n \\in ast.walk(node) \\land isinstance(n, ast.Call) \\land isinstance(n.func, ast.Name) } \\cup { n.func.attr | n \\in ast.walk(node) \\land isinstance(n, ast.Call) \\land isinstance(n.func, ast.Attribute) }`."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_9a5b67e1358646818b84d380aca31957_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_9a5b67e1358646818b84d380aca31957_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_9a5b67e1358646818b84d380aca31957_response.txt)

事件 2 — [llm_a36462f3fb8749c68039fe7dc066a7c7](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "A function definition with a decorator containing a call, for example:\n\n@decorator()\ndef foo():\n    pass\n\nWhen parsed, the resulting ast.FunctionDef node has a decorator_list with an ast.Call whose func is ast.Name(id='decorator').",
  "offending_statements": "Line 3: for n in ast.walk(node):",
  "reason": "The specification requires collecting only call-site names that occur within the function body. The code walks the entire ast.FunctionDef/AsyncFunctionDef node, which includes decorators (decorator_list). Calls inside decorators are not part of the function body but are nevertheless collected, adding extra names to the returned set."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_a36462f3fb8749c68039fe7dc066a7c7_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_a36462f3fb8749c68039fe7dc066a7c7_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_a36462f3fb8749c68039fe7dc066a7c7_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-284 — [src--scope-py--llm_rerank](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论；沿用旧清单的逐条审计结论。
- Validator：confirmed；attempts 1
- Phase / 模块：Phase 6 — Reasoning & Verification Engine / scope_analysis；原源码 src/scope.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** On success: returns a list of function name strings ordered by LLM-assessed relevance to issue, with length at most top_k and no duplicate names. Every returned name appears in funcs_info. On failure (the LLM response after 3 attempts is not a valid JSON array of non-empty function name strings): returns None. The initial LLM call includes the formatted function list, filepath, issue, and top_k; when the response is not a valid JSON array of non-empty strings, up to two additional retries are made with corrective instructions appended to the conversation, each spaced with increasing backoff delay. The function does not raise exceptions; all errors within attempts are caught and result in either a retry or None.
- **Actual behavior:** The function returns either a list of non-empty strings (function names) parsed from a valid JSON array in the LLM response, or None if all three attempts fail. Input arguments remain unchanged. All exceptions are caught internally and do not propagate. Side effects (logging, sleep, local messages list modifications) do not affect the caller. Formal post-condition: (return value = None) (return value is a list of strings s return value, isinstance(s, str) s.strip() != ""). The list may be empty. If returned, the list originates from the first successful LLM call within three attempts where the response contained a parseable JSON array of strings, each non-empty. Otherwise, after three failed attempts, None is returned.
- **Code evidence:** Line 29: names = _parse_json_response(text) Line 30: if not isinstance(names, list) or not all(Line 31: isinstance(name, str) and name.strip() for name in names Line 32:): Line 33: raise ValueError("LLM rerank response must be a JSON array of non-empty function names") Line 34: return [name.strip() for name in names]
- **Trigger condition:** The code only ensures the response is a JSON array of non-empty strings. It fails to check that the returned list has length top_k, contains no duplicates, or includes only names present in funcs_info, any of which would violate the specification.
- **Validator trigger:** Mock LLM returns 4 function names when top_k=2, exceeding the length constraint — the function returns all 4 without validation.
- **Probe stdout:** CONFIRMED — spec requires len <= 2, but got 4 names: ['foo', 'bar', 'baz', 'qux']

► SPEC sidecar (完整)

```
{
  "signature": "_llm_rerank(funcs_info: list[dict], source_lines: list[str], filepath: str, issue: str, top_k: int, llm_client: Any, model: str) -> list[str] | None",
  "pre_condition": "funcs_info is a non-empty list of function-metadata dicts each containing at least 'name' (string) and 'score' (float) keys, sorted by descending score. source_lines is the file content as a list of strings, one per line. filepath is a non-empty string identifying the source file. issue is a non-empty string describing the developer intent. top_k is a positive integer specifying the maximum number of functions to select. llm_client is a configured LLM API client whose chat.completions.create accepts model, messages, max_tokens, and temperature arguments. model is a non-empty string identifying the LLM model.",
  "post_condition": "On success: returns a list of function name strings ordered by LLM-assessed relevance to issue, with length at most top_k and no duplicate names. Every returned name appears in funcs_info. On failure (the LLM response after 3 attempts is not a valid JSON array of non-empty function name strings): returns None. The initial LLM call includes the formatted function list, filepath, issue, and top_k; when the response is not a valid JSON array of non-empty strings, up to two additional retries are made with corrective instructions appended to the conversation, each spaced with increasing backoff delay. The function does not raise exceptions; all errors within attempts are caught and result in either a retry or None."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_build_func_list_text",
      "signature": "_build_func_list_text(funcs_info: list[dict], source_lines: list[str]) -> str",
      "pre_condition": "funcs_info is a list of function-metadata dicts each containing at least 'name' and 'score' keys. source_lines is a list of strings representing the source file content, one per line.",
      "post_condition": "Returns a string representation of all functions in funcs_info suitable for embedding in an LLM prompt, including for each function: its name, the line range (start to end), and the body text excerpted from source_lines within that line range."
    },
    {
      "name": "_parse_json_response",
      "signature": "_parse_json_response(text: str) -> list[str]",
      "pre_condition": "text is a non-empty string potentially containing a JSON array of strings, possibly surrounded by markdown formatting or extra whitespace.",
      "post_condition": "Returns a list of non-empty strings parsed from the first JSON array found in text. Raises an exception when text does not contain a parseable JSON array of non-empty strings."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/scope-py/_llm_rerank.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "On success: returns a list of function name strings ordered by LLM-assessed relevance to issue, with length at
```


most top_k and no duplicate names. Every returned name appears in funcs_info. On failure (the LLM response after 3 attempts is not a valid JSON array of non-empty function name strings): returns None. The initial LLM call includes the formatted function list, filepath, issue, and top_k; when the response is not a valid JSON array of non-empty strings, up to two additional retries are made with corrective instructions appended to the conversation, each spaced with increasing backoff delay. The function does not raise exceptions; all errors within attempts are caught and result in either a retry or None.",

"actual_behavior": "The function returns either a list of non-empty strings (function names) parsed from a valid JSON array in the LLM response, or None if all three attempts fail. Input arguments remain unchanged. All exceptions are caught internally and do not propagate. Side effects (logging, sleep, local messages list modifications) do not affect the caller. Formal post-condition: (return value = None) (return value is a list of strings s return value, isinstance(s, str) s.strip() != ''). The list may be empty. If returned, the list originates from the first successful LLM call within three attempts where the response contained a parseable JSON array of strings, each non-empty. Otherwise, after three failed attempts, None is returned.",

"code_evidence": "Line 29: names = _parse_json_response(text)\nLine 30: if not isinstance(names, list) or not all(\nLine 31: isinstance(name, str) and name.strip() for name in names\nLine 32:):\nLine 33: raise ValueError(\"LLM rerank response must be a JSON array of non-empty function names\")\nLine 34: return [name.strip() for name in names]",

"trigger_condition": "The code only ensures the response is a JSON array of non-empty strings. It fails to check that the returned list has length top_k, contains no duplicates, or includes only names present in funcs_info, any of which would violate the specification."

```
}
}
```

► Bug Validator result (完整)

```
{
  "id": "src--scope-py--_llm_rerank",
  "source_file": "src/scope.py",
  "function_name": "_llm_rerank",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--scope-py--_llm_rerank.py",
  "detail_file": "fm_agent/bug_validation/src--scope-py--_llm_rerank.md",
  "probe_stdout": "CONFIRMED - spec requires len <= 2, but got 4 names: ['foo', 'bar', 'baz', 'qux']",
  "trigger_summary": "Mock LLM returns 4 function names when top_k=2, exceeding the length constraint - the function returns all 4 without validation."
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:46:20.815655Z	2448
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:46:48.139364Z	2696

事件 1 — [llm_d05b76387aee4dfcb0c6d76d3d40ad8a](#)

► parsed_json (完整)

```
{
  "post_condition": "The function returns either a list of non-empty strings (function names) parsed from a valid JSON array in the LLM response, or None if all three attempts fail. Input arguments remain unchanged. All exceptions are caught internally and do not propagate. Side effects (logging, sleep, local messages list modifications) do not affect the caller. Formal post-condition: (return value = None) v (return value is a list of strings ^ V s ∈ return value, isinstance(s, str) ^ s.strip() != ''). The list may be empty. If returned, the list originates from the first successful LLM call within three attempts where the response contained a parseable JSON array of strings, each non-empty. Otherwise, after three failed attempts, None is returned."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_d05b76387aee4dfcb0c6d76d3d40ad8a_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_d05b76387aee4dfcb0c6d76d3d40ad8a_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_d05b76387aee4dfcb0c6d76d3d40ad8a_response.txt)

事件 2 — [llm_f2f94e5a9ba64a42bf35ac0a0c27225a](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "Input: funcs_info = [{'name': 'foo'}, {'name': 'bar'}], top_k = 1, with a valid LLM response that parses to ['foo', 'bar']. The function returns ['foo', 'bar'] whereas the specification requires at most 1 name.",
  "offending_statements": "Line 29: names = _parse_json_response(text)\nLine 30: if not isinstance(names, list) or not all(\nLine 31:     isinstance(name, str) and name.strip() for name in names\nLine 32: ):\nLine 33:     raise ValueError(\"LLM rerank response must be a JSON array of non-empty function names\")\nLine 34: return [name.strip() for name in names]",
  "reason": "The code only ensures the response is a JSON array of non-empty strings. It fails to check that the returned list has"
```

```
length ≤ top_k, contains no duplicates, or includes only names present in funcs_info, any of which would violate the specification."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f2f94e5a9ba64a42bf35ac0a0c27225a_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f2f94e5a9ba64a42bf35ac0a0c27225a_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f2f94e5a9ba64a42bf35ac0a0c27225a_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-288 — [src--scope.py--_parse_issue_signals](#)

- 四类结论：可能有 **Bug**
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [scope_analysis](#); 原源码 [src/scope.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a dict with seven fixed keys 'traceback_funcs', 'backtick_idents', 'dotted_refs', 'dotted_classes', 'plain_idents', 'exception_types', 'all_words' each mapping to a set of lowercase strings. All extracted tokens are lowercased.

'traceback_funcs': function-name tokens appearing after the word "in" and immediately before a newline (Python traceback pattern).

'backtick_idents': identifier tokens extracted from backtick-quoted spans and fenced code blocks in the text, excluding programming-language keywords.

'dotted_refs': method-name tokens derived from Class.method references where the class part starts with an uppercase letter followed by alphanumerics and the method part starts with a lowercase letter or underscore followed by alphanumerics/underscores. The method name is lowercased and split on underscores; each constituent part of length greater than 1 is also included.

'dotted_classes': the class-name tokens from those same Class.method references, lowercased.

'plain_idents': CamelCase and snake_case identifier tokens appearing in prose text contiguous alphanumeric/underscore tokens that contain at least one uppercase letter or underscore transition beyond the first character. Each match is lowercased and decomposed into constituent parts on CamelCase boundaries and underscores; parts of length 3 or greater are included.

'exception_types': name tokens starting with an uppercase letter, consisting entirely of letters, and ending with one of the suffixes 'Error', 'Exception', or 'Warning'.

'all_words': every distinct alphabetic word of 4 or more characters in the lowercased text that is not a member of a predefined stop-word set.

- **Actual behavior**: The function returns a dictionary with keys: traceback_funcs, backtick_idents, dotted_refs, dotted_classes, plain_idents, exception_types, all_words. It raises no exceptions and always returns normally. The dictionary satisfies the following detailed postconditions:

traceback_funcs: set of words captured by the pattern 'in ' followed by optional whitespace and a newline in issue_text. The words retain their original case.

backtick_idents: the result of `_extract_backtick_idents(issue_text)`, a set of identifier strings from backtick-quoted or fenced code blocks, with programming keywords removed; case as in the original text.

dotted_refs: union of (a) all method names from matches of the pattern Class.method (where Class starts with uppercase letter, method starts with lowercase/underscore) after lowercasing; (b) any underscore-delimited parts of those method names that have length greater than 1.

dotted_classes: set of class names from those Class.method matches, lowercased.

plain_idents: union of (a) all words in issue_text that match the pattern for CamelCase or snake_case identifiers (i.e., words containing at least one uppercase letter or underscore), lowercased; (b) any parts obtained by inserting underscore before each uppercase letter in such a word, lowercasing, stripping leading/trailing underscores, splitting on underscore, and filtering for parts longer than 2 characters.

exception_types: set of words matching the pattern of exception names (capitalized word ending in Error, Exception, or Warning), lowercased.

all_words: set of alphabetic words of length ≥ 4 in issue_text (case-insensitive) that are not in the predefined stop word set `_STOP`.

All values are sets of strings; any set may be empty. No external side effects.

- **Code evidence:** Line 13: `signals['traceback_funcs'] = set(re.findall(r'\bin (\w+)\s*\n', issue_text))`
- **Trigger condition:** The `traceback_funcs` set is not lowercased, but the specification requires all extracted tokens to be lowercased. For input `'in MyFunc\n'`, the code returns `{'MyFunc'}` while the specification requires `{'myfunc'}`.
- **Validator trigger:** `traceback_funcs` set is not lowercased: input `'in MyFunc\n'` returns `{'MyFunc'}` instead of spec-required `{'myfunc'}`
- **Probe stdout:** CONFIRMED — actual: `{'MyFunc'}` | expected: `{'myfunc'}`

► SPEC sidecar (完整)

```
{
  "signature": "_parse_issue_signals(issue_text: str) -> dict[str, set[str]]",
  "pre_condition": "issue_text is a non-empty string describing developer intent for a code modification.",
  "post_condition": "Returns a dict with seven fixed keys – 'traceback_funcs', 'backtick_idsents', 'dotted_refs', 'dotted_classes', 'plain_idsents', 'exception_types', 'all_words' – each mapping to a set of lowercase strings. All extracted tokens are lowercased.\n\n'traceback_funcs': function-name tokens appearing after the word 'in' and immediately before a newline (Python traceback pattern).\n\n'backtick_idsents': identifier tokens extracted from backtick-quoted spans and fenced code blocks in the text, excluding programming-language keywords.\n\n'dotted_refs': method-name tokens derived from Class.method references where the class part starts with an uppercase letter followed by alphanumerics and the method part starts with a lowercase letter or underscore followed by alphanumerics/underscores. The method name is lowercased and split on underscores; each constituent part of length greater than 1 is also included.\n\n'dotted_classes': the class-name tokens from those same Class.method references, lowercased.\n\n'plain_idsents': CamelCase and snake_case identifier tokens appearing in prose text – contiguous alphanumeric/underscore tokens that contain at least one uppercase letter or underscore transition beyond the first character. Each match is lowercased and decomposed into constituent parts on CamelCase boundaries and underscores; parts of length 3 or greater are included.\n\n'exception_types': name tokens starting with an uppercase letter, consisting entirely of letters, and ending with one of the suffixes 'Error', 'Exception', or 'Warning'.\n\n'all_words': every distinct alphabetic word of 4 or more characters in the lowercased text that is not a member of a predefined stop-word set."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_extract_backtick_idsents",
      "signature": "_extract_backtick_idsents(issue_text: str) -> set[str]",
      "pre_condition": "issue_text is a non-empty string.",
      "post_condition": "Returns a set of identifier strings extracted from backtick-quoted spans and fenced code blocks within issue_text, with programming-language keywords filtered out. Each returned string is a valid identifier token from the text."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/scope-py/_parse_issue_signals.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a dict with seven fixed keys 'traceback_funcs', 'backtick_idsents', 'dotted_refs', 'dotted_classes', 'plain_idsents', 'exception_types', 'all_words' each mapping to a set of lowercase strings. All extracted tokens are lowercased.\n\n'traceback_funcs': function-name tokens appearing after the word 'in' and immediately before a newline (Python traceback pattern).\n\n'backtick_idsents': identifier tokens extracted from backtick-quoted spans and fenced code blocks in the text, excluding programming-language keywords.\n\n'dotted_refs': method-name tokens derived from Class.method references where the class part starts with an uppercase letter followed by alphanumerics and the method part starts with a lowercase letter or underscore followed by alphanumerics/underscores. The method name is lowercased and split on underscores; each constituent part of length greater than 1 is also included.\n\n'dotted_classes': the class-name tokens from those same Class.method references, lowercased.\n\n'plain_idsents': CamelCase and snake_case identifier tokens appearing in prose text contiguous alphanumeric/underscore tokens that contain at least one uppercase letter or underscore transition beyond the first character. Each match is lowercased and decomposed into constituent parts on CamelCase boundaries and underscores; parts of length 3 or greater are included.\n\n'exception_types': name tokens starting with an uppercase letter, consisting entirely of letters, and ending with one of the suffixes 'Error', 'Exception', or 'Warning'.\n\n'all_words': every distinct alphabetic word of 4 or more characters in the lowercased text that is not a member of a predefined stop-word set.",
    "actual_behavior": "The function returns a dictionary with keys: traceback_funcs, backtick_idsents, dotted_refs, dotted_classes, plain_idsents, exception_types, all_words. It raises no exceptions and always returns normally. The dictionary satisfies the following detailed postconditions:\n\n- traceback_funcs: set of words captured by the pattern 'in <word>' followed by optional whitespace and a newline in issue_text. The words retain their original case.\n- backtick_idsents: the result of _extract_backtick_idsents(issue_text), a set of identifier strings from backtick-quoted or fenced code blocks, with programming keywords removed; case as in the original text.\n- dotted_refs: union of (a) all method names from matches of the pattern Class.method (where Class starts with uppercase letter, method starts with lowercase/underscore) after lowercasing; (b) any underscore-delimited parts of those method names that have length greater than 1.\n- dotted_classes: set of class names from those Class.method matches, lowercased.\n- plain_idsents: union of (a) all words in issue_text that match the pattern for CamelCase or snake_case identifiers (i.e., words containing at least one uppercase letter or underscore), lowercased; (b) any parts obtained by inserting underscore before each uppercase letter in such a word, lowercasing, stripping leading/trailing underscores, splitting on underscore, and filtering for parts longer than 2 characters.\n- exception_types: set of words matching the pattern of exception names (capitalized word ending in Error, Exception, or Warning), lowercased.\n- all_words: set of alphabetic words of length >= 4 in issue_text (case-insensitive) that are not in the predefined stop word set _STOP.\n\nAll values are sets of strings; any set may be empty. No external side effects.",
    "code_evidence": "Line 13: signals['traceback_funcs'] = set(re.findall(r'\\bin (\\w+)\\s*\\n', issue_text))",
  }
}
```

```
"trigger_condition": "The traceback_funcs set is not lowercased, but the specification requires all extracted tokens to be lowercased. For input 'in MyFunc\\n', the code returns {'MyFunc'} while the specification requires {'myfunc'}."
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--scope-py--_parse_issue_signals",
  "source_file": "src/scope-py/_parse_issue_signals.py",
  "function_name": "_parse_issue_signals",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--scope-py--_parse_issue_signals.py",
  "detail_file": "fm_agent/bug_validation/src--scope-py--_parse_issue_signals.md",
  "probe_stdout": "CONFIRMED - actual: {'MyFunc'} | expected: {'myfunc'}\\n",
  "trigger_summary": "traceback_funcs set is not lowercased: input 'in MyFunc\\n' returns {'MyFunc'} instead of spec-required {'myfunc'}"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:28:08.880585Z	7111
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:29:44.638014Z	3801

事件 1 — [llm_2de7b64746fa4111919e32819662199d](#)

► parsed_json（完整）

```
{
  "post_condition": "The function returns a dictionary with keys: traceback_funcs, backtick_idsents, dotted_refs, dotted_classes, plain_idsents, exception_types, all_words. It raises no exceptions and always returns normally. The dictionary satisfies the following detailed postconditions:\\n\\n- traceback_funcs: set of words captured by the pattern 'in <word>' followed by optional whitespace and a newline in issue_text. The words retain their original case.\\n- backtick_idsents: the result of _extract_backtick_idsents(issue_text), a set of identifier strings from backtick-quoted or fenced code blocks, with programming keywords removed; case as in the original text.\\n- dotted_refs: union of (a) all method names from matches of the pattern Class.method (where Class starts with uppercase letter, method starts with lowercase/underscore) after lowercasing; (b) any underscore-delimited parts of those method names that have length greater than 1.\\n- dotted_classes: set of class names from those Class.method matches, lowercased.\\n- plain_idsents: union of (a) all words in issue_text that match the pattern for CamelCase or snake_case identifiers (i.e., words containing at least one uppercase letter or underscore), lowercased; (b) any parts obtained by inserting underscore before each uppercase letter in such a word, lowercasing, stripping leading/trailing underscores, splitting on underscore, and filtering for parts longer than 2 characters.\\n- exception_types: set of words matching the pattern of exception names (capitalized word ending in Error, Exception, or Warning), lowercased.\\n- all_words: set of alphabetic words of length >= 4 in issue_text (case-insensitive) that are not in the predefined stop word set _STOP.\\n\\nAll values are sets of strings; any set may be empty. No external side effects."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_2de7b64746fa4111919e32819662199d_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_2de7b64746fa4111919e32819662199d_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_2de7b64746fa4111919e32819662199d_response.txt)

事件 2 — [llm_b8da25c5e9a44a66a55a006c92630277](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "issue_text = 'in MyFunc\\n'",
  "offending_statements": "Line 13: signals['traceback_funcs'] = set(re.findall(r'\\bin (\\w+)\\s*\\n', issue_text))",
  "reason": "The traceback_funcs set is not lowercased, but the specification requires all extracted tokens to be lowercased. For input 'in MyFunc\\n', the code returns {'MyFunc'} while the specification requires {'myfunc'}."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_b8da25c5e9a44a66a55a006c92630277_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_b8da25c5e9a44a66a55a006c92630277_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_b8da25c5e9a44a66a55a006c92630277_response.txt) **复核动作**

► Logic verification result (完整)


```
by lang_key. The returned regex captures the function name in group 1 and ensures an opening parenthesis follows. The exact pattern depends on lang_key: for 'cpp', 'c', 'java', 'typescript', 'javascript', 'cuda', 'arkts' it includes optional template arguments '<...>'; for 'rust' it includes optional turbofish `::<...>`; for 'go' it includes optional type parameters `[...]`; and for any other lang_key (e.g., 'python', 'ruby') it matches a plain identifier followed by optional whitespace and '('. Formally: let L = {'cpp', 'c', 'java', 'typescript', 'javascript', 'cuda', 'arkts'}, then (lang_key ∈ L) ⇒ return = re.compile(r"\\b(\\w+)\\s*(?:<[>]*>)?\\s*\\(") (lang_key = 'rust') ⇒ return = re.compile(r"\\b(\\w+)\\s*(?:::<[>]*>)?\\s*\\(") (lang_key = 'go') ⇒ return = re.compile(r"\\b(\\w+)\\s*(?:\\[\\[\\]\\]\\s*)?\\s*\\(") (lang_key ∉ L ∪ {'rust', 'go'}) ⇒ return = re.compile(r"\\b(\\w+)\\s*\\(").",
    "code_evidence": "Line 5: return re.compile(r\"\\b(\\w+)\\s*(?:<[>]*>)?\\s*\\(\\\")\\nLine 8: return re.compile(r\"\\b(\\w+)\\s*(?:::<[>]*>)?\\s*\\(\\\")\\nLine 11: return re.compile(r\"\\b(\\w+)\\s*(?:\\[\\[\\]\\]\\s*)?\\s*\\(\\\")",
    "trigger_condition": "The regex patterns use [>]* and [\\[\\]]* which do not allow nested brackets typical of template/generic arguments in C++, Java, Rust, Go, etc., thus failing to match valid call sites with nested type arguments like foo<A<B>>(x)."
```

► Bug Validator result (完整)

```
{
  "id": "src--generate_topdown_layers-py--_get_call_regex",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/generate_topdown_layers-py/_get_call_regex.py",
  "function_name": "_get_call_regex",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--generate_topdown_layers-py--_get_call_regex.py",
  "detail_file": "fm_agent/bug_validation/src--generate_topdown_layers-py--_get_call_regex.md",
  "probe_stdout": "CONFIRMED - nested bracket regex bug reproduced (3 mismatches)\n [cpp] cpp nested template args: input='foo<A<B>>(x)' expected='foo' actual=None\n [rust] rust nested turbofish: input='bar::<Vec<u8>>(x)' expected='bar' actual=None\n [go] go nested type params: input='fn[map[string]int](x)' expected='fn' actual=None\n positive control (func<int>(x)) matched correctly",
  "trigger_summary": "Regex patterns use [>]* and [\\[\\]]* which fail on nested brackets in template/generic arguments (e.g., foo<A<B>>(x))"
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T05:57:04.200840Z	1200
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T05:57:16.922959Z	4026

事件 1 — [llm_f12643103eb942e0b6992f69407ead37](#)

► parsed_json (完整)

```
{
  "post_condition": "Returns a compiled regular expression (re.Pattern) for matching function call sites in the language given by lang_key. The returned regex captures the function name in group 1 and ensures an opening parenthesis follows. The exact pattern depends on lang_key: for 'cpp', 'c', 'java', 'typescript', 'javascript', 'cuda', 'arkts' it includes optional template arguments '<...>'; for 'rust' it includes optional turbofish `::<...>`; for 'go' it includes optional type parameters `[...]`; and for any other lang_key (e.g., 'python', 'ruby') it matches a plain identifier followed by optional whitespace and '('. Formally: let L = {'cpp', 'c', 'java', 'typescript', 'javascript', 'cuda', 'arkts'}, then (lang_key ∈ L) ⇒ return = re.compile(r"\\b(\\w+)\\s*(?:<[>]*>)?\\s*\\(") (lang_key = 'rust') ⇒ return = re.compile(r"\\b(\\w+)\\s*(?:::<[>]*>)?\\s*\\(") (lang_key = 'go') ⇒ return = re.compile(r"\\b(\\w+)\\s*(?:\\[\\[\\]\\]\\s*)?\\s*\\(") (lang_key ∉ L ∪ {'rust', 'go'}) ⇒ return = re.compile(r"\\b(\\w+)\\s*\\(")."
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f12643103eb942e0b6992f69407ead37_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f12643103eb942e0b6992f69407ead37_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f12643103eb942e0b6992f69407ead37_response.txt)

事件 2 — [llm_5bc427f2a1fb428892306cc1b26a2ac6](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "lang_key='cpp', source='foo<A<B>>(x)'",
  "offending_statements": "Line 5: return re.compile(r\"\\b(\\w+)\\s*(?:<[>]*>)?\\s*\\(\\\")\\nLine 8: return re.compile(r\"\\b(\\w+)\\s*(?:::<[>]*>)?\\s*\\(\\\")\\nLine 11: return re.compile(r\"\\b(\\w+)\\s*(?:\\[\\[\\]\\]\\s*)?\\s*\\(\\\")",
  "reason": "The regex patterns use [>]* and [\\[\\]]* which do not allow nested brackets typical of template/generic arguments in
```



```
C++, Java, Rust, Go, etc., thus failing to match valid call sites with nested type arguments like foo<A<B>>(x)."
```

```
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_5bc427f2a1fb428892306cc1b26a2ac6_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_5bc427f2a1fb428892306cc1b26a2ac6_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_5bc427f2a1fb428892306cc1b26a2ac6_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-303 — [src--generate_topdown_layers-py--_load_phases](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / [topdown_layers](#); 原源码 [src/generate_topdown_layers.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: Returns a Python dict representing the full parsed JSON content of proj_dir/phases.json, preserving all keys and nested values from the file. Raises a FileNotFoundError or OSError when the file does not exist or cannot be read. Raises a json.JSONDecodeError or ValueError when the file content is not structurally valid JSON. Does not return a partial or empty dict on failure every execution path either returns the complete parsed content or raises an error.
- **Actual behavior**: After execution, the function returns the parsed JSON object from the file 'phases.json' located at proj_dir. The return value is a Python dictionary that contains the key 'phases'. The file handle is closed. No exceptions are raised under the given pre-condition. Formally: (result = _load_phases(proj_dir)) (isinstance(result, dict) 'phases' result.keys())
- **Code evidence**: Line 5: return json.load(f)
- **Trigger condition**: Specification requires the function to "Return a Python dict representing the full parsed JSON content". When the file contains a JSON array, the code returns a Python list, not a dict, violating the specification. For example, if phases.json holds '[1,2,3]', the output is [1,2,3] which is not a dict.
- **Validator trigger**: When phases.json contains a JSON array (e.g. [1,2,3]), json.load() returns a list instead of a dict, violating the spec that requires a dict return.
- **Probe stdout**: CONFIRMED — actual: list [1, 2, 3] | spec requires: dict

► SPEC sidecar (完整)

```
{
  "signature": "_load_phases(proj_dir) -> dict",
  "pre_condition": "proj_dir is a string path to an existing directory. The file `phases.json` at proj_dir/phases.json exists, is readable, and contains valid JSON whose root value is a dict that includes a `phases` key.",
  "post_condition": "Returns a Python dict representing the full parsed JSON content of proj_dir/phases.json, preserving all keys and nested values from the file. Raises a FileNotFoundError or OSError when the file does not exist or cannot be read. Raises a json.JSONDecodeError or ValueError when the file content is not structurally valid JSON. Does not return a partial or empty dict on failure – every execution path either returns the complete parsed content or raises an error."
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/generate_topdown_layers-py/_load_phases.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "Returns a Python dict representing the full parsed JSON content of proj_dir/phases.json, preserving all keys and nested values from the file. Raises a FileNotFoundError or OSError when the file does not exist or cannot be read. Raises a json.JSONDecodeError or ValueError when the file content is not structurally valid JSON. Does not return a partial or empty dict on failure every execution path either returns the complete parsed content or raises an error.",
    "actual_behavior": "After execution, the function returns the parsed JSON object from the file 'phases.json' located at proj_dir. The return value is a Python dictionary that contains the key 'phases'. The file handle is closed. No exceptions are raised under the given pre-condition. Formally: (result = _load_phases(proj_dir)) (isinstance(result, dict) 'phases' result.keys())",
  }
}
```

```
"code_evidence": "Line 5: return json.load(f)",
"trigger_condition": "Specification requires the function to \"Return a Python dict representing the full parsed JSON content\". When the file contains a JSON array, the code returns a Python list, not a dict, violating the specification. For example, if phases.json holds '[1,2,3]', the output is [1,2,3] which is not a dict."
}
```

► Bug Validator result（完整）

```
{
  "id": "src--generate_topdown_layers-py--_load_phases",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/generate_topdown_layers-py/_load_phases.py",
  "function_name": "_load_phases",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--generate_topdown_layers-py--_load_phases.py",
  "detail_file": "fm_agent/bug_validation/src--generate_topdown_layers-py--_load_phases.md",
  "probe_stdout": "CONFIRMED – actual: list [1, 2, 3] | spec requires: dict",
  "trigger_summary": "When phases.json contains a JSON array (e.g. [1,2,3]), json.load() returns a list instead of a dict, violating the spec that requires a dict return."
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T05:38:46.924472Z	1004
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T05:39:02.210362Z	9979

事件 1 — [llm_c6ebb2cecabd4d44b72110989d7762bd](#)

► parsed_json（完整）

```
{
  "post_condition": "After execution, the function returns the parsed JSON object from the file 'phases.json' located at proj_dir. The return value is a Python dictionary that contains the key 'phases'. The file handle is closed. No exceptions are raised under the given pre-condition. Formally: (result = _load_phases(proj_dir)) => (isinstance(result, dict) ^ 'phases' ∈ result.keys())"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_c6ebb2cecabd4d44b72110989d7762bd_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_c6ebb2cecabd4d44b72110989d7762bd_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_c6ebb2cecabd4d44b72110989d7762bd_response.txt)

事件 2 — [llm_0705a7e6cdda4e49a113160120ba182a](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/tmp/test' with file '/tmp/test/phases.json' containing the JSON array [1,2,3]",
  "offending_statements": "Line 5: return json.load(f)",
  "reason": "Specification requires the function to \"Return a Python dict representing the full parsed JSON content\". When the file contains a JSON array, the code returns a Python list, not a dict, violating the specification. For example, if phases.json holds '[1,2,3]', the output is [1,2,3] which is not a dict."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_0705a7e6cdda4e49a113160120ba182a_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_0705a7e6cdda4e49a113160120ba182a_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_0705a7e6cdda4e49a113160120ba182a_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块：verification

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**： [not_confirmed](#)； [attempts 1](#)
- **Phase / 模块**： Phase 6 — Reasoning & Verification Engine / [verification](#)； 原源码 [src/verification.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**： When a result marker for the discrepancy corresponding to `result_json_rel` does not already exist at `<work_dir>/bug_validation/<bug_id>.result.json`, or when `resume` is `False`, a bug-validation agent is dispatched to evaluate whether the spec-code discrepancy recorded in `result_json_rel` constitutes a real, exploitable bug. A validated bug report is produced at `<work_dir>/bug_validation/<bug_id>.md` if and only if the discrepancy is confirmed to be an exploitable bug. The bug report contains: the behavioral assertion claimed by the specification that the code violates, the actual behavior observed from the code, concrete code evidence with line references, the condition under which the violation is triggered, step-by-step reproduction instructions, a probe script exercising the violation, and the raw output from executing the probe script. A result marker file is written to `<work_dir>/bug_validation/<bug_id>.result.json` upon completion of validation. When `resume` is `True` and the result marker already exists with valid, parseable JSON content, no re-validation occurs and no side effects are produced. The `bug_id` is uniquely derived from `result_json_rel` by stripping the standard verification-results path prefix and canonicalizing directory separators.
- **Actual behavior**： After the code block, one of three scenarios holds: (1) An unhandled exception was raised (e.g., from `os.makedirs`, file writing, `os.replace`, or `build_llm_cli_command`). In this case the program may have aborted with only partial side effects; no further guarantees are made about variables after the exception point. (2) No exception occurred and the function returned early at line 73 because `resume` was truthy, `result_path` existed, and the file could be successfully read and parsed as JSON. Then the following are true: the directory `os.path.join(work_dir, 'bug_validation')` exists; the file at `prompt_path` exists and its entire content equals `prompt_content`; the temporary file `tmp_path` no longer exists; the following variables are defined with values as given in the code: `prompt_content = '# Bug Validator\n\nTarget result file: {result_json_rel}\nBug ID: {bug_id}\n\n--\n\n'` + `user_knowledge_section` + `base_content`, `prompt_filename = 'fm_agent/bug_validation/bug_validator_{bug_id}.md'`, `prompt_path = os.path.join(proj_dir, prompt_filename)`, `tmp_path = prompt_path + '.tmp'`, `prompt = 'Follow the instructions in the attached file'`, `command = build_llm_cli_command(OPENCODE_BUG_VALIDATION_MODEL, prompt, cwd=proj_dir, files=[prompt_path])`, `result_relpth = 'fm_agent/bug_validation/{bug_id}.result.json'`, `result_path = os.path.join(proj_dir, result_relpth)`; the function returns normally. (3) No exception occurred and no early return (either `resume` is falsy, `result_path` does not exist, or JSON parsing raised an exception). Then the same directory and file side effects hold as in scenario (2), the same variable assignments hold as in scenario (2), and additionally: `max_attempts = config.BUG_VALIDATION_MAX_RETRIES`; the for loop for attempt in `range(1, max_attempts + 1)` has been entered and the first iteration began with `attempt = 1` and `run_failed = False`; the try block at line 80 has been entered but no inner statements have executed yet. Formally: `(exception_raised_at_line(l) | {49,55,56,57,59,60,61,62,63,64,65,66,68,69,70,71})` `(exception_early_return prompt_path_exists file_content(prompt_path) = prompt_content (v {prompt_content, prompt_filename, prompt_path, tmp_path, prompt, command, result_relpth, result_path} : assigned(v) = value_from_code) return_executed)` `(exception_early_return same_effects_as_above max_attempts = config.BUG_VALIDATION_MAX_RETRIES attempt = 1 run_failed = False)`. Here `early_return` is defined as `resume file_exists(result_path) json_load_succeeded(result_path)`.
- **Code evidence**： Line 41: through Line 80: The code block only performs prompt setup and a skip check; it never dispatches the bug-validation agent, never writes the validated bug report to `<work_dir>/bug_validation/<bug_id>.md`, and never writes the result marker file `<work_dir>/bug_validation/<bug_id>.result.json` upon completion of validation as required by the specification.
- **Trigger condition**： The specification requires that when `resume` is `False` or the result marker does not exist, a bug-validation agent is dispatched, leading to production of a bug report and a result marker. Condition A describes poststates where either an exception occurs (no dispatch) or the function returns early (skip) or enters a retry loop without executing any dispatch. In none of these scenarios does the code perform the mandated side effects.
- **Validator trigger**： When `resume=False` and no result marker exists, the function correctly dispatches a bug-validation agent via `run_opencode_traced` — the verification was a false positive caused by incomplete control-flow analysis stopping at line 80.
- **Probe stdout**： NOT CONFIRMED — actual: `'dispatch called' | expected: 'dispatch called' | output_files=['fm_agent/bug_validation/src--verification-py--_validate_single_bug.md', 'fm_agent/bug_validation/src--verification-py--_validate_single_bug.result.json'] | stage=bug_validation | summary=OpenCode bug validation for src--verification-py--_validate_single_bug\n`

► SPEC sidecar（完整）

```
{
  "signature": "_validate_single_bug(result_json_rel, proj_dir, work_dir=None, resume=False, bug_validator_path=None)",
  "pre_condition": "result_json_rel is a non-empty string identifying a relative path from proj_dir to an existing JSON file containing a reasoning verdict with at minimum a 'verdict' field. proj_dir is an existing directory. If provided, bug_validator_path is an absolute or relative path to an existing file whose content specifies custom bug-validation instructions. If work_dir is None, it defaults to proj_dir.",
  "post_condition": "When a result marker for the discrepancy corresponding to result_json_rel does not already exist at <work_dir>/bug_validation/<bug_id>.result.json, or when resume is False, a bug-validation agent is dispatched to evaluate whether the spec-code discrepancy recorded in result_json_rel constitutes a real, exploitable bug. A validated bug report is produced at
```

```
<work_dir>/bug_validation/<bug_id>.md if and only if the discrepancy is confirmed to be an exploitable bug. The bug report contains: the behavioral assertion claimed by the specification that the code violates, the actual behavior observed from the code, concrete code evidence with line references, the condition under which the violation is triggered, step-by-step reproduction instructions, a probe script exercising the violation, and the raw output from executing the probe script. A result marker file is written to <work_dir>/bug_validation/<bug_id>.result.json upon completion of validation. When resume is True and the result marker already exists with valid, parseable JSON content, no re-validation occurs and no side effects are produced. The bug_id is uniquely derived from result_json_rel by stripping the standard verification-results path prefix and canonicalizing directory separators."
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "function_id_from_result_path",
      "signature": "function_id_from_result_path(result_json_rel) -> str",
      "pre_condition": "result_json_rel is a non-empty string representing a path to a reasoning verdict JSON file.",
      "post_condition": "Returns a function identifier string derived from the result path, suitable for correlating verification results with their source function."
    },
    {
      "name": "list_staged_domain_knowledge_relpaths",
      "signature": "list_staged_domain_knowledge_relpaths(work_dir) -> list[str]",
      "pre_condition": "work_dir is the FM-Agent work directory that may contain staged domain knowledge files.",
      "post_condition": "Returns a list of relative paths (from work_dir) to all user-provided domain knowledge markdown files currently staged in the work directory. Returns an empty list when no domain knowledge files are staged."
    },
    {
      "name": "format_domain_knowledge_bullets",
      "signature": "format_domain_knowledge_bullets(paths) -> str",
      "pre_condition": "paths is a list of relative file path strings.",
      "post_condition": "Returns a markdown-formatted string containing a header section and a bullet list of the provided paths, suitable for embedding into a prompt as context references. When paths is empty, returns an empty string."
    },
    {
      "name": "build_llm_cli_command",
      "signature": "build_llm_cli_command(model, prompt, cwd, files) -> list[str]",
      "pre_condition": "model is a non-empty string identifying the LLM model. prompt is a non-empty string. cwd is an existing directory. files is a list of file paths to attach.",
      "post_condition": "Returns a list of CLI argument strings that, when executed as a subprocess, invokes the configured LLM backend with the given model, prompt, working directory, and attached files."
    },
    {
      "name": "run_opencode_traced",
      "signature": "run_opencode_traced(proj_dir, work_dir, command, stage, function_ids, input_files, output_files, summary, metadata) -> None",
      "pre_condition": "proj_dir is an existing directory. command is a non-empty list of CLI argument strings. stage is a non-empty string identifying the pipeline stage. output_files is a non-empty list of relative paths (from proj_dir) that must exist after successful execution.",
      "post_condition": "Executes command as a subprocess with proj_dir as the working directory. Produces a structured trace event recording the execution metadata, timing, input/output file references, and exit code. Raises a CalledProcessError when the subprocess exits with a non-zero code. The output files listed in output_files must be produced on disk after a successful run."
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/verification-py/_validate_single_bug.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "When a result marker for the discrepancy corresponding to result_json_rel does not already exist at <work_dir>/bug_validation/<bug_id>.result.json, or when resume is False, a bug-validation agent is dispatched to evaluate whether the spec-code discrepancy recorded in result_json_rel constitutes a real, exploitable bug. A validated bug report is produced at <work_dir>/bug_validation/<bug_id>.md if and only if the discrepancy is confirmed to be an exploitable bug. The bug report contains: the behavioral assertion claimed by the specification that the code violates, the actual behavior observed from the code, concrete code evidence with line references, the condition under which the violation is triggered, step-by-step reproduction instructions, a probe script exercising the violation, and the raw output from executing the probe script. A result marker file is written to <work_dir>/bug_validation/<bug_id>.result.json upon completion of validation. When resume is True and the result marker already exists with valid, parseable JSON content, no re-validation occurs and no side effects are produced. The bug_id is uniquely derived from result_json_rel by stripping the standard verification-results path prefix and canonicalizing directory separators.",
    "actual_behavior": "After the code block, one of three scenarios holds: (1) An unhandled exception was raised (e.g., from os.makedirs, file writing, os.replace, or build_llm_cli_command). In this case the program may have aborted with only partial side effects; no further guarantees are made about variables after the exception point. (2) No exception occurred and the function returned early at line 73 because resume was truthy, result_path existed, and the file could be successfully read and parsed as JSON. Then the following are true: the directory os.path.join(work_dir, 'bug_validation') exists; the file at prompt_path exists and its entire content equals prompt_content; the temporary file tmp_path no longer exists; the following variables are defined with values as given in the code: prompt_content = '# Bug Validator\\n\\n**Target result file:** `{result_json_rel}`\\n\\n**Bug ID:** `{bug_id}`\\n\\n---\\n\\n\\n' + user_knowledge_section + base_content, prompt_filename =
```

```
'fm_agent/bug_validation/bug_validation_{bug_id}.md', prompt_path = os.path.join(proj_dir, prompt_filename), tmp_path = prompt_path + '.tmp', prompt = 'Follow the instructions in the attached file', command = build_llm_cli_command(OPENCODE_BUG_VALIDATION_MODEL, prompt, cwd=proj_dir, files=[prompt_path]), result_relpth = 'fm_agent/bug_validation/{bug_id}.result.json', result_path = os.path.join(proj_dir, result_relpth); the function returns normally. (3) No exception occurred and no early return (either resume is falsy, result_path does not exist, or JSON parsing raised an exception). Then the same directory and file side effects hold as in scenario (2), the same variable assignments hold as in scenario (2), and additionally: max_attempts = config.BUG_VALIDATION_MAX_RETRIES; the for loop for attempt in range(1, max_attempts + 1) has been entered and the first iteration began with attempt = 1 and run_failed = False; the try block at line 80 has been entered but no inner statements have executed yet. Formally: (exception_raised_at_line(l) l {49,55,56,57,59,60,61,62,63,64,65,66,68,69,70,71}) (exception_early_return prompt_path_exists file_content(prompt_path) = prompt_content (v {prompt_content, prompt_filename, prompt_path, tmp_path, prompt, command, result_relpth, result_path} : assigned(v) = value_from_code) return_executed) (exception_early_return same_effects_as_above max_attempts = config.BUG_VALIDATION_MAX_RETRIES attempt = 1 run_failed = False). Here early_return is defined as resume file_exists(result_path) json_load_succeeded(result_path).",
    "code_evidence": "Line 41: through Line 80: The code block only performs prompt setup and a skip check; it never dispatches the bug-validation agent, never writes the validated bug report to <work_dir>/bug_validation/<bug_id>.md, and never writes the result marker file <work_dir>/bug_validation/<bug_id>.result.json upon completion of validation as required by the specification.",
    "trigger_condition": "The specification requires that when resume is False or the result marker does not exist, a bug-validation agent is dispatched, leading to production of a bug report and a result marker. Condition A describes poststates where either an exception occurs (no dispatch) or the function returns early (skip) or enters a retry loop without executing any dispatch. In none of these scenarios does the code perform the mandated side effects."
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--verification-py--_validate_single_bug",
  "source_file": "src/verification.py",
  "function_name": "_validate_single_bug",
  "confirmation_status": "not_confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--verification-py--_validate_single_bug.py",
  "detail_file": "fm_agent/bug_validation/src--verification-py--_validate_single_bug.md",
  "probe_stdout": "NOT CONFIRMED - actual: 'dispatch called' | expected: 'dispatch called' | output_files=[
'fm_agent/bug_validation/src--verification-py--_validate_single_bug.md', 'fm_agent/bug_validation/src--verification-py--_validate_single_bug.result.json'] | stage=bug_validation | summary=OpenCode bug validation for src--verification-py--_validate_single_bug\\n\\n",
  "trigger_summary": "When resume=False and no result marker exists, the function correctly dispatches a bug-validation agent via run_opencode_traced - the verification was a false positive caused by incomplete control-flow analysis stopping at line 80."
}
```

► Reasoner 流程 (3 条事件)

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/3	2026-07-28T04:11:29.985956Z	5490
2	generate_block_post_condition	success	1	1/3	2026-07-28T04:12:28.304371Z	6294
3	check_post_implies_spec	mismatch	1	1/3	2026-07-28T04:13:38.312810Z	6276

事件 1 — [llm_af91b2775ee34a3abe6bed5f2e929438](#)

► parsed_json (完整)

```
{
  "post_condition": "After the code block executes successfully, the following hold: The variable `work_dir` is set to `proj_dir` if the original `work_dir` argument was `None`; otherwise it retains the original value. `script_dir` is set to the parent directory of the directory containing the current file, and it exists. `bug_id` is a string computed from `result_json_rel` by removing any leading `\"fm_agent/logic_verification_results/\"` or `os.path.join(\"fm_agent\", \"logic_verification_results\")+os.sep`, then removing the file extension, and replacing all directory separators with `\"--\"`. `function_id` is the string returned by `function_id_from_result_path(result_json_rel)`, which is a function identifier for correlating verification results. `base_md_path` is set to `bug_validator_path` if that argument was truthy (a string), otherwise to `os.path.join(script_dir, \"md\", \"bug_validator.md\")`; this file exists and its entire content is read into `base_content`. The list `user_knowledge_paths` is the result of calling `list_staged_domain_knowledge_relpaths(work_dir)`, a possibly empty list of relative path strings. If `user_knowledge_paths` is non-empty, `user_knowledge_section` contains a markdown section header and a bullet list of those paths generated by `format_domain_knowledge_bullets`, followed by `\"\\n\\n--\\n\\n\"`; if the list is empty, `user_knowledge_section` is an empty string.\\n\\nFormally:\\n`W_initial` is None -> work_dir = P_dir) ^ (W_initial is not None -> work_dir = W_initial)`\\n`script_dir = os.path.dirname(os.path.abspath(__file__)) ^ exists(script_dir)`\\n`prefix1 = os.path.join(\"fm_agent\", \"logic_verification_results\") + os.sep`\\n`prefix2 = \"fm_agent/logic_verification_results/\"`\\n`parts = if P_rel.startswith(prefix1) then P_rel[len(prefix1):] else if P_rel.startswith(prefix2) then P_rel[len(prefix2):] else P_rel`\\n`bug_id = os.path.splitext(parts)[0].replace(os.sep, \"--\").replace(\"\\\\\\\\\", \"--\")`\\n`function_id = function_id_from_result_path(P_rel)`\\n`base_md_path = (BVP if BVP else os.path.join(script_dir, \"md\", \"bug_validator.md\")) ^ file_exists(base_md_path) ^ base_content = read_file_content(base_md_path)`\\n`user_knowledge_paths = list_staged_domain_knowledge_relpaths(work_dir)` (a list of strings)`\\n`(user_knowledge_paths != []) -> user_knowledge_section = \"## User-Provided Domain Knowledge\\n\\nRead these Markdown
```



```
files as additional context for intended behavior, terminology, data encodings, and invariants before validating the candidate
bug:\n\n\n" + format_domain_knowledge_bullets(user_knowledge_paths) + "\n\n\n---\n\n\n") ^ (user_knowledge_section = [] →
user_knowledge_section = "\n\n\n")
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_af91b2775ee34a3abe6bed5f2e929438_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_af91b2775ee34a3abe6bed5f2e929438_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_af91b2775ee34a3abe6bed5f2e929438_response.txt)

事件 2 — [llm_ac4d30153b7a49e49041781b65139bd7](#)

► parsed_json (完整)

```
{
  "post_condition": "After the code block, one of three scenarios holds: (1) An unhandled exception was raised (e.g., from
os.makedirs, file writing, os.replace, or build_llm_cli_command). In this case the program may have aborted with only partial side
effects; no further guarantees are made about variables after the exception point. (2) No exception occurred and the function
returned early at line 73 because resume was truthy, result_path existed, and the file could be successfully read and parsed as
JSON. Then the following are true: the directory os.path.join(work_dir, 'bug_validation') exists; the file at prompt_path exists
and its entire content equals prompt_content; the temporary file tmp_path no longer exists; the following variables are defined
with values as given in the code: prompt_content = '# Bug Validator\n\n\n**Target result file:** `{result_json_rel}`\n\n\n**Bug ID:**
`{bug_id}`\n\n\n---\n\n\n' + user_knowledge_section + base_content, prompt_filename =
'fm_agent/bug_validation/bug_validator_{bug_id}.md', prompt_path = os.path.join(proj_dir, prompt_filename), tmp_path = prompt_path
+ '.tmp', prompt = 'Follow the instructions in the attached file', command = build_llm_cli_command(OPENCODE_BUG_VALIDATION_MODEL,
prompt, cwd=proj_dir, files=[prompt_path]), result_relp_path = 'fm_agent/bug_validation/{bug_id}.result.json', result_path =
os.path.join(proj_dir, result_relp_path); the function returns normally. (3) No exception occurred and no early return (either
resume is falsy, result_path does not exist, or JSON parsing raised an exception). Then the same directory and file side effects
hold as in scenario (2), the same variable assignments hold as in scenario (2), and additionally: max_attempts =
config.BUG_VALIDATION_MAX_RETRIES; the for loop for attempt in range(1, max_attempts + 1) has been entered and the first iteration
began with attempt = 1 and run_failed = False; the try block at line 80 has been entered but no inner statements have executed
yet. Formally: (exception_raised_at_line(l) ^ l ∈ {49,55,56,57,59,60,61,62,63,64,65,66,68,69,70,71}) v (¬exception ^ early_return
^ prompt_path_exists ^ file_content(prompt_path) = prompt_content ^ (∀v ∈ {prompt_content, prompt_filename, prompt_path, tmp_path,
prompt, command, result_relp_path, result_path} : assigned(v) = value_from_code) ^ return_executed) v (¬exception ^ ¬early_return ^
same_effects_as_above ^ max_attempts = config.BUG_VALIDATION_MAX_RETRIES ^ attempt = 1 ^ run_failed = False). Here early_return is
defined as resume ^ file_exists(result_path) ^ json_load_succeeded(result_path)."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_ac4d30153b7a49e49041781b65139bd7_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_ac4d30153b7a49e49041781b65139bd7_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_ac4d30153b7a49e49041781b65139bd7_response.txt)

事件 3 — [llm_1f3e0ef1ab7345cbb2fe3f2780b2e8f5](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "resume=False, result_json_rel='some/path/result.json', bug_id='some_func', work_dir='/tmp/work',
proj_dir='/project', user_knowledge_section='', base_content='',
  "offending_statements": "Line 41: through Line 80: The code block only performs prompt setup and a skip check; it never
dispatches the bug-validation agent, never writes the validated bug report to <work_dir>/bug_validation/<bug_id>.md, and never
writes the result marker file <work_dir>/bug_validation/<bug_id>.result.json upon completion of validation as required by the
specification.",
  "reason": "The specification requires that when resume is False or the result marker does not exist, a bug-validation agent is
dispatched, leading to production of a bug report and a result marker. Condition A describes post-states where either an exception
occurs (no dispatch) or the function returns early (skip) or enters a retry loop without executing any dispatch. In none of these
scenarios does the code perform the mandated side effects."
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1f3e0ef1ab7345cbb2fe3f2780b2e8f5_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_1f3e0ef1ab7345cbb2fe3f2780b2e8f5_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_1f3e0ef1ab7345cbb2fe3f2780b2e8f5_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

运行完整性备注

- [trace/events.jsonl](#) 仍有 1 行并发写坏；本报告只索引可解析事件。
- 原始提示词和响应保留在 [fm_agent/trace/payloads/](#)，每条 Reasoner 事件均提供链接。
- 报告拆分不会修改任何验证结果、源码、SPEC 或 probe。

